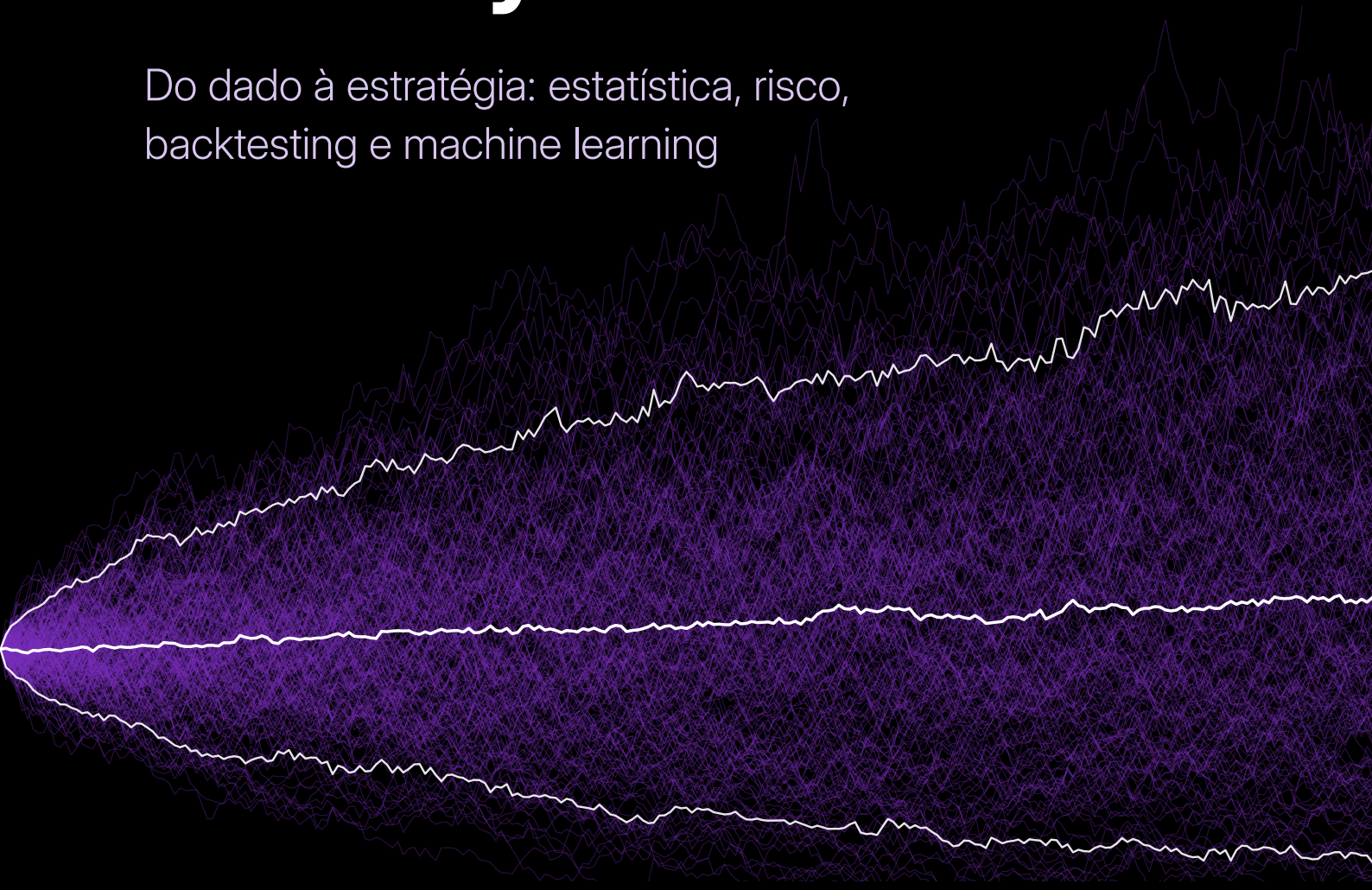
Liga de Mercado Financeiro

Finanças quantitativas com Python

Do dado à estratégia: estatística, risco, backtesting e machine learning



João Emanuel Zilli dos Santos

Núcleo de Análise Quantitativa

1ª edição, 2026

Finanças quantitativas com Python

Do dado à estratégia: estatística, risco, backtesting e machine learning

João Emanuel Zilli dos Santos

Estudante de Engenharia de Computação com IA Aplicada



Material didático do Núcleo de Análise Quantitativa
Liga de Mercado Financeiro, Universidade Federal de Uberlândia

1ª edição, 2026

Finanças quantitativas com Python
João Emanuel Zilli dos Santos

Material didático do Núcleo de Análise Quantitativa da Liga de Mercado Financeiro da Universidade Federal de Uberlândia (LMF UFU), escrito a partir do ciclo de aulas de 2026.1 e da ementa proposta para o Núcleo. 1ª edição, 2026.

Aviso. Este livro tem finalidade exclusivamente educacional. Nada nele é recomendação de investimento. As estratégias aparecem como exemplos de método, e os resultados históricos mostrados não garantem resultados futuros. Operar no mercado envolve risco de perda do capital.

Dados. As séries de preços usadas nos exemplos foram obtidas do Yahoo Finance por meio da biblioteca `yfinance`, cobrem de janeiro de 2010 a dezembro de 2025 e foram baixadas em outubro de 2026. As séries do CDI e do dólar PTAX vêm do Sistema Gerenciador de Séries Temporais do Banco Central do Brasil. A cópia bruta está em `livro/dados/bruto/`; os arquivos usados nos capítulos passaram pela limpeza descrita no Capítulo 2. São dados de uso educacional, sem garantia de exatidão.

Repositório. O livro, os códigos, os dados e o material das aulas estão em https://github.com/joaoemanuelzilli/aulas_quant. Os caminhos de arquivo citados no texto, como `aula5/Var_historico.py` ou `livro/scripts/cap05/`, são relativos à raiz desse repositório.

Código. Os códigos impressos e as figuras de dados são gerados por scripts em `livro/scripts/`, uma pasta por capítulo, executados com Python 3.12 e as versões de bibliotecas registradas em `livro/requirements.txt`. Os poucos trechos apresentados como esboço, que dependem de serviços externos, estão identificados no texto.

Composição. Texto em STIX Two, títulos em Inter e código em JetBrains Mono, fontes distribuídas sob a SIL Open Font License. Composto em LuaLaTeX.

A marca e o logotipo da Liga de Mercado Financeiro pertencem à Liga e são usados neste material institucional com essa finalidade.

Sobre este livro

O Núcleo de Análise Quantitativa nasceu em 2026 para dar à Liga um espaço de trabalho com dados, modelos e automação. No primeiro ciclo, o conteúdo ficou espalhado em slides, scripts, notebooks e estudos guiados, e parte da ementa planejada não chegou a ser dada em aula. Este livro reúne tudo em um só lugar: o que foi ensinado, revisto e ampliado, e o que estava previsto e faltou. A intenção é que quem entrar no Núcleo nos próximos semestres tenha um material único para estudar, e que os próximos gerentes tenham por onde começar em vez de recomeçar.

Para quem é

Para estudantes de graduação que sabem programar um pouco, lembram do básico de estatística e nunca trabalharam com dados de mercado. Não se supõe conhecimento prévio de finanças. Cada conceito é apresentado primeiro pela ideia e por um exemplo com números, e só depois pela definição formal.

Como está organizado

São doze capítulos, um para cada semana da ementa do Núcleo, em três partes.

A Parte I trata dos fundamentos: como transformar preços em retornos, como obter e limpar dados, como descrever retornos com estatística e como modelar sua dependência no tempo. A Parte II trata de risco e de backtesting: medidas de perda, construção de carteiras, métricas de desempenho, os vieses que invalidam um teste histórico e as formas de validar uma estratégia fora da amostra. A Parte III trata de aprendizado de máquina aplicado a mercados, da execução de ordens e seus custos, e termina com o projeto final, em que uma estratégia completa é construída e defendida.

Os capítulos foram escritos para serem lidos em ordem. Cada um abre com o que você será capaz de fazer ao terminá-lo e fecha com um resumo, exercícios em três níveis e indicações de leitura.

Como estudar por ele

Leia com o computador aberto. Todo código impresso é um arquivo que está em `livro/scripts/` e pode ser executado; os números que aparecem nas páginas são os que esses arquivos produzem a partir do conjunto de dados congelado em `livro/dados/`. Rodar, alterar um parâmetro e ver o que muda ensina mais do que reler. Os exercícios marcados como básicos conferem se a leitura foi entendida, os intermediários pedem que você escreva código, e os desafios são pequenas investigações.

Ao longo do texto aparecem blocos de seis tipos. As definições e os resultados numerados fixam o vocabulário formal. Os blocos de intuição dizem a mesma coisa em palavras. Os exemplos resolvem uma conta com

números. As armadilhas descrevem erros que quase todo mundo comete uma vez. E os blocos “No repositório” apontam para os scripts das aulas do ciclo 2026.1, que continuam disponíveis no repositório do Núcleo, em github.com/joaoemanuelzilli/aulas_quant.

Uma palavra sobre honestidade

Boa parte deste livro ensina a desconfiar. Você verá estratégias que parecem ótimas e deixam de funcionar quando o teste é feito direito, modelos sofisticados que perdem para regras simples e métricas cuja incerteza é maior do que o próprio valor. Isso é deliberado. A referência central do material, López de Prado (2018), trata o sobreajuste de backtests, o resultado que se obtém por tentativa e erro e se toma por descoberta, como um dos problemas centrais da área. Nenhum exemplo aqui deve ser lido como recomendação, e os resultados negativos foram mantidos onde apareceram.

Agradecimentos

Este material existe por causa do trabalho coletivo do Núcleo no ciclo 2026.1. Agradeço a Arthur Pires Lopes, Diretor de Pesquisa, que propôs a ementa em que o livro se baseia, e a Gustavo Kono, Wilson Ribeiro e Thiago Leal, que prepararam e deram aulas comigo. Agradeço também aos membros que assistiram às aulas, fizeram as entregas e apontaram o que não estava claro.

João Emanuel Zilli dos Santos

Sumário

Sobre este livro iii

Parte I Fundamentos

1 Python para finanças: preços e retornos 2

1.1 O que um analista quantitativo faz 2
1.2 Ambiente de trabalho 3
1.3 NumPy: contas sobre vetores inteiros 4
1.4 pandas: dados com rótulo de tempo 4
1.5 Por que retornos, e não preços 5
1.6 Retorno simples 6
1.7 Log-retorno 6
1.8 O preço certo: ajuste por proventos 9
1.9 Acumular e anualizar 10

2 Coleta, limpeza e visualização de dados 14

2.1 De onde vêm os dados 14
2.2 Baixar uma vez, guardar o bruto, limpar à parte 16
2.3 Dados faltantes: primeiro o diagnóstico 18
2.4 Remover, repetir ou interpolar 20
2.5 Erros de dados e retornos extremos 22
2.6 Anatomia de um gráfico no Matplotlib 27
2.7 Plotly para explorar 29
2.8 O efeito do câmbio 30
2.9 Um roteiro de análise exploratória 32

3 Estatística aplicada a retornos 38

3.1 Resumir uma série de retornos 38
3.2 Assimetria e curtose 40
3.3 A normal como ponto de partida, e onde ela falha 41
3.4 A distribuição *t* de Student 43
3.5 A lógica de um teste de hipótese 46
3.6 O retorno médio é diferente de zero? 46
3.7 Testes de normalidade e de autocorrelação 49
3.8 Muitos testes, alguns “achados” 51
3.9 Covariância e correlação 52
3.10 Correlação que muda no tempo 55

4 Séries temporais: estacionariedade, ARIMA e GARCH 58

4.1 Quando a ordem das observações importa 58
4.2 Estacionariedade 59
4.3 Regressão espúria 60
4.4 Testes de raiz unitária 61
4.5 Autocorrelação: ACF e PACF 64
4.6 Modelos AR, MA e ARMA 64
4.7 ARIMA e o roteiro de Box-Jenkins 66

4.8 Volatilidade que se agrupa 70
4.9 GARCH na prática 72
4.10 Entrega 1: análise exploratória 77

Parte II Risco e backtesting

5 Risco de mercado: VaR, CVaR e drawdown 81
5.1 Risco como perda em reais 81
5.2 Value at Risk: um quantil das perdas 83
5.3 VaR paramétrico 84
5.4 VaR histórico 87
5.5 VaR por simulação de Monte Carlo 89
5.6 Backtest do VaR: contar violações 90
5.7 O que o VaR não vê 92
5.8 Expected shortfall (CVaR) 94
5.9 Drawdown 96
5.10 Métricas de cauda e o tamanho do erro 98

6 Portfólios: Markowitz, diversificação e testes de estresse 102
6.1 Dois ativos: o ganho de misturar 102
6.2 N ativos em notação matricial 104
6.3 O limite da diversificação 104
6.4 O problema de Markowitz 106
6.5 Ativo livre de risco e a carteira tangente 107
6.6 Otimização numérica com restrições 108
6.7 O maximizador de erros 110
6.8 Remédios contra o erro de estimação 115
6.9 Teste de estresse e análise de cenários 117
6.10 Entrega 2: relatório de risco 121

7 Backtesting: métricas de desempenho e vieses 124
7.1 O que um backtest pode provar 124
7.2 Anatomia de um backtest 125
7.3 Métricas de desempenho 128
7.4 Quanto se pode confiar em um Sharpe 131
7.5 *Look-ahead*: usar o futuro sem perceber 133
7.6 Viés de sobrevivência 135
7.7 Sobreajuste e a garimpagem de dados 137
7.8 Backtests orientados a eventos: o backtrader 140
7.9 Lista de conferência 143

8 Estratégias e validação fora da amostra 146
8.1 De onde vem uma estratégia 146
8.2 Um motor de backtest em catorze linhas 147
8.3 Momentum 149
8.4 Reversão à média 152
8.5 Rompimento 154
8.6 Tamanho da posição e custos 157
8.7 Dentro e fora da amostra: walk-forward 157
8.8 Validação cruzada em séries financeiras 160
8.9 Testes de robustez 161
8.10 O relatório de backtest e as entregas 162

Parte III Machine learning, execução e projeto

9 Machine learning I: regressão, features e validação temporal 168

9.1 Por que mercados são um problema diferente 168
9.2 Regressão linear 169
9.3 Classificação e regressão logística 174
9.4 Engenharia de features com dados de mercado 177
9.5 Rotulagem: o que exatamente se quer prever 180
9.6 Validação temporal 182
9.7 Da previsão à estratégia 186

10 Machine learning II: ensembles, regularização e sobreajuste 190

10.1 O dilema entre viés e variância 190
10.2 Um painel para o capítulo inteiro 192
10.3 Regularização em modelos lineares 193
10.4 Árvores de decisão 196
10.5 Bagging e random forest 198
10.6 Gradient boosting 200
10.7 Uma comparação honesta 202
10.8 Interpretar o modelo 205
10.9 Da previsão à posição 207

11 Algoritmos de trading: ordens, custos e execução 212

11.1 Do backtest ao mercado 212
11.2 A anatomia de um bot 213
11.3 Microestrutura mínima: o livro de ofertas 214
11.4 Ordens: tipos, validade e ciclo de vida 215
11.5 Custos de transação 217
11.6 Quanto custo uma estratégia aguenta 219
11.7 Uma corretora simulada 222
11.8 Conectar-se a uma corretora 228
11.9 Dimensionamento de posições 230
11.10 Risco operacional 231

12 Projeto final: da tese ao resultado 234

12.1 O que o Núcleo pede 234
12.2 A tese 235
12.3 Dados e universo 238
12.4 Modelo: da tese à regra 238
12.5 Backtest 240
12.6 Robustez 244
12.7 Resultado e conclusão 246
12.8 O repositório de entrega 247
12.9 A defesa e a avaliação 249
12.10 Depois do ciclo 251

A Notação e formulário 254

A.1 Notação 255
A.2 Retornos (Capítulo 1) 255
A.3 Estatística (Capítulo 3) 256
A.4 Séries temporais (Capítulo 4) 256
A.5 Risco (Capítulo 5) 256
A.6 Carteiras (Capítulo 6) 257
A.7 Desempenho (Capítulo 7) 257

B Glossário inglês–português 258

Referências **260**

Índice remissivo **264**

Parte I

Fundamentos

Dados, estatística e séries temporais: as ferramentas com que se descreve o comportamento de um ativo antes de apostar qualquer coisa nele.



Capítulo 1

Python para finanças: preços e retornos

1

Quase tudo o que se faz em finanças quantitativas começa com a mesma conta: pegar uma série de preços e transformá-la em uma série de retornos. Parece um detalhe de preparação de dados, mas a escolha entre retorno simples e log-retorno, e o cuidado com o preço usado na conta, decidem se as estatísticas dos capítulos seguintes fazem sentido.

Ao terminar este capítulo você consegue

- montar um ambiente de trabalho reproduzível com Python, VS Code e Git;
- manipular séries de preços com NumPy e pandas sem escrever laços;
- calcular retornos simples e log-retornos e dizer quando usar cada um;
- acumular e anualizar retornos sem cometer os erros clássicos;
- explicar por que se usa o preço ajustado por proventos.

1.1 O que um analista quantitativo faz

Um analista quantitativo, ou *quant*, toma decisões de investimento a partir de regras escritas em código e testadas em dados, e não de impressões. O trabalho segue quase sempre o mesmo roteiro, que é também o roteiro deste livro: calcular retornos e obter e limpar dados (Capítulos 1 e 2), descrevê-los com estatística (Capítulos 3 e 4), medir o risco (Capítulos 5 e 6), testar uma estratégia no passado sem se enganar (Capítulos 7 e 8), usar modelos de aprendizado de máquina com parcimônia (Capítulos 9 e 10) e, por fim, levar a regra ao mercado (Capítulos 11 e 12).

A matemática raramente é a parte mais difícil. O que custa é a disciplina de não se deixar convencer por um resultado bonito. López de Prado (2018) atribui o fracasso da maior parte dos projetos de aprendizado de máquina em finanças à forma como a pesquisa é organizada e conduzida, mais do que à falta de modelos sofisticados. Por isso este material insiste, desde o primeiro

capítulo, em duas práticas: todo número precisa poder ser reproduzido por outra pessoa, e toda conta precisa de uma verificação independente.

1.2 Ambiente de trabalho

Você vai precisar de três coisas instaladas: Python 3.11 ou mais recente (o livro foi gerado com a versão 3.12), um editor (o Núcleo usa o VS Code, que também abre notebooks Jupyter) e o Git.

1.2.1 Um ambiente virtual por projeto

Bibliotecas mudam de versão, e um código que roda hoje pode quebrar daqui a um ano. O remédio é isolar cada projeto em um **ambiente virtual**, uma pasta com sua própria cópia do Python e das bibliotecas, e registrar as versões usadas em um arquivo `requirements.txt`.

```
python -m venv .venv                # cria o ambiente em .venv
source .venv/bin/activate           # ativa o ambiente
pip install numpy pandas matplotlib yfinance
pip freeze > requirements.txt       # registra as versões exatas
```

No Windows, o comando de ativação é `.venv\Scripts\activate`, sem o `source`.

Quem clonar o projeto recria o mesmo ambiente com `pip install -r requirements.txt`. A pasta `.venv` não vai para o repositório; ela entra no arquivo `.gitignore`.

1.2.2 Notebook ou script?

Um notebook Jupyter mistura código, texto e gráficos, e é ótimo para explorar dados e para as entregas do Núcleo. O risco é a ordem de execução: células rodadas fora de sequência deixam na memória variáveis que não existem mais no código. Antes de entregar qualquer notebook, use *Restart & Run All* e confira se ele roda de cima a baixo. Para código que será reaproveitado (um motor de backtest, uma função de risco), prefira arquivos `.py`.

1.2.3 Git em cinco comandos

O Git guarda o histórico do projeto e permite que várias pessoas trabalhem no mesmo código. No dia a dia do Núcleo bastam cinco comandos:

```
git clone <endereço-do-repositório>  # baixa uma cópia
git status                           # mostra o que mudou
git add analise.py                   # escolhe o que registrar
git commit -m "Calcula retornos"     # registra, com mensagem
git push                             # envia para o GitHub
```

No repositório

Todos os scripts das aulas estão no repositório do Núcleo, github.com/joaoemanuelzilli/aulas_quant, uma pasta por aula (`aula1/` a `aula7/`). Os códigos impressos neste livro estão em `livro/scripts/`, uma pasta por capítulo, e leem um conjunto de dados congelado em `livro/dados/`, de modo que os números que você obtiver ao rodá-los são os mesmos da página.

1.3 NumPy: contas sobre vetores inteiros

O NumPy fornece o `ndarray`, um vetor (ou matriz) de números do mesmo tipo guardados lado a lado na memória. A consequência prática é a **vetorização**: uma operação escrita uma vez é aplicada a todos os elementos por código compilado, sem laço em Python. O Código 1.1 calcula quatro retornos com uma única linha.

Código 1.1 Retornos com NumPy, sem laço (`cap01/l01_numpy.py`)

```
import numpy as np

precos = np.array([20.00, 20.50, 19.90, 21.10, 22.00])

# operação vetorizada: age sobre o array inteiro, sem laço
retornos = precos[1:] / precos[:-1] - 1

print(retornos.round(4))
print(f"média: {retornos.mean():.4%}   "
      f"desvio-padrão: {retornos.std(ddof=1):.4%}")
```

```
[ 0.025  -0.0293  0.0603  0.0427]
média: 2.4672%   desvio-padrão: 3.8740%
```

A expressão `precos[1:] / precos[:-1]` divide cada preço pelo anterior: `precos[1:]` é o vetor sem o primeiro elemento e `precos[:-1]` é o vetor sem o último. Repare também no argumento `ddof=1` do desvio-padrão. Ele manda dividir por $n-1$ em vez de n, que é o estimador amostral usual; o NumPy usa n por padrão e o pandas usa $n-1$, uma diferença que já causou muita discrepância entre planilhas.

1.4 pandas: dados com rótulo de tempo

O pandas acrescenta rótulos ao NumPy. Uma `Series` é um vetor com índice; um `DataFrame` é uma tabela em que cada coluna é uma `Series`. Em finanças o índice quase sempre é um `DatetimeIndex`, e é isso que permite fatiar por data, reamostrar por mês ou alinhar automaticamente dois ativos que negociam em calendários diferentes.

Código 1.2 Leitura e fatiamento de uma série de preços (`cap01/l02_pandas.py`)

```
import pandas as pd

precos = pd.read_csv("../../dados/precos_ajustados.csv",
                     index_col=0, parse_dates=True)

# uma coluna do DataFrame é uma Series
petr = precos["PETR4.SA"].dropna()
print(type(precos.index).__name__, "|", precos.shape)

# fatiar por data
print(petr.loc["2024-01-02":"2024-01-05"].round(2))

# último preço de cada ano
```

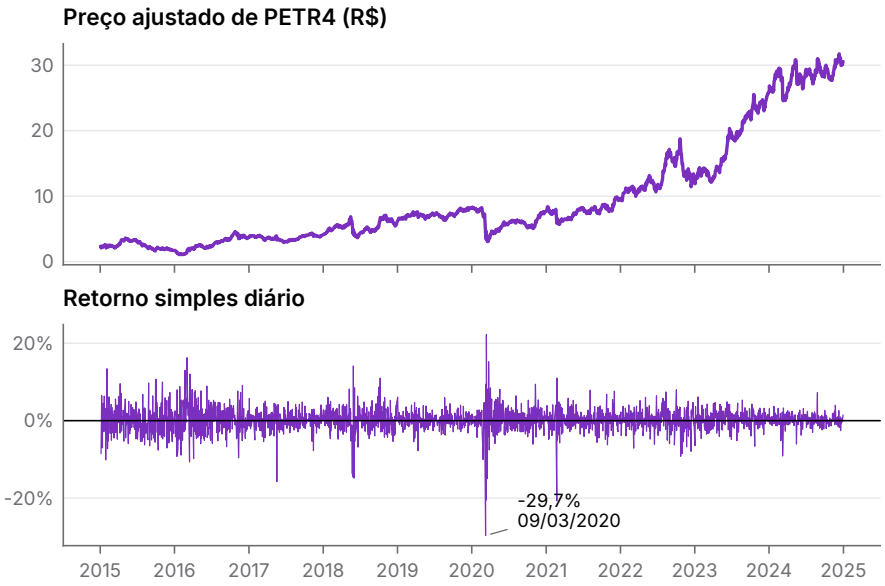


Figura 1.1 Preço e retorno diário de PETR4, de 2015 a 2024. O preço passeia sem voltar a um nível de referência; o retorno oscila em torno de zero. A pior queda diária da amostra, de 29,7%, ocorreu em 9 de março de 2020.

```
print(petr.resample("YE").last().tail(3).round(2))
```

```
DatetimeIndex | (4170, 11)
Date
2024-01-02    26.03
2024-01-03    26.84
2024-01-04    26.62
2024-01-05    26.68
Name: PETR4.SA, dtype: float64
Date
2023-12-31    25.66
2024-12-31    30.56
2025-12-31    28.98
Freq: YE-DEC, Name: PETR4.SA, dtype: float64
```

Três detalhes desse código voltam em todo o livro. O argumento `parse_dates` é o que transforma a primeira coluna do arquivo em datas de verdade. O método `.loc` seleciona por rótulo, e com datas aceita intervalos escritos como texto. Já o método `resample`, chamado com `"YE"`, agrupa as observações por ano; o rótulo de 31 de dezembro indica o grupo, não o dia do último pregão.

O sufixo `.SA` é a forma como o Yahoo Finance identifica ativos negociados na B3. O Capítulo 2 trata da coleta.

1.5 Por que retornos, e não preços

A Figura 1.1 mostra a mesma informação de duas formas. No painel de cima, o preço de PETR4 entre 2015 e 2024. No de baixo, a variação percentual de um dia para o outro.

Há três razões para trabalhar com a série de baixo.

- **Comparabilidade.** Uma alta de R$ 1 significa coisas muito diferentes em uma ação de R$ 5 e em uma de R$ 100. Um retorno de 2% significa a mesma coisa nas duas.

- **Comportamento estatístico.** O nível do preço muda de patamar ao longo dos anos, de modo que a média e a variância calculadas em uma janela pouco dizem sobre a janela seguinte. Os retornos flutuam em torno de um valor próximo de zero, com uma dispersão bem mais estável. O Capítulo 4 dá nome a essa propriedade (estacionariedade) e mostra como testá-la.
- **É o que o investidor recebe.** O resultado de uma posição é o capital investido multiplicado pelo retorno. Risco e desempenho são, por isso, definidos sobre retornos.

1.6 Retorno simples

Definição 1.1 Retorno simples

Sendo P_t o preço de um ativo no instante t, o **retorno simples** entre $t-1$ e t é

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \frac{P_t}{P_{t-1}} - 1. \tag{1.1}$$

A razão $P_t/P_{t-1} = 1 + R_t$ é chamada de retorno bruto.

O retorno simples responde à pergunta "quanto por cento meu dinheiro variou?". Ele tem uma propriedade que o torna indispensável em carteiras: o retorno de uma carteira é a média dos retornos simples dos ativos, ponderada pelos pesos,

$$R_{p,t} = \sum_{i=1}^{N} w_i R_{i,t}, \qquad \sum_{i=1}^{N} w_i = 1, \tag{1.2}$$

em que w_i é a fração do patrimônio aplicada no ativo i no início do período.

O que o retorno simples *não* faz é somar ao longo do tempo. Para acumular vários períodos é preciso multiplicar os retornos brutos:

$$1 + R_{0\to T} = \prod_{t=1}^{T} (1 + R_t). \tag{1.3}$$

Armadilha **Somar retornos simples de períodos diferentes**

Uma ação cai 50% em um dia e sobe 50% no dia seguinte. A soma dá zero, mas o investidor perdeu um quarto do capital: $0{,}5 \times 1{,}5 = 0{,}75$. Para voltar ao ponto de partida depois de perder 50% é preciso ganhar 100%. Pela mesma razão, a média aritmética de retornos simples superestima o crescimento de um investimento mantido por vários períodos.

1.7 Log-retorno

Definição 1.2 Log-retorno

O **log-retorno**, ou retorno continuamente composto, entre $t-1$ e t é

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln P_t - \ln P_{t-1} = \ln(1 + R_t). \tag{1.4}$$

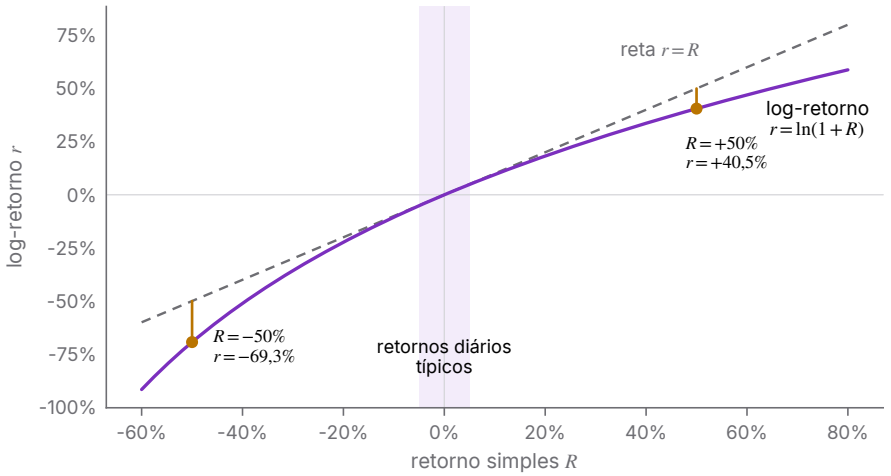


Figura 1.2 Relação entre retorno simples e log-retorno. Na faixa de ±5%, em que cai a maior parte dos retornos diários, as duas medidas praticamente coincidem. Fora dela a distância entre os dois cresce depressa.

Intuição

O log-retorno é a taxa que, capitalizada continuamente durante o período, leva de P_{t-1} a P_t: $P_t = P_{t-1}\,e^{r_t}$. O logaritmo troca multiplicações por somas, e é exatamente essa troca que interessa, porque acumular retornos é multiplicar.

Aplicando o logaritmo aos dois lados da Equação (1.3), o produto vira soma:

$$r_{0\to T} = \ln\left(\frac{P_T}{P_0}\right) = \sum_{t=1}^{T} r_t. \tag{1.5}$$

Os termos intermediários se cancelam, pois $\ln P_T - \ln P_0 = (\ln P_T - \ln P_{T-1}) + \cdots + (\ln P_1 - \ln P_0)$. Essa **aditividade no tempo** é a grande vantagem do log-retorno. Somas de variáveis aleatórias são muito mais fáceis de tratar do que produtos: a média de uma soma é a soma das médias e, se os retornos forem não correlacionados, a variância de uma soma é a soma das variâncias. É daí que sai a regra da raiz do tempo da Seção 1.9.

1.7.1 Quão diferentes são os dois retornos

A expansão em série de Taylor de $\ln(1+R)$ em torno de zero é

$$r = \ln(1+R) = R - \frac{R^2}{2} + \frac{R^3}{3} - \dots \tag{1.6}$$

Para retornos pequenos, $r \approx R$, com erro da ordem de $R^2/2$. Em um dia comum, com $R = 1\%$, a diferença é de 0,005 ponto percentual. Em um dia extremo ela deixa de ser desprezível, como mostra a Figura 1.2. Como $\ln(1+R) \le R$ para todo $R > -1$, o log-retorno nunca é maior que o retorno simples.

Exemplo 1.1 A ação que cai 50% e sobe 50%

Volte ao caso da armadilha anterior. Os log-retornos dos dois dias são $\ln 0{,}5 = -0{,}6931$ e $\ln 1{,}5 = +0{,}4055$. A soma é $-0{,}2877$, e $e^{-0{,}2877} - 1 = -25\%$, a perda verdadeira. Com log-retornos a soma dá a resposta certa; com retornos simples ela dá zero.

O log-retorno também é simétrico: dobrar de preço ($r = +0{,}693$) e cair pela metade ($r = -0{,}693$) têm a mesma magnitude, enquanto em retorno simples são $+100\%$ e -50%.

1.7.2 O que o log-retorno não faz

A aditividade no tempo tem um preço. O log-retorno de uma carteira *não* é a média ponderada dos log-retornos dos ativos, porque o logaritmo de uma soma não é a soma dos logaritmos. A Equação (1.2) vale apenas para retornos simples.

Tabela 1.1 Qual retorno usar

Tarefa	Retorno simples	Log-retorno
Agregar ativos em uma carteira no mesmo período	soma ponderada, exata	apenas aproximada
Acumular vários períodos de um mesmo ativo	produto de $(1 + R_t)$	soma, exata
Informar ganho ou perda a um investidor	sim	converter antes
Modelagem estatística de séries temporais	possível	preferido

Na prática usam-se os dois. Modelos de séries temporais e de volatilidade (Capítulo 4) costumam ser escritos em log-retornos; contas de carteira, de risco e de backtest (Capítulos 5 a 8) usam retornos simples, porque misturam ativos. A conversão é imediata: $R = e^r - 1$ e $r = \ln(1 + R)$.

1.7.3 As duas contas em pandas

Código 1.3 Retorno simples e log-retorno de PETR4 (`cap01/l03_retornos.py`)

```
import numpy as np
import pandas as pd

precos = pd.read_csv("../../dados/precos_ajustados.csv",
                     index_col=0, parse_dates=True)
petr = precos["PETR4.SA"].dropna()

simples = petr.pct_change().dropna()          # R_t = P_t/P_{t-1}-1
log = np.log(petr / petr.shift(1)).dropna()  # r_t = ln(1 + R_t)

ano = "2024"
composto = (1 + simples[ano]).prod() - 1     # compõe os simples
via_log = np.exp(log[ano].sum()) - 1         # soma os logs
soma = simples[ano].sum()                    # o erro clássico
dif = (simples - log).abs().max()

print(f"retorno de {ano}, compondo os simples: {composto:.4%}")
print(f"retorno de {ano}, somando os logs:     {via_log:.4%}")
print(f"soma ingênua dos retornos simples:      {soma:.4%}")
print(f"maior diferença diária |R - r|:         {dif:.4%}")
```

```
retorno de 2024, compondo os simples: 19.0859%
retorno de 2024, somando os logs:     19.0859%
soma ingênua dos retornos simples:      20.5969%
maior diferença diária |R - r|:         5.5389%
```

Os dois caminhos corretos chegam ao mesmo retorno de 19,09% em 2024, que é a variação do preço ajustado de R$ 25,66 para R$ 30,56 vista no Código 1.2. A soma dos retornos simples erra por um ponto e meio percentual em um único ano. A última linha mostra que, no pior dia da amostra, a diferença entre as duas medidas passou de cinco pontos percentuais.

Armadilha **A primeira linha vazia**

Tanto `pct_change()` quanto `shift(1)` produzem um valor ausente (`NaN`) na primeira data, porque não há preço anterior. Remova-o com `dropna()` logo após o cálculo. Nunca o substitua por zero: um retorno zero inventado reduz artificialmente a volatilidade.

1.8 O preço certo: ajuste por proventos

Quando uma empresa paga dividendos, o preço da ação cai aproximadamente o valor pago na data em que o papel passa a ser negociado sem direito ao provento (a data *ex*). Essa queda não é prejuízo: o acionista recebeu o dinheiro em conta. Desdobramentos e grupamentos provocam saltos ainda maiores, sem nenhum efeito econômico. Se você calcular retornos com o preço de tela, o preço pelo qual o papel foi de fato negociado em cada dia, vai registrar perdas que não existiram.

O **preço ajustado** corrige isso reescrevendo o histórico como se todos os proventos tivessem sido reinvestidos na própria ação. Na data em que a série é baixada, o preço ajustado mais recente coincide com o de tela, e os anteriores são reduzidos na proporção dos proventos pagos depois deles. O retorno calculado sobre a série ajustada é o **retorno total**.

O conjunto de dados do livro traz duas versões de cada série. O arquivo `precos_ajustados.csv` é ajustado por proventos e desdobramentos. O arquivo `precos_sem_ajuste.csv` guarda o fechamento que o Yahoo Finance distribui sem o ajuste por proventos; essa série já vem corrigida por desdobramentos, de modo que também não é exatamente o preço de tela de datas anteriores a um desdobramento.

O arquivo do livro foi baixado em 2026. Proventos pagos depois de dezembro de 2025 já reduziram toda a série, e por isso nem o último preço ajustado de 2025 (R$ 28,98 no Código 1.2) é a cotação daquele dia.

A Figura 1.3 mostra o tamanho do erro possível. Quem medir o desempenho de PETR4 pela série sem ajuste vai concluir que a ação rendeu 50% em seis anos, quando o acionista que reinvestiu os proventos multiplicou o capital por quase cinco.

Armadilha **Preço ajustado não é preço histórico**

O preço ajustado serve para calcular retornos, e só. Ele não é o preço pelo qual o papel foi negociado no passado, e muda a cada novo provento. Uma regra que dependa do nível do preço, como “comprar se a ação estiver abaixo de R$ 10”, precisa do preço efetivamente negociado naquela data, que nenhuma das duas séries do livro garante.

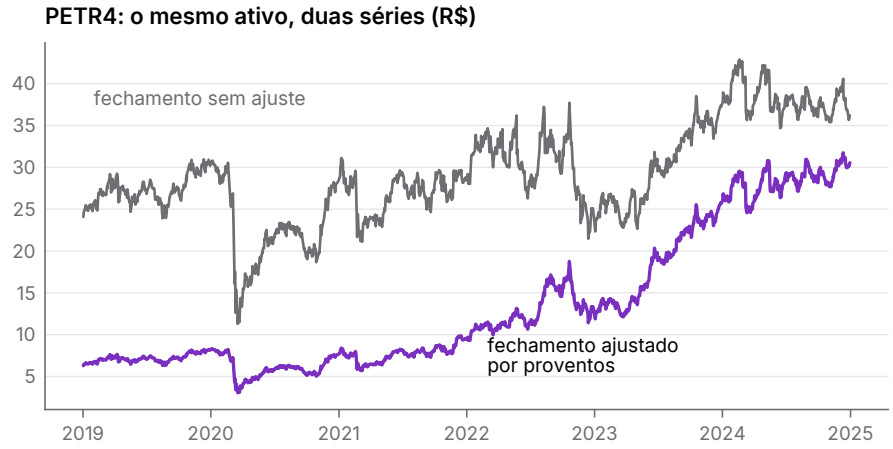


Figura 1.3 PETR4 de 2019 a 2024. Pelo fechamento sem ajuste por proventos a ação subiu 50,4% no período; pelo preço ajustado, 386,0%. Toda a diferença são dividendos e juros sobre capital próprio, que foram muito altos entre 2021 e 2023.

1.9 Acumular e anualizar

Dois números resumem o histórico de um ativo: quanto ele rendeu por ano e quanto oscilou por ano. Como os dados costumam ser diários, é preciso convertê-los. A convenção brasileira é de 252 dias úteis por ano.

Definição 1.3 Retorno anualizado (geométrico)

Para uma série de T retornos diários que leva o preço de P_0 a P_T em A anos, o retorno anualizado é a taxa constante que, composta ano a ano, produz o mesmo resultado final:

$$R_{\text{a.a.}} = \left(\frac{P_T}{P_0}\right)^{1/A} - 1 = \left[\prod_{t=1}^{T}(1+R_t)\right]^{1/A} - 1. \tag{1.7}$$

Em inglês essa medida é o CAGR (*compound annual growth rate*).

O número de anos A pode ser medido no calendário, como faz o Código 1.4, ou aproximado por $T/252$. A segunda forma é a convenção de mercado e a que os capítulos seguintes adotam; ela superestima um pouco a taxa, porque a B3 tem cerca de 248 pregões por ano, e não 252. Em log-retornos a mesma conta é uma soma dividida pelo tempo: $r_{\text{a.a.}} = \frac{1}{A}\sum_t r_t$, que com $A = T/252$ vira $252\,\bar{r}$, e $R_{\text{a.a.}} = e^{r_{\text{a.a.}}} - 1$.

Resultado 1.4 Regra da raiz do tempo

Se os log-retornos diários são não correlacionados entre si e têm a mesma variância σ_d^2, a variância do log-retorno de T dias é $T\sigma_d^2$. A volatilidade anualizada é, portanto,

$$\sigma_{\text{a.a.}} = \sigma_d\sqrt{252}. \tag{1.8}$$

A justificativa cabe em uma linha. Pela Equação (1.5), o retorno anual é a soma de 252 retornos diários, e a variância de uma soma de variáveis não correlacionadas é a soma das variâncias: $\operatorname{Var}(r_1 + \cdots + r_{252}) = 252\,\sigma_d^2$. O desvio-padrão é a raiz disso. O retorno esperado cresce proporcionalmente ao tempo e a volatilidade cresce com a raiz do tempo. A razão entre a volatilidade e o

retorno esperado cai com $1/\sqrt{T}$, e por isso, quando o retorno esperado em log é positivo, a probabilidade de terminar abaixo do ponto de partida diminui em horizontes longos. A dispersão do patrimônio final, porém, continua crescendo com o horizonte: prazo longo não elimina risco.

> **Armadilha** **A raiz do tempo é uma hipótese**
>
> A regra depende de os retornos não serem autocorrelacionados e de a variância ser constante. A primeira condição vale aproximadamente para ações líquidas. A segunda não: a volatilidade se agrupa no tempo, como se vê no painel de baixo da Figura 1.1. Use $\sqrt{252}$ como convenção de escala, sabendo que a volatilidade de um período de crise não se obtém multiplicando a de um período calmo.

Código 1.4 Retorno e volatilidade anualizados de cinco ativos (`cap01/l04_anualizar.py`)

```python
import numpy as np
import pandas as pd

precos = pd.read_csv("../../dados/precos_ajustados.csv",
                     index_col=0, parse_dates=True)
ativos = ["PETR4.SA", "VALE3.SA", "ABEV3.SA", "WEGE3.SA",
          "BOVA11.SA"]
p = precos.loc["2015":"2024", ativos].dropna()
log = np.log(p / p.shift(1)).dropna()

anos = (p.index[-1] - p.index[0]).days / 365.25    # calendário
crescimento = p.iloc[-1] / p.iloc[0]
resumo = pd.DataFrame({
    "retorno total": crescimento - 1,
    "retorno a.a.": crescimento ** (1 / anos) - 1,   # CAGR
    "vol a.a.": log.std() * np.sqrt(252),
})
print(f"{len(log)} retornos em {anos:.2f} anos")
print(resumo.map(lambda v: f"{v:.1%}"))
```

```
2462 retornos em 9.99 anos
          retorno total retorno a.a. vol a.a.
PETR4.SA        1203.1%        29.3%    48.2%
VALE3.SA         382.8%        17.1%    44.0%
ABEV3.SA          13.0%         1.2%    26.5%
WEGE3.SA         957.8%        26.6%    33.7%
BOVA11.SA        148.1%         9.5%    24.6%
```

A primeira linha da saída conta 2462 retornos, e não os cerca de 2480 pregões do período: o `dropna()` descarta as datas em que algum dos cinco papéis não tem preço, assunto do Capítulo 2. Leia a tabela com cuidado. PETR4 e WEGE3 tiveram retornos anualizados parecidos entre 2015 e 2024, mas a volatilidade da primeira foi quase uma vez e meia a da segunda. ABEV3 oscilou pouco e também rendeu pouco: volatilidade baixa não é sinônimo de bom investimento. E o BOVA11, um fundo que replica o Ibovespa, tem a menor volatilidade do grupo, um primeiro sinal do efeito da diversificação que o Capítulo 6 formaliza. Note ainda que dez anos de histórico de cinco papéis escolhidos hoje não dizem

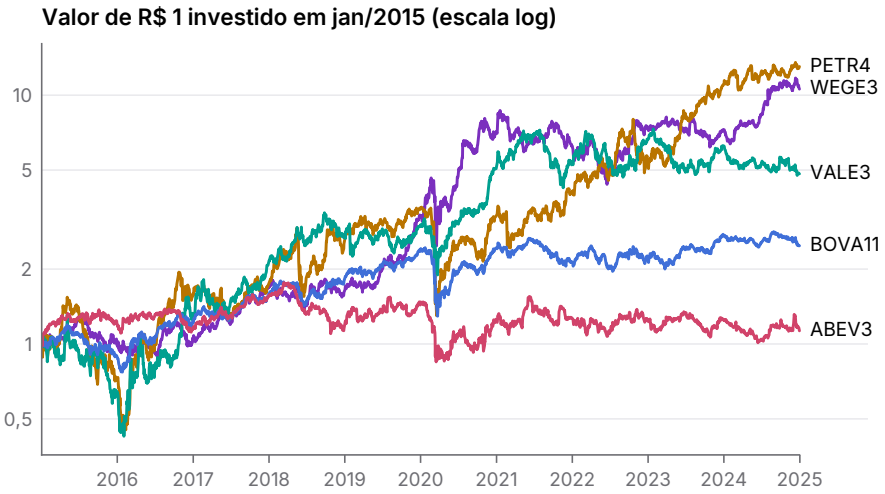


Figura 1.4 Valor de R$ 1 aplicado em janeiro de 2015. Na escala logarítmica, distâncias verticais iguais representam variações percentuais iguais, e uma reta corresponde a crescimento a taxa constante. É a escala adequada para comparar ativos em períodos longos.

nada sobre os próximos dez; o Capítulo 7 mostra como essa escolha a posteriori contamina resultados.

Intuição

Em escala linear, uma queda de 10 para 5 parece dez vezes maior do que uma de 1 para 0,5, embora ambas sejam perdas de 50%. A escala logarítmica do eixo vertical desenha as duas com a mesma altura. Ela é o equivalente gráfico do log-retorno.

O que levar deste capítulo

- Estatística, risco e desempenho são definidos sobre retornos, que são comparáveis entre ativos e muito mais estáveis do que preços.
- Retornos simples somam entre ativos de uma carteira; log-retornos somam ao longo do tempo. Nenhum dos dois faz as duas coisas.
- Para acumular retornos simples, multiplique os fatores $(1 + R_t)$. Somá-los é um erro que cresce com a volatilidade.
- Use sempre o preço ajustado por proventos para calcular retornos, e, para regras que dependam do nível do preço, o preço efetivamente negociado, que precisa vir de outra fonte.
- Retorno anualiza por composição (ou multiplicando a média dos logs por 252); volatilidade anualiza por $\sqrt{252}$, sob hipóteses que os dados só cumprem em parte.

Exercícios

1.1 [básico] Uma ação fecha a R$ 40,00, R$ 42,00, R$ 39,90 e R$ 41,50 em quatro pregões seguidos. Calcule à mão os retornos simples e os log-retornos diários, o retorno acumulado pelos dois caminhos, e confira que coincidem.

1.2 [básico] Um fundo perde 20% em um ano. Quanto precisa render no ano seguinte para recuperar o capital inicial? Generalize: depois de uma perda de fração x, qual é o ganho necessário?

1.3 [básico] Carregue `livro/dados/precos_ajustados.csv` e calcule o retorno e a volatilidade anualizados de `VALE3.SA` em cada ano civil de 2015 a 2024. Em que ano a volatilidade foi maior?

1.4 [intermediário] Mostre com um contraexemplo numérico de dois ativos que o log-retorno de uma carteira não é a média ponderada dos log-retornos dos ativos. Qual é o sinal do erro? (Dica: o logaritmo é uma função côncava.)

1.5 [intermediário] Repita o Código 1.4 usando `precos_sem_ajuste.csv`, que não é ajustado por proventos. Para quais ativos a diferença de retorno anualizado é maior, e por quê?

1.6 [intermediário] A média aritmética $\bar{R}$ dos retornos simples e o retorno geométrico por período G se relacionam aproximadamente por $G \approx \bar{R} - \sigma^2/2$. Verifique a aproximação com os retornos diários de PETR4 e explique, a partir dela, por que volatilidade "custa" retorno composto.

1.7 [desafio] Calcule a volatilidade de PETR4 a partir de retornos diários, semanais e mensais, anualizando cada uma pela raiz do tempo. Os três números coincidem? O que uma diferença sistemática indicaria sobre a autocorrelação dos retornos?

Para ir além

- McKinney (2022), capítulos 4, 5 e 11, para NumPy, pandas e séries temporais em profundidade.
- Hilpisch (2018), capítulos 4, 5 e 8, com a mesma matéria já voltada a dados financeiros.
- Tsay (2010), seção 1.1, para as definições formais de retorno e suas propriedades.
- Campbell et al. (1997), capítulo 1, que discute com cuidado a escolha entre retornos simples e continuamente compostos.

Capítulo 2

Coleta, limpeza e visualização de dados

2

Uma estratégia testada sobre dados errados produz um resultado errado com aparência de certo. Antes de estimar qualquer coisa é preciso saber de onde veio cada número, o que fazer quando ele falta, como reconhecer quando ele está errado e como desenhar a série de um jeito que mostre o problema em vez de escondê-lo. Este capítulo percorre esse caminho com ações brasileiras, o S&P 500 e o dólar.

Ao terminar este capítulo você consegue

- ler uma barra OHLCV e dizer o que cada fonte de dados entrega e o que ela não garante;
- baixar dados uma vez, congelá-los em arquivo e trabalhar sempre a partir da cópia;
- diagnosticar valores ausentes e tratá-los sem usar informação do futuro;
- distinguir um retorno extremo verdadeiro de um erro da fonte e registrar a limpeza em um script;
- construir gráficos com a interface orientada a objetos do Matplotlib e explorar dados com o Plotly;
- conduzir uma análise exploratória de ativos em moedas diferentes.

2.1 De onde vêm os dados

2.1.1 A barra OHLCV

O registro mais fino de um mercado é a lista de todos os negócios, cada um com horário, preço e quantidade. Para a maior parte das análises essa lista é resumida em intervalos fixos, e o resumo de cada intervalo é a **barra OHLCV**: o preço do primeiro negócio (*open*, abertura), o maior e o menor preço negociados (*high* e *low*, máxima e mínima), o preço do último negócio (*close*, fechamento) e a quantidade negociada (*volume*). Neste livro o intervalo é quase sempre de um pregão.

O fechamento é o campo mais usado porque é o preço pelo qual a maioria das carteiras é avaliada no fim do dia. A abertura é o primeiro preço em que uma decisão tomada na véspera pode ser executada, a distância entre máxima e mínima mede a agitação dentro do dia, e o volume diz se havia liquidez para negociar ao preço registrado. O Código 2.1 lê as barras de PETR4 tais como vieram da fonte e aplica testes de coerência que valem para qualquer base.

Código 2.1 Leitura e testes de coerência de barras OHLCV (`cap02/l01_ohlcv.py`)

```
import pandas as pd

ohlcv = pd.read_csv("../../dados/bruto/petr4_ohlcv.csv",
                    index_col=0, parse_dates=True)
ohlcv = ohlcv[["Open", "High", "Low", "Close", "Volume"]]
print(ohlcv.loc["2024-12-18":"2024-12-19"].round(2))

# testes de coerência: a máxima é o maior preço do dia,
# a mínima é o menor, e um pregão de verdade tem negócios
corpo = ohlcv[["Open", "Close"]]
fora = ((ohlcv["High"] < corpo.max(axis=1) - 0.005)
        | (ohlcv["Low"] > corpo.min(axis=1) + 0.005))
parado = ohlcv["Volume"] == 0

print(f"\nlinhas: {len(ohlcv)}   incoerentes: {fora.sum()}   "
      f"volume zero: {parado.sum()}")
print(ohlcv.loc["2011-08-15":"2011-08-17",
                ["Close", "Volume"]].round(3))
```

```
             Open   High    Low  Close    Volume
Date
2024-12-18  31.13  31.23  30.34  30.34  58126400
2024-12-19  30.63  30.85  30.07  30.22  43268400

linhas: 3974   incoerentes: 1   volume zero: 54
            Close    Volume
Date
2011-08-15  5.128         0
2011-08-16  5.128         0
2011-08-17  4.554  18831300
```

Os testes encontram defeitos. Há uma barra cujo fechamento está fora do intervalo entre mínima e máxima, e 54 linhas com volume zero em uma das ações mais negociadas do país. As últimas linhas da saída mostram o que acontece nelas: a fonte repete o preço, e quando os negócios reaparecem, em 17 de agosto de 2011, a série registra de uma vez uma queda de 11,2% ($4{,}554/5{,}128 - 1$). A Seção 2.5 investiga essas linhas. Os preços do arquivo são ajustados por proventos (Capítulo 1), e por isso os valores de 2011 são muito menores do que os de tela da época.

2.1.2 Fontes públicas e seus limites

Yahoo Finance, via yfinance. A biblioteca `yfinance` consulta as páginas do Yahoo Finance e devolve um `DataFrame` pronto. É gratuita e cobre ações, índices, fundos e moedas de muitos países. Ela não é um produto do Yahoo nem uma fonte oficial: não há garantia de disponibilidade, o formato de resposta já mudou entre versões, e os dados chegam com falhas como as que acabamos de

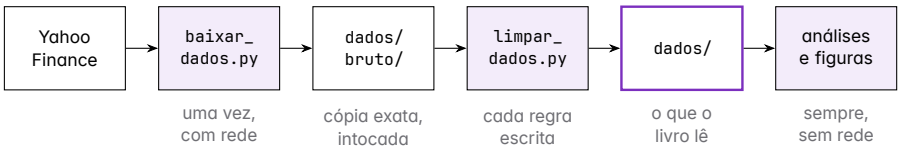


Figura 2.1 O caminho dos dados do livro. Só o primeiro passo fala com o mundo externo. A cópia bruta nunca é alterada, de modo que toda correção pode ser refeita e conferida.

ver. Cada ativo é identificado por um código, o **ticker**. No Yahoo, os tickers da B3 levam o sufixo `.SA` (`PETR4.SA`), índices começam com um circunflexo (`^BVSP`, `^GSPC`) e pares de moedas terminam em `=X` (`BRL=X` é o preço do dólar em reais).

No Yahoo, a coluna *Close* já vem corrigida por desdobramentos; a *Adj Close* corrige também dividendos. Com `auto_adjust=True` a biblioteca entrega só a série totalmente ajustada.

B3. A bolsa publica em seu site arquivos com as cotações históricas de todos os papéis negociados. São a referência oficial dos preços de tela, sem ajuste por proventos: servem para conferir um preço suspeito, e não para calcular retornos sem tratamento.

Banco Central. O Sistema Gerenciador de Séries Temporais (SGS) distribui séries econômicas por uma interface de programação aberta, que responde em JSON. Cada série tem um código numérico: 12 é a taxa CDI diária, 11 é a Selic diária e 1 é a cotação de venda do dólar PTAX. É a fonte natural para a taxa livre de risco usada no resto do livro.

Fontes pagas, como Bloomberg, LSEG e Economatica, costumam entregar dados mais conferidos e o histórico de empresas que deixaram de existir, mas o cuidado exigido de quem as usa é o mesmo. Entre as perguntas que Chan (2021) manda fazer a qualquer base antes de um teste estão duas que voltam neste livro: os preços estão ajustados por desdobramentos e dividendos? A base inclui os ativos que saíram do mercado? O Capítulo 7 trata da segunda.

2.2 Baixar uma vez, guardar o bruto, limpar à parte

Um script que baixa os dados toda vez que roda tem três defeitos. Ele para de funcionar quando a fonte sai do ar ou muda de formato. Ele produz números diferentes em dias diferentes, porque o histórico cresce e porque preços ajustados são reescritos a cada novo provento. E ele impede que outra pessoa confira o resultado, já que ela nunca verá os mesmos dados que você viu. A solução é separar o trabalho em três etapas, como na Figura 2.1: baixar uma vez e guardar a cópia exata do que a fonte devolveu; corrigir os defeitos em um script à parte, que lê a cópia bruta e grava arquivos limpos; e fazer todas as análises a partir dos arquivos limpos.

O Código 2.2 é o trecho central do script que baixou os dados deste livro em 6 de outubro de 2026. Ele depende da rede e do estado do Yahoo no dia da execução, e por isso não tem saída impressa aqui. As linhas omitidas definem a pasta `BRUTO`, a lista `TICKERS` com os onze ativos e o dicionário `PERIODO`, com `start="2010-01-01"` e `end="2026-01-01"`.

Código 2.2 Download único da cópia bruta (`scripts/baixar_dados.py`)

```
if (BRUTO / "precos_ajustados.csv").exists():
    raise SystemExit("dados/bruto/ já existe; nada foi baixado.")
BRUTO.mkdir(parents=True, exist_ok=True)

ajustado = yf.download(TICKERS, auto_adjust=True, **PERIODO)
ajustado["Close"].to_csv(BRUTO / "precos_ajustados.csv")
ajustado["Volume"].filter(like=".SA").to_csv(BRUTO/"volume.csv")

sem_ajuste = yf.download(TICKERS, auto_adjust=False, **PERIODO)
sem_ajuste["Close"].to_csv(BRUTO / "precos_sem_ajuste.csv")
```

As duas primeiras linhas são a guarda: se a cópia já existe, o script para sem baixar nada, e os números do livro não mudam por acidente. A data `end` não entra no resultado, e por isso se pede até 1º de janeiro de 2026 para incluir o último pregão de 2025. Com vários tickers, `yf.download` devolve colunas em dois níveis (campo e ativo), e `ajustado["Close"]` seleciona a tabela de fechamentos. O argumento `auto_adjust` deve ser escrito sempre, porque o valor padrão já mudou entre versões da biblioteca. Com `False`, o fechamento vem sem ajuste por proventos, mas já corrigido por desdobramentos, como descrito no Capítulo 1.

O Código 2.3 aplica o mesmo padrão ao Banco Central. Ele baixa o CDI e o dólar PTAX de 2010 a 2025, usados nas seções seguintes; as linhas finais, omitidas, leem o arquivo e imprimem uma amostra.

Código 2.3 CDI e dólar PTAX pelo SGS do Banco Central (`cap02/baixar_bcb.py`)

```
DADOS = Path("../../dados")
URL = ("https://api.bcb.gov.br/dados/serie/"
       "bcdata.sgs.{codigo}/dados")
SERIES = {"cdi": 12, "ptax_venda": 1}     # nome: código no SGS
JANELAS = [("01/01/2010", "31/12/2017"),  # o SGS limita a 10 anos
           ("01/01/2018", "31/12/2025")]  # por consulta diária


def baixar(codigo):
    partes = []
    for inicio, fim in JANELAS:
        resposta = requests.get(
            URL.format(codigo=codigo), timeout=30,
            params={"formato": "json", "dataInicial": inicio,
                    "dataFinal": fim})
        resposta.raise_for_status()       # erro visível
        partes.append(pd.DataFrame(resposta.json()))
    bruto = pd.concat(partes)
    datas = pd.to_datetime(bruto["data"], format="%d/%m/%Y")
    return pd.Series(bruto["valor"].astype(float).values,
                     index=datas.rename("Date"))


arquivo = DADOS / "c02_bcb.csv"
if not arquivo.exists():                  # baixa uma única vez
    series = {nome: baixar(c) for nome, c in SERIES.items()}
    pd.DataFrame(series).to_csv(arquivo)
```

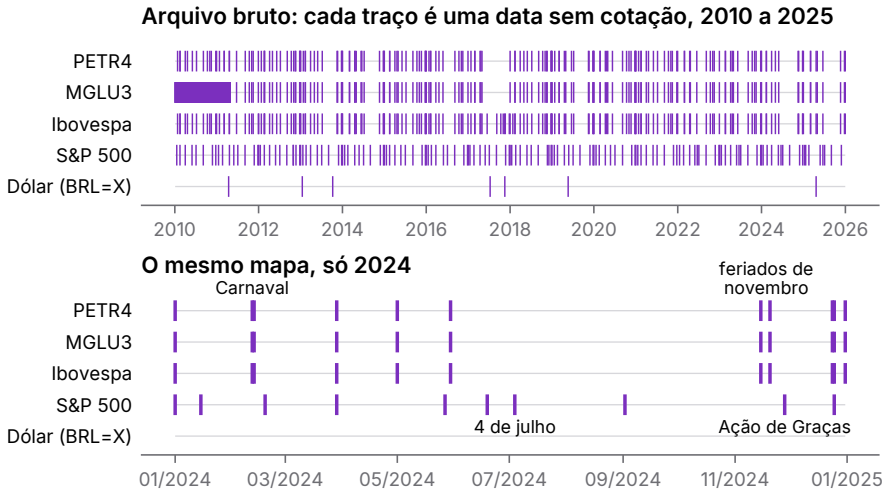


Figura 2.2 Mapa dos valores ausentes no arquivo bruto. No painel de cima, o bloco sólido de MGLU3 é o período anterior à abertura de capital, e a maior parte dos traços espalhados são feriados. No de baixo, vê-se que os feriados brasileiros e americanos raramente coincidem, e que o dólar tem cotação em quase todos os dias.

```
                cdi  ptax_venda
Date
2024-01-02  0.043739      4.8916
2024-01-03  0.043739      4.9212
2024-01-04  0.043739      4.9188
dias: 4019   CDI acumulado em 2024: 10.88%
```

O serviço recusa consultas de séries diárias com mais de dez anos, e por isso o pedido é feito em duas janelas de oito anos. A chamada `raise_for_status()` faz o script parar se o servidor responder com erro; seguir em frente com uma tabela vazia é o tipo de problema que só aparece semanas depois. O CDI vem em porcentagem ao dia: pela convenção de 252 dias (Seção 1.9), 0,043739% ao dia equivale a $1{,}00043739^{252} - 1 = 11{,}65\%$ ao ano, e a composição dos valores diários de 2024 dá 10,88%.

2.3 Dados faltantes: primeiro o diagnóstico

Um valor ausente aparece no pandas como `NaN`. Ele não gera erro. Somas e médias o ignoram em silêncio, e um retorno calculado a partir dele vira outro `NaN`, de modo que uma falha pode encurtar uma amostra sem que nada avise. Em séries financeiras diárias há três causas, que pedem tratamentos diferentes:

- **Calendário.** O mercado estava fechado naquele dia. Brasil e Estados Unidos têm feriados diferentes, e uma tabela com ativos dos dois países tem buracos dos dois lados.
- **O ativo não existia.** A série começa depois do início da tabela, ou termina antes do fim.
- **Falha da fonte.** Houve pregão, mas o dado não foi registrado ou veio corrompido.

O diagnóstico se faz sobre a cópia bruta. O Código 2.4 mede o tamanho de cada causa nos arquivos de `dados/bruto/`, e a Figura 2.2 mostra onde os buracos estão. Nesta e nas próximas listagens omitimos as linhas iniciais, que importam as bibliotecas e leem os arquivos nas variáveis `precos`, `volume` e `bcb`.

Código 2.4 Diagnóstico de valores ausentes (`cap02/l02_faltantes.py`)

```
ativos = ["PETR4.SA", "MGLU3.SA", "^BVSP", "^GSPC", "BRL=X"]
p = precos[ativos]

diagnostico = pd.DataFrame({
    "primeira data": p.apply(pd.Series.first_valid_index),
    "faltantes": p.isna().sum(),
    "% do total": (p.isna().mean() * 100).round(1),
})
print(diagnostico)

# quem estava aberto em cada data? Houve pregão na B3 se
# alguma ação teve negócio; na NYSE, se o S&P 500 tem valor
acoes = volume.filter(like=".SA")     # só as ações
b3 = (acoes > 0).any(axis=1).reindex(p.index)
nyse = p["^GSPC"].notna()
print(f"\nlinhas no arquivo:        {len(p)}")
print(f"B3 e NYSE abertas:        {(b3 & nyse).sum()}")
print(f"só a NYSE aberta:         {(~b3 & nyse).sum()}")
print(f"só a B3 aberta:           {(b3 & ~nyse).sum()}")
print(f"nenhuma aberta:           {(~b3 & ~nyse).sum()}")

sem_pregao = ~b3 & p["PETR4.SA"].notna()
print(f"\nPETR4 com preço em dia sem pregão: {sem_pregao.sum()}")
sem_indice = b3 & p["^BVSP"].isna()
print("pregão sem Ibovespa:",
      [f"{d:%d/%m/%Y}" for d in p.index[sem_indice]])
```

```
         primeira data  faltantes  % do total
PETR4.SA    2010-01-04        196         4.7
MGLU3.SA    2011-05-02        523        12.5
^BVSP       2010-01-04        205         4.9
^GSPC       2010-01-04        146         3.5
BRL=X       2010-01-04          7         0.2

linhas no arquivo:        4170
B3 e NYSE abertas:        3863
só a NYSE aberta:         161
só a B3 aberta:           102
nenhuma aberta:           44

PETR4 com preço em dia sem pregão: 9
pregão sem Ibovespa: ['02/01/2012']
```

A tabela separa as causas. MGLU3 só tem preço a partir de 2 de maio de 2011, e a maior parte de seus 523 valores ausentes é o tempo em que a ação não existia. Para os demais a falta é quase toda de calendário: das 4170 datas do arquivo, 3863 têm pregão nas duas bolsas. As 44 linhas em que ambas estavam fechadas existem porque o mercado de câmbio funciona em feriados, e a tabela conjunta guarda toda data em que pelo menos uma série tem valor. O dólar tem sete ausências isoladas em dias em que alguma das bolsas funcionou, o que as coloca na terceira categoria.

Repare em como o código decide se houve pregão na B3: pelo volume, e não pela presença de preço. As duas últimas linhas da saída mostram por quê. Em nove datas sem nenhum negócio na bolsa, PETR4 tem preço no arquivo: a

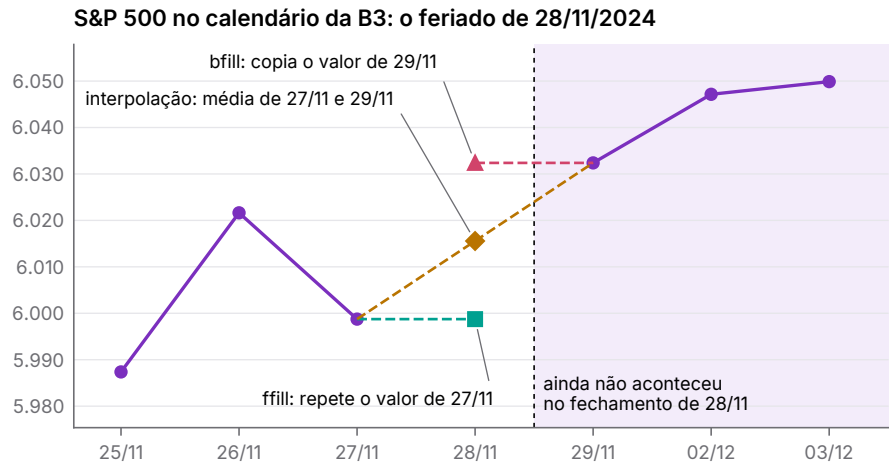


Figura 2.3 Em 28/11/2024 a bolsa americana não abriu. O `ffill` usa o fechamento de 27/11, que já era conhecido. A interpolação e o `bfill` usam o fechamento de 29/11, que ninguém conhecia no dia 28.

fonte criou linhas que não deveriam existir. Em oito delas o índice está ausente, como deveria; na nona, 29 de dezembro de 2017, a fonte preencheu também o índice. E em 2 de janeiro de 2012, pregão normal, falta o índice. Essas oito datas e a de 2012 são a diferença entre as 205 ausências do Ibovespa e as 196 de PETR4. Um valor presente pode ser tão errado quanto um ausente, assunto da Seção 2.5.

2.4 Remover, repetir ou interpolar

Há três tratamentos usuais. O método `dropna()` remove as linhas incompletas. O método `ffill()` (*forward fill*) repete o último valor conhecido. O método `interpolate()` liga por uma reta o valor anterior ao seguinte. Existe ainda `bfill()`, que copia para trás o próximo valor conhecido. A Figura 2.3 aplica todos ao S&P 500 no feriado americano de Ação de Graças de 2024, dia de pregão normal na B3.

Códigos antigos, como os de `aula2/yfinance.py` e `aula2/interpolacao.py`, escrevem `fillna(method="ffill")`. As versões atuais do pandas não aceitam mais essa forma.

> **Definição 2.1** Look-ahead
>
> Há *look-ahead*, ou **viés de antecipação**, quando o valor atribuído à data t depende de informação que só ficou disponível depois de t. Um dado assim serve para descrever o passado, mas qualquer regra de decisão testada sobre ele usa conhecimento que não existia no momento da decisão.

A interpolação e o `bfill` introduzem esse viés por construção. Na figura, o valor interpolado para o dia 28 é 6015,56, a média dos fechamentos de 27 e 29 de novembro: ele "sabe" que a bolsa americana subiria no dia seguinte. Em uma tabela que alimenta um modelo ou um *backtest*, esse vazamento melhora os resultados de forma artificial e é difícil de localizar depois. Preencher com a média da coluna tem o mesmo defeito, pois a média usa a amostra inteira. Em dados que alimentam decisões, use apenas `dropna` e `ffill`. O Capítulo 7 trata das outras portas por onde o futuro entra em um teste.

Entre os dois tratamentos seguros, a escolha depende do que a tabela vai calcular. O Código 2.5 compara, já nos arquivos limpos, os efeitos sobre a volatilidade anualizada do S&P 500 e sobre sua correlação com o Ibovespa; o laço final, omitido, imprime o resumo de cada caso.

Código 2.5 Efeito de cada tratamento sobre volatilidade e correlação (`cap02/l03_preencher.py`)

```
p = precos[["^BVSP", "^GSPC"]]
p = p.dropna(subset="^BVSP")          # só os dias de pregão na B3

sp = p["^GSPC"]
opcoes = pd.DataFrame({
    "original": sp,
    "ffill": sp.ffill(),              # repete o último valor
    "interpolate": sp.interpolate(),  # reta entre os vizinhos
    "bfill": sp.bfill(),              # copia o valor seguinte
})
print(opcoes.loc["2024-11-26":"2024-12-02"].round(2))


def resumo(tabela):
    r = tabela.pct_change().dropna()
    return (len(r), r["^GSPC"].std() * np.sqrt(252),
            (r["^GSPC"] == 0).sum(), r.corr().iloc[0, 1])


casos = {"dropna": p.dropna(), "ffill": p.ffill(),
         "interpolate": p.interpolate()}
```

```
            original    ffill  interpolate    bfill
Date
2024-11-26   6021.63  6021.63      6021.63  6021.63
2024-11-27   5998.74  5998.74      5998.74  5998.74
2024-11-28       NaN  5998.74      6015.56  6032.38
2024-11-29   6032.38  6032.38      6032.38  6032.38
2024-12-02   6047.15  6047.15      6047.15  6047.15

              dias   vol S&P  zeros  correlação
dropna        3862    17.59%      1      0.5679
ffill         3963    17.36%    102      0.5625
interpolate   3963    17.30%      1      0.5631
```

Repetir o preço cria um retorno igual a zero no dia do feriado. São cerca de cem zeros fabricados, que puxam a volatilidade estimada para baixo, de 17,59% para 17,36%, e enfraquecem a correlação, porque nesses dias o Ibovespa se moveu e o S&P 500 "ficou parado". Aqui o efeito é pequeno, pois os feriados são 2,5% da amostra. Em um ativo pouco negociado, com dias seguidos sem preço, ele deixa de ser.

Intuição

Preencher com o último valor responde à pergunta "qual era a melhor informação disponível naquele dia?". Remover a linha responde a "em que dias eu tenho uma observação de verdade?". Para marcar o valor de uma carteira em um feriado estrangeiro, a primeira pergunta é a certa. Para estimar volatilidade e correlação, é a segunda.

Na prática, para estatísticas de retornos de vários ativos, alinhe os preços com `dropna()` e só então calcule os retornos, de modo que cada retorno cubra o mesmo intervalo de tempo em todas as colunas. Reserve o `ffill()` para

grandezas que precisam existir todos os dias, como a taxa de câmbio que converte uma posição, de preferência com o argumento `limit`, para que um buraco longo continue visível. O período anterior à existência do ativo se trata cortando a amostra, nunca preenchendo. E falhas da fonte se investigam uma a uma.

> **Armadilha** **Preencher antes de olhar**
>
> Um `ffill()` aplicado à tabela inteira logo após a leitura esconde as três causas de uma vez. Conte os `NaN` por coluna, descubra a causa de cada grupo e só depois escolha o tratamento. Preencher retornos ausentes com zero é o mesmo erro com outro nome.

2.5 Erros de dados e retornos extremos

Um retorno diário de −20% pode ser uma notícia ou um defeito. Os dois casos pedem ações opostas: o retorno verdadeiro precisa ficar na amostra, porque é o tipo de dia que uma medida de risco tem de enxergar, e o defeito precisa sair. Os defeitos mais comuns em preços diários são quatro.

- **Preço repetido.** A fonte copia o último valor em dias sem informação, e a variação acumulada aparece toda no dia em que o dado volta.
- **Evento societário sem ajuste.** Em um desdobramento de 1 para 4, uma ação de R$ 80 passa a valer R$ 20 de um dia para o outro, e uma série não ajustada registra um retorno de −75% que nenhum acionista sofreu. O mesmo vale para grupamentos, bonificações e cisões.
- **Cotação errada isolada.** Um único preço fora do lugar gera dois retornos grandes e opostos em dias seguidos.
- **Data deslocada.** O valor está certo, mas com o rótulo do dia errado. A Seção 2.8 mostra um caso.

2.5.1 Preço repetido sem negócio

O Código 2.1 levantou a suspeita em PETR4. Para medir o problema em todas as ações é preciso o volume, que o download guardou no arquivo `volume.csv`. A regra, importada do script de limpeza do livro, marca um ponto quando o volume do papel é zero e o fechamento sem ajuste é igual ao último fechamento válido. O Código 2.6 a aplica e a compara com uma versão ingênua, que procura preços ajustados iguais em linhas vizinhas.

Código 2.6 Pontos com volume zero e preço repetido (`cap02/l04_repetidos.py`)

```
tela = ler("precos_sem_ajuste.csv")      # arquivos brutos
ajustado = ler("precos_ajustados.csv")
volume = ler("volume.csv")
acoes = list(volume.columns)

falso = pd.DataFrame(
    {a: sem_negocio(tela[a], volume[a]) for a in acoes})
feriado = falso.all(axis=1)              # nenhum papel negociou
# a regra ingênua: preço ajustado igual ao da linha anterior
ingenua = (ajustado[acoes].diff() == 0) & (volume == 0)

tabela = pd.DataFrame({
```

```
    "repetidos": falso.sum(),
    "em 2011": falso.loc["2011"].sum(),
    "regra ingênua": ingenua.sum(),
})
```

```
        repetidos  em 2011  regra ingênua
ABEV3          22        7             20
BBAS3          28       12             25
BOVA11         73       25             71
ITUB4          34        5             34
MGLU3          24        2             24
PETR4          54       33             51
VALE3          26       10             25
WEGE3          19        4             17

datas sem pregão que a fonte preencheu: 9, de 15/06/2017 a 25/01/2018
            sem ajuste  ajustado      volume
Date
2011-05-10        24.6     5.275  18303600.0
2011-05-11        24.6     5.275         0.0
2011-05-12        24.6     5.321         0.0
2011-05-13        24.6     5.321         0.0
```

O defeito está em todos os papéis e tem duas origens. Em nove datas entre junho de 2017 e janeiro de 2018, dias em que a bolsa não abriu, a fonte criou uma linha para cada ação com o preço da véspera. Os demais pontos são pregões, concentrados em 2011, em que a fonte não registrou negócios e repetiu o fechamento. As 54 linhas de volume zero de PETR4 são todas desse tipo.

As últimas linhas da saída mostram por que a comparação usa o preço sem ajuste. Em maio de 2011 a cotação de PETR4 ficou parada em R$ 24,60 por três linhas sem negócio, mas o preço ajustado mudou no dia 12, data em que a ação passou a ser negociada sem direito a um provento: o ajuste do Capítulo 1 altera o histórico mesmo quando a cotação foi apenas copiada. A regra ingênua não vê esse ponto, nem os que vêm logo depois de uma linha ausente, e deixa escapar 13 dos 280 casos. Em todos eles o retorno é fabricado, e a variação que faltou aparece somada no dia em que o dado volta. O tratamento é o da Seção 2.4: esses pontos são dados que faltam e devem virar valor ausente.

2.5.2 Retornos que o mercado não acompanhou

A primeira pergunta diante de um retorno extremo é se o resto do mercado se moveu junto. O Código 2.7 lista, ainda no arquivo bruto, os dias em que uma ação se afastou mais de 15 pontos percentuais do Ibovespa.

Código 2.7 Retornos muito diferentes do mercado (`cap02/l05_erros.py`)

```
acoes = ["PETR4.SA", "VALE3.SA", "ITUB4.SA", "BBAS3.SA",
         "ABEV3.SA", "WEGE3.SA"]
p = precos[acoes + ["^BVSP"]].dropna()
r = p.pct_change().dropna()

# retorno da ação menos o do mercado no mesmo dia
excesso = r[acoes].sub(r["^BVSP"], axis=0)
longo = excesso.stack().rename("excesso").to_frame()
longo["retorno"] = r[acoes].stack()
longo["Ibovespa"] = r["^BVSP"].reindex(
    longo.index.get_level_values(0)).values
longo["dia seguinte"] = r[acoes].shift(-1).stack()

suspeitos = longo[longo["excesso"].abs() > 0.15]
print(suspeitos.map(lambda v: f"{v:+.1%}"))
```

```
                   excesso retorno Ibovespa dia seguinte
Date
2016-03-15 BBAS3.SA  -17.6%  -21.2%    -3.6%        +3.4%
2018-06-01 PETR4.SA  -15.5%  -14.9%    +0.6%        +8.5%
2019-01-28 VALE3.SA  -22.3%  -24.6%    -2.3%        +0.9%
2020-03-09 PETR4.SA  -17.5%  -29.7%   -12.2%        +9.4%
2021-02-22 PETR4.SA  -15.6%  -20.7%    -5.1%       +11.0%
2021-10-04 ITUB4.SA  -15.7%  -18.0%    -2.2%        +2.4%
```

O filtro devolve seis candidatos em dezesseis anos de seis ações, quantidade que se confere à mão. Quatro têm explicação conhecida e são verdadeiros: a renúncia do presidente da Petrobras em 1º de junho de 2018, o primeiro pregão depois do rompimento da barragem de Brumadinho em janeiro de 2019, a guerra de preços do petróleo em 9 de março de 2020 e a troca do comando da Petrobras anunciada antes do pregão de 22 de fevereiro de 2021. O de ITUB4 é um defeito. Em 1º de outubro de 2021 a XP incorporou a XPart, empresa que guardava a participação do Itaú na XP, e os acionistas do banco receberam recibos de ações (BDRs) da XP. No pregão seguinte, 4 de outubro, ITUB4 passou a ser negociada sem direito a esses papéis, com um ajuste de −17,54% no preço, e a série "ajustada" do Yahoo não compensou a diferença. Quem tinha a ação recebeu outro papel no lugar dos 18% que a série diz ter perdido. O caso de BBAS3 fica para o Exercício 2.4.

A Figura 2.4 resume os sinais que ajudam na decisão. Nenhum deles é prova. Um retorno extremo que reverte no dia seguinte é suspeito, mas março de 2020 teve vários pares assim, todos reais. Um degrau que o mercado não acompanha sugere evento societário, e a confirmação está nos comunicados da empresa. Na dúvida, o que decide é uma segunda fonte: os arquivos da B3 para preços de tela, o Banco Central para câmbio e juros.

2.5.3 Da cópia bruta aos arquivos limpos

Os dois defeitos confirmados são corrigidos por `scripts/limpar_dados.py`, que lê `dados/bruto/` e grava em `dados/` os arquivos usados no resto do livro. O Código 2.8 mostra a função com a regra dos preços repetidos e o bloco principal, que a aplica.

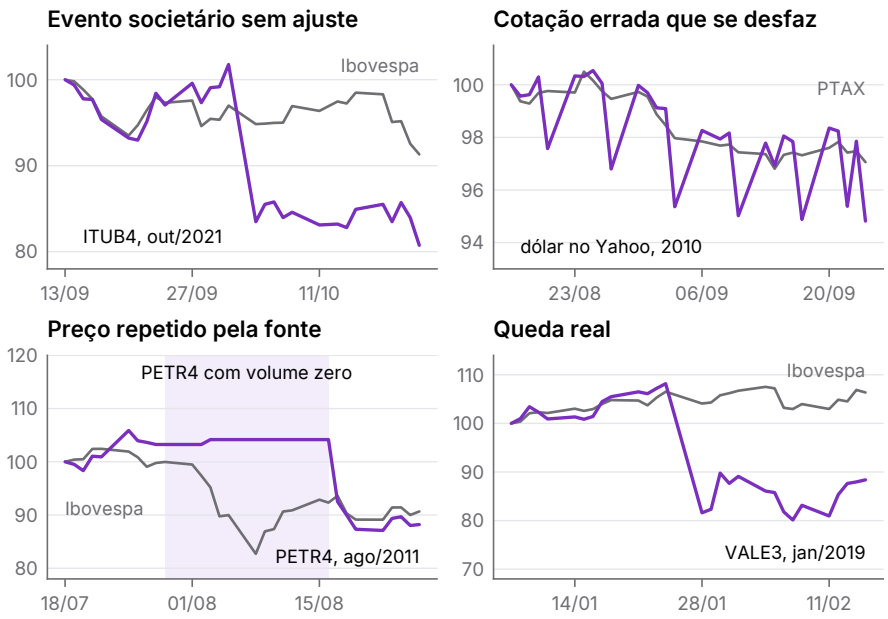


Figura 2.4 Quatro assinaturas no arquivo bruto, com a série examinada em roxo e a referência em cinza, ambas em base 100. Um evento societário sem ajuste desloca o nível em um dia sem queda comparável do mercado. Uma cotação errada volta no dia seguinte. Um preço repetido fica plano, com volume zero, enquanto o mercado se move. Uma queda real não se desfaz.

Código 2.8 As duas correções de `scripts/limpar_dados.py`

```
def sem_negocio(preco, volume):
    """Marca pontos com volume zero e preço repetido."""
    marca = pd.Series(False, index=preco.index)
    ultimo = float("nan")           # último fechamento válido
    for dia, p in preco.items():
        if volume.get(dia) == 0 and p == ultimo:
            marca[dia] = True
        elif pd.notna(p):
            ultimo = p
    return marca
if __name__ == "__main__":
    tela = ler("precos_sem_ajuste.csv")
    volume = ler("volume.csv")
    acoes = [c for c in tela.columns if c.endswith(".SA")]
    falso = pd.DataFrame(
        {a: sem_negocio(tela[a], volume[a]) for a in acoes})
    feriado = falso.all(axis=1)     # nenhum papel negociou

    for nome in ["precos_ajustados.csv", "precos_sem_ajuste.csv"]:
        p = ler(nome)
        p[acoes] = p[acoes].mask(falso)
        p.loc[feriado, "^BVSP"] = float("nan")
        antes = p.index < DATA_CISAO
        p.loc[antes, "ITUB4.SA"] *= FATOR_CISAO
        p.to_csv(DADOS / nome)
```

A função percorre a série guardando o último fechamento válido, de modo que uma sequência de repetições é marcada inteira, mesmo depois de uma lacuna. O bloco principal calcula as marcas uma vez, sobre o preço sem ajuste, e as aplica aos dois arquivos de preços com o método `mask`, que troca os pontos

marcados por valor ausente. Nas datas em que nenhum papel negociou, o Ibovespa também fica ausente. E os preços de ITUB4 anteriores a 4 de outubro de 2021 são multiplicados por $1 - 0{,}1754$, o ajuste do evento (as constantes `DATA_CISAO` e `FATOR_CISAO` ficam no início do arquivo), de modo que o degrau desaparece e os retornos dos outros dias não mudam. O arquivo de barras de PETR4 perde as mesmas linhas. O script `cap02/l06_limpeza.py` compara o antes e o depois:

```
           ausentes no bruto  no limpo
PETR4.SA                 196       250
ITUB4.SA                 196       230
BOVA11.SA                197       270
^BVSP                    205       206
ITUB4 em 04/10/2021, bruto: -18.0%
ITUB4 em 04/10/2021, limpo: -0.5%
barras de PETR4, bruto e limpo: [3974, 3920]
```

Limpar aumentou o número de valores ausentes, e é assim que deve ser: 54 preços de PETR4 que pareciam observações agora aparecem como o que são.

Tornar o ponto ausente tira o zero fabricado, mas não distribui a variação pelos dias em que ela ocorreu. O costume de escrever `dropna()` e depois `pct_change()` junta as pontas de cada lacuna, e o retorno seguinte cobre vários pregões embora entre na amostra como se fosse de um dia. A segunda parte do mesmo script conta esses casos.

Código 2.9 Retornos que cobrem mais de um pregão (`cap02/l06_limpeza.py`)

```python
# depois de dropna(), quantos retornos cobrem mais de um pregão?
acoes = [c for c in limpo.columns if c.endswith(".SA")]
pregoes = limpo.index[limpo[acoes].notna().any(axis=1)]
ordem = pd.Series(range(len(pregoes)), index=pregoes)
linhas = {}
for a in acoes:
    preco = limpo[a].dropna()
    passo = ordem[preco.index].diff()    # pregões entre preços
    linhas[a.replace(".SA", "")] = {
        "retornos longos": (passo > 1).sum(),
        "maior intervalo": int(passo.max())}
print("\n", pd.DataFrame(linhas).T, sep="")
```

```
        retornos longos  maior intervalo
ABEV3                13               2
BBAS3                17               4
BOVA11               63               3
ITUB4                19               5
MGLU3                15               2
PETR4                24              14
VALE3                15               4
WEGE3                10               2

PETR4 em 17/08/2011, depois de dropna(): -10.4%
com os ausentes no lugar: nan
```

O caso extremo é o que abriu o capítulo. Em 17 de agosto de 2011, o retorno de PETR4 calculado depois de `dropna()` é de $-10{,}4\%$ e cobre 14 pregões. Ele não é um erro, pois a ação de fato caiu nesse intervalo, mas não é um retorno

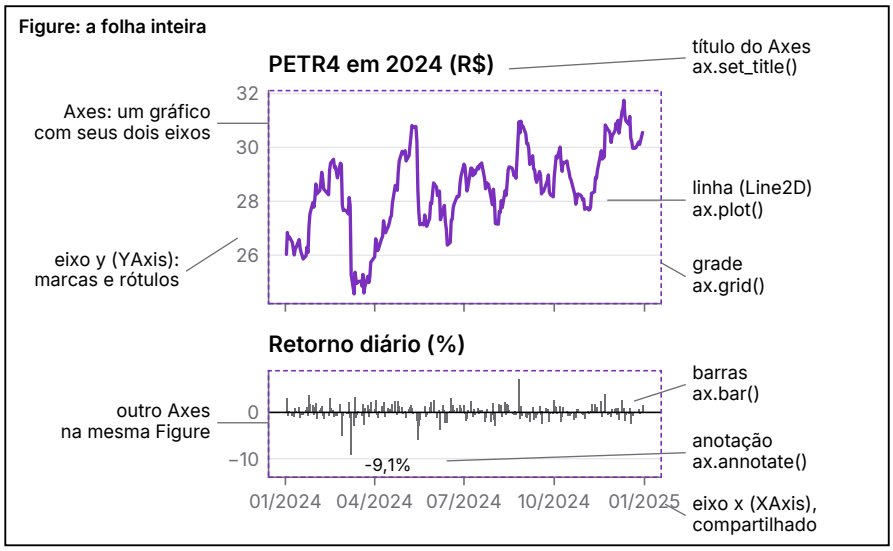


Figura 2.5 Os objetos de uma figura. A moldura preta delimita a `Figure`; cada retângulo tracejado é um `Axes`. Tudo o que se desenha é criado por um método do `Axes` a que pertence.

diário, e pesa em qualquer estatística de cauda como se fosse um dos piores dias do papel. Quando a análise depende de retornos de um pregão, calcule `pct_change(fill_method=None)` sobre a série com os ausentes no lugar: o retorno que se segue a uma lacuna fica ausente também, como na última linha da saída.

Ficam no conjunto, documentados, quatro problemas que o livro não corrige: a série `BRL=X`, tratada na Seção 2.8; o Ibovespa ausente em 2 de janeiro de 2012; o fato de os papéis terem sido escolhidos entre empresas que existem hoje, viés que o Capítulo 7 discute; e fechamentos que não coincidem com os oficiais. Em 8 de março de 2024, por exemplo, o arquivo traz R$ 36,70 para PETR4 (sem ajuste), queda de 9,14%, e o fechamento divulgado no dia foi de R$ 36,12, queda de 10,57%.

Armadilha **Limpar o que é verdadeiro**

Remover todo retorno acima de um limite, ou substituí-lo pelo próprio limite, apaga os dias que mais importam para medir risco. Dos seis candidatos do Código 2.7, só um era defeito comprovado. Corrija um ponto apenas quando souber o que aconteceu nele, e escreva a regra e o motivo em um script, como em `limpar_dados.py`. Nunca edite o arquivo bruto.

2.6 Anatomia de um gráfico no Matplotlib

O Matplotlib (Hunter, 2007) é a biblioteca de gráficos mais usada em Python, e todas as figuras de dados deste livro foram feitas com ela. Um gráfico do Matplotlib é uma árvore de objetos. A `Figure` é a folha inteira. Dentro dela ficam um ou mais `Axes`, cada um um gráfico completo com seus eixos, título, linhas e textos. A Figura 2.5 dá nome às partes.

O nome confunde: `Axes` é o gráfico inteiro, e não "os eixos". Os eixos propriamente ditos são os objetos `XAxis` e `YAxis` que cada `Axes` contém.

Há duas formas de usar a biblioteca. Na interface de estado, chama-se `plt.plot()`, `plt.title()` e assim por diante, e cada comando age sobre "o gráfico atual", que o Matplotlib escolhe por você. Na **interface orientada a objetos**, você cria a `Figure` e os `Axes` com `plt.subplots()` e chama os

métodos do objeto que quer alterar. Com dois painéis ou mais, só a segunda deixa claro em qual deles cada comando atua, e é a única usada daqui em diante. O Código 2.10 produz uma figura com a estrutura da Figura 2.5.

Código 2.10 Dois painéis com a interface orientada a objetos (`cap02/l07_matplotlib.py`)

```
petr = precos.loc["2024", "PETR4.SA"].dropna()
retorno = petr.pct_change().dropna()

# uma Figure com dois Axes empilhados, que dividem o eixo x
fig, (ax_preco, ax_ret) = plt.subplots(
    nrows=2, sharex=True, figsize=(7, 4.5),
    gridspec_kw={"height_ratios": [2, 1]})

ax_preco.plot(petr.index, petr, color="#7B2FBE")
ax_preco.set_title("PETR4 em 2024 (R$)", loc="left")
ax_preco.grid(axis="y")

ax_ret.bar(retorno.index, retorno * 100, color="#6E6E73")
ax_ret.axhline(0, color="black", linewidth=0.6)
ax_ret.set_title("Retorno diário (%)", loc="left")
pior = retorno.idxmin()                    # anota o pior dia
ax_ret.annotate(f"{retorno[pior]:.1%}",
                xy=(pior, retorno[pior] * 100),
                xytext=(8, 0), textcoords="offset points")

for ax in fig.axes:                        # menos tinta sem dado
    ax.spines[["top", "right"]].set_visible(False)
fig.savefig("petr4_2024.pdf")              # formato vetorial
print(type(fig).__name__, "com", len(fig.axes), "Axes;",
      f"pior dia: {pior:%d/%m/%Y}, {retorno[pior]:.2%}")
```

```
Figure com 2 Axes; pior dia: 08/03/2024, -9.14%
```

A correspondência com a interface de estado é quase sempre direta: `plt.title()` vira `ax.set_title()`, `plt.xlabel()` vira `ax.set_xlabel()`. O argumento `sharex=True` faz os dois painéis usarem a mesma escala de datas, e salvar em PDF mantém o gráfico vetorial, nítido em qualquer ampliação. O pior dia impresso, −9,14%, é o do arquivo; como visto na Seção 2.5, o fechamento oficial daquele pregão foi outro.

2.6.1 O que faz um bom gráfico

Tufte (2001) propõe um critério simples: a tinta de um gráfico deve ser gasta com os dados, e o que não informa deve sair. Quatro consequências práticas, que as figuras deste livro seguem:

- **Uma ideia por figura**, dita na legenda, com o rótulo de cada série junto da linha quando elas são poucas.
- **Escalas comparáveis.** Séries de ordens de grandeza diferentes vão para a mesma base (Seção 2.9) ou para painéis separados com o eixo horizontal compartilhado. Dois eixos verticais no mesmo painel permitem fabricar qualquer relação visual mudando as escalas. Para preços em períodos longos, a escala é a logarítmica (Capítulo 1).
- **Cor com significado.** Uma cor por série, a mesma em todas as figuras; tons de uma só cor para grandezas e duas cores opostas para valores com sinal.

- **Estilo definido uma vez.** Fontes, cores e espessuras ficam em um arquivo que todos os scripts importam, papel que `estilo.py` cumpre neste livro.

2.7 Plotly para explorar

Um gráfico estático responde à pergunta que o autor fez. Durante a exploração as perguntas ainda não existem, e ajuda poder aproximar um trecho, ler o valor exato sob o cursor e ligar ou desligar séries. O Plotly gera gráficos em HTML com esses recursos. O Código 2.11 monta barras OHLC de PETR4 no formato de *candlestick*, em que um corpo liga a abertura ao fechamento e dois traços alcançam a máxima e a mínima, com o volume embaixo.

Código 2.11 Gráfico interativo de barras e volume com Plotly (`cap02/l08_plotly.py`)

```
                    index_col=0, parse_dates=True).loc["2024"]

fig = go.Figure()
fig.add_trace(go.Candlestick(
    x=ohlcv.index, open=ohlcv["Open"], high=ohlcv["High"],
    low=ohlcv["Low"], close=ohlcv["Close"], name="PETR4"))
fig.add_trace(go.Bar(x=ohlcv.index, y=ohlcv["Volume"],
                     name="volume", yaxis="y2"))

# um eixo x para os dois painéis; cada eixo y ocupa uma faixa
fig.update_layout(
    yaxis=dict(domain=[0.30, 1.00]),         # preços, em cima
    yaxis2=dict(domain=[0.00, 0.24]),        # volume, embaixo
    xaxis=dict(anchor="y2", rangeslider_visible=False,
               rangebreaks=[dict(bounds=["sat", "mon"])]),
    hovermode="x unified", hoversubplots="axis",
    showlegend=False, title="PETR4 em 2024 (preços ajustados)")

fig.write_html("petr4_2024.html", include_plotlyjs="cdn")
print(len(fig.data), "séries;", len(ohlcv), "pregões")
```

```
2 séries; 251 pregões
```

O arquivo `petr4_2024.html` abre em qualquer navegador; em um notebook, `fig.show()` exibe o gráfico na própria célula. Como o livro é impresso, descrevemos o que a página oferece. As duas séries usam o mesmo eixo horizontal, e cada eixo vertical ocupa uma faixa da altura, definida em `domain`. Arrastar o mouse sobre um intervalo amplia aquele trecho nos dois painéis ao mesmo tempo. Passar o cursor sobre um pregão mostra, em um único rótulo, abertura, máxima, mínima, fechamento e volume daquele dia: `hovermode="x unified"` reúne as séries de um painel, e `hoversubplots="axis"` estende o rótulo aos painéis que dividem o eixo. A opção `rangebreaks` retira do eixo os fins de semana. Para um primeiro olhar, `plotly.express` desenha todas as colunas de um `DataFrame` em uma chamada, `px.line(precos)`, e um clique no nome de uma série a esconde.

A divisão de trabalho que usamos no Núcleo é Plotly para procurar e Matplotlib para mostrar: um defeito como o preço repetido de 2011 aparece em segundos quando se pode aproximar o trecho, e a figura do relatório precisa sair igual para todos os leitores.

No repositório

Na pasta `aula2/`: `yfinance.py` baixa preços e desenha o retorno acumulado; `interpolacao.py` compara `dropna`, média, `ffill` e `interpolate` em uma tabela de cadastro, onde todos são legítimos (os dois precisam da troca por `.ffill()` para rodar com o pandas do livro); `Matplotlib.py` e `Plotly.py` desenham preço, retorno e histograma de PETR4; `eda.py` traz o roteiro completo da aula, com um painel final de quatro gráficos, e `ex1_EDA.pdf`, o exercício.

2.8 O efeito do câmbio

O S&P 500 é cotado em dólares e o Ibovespa em reais. Um investidor brasileiro que compra o índice americano carrega duas posições ao mesmo tempo, uma em ações americanas e outra em dólar, e o retorno que ele recebe em reais combina as duas.

Resultado 2.2 Retorno em moeda local

Seja P_t o preço de um ativo em dólares e S_t o preço do dólar em reais. O valor do ativo em reais é $P_t S_t$, e seu retorno simples é

$$1 + R_t^{\text{R\$}} = \frac{P_t S_t}{P_{t-1} S_{t-1}} = \left(1 + R_t^{\text{US\$}}\right)\left(1 + R_t^{\text{câmbio}}\right). \tag{2.1}$$

Em log-retornos o produto vira soma: $r_t^{\text{R\$}} = r_t^{\text{US\$}} + r_t^{\text{câmbio}}$.

É a regra de composição do Capítulo 1, aplicada a dois preços em vez de dois períodos, e como lá os retornos simples não se somam. Se em um mês o S&P 500 sobe 10% e o dólar cai 5%, o investidor brasileiro ganha $1{,}10 \times 0{,}95 - 1 = 4{,}5\%$, e não 5%. A diferença é o termo cruzado $0{,}10 \times (-0{,}05) = -0{,}5\%$: a queda do dólar incide também sobre o ganho.

A conversão exige uma série de câmbio, e é aqui que aparece o defeito de data prometido na Seção 2.5. O Código 2.12 compara o `BRL=X` do arquivo bruto com a PTAX do Banco Central. A coluna `BRL=X seguinte` traz, em cada data, a cotação que o Yahoo rotula com a data seguinte.

Código 2.12 Conferência do câmbio do Yahoo contra a PTAX (`cap02/l09_cambio.py`)

```
yahoo = precos["BRL=X"].dropna()
d = pd.DataFrame({"Ibovespa": precos["^BVSP"],
                  "BRL=X": yahoo,
                  "BRL=X seguinte": yahoo.shift(-1),
                  "PTAX": bcb["ptax_venda"]}).dropna()
print(d.loc["2017-05-17":"2017-05-22"])

r = d.pct_change()
print("\ncorrelação com o retorno diário do Ibovespa")
for nome in ["BRL=X", "BRL=X seguinte", "PTAX"]:
    print(f"  {nome:<16}{r['Ibovespa'].corr(r[nome]):+.3f}")

# depois de corrigir a data, as duas fontes concordam?
desvio = d["BRL=X seguinte"] / d["PTAX"] - 1
print(f"\ndias com desvio acima de 3%: "
      f"{(desvio.abs() > 0.03).sum()} de {len(d)}")
vol_2010 = r.loc["2010"].std() * np.sqrt(252)
```

```
print(f"volatilidade em 2010:  BRL=X {vol_2010['BRL=X']:.1%}"
      f"   PTAX {vol_2010['PTAX']:.1%}")
```

```
            Ibovespa   BRL=X  BRL=X seguinte    PTAX
Date
2017-05-17   67540.0  3.0958          3.1334  3.1076
2017-05-18   61597.0  3.1334          3.3690  3.3807
2017-05-19   62639.0  3.3690          3.2533  3.2878
2017-05-22   61673.0  3.2533          3.2654  3.2863

correlação com o retorno diário do Ibovespa
  BRL=X           -0.020
  BRL=X seguinte  -0.409
  PTAX            -0.361

dias com desvio acima de 3%: 24 de 3959
volatilidade em 2010:  BRL=X 22.9%   PTAX 11.2%
```

Em 18 de maio de 2017, depois da divulgação de gravações que envolviam o presidente da República, o Ibovespa caiu 8,8% e a PTAX subiu 8,8%. Na série do Yahoo o salto do dólar só aparece com a data de 19 de maio. O padrão se repete na amostra inteira, de 2010 a 2025, nas datas em que as três séries têm valor: a correlação entre o Ibovespa e o `BRL=X` da mesma data é praticamente nula, e com o valor da data seguinte ela é de −0,41. O mercado de câmbio funciona 24 horas, cada fonte escolhe o instante que chama de fechamento, e nesta série o número rotulado com um dia corresponde ao que as bolsas viram no dia anterior. Mesmo com a data corrigida, as duas fontes divergem em mais de 3% em 24 dias, e em 2010 a série do Yahoo tem o dobro da volatilidade da PTAX por causa de cotações erradas como as do segundo painel da Figura 2.4.

A limpeza não tenta consertar essa série. O livro usa a PTAX como taxa de câmbio. Ela também tem uma limitação: é uma média de cotações apuradas pelo Banco Central ao longo do dia, e não um preço de fechamento. Por isso a correlação diária com o Ibovespa, de −0,36, ainda subestima a relação entre os dois.

> **Armadilha** **Correlação diária entre mercados desalinhados**
>
> Séries com data deslocada, ou fechadas em horários diferentes, têm a correlação diária reduzida, às vezes até zero. O teste é refazer a conta com retornos semanais, que quase não sentem um dia de desalinhamento. Se a correlação semanal for muito diferente da diária, desconfie das datas antes de concluir qualquer coisa sobre os ativos.

O Código 2.13 converte o S&P 500 para reais com a PTAX; as linhas omitidas calculam e imprimem os retornos totais, as volatilidades e as correlações.

Código 2.13 S&P 500 em reais (`cap02/l10_sp_reais.py`)

```
t = pd.DataFrame({"sp_usd": precos["^GSPC"],
                  "dolar": bcb["ptax_venda"],
                  "ibov": precos["^BVSP"]}).dropna()
t["sp_brl"] = t["sp_usd"] * t["dolar"]    # pontos x (R$ por US$)
```

```
04/01/2010 a 30/12/2025
```

```
S&P 500 em dólares:  +508.7%
dólar em reais:      +219.2%
S&P 500 em reais:   +1842.7%
(1+R_usd)(1+R_fx)-1 +1842.7%

volatilidade anualizada
sp_usd    17.6%
dolar     14.2%
ibov      23.3%
sp_brl    19.8%

correlação dos retornos diários
        sp_usd  dolar  ibov  sp_brl
sp_usd    1.00  -0.24  0.57    0.72
dolar    -0.24   1.00 -0.36    0.50
ibov      0.57  -0.36  1.00    0.25
sp_brl    0.72   0.50  0.25    1.00
```

O índice multiplicou-se por 6,1 em dólares e o dólar por 3,2 em reais; pela Equação (2.1), o investidor brasileiro multiplicou o capital por $6{,}087 \times 3{,}192 = 19{,}4$. Dois resultados menos óbvios estão nas tabelas de baixo. O S&P 500 em reais oscila 19,8% ao ano, mais do que em dólares, mas bem menos do que a soma das volatilidades do índice e do câmbio, porque os dois têm correlação negativa. Como os log-retornos se somam, a variância da soma dá a conta aproximada: $\sqrt{0{,}176^2 + 0{,}142^2 + 2 \times (-0{,}24) \times 0{,}176 \times 0{,}142} \approx 19{,}8\%$. E a correlação com o Ibovespa cai de 0,57 para 0,25 quando o índice americano é medido em reais. Nos dias ruins para a bolsa brasileira o dólar costuma subir, e essa alta compensa parte da queda das ações americanas.

2.9 Um roteiro de análise exploratória

A **análise exploratória de dados**, ou EDA (*exploratory data analysis*), é o exame dos dados antes de qualquer modelo. O nome foi consagrado por Tukey (1977), que defendia olhar para os dados com gráficos e resumos simples antes de testar hipóteses. Em finanças ela tem duas funções: encontrar defeitos, como os das seções anteriores, e formar uma primeira ideia de como os ativos se comportam e se relacionam. O Código 2.14 monta a base desta seção a partir dos arquivos limpos, com a PTAX como câmbio e apenas as datas com valor em todas as colunas. Quando uma data sai, o retorno seguinte cobre mais de um pregão em todas as colunas ao mesmo tempo, o que preserva o alinhamento entre os ativos.

Código 2.14 Base limpa e medidas exploratórias (`cap02/l11_eda.py`)

```
base = pd.DataFrame({
    "PETR4": precos["PETR4.SA"], "VALE3": precos["VALE3.SA"],
    "ITUB4": precos["ITUB4.SA"], "WEGE3": precos["WEGE3.SA"],
    "Ibovespa": precos["^BVSP"],
    "S&P 500 (US$)": precos["^GSPC"],
    "Dólar": bcb["ptax_venda"],
}).dropna()                          # só datas comuns a todos
base["S&P 500 (R$)"] = base["S&P 500 (US$)"] * base["Dólar"]

ret = base.pct_change().dropna()

base100 = 100 * base / base.iloc[0]
vol = ret.rolling(63).std() * np.sqrt(252)  # só olha para trás
queda = base / base.cummax() - 1            # drawdown
corr = ret.corr()
semanal = base.resample("W-FRI").last().pct_change()
corr_sem = semanal.corr()
```

```
3807 datas, de 04/01/2010 a 30/12/2025

               base 100 final vol mín vol máx drawdown máx data do fundo
Ibovespa                230.0   10.4%   80.3%       -48.6%    26/01/2016
S&P 500 (US$)           609.0    5.1%   62.6%       -33.9%    23/03/2020
S&P 500 (R$)           1943.0    9.3%   54.8%       -31.3%    11/10/2022
Dólar                   319.0    5.0%   29.7%       -27.3%    16/02/2017

correlação com o Ibovespa   diária  semanal
  PETR4                       0.74     0.72
  S&P 500 (US$)               0.57     0.54
  S&P 500 (R$)                0.24     0.08
  Dólar                      -0.37    -0.52
```

2.9.1 Preços na mesma régua

No fim de 2025 o Ibovespa está na casa dos 160 mil pontos, o S&P 500 perto de 7 mil e o dólar em R$ 5,50. Em um mesmo eixo, só o primeiro seria visível. Dividir cada série pelo seu primeiro valor e multiplicar por 100 coloca todas na mesma **base 100**: o valor 230 significa "2,3 vezes o nível inicial". A Figura 2.6 acrescenta o CDI acumulado, calculado com a série do Código 2.3.

Entre 2010 e 2025 o Ibovespa rendeu menos do que o CDI. Antes de concluir algo a partir disso, lembre que uma base 100 depende inteiramente da data inicial: a mesma figura iniciada em janeiro de 2016, data do fundo na tabela acima, contaria outra história. Há também uma diferença de construção entre os índices. O Ibovespa reinveste os proventos das ações que o compõem, e o `^GSPC` é um índice de preços, que os ignora. A comparação favorece o índice brasileiro e, ainda assim, ele fica atrás.

2.9.2 Volatilidade móvel e drawdown

A volatilidade de um ativo não é constante, como o Capítulo 1 adiantou. A forma mais simples de ver isso é calculá-la em uma **janela móvel**: em cada

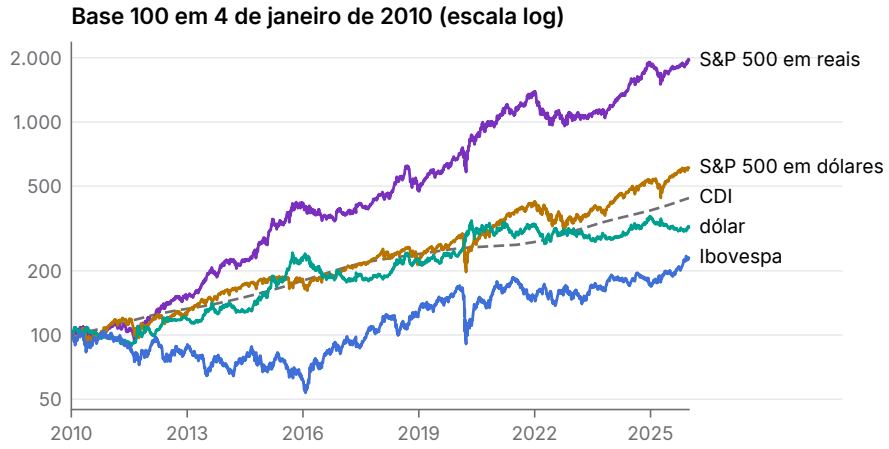


Figura 2.6 Índices, dólar e CDI em base 100. Em dezesseis anos o Ibovespa chegou a 230 e o CDI acumulado a 439. O S&P 500 foi a 609 em dólares e a 1943 em reais.

data t, o desvio-padrão dos últimos n retornos, anualizado,

$$\hat{\sigma}_t = \sqrt{252}\,\sqrt{\frac{1}{n-1}\sum_{i=0}^{n-1}\left(R_{t-i} - \bar{R}_t\right)^2}, \tag{2.2}$$

em que $\bar{R}_t$ é a média dos mesmos n retornos. O Código 2.14 faz essa conta com o método `rolling`, usando $n = 63$ pregões, cerca de três meses. A janela termina em t e só contém o passado, de modo que a medida pode ser usada em regras de decisão. As primeiras 63 datas da base ficam sem valor.

A segunda medida é o *drawdown*, a distância ao pico: quanto o preço de hoje está abaixo do maior preço atingido até hoje,

$$D_t = \frac{P_t}{\max_{s \le t} P_s} - 1. \tag{2.3}$$

O valor é zero quando o ativo está na máxima histórica e negativo no resto do tempo. O menor valor da série é o drawdown máximo do período. Aqui os dois servem como ferramenta de exploração; o Capítulo 5 retoma o drawdown como medida de risco.

Na Figura 2.7, a volatilidade do Ibovespa variou de 10% a 80% ao ano conforme a janela, e a do S&P 500, de 5% a 63%. Um único número de volatilidade para o período todo, como os da seção anterior, é uma média de regimes muito diferentes. O painel de baixo acrescenta a dimensão do tempo: a queda do S&P 500 em 2020 foi funda e curta, e a do Ibovespa entre 2010 e 2016 foi lenta e longa. A tabela do Código 2.14 traz ainda um detalhe que só aparece em reais. Para o investidor brasileiro, o drawdown máximo do S&P 500 foi o de 2022, quando a bolsa americana e o dólar recuaram juntos. Em março de 2020 o dólar subiu e amorteceu a perda.

2.9.3 Correlações

A correlação $\rho_{ij} = \mathrm{Cov}(R_i, R_j)/(\sigma_i \sigma_j)$ entre os retornos de dois ativos mede o quanto eles se movem juntos, em uma escala de -1 a $+1$. O método `corr` calcula todos os pares de uma vez e usa, em cada par, as datas em que ambos têm valor. A Figura 2.8 mostra a matriz.

Três blocos se destacam. As ações brasileiras têm correlações entre 0,20 e 0,54 umas com as outras e mais altas com o Ibovespa, do qual fazem parte. O dólar se move contra todas elas. E o S&P 500 em reais tem correlações baixas

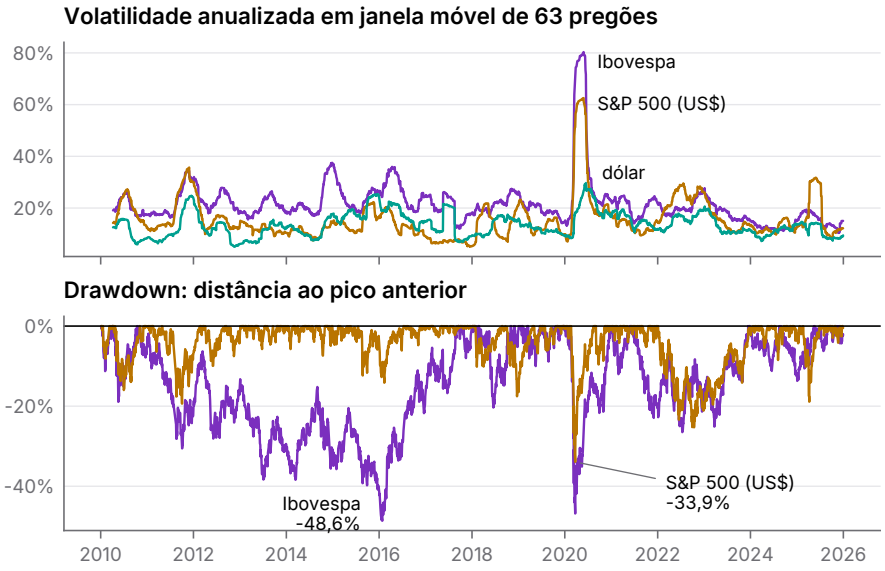


Figura 2.7 Em cima, a volatilidade móvel do Ibovespa (roxo), do S&P 500 em dólares (ocre) e do dólar (verde): longos períodos calmos interrompidos por saltos, o maior deles em 2020. Embaixo, o drawdown: o Ibovespa passou a maior parte da década de 2010 abaixo da máxima anterior.

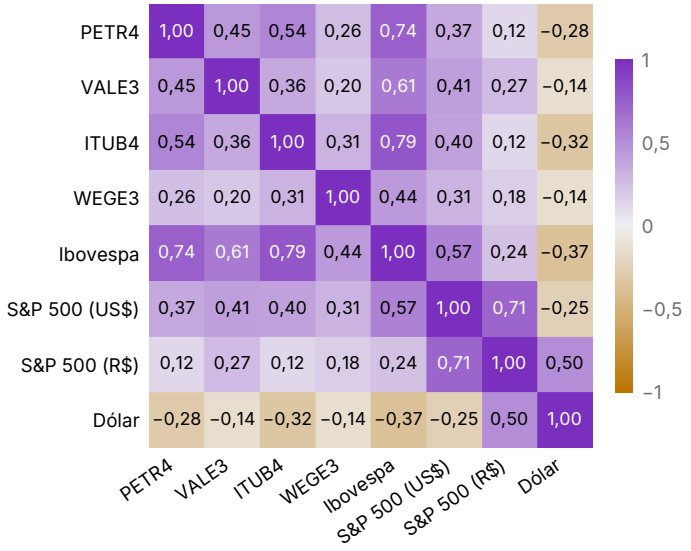


Figura 2.8 Correlação dos retornos diários, de 2010 a 2025. As ações brasileiras têm correlação positiva entre si e com o Ibovespa. O dólar tem correlação negativa com todas. Medido em reais, o S&P 500 quase se descola do mercado brasileiro.

com os ativos brasileiros, entre 0,12 e 0,27, a matéria-prima da diversificação que o Capítulo 6 vai quantificar. Uma matriz como essa resume dezesseis anos em um número por par; a correlação também muda no tempo, e a mesma janela móvel serve para acompanhá-la (Exercício 2.5).

As últimas linhas da saída do Código 2.14 repetem o teste da armadilha anterior. Em retornos semanais, a correlação do dólar com o Ibovespa passa de −0,37 para −0,52, e a do S&P 500 em reais cai de 0,24 para 0,08. (O valor diário era 0,25 no Código 2.13, cuja amostra não exige preço das quatro ações.) Os valores diários que envolvem a PTAX carregam o desalinhamento de horário, e os semanais são a melhor estimativa.

2.9.4 Checklist de EDA

Antes de usar um conjunto de preços em qualquer análise, percorra esta lista.

1. **Origem.** De que fonte veio cada série, em que data foi baixada, e a cópia bruta está guardada junto com o código?
2. **Significado.** O preço é de tela ou ajustado? Em que moeda? O índice inclui proventos? O que a fonte chama de fechamento?
3. **Cobertura.** Qual é a primeira e a última data válida de cada coluna? Quantos valores faltam, e por qual das três causas?
4. **Coerência.** Máxima e mínima contêm abertura e fechamento? Há volume zero ou preços repetidos em ativos líquidos?
5. **Extremos.** Os maiores retornos, em módulo, têm explicação? O mercado se moveu junto? Há degraus sem notícia?
6. **Alinhamento.** As datas das séries se referem ao mesmo instante? A correlação semanal confirma a diária?
7. **Tratamento.** Cada correção está escrita em um script que parte do arquivo bruto? Cada valor ausente foi removido ou preenchido por uma razão declarada, sem `bfill` nem interpolação?
8. **Retrato.** Base 100 em escala logarítmica, volatilidade móvel, drawdown e matriz de correlação foram vistos antes de qualquer modelo?

O que levar deste capítulo

- Nenhuma fonte gratuita é isenta de erros: o conjunto deste capítulo tinha preços repetidos, linhas em dias sem pregão, uma cisão sem ajuste e um câmbio com a data deslocada.
- Baixe uma vez, guarde a cópia bruta sem alterá-la e faça a limpeza em um script.
- Descubra por que um valor falta antes de tratá-lo. Use `dropna` para estatísticas de retornos e `ffill` para o último valor conhecido; `bfill` e interpolação usam o futuro. Depois de uma lacuna, o retorno seguinte cobre mais de um pregão.
- Um retorno extremo é verdadeiro até prova em contrário. Compare com o mercado, olhe o volume e o dia seguinte, e confirme em uma segunda fonte antes de corrigir.
- No Matplotlib, crie `Figure` e `Axes` explicitamente e desenhe pelos métodos do `Axes`.
- Para comparar ativos em moedas diferentes, converta para a moeda do investidor. O câmbio muda o retorno, a volatilidade e as correlações.

Exercícios

2.1 [básico] Uma ação fecha a R$ 50,00 na segunda-feira, não tem cotação na terça por falha da fonte e fecha a R$ 53,00 na quarta. Calcule os retornos diários de terça e quarta com `dropna`, `ffill` e interpolação linear. Em qual dos três o valor de terça usa informação de quarta?

2.2 [básico] Um fundo americano sobe 8% em dólares em um ano em que o dólar cai 12% em relação ao real. Qual foi o retorno em reais? De quanto o dólar precisaria ter caído para o retorno em reais ser zero?

2.3 [intermediário] Reescreva `aula2/Matplotlib.py` com a interface orientada a objetos, em uma única `Figure` com três `Axes` (preço, retorno diário e histograma), lendo os dados de `livro/dados/` em vez de baixá-los.

2.4 [intermediário] O Código 2.7 aponta um retorno de $-21{,}2\%$ em BBAS3 no dia 15 de março de 2016. Use o comportamento de outras ações na mesma data, os dias seguintes e uma segunda fonte para decidir se ele é verdadeiro.

2.5 [intermediário] Com a base do Código 2.14, calcule a correlação entre os retornos do Ibovespa e do S&P 500 em dólares em janela móvel de 252 pregões, usando `rolling(252).corr()`, e desenhe o resultado. Entre que valores ela variou?

2.6 [desafio] Refaça a matriz da Figura 2.8 com retornos semanais e com retornos mensais. Quais pares mudam mais? Separe, na explicação, o que pode ser efeito de desalinhamento de horário do que pode ser comportamento dos ativos.

2.7 [desafio] Usando a PTAX como referência, escreva uma função que localize na série `BRL=X` as cotações erradas isoladas (um desvio grande que desaparece no dia seguinte) e as substitua por `NaN`. Quantos pontos ela marca? Depois da limpeza, a volatilidade de 2010 se aproxima da volatilidade da PTAX?

Para ir além

- McKinney (2022), capítulos 7, 9 e 11, para tratamento de dados ausentes, gráficos e séries temporais no pandas.
- Hilpisch (2018), capítulos 7 a 9, para visualização com Matplotlib e Plotly, séries financeiras e leitura e gravação de arquivos.
- Chan (2021), capítulo 3, para os defeitos de dados que mais distorcem um teste de estratégia.
- Tufte (2001), para os princípios de desenho de gráficos estatísticos, e Tukey (1977), o livro que consagrou o nome e o método da análise exploratória.

Capítulo 3

Estatística aplicada a retornos

3

Dez anos de retornos diários de uma ação são cerca de 2 500 números. Ninguém decide nada olhando para 2 500 números; decide-se olhando para meia dúzia de resumos deles e para o resultado de alguns testes. Este capítulo trata de quais resumos calcular, de qual distribuição esperar e de quanto se pode confiar em cada conclusão, porque em finanças a amostra quase sempre é menor do que parece.

Ao terminar este capítulo você consegue

- resumir uma série de retornos com medidas de posição, dispersão, assimetria e curtose, sabendo o que cada biblioteca devolve;
- explicar por que a distribuição normal subestima a frequência de retornos extremos e usar a *t* de Student como alternativa;
- ler um histograma em escala logarítmica e um QQ-plot;
- interpretar corretamente um p-valor e reconhecer quando um teste tem pouco poder;
- aplicar os testes *t*, de Jarque-Bera e de Ljung-Box a retornos;
- calcular covariâncias e correlações, e desconfiar da estabilidade delas.

3.1 Resumir uma série de retornos

O Código 3.1 calcula, para PETR4, VALE3, ITUB4, WEGE3 e BOVA11 entre 2015 e 2024, os resumos que você vai pedir primeiro a qualquer série nova. Os log-retornos estão multiplicados por 100, isto é, em porcentagem ao dia.

Código 3.1 Medidas de posição e de dispersão (`cap03/l01_descritiva.py`)

```
import numpy as np
import pandas as pd

precos = pd.read_csv("../../dados/precos_ajustados.csv",
                     index_col=0, parse_dates=True)
```

```
ativos = ["PETR4.SA", "VALE3.SA", "ITUB4.SA", "WEGE3.SA",
          "BOVA11.SA"]
p = precos.loc["2015":"2024", ativos].dropna()
r = np.log(p / p.shift(1)).dropna() * 100    # em % ao dia

q = r.quantile([0.05, 0.25, 0.75, 0.95]).T   # uma linha por ativo
iqr = q[0.75] - q[0.25]
resumo = pd.DataFrame({
    "média": r.mean(), "mediana": r.median(),
    "desvio": r.std(), "IQR": iqr, "IQR/1,349": iqr / 1.349,
    "q5%": q[0.05], "q95%": q[0.95],
    "mín": r.min(), "máx": r.max(),
})
resumo.index = resumo.index.str.replace(".SA", "")
print(f"{len(r)} retornos diários por ativo, em %")
print(resumo.round(2).to_string())
```

```
2461 retornos diários por ativo, em %
        média  mediana  desvio   IQR  IQR/1,349    q5%  q95%    mín    máx
PETR4    0.10     0.13    3.03  2.86       2.12 -4.40  4.50 -35.24  20.07
VALE3    0.06     0.00    2.77  2.78       2.06 -4.03  4.45 -28.18  19.36
ITUB4    0.05     0.03    1.92  2.14       1.59 -2.91  3.10 -12.84  10.49
WEGE3    0.10     0.03    2.12  2.24       1.66 -3.11  3.42 -23.09  13.01
BOVA11   0.04     0.08    1.55  1.62       1.20 -2.28  2.25 -15.75  12.57
```

As duas primeiras colunas são **medidas de posição**, que localizam o centro da distribuição. A **média** amostral $\bar{r} = \frac{1}{n}\sum_t r_t$ usa todos os valores e por isso é sensível a dias extremos; a **mediana**, o valor que deixa metade das observações de cada lado, não é. As duas são minúsculas, de no máximo 0,13% ao dia, e a mediana de VALE3 é exatamente zero porque em 15 pregões o fechamento da ação não mudou, e esses zeros caem no meio da distribuição. O `dropna()` descarta as 16 datas em que falta o preço de algum dos cinco papéis, e o retorno seguinte a cada uma cobre mais de um pregão; nenhum mínimo ou máximo da tabela vem de um desses casos.

As colunas seguintes medem **dispersão**. O **desvio-padrão** amostral,

$$s = \sqrt{\frac{1}{n-1}\sum_{t=1}^{n}(r_t - \bar{r})^2}, \tag{3.1}$$

é a volatilidade diária do Capítulo 1. Compare-o com a média: em PETR4, 0,10% contra 3,03%. O retorno esperado de um dia é cerca de trinta vezes menor do que a oscilação típica desse dia, e essa desproporção é a origem de quase toda a dificuldade estatística em finanças. Ela volta na Seção 3.6.

Um **quantil** de ordem p é o valor abaixo do qual fica uma fração p dos dados. A coluna `q5%` diz que, em um a cada vinte pregões, o log-retorno de PETR4 ficou abaixo de $-4{,}40\%$. Esse número, com o sinal trocado, é a ideia por trás do *Value at Risk*, que o Capítulo 5 desenvolve. A distância entre os quantis de 75% e de 25% é o **intervalo interquartil** (IQR), uma medida de dispersão que ignora o que acontece nas caudas.

Em uma distribuição normal, o IQR vale 1,349 desvio-padrão. Dividir o IQR por 1,349 dá, portanto, uma segunda estimativa de σ, que deveria coincidir com s se os dados fossem normais. Não coincide em nenhuma linha: para PETR4 ela é 2,12%, contra $s = 3{,}03\%$. O miolo da distribuição é mais estreito do que o desvio-padrão sugere, e o desvio-padrão é grande porque uns poucos dias

muito ruins (o mínimo de $-35{,}24\%$ é o 9 de março de 2020 da Figura 1.1, uma queda de 29,7% em retorno simples) pesam muito em uma soma de quadrados.

3.2 Assimetria e curtose

Média e desvio-padrão descrevem onde a distribuição está e quão larga ela é. Para descrever a forma, usam-se os dois momentos seguintes da variável padronizada $(r-\mu)/\sigma$.

Definição 3.1 Assimetria e curtose

Para uma variável aleatória X com média μ e desvio-padrão σ,

$$S = \mathbb{E}\left[\left(\frac{X-\mu}{\sigma}\right)^3\right], \qquad K = \mathbb{E}\left[\left(\frac{X-\mu}{\sigma}\right)^4\right]. \tag{3.2}$$

S é a **assimetria** e K é a **curtose**. Toda distribuição normal tem $S=0$ e $K=3$. A diferença $K-3$ é o **excesso de curtose**.

O cubo preserva o sinal do desvio, de modo que $S<0$ indica que os desvios grandes ocorrem mais para baixo do que para cima: a cauda esquerda é mais longa. A quarta potência elimina o sinal e amplifica ainda mais os desvios grandes. Um dia a 5 desvios-padrão da média contribui com $5^4 = 625$ para a soma, o mesmo que 625 dias a 1 desvio. A curtose mede, por isso, o peso das caudas, e uma distribuição com $K>3$ é dita **leptocúrtica** ou de **caudas pesadas**.

O Código 3.2 calcula os dois momentos para o Ibovespa. Daqui em diante as listagens omitem as importações e a leitura do arquivo, iguais às do Código 3.1; os arquivos completos estão em `livro/scripts/cap03/`.

Código 3.2 Assimetria e curtose do Ibovespa, de 2010 a 2025 (`cap03/l02_momentos.py`)

```
ibov = precos["^BVSP"].dropna()
r = np.log(ibov / ibov.shift(1)).dropna()    # 2010 a 2025

print("n =", len(r))
print(f"assimetria  pandas {r.skew():.4f}"
      f"   scipy {stats.skew(r):.4f}")
k_pearson = stats.kurtosis(r, fisher=False)
print(f"curtose     pandas {r.kurt():.4f}"
      f"   scipy {stats.kurtosis(r):.4f}")
print(f"            scipy com fisher=False {k_pearson:.4f}")

# a mesma série em semanas e meses (log-retornos somam no tempo)
for nome, regra in [("semanal", "W"), ("mensal", "ME")]:
    ag = r.resample(regra).sum()
    print(f"{nome:8s} n = {len(ag):4d}"
          f"   assimetria {ag.skew():5.2f}"
          f"   excesso de curtose {ag.kurt():5.2f}")
```

```
n = 3963
assimetria  pandas -0.7802   scipy -0.7799
curtose     pandas 12.0940   scipy 12.0772
            scipy com fisher=False 15.0772
semanal  n =  835   assimetria -0.51   excesso de curtose  5.22
mensal   n =  192   assimetria -0.97   excesso de curtose  5.32
```

Armadilha **Curtose ou excesso de curtose?**

Tanto `Series.kurt()` do pandas quanto `scipy.stats.kurtosis` devolvem, por padrão, o *excesso* de curtose (a normal dá zero). Para obter K é preciso pedir `fisher=False` no SciPy ou somar 3. As duas bibliotecas também diferem ligeiramente porque o pandas aplica uma correção de viés para amostras finitas e o SciPy, por padrão, não. Ao ler "curtose de 12" em um relatório, descubra qual das duas definições foi usada.

O Ibovespa diário tem assimetria de $-0{,}78$ e excesso de curtose de $12{,}1$. As duas últimas linhas da saída repetem a conta com retornos semanais e mensais, obtidos somando os log-retornos diários. O excesso de curtose cai para perto de 5, mas não desaparece.

Antes de tirar conclusões desses números, veja o quanto eles dependem de poucos dias. Sem os 248 pregões de 2020, o excesso de curtose diário cai de $12{,}1$ para $2{,}0$ e a assimetria, de $-0{,}78$ para $-0{,}18$. Na série mensal, basta retirar um único mês, março de 2020, com log-retorno de $-35{,}5\%$, para o excesso de curtose ir de $5{,}3$ a $-0{,}2$. Estimativas de momentos de ordem alta são dominadas pelas maiores observações da amostra, e seu erro de estimação é grande. Isso não autoriza apagar a crise dos dados, já que ela é exatamente o tipo de evento que a gestão de risco existe para enfrentar. Autoriza desconfiar de qualquer comparação fina de curtoses entre ativos ou entre períodos.

Esses números, e os demais "sem 2020" do capítulo, saem de `cap03/c03_sensibilidade.py`.

3.3 A normal como ponto de partida, e onde ela falha

A distribuição normal $N(\mu, \sigma^2)$ tem densidade

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right] \tag{3.3}$$

e é o modelo de referência por boas razões. Dois parâmetros a descrevem por completo; somas de normais independentes são normais, o que combina com a aditividade dos log-retornos; e o teorema central do limite diz que somas de muitas parcelas independentes de variância finita tendem a ela. Se o retorno de um dia é a soma de milhares de pequenas variações, a normal parece uma hipótese natural.

O problema está no expoente. A densidade cai com o *quadrado* da distância à média, o que torna eventos distantes praticamente impossíveis: $95{,}45\%$ da probabilidade fica a menos de 2 desvios, $99{,}73\%$ a menos de 3, e um desvio de mais de 5 tem probabilidade de $5{,}7 \times 10^{-7}$. A Figura 3.1 confronta essa previsão com os 3 963 retornos diários do Ibovespa.

O histograma em escala linear esconde justamente a região que importa para o risco, porque um dia isolado em três mil não tem altura visível. Trocar o eixo vertical para escala logarítmica é o primeiro diagnóstico de cauda: o logaritmo da densidade normal é uma parábola, e tudo o que aparece fora dela

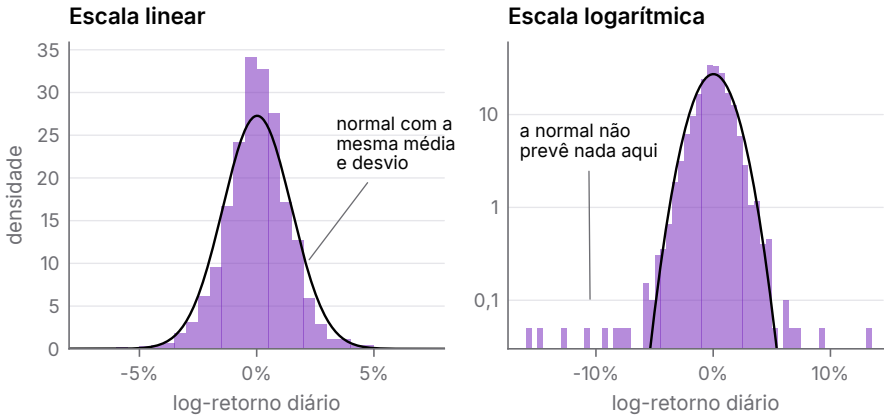


Figura 3.1 Log-retornos diários do Ibovespa, de 2010 a 2025, e a normal com mesma média e mesmo desvio-padrão. Em escala linear o ajuste parece razoável, com um pico mais alto e mais estreito. Em escala logarítmica a densidade normal vira uma parábola, e ficam visíveis os dias além de $\pm 6\%$, para os quais ela atribui probabilidade praticamente nula.

é excesso em relação à normal. O Código 3.3, na próxima seção, conta esses dias. Além de 4 desvios-padrão a normal previa 0,25 ocorrência em toda a amostra; houve 15.

Esse comportamento não é particular do Ibovespa nem de 2020. Mandelbrot (1963) já o documentava em preços de algodão, e Cont (2001) reuniu sob o nome de **fatos estilizados** as regularidades que aparecem em praticamente todos os ativos e mercados. Quatro delas organizam o resto deste capítulo e o próximo.

- **Caudas pesadas.** A distribuição incondicional dos retornos tem mais massa nos extremos do que a normal. Cont relata índices de cauda entre 2 e 5 na maioria das séries que estudou, o que é compatível com variância finita e deixa em dúvida a existência do quarto momento.
- **Ausência de autocorrelação linear.** O retorno de hoje praticamente não ajuda a prever, por uma regra linear, o de amanhã, exceto em escalas intradiárias muito curtas.
- **Agrupamento de volatilidade.** Dias de grande oscilação, para cima ou para baixo, tendem a vir juntos. O *tamanho* dos retornos é autocorrelacionado mesmo quando os retornos não são.
- **Normalidade agregacional.** Quanto maior o intervalo sobre o qual se calcula o retorno (semana, mês, ano), mais a distribuição se aproxima da normal.

A Figura 3.2 mostra o segundo e o terceiro fatos. A **autocorrelação** de ordem k é a correlação da série com ela mesma defasada de k dias. Para os retornos, ela nunca passa de 0,08 em módulo, embora saia em algumas defasagens da faixa em que estaria por puro acaso (a Seção 3.7 volta a esse ponto); para os retornos ao quadrado, que medem o tamanho do movimento sem o sinal, ela começa em 0,5 e leva semanas para se dissipar.

O agrupamento explica por que a normalidade agregacional é lenta nos nossos dados. A versão mais simples do teorema central do limite pede parcelas independentes. Com a dependência que os retornos têm (não correlacionados, mas com volatilidade agrupada e variância finita) ele continua valendo, só que a convergência é mais lenta: dos nove dias da amostra com mais de 5 desvios-padrão, sete ocorreram no mesmo mês, março de 2020. Dias extremos que chegam em bloco não se compensam dentro do mês. O Capítulo 4 modela essa dependência com a família GARCH.

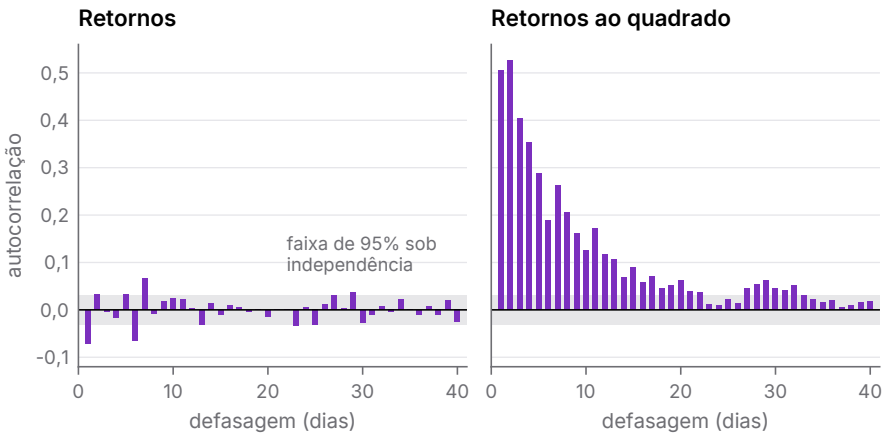


Figura 3.2 Autocorrelação dos log-retornos diários do Ibovespa e dos seus quadrados, de 2010 a 2025. O sinal do retorno de amanhã é quase imprevisível a partir do passado; o tamanho, não. A faixa cinza é $\pm 1{,}96/\sqrt{n}$.

3.4 A distribuição t de Student

Uma distribuição com caudas pesadas, um único parâmetro a mais do que a normal e disponível em qualquer biblioteca é a t de Student.

Definição 3.2 Distribuição t de Student

Uma variável T tem distribuição t com $\nu > 0$ **graus de liberdade** se sua densidade é

$$f(t) = c_\nu \left(1 + \frac{t^2}{\nu}\right)^{-\frac{\nu+1}{2}}, \qquad c_\nu = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\,\Gamma\left(\frac{\nu}{2}\right)}. \tag{3.4}$$

A média é zero para $\nu > 1$. A variância é $\nu/(\nu-2)$ e só existe para $\nu > 2$. O excesso de curtose é $6/(\nu-4)$ e só existe para $\nu > 4$.

A diferença em relação à Equação (3.3) é a forma da queda. Para $|t|$ grande, a densidade da t se comporta como $|t|^{-(\nu+1)}$, uma potência, enquanto a da normal cai como a exponencial de um quadrado. Uma potência decresce muito mais devagar, e quanto menor ν, mais devagar. É por isso que os momentos deixam de existir: a integral que define o momento de ordem k só converge se $k < \nu$. No outro extremo, quando $\nu \to \infty$ a t converge para a normal padrão, e com $\nu = 30$ as duas já são difíceis de distinguir a olho.

Para comparar caudas de forma justa é preciso igualar as variâncias. Como $\mathrm{Var}(T) = \nu/(\nu-2)$, a variável $X = T\sqrt{(\nu-2)/\nu}$ tem variância 1; é a t **padronizada**. A Figura 3.3 e a Tabela 3.1 comparam a normal padrão com a t padronizada.

Em 2 desvios-padrão as quatro distribuições dão probabilidades parecidas, e a t com $\nu = 3$ dá até um pouco menos do que a normal. A diferença aparece depois. Sob a normal, uma queda de 5 desvios acontece uma vez a cada 3,5 milhões de pregões, quase 14 mil anos de bolsa. Sob uma t com $\nu = 5$, uma vez a cada 1 506 pregões, ou seis anos. Quem dimensiona risco com a normal trata como impossível algo que um investidor vê algumas vezes na vida.

A tabela e os números deste parágrafo saem de `cap03/c03_tabela_caudas.py`.

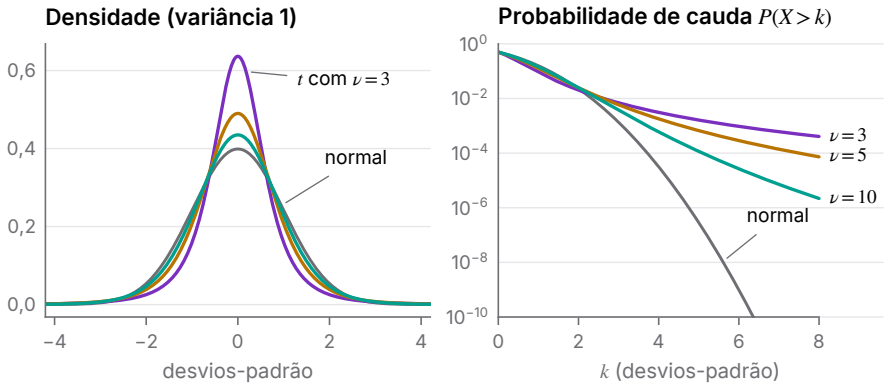


Figura 3.3 Normal padrão e t de Student padronizada para variância 1. À esquerda, a t concentra mais massa no centro e nas caudas, e menos nos "ombros". À direita, em escala logarítmica, a probabilidade de um valor acima de k desvios-padrão: as curvas se cruzam perto de $k = 2$ e depois se separam por ordens de grandeza.

Tabela 3.1 Uma queda de mais de k desvios-padrão ocorre, em média, uma vez a cada quantos pregões? Valores de $1/P(X < -k)$ para distribuições com variância 1.

k	Normal	$t, \nu = 10$	$t, \nu = 5$	$t, \nu = 3$
2	44	41	41	49
3	741	273	171	144
4	31 574	1 676	560	324
5	3 488 556	8 665	1 506	617
6	1 013 594 692	37 663	3 489	1 052

3.4.1 Ajuste aos dados

Retornos não têm média zero nem variância $\nu/(\nu-2)$, de modo que na prática se usa a família de **locação e escala** $r = \mu + \lambda\, T$. São três parâmetros: ν, a locação μ e a escala λ. O Código 3.3 os estima por máxima verossimilhança e compara, para cada k, o número de dias a mais de k desvios-padrão da média com o que cada distribuição previa.

Código 3.3 Ajuste da t de Student e contagem de dias extremos (`cap03/l03_caudas.py`)

```
ibov = precos["^BVSP"].dropna()
r = np.log(ibov / ibov.shift(1)).dropna()
n, mu, sigma = len(r), r.mean(), r.std()

nu, loc, escala = stats.t.fit(r)       # máxima verossimilhança
t_aj = stats.t(nu, loc, escala)
print(f"amostra:    n = {n}  média = {mu:.5f}"
      f"  desvio = {sigma:.5f}")
print(f"t ajustada: nu = {nu:.2f}  loc = {loc:.5f}"
      f"  escala = {escala:.5f}  desvio = {t_aj.std():.5f}")

print(" k   observado   normal   t ajustada")
for k in [2, 3, 4, 5]:
    obs = (np.abs(r - mu) > k * sigma).sum()
    esp_n = n * 2 * stats.norm.sf(k)
    esp_t = n * (t_aj.cdf(mu - k * sigma)
                 + t_aj.sf(mu + k * sigma))
    print(f"{k:2d}   {obs:9d}   {esp_n:6.2f}   {esp_t:10.2f}")
```

```
amostra:    n = 3963  média = 0.00021  desvio = 0.01463
t ajustada: nu = 4.86  loc = 0.00040  escala = 0.01089  desvio = 0.01420
 k   observado   normal   t ajustada
 2         157   180.32       177.68
 3          39    10.70        42.21
 4          15     0.25        13.03
 5           9     0.00         4.93
```

O ajuste devolve $\hat{\nu} = 4{,}86$, dentro da faixa relatada por Cont (2001). Repare que a escala, 0,01089, não é o desvio-padrão: pela fórmula da variância, o desvio-padrão da t ajustada é $\lambda\sqrt{\nu/(\nu-2)} = 0{,}0142$, próximo do amostral. Confundir os dois parâmetros ao simular ou ao calcular quantis subestima a volatilidade em cerca de um quarto.

A t acerta a contagem de dias além de 3 e de 4 desvios, que a normal erra por fatores de 3,6 e de 60. Além de 5 desvios ela ainda prevê menos (cerca de 5) do que se observou (9). Parte dessa diferença vem do agrupamento: uma distribuição única para todos os dias, por mais pesada que seja a cauda, não descreve o fato de que sete daqueles nove dias foram vizinhos.

3.4.2 QQ-plot

O **QQ-plot** (gráfico quantil-quantil) compara uma amostra com uma distribuição candidata ponto a ponto. Ordenam-se as n observações; a i-ésima menor é o quantil empírico de ordem aproximadamente $(i-0{,}5)/n$; no eixo horizontal vai o quantil de mesma ordem da distribuição teórica. Se a candidata descreve bem os dados, os pontos caem sobre a reta de 45°. Caudas pesadas produzem um S: os pontos ficam abaixo da reta à esquerda e acima à direita, porque os extremos observados são maiores do que os teóricos.

A Figura 3.4 resume a seção. O QQ-plot tem uma vantagem sobre o histograma: não depende da escolha de classes, e cada observação extrema aparece individualmente.

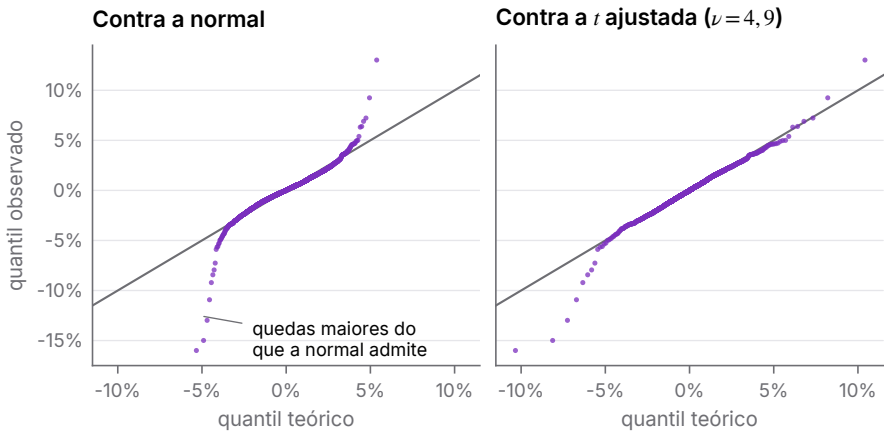


Figura 3.4 QQ-plots dos log-retornos diários do Ibovespa. Contra a normal, os pontos formam o S típico de caudas pesadas, e a pior queda ($-16{,}0\%$) aparece onde a normal esperava $-5{,}3\%$. Contra a t ajustada, o alinhamento vai até perto de $\pm 6\%$; os dias de março de 2020 continuam fora.

No repositório

`aula3/s3_01_estatistica_descritiva.py` monta a tabela descritiva para PETR4, VALE3 e ITUB4, e `aula3/s3_02_distribuicoes.py` ajusta a normal e a t ao Ibovespa e compara os quantis de cada uma. Os dois baixam os dados do Yahoo Finance na hora, e por isso os números variam um pouco em relação aos desta página.

3.5 A lógica de um teste de hipótese

Um teste de hipótese transforma a impressão deixada por um gráfico em uma probabilidade. O raciocínio é o mesmo em todos os testes deste capítulo. Formula-se uma **hipótese nula** H_0, uma afirmação precisa sobre o processo que gerou os dados ("o retorno esperado é zero", "a distribuição é normal"), e uma alternativa H_1. Escolhe-se uma **estatística de teste**, um número calculado da amostra cuja distribuição é conhecida *se H_0 for verdadeira*. Calcula-se a estatística nos dados e pergunta-se quão surpreendente é o valor obtido.

Definição 3.3 p-valor

O **p-valor** é a probabilidade, calculada supondo H_0 verdadeira, de a estatística de teste assumir um valor tão ou mais extremo do que o observado.

Um p-valor pequeno diz que os dados seriam raros em um mundo no qual H_0 vale, e por isso depõe contra ela. Fixa-se antes um **nível de significância** α, por convenção 5%, e rejeita-se H_0 quando o p-valor fica abaixo dele.

Armadilha **O que o p-valor não é**

O p-valor não é a probabilidade de H_0 ser verdadeira, nem a probabilidade de o resultado ter ocorrido "por acaso". Ele é calculado *dentro* da hipótese nula e nada diz sobre a chance dela. Também não mede o tamanho do efeito: com amostra grande, uma diferença irrelevante na prática produz p-valor minúsculo. E um p-valor alto não prova H_0; pode apenas indicar que a amostra é pequena demais para enxergar o efeito.

A decisão pode errar de duas maneiras. O **erro tipo I** é rejeitar uma H_0 verdadeira, um alarme falso, e sua probabilidade é α por construção. O **erro tipo II** é não rejeitar uma H_0 falsa, com probabilidade β. O **poder** do teste é $1-\beta$, a probabilidade de detectar um efeito que existe. O painel esquerdo da Figura 3.5 mostra as três áreas. O poder cresce com o tamanho do efeito e com o tamanho da amostra, e cai com a dispersão dos dados. Em finanças os efeitos são pequenos e a dispersão é grande, e por isso o poder costuma ser baixo.

3.6 O retorno médio é diferente de zero?

A pergunta mais básica que se pode fazer a um ativo, ou a uma estratégia, é se o retorno esperado é positivo. A média amostral $\bar{r}$ é uma estimativa dele, e, como toda estimativa, tem erro.

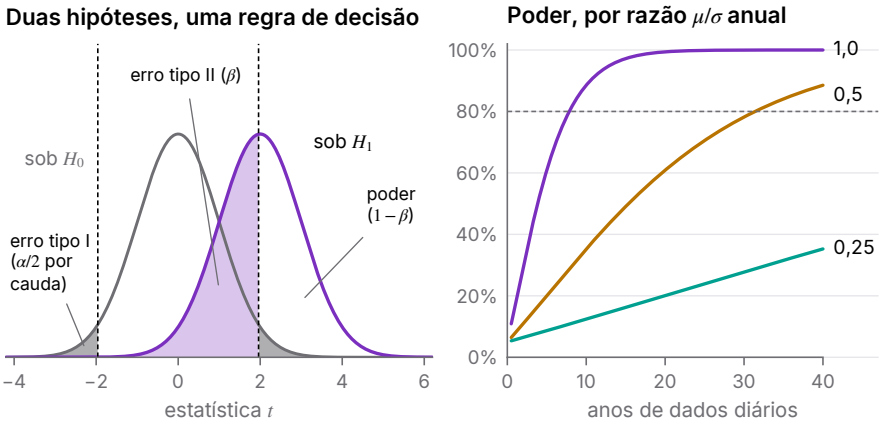


Figura 3.5 À esquerda, a distribuição da estatística t sob H_0 e sob uma alternativa em que seu valor esperado é 2; as linhas tracejadas marcam $\pm 1{,}96$. À direita, o poder do teste de média zero em função dos anos de dados, para três valores da razão entre retorno e volatilidade anuais. Com razão de 0,5, são precisos 31 anos para um poder de 80%.

Resultado 3.4 Erro-padrão da média e estatística t

Se $r_1, \ldots, r_n$ são não correlacionados, com média μ e variância σ^2, então $\mathrm{Var}(\bar{r}) = \sigma^2/n$. O desvio-padrão de $\bar{r}$, estimado por $s/\sqrt{n}$, é o **erro-padrão da média**. Para testar $H_0 : \mu = 0$ usa-se

$$t = \frac{\bar{r}}{s/\sqrt{n}}, \tag{3.5}$$

que, sob H_0, com n grande e dependência fraca entre as observações (o caso dos retornos), tem distribuição aproximadamente normal padrão. Rejeita-se H_0 a 5% quando $|t| > 1{,}96$.

Se os retornos fossem normais e independentes, a distribuição exata de t sob H_0 seria a t de Student com $n-1$ graus de liberdade, que é de onde vem o nome da distribuição. Com milhares de observações a distinção não tem efeito prático.

"Student" era o pseudônimo de William Gosset, que publicou o resultado em 1908.

Código 3.4 Teste t para a média do log-retorno do Ibovespa (`cap03/l04_teste_media.py`)

```
ibov = precos["^BVSP"].dropna()
r = np.log(ibov / ibov.shift(1)).dropna()
n, media, s = len(r), r.mean(), r.std()

ep = s / np.sqrt(n)                    # erro-padrão da média
t = media / ep                         # H0: média igual a zero
p = 2 * stats.t.sf(abs(t), df=n - 1)   # p-valor bilateral
print(f"n = {n} ({n / 252:.1f} anos)   média = {media:.4%} ao dia"
      f" = {252 * media:.2%} ao ano")
print(f"desvio = {s:.4%}   erro-padrão = {ep:.4%}")
vol = s * np.sqrt(252)
print(f"vol. anual = {vol:.1%}   média/vol. anual = "
      f"{252 * media / vol:.3f}")
print(f"t = {t:.3f}   p-valor = {p:.3f}")
res = stats.ttest_1samp(r, 0)
print(f"scipy: t = {res.statistic:.3f}"
      f"   p-valor = {res.pvalue:.3f}")
```

```
ic = media + np.array([-1, 1]) * stats.t.ppf(0.975, n - 1) * ep
print(f"IC de 95% para a média anual: {252 * ic[0]:.1%}"
      f" a {252 * ic[1]:.1%}")

# dias necessários se a média verdadeira fosse a amostral
z_a, z_b = stats.norm.ppf(0.975), stats.norm.ppf(0.80)
for nome, z in [("t = 1,96", z_a), ("poder de 80%", z_a + z_b)]:
    dias = (z * s / media) ** 2
    anos = dias / 252
    print(f"{nome:13s}: {dias:6.0f} dias = {anos:4.0f} anos")
```

```
n = 3963 (15.7 anos)   média = 0.0210% ao dia = 5.30% ao ano
desvio = 1.4626%   erro-padrão = 0.0232%
vol. anual = 23.2%   média/vol. anual = 0.228
t = 0.905   p-valor = 0.366
scipy: t = 0.905   p-valor = 0.366
IC de 95% para a média anual: -6.2% a 16.8%
t = 1,96     :  18598 dias =   74 anos
poder de 80% :  38000 dias =  151 anos
```

A anualização segue a convenção de 252 dias (Seção 1.9), pela qual os dezesseis anos de calendário da amostra contam como 15,7. Com todo esse histórico, não se rejeita a hipótese de que o log-retorno esperado do Ibovespa seja zero. O intervalo de confiança de 95% para a média anual vai de $-6{,}2\%$ a $16{,}8\%$: os dados são compatíveis com um índice que perde dinheiro e com um que rende muito bem. As duas últimas linhas fazem a conta inversa. Se a média verdadeira fosse exatamente a amostral, seriam necessários 74 anos de dados para a estatística esperada chegar a 1,96, e 151 anos para o teste detectar o efeito em 80% das vezes.

Intuição

A estatística t pode ser reescrita em unidades anuais. Com $\mu_a = 252\,\bar{r}$ e $\sigma_a = s\sqrt{252}$,

$$t = \frac{\bar{r}}{s/\sqrt{n}} = \frac{\mu_a}{\sigma_a}\sqrt{\frac{n}{252}} = \frac{\mu_a}{\sigma_a}\sqrt{\text{anos}}.$$

A precisão da média depende do número de anos cobertos pela amostra, qualquer que seja a frequência das observações. Trocar dados diários por dados de cinco minutos multiplica n sem acrescentar um único ano, e o erro-padrão da média anual, $\sigma_a/\sqrt{\text{anos}}$, continua igual.

No Ibovespa, $\mu_a/\sigma_a = 0{,}228$ e $\sqrt{15{,}7} = 3{,}97$, o que dá $t = 0{,}905$. A razão entre retorno e volatilidade anuais reaparece no Capítulo 6, descontada a taxa livre de risco, com o nome de índice de Sharpe. O painel direito da Figura 3.5 generaliza a conta: para uma razão de 0,5, respeitável para um ativo, dez anos de dados dão poder de 35%. É por isso que médias históricas de retorno são estimativas tão frágeis, um problema que volta na otimização de carteiras (Capítulo 6) e na avaliação de *backtests* (Capítulo 7). A volatilidade, em contraste, é estimada com precisão muito maior a partir da mesma amostra.

3.7 Testes de normalidade e de autocorrelação

3.7.1 Jarque-Bera

O teste de Jarque e Bera (1987) transforma a comparação visual da Seção 3.3 em uma estatística. A hipótese nula é que os dados vêm de uma distribuição normal, caso em que a assimetria amostral $\hat{S}$ e o excesso de curtose amostral $\hat{K} - 3$ devem estar perto de zero.

Resultado 3.5 Estatística de Jarque-Bera

$$JB = \frac{n}{6}\left[\hat{S}^2 + \frac{(\hat{K}-3)^2}{4}\right]. \tag{3.6}$$

Sob H_0 (normalidade) e para n grande, JB tem distribuição qui-quadrado com 2 graus de liberdade. Rejeita-se a normalidade a 5% quando $JB > 5{,}99$.

A fórmula vem de que, em amostras normais grandes, $\hat{S}$ tem variância aproximada $6/n$ e $\hat{K}$, $24/n$. Cada parcela de JB é o quadrado de uma dessas estimativas dividido pela sua variância, e a soma de dois quadrados de normais padrão independentes é uma qui-quadrado com 2 graus de liberdade.

Código 3.5 Jarque-Bera e Shapiro-Wilk em três frequências (`cap03/l05_normalidade.py`)

```python
def jarque_bera(x):
    n = len(x)
    s = stats.skew(x)                  # assimetria amostral
    k = stats.kurtosis(x)              # excesso de curtose
    jb = n / 6 * (s ** 2 + k ** 2 / 4)
    return jb, stats.chi2.sf(jb, df=2)

series = {"diário": r, "mensal": r.resample("ME").sum(),
          "anual": r.resample("YE").sum()}
for nome, x in series.items():
    jb, p = jarque_bera(x)
    sw = stats.shapiro(x)
    print(f"{nome:7s} n = {len(x):4d}   JB = {jb:8.1f}"
          f"  p = {p:8.2g}   Shapiro-Wilk: p = {sw.pvalue:.2g}")

print(f"scipy, diário: JB = {stats.jarque_bera(r).statistic:.1f}")
```

```
diário  n = 3963   JB =  24486.9  p =        0   Shapiro-Wilk: p = 5.1e-40
mensal  n =  192   JB =    241.9  p =    3e-53   Shapiro-Wilk: p = 1e-06
anual   n =   16   JB =      1.2  p =     0.54   Shapiro-Wilk: p = 0.3
scipy, diário: JB = 24486.9
```

Para os retornos diários, $JB = 24\,487$ contra um valor crítico de 5,99; o p-valor é tão pequeno que o computador o arredonda para zero. A normalidade também é rejeitada na frequência mensal, embora, como se viu na Seção 3.2, essa rejeição dependa de um único mês. Na frequência anual o teste não rejeita. Leia esse resultado com a armadilha do p-valor em mente: com 16 observações o teste quase não tem poder, a aproximação qui-quadrado é ruim (o teste rejeita menos do que os 5% nominais), e não rejeitar está longe de demonstrar que retornos anuais são normais.

Outros dois testes aparecem com frequência. O de Shapiro-Wilk, na segunda coluna de p-valores, costuma ter bom poder em amostras pequenas; a

documentação do SciPy avisa que seu p-valor pode não ser preciso acima de 5 000 observações. O de Kolmogorov-Smirnov compara a função de distribuição empírica com a teórica, mas na forma usual exige que média e variância sejam conhecidas de antemão. Estimá-las da própria amostra, como quase sempre se faz, invalida o p-valor tabelado; nesse caso use a variante de Lilliefors, disponível no `statsmodels`.

3.7.2 Ljung-Box

Para testar se uma série tem autocorrelação, olhar uma defasagem de cada vez leva ao problema da Seção 3.8. O teste de Ljung e Box (1978) junta as m primeiras autocorrelações amostrais $\hat{\rho}_1, \dots, \hat{\rho}_m$ em uma só estatística.

Resultado 3.6 Estatística de Ljung-Box

$$Q(m) = n(n+2)\sum_{k=1}^{m}\frac{\hat{\rho}_k^2}{n-k}. \tag{3.7}$$

Sob H_0 (observações independentes e identicamente distribuídas), $Q(m)$ tem distribuição aproximada qui-quadrado com m graus de liberdade.

Código 3.6 Ljung-Box nos retornos e nos retornos ao quadrado (`cap03/l06_ljungbox.py`)

```
series = {"retornos": r, "retornos ao quadrado": r ** 2}
for nome, x in series.items():
    rho = acf(x, nlags=3)[1:]
    print(f"{nome}: autocorrelações 1 a 3 =", rho.round(3))
    print(acorr_ljungbox(x, lags=[5, 10, 20]).round(4))
```

```
retornos: autocorrelações 1 a 3 = [-0.071  0.033 -0.004]
    lb_stat  lb_pvalue
5   30.0408        0.0
10  68.1284        0.0
20  76.0275        0.0
retornos ao quadrado: autocorrelações 1 a 3 = [0.505 0.527 0.405]
      lb_stat  lb_pvalue
5   3592.4921        0.0
10  4345.8839        0.0
20  4681.2280        0.0
```

O resultado dos quadrados é o esperado: $Q(10) = 4\,346$, uma rejeição esmagadora, que confirma o agrupamento de volatilidade. Aplicado a quadrados de retornos ou de resíduos, o teste é o diagnóstico usual desse efeito e dos modelos de volatilidade do Capítulo 4.

O resultado dos retornos parece contradizer o segundo fato estilizado, pois o teste também rejeita, com $Q(10) = 68{,}1$ e autocorrelação de ordem 1 igual a $-0{,}071$. Duas verificações mostram que a conclusão é mais fraca do que parece. Retirando 2020 da amostra, $Q(10)$ cai para 8,4, com p-valor de 0,59: a rejeição vem das semanas de março de 2020, em que quedas de mais de 10% e repiques de 9% a 13% se sucederam em dias próximos. Além disso, a hipótese nula do teste é de observações independentes, e não apenas não correlacionadas. Com volatilidade agrupada, as autocorrelações amostrais variam mais do que $1/\sqrt{n}$, e o teste rejeita com frequência maior do que α. Refazendo o teste da autocorrelação de ordem 1 com um erro-padrão que admite variância mutável

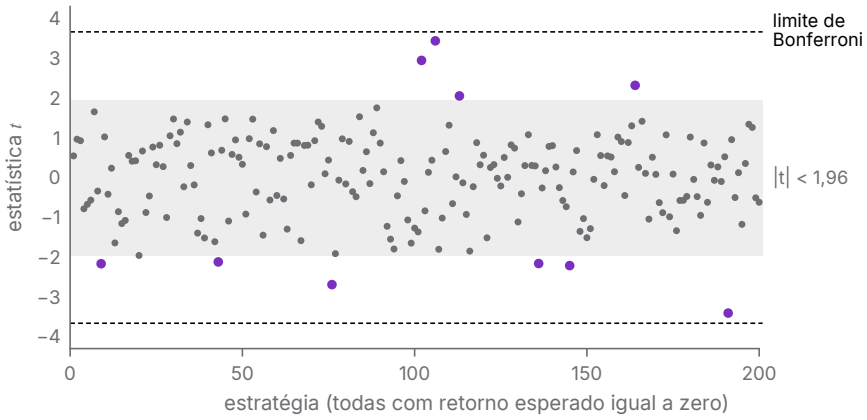


Figura 3.6 Estatísticas t de 200 séries simuladas de puro ruído, cada uma com cinco anos de retornos diários. Dez delas, em roxo, passam do limite de 5% sem que haja nada a descobrir. Nenhuma passa do limite de Bonferroni.

(o de White, opção `cov_type="HC0"` do `statsmodels`), a estatística t passa de $-4{,}49$ para $-1{,}66$, que não é significativa a 5%.

3.7.3 Uma hipótese por trás de todas as outras

Todas as contas deste capítulo tratam a amostra como repetições de um mesmo experimento: supõem que média, variância e autocorrelações não mudam com o tempo. Uma série com essa propriedade é dita **estacionária**. Preços não são estacionários, e essa é a razão formal para trabalhar com retornos. O teste de Dickey e Fuller (1979), em sua versão aumentada (ADF), tem como hipótese nula a presença de uma raiz unitária, isto é, de não estacionariedade. O Capítulo 4 o apresenta, junto com o que fazer quando a variância dos retornos muda no tempo.

3.8 Muitos testes, alguns “achados”

Um nível de significância de 5% significa que, em um teste com H_0 verdadeira, há 5% de chance de alarme falso. Em m testes independentes, todos com H_0 verdadeira, a probabilidade de ao menos um alarme falso é $1 - 0{,}95^m$: 64% para $m = 20$ e praticamente 100% para $m = 200$. Quem testa muitas ideias encontra resultados “significativos” mesmo que nenhuma funcione.

A Figura 3.6 mostra uma simulação com 200 “estratégias” cujos retornos são números aleatórios de média zero (`cap03/l07_multiplos.py`). Dez têm $|t| > 1{,}96$, perto dos 5% esperados (o número exato varia com a semente). A melhor tem $t = 3{,}44$, o que em cinco anos corresponde a uma razão anual entre retorno e volatilidade de 1,54, um número que qualquer gestor apresentaria com orgulho, obtido aqui de um sorteio.

A correção mais simples é a de **Bonferroni**: para manter em α a probabilidade de qualquer alarme falso em m testes, exige-se de cada um p-valor abaixo de α/m. Com $m = 200$, o limite sobe de $|t| > 1{,}96$ para $|t| > 3{,}66$, e nenhuma das séries simuladas o ultrapassa. O custo é a perda de poder: efeitos verdadeiros e modestos também deixam de ser detectados.

O problema é sério na pesquisa em finanças porque os testes já feitos por outras pessoas, sobre os mesmos dados, também contam. Harvey et al. (2016) catalogaram centenas de fatores publicados como explicativos de retornos de ações e argumentam que, dado esse volume de tentativas, um fator novo

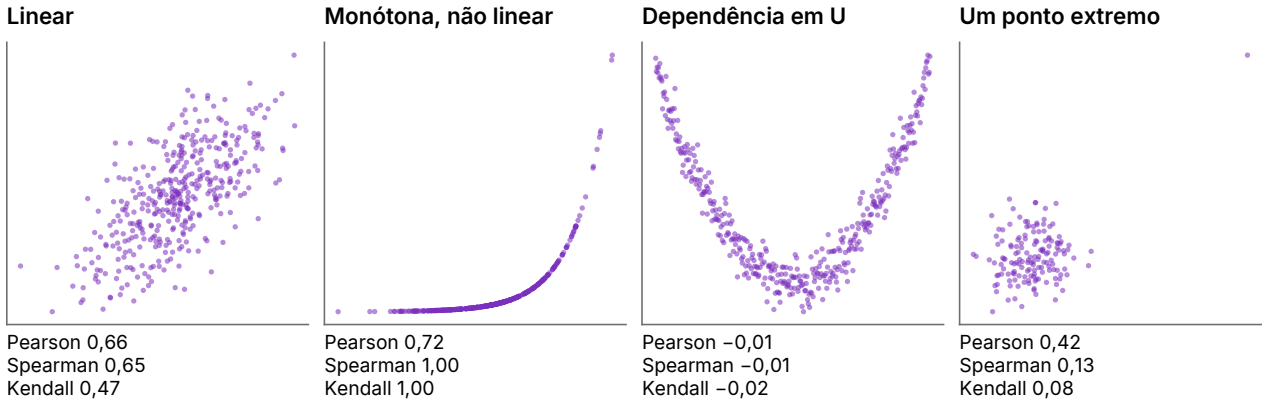


Figura 3.7 Três medidas de associação em quatro situações simuladas. Na relação monótona mas curva, Pearson fica abaixo de 1 e as medidas de posto valem 1. Na dependência em U, em que y é quase determinado por x, as três ficam perto de zero. No último painel, 149 pontos sem relação e um único ponto extremo bastam para um Pearson de $0{,}42$.

deveria superar uma estatística t de 3,0, e não o 2,0 habitual. O Capítulo 7 retoma o assunto no contexto em que ele mais prejudica: a escolha da melhor entre muitas variantes de uma estratégia.

3.9 Covariância e correlação

Até aqui cada ativo foi tratado isoladamente. Uma carteira depende de como os ativos se movem em conjunto.

Definição 3.7 Covariância e correlação de Pearson

Para duas variáveis X e Y com médias μ_X, μ_Y e desvios-padrão σ_X, σ_Y,

$$\operatorname{Cov}(X,Y) = \mathbb{E}\big[(X-\mu_X)(Y-\mu_Y)\big], \qquad \rho_{XY} = \operatorname{Corr}(X,Y) = \frac{\operatorname{Cov}(X,Y)}{\sigma_X\,\sigma_Y}. \tag{3.8}$$

A covariância é positiva quando desvios do mesmo sinal tendem a ocorrer juntos. Sua unidade é o produto das unidades das duas variáveis, o que a torna difícil de interpretar. A **correlação** divide pelos desvios-padrão e produz um número puro entre -1 e $+1$, que não muda se as variáveis forem reescaladas. Os extremos só são atingidos quando uma variável é função linear exata da outra.

O que a correlação de Pearson mede é a intensidade da relação *linear*. Duas consequências são fonte permanente de erro. Variáveis independentes têm correlação zero, mas a recíproca é falsa: correlação zero não implica independência. Os próprios retornos dão o exemplo, pois r_t e r_{t-1} são quase não correlacionados e claramente dependentes, já que o tamanho de um ajuda a prever o tamanho do outro. E a correlação, sendo construída com médias de produtos, é tão sensível a observações extremas quanto a variância.

Duas alternativas trabalham com a ordem dos dados em vez dos valores. A **correlação de Spearman** é a correlação de Pearson calculada sobre os postos (a posição de cada observação na amostra ordenada). O **tau de Kendall** toma todos os pares de observações e calcula a proporção de pares concordantes, em que x e y variam no mesmo sentido, menos a proporção de discordantes. Ambas medem associação *monótona*, não mudam sob transformações crescentes das

variáveis e são pouco afetadas por valores extremos. Como mostra a Figura 3.7, nenhuma das três enxerga uma dependência que não seja monótona.

No Código 3.7, `r` é a tabela de retornos simples diários de quatro ações, de 2015 a 2024. Os métodos `cov` e `corr` de um `DataFrame` devolvem as matrizes de que trata a próxima subseção.

Código 3.7 Covariância, correlação e variância de uma carteira (`cap03/l08_correlacao.py`)

```python
print("covariância anualizada:")
print((r.cov() * 252).round(3))
print("correlação de Pearson:")
print(r.corr().round(2))

# a correlação é a covariância dividida pelos desvios
dp = np.sqrt(np.diag(r.cov()))
print(np.allclose(r.cov() / np.outer(dp, dp), r.corr()))

for metodo in ["pearson", "spearman", "kendall"]:
    c = r["PETR4"].corr(r["VALE3"], method=metodo)
    print(f"PETR4 x VALE3, {metodo:8s}: {c:.3f}")

# variância de uma carteira com pesos iguais: w' Sigma w
w = np.full(4, 0.25)
vol_cart = np.sqrt(w @ (r.cov() * 252) @ w)
vol_media = (dp * np.sqrt(252)).mean()
print(f"vol. da carteira: {vol_cart:.1%}"
      f"   média das vols.: {vol_media:.1%}")
```

```
covariância anualizada:
       PETR4  VALE3  ITUB4  WEGE3
PETR4  0.224  0.092  0.080  0.047
VALE3  0.092  0.191  0.045  0.031
ITUB4  0.080  0.045  0.093  0.034
WEGE3  0.047  0.031  0.034  0.112
correlação de Pearson:
       PETR4  VALE3  ITUB4  WEGE3
PETR4   1.00   0.44   0.55   0.29
VALE3   0.44   1.00   0.34   0.21
ITUB4   0.55   0.34   1.00   0.34
WEGE3   0.29   0.21   0.34   1.00
True
PETR4 x VALE3, pearson : 0.443
PETR4 x VALE3, spearman: 0.385
PETR4 x VALE3, kendall : 0.268
vol. da carteira: 28.3%   média das vols.: 38.8%
```

Para PETR4 e VALE3 as três medidas dão 0,44, 0,39 e 0,27. O tau de Kendall é menor em valor absoluto por construção, e não por indicar associação mais fraca: em uma normal bivariada com correlação ρ ele vale $(2/\pi)\arcsin\rho$, que para $\rho = 0{,}443$ dá 0,29 (McNeil et al., 2015). Compare cada medida apenas com ela mesma.

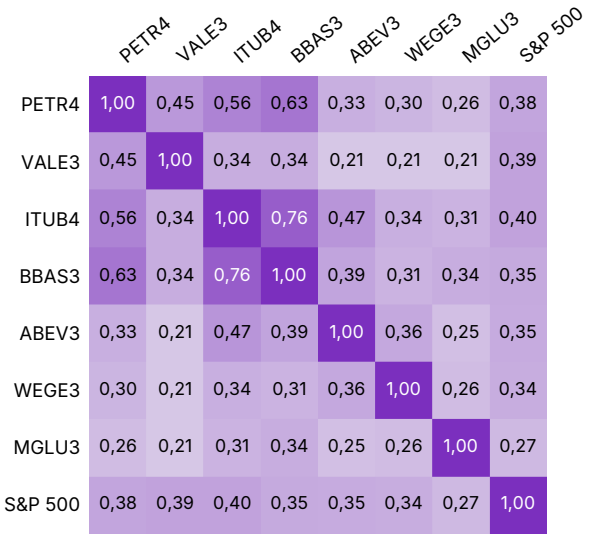


Figura 3.8 Correlações de Pearson entre retornos diários de sete ações brasileiras e do S&P 500, de 2015 a 2024, nos 2 409 dias com pregão nos dois países. Todas são positivas. Os dois bancos formam o par mais correlacionado ($0{,}76$), e as correlações entre ações vão de $0{,}21$ a $0{,}76$, com média de $0{,}36$. Alguns valores diferem na segunda casa dos do Código 3.7, cuja amostra inclui os dias sem pregão em Nova York.

3.9.1 As matrizes de covariância e de correlação

Para N ativos com vetor de retornos $\mathbf{R}$ e médias $\boldsymbol{\mu}$, as covariâncias de todos os pares são reunidas na **matriz de covariância**

$$\boldsymbol{\Sigma} = \mathbb{E}\left[(\mathbf{R}-\boldsymbol{\mu})(\mathbf{R}-\boldsymbol{\mu})^{\top}\right], \tag{3.9}$$

simétrica, com as variâncias na diagonal. A **matriz de correlação** tem a mesma estrutura, com 1 na diagonal. A importância de $\boldsymbol{\Sigma}$ está em que ela é tudo o que se precisa para a variância de qualquer carteira. Com pesos $\mathbf{w}$, o retorno da carteira é $\mathbf{w}^{\top}\mathbf{R}$ (Equação (1.2)) e

$$\operatorname{Var}(\mathbf{w}^{\top}\mathbf{R}) = \mathbf{w}^{\top}\boldsymbol{\Sigma}\,\mathbf{w} = \sum_{i=1}^{N}\sum_{j=1}^{N} w_i w_j \sigma_i \sigma_j \rho_{ij}. \tag{3.10}$$

Como uma variância não pode ser negativa, $\mathbf{w}^{\top}\boldsymbol{\Sigma}\,\mathbf{w} \geq 0$ para todo $\mathbf{w}$: a matriz é positiva semidefinida. A última linha da saída do Código 3.7 aplica a fórmula a quatro ações com pesos iguais. A volatilidade da carteira, de $28{,}3\%$ ao ano, é bem menor do que a média das volatilidades individuais, de $38{,}8\%$, porque as correlações estão longe de 1. Essa é a aritmética da diversificação, e o Capítulo 6 parte dela para chegar à fronteira eficiente de Markowitz (1952).

Uma matriz com N ativos tem $N(N+1)/2$ elementos distintos: 28 para as sete ações da Figura 3.8, 5 050 para uma carteira de cem. Cada um é estimado com erro, a partir de uma amostra que não cresce com N, e a matriz amostral se torna pouco confiável quando o número de ativos se aproxima do número de observações. Um remédio usual é o encolhimento (*shrinkage*) proposto por Ledoit e Wolf (2004a), e o problema volta no Capítulo 6.

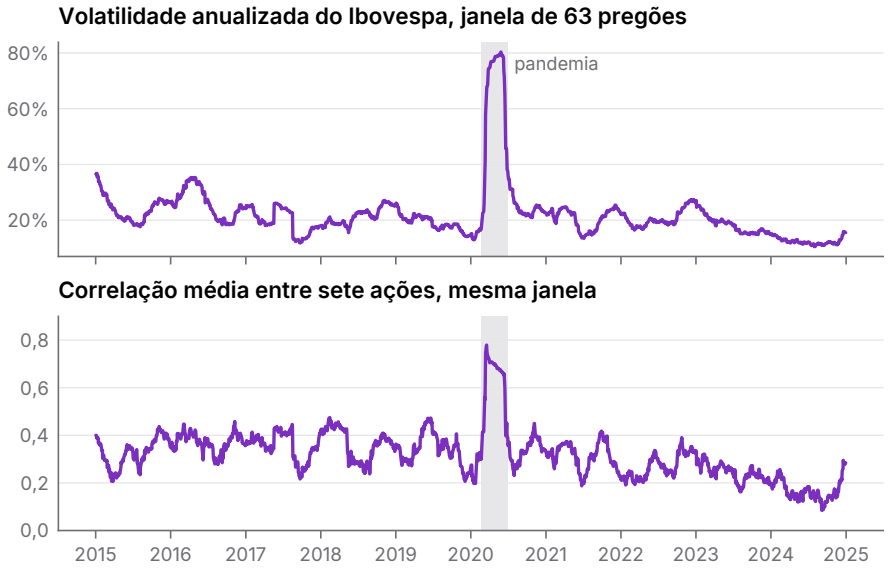


Figura 3.9 Volatilidade do Ibovespa e correlação média entre sete ações, em janelas móveis de 63 pregões. A correlação média oscila entre 0,09 e 0,78, com mediana de 0,32. O máximo, em 18 de março de 2020, coincide com o salto da volatilidade.

Armadilha **Correlacionar séries dessincronizadas**

De 2015 a 2024, no arquivo `precos_ajustados.csv`, a correlação entre o retorno diário do Ibovespa e o do dólar (`BRL=X`) na mesma data é 0,01. Deslocando o dólar em um dia, ela vai a −0,46 (`cap03/c03_sincronia.py`). A relação sempre esteve lá: nessa série, a variação do dólar registrada em uma data corresponde ao que a B3 viu no pregão anterior. A Seção 2.8 documenta o defeito, com números um pouco diferentes (−0,02 e −0,41) porque usa o período de 2010 a 2025, e mostra que a PTAX do Banco Central não o tem. Antes de interpretar uma correlação entre mercados diferentes, confira como cada série marca a data e a hora do fechamento, e calcule também as correlações defasadas.

3.10 Correlação que muda no tempo

Um único número para dez anos esconde variação. A **correlação móvel** recalcula a correlação a cada dia usando apenas as últimas J observações; em pandas, `r["A"].rolling(63).corr(r["B"])`. A Figura 3.9 faz isso com janela de 63 pregões (um trimestre) para os 21 pares formados pelas sete ações e mostra a média, ao lado da volatilidade do Ibovespa na mesma janela.

Entre 20 de fevereiro e 30 de abril de 2020, a correlação média entre as sete ações foi de 0,71, contra 0,36 no período inteiro. A de PETR4 com VALE3, que em dez anos é 0,44, foi de 0,83 nesse intervalo; a correlação móvel de 63 pregões do par chegou a 0,89 em 16 de março. Para quem monta carteiras a consequência é desagradável: a diversificação medida em tempos normais superestima a proteção disponível justamente quando ela é mais necessária.

Três cuidados impedem que essa observação vire uma conclusão maior do que os dados sustentam.

O primeiro é o erro de estimação. Para dados normais, o erro-padrão de uma correlação amostral é aproximadamente $(1-\rho^2)/\sqrt{n}$. Com $\rho = 0{,}3$ e $n = 63$ isso dá 0,11, ou uma faixa de $\pm 0{,}22$ em torno do valor verdadeiro. Boa parte

`cap03/c03_fator.py` confere por simulação essa aproximação e o exemplo do fator comum.

do sobe e desce de uma correlação móvel entre dois ativos é ruído de amostra, e a média de 21 pares da figura é mais estável do que qualquer par.

O segundo é que a amostra contém uma única crise dessa magnitude. Ela ilustra o fenômeno, mas não permite medi-lo com precisão. A evidência de que correlações entre mercados de ações aumentam nas quedas acentuadas vem de estudos com amostras muito mais longas, como o de Longin e Solnik (2001), com retornos mensais de 1959 a 1996 dos cinco maiores mercados. Eles encontram correlação maior nas grandes quedas, mas não nas grandes altas.

O terceiro é mais sutil: a correlação pode subir sem que a ligação entre os ativos tenha mudado.

Exemplo 3.1 Correlação e volatilidade de um fator comum

Suponha que duas ações sigam $R_i = M + \varepsilon_i$, em que M é um movimento de mercado com desvio-padrão σ_M e ε_1, ε_2 são componentes específicos, independentes entre si e de M, com desvio-padrão σ_ε. Então $\mathrm{Cov}(R_1, R_2) = \sigma_M^2$, $\mathrm{Var}(R_i) = \sigma_M^2 + \sigma_\varepsilon^2$ e

$$\rho = \frac{\sigma_M^2}{\sigma_M^2 + \sigma_\varepsilon^2}.$$

Com $\sigma_\varepsilon = 1{,}5\%$ e $\sigma_M = 1\%$ ao dia, $\rho = 1/3{,}25 = 0{,}31$. Se a volatilidade do mercado quadruplica, para $\sigma_M = 4\%$, e nada mais muda, $\rho = 16/18{,}25 = 0{,}88$.

Nesse modelo a sensibilidade de cada ação ao mercado é a mesma nos dois regimes; o que muda é a proporção da variância que vem do fator comum. Forbes e Rigobon (2002) mostraram que esse efeito enviesa para cima as correlações medidas em períodos turbulentos e que, depois de corrigi-lo, boa parte do aparente "contágio" entre mercados em crises dos anos 1980 e 1990 desaparece. Uma correlação alta em 2020 não exige, por si só, um mecanismo novo de dependência. O exemplo, porém, é simétrico: a volatilidade do fator sobe tanto nas quedas quanto nas altas, e sozinho ele não explica a diferença entre umas e outras encontrada por Longin e Solnik (2001).

No repositório

`aula3/s3_03_testes_hipotese.py` aplica o teste t, o de Ljung-Box e os de normalidade, além de uma comparação de duas amostras. `aula3/s3_04_correlacao.py` calcula as três correlações, a correlação móvel e a comparação entre períodos de crise e de calma para um conjunto maior de ativos.

O que levar deste capítulo

- A média diária de um retorno é dezenas de vezes menor do que seu desvio-padrão. Por isso o retorno esperado é estimado com enorme incerteza, que só diminui com mais anos de dados, e não com dados mais frequentes.
- Retornos diários têm caudas pesadas, autocorrelação linear quase nula e volatilidade agrupada. A normal subestima a frequência de dias extremos por ordens de grandeza; a t de Student com poucos graus de liberdade descreve melhor as caudas, mas não o agrupamento.
- Assimetria, curtose e os testes construídos sobre elas são dominados por poucas observações. Refaça a conta sem o período extremo antes de acreditar nela.

- O p-valor é calculado supondo H_0 verdadeira. Ele não é a probabilidade de H_0, e não rejeitar não equivale a confirmar, sobretudo quando o poder é baixo.
- Testar muitas hipóteses ao nível de 5% torna os falsos achados quase certos. O limite de significância precisa subir com o número de tentativas.
- A correlação de Pearson mede apenas dependência linear, é sensível a extremos e muda no tempo. Ela sobe nas crises, em parte porque a volatilidade do fator comum sobe.

Exercícios

3.1 [básico] Carregue os preços ajustados de `WEGE3.SA` e `MGLU3.SA` de 2015 a 2024 e monte, para os log-retornos diários, a tabela do Código 3.1 acrescida de assimetria e excesso de curtose. Em qual dos dois ativos a diferença entre s e IQR/1,349 é maior?

3.2 [básico] Uma t de Student tem $\nu = 6$ graus de liberdade. Calcule a variância e o excesso de curtose. Qual fator de escala a transforma em uma variável de variância 1? Confira com `scipy.stats.t.stats(6, moments="vk")`.

3.3 [básico] Uma estratégia rendeu em média 8% ao ano, com volatilidade de 16% ao ano, durante 6 anos. Calcule a estatística t para a hipótese de retorno esperado nulo. Quantos anos seriam necessários para chegar a $t = 1{,}96$ com a mesma média e a mesma volatilidade?

3.4 [intermediário] Faça o QQ-plot dos log-retornos diários de `PETR4.SA` contra a normal e contra a t ajustada por máxima verossimilhança. Repita com retornos mensais. O que muda, e qual fato estilizado explica a mudança?

3.5 [intermediário] Implemente a estatística da Equação (3.7) com NumPy, para $m = 10$, e confira com `acorr_ljungbox` nos retornos e nos retornos ao quadrado de `VALE3.SA`. Repita excluindo 2020 e comente.

3.6 [intermediário] Simule 1 000 amostras de 250 observações de uma t com $\nu = 4$ e aplique o teste de Jarque-Bera a cada uma. Em que fração das amostras a normalidade é rejeitada a 5%? Repita com amostras de 25 observações. Que conceito da Seção 3.5 os dois resultados ilustram?

3.7 [desafio] Refaça a simulação da Figura 3.6 trocando o ruído normal por uma t com $\nu = 3$ reescalada para o mesmo desvio-padrão. A proporção de falsos positivos do teste t a 5% muda? Relacione o resultado com o teorema central do limite.

3.8 [desafio] Verifique por simulação o exemplo do fator comum: gere R_1 e R_2 com σ_M alternando entre 1% e 4% em blocos de 250 dias e $\sigma_\varepsilon = 1{,}5\%$. Calcule a correlação em cada regime e a correlação da amostra inteira. Por que esta última não é a média das duas?

Para ir além

- Cont (2001), o artigo inteiro, curto e legível, para a lista completa de fatos estilizados e os cuidados estatísticos ao medi-los.
- Tsay (2010), capítulo 1, para momentos amostrais, testes de normalidade e distribuições de retornos com exemplos de dados reais.
- Campbell et al. (1997), os capítulos iniciais, para os testes de previsibilidade, incluindo versões que admitem heterocedasticidade.
- McNeil et al. (2015), capítulos 3 e 7, para caudas pesadas e para os limites da correlação como medida de dependência.
- Harvey et al. (2016), para múltiplos testes na pesquisa em finanças.

Capítulo 4

Séries temporais: estacionariedade, ARIMA e GARCH

4

Os retornos de ontem ajudam a prever os de hoje? Para o sinal e o tamanho médio do retorno, quase nada. Para a dispersão, bastante: dias agitados vêm em sequência. Este capítulo dá as ferramentas para verificar as duas coisas com dados, e para transformar a segunda em um modelo de volatilidade que muda a cada dia.

Ao terminar este capítulo você consegue

- explicar o que é estacionariedade e por que preços não a têm e retornos, em boa aproximação, sim;
- aplicar e interpretar os testes ADF e KPSS sem inverter as hipóteses;
- ler uma ACF e uma PACF, incluindo o que as bandas de confiança garantem e o que não garantem;
- identificar, estimar e diagnosticar um modelo ARIMA, e avaliá-lo fora da amostra contra uma previsão ingênua;
- estimar um GARCH(1,1) com a biblioteca `arch` e extrair dele persistência, volatilidade de longo prazo e previsões.

4.1 Quando a ordem das observações importa

Uma **série temporal** é uma sequência de observações indexadas pelo tempo, $x_1, x_2, \ldots, x_T$. A média, o desvio-padrão e o histograma do Capítulo 3 não mudam se você embaralhar as datas. Tudo o que este capítulo estuda muda: a relação entre x_t e x_{t-1}, a tendência de dias turbulentos virem juntos, o que se pode dizer sobre amanhã sabendo o que houve hoje.

O modelo matemático de uma série temporal é o **processo estocástico**: uma variável aleatória para cada instante. A dificuldade prática é que só observamos uma trajetória. Há um único histórico do Ibovespa, e não é possível repetir o ano de 2020 para ver o que mais poderia ter acontecido. Para aprender alguma coisa com uma única trajetória é preciso supor que o comportamento estatístico

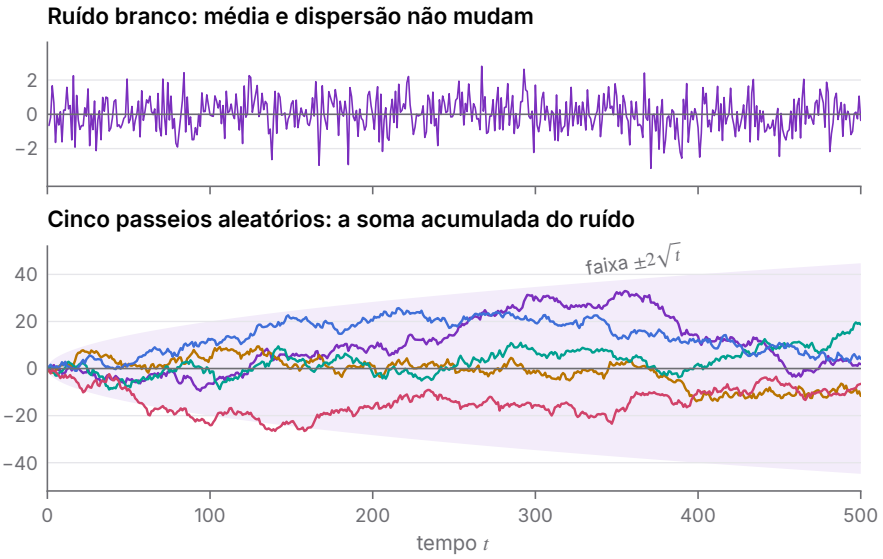


Figura 4.1 Em cima, 500 observações de um ruído branco gaussiano. Embaixo, cinco passeios aleatórios construídos acumulando ruídos do mesmo tipo. As trajetórias se afastam de zero sem força que as traga de volta, e a faixa $\pm 2\sqrt{t}$ (dois desvios-padrão) se alarga com o tempo.

do processo se repete ao longo do tempo, de modo que médias calculadas no tempo façam o papel de médias entre trajetórias. Essa suposição tem nome.

4.2 Estacionariedade

Definição 4.1 Estacionariedade fraca

Um processo $\{x_t\}$ é **fracamente estacionário** (ou estacionário em covariância) se, para todo t e toda defasagem k,

$$\mathbb{E}[x_t] = \mu, \qquad \operatorname{Var}(x_t) = \gamma_0 < \infty, \qquad \operatorname{Cov}(x_t, x_{t-k}) = \gamma_k. \tag{4.1}$$

A média e a variância não dependem de t, e a covariância entre duas observações depende apenas da distância k entre elas.

A **estacionariedade estrita** pede mais: que a distribuição conjunta de qualquer bloco $(x_{t_1}, \dots, x_{t_n})$ seja a mesma do bloco deslocado $(x_{t_1+k}, \dots, x_{t_n+k})$. Na prática trabalha-se com a versão fraca, que só envolve média, variância e covariâncias, e é a ela que a palavra "estacionária" se refere no resto do capítulo.

A versão estrita implica a fraca quando a variância é finita. A recíproca só vale em casos particulares, como o de processos gaussianos.

Dois processos servem de referência para tudo o que vem depois. O **ruído branco** é uma sequência $\{\varepsilon_t\}$ com média zero, variância constante σ^2 e nenhuma correlação entre instantes diferentes. Ele é estacionário e não tem memória alguma. O **passeio aleatório** é a soma acumulada de um ruído branco:

$$x_t = x_{t-1} + \varepsilon_t \quad \Longrightarrow \quad x_t = x_0 + \varepsilon_1 + \varepsilon_2 + \cdots + \varepsilon_t. \tag{4.2}$$

Cada choque entra na soma e nunca mais sai. A consequência aparece na variância: como os choques são não correlacionados, $\operatorname{Var}(x_t) = t\,\sigma^2$ para x_0 fixo. A variância cresce com o tempo, a condição (4.1) falha, e o processo não tem um nível ao qual retornar. Com uma constante δ somada a cada passo, $x_t = \delta + x_{t-1} + \varepsilon_t$, tem-se o passeio aleatório com **deriva**, que acrescenta uma tendência δt sem mudar a natureza do problema.

A Figura 4.1 mostra os dois. Olhe para qualquer trajetória do painel de baixo isoladamente: é fácil enxergar nela "tendências de alta", "topos" e "suportes". Todas foram geradas somando números sorteados de forma independente.

4.2.1 Raiz unitária e diferenciação

Escreva o passeio aleatório como caso particular de $x_t = \phi\, x_{t-1} + \varepsilon_t$. Com $|\phi| < 1$ o efeito de um choque decai como ϕ^k e o processo é estacionário (Seção 4.6). Com $\phi = 1$ o choque é permanente. Diz-se então que o processo tem uma **raiz unitária**.

O nome vem do polinômio $1 - \phi z$ associado ao processo, cuja raiz é $z = 1/\phi$. Quando $\phi = 1$, a raiz é 1.

O remédio é simples. Se x_t é um passeio aleatório, a **primeira diferença** $\Delta x_t = x_t - x_{t-1} = \varepsilon_t$ é um ruído branco. Uma série que se torna estacionária depois de diferenciada d vezes é chamada de integrada de ordem d, ou $I(d)$; a série estacionária é $I(0)$.

Você já fez essa operação sem lhe dar esse nome. Pela Equação (1.4) do Capítulo 1, o log-retorno é $r_t = \ln P_t - \ln P_{t-1}$, a primeira diferença do logaritmo do preço. Dizer que o log do preço se comporta aproximadamente como um passeio aleatório com deriva e dizer que os log-retornos são aproximadamente um ruído em torno de uma média pequena são a mesma afirmação. É a justificativa estatística para a regra prática de trabalhar com retornos.

4.3 Regressão espúria

A não estacionariedade cobra seu preço mais alto quando se relacionam duas séries. Tome dois passeios aleatórios gerados de forma independente e faça a regressão de um sobre o outro. Como não há relação nenhuma, espera-se um coeficiente angular estatisticamente nulo em 95% das tentativas. O Código 4.1 repete o experimento 2 000 vezes.

Código 4.1 Regressão entre passeios aleatórios independentes (`cap04/l01_espuria.py`)

```
import numpy as np
import statsmodels.api as sm
rng = np.random.default_rng(42)
T, N = 500, 2000
t_nivel, t_dif, r2 = [], [], []
for _ in range(N):
    # dois passeios aleatórios sem nenhuma relação entre si
    x = rng.standard_normal(T).cumsum()
    y = rng.standard_normal(T).cumsum()
    nivel = sm.OLS(y, sm.add_constant(x)).fit()
    dx, dy = np.diff(x), np.diff(y)
    dif = sm.OLS(dy, sm.add_constant(dx)).fit()
    t_nivel.append(nivel.tvalues[1])
    t_dif.append(dif.tvalues[1])
    r2.append(nivel.rsquared)
t_nivel, t_dif, r2 = map(np.array, (t_nivel, t_dif, r2))
rej_nivel = np.mean(abs(t_nivel) > 1.96)
rej_dif = np.mean(abs(t_dif) > 1.96)
print(f"|t| > 1,96 em níveis:     {rej_nivel:.1%}")
print(f"|t| > 1,96 em diferenças: {rej_dif:.1%}")
print(f"R² acima de 0,5 em níveis: {np.mean(r2 > 0.5):.1%}")
```

```
|t| > 1,96 em níveis:     90.0%
|t| > 1,96 em diferenças: 5.3%
R² acima de 0,5 em níveis: 18.0%
```

Em nove de cada dez simulações o teste t usual declara "significativa" uma relação que não existe, e em quase uma de cada cinco o R^2 passa de 0,5. Refeita nas diferenças, a mesma regressão rejeita em 5,3% dos casos, o que se espera de um teste a 5%. O fenômeno foi documentado por simulação por Granger e Newbold (1974), que o chamaram de **regressão espúria**. A razão é que os resíduos de uma regressão entre passeios aleatórios herdam a raiz unitária, o que viola as hipóteses que dão à estatística t a sua distribuição; ela cresce com o tamanho da amostra em vez de se estabilizar.

Armadilha **Correlação entre preços**

Dois ativos cujos preços subiram na mesma década terão correlação de preços alta, qualquer que seja a relação entre eles. Correlações, betas e regressões devem ser calculados com retornos. A exceção legítima é a **cointegração**, em que uma combinação de séries $I(1)$ é estacionária; ela exige teste próprio e não se presume a partir de um gráfico.

4.4 Testes de raiz unitária

4.4.1 Dickey-Fuller e sua versão aumentada

Para testar se $\phi = 1$ em $x_t = \phi\, x_{t-1} + \varepsilon_t$, subtraia x_{t-1} dos dois lados. O resultado é $\Delta x_t = \gamma\, x_{t-1} + \varepsilon_t$, com $\gamma = \phi - 1$, e a hipótese de raiz unitária vira $\gamma = 0$. A regressão diz, em palavras, que em uma série estacionária o nível de ontem ajuda a prever a variação de hoje (quem está acima da média tende a cair, logo $\gamma < 0$), e em um passeio aleatório não ajuda.

Na prática as variações Δx_t podem ter autocorrelação própria, que contaminaria o teste. A versão aumentada do teste de Dickey e Fuller (1979), cuja validade para processos ARMA de ordem desconhecida foi estabelecida por

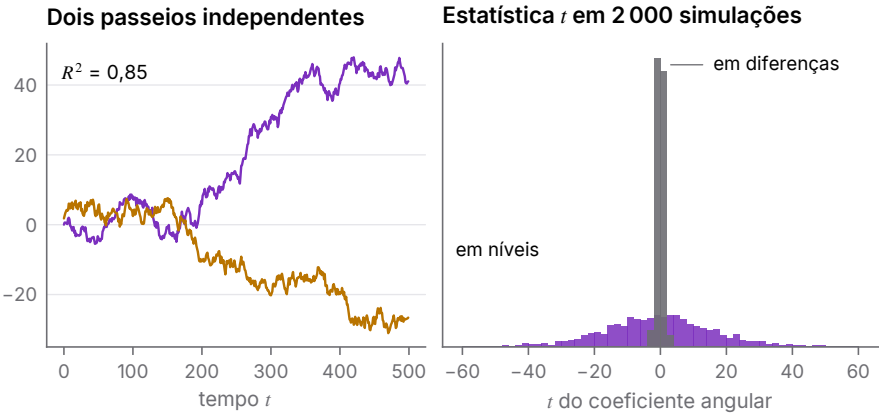


Figura 4.2 À esquerda, um dos pares de passeios independentes simulados, escolhido pelo R^2 alto. À direita, a distribuição da estatística t do coeficiente angular nas 2 000 simulações: em níveis ela se espalha por dezenas de unidades; em diferenças fica concentrada entre -2 e 2, como deveria.

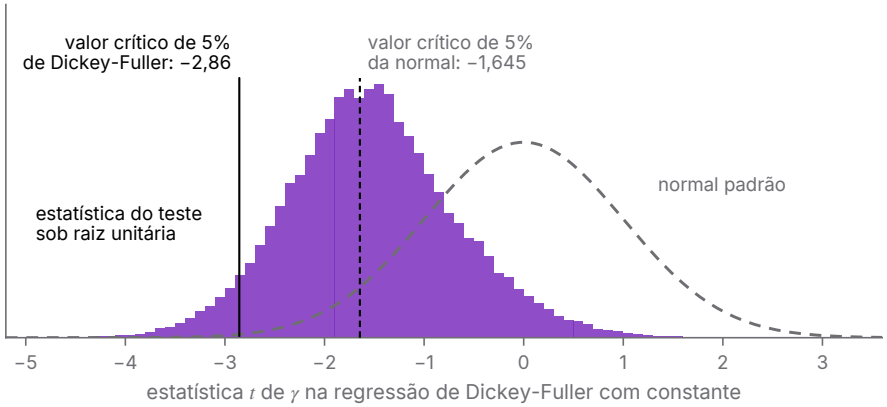


Figura 4.3 Distribuição simulada da estatística de Dickey-Fuller com constante quando a série é de fato um passeio aleatório (50 mil simulações de 500 observações). O valor crítico de 5% é $-2{,}86$. Quem usasse o valor $-1{,}645$ da normal rejeitaria uma raiz unitária verdadeira em 45,6% das vezes.

Said e Dickey (1984), acrescenta defasagens de Δx_t para absorvê-la:

$$\Delta x_t = a + b\,t + \gamma\, x_{t-1} + \sum_{i=1}^{p} \delta_i\, \Delta x_{t-i} + \varepsilon_t. \tag{4.3}$$

Esse é o teste **ADF** (*augmented Dickey-Fuller*), que confronta $H_0 : \gamma = 0$ com $H_1 : \gamma < 0$. A estatística é a razão t usual de $\hat{\gamma}$, e três detalhes decidem se ele é usado direito.

- **A hipótese nula é a raiz unitária.** Rejeitar H_0 é evidência de estacionariedade. Não rejeitar significa apenas que os dados são compatíveis com uma raiz unitária; o teste tem pouco poder quando ϕ está perto de 1, e uma série estacionária muito persistente passa facilmente por não estacionária em amostras curtas.
- **Os valores críticos não são os da t de Student.** Sob H_0 o regressor x_{t-1} é não estacionário, e a estatística tem uma distribuição própria, deslocada para a esquerda (Figura 4.3). O `statsmodels` já usa a tabela certa; o erro é comparar o número com $-1{,}645$ por conta própria.
- **Os termos determinísticos mudam a tabela.** A constante a e a tendência $b\,t$ entram conforme a alternativa que faz sentido: só constante, se a alternativa é uma série que oscila em torno de um nível fixo (`regression="c"`); constante e tendência, se a alternativa é oscilar em torno de uma reta (`"ct"`). O script da Figura 4.3 simula os três casos com 500 observações e obtém valores críticos de 5% de $-1{,}94$ (sem constante), $-2{,}86$ (com constante) e $-3{,}42$ (com constante e tendência). O `statsmodels` calcula os valores críticos para o tamanho exato da amostra por uma aproximação numérica, e por isso a saída do Código 4.2 traz $-3{,}41$ no último caso. O número de defasagens p costuma ser escolhido por critério de informação; é o que faz `autolag="AIC"`.

4.4.2 KPSS: a hipótese ao contrário

O teste **KPSS**, de Kwiatkowski et al. (1992), inverte os papéis: a hipótese nula é que a série é estacionária (em torno de um nível ou de uma tendência) e a alternativa é a raiz unitária. Valores altos da estatística rejeitam a estacionariedade. Usar os dois testes juntos protege contra a leitura apressada de um "não rejeitei". Se o ADF rejeita e o KPSS não, a evidência aponta para

estacionariedade; se o ADF não rejeita e o KPSS rejeita, para raiz unitária; se discordam, a amostra não decide. Os dois desacordos não são equivalentes: quando nenhum rejeita, falta poder; quando ambos rejeitam, a série pode não ser nem $I(0)$ nem $I(1)$ simples, como ocorre com quebras de nível.

Código 4.2 ADF e KPSS no Ibovespa, 2010 a 2025 (`cap04/l02_raiz_unitaria.py`)

```
import warnings
import numpy as np
import pandas as pd
from statsmodels.tools.sm_exceptions import InterpolationWarning
from statsmodels.tsa.stattools import adfuller, kpss
# o kpss avisa quando o p-valor cai fora da faixa tabelada
warnings.simplefilter("ignore", InterpolationWarning)
precos = pd.read_csv("../../dados/precos_ajustados.csv",
                     index_col=0, parse_dates=True)
ibov = precos["^BVSP"].dropna()
log_p = np.log(ibov)               # log do índice
r = log_p.diff().dropna()          # log-retorno: 1ª diferença
print("série        teste  estat.  crít. 5%  defas.")
for nome, s in [("log do preço", log_p), ("log-retorno ", r)]:
    est, pv, lags, n, crit, _ = adfuller(s, regression="c",
                                         autolag="AIC")
    c5 = crit["5%"]
    print(f"{nome} ADF   {est:7.2f}  {c5:8.2f}  {lags:6d}")
    est, pv, lags, crit = kpss(s, regression="c", nlags="auto")
    c5 = crit["5%"]
    print(f"{nome} KPSS  {est:7.2f}  {c5:8.2f}  {lags:6d}")
est, pv, _, _, crit, _ = adfuller(log_p, regression="ct",
                                  autolag="AIC")
print(f"log do preço, ADF com tendência: {est:.2f}"
      f"  (crít. 5%: {crit['5%']:.2f})")
```

```
série        teste  estat.  crít. 5%  defas.
log do preço ADF     -0.49     -2.86       7
log do preço KPSS     8.34      0.46      39
log-retorno  ADF    -23.38     -2.86       6
log-retorno  KPSS     0.15      0.46      14
log do preço, ADF com tendência: -2.87  (crít. 5%: -3.41)
```

Os dois testes contam a mesma história. Para o log do índice, o ADF fica longe de rejeitar a raiz unitária ($-0{,}49$ contra um valor crítico de $-2{,}86$) e o KPSS rejeita a estacionariedade com folga ($8{,}34$ contra $0{,}46$). Para os log-retornos ocorre o inverso. A última linha responde a uma objeção natural, a de que o índice poderia ser estacionário em torno de uma tendência de alta: com tendência na regressão a estatística é $-2{,}87$, ainda acima do valor crítico de $-3{,}41$. O Ibovespa se comporta como uma série $I(1)$ e seus retornos como $I(0)$.

O filtro silencia apenas o `InterpolationWarning`, que o `kpss` emite quando o p-valor cai fora da faixa tabelada, de 1% a 10%.

Armadilha **Estacionário não quer dizer comportado**

Os testes acima tratam do nível da série. Passar neles não garante retornos normais, nem independentes, nem com volatilidade parecida todos os dias. A Seção 4.8 mostra que uma série pode ter variância de longo prazo constante, como pede a estacionariedade fraca, e ainda assim alternar períodos calmos e turbulentos.

4.5 Autocorrelação: ACF e PACF

Em uma série estacionária, a dependência linear entre observações separadas por k períodos é medida pela **autocorrelação** de ordem k,

$$\rho_k = \frac{\gamma_k}{\gamma_0} = \operatorname{Corr}(x_t, x_{t-k}), \qquad \hat{\rho}_k = \frac{\sum_{t=k+1}^{T} (x_t - \bar{x})(x_{t-k} - \bar{x})}{\sum_{t=1}^{T} (x_t - \bar{x})^2}, \tag{4.4}$$

em que $\hat{\rho}_k$ é a estimativa amostral. O gráfico de $\hat{\rho}_k$ contra k é a **função de autocorrelação** (ACF).

A ACF mistura efeitos diretos e indiretos. Se hoje depende de ontem e ontem depende de anteontem, hoje e anteontem serão correlacionados mesmo que não exista ligação direta entre eles. A **função de autocorrelação parcial** (PACF) isola a ligação direta: seu valor na defasagem k, ϕ_{kk}, é o coeficiente de x_{t-k} em uma regressão de x_t sobre $x_{t-1}, \dots, x_{t-k}$, isto é, a correlação entre x_t e x_{t-k} depois de descontado o que as defasagens intermediárias explicam.

4.5.1 O que as bandas de confiança dizem

Os gráficos de ACF trazem uma faixa em torno de zero. Ela vem do seguinte resultado: se a série é uma sequência independente e identicamente distribuída com variância finita, então para T grande cada $\hat{\rho}_k$ é aproximadamente normal com média zero e variância $1/T$ (Box et al., 2015). A faixa de 95% é, portanto,

$$\pm \frac{1{,}96}{\sqrt{T}}. \tag{4.5}$$

Uma barra fora da faixa é evidência contra a hipótese de independência naquela defasagem. Duas ressalvas evitam leituras erradas. A primeira: com 40 defasagens, espera-se que cerca de duas saiam da faixa por acaso; para julgar o conjunto use o teste de Ljung-Box (Capítulo 3). A segunda: com milhares de observações a faixa fica estreita, e uma autocorrelação de 0,05 pode ser estatisticamente significativa e economicamente irrelevante, porque explica $0{,}05^2 = 0{,}25\%$ da variância.

4.6 Modelos AR, MA e ARMA

Três famílias de modelos lineares descrevem como uma série estacionária depende do próprio passado. Em todas, ε_t é um ruído branco de variância σ^2.

Definição 4.2 Processos AR, MA e ARMA

O processo **autorregressivo** de ordem p, AR(p), explica x_t por seus p valores anteriores; o de **médias móveis** de ordem q, MA(q), pelos q choques anteriores; o ARMA(p, q) combina os dois:

$$x_t = c + \underbrace{\phi_1 x_{t-1} + \cdots + \phi_p x_{t-p}}_{\text{parte AR}} + \varepsilon_t + \underbrace{\theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}}_{\text{parte MA}}. \tag{4.6}$$

O AR(1), $x_t = c + \phi\, x_{t-1} + \varepsilon_t$, mostra o essencial com pouca álgebra.

Resultado 4.3 Momentos do AR(1)

Se $|\phi| < 1$, o AR(1) é estacionário, com

$$\mathbb{E}[x_t] = \mu = \frac{c}{1-\phi}, \qquad \operatorname{Var}(x_t) = \frac{\sigma^2}{1-\phi^2}, \qquad \rho_k = \phi^k. \tag{4.7}$$

As três fórmulas saem de impor a estacionariedade na própria equação do processo. Tomando a esperança dos dois lados, $\mu = c + \phi\mu$. Tomando a variância, e lembrando que ε_t não é correlacionado com x_{t-1}, $\gamma_0 = \phi^2\gamma_0 + \sigma^2$. Multiplicando a equação (em desvios da média) por $x_{t-k} - \mu$ e tomando a esperança, $\gamma_k = \phi\,\gamma_{k-1}$, de onde $\rho_k = \phi^k$. Quando $|\phi| \geq 1$ não existe variância finita que satisfaça a segunda equação. Para um AR(p) a condição equivalente é que todas as raízes do polinômio $1 - \phi_1 z - \cdots - \phi_p z^p$ estejam fora do círculo unitário (Hamilton, 1994).

Exemplo 4.1 Um AR(1) conferido por simulação

Com $c = 0{,}5$, $\phi = 0{,}8$ e $\sigma = 1$, as fórmulas dão média $0{,}5/0{,}2 = 2{,}5$ e variância $1/(1-0{,}64) = 2{,}778$. O script `cap04/l03_ar1.py` simula 200 mil observações desse processo e obtém média 2,505, variância 2,790 e autocorrelações 0,801, 0,643 e 0,516 nas três primeiras defasagens, contra 0,8, 0,64 e 0,512 teóricos. Note que a média do processo não é a constante c: a persistência a amplifica por $1/(1-\phi) = 5$.

O MA(1), $x_t = \mu + \varepsilon_t + \theta\,\varepsilon_{t-1}$, é estacionário para qualquer θ e tem memória de exatamente um período: $\rho_1 = \theta/(1+\theta^2)$ e $\rho_k = 0$ para $k \geq 2$, porque observações separadas por dois ou mais períodos não compartilham nenhum choque.

Intuição

No AR, um choque se propaga por meio da própria série e seu efeito decai aos poucos, como ϕ^k, sem nunca zerar. No MA, o choque atua por um número fixo de períodos e desaparece. A ACF e a PACF são as ferramentas que distinguem as duas formas de memória.

A Figura 4.4 mostra essas assinaturas, e a Tabela 4.1 as generaliza. Em dados reais os padrões raramente são tão limpos, e ordens diferentes produzem gráficos parecidos; a tabela sugere candidatos, e a escolha final fica para os critérios da próxima seção.

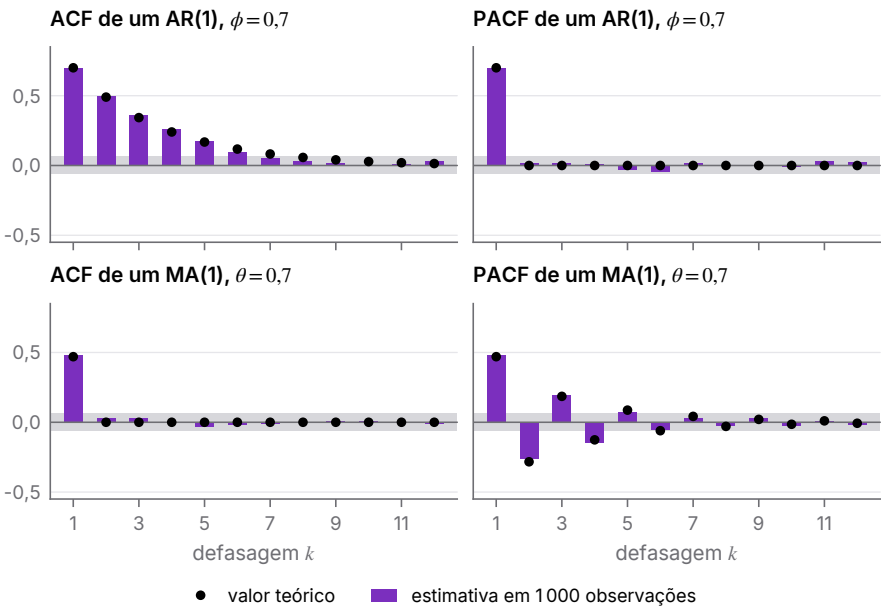


Figura 4.4 ACF e PACF de um AR(1) e de um MA(1) simulados, com 1 000 observações cada. No AR(1) a ACF decai geometricamente e a PACF tem um único valor fora da faixa cinza. No MA(1) os papéis se invertem: a ACF corta depois da primeira defasagem e a PACF decai alternando o sinal.

Tabela 4.1 Assinaturas dos processos estacionários na ACF e na PACF

Processo	ACF	PACF
Ruído branco	nula em todas as defasagens	nula em todas as defasagens
AR(p)	decai gradualmente (exponencial ou senoide amortecida)	corta: nula depois da defasagem p
MA(q)	corta: nula depois da defasagem q	decai gradualmente
ARMA(p, q)	decai gradualmente	decai gradualmente

4.7 ARIMA e o roteiro de Box-Jenkins

Um modelo **ARIMA**(p, d, q) é um ARMA(p, q) aplicado à série diferenciada d vezes. A letra I vem de "integrado". Um ARIMA(1, 1, 0) no log do preço é o mesmo modelo que um AR(1) nos log-retornos, e é assim, com $d = 0$ sobre retornos, que os ajustes abaixo estão escritos.

Box et al. (2015) organizaram a construção desses modelos em um ciclo de três etapas que continua sendo o procedimento padrão.

1. **Identificação.** Decida d com o gráfico da série e os testes da Seção 4.4. Com a série estacionária em mãos, use a ACF e a PACF para propor valores pequenos de p e q.
2. **Estimação.** Ajuste os candidatos por máxima verossimilhança e compare-os por um critério de informação, que premia o ajuste e pune o número k de parâmetros: $\mathrm{AIC} = -2\ln L + 2k$ e $\mathrm{BIC} = -2\ln L + k\ln T$, em que L é a verossimilhança maximizada. Quanto menor, melhor. O BIC pune mais e escolhe modelos menores.
3. **Diagnóstico.** Se o modelo capturou a dependência linear, os resíduos devem parecer ruído branco. Examine a ACF dos resíduos e aplique o teste

de Ljung-Box. Se houver estrutura, volte à primeira etapa.

O Código 4.3 percorre o ciclo com os log-retornos diários do Ibovespa de 2010 a 2021, em porcentagem. Os anos de 2022 a 2025 ficam guardados para a próxima subseção.

Código 4.3 Estimação e diagnóstico de modelos ARMA para os retornos do Ibovespa (`cap04/l04_box_jenkins.py`)

```
import numpy as np
import pandas as pd
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.arima.model import ARIMA
precos = pd.read_csv("../../dados/precos_ajustados.csv",
                     index_col=0, parse_dates=True)
ibov = precos["^BVSP"].dropna()
r = 100 * np.log(ibov).diff().dropna()      # log-retornos em %
treino = r[:"2021"].to_numpy()              # 2010 a 2021
print("ordem          AIC      BIC   p (Ljung-Box)")
ordens = [(0, 0, 0), (1, 0, 0), (0, 0, 1), (1, 0, 1), (2, 0, 2)]
for ordem in ordens:
    m = ARIMA(treino, order=ordem).fit()
    # diagnóstico: resta autocorrelação nos resíduos?
    lb = acorr_ljungbox(m.resid, lags=[10],
                        model_df=ordem[0] + ordem[2])
    print(f"{ordem}  {m.aic:.1f}  {m.bic:.1f}"
          f"   {lb.lb_pvalue[10]:.4f}")
ar1 = ARIMA(treino, order=(1, 0, 0)).fit(cov_type="robust")
print(f"AR(1): phi = {ar1.params[1]:.4f}"
      f"   erro-padrão robusto = {ar1.bse[1]:.4f}")
lb2 = acorr_ljungbox(ar1.resid**2, lags=[10])
print(f"Ljung-Box dos resíduos ao quadrado: "
      f"Q = {lb2.lb_stat[10]:.0f}")
```

```
ordem          AIC      BIC   p (Ljung-Box)
(0, 0, 0)  11106.4  11118.4   0.0000
(1, 0, 0)  11086.0  11104.0   0.0000
(0, 0, 1)  11087.7  11105.7   0.0000
(1, 0, 1)  11085.0  11109.0   0.0000
(2, 0, 2)  11062.6  11098.5   0.2886
AR(1): phi = -0.0867   erro-padrão robusto = 0.0486
Ljung-Box dos resíduos ao quadrado: Q = 2757
```

Pelo manual, o resultado é claro. O modelo sem dinâmica nenhuma, $(0, 0, 0)$, tem os piores critérios e resíduos autocorrelacionados. O ARMA$(2, 2)$ vence no AIC e no BIC e é o único cujos resíduos passam no Ljung-Box com dez defasagens. Há, porém, dois sinais de alerta na saída. O coeficiente do AR(1) é pequeno, $-0{,}087$, e seu erro-padrão robusto a heterocedasticidade é $0{,}049$: a estimativa está a menos de dois erros-padrão de zero. E a estatística de Ljung-Box dos resíduos ao quadrado é enorme, o que indica que a variância dos resíduos não é constante. As duas observações estão ligadas, como se verá.

O argumento `model_df` desconta dos graus de liberdade do teste os $p + q$ coeficientes estimados, como pede o teste quando aplicado a resíduos.

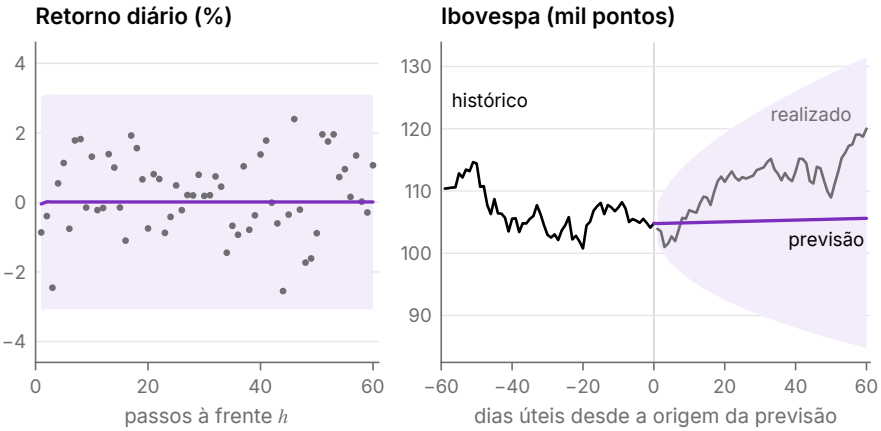


Figura 4.5 Previsão do AR(1) estimado até 30 de dezembro de 2021 para os 60 pregões seguintes, com intervalo de 95%. À esquerda, o retorno diário: a previsão é praticamente a média, e os pontos são os retornos realizados. À direita, o nível do índice implicado pelo mesmo modelo: a incerteza se acumula e o intervalo se abre aproximadamente com $\sqrt{h}$.

4.7.1 Previsão e a prova fora da amostra

A previsão de um modelo ARMA para h passos à frente é a esperança condicional de x_{T+h} dado o histórico. No AR(1) ela tem forma fechada:

$$\hat{x}_{T+h} = \mu + \phi^h (x_T - \mu), \qquad \operatorname{Var}(x_{T+h} - \hat{x}_{T+h}) = \sigma^2 \frac{1 - \phi^{2h}}{1 - \phi^2}. \tag{4.8}$$

A previsão parte do último valor observado e converge para a média μ à taxa ϕ^h; a variância do erro cresce até a variância incondicional do processo. Com $\phi = -0{,}087$ a convergência é quase imediata. No fim de 2021 o último retorno observado foi de 0,68%; o modelo prevê $-0{,}04\%$ para o dia seguinte, 0,02% para o segundo dia e, do terceiro em diante, a média de 0,01%. O intervalo de 95% tem meia-largura de 3,07 pontos percentuais já no primeiro passo, contra 3,09 da distribuição incondicional (Figura 4.5, à esquerda). A previsão condicional quase não difere da incondicional.

Para o nível do índice a figura da direita mostra o outro lado da raiz unitária: os erros de previsão dos retornos se somam, e o intervalo se alarga sem limite. Em 60 pregões, partindo de 104 822 pontos, o intervalo de 95% vai de 84 807 a 131 531 pontos. O índice fechou esse período em 120 014, dentro de um intervalo largo demais para orientar qualquer decisão.

Falta a pergunta que importa: o modelo prevê melhor do que não usar modelo? O Código 4.4 fixa os parâmetros estimados até 2021 e produz, para cada um dos 999 pregões de 2022 a 2025, a previsão de um passo à frente usando os retornos observados até a véspera. As referências são duas previsões ingênuas: retorno zero e a média do período de treino.

Código 4.4 Previsão um passo à frente fora da amostra (`cap04/l05_fora_da_amostra.py`; a leitura dos dados é a do Código 4.3)

```
treino = r[:"2021"].to_numpy()              # 2010 a 2021
teste = r["2022":].to_numpy()               # 2022 a 2025
def rmse(previsao):
    return np.sqrt(np.mean((teste - previsao) ** 2))
media = treino.mean()
e2_media = (teste - media) ** 2             # erro² da ingênua
sinal = np.mean(np.sign(media) == np.sign(teste))
print(f"treino: {len(treino)} dias   teste: {len(teste)} dias")
```

```
print(f"previsão zero:    RMSE = {rmse(0.0):.4f}")
print(f"média do treino:  RMSE = {rmse(media):.4f}"
      f"   acerto de sinal = {sinal:.1%}")
for ordem in [(1, 0, 0), (2, 0, 2)]:
    m = ARIMA(treino, order=ordem).fit()
    # parâmetros fixos do treino; a cada dia entra o dado novo
    m = m.append(teste, refit=False)
    prev = m.predict(start=len(treino))    # um passo à frente
    acerto = np.mean(np.sign(prev) == np.sign(teste))
    # t da diferença média de erro² contra a média do treino
    d = (teste - prev) ** 2 - e2_media
    t = d.mean() / (d.std(ddof=1) / np.sqrt(len(d)))
    print(f"ARIMA{ordem}:   RMSE = {rmse(prev):.4f}"
          f"   acerto de sinal = {acerto:.1%}   t = {t:.2f}")
# de onde vinha a autocorrelação do treino?
s = r[:"2021"]
sem2020 = pd.concat([s[:"2019"], s["2021":]])
print(f"autocorrelação de ordem 1 no treino: {s.autocorr(1):.3f}")
print(f"  só 2020: {s['2020'].autocorr(1):.3f}"
      f"   treino sem 2020: {sem2020.autocorr(1):.3f}"
      f"   teste: {r['2022':].autocorr(1):.3f}")
```

```
treino: 2964 dias   teste: 999 dias
previsão zero:    RMSE = 1.0632
média do treino:  RMSE = 1.0627   acerto de sinal = 51.4%
ARIMA(1, 0, 0):   RMSE = 1.0695   acerto de sinal = 47.9%   t = 2.18
ARIMA(2, 0, 2):   RMSE = 1.0763   acerto de sinal = 49.3%   t = 3.53
autocorrelação de ordem 1 no treino: -0.087
  só 2020: -0.262   treino sem 2020: -0.020   teste: 0.030
```

A raiz do erro quadrático médio (RMSE) dos dois modelos é maior que a das previsões ingênuas, e o ARMA(2, 2), campeão dos critérios de informação e do diagnóstico, é o pior dos quatro. As diferenças são pequenas, de 0,6% e 1,3% do RMSE da média do treino, mas sistemáticas: a coluna t traz a estatística t simples da diferença média entre os erros quadráticos de cada modelo e os da média do treino, 2,18 e 3,53, ambas acima de 2. Leia esses valores como indicação, e não como p-valor exato, porque diferenças de erros quadráticos têm cauda pesada e poucos dias extremos pesam muito na média. No sinal, a referência correta não é 50%. Prever sempre o sinal da média do treino, que é positiva, acerta 51,4% dos dias de teste, e os dois modelos ficam abaixo disso. As duas últimas linhas explicam o que houve. A autocorrelação negativa do período de treino vinha quase toda de 2020, quando quedas e repiques violentos se alternaram em dias seguidos: nesse ano ela foi de $-0{,}26$; no treino sem 2020, de $-0{,}02$; no período de teste, de $+0{,}03$. A dependência que os modelos estimaram pertencia a um episódio e não se repetiu fora dele.

Armadilha **Ajuste dentro da amostra não é capacidade de previsão**

AIC, BIC e Ljung-Box são calculados nos mesmos dados usados para estimar. Eles comparam modelos entre si e não dizem se algum deles prevê. Antes de acreditar em um modelo de previsão, separe um período que ele nunca viu e compare-o com a alternativa mais simples possível. O Capítulo 7 transforma essa regra em método.

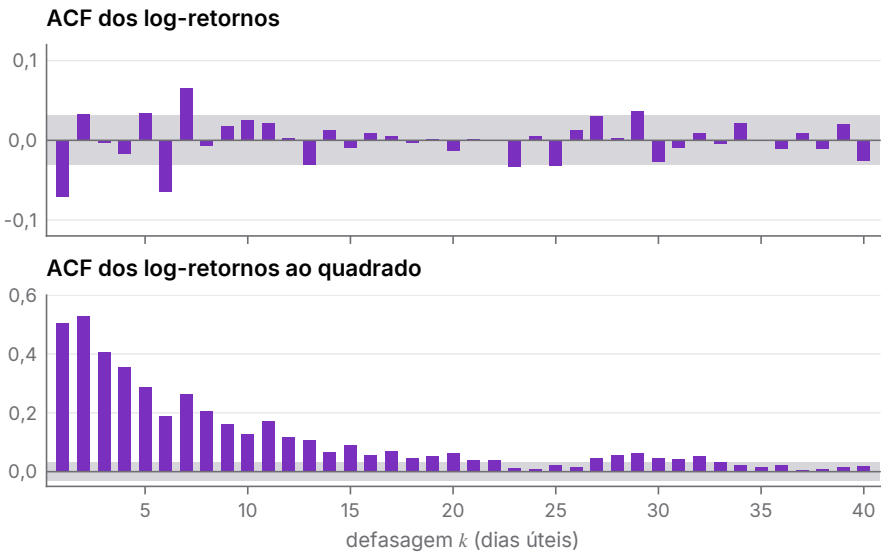


Figura 4.6 ACF dos log-retornos diários do Ibovespa de 2010 a 2025 e dos mesmos retornos ao quadrado. As escalas verticais são diferentes: nos retornos nenhuma barra passa de 0,1 em módulo; nos quadrados a autocorrelação começa em 0,5 e leva cerca de um mês para se aproximar da faixa.

Esse resultado, obtido aqui com um índice, um período de teste e parâmetros fixos, não é uma particularidade da amostra: a ausência de autocorrelação linear relevante em retornos de ativos líquidos é um dos fatos estilizados catalogados por Cont (2001), e Campbell et al. (1997) dedicam um capítulo inteiro às formas de testá-la. Faz sentido econômico: uma previsibilidade grande e estável da média seria explorada até desaparecer. Os modelos ARMA continuam úteis para séries com dependência genuína, como taxas de juros, inflação, volumes e *spreads* entre ativos, e são o ponto de partida para o que os retornos de fato têm de previsível.

No repositório

Os scripts `aula4/precos.py` e `aula4/retorno.py` aplicam o `adfuller` a preços e retornos baixados na hora com o `yfinance`, de modo que os números mudam conforme a data da execução; o `aula4/testeadf.py` usa uma série de 20 pontos, boa para ver a mecânica da função e curta demais para o teste ter poder. O notebook `aula4/ACF e PACF.ipynb` traça a ACF e a PACF de preços, retornos e retornos ao quadrado de PETR4 em frequência mensal e semanal. O script `aula4/arima.py` reestima um ARIMA(1, 0, 1) a cada dia e converte as previsões de retorno em preços encadeando-as a partir do preço inicial; ao adaptá-lo, avalie o erro nos retornos e inclua a previsão ingênua na comparação, como no Código 4.4.

4.8 Volatilidade que se agrupa

A Figura 4.6 repete a análise de autocorrelação em toda a amostra do Ibovespa, 3 963 retornos, duas vezes: nos retornos e nos retornos ao quadrado. O quadrado descarta o sinal e mede o tamanho do movimento.

Nos retornos, as autocorrelações são pequenas: a maior em módulo é a da primeira defasagem, $-0{,}071$. Nos quadrados elas são grandes, positivas e persistentes: 0,51 na primeira defasagem e 0,29 na quinta. O tamanho do retorno de hoje diz bastante sobre o tamanho do de amanhã, embora nada diga sobre o sinal. Mandelbrot (1963) já havia observado que variações grandes

tendem a ser seguidas por variações grandes, de qualquer sinal, e variações pequenas por variações pequenas. O fenômeno é chamado de **agrupamento de volatilidade** (*volatility clustering*).

Ele também resolve a pendência da seção anterior. Oito das 40 barras do painel de cima saem da faixa, e a primeira tem estatística t de $-4{,}5$ pela conta usual. Mas a faixa da Equação (4.5) supõe observações independentes, e retornos com volatilidade agrupada não são independentes, ainda que não sejam correlacionados. Com erro-padrão robusto a heterocedasticidade, a estatística t da primeira autocorrelação cai para $-1{,}66$. E em 2 000 séries simuladas de um GARCH(1,1) sem autocorrelação alguma, com os parâmetros estimados adiante, a primeira barra sai da faixa em 10,2% dos casos, o dobro dos 5% nominais. Os números são do script `cap04/fig_acf_ibov.py`.

4.8.1 De ARCH a GARCH

A ideia de Engle (1982) foi modelar a variância *condicional*, a variância do retorno de amanhã dada a informação de hoje, como função dos choques recentes. Escreva o retorno como

$$r_t = \mu + \varepsilon_t, \qquad \varepsilon_t = \sigma_t \, z_t, \tag{4.9}$$

em que z_t é uma sequência independente e identicamente distribuída, de média zero e variância um, e σ_t^2 é a variância de ε_t condicional ao passado. No modelo **ARCH**(q) de Engle, $\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \cdots + \alpha_q \varepsilon_{t-q}^2$: a variância de hoje é alta se os choques recentes foram grandes. Para reproduzir a persistência vista na Figura 4.6 são necessárias muitas defasagens. Bollerslev (1986) resolveu isso acrescentando a própria variância defasada, no modelo que se tornou o padrão da área.

Definição 4.4 GARCH(1,1)

No modelo **GARCH**(1,1), a variância condicional do choque ε_t da Equação (4.9) segue

$$\sigma_t^2 = \omega + \alpha \, \varepsilon_{t-1}^2 + \beta \, \sigma_{t-1}^2, \qquad \omega > 0, \quad \alpha \geq 0, \quad \beta \geq 0. \tag{4.10}$$

As restrições de sinal garantem que a variância seja sempre positiva. A soma $\alpha + \beta$ é a **persistência**, e dela dependem as três propriedades que se usam na prática.

Resultado 4.5 Longo prazo, previsão e meia-vida no GARCH(1,1)

Se $\alpha + \beta < 1$, o choque ε_t tem variância incondicional finita,

$$\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \beta}, \tag{4.11}$$

e a previsão, feita em t, da variância de h dias à frente é

$$\mathbb{E}_t\big[\sigma_{t+h}^2\big] = \bar{\sigma}^2 + (\alpha + \beta)^{h-1} \left(\sigma_{t+1}^2 - \bar{\sigma}^2\right). \tag{4.12}$$

A distância até $\bar{\sigma}^2$ cai pela metade a cada $\ln 0{,}5 / \ln(\alpha + \beta)$ dias, a **meia-vida** de um choque de volatilidade.

A justificativa usa um fato só: como $\mathbb{E}_{t-1}[\varepsilon_t^2] = \sigma_t^2$, o valor esperado de ε_{t+1}^2 visto de hoje é σ_{t+1}^2. Aplicando a esperança incondicional à Equação (4.10),

$\bar{\sigma}^2 = \omega + (\alpha + \beta)\,\bar{\sigma}^2$, o que dá (4.11). Aplicando a esperança condicional um passo de cada vez, $\mathbb{E}_t[\sigma^2_{t+2}] = \omega + (\alpha+\beta)\,\sigma^2_{t+1}$; subtraindo $\bar{\sigma}^2$ dos dois lados e repetindo, chega-se a (4.12). Note que σ^2_{t+1} não é uma previsão: ele é conhecido no fim do dia t, porque só depende de ε_t e de σ_t.

Intuição

Substitua $\omega = (1-\alpha-\beta)\,\bar{\sigma}^2$ na Equação (4.10). A variância de amanhã é uma média ponderada de três coisas: o nível de longo prazo, com peso $1-\alpha-\beta$; a surpresa de hoje ao quadrado, com peso α; e a variância que valia hoje, com peso β. O parâmetro α mede a reação a notícias, e β, a memória.

Esse processo é fracamente estacionário: a variância *incondicional* é a constante $\bar{\sigma}^2$. O que varia é a variância condicional. É a distinção anunciada na Seção 4.4, e ela explica também as caudas pesadas do Capítulo 3: uma mistura de dias de variância baixa e dias de variância alta tem curtose maior que 3 mesmo quando cada dia, dado o passado, é normal (Tsay, 2010).

4.9 GARCH na prática

Os parâmetros são estimados por máxima verossimilhança. Supondo z_t normal, cada retorno contribui com $-\frac{1}{2}\left[\ln(2\pi) + \ln\sigma_t^2 + \varepsilon_t^2/\sigma_t^2\right]$ para o logaritmo da verossimilhança, e σ_t^2 é calculado recursivamente pela Equação (4.10). A biblioteca `arch` faz esse trabalho.

Código 4.5 GARCH(1,1) para o Ibovespa, 2010 a 2025 (`cap04/l06_garch.py`)

```
import numpy as np
import pandas as pd
from arch import arch_model
precos = pd.read_csv("../../dados/precos_ajustados.csv",
                     index_col=0, parse_dates=True)
ibov = precos["^BVSP"].dropna()
r = 100 * np.log(ibov).diff().dropna()     # retornos em %
modelo = arch_model(r, mean="Constant", vol="GARCH", p=1, q=1,
                    dist="normal")
res = modelo.fit(disp="off")
print(f"mu estimado: {res.params['mu']:.4f}%"
      f"   média amostral: {r.mean():.4f}%")
print(res.summary().tables[2])             # equação da variância
omega, alfa, beta = res.params[1:]         # depois de mu
persistencia = alfa + beta
var_inc = omega / (1 - persistencia)       # de longo prazo
meia_vida = np.log(0.5) / np.log(persistencia)
print(f"persistência alfa + beta: {persistencia:.4f}")
print(f"vol. incondicional: {np.sqrt(var_inc):.3f}% ao dia, "
      f"{np.sqrt(252 * var_inc):.1f}% ao ano")
print(f"desvio-padrão amostral: {r.std():.3f}% ao dia")
print(f"meia-vida da variância: {meia_vida:.1f} dias úteis")
vol = res.conditional_volatility * np.sqrt(252)
print(f"vol. condicional anualizada: mínima {vol.min():.1f}%, "
      f"máxima {vol.max():.1f}% em {vol.idxmax():%d/%m/%Y}")
```

```
mu estimado: 0.0414%   média amostral: 0.0210%
                             Volatility Model
============================================================================
                 coef    std err          t      P>|t|      95.0% Conf. Int.
----------------------------------------------------------------------------
omega          0.0488  1.313e-02      3.718  2.012e-04 [2.307e-02,7.454e-02]
alpha[1]       0.0749  1.199e-02      6.246  4.201e-10 [5.139e-02,9.838e-02]
beta[1]        0.9003  1.501e-02     59.980      0.000     [  0.871,  0.930]
============================================================================
persistência alfa + beta: 0.9752
vol. incondicional: 1.403% ao dia, 22.3% ao ano
desvio-padrão amostral: 1.463% ao dia
meia-vida da variância: 27.6 dias úteis
vol. condicional anualizada: mínima 13.0%, máxima 123.9% em 17/03/2020
```

Armadilha **Retornos em fração no `arch`**

Repare no fator 100 na criação de `r`. Com retornos em fração, as variâncias diárias são da ordem de 10^{-4}, o otimizador numérico trabalha mal e a biblioteca emite um aviso recomendando reescalar a série por 100. Estime sempre com retornos em porcentagem e lembre que, nessa escala, ω está em pontos percentuais ao quadrado.

A primeira linha traz o μ estimado, 0,041% ao dia, quase o dobro da média amostral da mesma série, 0,021%. Não leia esse número como uma estimativa melhor do retorno esperado. No GARCH o μ também entra na equação da variância, porque define o choque $\varepsilon_{t-1} = r_{t-1} - \mu$, e a estimativa depende da forma dessa equação: o Código 4.6, adiante, mostra que ela vai de 0,019% a 0,049% conforme a especificação. A tabela, extraída de `res.summary()`, traz os três parâmetros da Equação (4.10) com erros-padrão, estatísticas t, p-valores e intervalos de confiança; os erros-padrão são calculados, por padrão, de forma robusta a erros na hipótese de normalidade. A leitura é a seguinte. A variância de amanhã dá peso 0,90 à variância de hoje, 0,075 ao choque de hoje ao quadrado e os 0,025 restantes ao nível de longo prazo. A persistência é 0,975, abaixo de 1, e a volatilidade de longo prazo implícita, 1,40% ao dia ou 22,3% ao ano pela convenção de 252 dias (Seção 1.9), fica próxima do desvio-padrão amostral de 1,46%. A distância da variância condicional até o nível de longo prazo cai pela metade a cada 27,6 pregões, pouco mais de um mês. Em unidades de volatilidade, depois de um choque grande, a queda é mais lenta: partindo do pico de 17 de março de 2020, a distância da volatilidade até os 22,3% só cai à metade 45 pregões depois (script `cap04/fig_prev_vol.py`).

Exemplo 4.2 Um passo da recursão, à mão

Suponha que a volatilidade condicional de hoje seja $\sigma_t = 1{,}2\%$ ao dia, cerca de 19% ao ano, e que o índice caia 4 pontos percentuais além da média: $\varepsilon_t = -4$. Com os parâmetros estimados,

$$\begin{aligned}\sigma_{t+1}^2 &= 0{,}0488 + 0{,}0749 \times 4^2 + 0{,}9003 \times 1{,}2^2 \\ &= 0{,}0488 + 1{,}1984 + 1{,}2964 = 2{,}5436,\end{aligned}$$

e $\sigma_{t+1} = \sqrt{2{,}5436} = 1{,}59\%$ ao dia, ou $1{,}59 \times \sqrt{252} = 25{,}3\%$ ao ano. Um único dia ruim elevou a volatilidade prevista em um terço. Um choque de +4 teria exatamente o mesmo efeito, porque o modelo só enxerga ε_t^2.

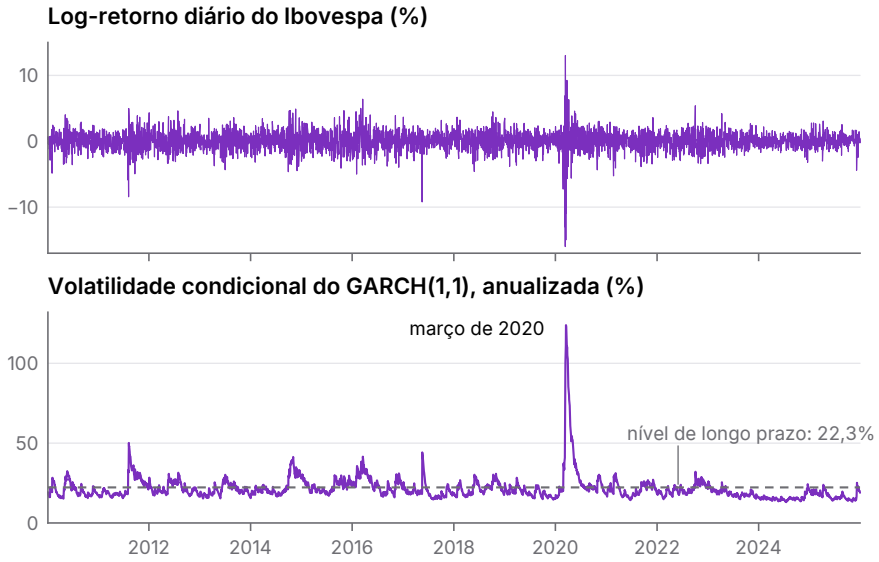


Figura 4.7 Retornos diários do Ibovespa e a volatilidade condicional estimada pelo GARCH(1,1), anualizada por $\sqrt{252}$. A volatilidade passa a maior parte do tempo abaixo do nível de longo prazo e dá saltos nos episódios de estresse, dos quais retorna aos poucos. O pico, de 124% ao ano, ocorre em 17 de março de 2020.

O atributo `conditional_volatility` do resultado contém σ_t para cada dia da amostra, e é o que a Figura 4.7 mostra. A volatilidade condicional do Ibovespa variou de 13% a 124% ao ano no período. Resumir esses dezesseis anos em um único número, os 22% de longo prazo, superestima o risco na maior parte dos dias, já que a volatilidade condicional ficou abaixo do nível de longo prazo em 69% deles, e o subestima muito justamente quando ele importa.

4.9.1 Os resíduos padronizados e a distribuição dos choques

Se o modelo está certo, os **resíduos padronizados** $\hat{z}_t = \hat{\varepsilon}_t / \hat{\sigma}_t$ devem se comportar como a sequência z_t suposta: sem agrupamento de volatilidade e com a distribuição escolhida. O Código 4.6 examina a segunda parte e compara quatro especificações.

Código 4.6 Quatro especificações e os resíduos padronizados (`cap04/l07_garch_t.py`; `r` é a série do Código 4.5)

```
especs = {
    "GARCH normal": dict(p=1, q=1, dist="normal"),
    "GARCH t     ": dict(p=1, q=1, dist="t"),
    "GJR normal  ": dict(p=1, o=1, q=1, dist="normal"),
    "GJR t       ": dict(p=1, o=1, q=1, dist="t"),
}
print("modelo        log-veros.      AIC      BIC      mu")
ajustes = {}
for nome, kw in especs.items():
    m = arch_model(r, vol="GARCH", **kw).fit(disp="off")
    ajustes[nome] = m
    print(f"{nome}  {m.loglikelihood:10.1f}"
          f"  {m.aic:.1f}  {m.bic:.1f}  {m.params['mu']:.4f}")
z = ajustes["GARCH normal"].std_resid      # resíduos padronizados
k_r = stats.kurtosis(r, fisher=False)
k_z = stats.kurtosis(z, fisher=False)
```

```
print(f"curtose: retornos {k_r:.2f}"
      f"   resíduos padronizados {k_z:.2f}")
print(f"|z| > 3: {np.mean(abs(z) > 3):.2%} dos dias"
      f"   (normal: {2 * stats.norm.sf(3):.2%})")
print(f"graus de liberdade da t: "
      f"{ajustes['GARCH t     '].params['nu']:.1f}")
gjr = ajustes["GJR t       "].params
print(f"GJR t: alfa {gjr['alpha[1]']:.3f}"
      f"   gama {gjr['gamma[1]']:.3f}"
      f"   beta {gjr['beta[1]']:.3f}")
```

```
modelo        log-veros.      AIC      BIC      mu
GARCH normal     -6676.3  13360.7  13385.8  0.0414
GARCH t          -6618.8  13247.5  13279.0  0.0485
GJR normal       -6654.0  13318.0  13349.4  0.0189
GJR t            -6600.2  13212.4  13250.1  0.0314
curtose: retornos 15.08   resíduos padronizados 4.83
|z| > 3: 0.73% dos dias   (normal: 0.27%)
graus de liberdade da t: 9.2
GJR t: alfa 0.018   gama 0.082   beta 0.917
```

O GARCH explica boa parte das caudas pesadas, mas não todas. A curtose cai de 15,1 nos retornos para 4,8 nos resíduos padronizados, ainda acima dos 3 da normal, e resíduos além de três desvios aparecem em 0,73% dos dias, quase o triplo do que a normal prevê. Por isso Bollerslev (1987) propôs trocar a normal por uma t de Student padronizada, cujo número de graus de liberdade ν é estimado junto com os demais parâmetros (`dist="t"`). Para o Ibovespa, $\hat{\nu} = 9{,}2$, e o AIC e o BIC melhoram em mais de cem pontos. A Figura 4.8 mostra o ganho e o que resta: a cauda esquerda continua mais pesada que a direita.

Essa assimetria tem tratamento. Em ações, quedas costumam elevar a volatilidade mais do que altas da mesma magnitude (Cont, 2001). O modelo GJR-GARCH, de Glosten et al. (1993), dá coeficientes diferentes aos choques positivos e negativos. Na forma usual hoje na literatura e nos pacotes, acrescenta-se à Equação (4.10) o termo $\gamma\, \varepsilon_{t-1}^2\, I(\varepsilon_{t-1} < 0)$, em que $I(\cdot)$ vale 1 quando o choque é negativo e 0 caso contrário; no `arch`, basta `o=1`. Na última linha da saída, um choque positivo entra na variância do dia seguinte com peso 0,018 e um negativo com peso $0{,}018 + 0{,}082 = 0{,}100$, e o GJR com choques t tem os melhores critérios dos quatro. A coluna `mu` completa a observação feita sobre o Código 4.5: no GJR com choques normais o μ estimado é 0,019%, praticamente a média amostral, enquanto nos dois modelos simétricos ele fica em 0,041% e 0,049%. O deslocamento aparece quando a equação da variância não distingue quedas de altas e diminui quando ela distingue. O EGARCH de Nelson (1991) obtém efeito semelhante modelando o logaritmo da variância. Outro parente conhecido é a média móvel exponencial (EWMA) popularizada pelo RiskMetrics, $\sigma_t^2 = \lambda\, \sigma_{t-1}^2 + (1-\lambda)\, r_{t-1}^2$ com $\lambda = 0{,}94$ para dados diários (Hull, 2018). Ela é um GARCH(1,1) com $\omega = 0$, $\alpha = 1-\lambda$ e $\beta = \lambda$: a persistência é exatamente 1, não existe nível de longo prazo, e a previsão para qualquer horizonte é a volatilidade de hoje.

4.9.2 Previsão de volatilidade

A Equação (4.12) já diz como prever a variância de qualquer dia futuro a partir de σ_{t+1}^2. Na biblioteca, a conta é feita pelo método `forecast`, que devolve a variância prevista para cada um dos próximos h dias.

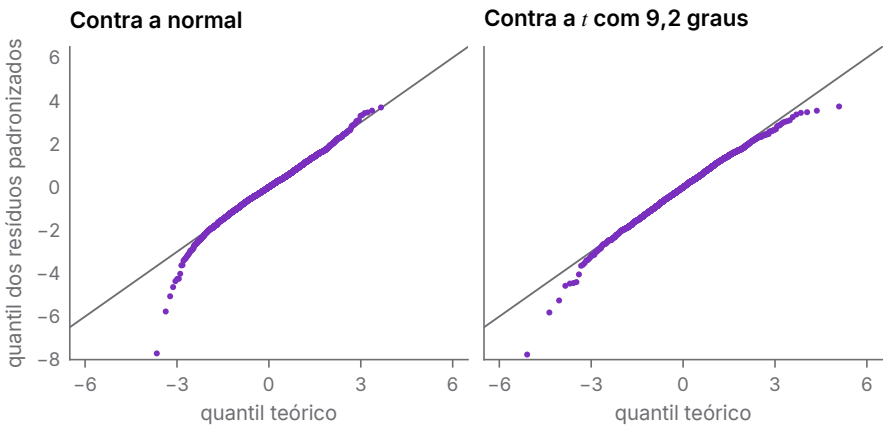


Figura 4.8 Quantis dos resíduos padronizados do GARCH(1,1) contra os quantis teóricos. Sob a normal, a cauda esquerda se afasta muito da reta. Com a t de Student o ajuste melhora, mas as piores quedas ainda ficam abaixo da reta. O ponto extremo é 18 de maio de 2017, quando o log-retorno do índice foi de $-9{,}2\%$ em um dia de volatilidade prevista de 1,2%.

Código 4.7 Previsão da variância para os próximos 252 pregões (trecho de `cap04/l08_previsao_vol.py`)

```
res = arch_model(r, vol="GARCH", p=1, q=1).fit(disp="off")
H = 252
prev = res.forecast(horizon=H).variance.iloc[-1].to_numpy()
```

```
maior diferença biblioteca x fórmula: 1.6e-15
origem: 30/12/2025   vol. de longo prazo: 1.403%
 h   vol. do dia h   vol. acumulada   raiz do tempo
 1        1.156%          1.16%          1.16%
 5        1.182%          2.61%          2.59%
21        1.260%          5.56%          5.30%
63        1.355%         10.19%          9.18%
252       1.403%         21.70%         18.36%
```

O restante do script, não impresso, refaz a conta pela Equação (4.12) e monta a tabela; a primeira linha da saída confirma que biblioteca e fórmula coincidem. No fim de 2025 o mercado estava calmo: a volatilidade prevista para o dia seguinte, 1,16%, estava abaixo do nível de longo prazo de 1,40%, e a previsão sobe em direção a ele conforme o horizonte aumenta. A coluna "vol. acumulada" soma as variâncias previstas dia a dia, o que dá a volatilidade do retorno de h dias se os retornos não forem autocorrelacionados. A última coluna aplica a regra da raiz do tempo do Capítulo 1 à volatilidade de amanhã. Para um ano, a regra dá 18,4% e o modelo, 21,7%. Em um dia de crise o erro teria o sinal contrário e seria muito maior.

A Figura 4.9 resume o comportamento do modelo. A volatilidade condicional é persistente: o desvio em relação ao nível de longo prazo dura semanas, e é essa persistência que a torna previsível. O retorno volta à média em um dia, e por isso sua previsão é praticamente a média. O Capítulo 5 leva a ideia ao VaR em versão simplificada. No backtest do VaR, ele compara a janela histórica de 250 dias, a normal com desvio-padrão da mesma janela e a normal com volatilidade EWMA de $\lambda = 0{,}94$, que reage ao estado do mercado de um dia para o outro; o GARCH com choques t fica como exercício daquele capítulo, e a simulação de vários dias de um GARCH é citada como alternativa à raiz do tempo.

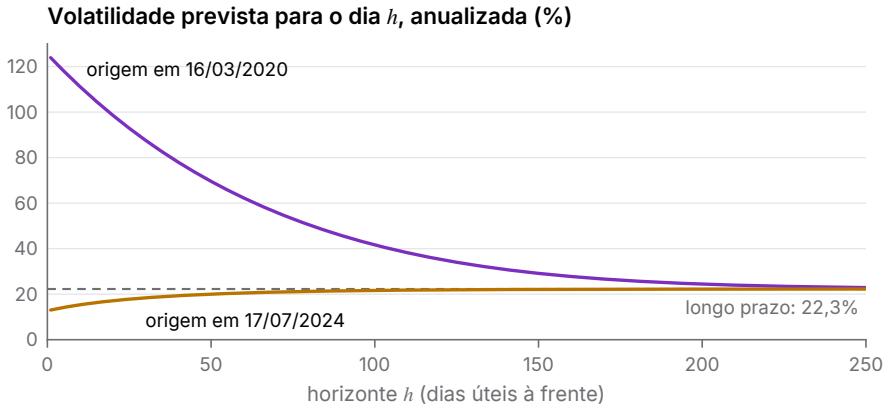


Figura 4.9 Volatilidade prevista pelo GARCH(1,1) a partir de duas origens: 16 de março de 2020, véspera do dia de maior volatilidade condicional da amostra, e 17 de julho de 2024, véspera do dia de menor. Como na Equação (4.12), $h = 1$ é o pregão seguinte à origem. As duas curvas convergem para o mesmo nível de longo prazo, e a distância da variância (não da volatilidade) até ele cai pela metade a cada 27,6 pregões.

No repositório

O script `aula4/garch.py` estima o mesmo GARCH(1,1) para PETR4 com dados do `yfinance`. Antes de rodar, amplie o intervalo do `yf.download`: como está, ele baixa apenas duas semanas de janeiro de 2023, e um GARCH precisa de alguns anos de retornos diários para que seus parâmetros sejam estimados com alguma precisão.

4.10 Entrega 1: análise exploratória

Este capítulo fecha o primeiro bloco do ciclo, e com ele vem a primeira entrega, prevista para o quarto encontro. O enunciado na proposta do Núcleo é: *notebook com coleta, tratamento e visualização de dados de pelo menos 5 ativos; estatísticas descritivas e correlação*. O notebook deve rodar de cima a baixo e conter, nesta ordem:

- **Coleta e tratamento** (Capítulo 2). Dados baixados uma vez e lidos de arquivo, com a fonte e a data da coleta declaradas; contagem de valores ausentes por ativo e a decisão tomada para cada caso, sem preencher retorno com zero.
- **Retornos** (Capítulo 1). Preço ajustado por proventos, retornos simples ou log-retornos conforme o uso, retorno e volatilidade anualizados de cada ativo.
- **Visualização** (Capítulo 2). Preços em base comum e escala logarítmica, retornos no tempo e histogramas, com eixos rotulados e uma frase de leitura por gráfico.
- **Estatísticas descritivas e correlação** (Capítulo 3). Média, desvio-padrão, assimetria e curtose; um teste de normalidade interpretado; matriz de correlação calculada com retornos e um gráfico de correlação móvel que mostre quanto ela varia.
- **Dependência no tempo** (este capítulo). ADF e KPSS em preços e em retornos de ao menos um ativo, e a ACF dos retornos e dos retornos ao quadrado.

A avaliação segue a rubrica de quatro dimensões da proposta (rigor técnico, clareza, reprodutibilidade e interpretação), detalhada no Capítulo 8. Nesta

entrega, interpretar significa dizer o que cada número implica para quem fosse investir nesses ativos e apontar o que a amostra não permite concluir.

O que levar deste capítulo

- Preços têm raiz unitária: choques são permanentes e não há nível ao qual o preço retorne. Log-retornos são a primeira diferença do log do preço e, em boa aproximação, estacionários. Estatística entre séries se faz com retornos, sob pena de regressão espúria.
- No ADF a hipótese nula é a raiz unitária e os valores críticos são próprios; no KPSS a nula é a estacionariedade. Usados juntos, um confere o outro; quando discordam, a amostra não decide.
- A ACF e a PACF indicam o tipo de memória linear de uma série. A faixa $\pm 1{,}96/\sqrt{T}$ supõe independência e fica estreita demais quando a volatilidade se agrupa.
- Um ARIMA bem ajustado dentro da amostra pode prever pior que a média histórica. Para retornos de ações, a previsibilidade da média é mínima, e um teste fora da amostra contra uma previsão ingênua deixa isso claro.
- A volatilidade é persistente e previsível. O GARCH(1,1) a descreve com três parâmetros, dos quais saem a persistência $\alpha + \beta$, a variância de longo prazo $\omega/(1-\alpha-\beta)$ e a previsão para qualquer horizonte.
- Estime o GARCH com retornos em porcentagem, confira os resíduos padronizados e considere choques com distribuição t e resposta assimétrica a quedas.

Exercícios

4.1 [básico] Para um AR(1) com $c = 0{,}2$, $\phi = 0{,}5$ e $\sigma = 2$, calcule a média, a variância e as três primeiras autocorrelações. Confira adaptando `cap04/l03_ar1.py`.

4.2 [básico] Um GARCH(1,1) estimado com retornos em porcentagem tem $\omega = 0{,}02$, $\alpha = 0{,}10$ e $\beta = 0{,}88$. Calcule a volatilidade de longo prazo diária e anualizada e a meia-vida da variância. Se $\sigma_t = 1\%$ e $\varepsilon_t = 3$, quanto vale σ_{t+1}?

4.3 [básico] Aplique o ADF e o KPSS ao log do preço e aos log-retornos de `VALE3.SA` e do dólar (a PTAX, coluna `ptax_venda` de `dados/c02_bcb.csv`), adaptando o Código 4.2. As conclusões são as mesmas do Ibovespa?

4.4 [intermediário] Refaça o Código 4.1 com $T = 100$ e $T = 2000$. A taxa de rejeição em níveis aumenta ou diminui com o tamanho da amostra? Relacione a resposta com a explicação da Seção 4.3.

4.5 [intermediário] Mostre que a ACF de um MA(1) na primeira defasagem, $\theta/(1+\theta^2)$, nunca passa de 0,5 em módulo. Que valor de θ atinge o máximo? O que você concluiria ao estimar $\hat{\rho}_1 = 0{,}7$ e $\hat{\rho}_2 \approx 0$ em uma série?

4.6 [intermediário] Estime um GARCH(1,1) com distribuição t para `PETR4.SA` e para `BOVA11.SA`. Compare persistência, volatilidade de longo prazo e graus de liberdade. Qual dos dois tem caudas mais pesadas depois de filtrada a volatilidade?

4.7 [intermediário] Aplique o teste de Ljung-Box aos resíduos padronizados do Código 4.5 e aos seus quadrados, com dez defasagens. O modelo eliminou a autocorrelação dos quadrados vista na Figura 4.6?

4.8 [desafio] Repita o Código 4.4 para a volatilidade. Estime o GARCH(1,1) até 2021, gere previsões de um passo para 2022 a 2025 com os parâmetros fixos e compare-as, usando r_t^2 como medida ruidosa da variância realizada, com duas alternativas:

a variância amostral do treino e o EWMA com $\lambda = 0{,}94$. Qual erra menos? Por que r_t^2 é uma referência tão ruidosa?

Para ir além

- Tsay (2010), capítulos 2 e 3, para modelos lineares e de volatilidade condicional com foco em retornos financeiros.
- Hamilton (1994), capítulos 3, 4 e 17, para a teoria de processos ARMA, previsão e raiz unitária com todas as demonstrações.
- Box et al. (2015), a edição atual do livro que criou o roteiro de identificação, estimação e diagnóstico.
- Campbell et al. (1997), capítulo 2, sobre os testes de previsibilidade de retornos e a hipótese de passeio aleatório.
- Engle (1982) e Bollerslev (1986), os artigos que criaram os modelos ARCH e GARCH.

Parte II

Risco e backtesting

Quanto se pode perder, como combinar ativos e como testar uma ideia no passado sem cair nos vieses que fazem qualquer estratégia parecer boa.



Capítulo 5

Risco de mercado: VaR, CVaR e drawdown

5

Uma carteira de ações com volatilidade de 25% ao ano é arriscada, mas esse número não diz ao gestor quantos reais ele pode perder amanhã, nem de que tamanho é o prejuízo nos dias realmente ruins, nem quanto tempo a carteira pode passar abaixo do seu melhor momento. Este capítulo apresenta as três medidas que respondem a essas perguntas, o VaR, o *expected shortfall* e o *drawdown*, e mostra como verificar se os números que elas produzem merecem confiança.

Ao terminar este capítulo você consegue

- definir o VaR como um quantil da distribuição de perdas e interpretá-lo sem exagerar o que ele diz;
- calcular o VaR de uma carteira pelos métodos paramétrico, histórico e de Monte Carlo, e explicar por que os resultados diferem;
- testar um modelo de VaR contando violações e aplicando o teste de Kupiec;
- calcular o *expected shortfall* e explicar por que ele substituiu o VaR na regulação bancária;
- construir a série de *drawdown* de uma carteira e extrair dela o drawdown máximo, a duração e o tempo de recuperação;
- reconhecer o tamanho do erro de estimação em qualquer medida de cauda.

5.1 Risco como perda em reais

A volatilidade, que você aprendeu a calcular no Capítulo 1 e a modelar no Capítulo 4, mede a dispersão dos retornos em torno da média. Ela tem duas limitações como medida de risco. Trata da mesma forma um desvio para cima e um desvio para baixo, embora ninguém considere um ganho inesperado um problema. E resume a distribuição inteira em um número que só descreve bem as caudas quando os retornos são normais, o que o Capítulo 3 mostrou não

ser o caso: duas carteiras com a mesma volatilidade podem ter piores dias de tamanhos muito diferentes.

As medidas deste capítulo olham diretamente para o lado das perdas e são expressas na unidade que interessa a quem responde pelo dinheiro. Para isso é preciso fixar uma convenção de sinal, que será mantida até o fim do capítulo.

Definição 5.1 Perda

Para uma carteira de valor V hoje e retorno simples R_p no horizonte considerado, a **perda** é

$$L = -VR_p. \tag{5.1}$$

Perdas são números positivos e ganhos são perdas negativas.

Todos os exemplos com dados usam a mesma carteira: R$ 1 milhão dividido igualmente entre PETR4, VALE3, ITUB4, WEGE3 e ABEV3. O Código 5.1 monta a série de perdas diárias que essa carteira teria sofrido em cada dia do histórico, e os demais códigos do capítulo a importam.

Código 5.1 A carteira do capítulo e sua série de perdas (`cap05/carteira.py`)

```
import numpy as np
import pandas as pd

ATIVOS = ["PETR4.SA", "VALE3.SA", "ITUB4.SA", "WEGE3.SA",
          "ABEV3.SA"]
V = 1_000_000                  # valor da carteira, em reais
w = np.full(5, 0.2)            # pesos iguais

precos = pd.read_csv("../../dados/precos_ajustados.csv",
                     index_col=0, parse_dates=True)
pregoes = precos[ATIVOS].dropna(how="all")
p = pregoes.dropna()           # datas com os cinco preços
R_todos = p.pct_change().dropna()

# depois de uma data sem preço o retorno cobre vários pregões:
# ele entra na cota, mas não é uma perda diária
ordem = pd.Series(np.arange(len(pregoes)), pregoes.index)
um_pregao = ordem[p.index].diff().dropna() == 1
cota = (1 + R_todos @ w).cumprod()   # valor de R$ 1 aplicado
R = R_todos[um_pregao]
Rp = R @ w                     # retorno simples da carteira
perda = -V * Rp                # perda diária em reais
```

```
2471 dias, de 02/01/2015 a 30/12/2024
perda média: R$ -905
desvio-padrão: R$ 16,026
maior perda: R$ 150,307 em 12/03/2020
retornos de mais de um pregão: 30 (3 de 2015 a 2024)
```

A saída, impressa pelo bloco final do arquivo, resume os dez anos de 2015 a 2024, que servem de amostra de estimação no restante do capítulo. A perda média é negativa porque a carteira ganhou, em média, R$ 905 por dia. A série de perdas diz quanto a carteira de *hoje*, com os pesos de hoje, perderia se o mercado repetisse cada dia do passado, que é o que interessa para medir o risco

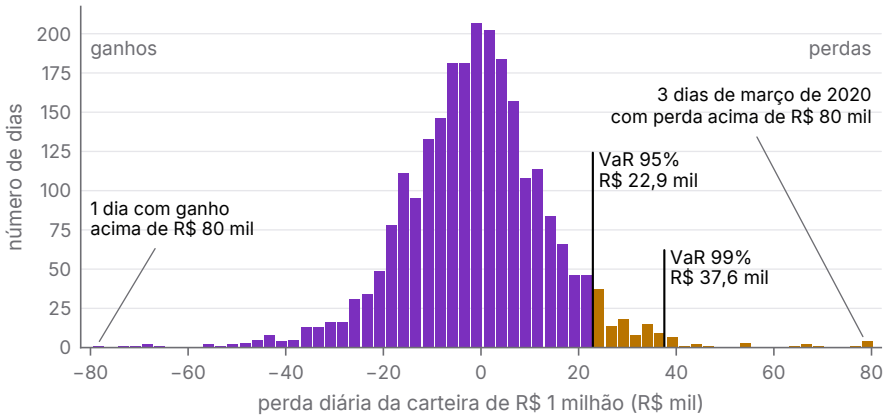


Figura 5.1 Perdas diárias da carteira de R$ 1 milhão entre 2015 e 2024. O VaR de 95% é o valor que separa os 5% piores dias (em ocre) dos demais. As barras das duas bordas reúnem os dias além de R$ 80 mil, de ganho ou de perda. O histograma mostra onde a cauda começa; o VaR nada diz sobre até onde ela vai.

de amanhã; ela é diferente do resultado de quem aplicou R$ 1 milhão em 2015 e deixou render.

A base de preços tem valores ausentes nos pregões em que um papel não teve negócio registrado. Essas datas saem, e o retorno seguinte cobre mais de um pregão: o de 17 de agosto de 2011 reúne 14 pregões, porque PETR4 ficou sem preço na base. Há 30 retornos assim de 2010 a 2025, quase todos em 2011 e 2012, e três entre 2015 e 2024. Eles entram na `cota`, usada no drawdown da Seção 5.9, que precisa de todo o caminho, e ficam fora da série de perdas diárias, do VaR e do backtest, em que apareceriam como perdas de um dia que não ocorreram.

5.2 Value at Risk: um quantil das perdas

A Figura 5.1 mostra o histograma das 2471 perdas diárias. Em 124 desses dias, 5% do total, a carteira perdeu mais de R$ 22,9 mil. Esse valor de corte é o *Value at Risk* de 95% da carteira para um dia, estimado da forma mais simples possível. Em 25 dias, 1% do total, a perda passou de R$ 37,6 mil: é o VaR de 99%.

Definição 5.2 Value at Risk

O **Value at Risk** de nível de confiança α é o quantil α da distribuição da perda L em um horizonte fixado:

$$\mathrm{VaR}_\alpha(L) = \inf\{\ell \in \mathbb{R} : P(L \le \ell) \ge \alpha\}. \tag{5.2}$$

Se L tem distribuição contínua, VaR_α é o valor que a perda supera com probabilidade exatamente $1-\alpha$.

A definição segue McNeil et al. (2015). Em termos de retornos, a Equação (5.1) dá $\mathrm{VaR}_\alpha(L) = -V q_{1-\alpha}(R_p)$, em que $q_{1-\alpha}$ é o quantil $1-\alpha$ dos retornos: o VaR de 95% é o quinto percentil dos retornos, com o sinal trocado e multiplicado pelo valor da carteira.

Um VaR só tem significado acompanhado de duas escolhas. O **nível de confiança** α costuma ser 95% ou 99%; quanto maior, mais para dentro da cauda se olha e, como se verá na Seção 5.10, menos dados sustentam a estimativa. O **horizonte** é o intervalo em que a perda é medida, normalmente um dia para

Muitos textos e os scripts da aula informam o VaR como um retorno negativo, por exemplo $-2{,}3\%$. É a mesma medida com outra convenção de sinal. Ao ler um relatório, descubra primeiro qual convenção ele usa.

mesas de operação e dez dias úteis na regulação bancária tradicional. A frase completa é sempre do tipo "o VaR de um dia a 99% é de R$ 37,6 mil".

Armadilha **O VaR não é a perda máxima**

O VaR de 99% não diz que a carteira perde no máximo R$ 37,6 mil. Diz que em um dia a cada cem, em média, ela perde *mais* do que isso, sem informar quanto mais. Na amostra da Figura 5.1 houve três dias com perda acima de R$ 130 mil, mais de três vezes o VaR de 99%.

O que muda de um método de cálculo para outro é a forma de obter a distribuição de L: supor uma família de distribuições, usar o histograma do passado ou simular.

5.3 VaR paramétrico

5.3.1 Sob normalidade

Se a perda tem distribuição normal de média μ_L e desvio-padrão σ_L, seus quantis são conhecidos e o VaR sai de uma conta.

Resultado 5.3 VaR normal

Se $L \sim N(\mu_L, \sigma_L^2)$, então

$$\mathrm{VaR}_\alpha(L) = \mu_L + \sigma_L\, z_\alpha, \tag{5.3}$$

em que z_α é o quantil α da normal padrão: $z_{0,95} = 1{,}645$ e $z_{0,99} = 2{,}326$.

Com $\mu_L = -V\mu_p$ e $\sigma_L = V\sigma_p$, a fórmula em retornos é $\mathrm{VaR}_\alpha = V(z_\alpha\sigma_p - \mu_p)$. Em horizonte de um dia a média é pequena diante do desvio-padrão (na nossa carteira, R$ 905 contra R$ 16 mil) e muito mal estimada, e por isso é prática comum fixá-la em zero.

Para uma carteira de vários ativos com vetor de pesos $\mathbf{w}$, vetor de médias $\boldsymbol{\mu}$ e matriz de covariância $\boldsymbol{\Sigma}$ dos retornos, a Equação (1.2) do Capítulo 1 implica

$$\mu_p = \mathbf{w}^\top \boldsymbol{\mu}, \qquad \sigma_p = \sqrt{\mathbf{w}^\top \boldsymbol{\Sigma}\, \mathbf{w}}. \tag{5.4}$$

A segunda expressão carrega todo o efeito da diversificação, e o Capítulo 6 a estuda em detalhe. Aqui basta um exemplo para ver o que ela faz com o VaR.

Exemplo 5.1 VaR de dois ativos, à mão

Uma carteira de R$ 100 mil tem 60% no ativo A, com volatilidade diária de 2%, e 40% no ativo B, com volatilidade diária de 3%. A correlação entre eles é 0,3 e as médias são tomadas como zero. A variância da carteira é

$$\sigma_p^2 = 0{,}6^2 \cdot 0{,}02^2 + 0{,}4^2 \cdot 0{,}03^2 + 2 \cdot 0{,}6 \cdot 0{,}4 \cdot 0{,}3 \cdot 0{,}02 \cdot 0{,}03 = 0{,}0003744,$$

de modo que $\sigma_p = 1{,}935\%$. O VaR de um dia a 95% é $1{,}645 \times 0{,}01935 \times 100\,000 =$ R$ 3 183.

Isoladas, a posição em A tem VaR de $1{,}645 \times 0{,}02 \times 60\,000 =$ R$ 1 974 e a posição em B, por coincidência, o mesmo valor: $1{,}645 \times 0{,}03 \times 40\,000 =$ R$ 1 974. A soma é R$ 3 948. Os R$ 765 de diferença são o ganho de diversificação, que só desapareceria se a correlação fosse igual a 1.

O Código 5.2 faz a mesma conta para a carteira de cinco ações, agora com a média estimada.

Código 5.2 VaR paramétrico normal a partir de $\mathbf{w}$ e $\boldsymbol{\Sigma}$ (cap05/l01_parametrico.py)

```
import numpy as np
from scipy import stats
from carteira import R, w, V

Ra = R["2015":"2024"]
mu = Ra.mean().values          # médias diárias dos 5 ativos
Sigma = Ra.cov().values        # matriz de covariância 5 x 5

mu_L = -V * (w @ mu)                    # média da perda
sig_L = V * np.sqrt(w @ Sigma @ w)      # desvio-padrão da perda
print(f"mu_L = {mu_L:,.0f}   sigma_L = {sig_L:,.0f}")

for a in (0.95, 0.99):
    z = stats.norm.ppf(a)
    print(f"VaR normal {a:.0%}: z = {z:.3f}  "
          f"R$ {mu_L + z * sig_L:,.0f}")

# VaR de cada posição isolada e ganho de diversificação
sig_i = V * w * np.sqrt(np.diag(Sigma))
z = stats.norm.ppf(0.99)
print("soma dos VaR 99% isolados (média zero): "
      f"R$ {z * sig_i.sum():,.0f}")
print(f"VaR 99% da carteira (média zero):       "
      f"R$ {z * sig_L:,.0f}")
```

```
mu_L = -905   sigma_L = 16,026
VaR normal 95%: z = 1.645  R$ 25,456
VaR normal 99%: z = 2.326  R$ 36,377
soma dos VaR 99% isolados (média zero): R$ 53,192
VaR 99% da carteira (média zero):       R$ 37,282
```

O desvio-padrão de R$ 16.026 obtido por $\mathbf{w}^{\top}\boldsymbol{\Sigma}\mathbf{w}$ é idêntico ao da série de perdas do Código 5.1, como tem de ser: são duas maneiras de calcular a mesma variância amostral. As duas últimas linhas mostram que a diversificação entre as cinco ações reduz o VaR em 30% em relação à soma das posições isoladas.

5.3.2 Com a t de Student

O VaR normal de 95% (R$ 25.456) ficou acima do valor de R$ 22,9 mil lido no histograma, e o de 99% (R$ 36.377) ficou abaixo dos R$ 37,6 mil. O desencontro tem a assinatura das caudas pesadas descritas no Capítulo 3: em relação à normal de mesma variância, os retornos reais têm mais massa perto do centro e nos extremos, e menos nos "ombros". Uma alternativa paramétrica que acomoda isso é a t de Student.

Uma perda com distribuição t de Student de ν graus de liberdade, locação m e escala s tem

$$\mathrm{VaR}_{\alpha}(L) = m + s\, t_{\nu,\alpha}, \tag{5.5}$$

em que $t_{\nu,\alpha}$ é o quantil α da t padrão. Há um cuidado de escala: s não é o desvio-padrão. Uma variável com distribuição t padrão e $\nu > 2$ graus de liberdade tem variância $\nu/(\nu-2)$, maior que 1, de modo que o desvio-padrão da perda é

$s\sqrt{\nu/(\nu-2)}$. Quem parte da média e do desvio-padrão amostrais precisa usar $m = \mu_L$ e $s = \sigma_L\sqrt{(\nu-2)/\nu}$.

Código 5.3 VaR com t de Student ajustada por máxima verossimilhança (`cap05/l02_student.py`)

```
import numpy as np
from scipy import stats
from carteira import perda

L = perda["2015":"2024"]
nu, m, s = stats.t.fit(L)      # máxima verossimilhança
print(f"nu = {nu:.2f}   m = {m:,.0f}   s = {s:,.0f}")
print(f"desvio-padrão implícito: "
      f"{s * np.sqrt(nu / (nu - 2)):,.0f}"
      f"   amostral: {L.std():,.0f}")

s_mom = L.std() * np.sqrt((nu - 2) / nu)   # escala pelos momentos
for a in (0.95, 0.99):
    q = stats.t.ppf(a, nu)
    print(f"VaR t {a:.0%}: quantil = {q:.3f}  "
          f"ajuste: R$ {m + s * q:,.0f}  "
          f"momentos: R$ {L.mean() + s_mom * q:,.0f}")

errado = m + L.std() * stats.t.ppf(0.99, nu)
print(f"VaR t 99% sem corrigir a escala: R$ {errado:,.0f}")
```

```
nu = 4.48   m = -957   s = 11,660
desvio-padrão implícito: 15,672   amostral: 16,026
VaR t 95%: quantil = 2.068  ajuste: R$ 23,156  momentos: R$ 23,752
VaR t 99%: quantil = 3.535  ajuste: R$ 40,258  momentos: R$ 41,240
VaR t 99% sem corrigir a escala: R$ 55,688
```

A função `stats.t.fit` estima de uma vez ν, m e s por máxima verossimilhança. O ajuste encontra $\nu = 4{,}48$, caudas bem mais pesadas que as da normal, e uma escala de R$ 11.660, que corresponde a um desvio-padrão de R$ 15.672, um pouco menor que o amostral. Por isso as duas colunas da saída diferem: "ajuste" aplica a Equação (5.5) com o m e o s estimados, e "momentos" usa o mesmo ν com $m = \mu_L$ e $s = \sigma_L\sqrt{(\nu-2)/\nu}$, o que dá um VaR cerca de 2,5% maior (R$ 41.240 contra R$ 40.258 a 99%). As duas são leituras legítimas do mesmo modelo, e a diferença entre elas é erro de estimação. A última linha mostra um erro de outra natureza, o de usar o desvio-padrão amostral diretamente como escala: o VaR de 99% sobe para R$ 56 mil, um exagero de quase 40%.

A Figura 5.2 explica os números. As duas curvas têm a mesma variância e se cruzam perto do nível de 97%. A 95% a hipótese normal é a mais conservadora das duas; a 99% ela subestima a perda, e a diferença cresce rapidamente depois disso. Os quantis empíricos da carteira ficam junto da curva da t, e abaixo da normal, até 97,5%, entre as duas a 99% e a 99,5%, e acima das duas a partir de 99,8%.

5.3.3 Mudança de horizonte e a raiz do tempo

Se os retornos diários forem independentes, com a mesma distribuição normal e média zero, a perda acumulada em h dias tem desvio-padrão $\sigma_L\sqrt{h}$ (é a regra

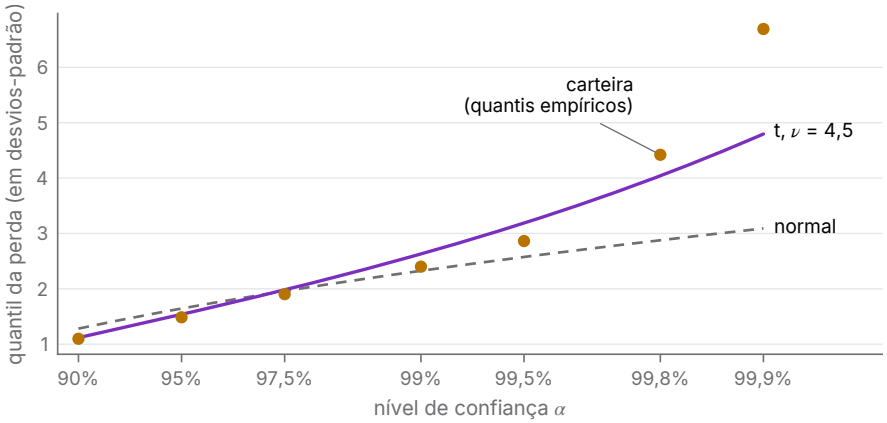


Figura 5.2 Quantis da perda, em desvios-padrão, em função do nível de confiança. A normal e a t de mesma variância se cruzam perto de 97%: abaixo desse nível a normal dá o VaR maior, acima dele a t dá o maior. Os pontos são os quantis empíricos da carteira; os dois últimos se apoiam em cinco e em duas ou três observações.

da raiz do tempo da Seção 1.9), e

$$\mathrm{VaR}_\alpha^{(h\ \text{dias})} = \sqrt{h}\ \mathrm{VaR}_\alpha^{(1\ \text{dia})}. \tag{5.6}$$

Para a carteira do capítulo, o VaR normal de dez dias a 99% com média zero seria $\sqrt{10} \times 37.282$, cerca de R$ 117,9 mil.

A regra é uma convenção de escala, e as hipóteses por trás dela falham. O script `cap05/horizonte.py` compara $\sqrt{h}$ vezes o VaR histórico de um dia com o quantil das perdas acumuladas em janelas sobrepostas de h dias. A 99%, a regra dá R$ 84,0 mil contra R$ 74,6 mil observados em cinco dias, R$ 118,8 mil contra R$ 121,0 mil em dez dias e R$ 172,1 mil contra R$ 198,7 mil em 21 dias; a 95% e dez dias, R$ 72,5 mil contra R$ 60,8 mil. Duas forças atuam em sentidos opostos. A média positiva e a autocorrelação levemente negativa dos retornos diários desta carteira ($-0{,}08$) contribuem para que a perda acumulada cresça menos que $\sqrt{h}$ no miolo da distribuição; o agrupamento de volatilidade (Capítulo 4) atua no sentido de engordar a cauda extrema. Leia os números de 99% com reserva: as janelas se sobrepõem, e 20 das 25 janelas de 21 dias acima do quantil terminam em março ou abril de 2020. A comparação nesse nível se apoia em um único episódio.

5.4 VaR histórico

O método histórico, ou de simulação histórica, não supõe família alguma de distribuições. Ele toma as perdas que a carteira atual teria sofrido em cada dia de uma janela do passado e usa o quantil empírico:

$$\widehat{\mathrm{VaR}}_\alpha = \text{quantil empírico de nível } \alpha \text{ de } \{L_1, \ldots, L_n\}. \tag{5.7}$$

Foi o que se fez na Figura 5.1.

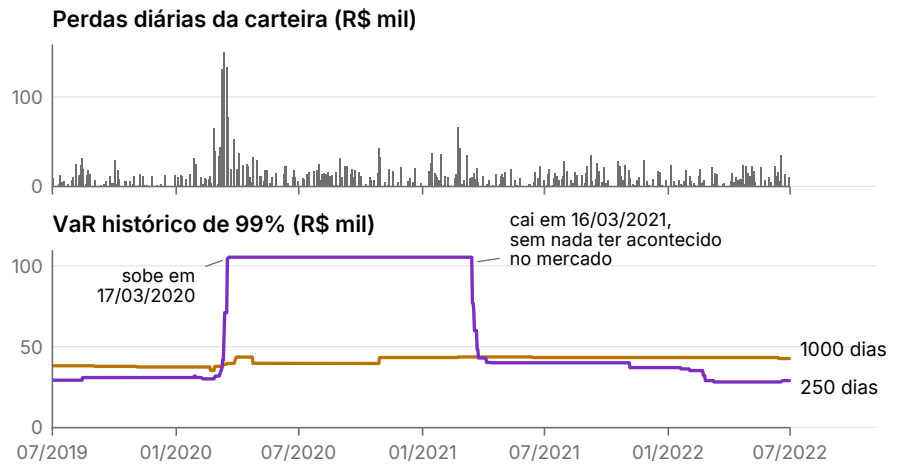


Figura 5.3 O efeito fantasma. Com janela de 250 dias, o VaR histórico de 99% triplica em março de 2020 e permanece no patamar novo por um ano, até despencar no dia em que as perdas daquele mês saem da janela. Com 1000 dias, o mesmo choque mal altera o quantil.

Código 5.4 VaR histórico e sua dependência da janela (`cap05/l03_historico.py`)

```python
import numpy as np
from carteira import perda

L = perda["2015":"2024"]
for a in (0.95, 0.99):
    print(f"VaR histórico {a:.0%}: "
          f"R$ {np.quantile(L, a):,.0f}")

# a resposta depende da janela
for ini, fim in (("2015", "2019"), ("2020", "2024"),
                 ("2024", "2024")):
    print(f"VaR histórico 99%, {ini} a {fim}: "
          f"R$ {np.quantile(perda[ini:fim], 0.99):,.0f}")
```

```
VaR histórico 95%: R$ 22,914
VaR histórico 99%: R$ 37,561
VaR histórico 99%, 2015 a 2019: R$ 37,390
VaR histórico 99%, 2020 a 2024: R$ 37,603
VaR histórico 99%, 2024 a 2024: R$ 20,168
```

Com 2471 observações, o quantil de 99% cai entre a 25ª e a 26ª maior perda, e `np.quantile` interpola entre as duas. A vantagem do método é evidente. Assimetria, caudas pesadas e a dependência entre os ativos nos dias ruins entram no cálculo sem que seja preciso modelá-las, porque já estão nos dados.

O preço é a dependência da janela, visível nas três últimas linhas da saída. Com apenas os 249 dias de 2024, um ano calmo, o VaR de 99% cai para R$ 20 mil, pouco mais da metade do valor obtido com dez anos. O método só conhece o que aconteceu dentro da janela e supõe, implicitamente, que o futuro próximo será sorteado da mesma distribuição. Uma janela curta reage depressa a mudanças de regime, mas apoia o quantil em duas ou três observações; uma janela longa dá mais observações, mas mistura regimes de volatilidade muito diferentes e reage devagar.

A Figura 5.3 mostra a consequência mais estranha dessa mecânica, conhecida como **efeito fantasma**. Com 250 observações, o quantil de 99% fica entre

a terceira e a quarta maior perda da janela. Em março de 2020 três perdas acima de R$ 130 mil entram na janela e o VaR salta de R$ 32 mil para R$ 105 mil. Ele fica nesse patamar por um ano, embora o mercado tenha se acalmado em poucos meses, e em 16 de março de 2021 cai de R$ 105 mil para R$ 77 mil de um dia para o outro, sem que nada tenha acontecido no mercado: a perda de 9 de março de 2020 deixou de estar entre os 250 pregões mais recentes e saiu da conta. A janela móvel dá o mesmo peso a todas as observações que contém e peso zero às demais, e essa descontinuidade produz o degrau.

5.5 VaR por simulação de Monte Carlo

O terceiro caminho consiste em escrever um modelo para os retornos dos ativos, sortear dele um grande número de cenários, calcular a perda da carteira em cada cenário e tomar o quantil empírico das perdas simuladas. É o método histórico aplicado a um passado artificial, do tamanho que se quiser.

Para sortear retornos de vários ativos é preciso reproduzir a covariância entre eles. O instrumento é a **decomposição de Cholesky**: toda matriz de covariância positiva definida pode ser escrita como $\boldsymbol{\Sigma} = \mathbf{C}\mathbf{C}^{\mathsf{T}}$, com $\mathbf{C}$ triangular inferior. Se $\mathbf{z}$ é um vetor de normais padrão independentes, então $\mathbf{x} = \mathbf{C}\mathbf{z}$ tem covariância

$$\operatorname{Cov}(\mathbf{C}\mathbf{z}) = \mathbf{C}\,\operatorname{Cov}(\mathbf{z})\,\mathbf{C}^{\mathsf{T}} = \mathbf{C}\mathbf{C}^{\mathsf{T}} = \boldsymbol{\Sigma}, \tag{5.8}$$

pois $\operatorname{Cov}(\mathbf{z})$ é a identidade. O Código 5.5 usa essa ideia para gerar 200 mil cenários de um dia.

Código 5.5 VaR por Monte Carlo com choques correlacionados (`cap05/l04_montecarlo.py`)

```python
import numpy as np
from carteira import R, w, V

Ra = R["2015":"2024"]
mu, Sigma = Ra.mean().values, Ra.cov().values

rng = np.random.default_rng(42)
N = 200_000
C = np.linalg.cholesky(Sigma)       # Sigma = C C^T
Z = rng.standard_normal((N, 5))     # normais independentes
X = Z @ C.T                         # covariância Sigma

L_normal = -V * ((mu + X) @ w)

# t multivariada com nu graus de liberdade e mesma covariância
nu = 4.48
g = rng.chisquare(nu, N) / nu
X_t = X * np.sqrt((nu - 2) / nu) / np.sqrt(g)[:, None]
L_t = -V * ((mu + X_t) @ w)

for nome, L in (("normal", L_normal), ("t", L_t)):
    print(f"Monte Carlo {nome:6s}  VaR 95%: "
          f"R$ {np.quantile(L, 0.95):,.0f}   VaR 99%: "
          f"R$ {np.quantile(L, 0.99):,.0f}")
```

```
Monte Carlo normal  VaR 95%: R$ 25,381   VaR 99%: R$ 36,319
Monte Carlo t       VaR 95%: R$ 23,730   VaR 99%: R$ 41,533
```

Cada linha de `Z` é um cenário e `Z @ C.T` aplica $\mathbf{C}$ a todos de uma vez. A segunda metade do código troca a normal por uma t multivariada: dividir um vetor normal pela raiz de uma qui-quadrado independente, dividida por seus graus de liberdade, produz uma t, e o fator $\sqrt{(\nu-2)/\nu}$ é a mesma correção de escala da Equação (5.5), agora para que a covariância continue sendo $\boldsymbol{\Sigma}$. O valor de ν é o estimado no Código 5.3.

A Tabela 5.1 reúne os resultados. O Monte Carlo normal reproduz o paramétrico normal a menos de 0,3%, e o Monte Carlo t reproduz a menos de 0,7% o paramétrico t pelos momentos, com o qual compartilha a média e a covariância amostrais. Isso não é coincidência. Para uma carteira que é combinação linear dos ativos, sob um modelo normal ou t multivariado, a distribuição da perda é conhecida em forma fechada, e a simulação apenas a aproxima com ruído. A diferença que importa na tabela é entre hipóteses de distribuição, e não entre métodos de cálculo: a 99%, os modelos com t de Student pedem de 11% a 14% a mais que os normais.

A simulação acrescenta, além disso, um erro próprio. Repetindo 200 vezes o Monte Carlo normal (script `cap05/mc_erro.py`), o VaR de 99% estimado com mil cenários varia de R$ 32 mil a R$ 41 mil entre rodadas, com desvio-padrão de 5% do valor exato; com 100 mil cenários o desvio-padrão cai para 0,5%. Como o erro de simulação cai com $1/\sqrt{N}$, cada casa decimal a mais exige cem vezes mais cenários.

O Monte Carlo compensa esse custo quando não existe fórmula fechada. O caso clássico é o de carteiras com opções, cujo valor não varia linearmente com o preço do ativo: a perda da carteira deixa de ser normal mesmo que os retornos dos ativos sejam, e é preciso reavaliar cada posição em cada cenário (Hull, 2018; Jorion, 2007). Os outros casos são modelos dinâmicos, como simular dez dias de um GARCH (Capítulo 4) para obter o VaR de dez dias sem recorrer à raiz do tempo, e regras que dependem da trajetória, como uma ordem de *stop*, que encerra a posição quando a perda atinge um limite. Em todos eles vale a mesma ressalva: a simulação reproduz fielmente o modelo que recebeu, inclusive os defeitos dele.

5.6 Backtest do VaR: contar violações

Um modelo de VaR faz uma afirmação que pode ser conferida. Se o VaR de 99% estiver certo, a perda do dia seguinte deve superá-lo em 1% dos dias. Cada dia em que isso ocorre é uma **violação** (ou exceção). O *backtest* de um modelo de VaR consiste em recalcular o VaR a cada dia usando só os dados disponíveis até a véspera, comparar com a perda realizada e contar.

Tabela 5.1 VaR de um dia da carteira de R$ 1 milhão por seis caminhos, com os mesmos dados de 2015 a 2024 (valores em reais, saídas dos Códigos 5.2 a 5.5)

Método	VaR 95%	VaR 99%
Paramétrico normal	25.456	36.377
Paramétrico t, máxima verossimilhança	23.156	40.258
Paramétrico t, momentos amostrais	23.752	41.240
Histórico	22.914	37.561
Monte Carlo normal, 200 mil cenários	25.381	36.319
Monte Carlo t, 200 mil cenários	23.730	41.533

Violações de menos também são defeito: indicam um VaR alto demais, que imobiliza capital sem necessidade. O teste formal mais usado para a contagem é o de Kupiec (1995).

Resultado 5.4 Teste de proporção de falhas de Kupiec

Sejam T o número de dias do backtest, x o número de violações, $p = 1-\alpha$ a probabilidade de violação prometida pelo modelo e $\hat{p} = x/T$ a frequência observada. Sob a hipótese nula de que as violações são independentes e têm probabilidade p, a estatística

$$LR = -2\ln\left[\frac{(1-p)^{T-x}\,p^{x}}{(1-\hat{p})^{T-x}\,\hat{p}^{x}}\right] \tag{5.9}$$

tem, para T grande, distribuição aproximadamente qui-quadrado com um grau de liberdade. Rejeita-se o modelo ao nível de 5% quando $LR > 3{,}84$.

O numerador é a verossimilhança de x violações em T dias se a probabilidade for a prometida, e o denominador é a mesma verossimilhança no valor de $\hat{p}$ que melhor explica os dados. A razão é próxima de 1, e LR próxima de zero, quando $\hat{p}$ está perto de p. Para um ano de 250 pregões a 99%, esperam-se 2,5 violações. Com seis, $LR = 3{,}56$ e o modelo ainda passa; com sete, $LR = 5{,}50$ e ele é rejeitado. No outro extremo, nenhuma violação em 250 dias dá $LR = 5{,}03$ (as três contas estão em `cap05/exemplos.py`) e a aproximação qui-quadrado rejeitaria o modelo, embora um modelo correto passe um ano inteiro sem violação com probabilidade $0{,}99^{250} = 8\%$. Com tão poucos eventos esperados a aproximação é grosseira, e o teste tem pouco poder para distinguir um modelo bom de um medíocre.

O Comitê de Basileia usa a mesma contagem em 250 dias a 99% e só pune o excesso: até quatro violações, zona verde; de cinco a nove, amarela; dez ou mais, vermelha. Jorion (2007) expõe a regra.

O Código 5.6 testa três modelos nos 3624 dias de 2011 a 2025 para os quais há uma janela completa de 250 observações: o histórico com janela de 250 dias, o normal com média e desvio-padrão da mesma janela, e o normal com volatilidade EWMA, a média móvel exponencial dos retornos ao quadrado, $\sigma_t^2 = 0{,}94\,\sigma_{t-1}^2 + 0{,}06\,R_{t-1}^2$, o parente mais simples dos modelos de volatilidade condicional do Capítulo 4. É a única previsão de volatilidade usada neste capítulo; o VaR com a previsão de um GARCH fica para o último exercício.

Código 5.6 Backtest de três modelos de VaR e teste de Kupiec (`cap05/l05_kupiec.py`)

```
import numpy as np
from scipy import stats
from scipy.special import xlogy
from carteira import perda, V, Rp

def kupiec(x, T, p):
    """Teste de proporção de falhas: estatística LR e p-valor."""
    def logver(q):      # log-verossimilhança de x falhas em T
        return xlogy(x, q) + xlogy(T - x, 1 - q)
    lr = -2 * (logver(p) - logver(x / T))
    return lr, stats.chi2.sf(lr, df=1)

if __name__ == "__main__":
    jan = perda.rolling(250)
    com_var = jan.mean().shift(1).notna()   # dias com previsão
    ewma = (Rp ** 2).ewm(alpha=0.06, adjust=False).mean()
    for a in (0.95, 0.99):
```

```
        z = stats.norm.ppf(a)
        modelos = {
            "histórico 250d": jan.quantile(a),
            "normal 250d": jan.mean() + z * jan.std(),
            "normal EWMA": z * V * np.sqrt(ewma),
        }
        for nome, var in modelos.items():
            var = var.shift(1)      # só dados até a véspera
            viol = (perda > var)[com_var]["2011":"2025"]
            x, T = viol.sum(), len(viol)
            lr, pv = kupiec(x, T, 1 - a)
            print(f"{a:.0%} {nome:14s} {x:3d} em {T} "
                  f"({x / T:.2%})  LR = {lr:5.2f}  p = {pv:.4f}")
```

```
95% histórico 250d 199 em 3624 (5.49%)  LR =  1.79  p = 0.1814
95% normal 250d    181 em 3624 (4.99%)  LR =  0.00  p = 0.9878
95% normal EWMA    162 em 3624 (4.47%)  LR =  2.22  p = 0.1365
99% histórico 250d  46 em 3624 (1.27%)  LR =  2.45  p = 0.1178
99% normal 250d     47 em 3624 (1.30%)  LR =  2.95  p = 0.0858
99% normal EWMA     58 em 3624 (1.60%)  LR = 11.16  p = 0.0008
```

A função usa `xlogy`, que calcula $x \ln y$ e devolve zero quando $x = 0$, para que os casos de nenhuma violação e de violação todos os dias não quebrem a conta. A linha `var.shift(1)` é a que impede o modelo de enxergar a perda que ele deveria prever; sem ela o backtest usa informação do futuro, o *look-ahead* definido no Capítulo 2. A 95% os três modelos passam. A 99% o EWMA normal é rejeitado: ele acompanha bem o nível da volatilidade, mas o quantil normal de 2,33 desvios-padrão é pequeno demais para retornos de cauda pesada, e sobram violações.

O histórico passa no teste nos dois níveis, e a Figura 5.4 mostra por que isso não basta. O teste de Kupiec só olha para o total. As 46 violações do VaR histórico de 99% vêm em blocos: oito em menos de três semanas em fevereiro e março de 2020, quando a janela de 250 dias ainda refletia um mercado calmo, e nenhuma em anos inteiros. Um modelo correto deveria errar com a mesma probabilidade todos os dias, sem que uma violação anunciasse a seguinte. O teste de Christoffersen (1998) acrescenta ao de Kupiec exatamente essa verificação de independência.

> **Armadilha** **Passar no teste de Kupiec não valida o modelo**
>
> A contagem certa de violações no total é compatível com um VaR alto demais durante anos e baixo demais nas semanas em que mais se precisa dele. Olhe sempre o gráfico das violações no tempo. Blocos de violações indicam que o modelo demora a reagir a mudanças de volatilidade.

5.7 O que o VaR não vê

O VaR tem dois defeitos que não dependem do método de cálculo. O primeiro já apareceu: ele informa onde a cauda começa e ignora o que há dentro dela. Um operador avaliado apenas pelo VaR tem incentivo para montar posições que rendem um pouco quase sempre e perdem muito com probabilidade menor que $1 - \alpha$.

O segundo defeito é menos evidente e aparece quando se juntam carteiras. Artzner et al. (1999) propuseram quatro propriedades que uma medida de risco ρ deveria ter para merecer o nome. Na convenção de perdas deste capítulo, elas são:

- **Monotonicidade.** Se $L_1 \le L_2$ em todos os cenários, então $\rho(L_1) \le \rho(L_2)$.
- **Invariância por translação.** Para qualquer constante c, $\rho(L + c) = \rho(L) + c$: somar uma perda certa c aumenta o risco em c, e acrescentar caixa o reduz no mesmo valor.
- **Homogeneidade positiva.** Para $\lambda > 0$, $\rho(\lambda L) = \lambda\,\rho(L)$: dobrar a posição dobra o risco.
- **Subaditividade.** $\rho(L_1 + L_2) \le \rho(L_1) + \rho(L_2)$: juntar duas carteiras não pode criar risco.

Uma medida que satisfaz as quatro é chamada de **medida coerente de risco**. O VaR satisfaz as três primeiras e, em geral, falha na quarta.

Exemplo 5.2 Diversificar aumenta o VaR

Dois títulos de emissores diferentes têm, cada um, 4% de probabilidade de calote, com perda de R$ 100 nesse caso e zero no contrário. Os calotes são independentes. Para um título sozinho, $P(L \le 0) = 96\% \ge 95\%$, logo $\mathrm{VaR}_{0,95} = 0$. Para a carteira com os dois, a probabilidade de pelo menos um calote é $1 - 0{,}96^2 = 7{,}84\%$, maior que 5%, e portanto $\mathrm{VaR}_{0,95} = 100$. O VaR da soma (100) é maior que a soma dos VaR ($0 + 0$): a medida pune a diversificação e premia a concentração em um único emissor.

A falha não ocorre em qualquer situação. Para carteiras lineares de ativos com distribuição conjunta elíptica, família que inclui a normal e a t multivariadas, o VaR com $\alpha \ge 50\%$ é subaditivo (McNeil et al., 2015); é o caso do exemplo de dois ativos da Seção 5.3. O problema surge com distribuições muito assimétricas ou discretas, típicas de crédito e de opções vendidas, e basta para que limites de risco definidos por VaR em cada mesa não garantam o VaR do banco inteiro.

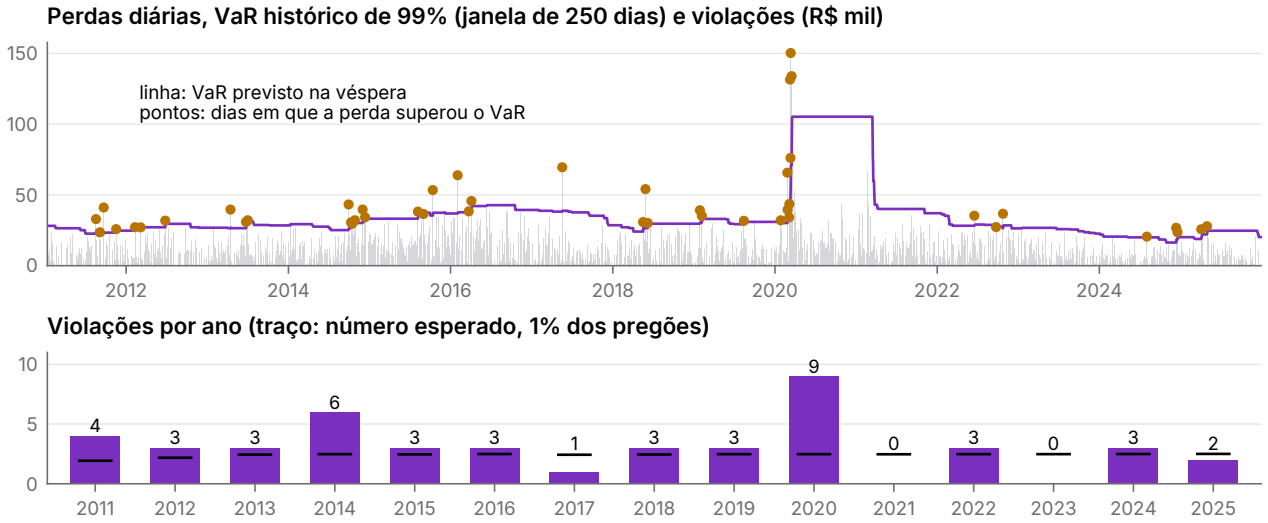


Figura 5.4 Backtest do VaR histórico de 99% com janela de 250 dias, de 2011 a 2025. O total de 46 violações em 3624 dias passa no teste de Kupiec, mas elas não se distribuem ao acaso: nove ocorrem em 2020, oito delas entre 26 de fevereiro e 16 de março, enquanto 2021 e 2023 não têm nenhuma.

5.8 Expected shortfall (CVaR)

A medida que corrige os dois defeitos responde à pergunta que o VaR deixa em aberto: nos dias em que a perda supera o VaR, de quanto ela é em média?

Definição 5.5 Expected shortfall

O **expected shortfall** (ES) de nível α, que este livro escreve CVaR_α, é a média dos VaR de todos os níveis acima de α:

$$\mathrm{CVaR}_\alpha(L) = \frac{1}{1-\alpha}\int_\alpha^1 \mathrm{VaR}_u(L)\,\mathrm{d}u. \tag{5.10}$$

Se L tem distribuição contínua, isso equivale à perda esperada condicional a superar o VaR: $\mathrm{CVaR}_\alpha(L) = \mathbb{E}[L \mid L \geq \mathrm{VaR}_\alpha(L)]$.

O nome **CVaR** (*conditional value at risk*) é o usado por Rockafellar e Uryasev (2000), que mostraram como minimizar a medida por programação linear; os dois nomes designam a mesma medida, e este capítulo usa ES e CVaR como sinônimos. Por construção $\mathrm{CVaR}_\alpha \geq \mathrm{VaR}_\alpha$, e o ES é uma medida coerente (Acerbi e Tasche, 2002). No exemplo dos dois títulos, o ES de 95% de um título é 80, a soma dos dois é 160, e o ES da carteira é 103,2 (as contas estão em `cap05/exemplos.py`): a diversificação volta a reduzir o risco.

Intuição

O VaR de 95% é a melhor das perdas entre os 5% piores dias. O ES de 95% é a média de todas elas. Na Figura 5.1, o VaR é a borda esquerda da região em ocre e o ES é o seu centro de gravidade, que fica em R$ 34,9 mil.

5.8.1 Fórmulas fechadas

Resultado 5.6 ES normal

Se $L \sim N(\mu_L, \sigma_L^2)$, então

$$\mathrm{CVaR}_\alpha(L) = \mu_L + \sigma_L\,\frac{\varphi(z_\alpha)}{1-\alpha}, \tag{5.11}$$

em que φ é a densidade da normal padrão.

A demonstração usa a propriedade $\varphi'(x) = -x\,\varphi(x)$ da densidade normal e fica como exercício. Os multiplicadores de σ_L são 2,063 a 95%, 2,338 a 97,5% e 2,665 a 99%.

Para a t de Student com $\nu > 1$ graus de liberdade, locação μ_L e escala s, a fórmula correspondente é (McNeil et al., 2015)

$$\mathrm{CVaR}_\alpha(L) = \mu_L + s\,\frac{f_\nu(t_{\nu,\alpha})}{1-\alpha}\cdot\frac{\nu + t_{\nu,\alpha}^2}{\nu-1}, \tag{5.12}$$

com f_ν a densidade da t padrão. O estimador histórico é a média das perdas da amostra que igualam ou superam o VaR histórico.

Código 5.7 VaR e ES pelos três caminhos (`cap05/l06_es.py`)

```python
import numpy as np
from scipy import stats
from carteira import perda

L = perda["2015":"2024"]
mu, sig = L.mean(), L.std()
nu, m, s = stats.t.fit(L)

print("nível   método      VaR       ES    ES/VaR")
for a in (0.95, 0.975, 0.99):
    z, q = stats.norm.ppf(a), stats.t.ppf(a, nu)
    v_h = np.quantile(L, a)
    linhas = {
        # média das perdas iguais ou maiores que o VaR
        "histórico": (v_h, L[L >= v_h].mean()),
        "normal": (mu + sig * z,
                   mu + sig * stats.norm.pdf(z) / (1 - a)),
        "t": (m + s * q,
              m + s * stats.t.pdf(q, nu) / (1 - a)
              * (nu + q ** 2) / (nu - 1)),
    }
    for nome, (v, es) in linhas.items():
        print(f"{a:6.1%}  {nome:9s} {v:8,.0f} {es:8,.0f}"
              f"   {es / v:.2f}")
```

nível	método	VaR	ES	ES/VaR
95.0%	histórico	22,914	34,929	1.52
95.0%	normal	25,456	32,152	1.26
95.0%	t	23,156	34,354	1.48
97.5%	histórico	29,595	44,412	1.50
97.5%	normal	30,506	36,561	1.20
97.5%	t	30,095	42,521	1.41
99.0%	histórico	37,561	60,794	1.62
99.0%	normal	36,377	41,808	1.15
99.0%	t	40,258	54,853	1.36

A coluna ES/VaR é a que mais ensina. Sob normalidade a cauda decai tão depressa que o ES fica pouco acima do VaR, e a razão diminui com o nível de confiança: 1,26, 1,20, 1,15. Nos dados ela fica em torno de 1,5 e não diminui. A 99%, o VaR normal e o histórico diferem em 3%, e alguém poderia concluir que a hipótese normal é inofensiva. No ES de 99% a diferença é de R$ 41,8 mil para R$ 60,8 mil, 45% a mais. A normal erra pouco a posição do quantil e erra muito o que vem depois dele; a t chega mais perto, e ainda assim fica abaixo do histórico.

5.8.2 Do VaR de 99% ao ES de 97,5%

Na revisão das regras de capital para risco de mercado conhecida como FRTB (*Fundamental Review of the Trading Book*), o Comitê de Basileia substituiu o VaR de 99% pelo ES de 97,5% nos modelos internos dos bancos (BCBS, 2019). A escolha do nível tem uma lógica que a Equação (5.11) revela: sob normalidade, o ES de 97,5% vale $2{,}338\,\sigma_L$ e o VaR de 99% vale $2{,}326\,\sigma_L$, praticamente o mesmo número. Para uma carteira de cauda normal a troca não altera a exigência; para caudas pesadas, ela a aumenta. Na carteira do capítulo, o ES histórico de 97,5% é de R$ 44,4 mil, contra um VaR histórico de 99% de R$ 37,6 mil.

O ES tem um custo estatístico: é uma média de observações raras, e por isso sua estimativa é mais instável que a do VaR de mesmo nível (Seção 5.10) e mais difícil de conferir em um backtest.

5.9 Drawdown

VaR e ES olham para um dia de cada vez. O investidor sente o risco de outra forma: pela queda acumulada desde o melhor momento da carteira e pelo tempo que ela leva para voltar até lá. Uma sequência de trinta perdas pequenas, nenhuma delas uma violação de VaR, pode custar mais do que um dia de pânico.

Definição 5.7 Drawdown

Seja V_t o valor da carteira (ou da cota de um fundo) no instante t. O máximo atingido até t, $M_t = \max_{s \le t} V_s$, é o **high-water mark**. O **drawdown** em t é a distância percentual até ele,

$$D_t = \frac{V_t}{M_t} - 1 \le 0, \tag{5.13}$$

e o **drawdown máximo** de um período é o menor valor de D_t nesse período.

Três números descrevem um episódio de drawdown: a profundidade (o valor de D_t no vale), a duração da queda, do pico ao vale, e o **tempo de recuperação**,

do vale até a cota voltar ao pico anterior. A soma das duas durações, de um pico ao seguinte, é o **tempo submerso**. O Código 5.8 calcula os quatro para a `cota` do Código 5.1, com os dezesseis anos de dados, de 2010 a 2025, e supondo que os pesos iguais são refeitos todos os dias.

Código 5.8 Série de drawdown, drawdown máximo e recuperação (`cap05/l07_drawdown.py`)

```
import numpy as np
from carteira import cota, pregoes

hwm = cota.cummax()                # high-water mark
dd = cota / hwm - 1                # drawdown: sempre <= 0

vale = dd.idxmin()
pico = cota[:vale].idxmax()
depois = cota[vale:]
volta = depois[depois >= cota[pico]].index[0]
queda = len(cota[pico:vale]) - 1
recuperacao = len(cota[vale:volta]) - 1
print(f"drawdown máximo: {dd.min():.1%}")
print(f"pico {pico:%d/%m/%Y}  vale {vale:%d/%m/%Y}  "
      f"recuperação {volta:%d/%m/%Y}")
print(f"queda: {queda} pregões   recuperação: {recuperacao}"
      f"   tempo submerso: {queda + recuperacao}")

anos = len(pregoes[cota.index[0]:]) / 252   # pregões da B3
cagr = cota.iloc[-1] ** (1 / anos) - 1
print(f"retorno a.a.: {cagr:.1%}   "
      f"Calmar: {cagr / abs(dd.min()):.2f}")
print(f"fração dos dias abaixo do pico: {(dd < 0).mean():.1%}")
```

```
drawdown máximo: -42.8%
pico 05/05/2015  vale 26/01/2016  recuperação 15/08/2016
queda: 179 pregões   recuperação: 139   tempo submerso: 318
retorno a.a.: 16.1%   Calmar: 0.38
fração dos dias abaixo do pico: 92.1%
```

O método `cummax()` devolve o máximo acumulado e resolve a conta em uma linha. O pior episódio foi a queda de 42,8% entre maio de 2015 e janeiro de 2016, durante a recessão brasileira: 179 pregões de descida e mais 139 para recuperar, 318 de tempo submerso (o código conta as datas da cota, que no episódio coincidem com os pregões). O retorno é anualizado pela convenção de 252 dias (Seção 1.9). A última linha merece atenção. A carteira multiplicou o capital por mais de dez no período e, mesmo assim, passou 92% dos dias abaixo de algum pico anterior.

O gráfico de baixo da Figura 5.5 é chamado de *underwater*, porque mostra quanto tempo a carteira passa "debaixo d'água". Ele deixa ver o que o número isolado esconde: 2015–2016 e 2020 têm quase a mesma profundidade, mas o primeiro foi uma erosão de nove meses e o segundo um choque de menos de um mês.

A recuperação exige mais do que a queda tirou, pela aritmética da armadilha sobre somar retornos do Capítulo 1: depois de um drawdown D, o retorno necessário para voltar ao pico é $1/(1+D)-1$. Para $D=-42{,}8\%$, são $+74{,}8\%$.

O **índice de Calmar** resume a relação entre o que a carteira rendeu e o pior momento que impôs: divide o retorno anualizado pelo módulo do drawdown

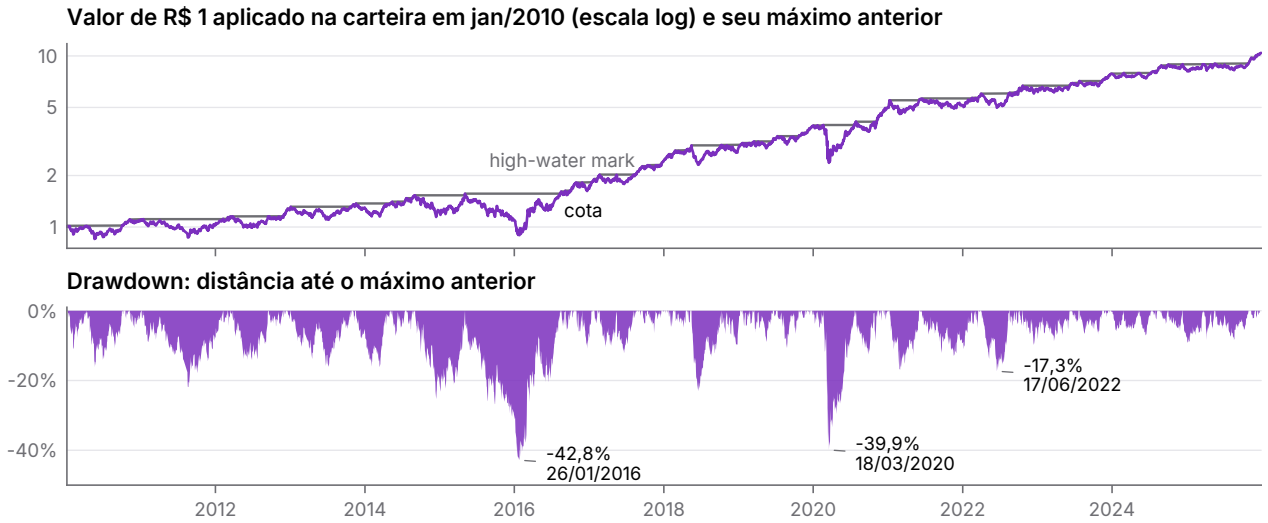


Figura 5.5 Cota da carteira, *high-water mark* e drawdown, de 2010 a 2025. Os dois maiores episódios têm profundidade parecida e formatos opostos: o de 2015–2016 levou 179 pregões do pico ao vale; o de 2020 levou 18 pregões para cair 39,9% e 86 para recuperar.

máximo do mesmo período. A medida foi proposta por Young (1991), que usava o retorno anual médio e o drawdown máximo dos últimos 36 meses. Para a carteira, $16{,}1\%/42{,}8\% = 0{,}38$: cada ponto de retorno anual veio acompanhado de cerca de 2,7 pontos de drawdown máximo. O Capítulo 7 retoma o Calmar ao lado das demais métricas de desempenho.

Armadilha **O drawdown máximo é uma única observação**

O drawdown máximo depende de um só episódio e do tamanho da amostra: quanto mais longo o histórico, maior tende a ser o pior episódio encontrado, e por isso não se comparam drawdowns de fundos com idades diferentes. Há ainda um viés nesta carteira em particular: as cinco ações foram escolhidas hoje, entre empresas que sobreviveram e prosperaram.

5.10 Métricas de cauda e o tamanho do erro

Além de VaR, ES e drawdown, relatórios de risco costumam trazer um conjunto de estatísticas descritivas da cauda. O Código 5.9 calcula as mais comuns.

Código 5.9 Métricas descritivas de cauda (`cap05/l08_cauda.py`)

```python
import numpy as np
from scipy import stats
from carteira import Rp, cota

r = Rp["2015":"2024"]
print(f"assimetria: {stats.skew(r):.2f}")
print(f"excesso de curtose: {stats.kurtosis(r):.2f}")
q05, q95 = np.quantile(r, [0.05, 0.95])
print(f"razão de cauda q95/|q05|: {q95 / abs(q05):.2f}")
print(f"pior dia: {r.min():.1%} em {r.idxmin():%d/%m/%Y}")

mensal = cota.resample("ME").last().pct_change()["2015":"2024"]
print(f"pior mês: {mensal.min():.1%} em {mensal.idxmin():%m/%Y}")

# quantos desvios-padrão foi o pior dia, e o que a normal diz
k = (r.min() - r.mean()) / r.std()
print(f"pior dia em desvios-padrão: {k:.1f}")
print(f"probabilidade normal de um dia assim: "
      f"{stats.norm.cdf(k):.1e}")
```

```
assimetria: -0.53
excesso de curtose: 10.43
razão de cauda q95/|q05|: 1.07
pior dia: -15.0% em 12/03/2020
pior mês: -22.6% em 03/2020
pior dia em desvios-padrão: -9.4
probabilidade normal de um dia assim: 1.9e-21
```

A assimetria e a curtose foram definidas no Capítulo 3. Assimetria negativa indica que os desvios grandes ocorrem mais para o lado das perdas, e um excesso de curtose de 10 indica caudas muito mais pesadas que as da normal, cujo excesso é zero. A **razão de cauda** divide o quantil de 95% dos retornos pelo módulo do quantil de 5%: acima de 1, os ganhos típicos dos dias bons superam as perdas típicas dos dias ruins. Aqui ela vale 1,07 enquanto a assimetria é negativa, e não há contradição: a razão olha para os quantis de 5% e a assimetria é dominada pelos poucos dias extremos. Pior dia e pior mês são os números mais fáceis de comunicar e os mais frágeis, porque cada um é uma observação.

As duas últimas linhas dão a medida do fracasso da normal. O pior dia ficou a 9,4 desvios-padrão da média, evento ao qual a normal atribui probabilidade da ordem de 10^{-21}.

No repositório

A pasta `aula5/` traz os três tipos de VaR para VALE3 (`Var_parametrico.py`, `Var_historico.py`, `Var_MonteCarlo.py`), notebooks de CVaR e de drawdown máximo, e o cálculo de assimetria e curtose com laços (`sk.py`, `kurtose.py`). Os scripts de VaR informam o resultado como log-retorno negativo; refaça-os com perdas positivas em reais e confira com os códigos deste capítulo.

5.10.1 Por que toda medida de cauda é imprecisa

Uma medida de cauda é estimada com as poucas observações que caem na cauda. Em um ano de 250 pregões, o VaR de 95% se apoia em 12 ou 13 perdas, o de 99% em duas ou três, e o ES de 99% é a média dessas duas ou três. A Figura 5.6

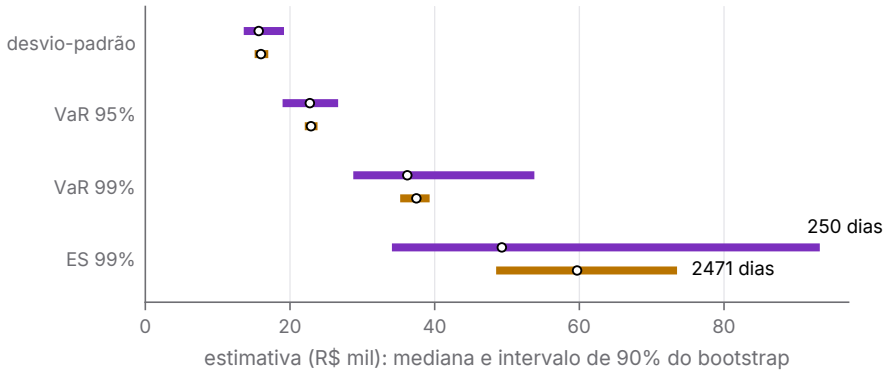


Figura 5.6 Intervalos de 90% de quatro medidas de risco em 5000 reamostragens das perdas diárias da carteira (2015–2024). Com 250 dias, o VaR de 99% pode sair entre R$ 29 mil e R$ 54 mil e o ES de 99% entre R$ 34 mil e R$ 93 mil. Quanto mais para dentro da cauda, maior o erro.

mede a consequência por *bootstrap*: sorteiam-se, com reposição, amostras de 250 dias das perdas da carteira, recalcula-se cada medida e observa-se quanto ela varia de uma amostra para outra.

Com um ano de dados, o desvio-padrão é estimado com um intervalo de largura igual a um terço do seu valor; o VaR de 99%, com um intervalo de 70% do valor; o ES de 99%, com um intervalo maior que a própria estimativa. Com dez anos os intervalos encolhem, mas o do ES de 99% ainda vai de R$ 48,5 mil a R$ 73,5 mil. E a figura é otimista: o sorteio trata os dias como independentes, quando na realidade os dias ruins vêm agrupados e a informação contida na amostra é menor do que o número de observações sugere.

A lição prática é desconfiar de casas decimais. Um VaR de 99% informado como R$ 37.561 tem bem menos precisão do que aparenta: mesmo com dez anos de dados, o intervalo de 90% do bootstrap vai de R$ 35 mil a R$ 39 mil, e o segundo algarismo já é incerto. A diferença entre os métodos da Tabela 5.1 é menor que o erro de estimação de qualquer um deles com um ano de dados, e por isso a escolha da janela e a reação do modelo a mudanças de volatilidade pesam mais do que a escolha entre fórmulas. McNeil et al. (2015) tratam da teoria dos valores extremos, o conjunto de técnicas estatísticas desenvolvido para extrapolar a cauda além do que a amostra mostra.

O que levar deste capítulo

- O VaR de nível α é o quantil α das perdas em um horizonte dado. Ele marca onde a cauda começa e não é a perda máxima.
- O VaR paramétrico normal é $\mu_L + \sigma_L z_\alpha$, com $\sigma_L = V\sqrt{\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}}$. A normal tende a superestimar o VaR de 95% e a subestimar o de 99%; com a t de Student, a escala precisa do fator $\sqrt{(\nu-2)/\nu}$.
- O VaR histórico dispensa hipóteses de distribuição e depende da janela. O Monte Carlo só acrescenta algo quando a carteira é não linear ou o modelo é dinâmico.
- Um modelo de VaR se testa contando violações fora da amostra: o teste de Kupiec confere o total, e o gráfico no tempo mostra se vêm em blocos.
- O VaR não é subaditivo em geral. O *expected shortfall*, a perda média além do VaR, é coerente e sensível à forma da cauda, e foi adotado pela regulação bancária a 97,5%.

- O drawdown mede a queda acumulada desde o pico. Toda medida de cauda, do VaR de 99% ao drawdown máximo, se apoia em poucas observações e tem erro de estimação grande.

Exercícios

5.1 [básico] Uma carteira de R$ 500 mil tem volatilidade diária de 1,2%. Supondo normalidade e média zero, calcule à mão o VaR de um dia a 95% e a 99%, o VaR de dez dias a 99% pela raiz do tempo e o ES de um dia a 97,5%.

5.2 [básico] Um relatório informa: "VaR de um dia a 99%: R$ 40 mil". Verdadeiro ou falso, e por quê: (a) a carteira nunca perde mais de R$ 40 mil em um dia; (b) em dois ou três dias por ano a perda deve superar R$ 40 mil; (c) nos dias de violação a perda média é de R$ 40 mil.

5.3 [intermediário] Um modelo de VaR de 95% teve 36 violações em 500 dias. Calcule a estatística de Kupiec pela Equação (5.9) e decida ao nível de 5%. Para que números de violações, acima e abaixo do esperado (25), o modelo é rejeitado?

5.4 [intermediário] Demonstre a Equação (5.11) e encontre numericamente o nível α em que o ES normal iguala o VaR normal de 99%.

5.5 [intermediário] Adapte `cap05/carteira.py` para uma posição de R$ 100 mil em `PETR4.SA`. Calcule o VaR e o ES históricos de 99% de 2015 a 2024 e liste os três drawdowns mais profundos de 2010 a 2025, com profundidade, duração da queda e tempo de recuperação.

5.6 [desafio] A expansão de Cornish–Fisher corrige o quantil normal pela assimetria S e pelo excesso de curtose $K-3$ da perda (notação do Capítulo 3): $\tilde{z}_\alpha = z_\alpha + (z_\alpha^2-1)S/6 + (z_\alpha^3-3z_\alpha)(K-3)/24 - (2z_\alpha^3-5z_\alpha)S^2/36$, com $\mathrm{VaR}_\alpha = \mu_L + \sigma_L \tilde{z}_\alpha$. Aplique-a às perdas diárias e mensais da carteira e compare com o VaR histórico. O que acontece quando o excesso de curtose é da ordem de 10?

5.7 [desafio] Acrescente ao Código 5.6 um modelo com volatilidade prevista por um GARCH(1,1) com inovações t (Capítulo 4) e quantil da t com a escala correta. Ele passa no teste de Kupiec a 99%? As violações de 2020 continuam agrupadas?

Para ir além

- Jorion (2007), a referência clássica sobre VaR, para os métodos de cálculo, o backtest e a simulação de Monte Carlo.
- Hull (2018), para carteiras com opções e para a regulação de Basileia.
- McNeil et al. (2015), para o tratamento rigoroso de VaR, ES, medidas coerentes e teoria dos valores extremos.
- Artzner et al. (1999), o artigo que formulou os axiomas de coerência, e Acerbi e Tasche (2002), que demonstram que o ES os satisfaz.

Capítulo 6

Portfólios: Markowitz, diversificação e testes de estresse

6

Os capítulos anteriores mediram retorno e risco de um ativo por vez. Quem investe, porém, carrega vários ativos ao mesmo tempo, e o risco do conjunto não é a soma dos riscos das partes. Este capítulo mostra como a correlação entre os ativos reduz o risco de uma carteira, como escolher os pesos de forma sistemática, por que a receita matemática mais famosa das finanças falha quando alimentada com médias históricas, e como perguntar à carteira o que acontece em um dia ruim que o histórico recente não contém.

Ao terminar este capítulo você consegue

- calcular retorno esperado e variância de uma carteira em notação matricial;
- explicar por que a diversificação reduz o risco e por que essa redução tem um piso;
- obter a carteira de mínima variância, a carteira tangente e a fronteira eficiente, por fórmula e por otimização numérica;
- reconhecer a sensibilidade da otimização aos insumos e testar uma regra de alocação fora da amostra;
- aplicar remédios simples contra o erro de estimação: restrições, encolhimento e regras que dispensam médias;
- montar e reportar testes de estresse históricos, hipotéticos e de correlação.

6.1 Dois ativos: o ganho de misturar

Considere dois ativos. O ativo A tem retorno esperado de 10% ao ano e volatilidade de 20%; o ativo B, 15% e 30%. Uma carteira com metade do patrimônio em cada um tem retorno esperado de 12,5%, a média dos dois, como garante a Equação (1.2) do Capítulo 1. A volatilidade, porém, só será a média (25%) em um caso extremo. Em todos os outros ela é menor.

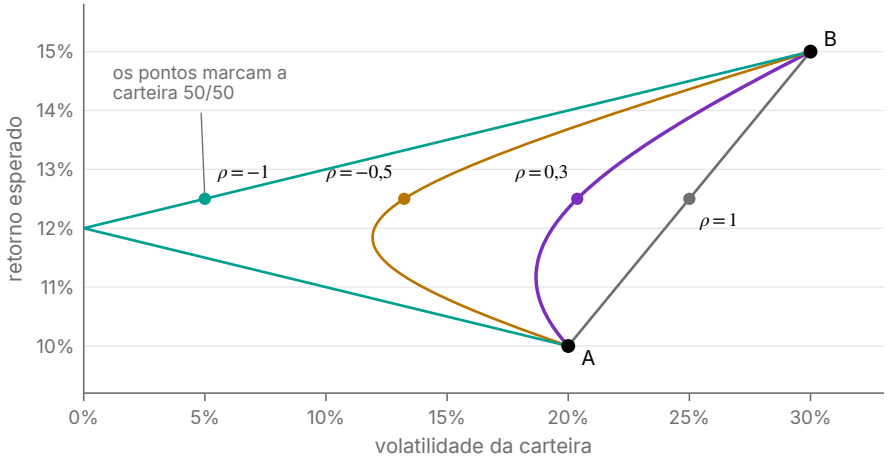


Figura 6.1 Todas as carteiras formadas com os ativos A e B, para quatro valores da correlação. Com $\rho = 1$ as combinações ficam sobre a reta que liga os dois ativos. Quanto menor a correlação, mais a curva se dobra para a esquerda: mesmo retorno esperado, menos risco.

Sejam w o peso em A e $1 - w$ o peso em B. O retorno da carteira é $R_p = wR_A + (1 - w)R_B$, e a variância de uma soma de duas variáveis aleatórias (Capítulo 3) dá

$$\sigma_p^2 = w^2\sigma_A^2 + (1-w)^2\sigma_B^2 + 2\,w(1-w)\,\rho\,\sigma_A\sigma_B, \tag{6.1}$$

em que ρ é a correlação entre os dois retornos e $\rho\,\sigma_A\sigma_B$ é a covariância. O retorno esperado não depende de ρ; o risco depende, e muito.

Exemplo 6.1 A carteira meio a meio

Com $\sigma_A = 0{,}20$, $\sigma_B = 0{,}30$, $\rho = 0{,}3$ e $w = 0{,}5$:

$$\begin{aligned}\sigma_p^2 &= 0{,}25 \times 0{,}04 + 0{,}25 \times 0{,}09 + 2 \times 0{,}25 \times 0{,}3 \times 0{,}06 \\ &= 0{,}0100 + 0{,}0225 + 0{,}0090 = 0{,}0415,\end{aligned}$$

e $\sigma_p = \sqrt{0{,}0415} = 20{,}37\%$. A carteira rende a média dos dois ativos (12,5%) com risco bem abaixo da média dos riscos (25%). A diferença de 4,6 pontos percentuais é o ganho de diversificação. O script `cap06/l01_dois_ativos.py` repete a conta.

Se $\rho = 1$, a Equação (6.1) vira um quadrado perfeito, $\sigma_p = w\sigma_A + (1-w)\sigma_B$, e não há ganho algum. Para qualquer $\rho < 1$ e pesos estritamente entre zero e um, a volatilidade da carteira fica abaixo da média ponderada das volatilidades. No outro extremo, com $\rho = -1$, existe uma combinação que zera o risco: $w = \sigma_B/(\sigma_A + \sigma_B) = 0{,}6$. A Figura 6.1 mostra as quatro situações.

A curva de $\rho = 0{,}3$ tem ainda uma propriedade que surpreende na primeira vez. Seu ponto mais à esquerda, a carteira de menor variância, tem 76,6% em A e volatilidade de 18,67%, abaixo dos 20% do ativo menos arriscado. Acrescentar um pouco do ativo mais volátil *reduziu* o risco. Derivando a Equação (6.1) em relação a w e igualando a zero, o peso de mínima variância é

$$w^* = \frac{\sigma_B^2 - \rho\,\sigma_A\sigma_B}{\sigma_A^2 + \sigma_B^2 - 2\rho\,\sigma_A\sigma_B}, \tag{6.2}$$

e ele é menor que 1 sempre que $\rho < \sigma_A/\sigma_B$. Markowitz (1952) foi quem transformou essa observação em método: o investidor não deve avaliar um ativo

isoladamente, e sim pelo que ele faz com a média e a variância da carteira inteira.

6.2 N ativos em notação matricial

Com N ativos as somas ficam longas e a notação matricial passa a compensar. Reúna os pesos em um vetor $\mathbf{w} = (w_1, \dots, w_N)^\top$, os retornos esperados em $\boldsymbol{\mu}$ e as covariâncias na matriz $\boldsymbol{\Sigma}$, cujo elemento σ_{ij} é $\mathrm{Cov}(R_i, R_j)$ e cuja diagonal traz as variâncias.

Resultado 6.1 Média e variância de uma carteira

Para uma carteira com pesos $\mathbf{w}$, com $\mathbf{w}^\top \mathbf{1} = 1$,

$$\mu_p = \mathbf{w}^\top \boldsymbol{\mu} = \sum_{i=1}^{N} w_i \mu_i, \qquad \sigma_p^2 = \mathbf{w}^\top \boldsymbol{\Sigma}\, \mathbf{w} = \sum_{i=1}^{N}\sum_{j=1}^{N} w_i w_j \sigma_{ij}. \tag{6.3}$$

A soma dupla tem N^2 parcelas: N variâncias, quando $i = j$, e $N(N-1)$ covariâncias. Com dois ativos são duas variâncias e duas covariâncias iguais, e a expressão se reduz à Equação (6.1). Em NumPy as duas contas são `w @ mu` e `w @ cov @ w`.

Intuição

Em uma carteira de 50 ações há 50 variâncias e 2 450 covariâncias. O risco do conjunto é quase todo determinado pela forma como os ativos se movem *juntos*, e muito pouco pelo quanto cada um oscila sozinho. Por isso a matriz de covariância, e não a lista de volatilidades, é o insumo central da gestão de carteiras.

Ao longo do capítulo usamos as sete ações do conjunto de dados do livro (PETR4, VALE3, ITUB4, WEGE3, ABEV3, BBAS3 e MGLU3). Os insumos são estimados com retornos simples diários, que são os que se somam entre ativos, e anualizados pela convenção de 252 dias (Seção 1.9): $\hat{\boldsymbol{\mu}}$ é a média diária multiplicada por 252 e $\hat{\boldsymbol{\Sigma}}$ é a covariância amostral multiplicada por 252. Datas em que alguma das sete ações não tem preço são descartadas, e o retorno seguinte cobre o intervalo inteiro; por isso a amostra de 2015 a 2024 tem 2 472 retornos. As volatilidades também diferem um pouco das do Capítulo 1, calculadas com log-retornos. A Figura 6.2 mostra as correlações entre 2015 e 2024.

Nenhuma correlação é negativa, e isso é a regra entre ações de um mesmo país: todas respondem aos mesmos juros, ao mesmo câmbio e ao mesmo humor do mercado. O ganho de diversificação existe, porque as correlações estão longe de 1, mas é limitado, como mostra a próxima seção.

A média aritmética anualizada supera o retorno geométrico, e tanto mais quanto maior a volatilidade. Para PETR4, de 2015 a 2024, 36,9% contra 29,0% (o Capítulo 1 mostra 29,3% porque usa as datas de cinco papéis e o tempo de calendário).

6.3 O limite da diversificação

Tome a carteira mais simples possível, com pesos iguais $w_i = 1/N$. Pela Equação (6.3),

$$\sigma_p^2 = \frac{1}{N^2}\sum_{i=1}^{N} \sigma_i^2 + \frac{1}{N^2}\sum_{i \neq j} \sigma_{ij}.$$

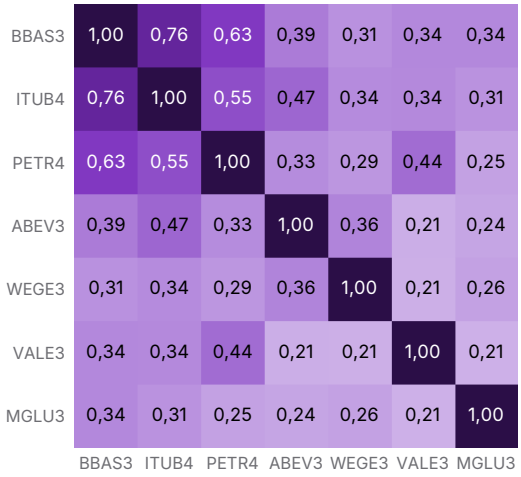


Figura 6.2 Correlações entre os retornos diários das sete ações, de 2015 a 2024, ordenadas da mais para a menos correlacionada com as demais. Todas são positivas. Os dois bancos formam o par mais parecido (0,76); a média fora da diagonal é 0,36.

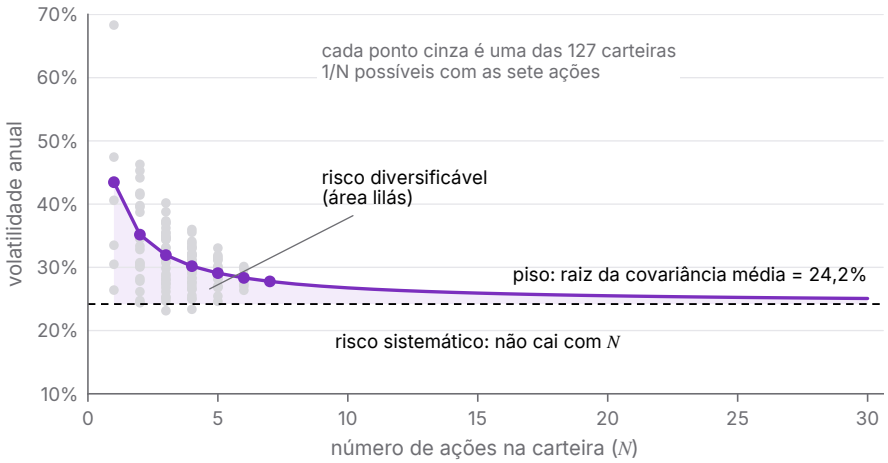


Figura 6.3 Volatilidade de carteiras igualmente ponderadas formadas com as sete ações (2015 a 2024). Os pontos roxos são a raiz da variância média entre todas as carteiras de mesmo tamanho, que é o que a Equação (6.4) calcula; a curva a extrapola para N maior, supondo que novas ações tenham variância e covariância médias iguais às das sete.

Chame de $\bar{v}$ a média das N variâncias e de $\bar{c}$ a média das $N(N-1)$ covariâncias. A primeira soma vale $N\bar{v}$ e a segunda, $N(N-1)\,\bar{c}$.

Resultado 6.2 Variância da carteira igualmente ponderada

$$\sigma_p^2 = \frac{1}{N}\,\bar{v} + \left(1 - \frac{1}{N}\right)\bar{c} \xrightarrow[N\to\infty]{} \bar{c}. \tag{6.4}$$

A parcela das variâncias individuais é dividida por N e desaparece à medida que a carteira cresce. É o **risco diversificável**, também chamado de específico ou idiossincrático: a greve em uma fábrica, o balanço ruim de uma empresa. A parcela da covariância média não diminui com N. É o **risco sistemático**, o que os ativos têm em comum, e nenhuma quantidade de ações o elimina.

A Figura 6.3 aplica a fórmula aos dados. Uma ação escolhida ao acaso entre as sete tem variância média $\bar{v} = 0{,}189$, ou volatilidade de 43,5% ao ano. Com as sete em pesos iguais a volatilidade cai para 27,8%. A covariância média é $\bar{c} = 0{,}058$, de modo que o piso é $\sqrt{\bar{c}} = 24{,}2\%$. Na carteira de sete ações, cerca

de dois terços da variância ($\frac{6}{7}\,\bar{c}/\sigma_p^2 = 0{,}65$, com os valores sem arredondar) já vêm das covariâncias, e trinta ações com as mesmas características levariam a volatilidade apenas a 25,1%. O piso fica perto da volatilidade do BOVA11 no mesmo período, 24,4% em retornos simples. Faz sentido, porque um índice amplo é, em boa medida, o que sobra depois de diversificar, mas a proximidade dos dois números é em parte coincidência: $\bar{c}$ depende das sete ações escolhidas.

Duas consequências práticas. A maior parte do ganho vem dos primeiros ativos, e passar de 20 para 200 ações quase não muda o risco. E quem quiser ir abaixo do piso precisa de ativos com covariância menor com a bolsa (renda fixa, câmbio, ações de outros países), não de mais ações do mesmo mercado.

6.4 O problema de Markowitz

Nada garante que pesos iguais sejam a melhor escolha. Markowitz (1952) propôs tratar a seleção de carteiras como um problema de otimização: entre todas as carteiras com um dado retorno esperado μ^*, procure a de menor variância,

$$\min_{\mathbf{w}} \ \mathbf{w}^\top \boldsymbol{\Sigma}\, \mathbf{w} \quad \text{sujeito a} \quad \mathbf{w}^\top \boldsymbol{\mu} = \mu^*, \qquad \mathbf{w}^\top \mathbf{1} = 1. \tag{6.5}$$

Resolvendo para cada μ^* obtém-se, no plano de volatilidade e retorno, uma curva que delimita o que é possível: à esquerda dela não há carteira.

Definição 6.3 Fronteira eficiente

Uma carteira é **eficiente** se nenhuma outra tem retorno esperado maior com a mesma variância, nem variância menor com o mesmo retorno esperado. A **fronteira eficiente** é o conjunto dessas carteiras: o trecho da curva de mínima variância que fica acima da carteira de menor variância de todas.

O artigo original já proibia pesos negativos. Um peso negativo é uma **venda a descoberto**: vender um ativo que não se tem, tomado em aluguel, para recomprá-lo depois. Se essa restrição é retirada, o problema (6.5) tem solução em fórmula fechada, e vale a pena conhecê-la porque revela a estrutura do problema.

Resultado 6.4 Carteira de mínima variância global

Sem restrição de sinal sobre os pesos, a carteira de menor variância entre todas as que somam 1 é

$$\mathbf{w}_{\text{mv}} = \frac{\boldsymbol{\Sigma}^{-1}\mathbf{1}}{\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}\mathbf{1}}, \qquad \text{com variância} \qquad \sigma^2_{\text{mv}} = \frac{1}{\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}\mathbf{1}}. \tag{6.6}$$

A demonstração usa um multiplicador de Lagrange, a técnica do cálculo para minimizar uma função sujeita a uma restrição de igualdade. Minimizar $\frac{1}{2}\mathbf{w}^\top\boldsymbol{\Sigma}\mathbf{w} - \lambda(\mathbf{w}^\top\mathbf{1} - 1)$ exige $\boldsymbol{\Sigma}\mathbf{w} = \lambda\mathbf{1}$, isto é, $\mathbf{w} = \lambda\,\boldsymbol{\Sigma}^{-1}\mathbf{1}$. Impondo que os pesos somem 1, $\lambda = 1/(\mathbf{1}^\top\boldsymbol{\Sigma}^{-1}\mathbf{1})$, e a variância é $\mathbf{w}^\top\boldsymbol{\Sigma}\mathbf{w} = \lambda\,\mathbf{w}^\top\mathbf{1} = \lambda$. Repare que $\boldsymbol{\mu}$ não aparece: essa é a única carteira da fronteira que dispensa qualquer estimativa de retorno esperado, um detalhe que ganhará importância na Seção 6.7.

6.5 Ativo livre de risco e a carteira tangente

Acrescente agora um ativo sem risco, que rende R^f com certeza. No Brasil a referência é o CDI; neste capítulo R^f é a taxa anual, tratada como constante, e R_t^f fica reservado ao CDI de cada dia. Uma carteira com fração a em uma carteira arriscada p e $1-a$ no ativo livre de risco tem retorno esperado $R^f + a(\mu_p - R^f)$ e volatilidade $a\,\sigma_p$. Eliminando a, as combinações formam uma reta no plano de volatilidade e retorno,

$$\mu = R^f + \frac{\mu_p - R^f}{\sigma_p}\,\sigma, \tag{6.7}$$

chamada **linha de alocação de capital**. A inclinação é o retorno em excesso por unidade de risco, o **índice de Sharpe** da carteira p (Sharpe, 1966):

$$S_p = \frac{\mu_p - R^f}{\sigma_p}. \tag{6.8}$$

Essa definição operacional, com média e volatilidade anualizadas dos insumos, basta por ora. O Capítulo 7 trata o índice com mais cuidado, inclusive seu erro de estimação, e o calcula sobre a série diária de retornos em excesso ao CDI (média sobre desvio-padrão, vezes $\sqrt{252}$); é assim que ele é medido fora da amostra na Seção 6.7. Como o CDI quase não varia de um dia para o outro, as duas contas diferem muito pouco.

Entre todas as retas que partem de R^f, a mais inclinada é a que tangencia a fronteira eficiente. A carteira no ponto de tangência é a de máximo Sharpe.

Resultado 6.5 Carteira tangente

Sem restrição de sinal, a carteira de ativos arriscados com o maior índice de Sharpe é

$$\mathbf{w}_{\text{tg}} = \frac{\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - R^f\mathbf{1})}{\mathbf{1}^\top\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - R^f\mathbf{1})}, \tag{6.9}$$

desde que o denominador seja positivo, o que ocorre exatamente quando a carteira de mínima variância global rende mais que R^f.

A justificativa segue o mesmo caminho. O índice de Sharpe não muda quando os pesos são multiplicados por uma constante positiva, de modo que se pode maximizar $\mathbf{w}^\top(\boldsymbol{\mu} - R^f\mathbf{1})$ para uma variância fixa. A condição de primeira ordem é $\boldsymbol{\Sigma}\mathbf{w} \propto \boldsymbol{\mu} - R^f\mathbf{1}$, e a divisão pela soma apenas normaliza os pesos.

Com o ativo livre de risco, a decisão se separa em duas. Primeiro encontra-se a carteira tangente, que é a mesma para todos os investidores que concordam com $\boldsymbol{\mu}$ e $\boldsymbol{\Sigma}$. Depois cada um escolhe quanto aplicar nela e quanto no CDI, conforme sua tolerância a risco. Se todos os investidores agissem assim, a carteira tangente teria de ser a carteira de mercado, com todos os ativos na proporção de seu valor; nesse caso a reta (6.7) recebe o nome de **linha de mercado de capitais** (CML, de *capital market line*).

Código 6.1 Mínima variância e carteira tangente pelas fórmulas fechadas (`cap06/l02_formulas_fechadas.py`)

```
r = retornos("2015", "2024")
mu, cov = insumos(r)
rf = cdi_anual("2015", "2024")

um = np.ones(len(mu))
x = np.linalg.solve(cov, um)         # resolve Σ x = 1
w_mv = x / x.sum()                   # mínima variância global
y = np.linalg.solve(cov, mu - rf)    # resolve Σ y = μ - r_f
w_tg = y / y.sum()                   # carteira tangente
```

```
CDI médio do período: 9.01% a.a.
         mu   vol  w_mv   w_tg
PETR4  36.9  47.4  -3.6  113.9
VALE3  25.3  43.7  13.1   24.5
ITUB4  16.8  30.5  29.0  -10.6
WEGE3  29.8  33.5  22.3  178.8
ABEV3   4.7  26.4  45.8 -222.1
BBAS3  22.4  40.6  -6.1  -16.4
MGLU3  34.3  68.3  -0.5   31.9
mín. variância  retorno   14.1%  vol  22.1%  Sharpe 0.23
tangente        retorno   96.6%  vol  91.8%  Sharpe 0.95
```

O Código 6.1 aplica as duas fórmulas às sete ações, com dados de 2015 a 2024 e, como R^f, o CDI do mesmo período, baixado do Banco Central por `cap06/baixar_cdi.py`: acumula-se a taxa entre pregões consecutivos das sete ações e multiplica-se a média por 252, a mesma base das médias dos retornos (as funções `retornos`, `insumos` e `cdi_anual` estão em `cap06/carteira.py`). A carteira de mínima variância é razoável: concentra-se nas ações menos voláteis e tem volatilidade de 22,1%, abaixo do piso de 24,2% da carteira de pesos iguais. A carteira tangente é outra história. Ela manda vender a descoberto o equivalente a 222,1% do patrimônio em ABEV3, além de 16,4% em BBAS3 e 10,6% em ITUB4, para comprar 178,8% em WEGE3 e 113,9% em PETR4, e promete 96,6% ao ano com volatilidade de 91,8%. É uma posição que nenhum gestor montaria. A fórmula foi aplicada corretamente, e a Seção 6.7 mostra que o defeito está nos insumos.

6.6 Otimização numérica com restrições

Carteiras reais têm restrições que as fórmulas fechadas ignoram: proibição de venda a descoberto, teto por ativo, limites por setor. Com elas o problema (6.5) deixa de ter solução analítica e passa a ser resolvido numericamente. Ele continua sendo um problema bem-comportado: a função objetivo é quadrática e convexa, e as restrições são lineares. O Código 6.2 usa o método SLSQP (programação quadrática sequencial) da `scipy.optimize`, que aceita restrições de igualdade e limites para cada variável.

Código 6.2 Otimizadores de carteira com soma 1 e sem venda a descoberto (`cap06/carteira.py`)

```python
def vol(w, cov):
    return np.sqrt(w @ cov @ w)


SOMA_1 = {"type": "eq", "fun": lambda w: w.sum() - 1}


def otimizar(objetivo, n, restricoes=(), teto=1.0):
    """Minimiza `objetivo` com soma 1 e pesos entre 0 e teto."""
    res = minimize(objetivo, np.full(n, 1 / n), method="SLSQP",
                   bounds=[(0, teto)] * n,
                   constraints=[SOMA_1, *restricoes],
                   options={"ftol": 1e-12, "maxiter": 500})
    assert res.success, res.message
    return res.x


def minima_variancia(cov, teto=1.0):
    return otimizar(lambda w: w @ cov @ w, len(cov), teto=teto)


def maximo_sharpe(mu, cov, rf, teto=1.0):
    def menos_sharpe(w):
        return -(w @ mu - rf) / vol(w, cov)
    return otimizar(menos_sharpe, len(mu), teto=teto)
```

A função `otimizar` parte de pesos iguais e impõe duas condições: a soma dos pesos é 1 (`SOMA_1`) e cada peso fica entre zero e um teto (`bounds`). A carteira de máximo Sharpe é obtida minimizando o índice com sinal trocado. Cada ponto da fronteira é uma minimização de variância com a restrição adicional de retorno-alvo, passada em `restricoes` pela função `fronteira` do mesmo arquivo. Sem os limites de sinal, o mesmo método reproduz os pesos da Equação (6.6) com diferença inferior a 10^{-7} (conferência em `cap06/conf_diversos.py`). Aplicada aos mesmos dados de 2015 a 2024, por `cap06/l03_otimizacao.py`, ela devolve:

```
        1/N  mín. var.  máx. Sharpe
PETR4  14.3        0.0         31.7
VALE3  14.3       11.6          7.1
ITUB4  14.3       21.1          0.0
WEGE3  14.3       21.7         53.3
ABEV3  14.3       45.7          0.0
BBAS3  14.3        0.0          0.0
MGLU3  14.3        0.0          8.0
1/N          retorno  24.3%  vol  27.8%  Sharpe 0.55
mín. var.    retorno  15.0%  vol  22.2%  Sharpe 0.27
máx. Sharpe  retorno  32.1%  vol  30.1%  Sharpe 0.77
```

A proibição de venda a descoberto domou os pesos, ao custo de reduzir o Sharpe prometido de 0,95 para 0,77. Ainda assim a carteira de máximo Sharpe põe 85% do patrimônio em duas ações e zera três das sete. A Figura 6.4 reúne os resultados.

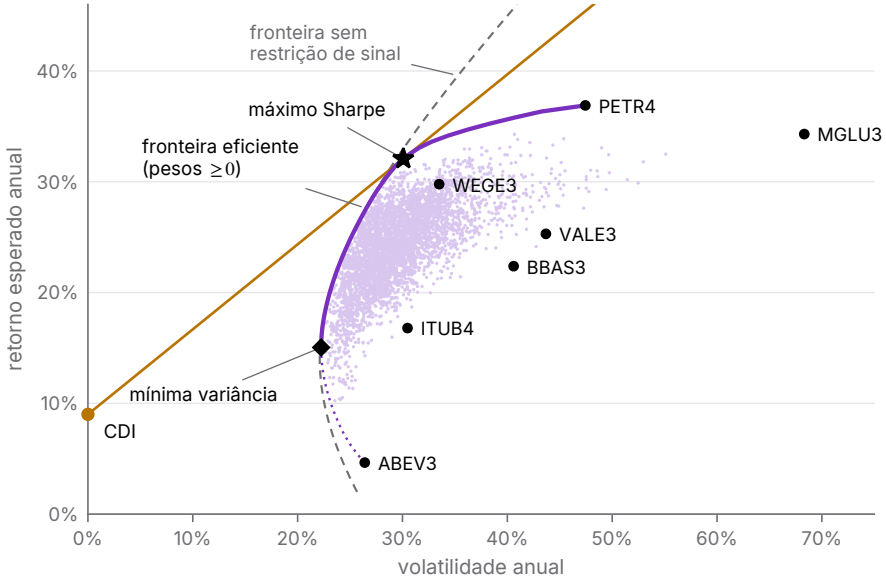


Figura 6.4 Fronteira eficiente das sete ações, com insumos de 2015 a 2024. A curva roxa cheia é a fronteira sem venda a descoberto; a tracejada cinza, a fronteira sem restrição de sinal. A reta parte do CDI médio de 2015 a 2024 (9,01% ao ano) e tangencia a fronteira na carteira de máximo Sharpe. A nuvem lilás são 4 000 carteiras com pesos sorteados.

Armadilha **A nuvem de carteiras aleatórias não é a fronteira**

Sortear milhares de pesos e ficar com o melhor resultado é uma boa ilustração e um mau otimizador. Na Figura 6.4, o maior Sharpe entre as 4 000 carteiras sorteadas é 0,74, contra 0,77 da solução ótima, e a nuvem nem se aproxima dos extremos da fronteira, onde a carteira se concentra em poucos ativos. Com sete ações o erro é pequeno; com cinquenta, as carteiras sorteadas ficam todas parecidas com a de pesos iguais, e a nuvem deixa de dizer qualquer coisa sobre a fronteira.

No repositório

Três scripts da aula 6 cobrem esta parte: `aula6/matrix_cov.py` (mapa de calor da covariância, convertendo ativos em dólar para reais), `aula6/marco.py` (fronteira de três moedas e uma ação, com SLSQP) e `markowitz.py`, na pasta `aulas/Aulas/markowitz/` (painel completo em Plotly). Ao estudá-los, repare em três simplificações: `marco.py` colore a nuvem pela razão entre retorno e volatilidade, sem subtrair a taxa livre de risco; `markowitz.py` usa médias de log-retornos como $\boldsymbol{\mu}$ e uma taxa livre de risco fixa de 10,75%; e os dois avaliam a carteira na mesma amostra em que ela foi otimizada.

6.7 O maximizador de erros

Tudo o que foi feito até aqui supôs que $\boldsymbol{\mu}$ e $\boldsymbol{\Sigma}$ são conhecidos. Não são: foram estimados com dez anos de dados, e a qualidade dessas estimativas é muito desigual.

A covariância é estimada com precisão razoável, porque cada dia traz informação nova sobre o quanto os ativos oscilam. A média é outro caso. O erro-padrão da média anualizada é $\sigma/\sqrt{T}$, com T medido em anos, e não melhora com dados mais frequentes (Merton, 1980): o que importa para a média

é quanto o preço andou do início ao fim da amostra. Para WEGE3, com volatilidade de 33,5% e 2 472 pregões (9,8 anos pela convenção de 252 dias), o erro-padrão é de 10,7 pontos percentuais. O intervalo de 95% de confiança para o retorno esperado vai de 8,8% a 50,7% ao ano. Os intervalos das sete ações se sobrepõem todos (`cap06/conf_diversos.py`): dez anos de dados dizem muito pouco sobre qual delas tem o maior retorno esperado.

Intuição

O otimizador não sabe que os insumos são estimativas. Ele procura o ativo com retorno alto, variância baixa e covariância baixa com os demais, e são justamente esses os ativos cujos números mais provavelmente foram superestimados por sorte na amostra. A otimização seleciona os erros de estimação favoráveis e aposta neles com o máximo de convicção. Michaud (1989) resumiu o fenômeno chamando os otimizadores de média-variância de "maximizadores de erros de estimação".

6.7.1 Duas janelas, duas carteiras

O primeiro sintoma é a instabilidade. O script `cap06/l04_duas_janelas.py` estima as carteiras de máximo Sharpe (MS) e de mínima variância (MV), sem venda a descoberto, em duas janelas vizinhas de cinco anos.

```
       mu 15-19  mu 20-24  MS 15-19  MS 20-24  MV 15-19  MV 20-24
PETR4      36.6      37.2       3.5      49.2       0.0       0.0
VALE3      33.7      16.9       9.4       0.0       5.2      21.4
ITUB4      24.0       9.6       0.0       0.0      12.4      25.6
WEGE3      28.2      31.3      34.2      50.8      25.5      17.8
ABEV3       9.7      -0.3       0.0       0.0      56.3      35.2
BBAS3      31.8      13.1       0.0       0.0       0.0       0.0
MGLU3     101.9     -32.9      53.0       0.0       0.7       0.0

MS de 2015-19: 62.3% do patrimônio trocaria de ativo em 2020-24
  prometido: retorno  68.1%  vol 39.8%  Sharpe  1.47
  realizado: retorno  -3.8%  vol 44.8%  Sharpe -0.27

MV de 2015-19: 29.5% do patrimônio trocaria de ativo em 2020-24
  prometido: retorno  18.0%  vol 19.4%  Sharpe  0.43
  realizado: retorno   9.6%  vol 25.5%  Sharpe  0.05
```

Entre 2015 e 2019, MGLU3 teve média de 101,9% ao ano e recebeu 53,0% da carteira de máximo Sharpe. Nos cinco anos seguintes a média da mesma ação foi de $-32{,}9\%$. Quem montou a carteira no fim de 2019 esperava, pelas contas dentro da amostra, um Sharpe de 1,47; com os mesmos pesos, as médias e covariâncias de 2020 a 2024 dão $-0{,}27$. A carteira de mínima variância também decepcionou (a volatilidade realizada foi de 25,5%, contra 19,4% estimados, em um período que inclui a pandemia), mas sua estrutura mudou muito menos, como mostra a Figura 6.5: as médias históricas quase nada dizem sobre as médias futuras, enquanto as ações menos voláteis de uma janela tendem a continuar entre as menos voláteis na seguinte.

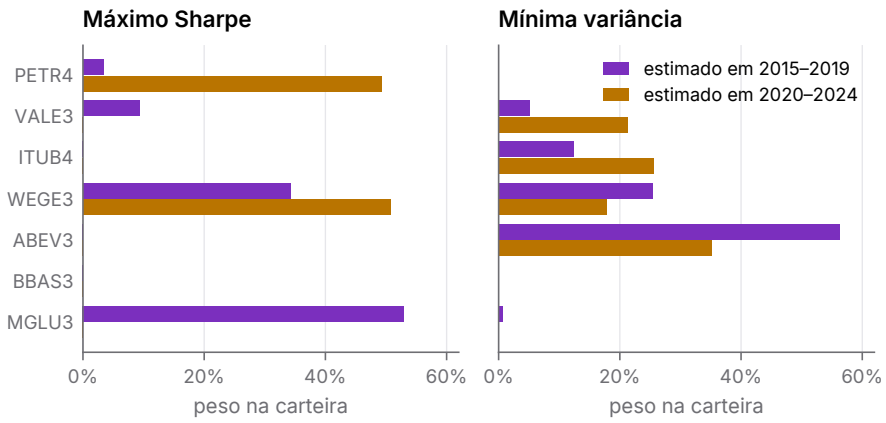


Figura 6.5 Pesos ótimos estimados em duas janelas consecutivas de cinco anos. A carteira de máximo Sharpe muda quase por completo de uma janela para a outra; a de mínima variância, que não usa médias, muda bem menos (29,5% do patrimônio, contra 62,3%) e mantém os mesmos ativos.

Armadilha **Otimizar e avaliar na mesma amostra**

O Sharpe de 0,77 da Figura 6.4 não é uma previsão. Ele é o melhor resultado que se poderia ter obtido conhecendo de antemão as médias de 2015 a 2024, e por construção nenhuma outra carteira sem venda a descoberto o supera nessa amostra. O desempenho que interessa é o de uma carteira montada só com dados anteriores à data da decisão. O Capítulo 7 trata desse princípio em detalhe.

6.7.2 Um teste fora da amostra

O teste honesto imita o que um gestor faria. No fechamento do último pregão de cada mês, estimam-se $\hat{\boldsymbol{\mu}}$ e $\hat{\boldsymbol{\Sigma}}$ com os 756 pregões até aquele dia (três anos) e calculam-se os pesos; supõe-se que a carteira é montada a esse mesmo preço de fechamento e mantida até o fim do mês seguinte. É a *execução no fechamento do sinal* do Capítulo 7: não usa informação futura, mas é uma hipótese otimista, testada adiante com um pregão de atraso. O Código 6.3 faz isso para seis regras: pesos iguais; mínima variância com a covariância amostral e com a covariância encolhida de Ledoit e Wolf; pesos proporcionais ao inverso da volatilidade; e máximo Sharpe, sem teto e com teto de 30% por ação (as três regras adicionais são explicadas na Seção 6.8). Custos de transação são ignorados, e por isso a tabela informa o giro: a fração do patrimônio que troca de ativo a cada rebalanceamento.

Este giro é metade da soma das variações absolutas dos pesos. O dos Capítulos 7 e 8 conta as duas pontas, a venda e a compra, e seria o dobro.

Código 6.3 Regras de alocação comparadas fora da amostra (`cap06/experimento.py`)

```
def pesos(passado, rf):
    """Pesos de cada regra, estimados só com `passado`."""
    n = passado.shape[1]
    mu, cov = insumos(passado)
    lw = LedoitWolf().fit(passado.values).covariance_ * 252
    inv_vol = 1 / np.sqrt(np.diag(cov))
    return {"1/N": np.full(n, 1 / n),
            "mín. variância": minima_variancia(cov),
            "mín. var. (LW)": minima_variancia(lw),
            "inverso da vol": inv_vol / inv_vol.sum(),
            "máx. Sharpe": maximo_sharpe(mu, cov, rf),
            "máx. Sharpe 30%": maximo_sharpe(mu, cov, rf, 0.30)}
```

A função `simular`, do mesmo arquivo, aplica essas regras mês a mês e devolve os retornos diários e o giro de cada uma:

```
def simular(r, cdi, atraso=0):
    """Pesos calculados com dados até o último fechamento do
    mês anterior (ou `atraso` pregões antes), assumidos nesse
    fechamento e mantidos por um mês. Devolve retornos e giro."""
    diario, giro, antes = {}, {}, {}
    meses = r.index.to_period("M")
    for mes in meses.unique():
        hoje = np.argmax(meses == mes)       # 1º pregão do mês
        if hoje < JANELA + atraso:
            continue
        fim = hoje - atraso                  # não inclui `fim`
        passado = r.iloc[fim - JANELA:fim]
        rf = cdi.iloc[fim - JANELA:fim].mean() * 252
        cotas = (1 + r[meses == mes]).cumprod()
        for nome, w in pesos(passado, rf).items():
            valor = cotas @ w
            ret = valor.pct_change().fillna(valor.iloc[0] - 1)
            diario.setdefault(nome, []).append(ret)
            if nome in antes:
                troca = np.abs(w - antes[nome]).sum() / 2
                giro.setdefault(nome, []).append(troca)
            antes[nome] = w * cotas.iloc[-1] / valor.iloc[-1]
    diario = pd.DataFrame(
        {k: pd.concat(v) for k, v in diario.items()})
    return diario, pd.DataFrame(giro)
```

```
02/03/2015 a 30/12/2025
                ret a.a.  vol a.a.  Sharpe  pior queda  giro/mês
1/N                23.41     26.98    0.57      -45.60      3.41
mín. variância     10.90     21.64    0.15      -35.06      3.06
mín. var. (LW)     11.61     21.59    0.18      -34.88      2.99
inverso da vol     19.87     24.43    0.48      -39.86      3.06
máx. Sharpe        41.33     34.68    0.90      -53.58     11.32
máx. Sharpe 30%    30.46     26.82    0.78      -44.15      8.41
```

A saída é a primeira parte do que imprime `cap06/l05_fora_da_amostra.py`, e contraria a expectativa criada pela subseção anterior: com as sete ações, a regra de máximo Sharpe teve o maior índice fora da amostra, 0,90 contra 0,57

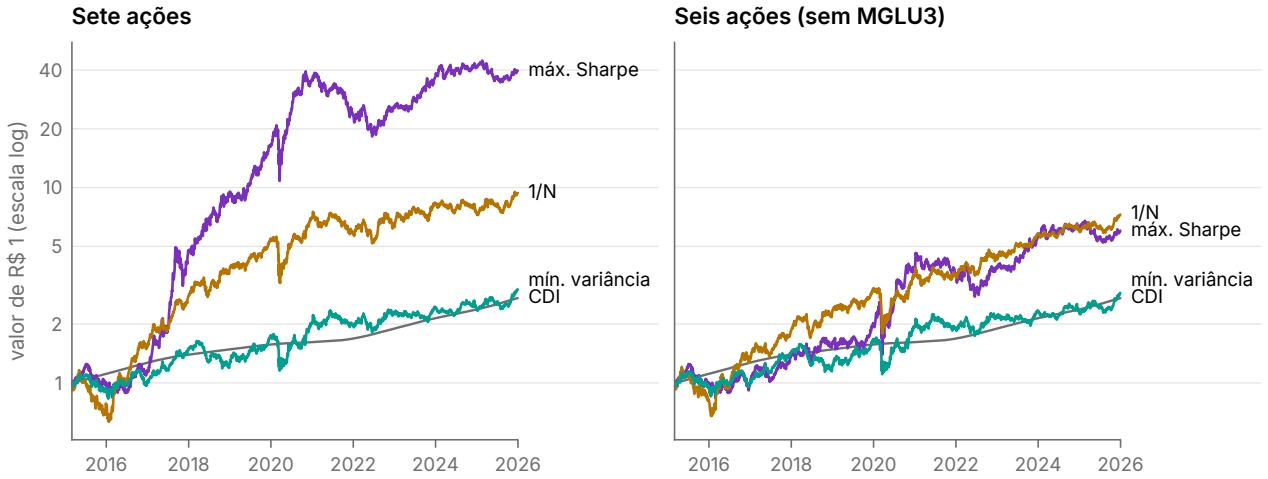


Figura 6.6 Valor de R$ 1 aplicado no fim de fevereiro de 2015 segundo três regras de alocação e no CDI (linha cinza), com pesos recalculados todo mês a partir dos três anos anteriores. À esquerda, o universo de sete ações; à direita, o mesmo exercício sem MGLU3. A ordem entre máximo Sharpe e $1/N$ se inverte com a retirada de um único ativo.

da carteira de pesos iguais. Antes de comemorar, veja o que o mesmo script imprime ao repetir a simulação sem uma única ação, MGLU3:

```
sem MGLU3:
                 ret a.a.  vol a.a.  Sharpe  pior queda  giro/mês
1/N                 20.49     25.37    0.49       -43.94      2.67
mín. variância      10.45     21.64    0.13       -35.06      2.95
mín. var. (LW)      10.90     21.58    0.15       -34.89      2.88
inverso da vol      18.46     23.73    0.44       -39.39      2.62
máx. Sharpe         18.36     28.86    0.40       -46.27     14.42
máx. Sharpe 30%     19.75     25.07    0.47       -40.61      9.46
```

O Sharpe da regra de máximo Sharpe cai para 0,40, abaixo do de $1/N$ (0,49), com mais volatilidade, queda máxima maior e um giro mais de cinco vezes maior. O resultado do primeiro bloco se deve quase todo a uma aposta concentrada em uma ação cujo preço ajustado se multiplicou por 370 entre o fim de 2015 e o fim de 2020, e que perdeu 97% do valor nos quatro anos seguintes (Figura 6.6).

Três leituras cabem aqui, todas modestas.

- **A diferença não é estatisticamente distinguível de zero.** Um *bootstrap* em blocos da diferença de Sharpe entre máximo Sharpe e $1/N$ no universo de sete ações (o *bootstrap* do Capítulo 5, sorteando blocos de 21 pregões seguidos em vez de dias isolados, para preservar a dependência entre dias vizinhos) dá um intervalo de 95% de $-0{,}27$ a 0,84 (`cap06/conf_sharpe.py`). Quase onze anos são pouco para separar duas regras, e o resultado depende do período: o Sharpe de $1/N$ foi de 1,08 entre 2015 e 2019 e de 0,14 entre 2020 e 2025. A hipótese de execução pesa pouco: com um pregão de atraso entre o cálculo dos pesos e a montagem da carteira, os índices são 0,88 e 0,57 com sete ações e 0,41 e 0,49 sem MGLU3.
- **O que se estima bem funciona fora da amostra.** As duas carteiras de mínima variância tiveram, de fato, a menor volatilidade realizada (21,6%) e a menor queda máxima. A promessa sobre o risco foi cumprida. O retorno baixo veio da concentração em ABEV3 (44% da carteira em média), que rendeu 1,6% ao ano no período (`cap06/conf_sharpe.py`), e mostra que minimizar risco não é o mesmo que ter bom desempenho.

- **O universo já contém uma escolha.** As sete ações foram selecionadas hoje, e toda regra aplicada a elas herda o viés de sobrevivência (Seção 7.6), inclusive $1/N$: seus 23,4% ao ano não são o que um investidor obteria escolhendo ações em 2015.

Esses números conversam com o estudo mais citado sobre o tema. DeMiguel et al. (2009) compararam a regra $1/N$ com 14 modelos de otimização em sete conjuntos de dados e concluíram que nenhum deles a superava de forma consistente fora da amostra em índice de Sharpe, equivalente-certeza (o retorno sem risco que o investidor aceitaria em troca da carteira) ou giro. Estimaram ainda que, para uma carteira de 25 ativos, a otimização com médias e covariâncias amostrais precisaria de cerca de 3 000 meses de dados para vencer $1/N$. A conclusão comedida é que a otimização de média-variância com médias históricas é uma aposta de baixa confiabilidade, e não que ela sempre perde: no nosso exemplo ela ganhou e perdeu, conforme a presença de uma ação.

6.8 Remédios contra o erro de estimação

Nenhum dos remédios a seguir cria informação que os dados não têm. Todos trocam um pouco de otimalidade teórica por estabilidade.

- **Restringir os pesos.** Proibir venda a descoberto e impor um teto por ativo impede que o otimizador transforme um erro de estimação em uma posição gigante. No experimento, o teto de 30% reduziu a volatilidade, a queda máxima e o giro da regra de máximo Sharpe nos dois universos. Jagannathan e Ma (2003) mostraram que, na carteira de mínima variância, proibir pesos negativos equivale a encolher as covariâncias estimadas mais altas, o que explica por que uma restrição "errada" em teoria pode ajudar na prática.
- **Abrir mão das médias.** A carteira de mínima variância e a de pesos iguais não usam $\hat{\boldsymbol{\mu}}$, o insumo mais ruidoso. Chopra e Ziemba (1993) estimaram que erros nas médias custam ao investidor cerca de dez vezes mais que erros de mesma magnitude nas variâncias, e mais ainda em relação às covariâncias; a razão exata depende da tolerância a risco do investidor. É um argumento a favor de regras que deixam as médias de fora.
- **Encolher a covariância.** A matriz amostral $\mathbf{S}$ tem $N(N+1)/2$ parâmetros, e quando o número de ativos se aproxima do número de observações seus valores extremos são em boa parte ruído. O **encolhimento** (*shrinkage*) substitui $\mathbf{S}$ por uma média ponderada com um alvo $\mathbf{F}$ muito mais simples,

$$\hat{\boldsymbol{\Sigma}} = \alpha\,\mathbf{F} + (1-\alpha)\,\mathbf{S}, \qquad 0 \le \alpha \le 1, \tag{6.10}$$

  com a intensidade α escolhida para minimizar o erro quadrático esperado. Ledoit e Wolf (2004a) propõem como alvo a matriz em que todas as correlações são iguais à correlação média. A classe `LedoitWolf` da biblioteca scikit-learn implementa outra variante dos mesmos autores, em que o alvo é a identidade multiplicada pela variância média (Ledoit e Wolf, 2004b). No nosso experimento o efeito foi quase nulo, e a razão é instrutiva: com 7 ativos e 756 observações a matriz amostral já é bem estimada, e a intensidade calculada é de apenas 0,014. Com a mesma matriz estimada em 21 pregões ela sobe para 0,155 (`cap06/conf_diversos.py`). O encolhimento é um remédio para carteiras com dezenas ou centenas de ativos.
- **Paridade de risco.** Em vez de igualar os pesos, iguala-se o risco. A **contribuição de risco** do ativo i é $\mathrm{CR}_i = w_i\,(\boldsymbol{\Sigma}\mathbf{w})_i/\sigma_p$, e as contribuições somam

σ_p. Na carteira de pesos iguais de 2015 a 2024, MGLU3 tem 14,3% do patrimônio e responde por 23,0% do risco, enquanto ABEV3 responde por 7,7%. A **paridade de risco** procura os pesos que tornam todas as contribuições iguais. Quando todas as correlações são iguais, a solução é dar a cada ativo um peso proporcional ao inverso de sua volatilidade (Maillard et al., 2010), e essa versão simplificada é a regra "inverso da vol" do experimento. Sua volatilidade ficou entre a da mínima variância e a de $1/N$, ordenação que os mesmos autores demonstram para a paridade de risco exata.

6.8.1 Black–Litterman: partir do equilíbrio

O quinto remédio ataca o problema pela média. O modelo de Black e Litterman (1992) tem três passos. Primeiro, em vez de médias históricas, parte-se dos retornos de equilíbrio: os retornos em excesso $\boldsymbol{\pi}$ que fariam um investidor de média-variância, com aversão a risco δ, escolher exatamente a carteira de mercado $\mathbf{w}_m$. Invertendo a condição de primeira ordem, $\boldsymbol{\pi} = \delta\,\boldsymbol{\Sigma}\,\mathbf{w}_m$. Segundo, o analista declara suas visões na forma $\mathbf{P}\boldsymbol{\mu} = \mathbf{q} + \boldsymbol{\varepsilon}$, em que cada linha de $\mathbf{P}$ é uma combinação de ativos, $\mathbf{q}$ é o retorno que ele espera dessa combinação e a covariância $\boldsymbol{\Omega}$ de $\boldsymbol{\varepsilon}$ mede sua incerteza. Terceiro, as duas fontes são combinadas como se combinam duas medições ruidosas da mesma grandeza, cada uma pesada pelo inverso de sua variância. O equilíbrio faz o papel de informação anterior, com covariância $\tau\boldsymbol{\Sigma}$, e as visões, o de dado novo (é a regra de Bayes aplicada a distribuições normais). O resultado, que está no apêndice do artigo, é

$$\boldsymbol{\mu}_{\mathrm{BL}} = \left[(\tau\boldsymbol{\Sigma})^{-1} + \mathbf{P}^{\top}\boldsymbol{\Omega}^{-1}\mathbf{P}\right]^{-1}\left[(\tau\boldsymbol{\Sigma})^{-1}\boldsymbol{\pi} + \mathbf{P}^{\top}\boldsymbol{\Omega}^{-1}\mathbf{q}\right]. \tag{6.11}$$

É uma média de $\boldsymbol{\pi}$ e das visões, ponderada pela precisão de cada uma. Sem visões, ou com visões muito incertas, $\boldsymbol{\mu}_{\mathrm{BL}} = \boldsymbol{\pi}$ e a carteira ótima é a de mercado; quanto mais confiante a visão, mais os pesos se afastam do mercado, e apenas na direção dela.

O script `cap06/l06_black_litterman.py` aplica a fórmula às sete ações, com uma única visão: WEGE3 renderá 5 pontos percentuais a mais que ITUB4.

```
          PETR4  VALE3  ITUB4  WEGE3  ABEV3  BBAS3  MGLU3
pi (%)     24.6   18.0   16.0   12.7   10.4   22.1   31.1
mu_BL (%)  23.6   17.6   14.2   15.1   10.2   20.5   31.0
w_ref (%)  14.3   14.3   14.3   14.3   14.3   14.3   14.3
w_BL (%)   14.3   14.3    2.2   26.4   14.3   14.3   14.3
visão embutida no equilíbrio: -3.26%; a posteriori: 0.87%
```

O exemplo usa a carteira de pesos iguais (`w_ref`) no lugar da carteira de mercado, porque o conjunto de dados não traz o valor de mercado das empresas, e os valores $\delta = 2{,}5$ e $\tau = 0{,}05$. O artigo original não fixa esses números; eles e a escolha de $\boldsymbol{\Omega}$ proporcional à incerteza do equilíbrio são convenções da literatura posterior, e aqui entram como hipóteses. O equilíbrio embute que WEGE3 renderia 3,26 pontos a menos que ITUB4. A visão diz 5 pontos a mais, com incerteza $\boldsymbol{\Omega} = \mathbf{P}(\tau\boldsymbol{\Sigma})\mathbf{P}^{\top}$ igual à do equilíbrio, e o resultado fica no meio do caminho: 0,87. Os pesos ótimos, $(\delta\boldsymbol{\Sigma})^{-1}\boldsymbol{\mu}_{\mathrm{BL}}$, mudam somente nas duas ações da visão. Compare com a carteira tangente da Seção 6.5: o ponto de partida deixou de ser um vetor de médias ruidosas e passou a ser uma carteira que já faz sentido.

No repositório

O arquivo `aula6/litterman.py` aplica a Equação (6.11) a três ativos com duas visões, uma absoluta e uma relativa, e uma escolha análoga de $\boldsymbol{\Omega}$ (a diagonal de $\mathbf{P}(\tau\boldsymbol{\Sigma})\mathbf{P}^{\mathsf{T}}$). Dois cuidados ao rodá-lo: o comentário anuncia uma visão de 15% para PETR4 e o vetor `Q` traz 0,20; e os pesos negativos são zerados e renormalizados no fim, um atalho que não equivale a otimizar com a restrição.

6.9 Teste de estresse e análise de cenários

O VaR do Capítulo 5 responde a uma pergunta sobre dias comuns: qual perda não deve ser superada em 99% dos dias, *se o futuro próximo se parecer com a janela de estimação*. Ele nada diz sobre o tamanho das perdas além do quantil, e herda a calma ou a turbulência da janela usada. Um **teste de estresse** inverte a lógica. Em vez de perguntar pela probabilidade, fixa-se um cenário severo e plausível e calcula-se quanto a carteira perderia nele. Jorion (2007) e Hull (2018) tratam os dois instrumentos como complementares: o VaR para o risco do dia a dia, o estresse para os eventos que a janela não contém.

Definição 6.6 Teste de estresse

Um teste de estresse é a reavaliação da carteira atual sob um conjunto de variações extremas, mas plausíveis, dos fatores de risco. O resultado é uma perda em reais ou em porcentagem do patrimônio, sem probabilidade associada.

6.9.1 Cenários históricos

O cenário mais fácil de defender é um que já aconteceu. Tomam-se os retornos dos ativos em um episódio do passado e aplicam-se aos pesos de hoje:

$$\text{resultado no cenário} = \sum_{i=1}^{N} w_i^{\text{hoje}}\, R_i^{\text{episódio}}. \tag{6.12}$$

O Código 6.4 faz isso (o dicionário `EPISODIOS` guarda as datas de início e fim) para três carteiras “de hoje”, estimadas com dados de 2023 a 2025, e quatro episódios escolhidos entre as maiores quedas do Ibovespa no conjunto de dados (a maior de todas, de 48,6% entre novembro de 2010 e janeiro de 2016, fica de fora porque uma queda de cinco anos não serve de cenário de estresse): a pandemia de Covid-19, do pico de 23 de janeiro ao fundo de 23 de março de 2020; o *Joesley Day*, 18 de maio de 2017, pregão seguinte à divulgação de uma gravação envolvendo o presidente da República; a queda de fevereiro a junho de 2018, período que inclui a greve dos caminhoneiros; e a queda de junho de 2021 a julho de 2022, durante o ciclo de alta da Selic.

Código 6.4 Cenários históricos aplicados às carteiras atuais (`cap06/l07_estresse_historico.py`)

```
linhas = {}
for nome, (ini, fim) in EPISODIOS.items():
    choque = p.loc[fim] / p.loc[ini] - 1   # retorno no episódio
    linhas[nome] = choque[ACOES] @ hoje
    linhas[nome]["Ibovespa"] = choque["^BVSP"]
    linhas[nome]["pregões"] = len(p.loc[ini:fim, "PETR4.SA"]
                                  .dropna()) - 1
cenarios = pd.DataFrame(linhas).T
```

```
              PETR4  VALE3  ITUB4  WEGE3  ABEV3  BBAS3  MGLU3
1/N            14.3   14.3   14.3   14.3   14.3   14.3   14.3
mín. var.      12.2   19.4   15.1   16.5   25.3   11.5    0.0
máx. Sharpe    35.6    0.0   64.4    0.0    0.0    0.0    0.0

               1/N  mín. var.  máx. Sharpe  Ibovespa  pregões
Covid-19     -41.5      -38.7        -46.7     -46.8       40
Joesley Day  -10.1       -7.6        -13.4      -8.8        1
fev-jun/2018 -15.1      -19.0        -30.1     -20.4       77
2021-2022    -19.3      -12.3          3.8     -26.5      276
```

A comparação com o VaR é o ponto central, e o mesmo script a imprime em seguida:

```
VaR histórico 99%, 1 dia (2023-2025), em %:
1/N            2.89
mín. var.      2.10
máx. Sharpe    2.75
escalado pela raiz de 40 pregões, em %:
1/N            18.3
mín. var.      13.3
máx. Sharpe    17.4
```

O VaR histórico de 99% para um dia da carteira de pesos iguais, medido na janela calma de 2023 a 2025, é de 2,89%. No *Joesley Day* ela teria perdido 10,1% em um pregão, três vezes e meia esse valor. Escalado pela raiz do tempo para 40 pregões, o VaR sugere 18,3%; a repetição da Covid-19 custaria 41,5%, mais que o dobro. Os VaR das duas carteiras otimizadas foram medidos na mesma janela em que elas foram otimizadas e são, por isso, otimistas. Na crise, as perdas diárias são maiores e se acumulam na mesma direção, duas coisas que a regra da raiz do tempo descarta. Repare também que a carteira de mínima variância, desenhada para ter pouco risco em dias normais, perderia quase tanto quanto as outras na pandemia (Figura 6.7).

O cenário histórico tem a virtude de ser incontestável (aconteceu) e de trazer correlações de crise já embutidas. Seu limite é o mesmo: a próxima crise não repetirá a anterior, e ativos que não existiam na época ficam sem retorno para aplicar.

6.9.2 Cenários hipotéticos e análise de sensibilidade

No cenário hipotético o analista define o choque. O risco é esquecer que um choque em um fator arrasta os demais. Há duas formas simples de propagar um choque de maneira coerente com os dados.

A primeira usa os betas. Se $\beta_i = \mathrm{Cov}(R_i, R_m)/\mathrm{Var}(R_m)$ é a sensibilidade do ativo i ao índice de mercado, um choque s no índice produz, em média, $\beta_i\, s$ no ativo e $s \sum_i w_i \beta_i$ na carteira. A segunda generaliza a ideia para um choque em qualquer ativo ou conjunto de ativos: se o bloco 1 sofre o choque $\mathbf{s}$, o retorno esperado do bloco 2 é

$$\mathbb{E}[\mathbf{R}_2 \mid \mathbf{R}_1 = \mathbf{s}] = \boldsymbol{\Sigma}_{21}\, \boldsymbol{\Sigma}_{11}^{-1}\, \mathbf{s}, \tag{6.13}$$

medindo os retornos como desvios em relação à média. A expressão é a média condicional exata sob normalidade conjunta e, fora dela, a melhor previsão linear. Com um único ativo chocado ela se reduz a σ_{i1}/σ_1^2 vezes o choque, que é um beta em relação a esse ativo.

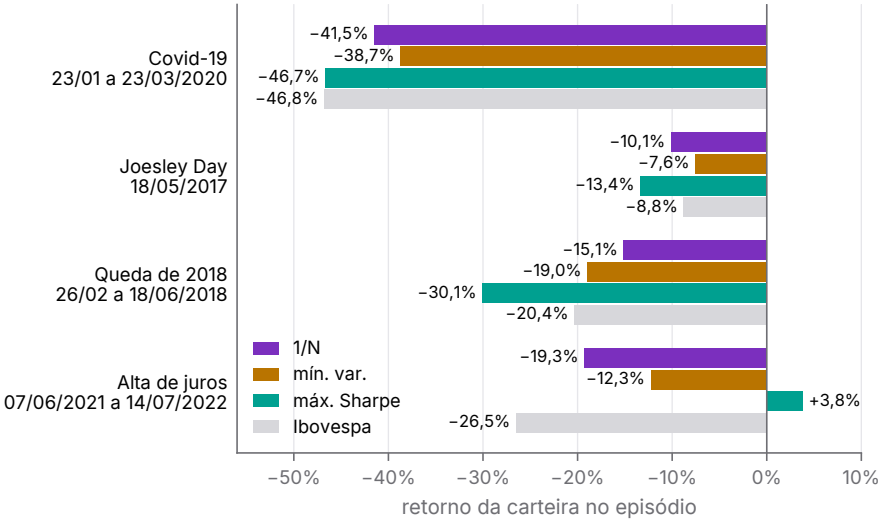


Figura 6.7 Retorno que três carteiras atuais (pesos estimados com dados de 2023 a 2025) teriam em quatro episódios históricos, ao lado do retorno do Ibovespa. A carteira de máximo Sharpe, com duas ações, teria ganhado 3,8% em um episódio e é a pior nos outros três.

Código 6.5 Propagação de choques por betas e por covariância (`cap06/l08_estresse_hipotetico.py`)

```
# (a) choque no índice, propagado pelos betas
beta = acoes.apply(lambda x: x.cov(ibov)) / ibov.var()
via_beta = beta * -0.20

# (b) choque em um ativo, propagado pela covariância
i = ACOES.index("PETR4.SA")
via_cov = pd.Series(cov[:, i] / cov[i, i] * -0.30, index=ACOES)
```

```
                PETR4  VALE3  ITUB4  WEGE3  ABEV3  BBAS3  MGLU3
beta             0.88   0.80   1.01   0.57   0.63   0.88   2.44
Ibov -20% (%)  -17.51 -15.98 -20.25 -11.33 -12.66 -17.70 -48.74
PETR4 -30% (%) -30.00  -5.54  -4.94  -3.09  -2.90  -7.91 -11.36

perda da carteira em %:
             Ibov -20%  PETR4 -30%
1/N              -20.6        -9.4
mín. var.        -15.4        -7.6
máx. Sharpe      -19.3       -13.9

correlação média: 0.21 em 2023-2025, 0.75 na Covid-19
volatilidade anual da carteira em %:
             λ = 0.00  λ = 0.68  λ = 1.00
1/N              17.9      26.9      30.3
mín. var.        13.7      20.9      23.6
máx. Sharpe      18.2      21.7      23.1
```

Com betas estimados de 2023 a 2025, uma queda de 20% do Ibovespa custaria 20,6% à carteira de pesos iguais e 15,4% à de mínima variância, que evita a ação de beta mais alto. Um tombo de 30% em PETR4, sozinho, levaria 4,3% da carteira de pesos iguais (0,143 × 30%); propagado pelas covariâncias, 9,4%. Ignorar o contágio teria subestimado a perda em mais da metade.

A **análise de sensibilidade** é o caso particular em que se move um fator por vez em uma grade de valores (o índice a −10%, −20%, −30%) e se tabula

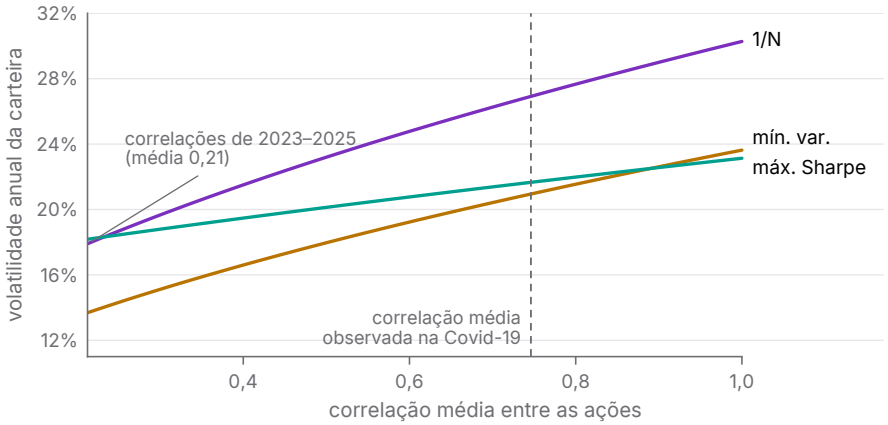


Figura 6.8 Volatilidade das três carteiras atuais quando as correlações de 2023 a 2025 são deslocadas em direção a 1, com as volatilidades individuais mantidas. A carteira de máximo Sharpe, concentrada em duas ações, é a que menos muda: tinha pouca diversificação a perder.

o resultado. Para uma carteira de ações a resposta é linear e a tabela é uma régua. Ela se torna informativa quando há instrumentos não lineares, como opções, em que a perda para -20% não é o dobro da perda para -10%.

6.9.3 Estresse de correlação

Os betas e as covariâncias acima foram medidos em anos calmos, e esse é o ponto fraco dos cenários hipotéticos. Em crises as correlações sobem: a média entre as sete ações foi de 0,21 entre 2023 e 2025 e de 0,75 nos 40 pregões da queda da Covid-19. O ganho de diversificação medido em tempos normais encolhe exatamente quando se precisa dele.

O estresse de correlação mantém as volatilidades e desloca a matriz de correlação $\mathbf{C}$ em direção à matriz $\mathbf{J}$ em que todas as correlações valem 1:

$$\mathbf{C}(\lambda) = (1-\lambda)\,\mathbf{C} + \lambda\,\mathbf{J}, \qquad 0 \le \lambda \le 1. \tag{6.14}$$

Com $\lambda = 1$ toda a diversificação desaparece e a volatilidade de uma carteira sem posições vendidas passa a ser a média ponderada das volatilidades, como na reta de $\rho = 1$ da Figura 6.1. A última tabela da saída anterior traz o resultado: com as volatilidades de 2023 a 2025 e a correlação média da pandemia ($\lambda = 0{,}68$), a volatilidade da carteira de pesos iguais sobe de 17,9% para 26,9%, e a de mínima variância, de 13,7% para 20,9%. A Figura 6.8 mostra o caminho completo. Em uma crise real as volatilidades também sobem, de modo que esse exercício isola apenas uma parte do problema.

Armadilha **Alterar correlações à mão**

Uma matriz de correlação precisa ser positiva semidefinida; caso contrário alguma carteira teria variância negativa. Trocar um elemento isolado (“e se a correlação entre PETR4 e VALE3 fosse 0,95?”) pode violar essa condição sem aviso, e o cálculo seguinte devolve um número sem sentido ou a raiz de um negativo. A mistura da Equação (6.14) é segura porque combina duas matrizes positivas semidefinidas com pesos não negativos. Depois de qualquer outra alteração, confira os autovalores com `np.linalg.eigvalsh`: nenhum pode ser negativo.

6.9.4 Como reportar

Um relatório de estresse útil cabe em uma página e responde a cinco perguntas: qual é a carteira e em que data foi avaliada; quais cenários foram aplicados, com as hipóteses de cada um escritas por extenso (datas do episódio, tamanho do choque, janela dos betas); quanto se perde em cada cenário, em reais e em porcentagem do patrimônio; quais posições respondem pela maior parte da perda no pior cenário; e como o resultado se compara ao limite de perda que o investidor declarou tolerar. Se a perda no pior cenário supera esse limite, o relatório deve terminar com uma proposta: reduzir a posição, comprar proteção ou aceitar o risco por escrito.

Três cuidados completam a prática. A lista de cenários deve ser fixada antes de olhar os resultados e mantida ao longo do tempo, para que as leituras sejam comparáveis. Cenários que só machucam carteiras alheias são inúteis: pelo menos um deve atacar a concentração específica da sua (na carteira de máximo Sharpe de hoje, bancos e petróleo). E vale fazer a pergunta ao contrário, no chamado teste de estresse reverso: que combinação de choques produziria uma perda inaceitável, e quão implausível ela é?

No repositório

O notebook `aula6/Stress Testing e Análise de Cenários.ipynb` percorre as mesmas etapas com quatro ações: recorte de crises históricas, choques definidos à mão por cenário, simulação de Monte Carlo com decomposição de Cholesky e estresse de volatilidade multiplicando a matriz de covariância. Compare os choques escolhidos à mão com os que a Equação (6.13) produziria.

6.10 Entrega 2: relatório de risco

A segunda entrega do ciclo, prevista para a sexta semana, tem o seguinte enunciado na proposta do Núcleo: *cálculo de VaR, CVaR e drawdown para um portfólio; otimização de Markowitz com fronteira eficiente*. O produto é um notebook que roda de cima a baixo sobre uma carteira de pelo menos cinco ativos, com os pesos e o período declarados no início, e que contém:

- **Medidas de risco** (Capítulo 5). VaR e CVaR de um dia, a 95% e a 99%, por pelo menos dois métodos (histórico e paramétrico), em reais e em porcentagem do patrimônio; contagem de violações do VaR em uma janela que não foi usada para estimá-lo; série de drawdown com o drawdown máximo, sua duração e o tempo de recuperação.
- **Otimização** (este capítulo). Matriz de correlação, insumos $\hat{\boldsymbol{\mu}}$ e $\hat{\boldsymbol{\Sigma}}$ com a taxa livre de risco declarada, fronteira eficiente sem venda a descoberto, carteiras de mínima variância e de máximo Sharpe, e a comparação das duas com $1/N$ em um período posterior ao da estimação.
- **Estresse.** Pelo menos um cenário histórico e um hipotético aplicados à carteira escolhida, ao lado do VaR.
- **Interpretação.** Um parágrafo que diga qual carteira você manteria e por quê, o que os números não garantem (erro de estimação das médias, janela calma, universo escolhido hoje) e qual limite de perda a carteira respeita.

A avaliação segue a rubrica de quatro dimensões, com pesos iguais: rigor técnico, clareza, reprodutibilidade e interpretação. Ela está detalhada no Capítulo 8, junto da terceira entrega.

O que levar deste capítulo

- O retorno esperado de uma carteira é a média ponderada dos retornos dos ativos; a variância é $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$ e depende sobretudo das covariâncias.
- Diversificar elimina o risco específico e deixa o sistemático. Em uma carteira de pesos iguais a variância tende à covariância média, e a maior parte do ganho vem dos primeiros ativos.
- A carteira de mínima variância global é $\boldsymbol{\Sigma}^{-1}\mathbf{1}$ normalizado e a tangente é $\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - R^f\mathbf{1})$ normalizado; com restrições, resolve-se numericamente.
- Médias históricas são estimadas com erro enorme, e o otimizador amplifica esse erro. Pesos de máximo Sharpe são instáveis, e seu desempenho dentro da amostra não é previsão.
- Restrições de peso, encolhimento da covariância, regras sem médias e Black–Litterman trocam otimalidade teórica por estabilidade. Compare qualquer regra com $1/N$ fora da amostra antes de adotá-la.
- O teste de estresse complementa o VaR: aplica à carteira de hoje episódios históricos, choques hipotéticos propagados pelas covariâncias e correlações de crise, e reporta a perda sem atribuir probabilidade.

Exercícios

6.1 [básico] Dois ativos têm volatilidades de 25% e 40% e correlação de 0,2. Calcule à mão a volatilidade da carteira com 60% no primeiro, e depois o peso de mínima variância pela Equação (6.2). A partir de que correlação a carteira de mínima variância passa a ter 100% no ativo menos volátil?

6.2 [básico] Use a Equação (6.4) com $\bar{v} = 0{,}189$ e $\bar{c} = 0{,}058$ para calcular a volatilidade de carteiras de pesos iguais com 2, 5, 10 e 50 ações. Quantas ações são necessárias para ficar a menos de um ponto percentual do piso?

6.3 [básico] Mostre, a partir da condição $\boldsymbol{\Sigma}\mathbf{w} = \lambda\mathbf{1}$, que todo ativo tem a mesma covariância com a carteira de mínima variância global, e que essa covariância é a variância da própria carteira. Confira numericamente com as sete ações.

6.4 [intermediário] Refaça o Código 6.1 com $R^f = 6\%$ e com $R^f = 13\%$. Como mudam os pesos da carteira tangente? O que acontece com a Equação (6.9) quando R^f supera o retorno da carteira de mínima variância?

6.5 [intermediário] Some dois pontos percentuais à média estimada de ITUB4, um valor bem menor que seu erro-padrão, e recalcule a carteira de máximo Sharpe sem venda a descoberto com `maximo_sharpe`. Quanto do patrimônio muda de lugar? Repita o teste com a carteira de mínima variância, multiplicando por 1,02 a linha e a coluna de ITUB4 na matriz de covariância (o que eleva sua volatilidade em 2%).

6.6 [intermediário] Altere a constante `JANELA` de `experimento.py` para 252 e para 1 260 pregões e rode de novo `l05_fora_da_amostra.py`. O ordenamento das regras se mantém? Qual regra é mais sensível ao tamanho da janela, e por quê?

6.7 [intermediário] Escreva uma função que calcule as contribuições de risco de uma carteira e verifique que elas somam σ_p. Em seguida, encontre com SLSQP os pesos de paridade de risco para as sete ações, minimizando a soma dos quadrados das diferenças entre as contribuições, e compare com a regra do inverso da volatilidade.

6.8 [desafio] Acrescente ao experimento fora da amostra o custo por ponta de 0,10% (Capítulo 7) sobre o valor negociado em cada rebalanceamento, que é o dobro do giro informado na tabela. Quanto do retorno anual de cada regra é consumido pelo custo, e qual regra perde mais?

6.9 [desafio] Monte um relatório de estresse de uma página para a carteira de mínima variância de hoje: os quatro episódios históricos, um choque de -25% no Ibovespa via betas, um choque de -20% simultâneo em ITUB4 e BBAS3 propagado pela Equação (6.13), e o estresse de correlação com $\lambda = 0{,}68$ somado a volatilidades 50% maiores, convertendo a volatilidade resultante em perda pelo VaR normal de 99% em 40 pregões. Indique a posição que mais contribui para a perda no pior cenário.

Para ir além

- Markowitz (1952), o artigo original, curto e legível, com a geometria da fronteira para três e quatro ativos.
- Ang (2014), capítulo 3, para a teoria de média-variância discutida com espírito crítico e com dados.
- Grinold e Kahn (2000), para o uso de otimização em gestão ativa, com retornos esperados construídos a partir de previsões e não de médias históricas.
- DeMiguel et al. (2009) e Michaud (1989), para a evidência e os argumentos sobre erro de estimação.
- López de Prado (2018), capítulo 16, que apresenta a paridade de risco hierárquica (HRP), uma alternativa que dispensa a inversão da matriz de covariância; López de Prado (2020), capítulo 2, discute a limpeza da matriz de covariância antes da otimização.
- Hull (2018), capítulo 22, e Jorion (2007), no capítulo sobre testes de estresse, para a prática em instituições financeiras e a visão dos reguladores.

Capítulo 7

Backtesting: métricas de desempenho e vieses

7

Testar uma regra de investimento no passado é barato, rápido e perigosamente convincente. Com meia dúzia de linhas de pandas você produz uma curva de capital que sobe, e quase sempre há um erro escondido nela: uma informação usada antes de existir, um conjunto de ativos escolhido depois dos fatos, um parâmetro ajustado até o gráfico ficar bonito. Este capítulo ensina a montar um backtest, a medir o resultado com as métricas usuais e, principalmente, a desconfiar dele.

Ao terminar este capítulo você consegue

- montar um backtest vetorizado em pandas com a defasagem correta entre sinal e retorno, declarando a hipótese de execução e os custos;
- calcular e interpretar os índices de Sharpe, Sortino, Calmar e Treynor, sabendo o que cada um penaliza;
- estimar a incerteza estatística de um índice de Sharpe e o tamanho de histórico necessário para confiar nele;
- reconhecer e demonstrar o *look-ahead*, o viés de sobrevivência e o sobreajuste;
- escrever e rodar uma estratégia no *backtrader*;
- aplicar uma lista de conferência antes de acreditar em qualquer resultado.

7.1 O que um backtest pode provar

Um **backtest** é a simulação de uma regra de investimento sobre dados históricos: aplica-se a regra a cada data do passado, como se ela estivesse sendo seguida na época, e contabiliza-se o resultado. Serve para três coisas. Descarta ideias que nem no passado funcionaram, o que já elimina a maioria. Mostra o perfil de risco de uma regra, isto é, o tamanho das perdas, a duração dos períodos ruins e a frequência das operações. E revela erros de implementação, porque obriga a escrever a regra sem ambiguidade.

O que ele não faz é provar que a regra vai dar dinheiro. Um experimento científico pode ser repetido em condições controladas; a história dos mercados aconteceu uma única vez, e o backtest é uma releitura dessa única trajetória. Pior, quem conduz o teste já conhece o desfecho: sabe quais crises ocorreram, quais empresas prosperaram e que tipo de regra andou funcionando. López de Prado (2018) dedica um capítulo inteiro aos perigos do backtesting e defende que ele nunca seja usado como instrumento de busca. A razão é estatística e ocupa a Seção 7.7: o melhor de muitos testes parece bom mesmo quando nenhuma das regras testadas tem valor.

Convém, por isso, ler um backtest de modo assimétrico. Um resultado ruim é informativo. Um resultado bom é apenas o começo de uma investigação sobre o que pode estar errado, e o restante do capítulo é o roteiro dessa investigação.

7.2 Anatomia de um backtest

Todo backtest percorre a mesma cadeia. Os *dados* disponíveis até certo instante alimentam uma regra que produz um *sinal*; o sinal vira uma *posição* (quanto do patrimônio fica em cada ativo); a posição, carregada durante o período seguinte, rende o *retorno* da estratégia. O ponto delicado é a palavra "seguinte".

O exemplo deste capítulo é o cruzamento de médias móveis em PETR4: ficar comprado quando a média dos últimos 20 fechamentos está acima da média dos últimos 100, e fora da ação, com o dinheiro rendendo o CDI, no caso contrário. Chame de s_t o sinal calculado com os preços até o fechamento do dia t: vale 1 se a média rápida supera a lenta e 0 se não. Esse número só existe quando o pregão de t termina, e por isso o primeiro retorno que ele pode capturar é o do pregão de $t+1$.

Definição 7.1 Execução no fechamento do sinal

Na convenção padrão deste livro, a **execução no fechamento do sinal**, a posição w_t é assumida no próprio fechamento em que o sinal é calculado, $w_t = s_t$, e rende do fechamento de t ao de $t+1$. O retorno da estratégia no dia t é

$$R_t = w_{t-1}\, R_t^a + (1 - w_{t-1})\, R_t^f - c\,|w_{t-1} - w_{t-2}|, \tag{7.1}$$

em que $R_t^a = P_t/P_{t-1} - 1$ é o retorno simples do ativo entre os fechamentos de $t-1$ e t, R_t^f é o CDI acumulado entre esses dois fechamentos e c é o custo por ponta, cobrado sobre a fração do patrimônio negociada.

O índice $t-1$ é a parte mais importante da Equação (7.1): o retorno de hoje é multiplicado pela posição decidida ontem. A Figura 7.1 desenha a sequência. Trocar w_{t-1} por w_t faz o sinal ganhar o retorno do mesmo dia cujo fechamento ele usou; isso é *look-ahead*, o erro da Seção 7.5. A convenção padrão não comete esse erro, mas tem uma fragilidade própria: o preço de fechamento só é conhecido quando o pregão termina, e operar nesse mesmo preço exige ter o sinal pronto instantes antes. Há quem classifique também esse caso como look-ahead, e o enunciado do projeto final do Núcleo é um deles (Capítulo 12). O livro a usa por ser a forma mais simples de escrever um backtest, sempre declarada e acompanhada do teste com um pregão de atraso.

A execução no fechamento do sinal não usa informação do futuro, mas é uma hipótese otimista, e precisa ser declarada em qualquer relatório. Para receber R_t^a inteiro, a compra tem de sair exatamente ao preço P_{t-1}, o mesmo

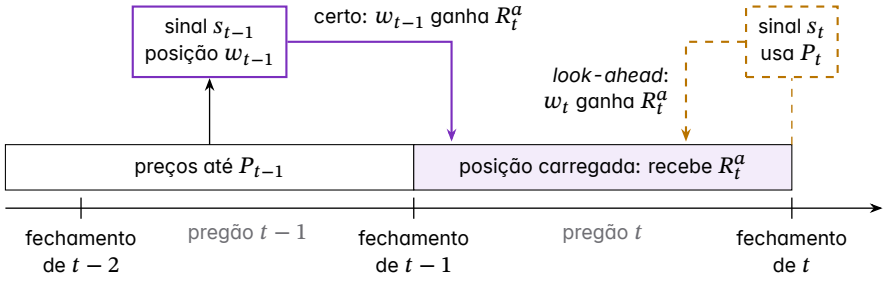


Figura 7.1 Linha do tempo de um backtest diário com execução no fechamento do sinal. A posição assumida no fechamento de $t-1$ é carregada durante o pregão de t e recebe o retorno R_t^a. Usar a posição decidida com P_t para ganhar esse mesmo retorno (tracejado) é *look-ahead*.

fechamento que entrou no cálculo do sinal. Isso supõe que o sinal fique pronto instantes antes do fim do pregão, com um preço quase igual ao de fechamento, e que a ordem seja atendida no leilão de fechamento sem mover o preço. Há duas hipóteses mais conservadoras. Uma é executar na abertura do pregão seguinte, que é o que o backtrader faz por padrão (Seção 7.8). A outra é esperar um pregão inteiro e executar no fechamento seguinte, $w_t = s_{t-1}$. Toda regra deve ser testada também com esse pregão de atraso: se o resultado desaparece, ele dependia de uma execução que talvez não exista.

7.2.1 O backtest em pandas

O Código 7.1 implementa a Equação (7.1) sem nenhum laço. A linha que importa é a do `shift`: `sinal.shift(1)` desloca a série um dia para a frente, de modo que a linha de cada data contém a posição assumida na véspera, o w_{t-1} da fórmula. Com `defasagem=2` a mesma função faz o teste do pregão de atraso.

Código 7.1 Motor vetorizado do cruzamento de médias (`cap07/motor.py`)

```python
def cruzamento(preco, cdi, rapida=20, lenta=100,
               custo=0.001, defasagem=1):
    """Comprado quando a média rápida supera a lenta."""
    media_r = preco.rolling(rapida).mean()
    media_l = preco.rolling(lenta).mean()
    # o sinal de t só é conhecido no fechamento de t
    sinal = (media_r > media_l).astype(float)
    # por isso a posição do dia t usa o sinal de t - defasagem
    posicao = sinal.shift(defasagem).fillna(0.0)
    giro = posicao.diff().abs().fillna(0.0)  # fração negociada
    bruto = posicao * preco.pct_change().fillna(0.0)
    caixa = (1 - posicao) * cdi              # fora da ação: CDI
    return pd.DataFrame({"posicao": posicao, "giro": giro,
                         "retorno": bruto + caixa - custo * giro})
```

O **giro** (*turnover*) é a fração do patrimônio que troca de mãos: aqui vale 1 em cada entrada e em cada saída e 0 nos demais dias. O custo de transação é proporcional a ele. Usamos o custo padrão do livro, $c = 0{,}10\%$ por ponta (10 pontos-base do valor negociado em cada compra e em cada venda), hipótese única que resume taxas, metade do *spread* e *slippage* de ordens pequenas; não inclui o impacto de mercado de ordens grandes, tratado no Capítulo 11. O script `cap07/l01_vetorizado.py` chama essa função e mede o resultado de 2011 a 2025 (o ano de 2010 fica reservado para aquecer as médias):

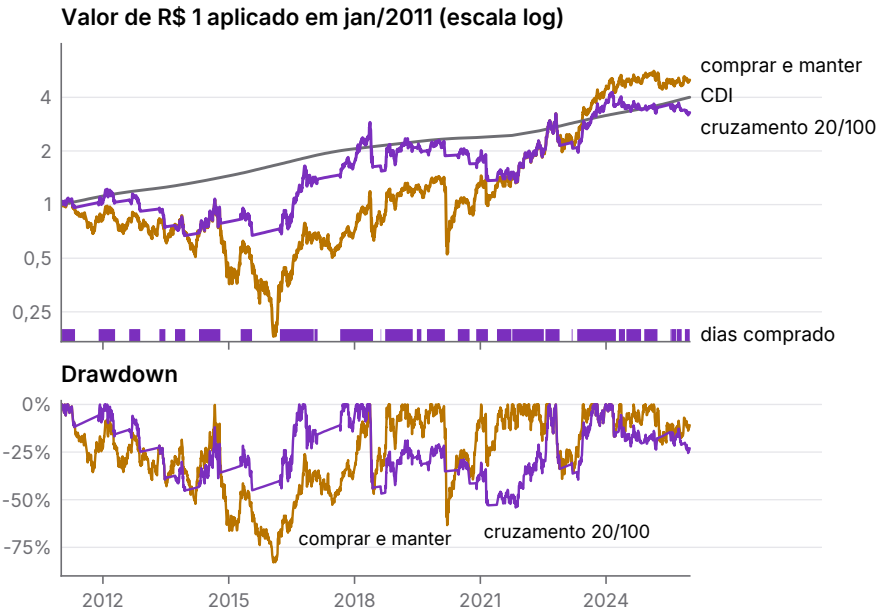


Figura 7.2 Cruzamento de médias 20/100 em PETR4, de 2011 a 2025, líquido de custos, contra comprar e manter a ação e contra o CDI. A faixa na base do painel de cima marca os dias com posição comprada. A regra evitou boa parte da queda de 2014 e 2015, mas terminou atrás das duas alternativas.

```
pregões: 3674
tempo comprado: 61.6%
ordens: 55   custo total: 5.50%
R$ 1 na estratégia vira  3.29
R$ 1 em ativo vira       5.01
R$ 1 em cdi   vira       4.00
com um pregão de atraso: 3.27
```

O resultado é honesto e pouco animador. Em quinze anos a regra multiplicou o capital por 3,29, menos do que os 4,00 do CDI e do que os 5,01 de quem simplesmente comprou a ação. Foram 55 ordens, menos de quatro por ano, que custaram cerca de 5,5% do patrimônio. A última linha é o teste da hipótese de execução: com um pregão de atraso o capital final passa de 3,29 para 3,27, de modo que este resultado não depende de executar no fechamento do sinal, o que é de esperar em uma regra que negocia tão pouco. A Figura 7.2 mostra onde a regra ajudou (ficou fora da ação em boa parte da queda de 2014 e 2015) e onde atrapalhou (em anos como 2018 e 2021, nos quais a ação subiu aos solavancos, ela entrou tarde e saiu tarde). Os termos de comparação de um backtest, aqui o CDI e a ação comprada e mantida, formam o seu **benchmark**: a alternativa simples que a estratégia precisa superar para justificar o trabalho. Guarde esses números, porque é contra eles que os vieses das próximas seções serão medidos.

No repositório

Os scripts `aula7/01_lookahead_bias.py` e `aula7/03_overfitting.py` usam esta mesma estrutura (`sinal.shift(1) * retorno`) com dados baixados na hora pelo `yfinance`. As versões do livro, em `livro/scripts/cap07/`, leem os dados congelados, incluem custos e remuneram o caixa pelo CDI (série do Banco Central, `cap07/baixar_cdi.py`).

7.3 Métricas de desempenho

Retorno acumulado sozinho não compara estratégias, porque ignora o risco corrido para obtê-lo. As métricas desta seção são todas razões entre um ganho e uma medida de risco. O numerador muda pouco de uma para outra; a diferença está no que cada uma chama de risco.

O ponto de partida são o retorno anualizado (Equação (1.7) do Capítulo 1) e a volatilidade anualizada pela regra da raiz do tempo, os dois pela convenção de 252 dias (Seção 1.9). O retorno que interessa é o **excesso de retorno** sobre a taxa livre de risco, $d_t = R_t - R_t^f$: render 10% ao ano com o CDI a 13% não é mérito. No Brasil o livre de risco é o CDI, que de 2011 a 2025 rendeu 9,7% ao ano em tempo de calendário. Pela convenção de 252 dias o mesmo CDI aparece como 10,0% ao ano, porque a série de PETR4 tem preço em 3.674 pregões nesses quinze anos, cerca de 245 por ano (a B3 teve cerca de 248; faltam os dias de 2011 e 2012 sem negócio registrado na fonte, Capítulo 2; o retorno que vem depois de uma dessas lacunas cobre mais de um pregão e entra nas contas como se fosse de um dia); como todas as taxas deste capítulo usam a mesma convenção, as comparações entre elas não são afetadas.

7.3.1 Índice de Sharpe

Definição 7.2 Índice de Sharpe

Sendo $\bar{d}$ a média e $\hat{\sigma}_d$ o desvio-padrão amostral dos excessos de retorno diários $d_t = R_t - R_t^f$, o índice de Sharpe anualizado é

$$\mathrm{SR} = \frac{\bar{d}}{\hat{\sigma}_d}\sqrt{252}. \tag{7.2}$$

A medida foi proposta por Sharpe (1966) para comparar fundos e reformulada por Sharpe (1994), que passou a defini-la sobre o retorno diferencial em relação a um benchmark, com o desvio-padrão desse diferencial no denominador. No Capítulo 6 o índice apareceu com o desvio-padrão do próprio retorno no denominador; como o CDI quase não varia de um dia para o outro, os dois denominadores praticamente coincidem. Ela responde quantas unidades de excesso de retorno a estratégia entregou por unidade de oscilação. O fator $\sqrt{252}$ vem de anualizar o numerador multiplicando por 252 e o denominador multiplicando por $\sqrt{252}$, e por isso herda a hipótese da raiz do tempo: retornos sem autocorrelação e com variância constante. Lo (2002) mostra que, quando os retornos são autocorrelacionados, a regra pode errar muito; em sua amostra de fundos de *hedge*, a autocorrelação positiva dos retornos mensais chega a superestimar o Sharpe anualizado em 65%.

Intuição

O índice de Sharpe é a média do excesso de retorno medida em desvios-padrão. Se você dividir o Sharpe anualizado por $\sqrt{252}$ e multiplicar pela raiz do número de dias da amostra, obtém a estatística t do teste de que o excesso médio é zero. Ler um Sharpe sem perguntar pelo tamanho da amostra é ler uma estatística de teste sem o valor crítico; a Seção 7.4 faz essa conta.

7.3.2 Índice de Sortino

O desvio-padrão trata da mesma forma um dia de +3% e um dia de −3%. O índice de Sortino (Sortino e Price, 1994) substitui o denominador por uma medida que só enxerga os resultados abaixo de um alvo A_t (um retorno mínimo aceitável; aqui, o próprio CDI).

Definição 7.3 Desvio abaixo do alvo e índice de Sortino

O **desvio abaixo do alvo** (*downside deviation*) de uma série de T retornos é

$$\mathrm{DD} = \sqrt{\frac{1}{T}\sum_{t=1}^{T} \min(R_t - A_t,\, 0)^2}, \tag{7.3}$$

e o **índice de Sortino** anualizado é a média de $R_t - A_t$ dividida por DD e multiplicada por $\sqrt{252}$.

Armadilha **Dividir só pelos dias ruins**

Na Equação (7.3) a soma é dividida pelo total de observações T, e os dias acima do alvo entram com valor zero. Um erro frequente é filtrar os retornos negativos e calcular o desvio-padrão só deles. Isso responde outra pergunta (quão dispersas são as perdas, quando ocorrem) e premia a estratégia que perde raramente mas muito: duas perdas idênticas de 30% em mil dias teriam, por essa conta, dispersão nula.

Para retornos simétricos de média pequena, metade dos dias fica abaixo do alvo e $\mathrm{DD} \approx \hat{\sigma}_d/\sqrt{2}$, de modo que o Sortino vale cerca de $\sqrt{2} \approx 1{,}41$ vez o Sharpe. A informação está no desvio em relação a essa proporção: um Sortino bem acima dela indica oscilações concentradas no lado dos ganhos, e bem abaixo, perdas raras e grandes.

Por isso não se compara o Sortino de uma estratégia com o Sharpe de outra: as escalas são diferentes.

7.3.3 Índice de Calmar

O drawdown máximo é a maior queda da curva de capital em relação ao pico anterior, e o índice de Calmar (Young, 1991) divide o retorno anualizado por ele; os dois foram definidos no Capítulo 5:

$$\mathrm{Calmar} = \frac{R_{\text{a.a.}}}{|\text{drawdown máximo}|}. \tag{7.4}$$

É a métrica de quem pergunta “quanto eu ganho por ano para cada real que cheguei a ver sumir?”. Tem duas fraquezas. Depende de um único episódio da amostra, o que a torna instável. E o drawdown máximo só pode crescer quando a amostra aumenta, de modo que índices de Calmar calculados em janelas de tamanhos diferentes não são comparáveis.

7.3.4 Treynor, alfa de Jensen e *information ratio*

As três métricas anteriores medem o risco da estratégia isolada. Para quem a coloca dentro de uma carteira diversificada, a parte do risco que importa é a que não se dilui: a sensibilidade ao mercado, medida pelo beta do Capítulo 6. Estima-se o beta pela regressão dos excessos de retorno da estratégia sobre os do mercado,

$$R_t - R_t^f = \alpha + \beta\,(R_t^m - R_t^f) + \varepsilon_t, \tag{7.5}$$

Tabela 7.1 O que cada métrica trata como risco

Métrica	Risco no denominador	Ponto cego
Sharpe	desvio-padrão do excesso sobre o CDI: toda oscilação, para cima ou para baixo	perdas raras e grandes; autocorrelação distorce o número
Sortino	desvio abaixo do alvo: só os dias ruins	poucos dias ruins na amostra tornam a estimativa instável
Calmar	drawdown máximo: a pior queda acumulada	depende de um único episódio e do tamanho da janela
Treynor	beta: só o risco de mercado	ignora o risco próprio; sem sentido com beta perto de zero
Information ratio	*tracking error*: o desvio em relação ao benchmark	depende inteiramente do benchmark escolhido

cuja inclinação de mínimos quadrados é $\hat{\beta} = \mathrm{Cov}(d, d^m)/\mathrm{Var}(d^m)$. O **índice de Treynor** (Treynor, 1965) é o excesso de retorno médio anualizado dividido por $\hat{\beta}$, e se lê como o prêmio recebido por unidade de risco de mercado. O intercepto anualizado da mesma regressão é o **alfa de Jensen** (Jensen, 1968): o excesso de retorno que sobra depois de descontada a exposição ao mercado. O Treynor perde o sentido quando o beta é próximo de zero ou negativo, caso de muitas estratégias compradas e vendidas ao mesmo tempo; para elas olhe o alfa.

Por fim, quando a estratégia é avaliada contra um benchmark R^b que ela tenta superar, usa-se o **information ratio**: a média do retorno ativo $R_t - R_t^b$ dividida pelo seu desvio-padrão (o *tracking error*), anualizada como o Sharpe. O Sharpe é o caso particular em que o benchmark é o CDI. Grinold e Kahn (2000), que fazem dessa razão a peça central da gestão ativa, trabalham com uma versão que antes desconta o beta em relação ao benchmark; as duas coincidem quando esse beta é 1. A Tabela 7.1 põe lado a lado o que cada razão trata como risco.

7.3.5 As métricas do exemplo

O Código 7.2 mostra quatro das definições como funções de poucas linhas; o arquivo traz também o drawdown máximo, o Calmar e o *information ratio*. Todas recebem séries de retornos diários já alinhadas.

Código 7.2 Métricas a partir de retornos diários (`cap07/metricas.py`, trecho)

```
def sharpe(r, rf):
    excesso = r - rf
    return excesso.mean() / excesso.std() * np.sqrt(DIAS)


def sortino(r, alvo):
    desvio = r - alvo
    abaixo = np.minimum(desvio, 0.0)         # zero nos dias bons
    downside = np.sqrt((abaixo ** 2).mean())  # divide por T
    return desvio.mean() / downside * np.sqrt(DIAS)


def beta_alfa(r, mercado, rf):
    y, x = r - rf, mercado - rf
```

```
    beta = np.cov(y, x)[0, 1] / x.var()      # inclinação de MQO
    alfa = (y.mean() - beta * x.mean()) * DIAS
    return beta, alfa


def treynor(r, mercado, rf):
    beta, _ = beta_alfa(r, mercado, rf)
    return (r - rf).mean() * DIAS / beta
```

Aplicadas à estratégia, à ação e ao Ibovespa de 2011 a 2025 (script `cap07/l02_metricas.py`), elas produzem a tabela abaixo. A linha "excesso a.a." é a média aritmética diária do excesso sobre o CDI multiplicada por 252, o numerador do Sharpe e do Treynor.

```
              estratégia   PETR4 Ibovespa
retorno a.a.        8.5%   11.7%     6.0%
vol a.a.           30.7%   44.3%    23.2%
excesso a.a.        3.4%   11.5%    -1.0%
drawdown máx.     -54.1%  -82.9%   -47.7%
Sharpe              0.11    0.26    -0.04
Sortino             0.15    0.36    -0.06
Calmar              0.16    0.14     0.13
beta                0.62    1.41     1.00
Treynor             5.5%    8.1%    -1.0%
alfa a.a.           4.0%   12.9%     0.0%
CDI a.a.: 10.0% (252 dias)   9.7% (calendário)
information ratio da estratégia contra PETR4: -0.25
```

Quatro leituras. Primeira: o Ibovespa perdeu para o CDI nesses quinze anos, e seu Sharpe é negativo; qualquer estratégia brasileira precisa ser comparada com a renda fixa, e não com zero. Segunda: por ficar fora da ação em parte do tempo, a estratégia tem beta de 0,62, contra 1,41 de PETR4, e cortou o drawdown máximo de 83% para 54%; o Calmar é a única das razões em que ela supera a ação. Terceira: o Sortino vale cerca de 1,4 vez o Sharpe nas três colunas, a proporção esperada para retornos sem assimetria marcante, de modo que aqui ele não acrescenta informação. Quarta, e menos óbvia: a estratégia rendeu 8,5% ao ano, menos que os 10,0% do CDI na mesma convenção, e mesmo assim tem Sharpe positivo.

Armadilha **Sharpe positivo com retorno abaixo do CDI**

Não há erro de conta. O Sharpe usa a média *aritmética* dos excessos diários, que foi de 3,4% ao ano. O retorno anualizado é *geométrico* e fica abaixo da média aritmética em cerca de $\sigma^2/2$, relação que um exercício do Capítulo 1 pede para verificar: com volatilidade de 30,7%, são $0{,}307^2/2 \approx 4{,}7$ pontos percentuais por ano, mais do que o excesso médio. Sempre que a volatilidade for alta, informe as duas coisas: o Sharpe e o retorno composto.

7.4 Quanto se pode confiar em um Sharpe

O Sharpe de 0,11 da estratégia é uma estimativa feita com uma amostra, e como toda estimativa tem erro. Lo (2002) deriva a distribuição assintótica do estimador para retornos independentes e identicamente distribuídos.

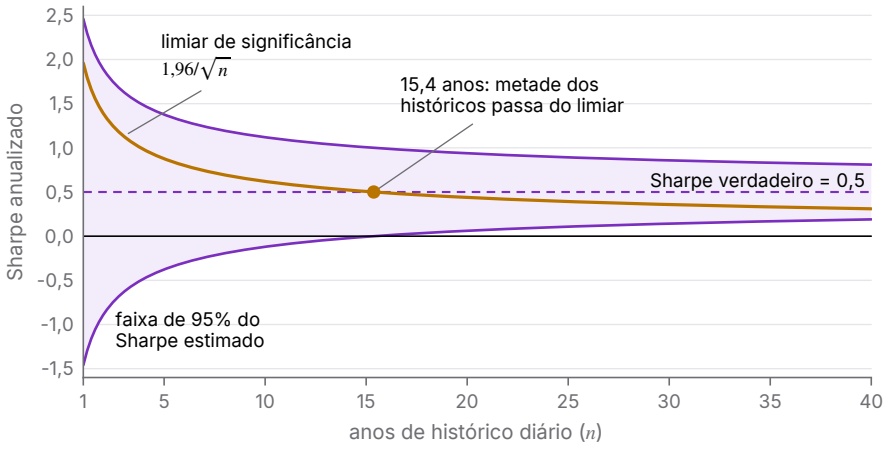


Figura 7.3 Sharpe estimado quando o Sharpe verdadeiro é 0,5: faixa que contém 95% das estimativas, pela Equação (7.6), e limiar que a estimativa precisa superar para ser significativa a 5%. Com cinco anos a faixa vai de $-0{,}4$ a $1{,}4$. Aos 15,4 anos o limiar cruza 0,5, e só metade dos históricos o supera.

Resultado 7.4 Erro-padrão do índice de Sharpe (retornos iid)

Se os excessos de retorno são iid com índice de Sharpe por período SR, o estimador $\widehat{\mathrm{SR}}$ calculado com T observações é aproximadamente normal com erro-padrão

$$\mathrm{ep}\big(\widehat{\mathrm{SR}}\big) \approx \sqrt{\frac{1 + \frac{1}{2}\mathrm{SR}^2}{T}}. \tag{7.6}$$

Na fórmula, SR e T estão na mesma frequência: Sharpe diário com T em dias.

Com dados diários o termo $\frac{1}{2}\mathrm{SR}^2$ é desprezível, porque o Sharpe por dia é o anualizado dividido por $\sqrt{252}$. Multiplicando os dois lados por $\sqrt{252}$ para voltar à escala anual, e escrevendo $T = 252\,n$ para n anos de histórico, sobra uma regra de bolso:

$$\mathrm{ep}\big(\widehat{\mathrm{SR}}_{\text{a.a.}}\big) \approx \frac{1}{\sqrt{n}}, \qquad t \approx \mathrm{SR}_{\text{a.a.}}\sqrt{n}. \tag{7.7}$$

O erro-padrão do Sharpe anualizado é aproximadamente um sobre a raiz do número de anos, quase sem depender do próprio Sharpe ou da frequência dos dados.

Exemplo 7.1 Quantos anos para um Sharpe de 0,5 ser significativo?

Para rejeitar a hipótese de Sharpe nulo com 95% de confiança (teste bilateral) é preciso $t \geq 1{,}96$. Pela Equação (7.7), um Sharpe *estimado* de 0,5 só passa no teste se $0{,}5\sqrt{n} \geq 1{,}96$, isto é, com $n \geq (1{,}96/0{,}5)^2 = 15{,}4$ anos de histórico. Para um Sharpe estimado de 1,0 bastam $1{,}96^2 = 3{,}8$ anos, e para 2,0, um ano. A correção do termo $\frac{1}{2}\mathrm{SR}^2$ não altera nenhuma dessas casas decimais. Esse prazo não garante a detecção: se o Sharpe *verdadeiro* é 0,5, a estimativa fica abaixo de 0,5 em metade dos históricos de 15,4 anos, e nesses o teste não rejeita. Para rejeitar em 80% dos históricos são necessários $\big((1{,}96 + 0{,}84)/0{,}5\big)^2 \approx 31$ anos.

A Figura 7.3 mostra a consequência. Um Sharpe de 0,5 não é pequeno (o de PETR4 no exemplo foi 0,26), e ainda assim uma estratégia que o tenha de verdade precisa de quinze anos de dados diários para ter metade de chance de ser distinguida de sorte, e de mais de trinta para que isso seja provável. Cinco anos de histórico, o tamanho típico de um backtest de estudante, produzem um erro-padrão de 0,45: Sharpes estimados de 0,0 e de 0,8 são estatisticamente indistinguíveis. O script `cap07/l03_sharpe_erro.py` faz a conta para o exemplo e confere a fórmula por simulação:

```
Sharpe 0.11   erro-padrão 0.26   IC 95%: [-0.40, 0.62]
Sharpe 0.5:  15.4 anos para t = 1.96
Sharpe 1.0:   3.8 anos para t = 1.96
Sharpe 2.0:   1.0 anos para t = 1.96
Sharpe 0.5, poder de 80%: 31.4 anos
simulado: 0.316   fórmula: 0.316
```

O intervalo de 95% para o Sharpe do cruzamento de médias vai de $-0{,}40$ a $0{,}62$: os dados não permitem dizer nem que a regra é melhor que o CDI, nem que é pior. A última linha compara o desvio-padrão do Sharpe em 20 mil históricos simulados de dez anos com a Equação (7.6); os dois valores coincidem em $0{,}316 = 1/\sqrt{10}$.

A hipótese iid é uma simplificação. Para medir quanto ela custa, o script `cap07/l08_sharpe_dependencia.py` repete a simulação anterior (históricos de dez anos, Sharpe verdadeiro de 0,5) com retornos dependentes no tempo. Com volatilidade agrupada, em um GARCH(1,1) como os do Capítulo 4, o desvio-padrão do Sharpe estimado é 0,322, contra 0,317 no caso iid: praticamente o mesmo. Com autocorrelação de primeira ordem de $+0{,}1$ ele sobe para 0,352, e com $-0{,}1$ cai para 0,278. O que pesa, portanto, é a autocorrelação dos retornos, caso tratado por Lo (2002): quando é positiva, o intervalo se alarga e o Sharpe anualizado pela raiz do tempo fica superestimado; quando é negativa, ocorre o contrário. A assimetria e a curtose, incluídas na versão do erro-padrão apresentada por Bailey e López de Prado (2012), mudam o resultado em poucos por cento em dados diários, em que o Sharpe por período é muito pequeno. Trate os prazos do Exemplo acima como ordem de grandeza, e como pisos para estratégias de retornos positivamente autocorrelacionados.

7.5 *Look-ahead*: usar o futuro sem perceber

O *look-ahead bias*, ou viés de antecipação, já apareceu no Capítulo 2: ocorre quando o backtest usa, em uma data, informação que só ficou disponível depois. É o erro mais comum e o mais destrutivo, porque um programa não reclama quando lê o futuro: o arquivo de dados contém todas as datas, e nada impede o código de consultar a linha de amanhã. Ele aparece de quatro formas principais.

- **Sinal e retorno no mesmo dia.** Esquecer o `shift(1)` do Código 7.1 equivale a trocar w_{t-1} por w_t na Equação (7.1): a posição decidida com o fechamento de t recebe o retorno que termina nesse mesmo fechamento. É a seta tracejada da Figura 7.1. Não confunda com a execução no fechamento do sinal, em que a posição decidida com P_t só rende de t para $t+1$. Variantes do erro: usar a máxima ou a mínima do dia para decidir uma entrada na abertura; médias móveis centradas (`rolling(..., center=True)`); `shift(-1)` em qualquer variável explicativa.

- **Estatísticas da amostra inteira.** Padronizar uma série com a média e o desvio-padrão de todo o histórico, ou escalá-la pelo mínimo e pelo máximo globais, coloca em cada data um número que depende dos preços futuros. A versão correta usa janelas que terminam na data da decisão (`rolling` ou `expanding`). O mesmo vale para qualquer parâmetro estimado; em aprendizado de máquina o erro se chama vazamento de informação (Capítulo 9).
- **Dados fundamentalistas na data de referência.** O balanço do quarto trimestre tem data de referência em 31 de dezembro, mas só é divulgado semanas ou meses depois. Um backtest que compra "as empresas de maior lucro" em 2 de janeiro usando esse balanço negocia com informação que ninguém tinha. É preciso indexar cada dado pela data de divulgação, e desconfiar de bases que guardam apenas o valor revisado.
- **Preço ajustado usado como nível.** A série ajustada por proventos é recalculada a cada novo dividendo (Capítulo 1), de modo que o preço ajustado de 2012 que você vê hoje embute dividendos pagos até hoje. Ela serve para calcular retornos e sinais que dependem só de razões entre preços próximos, como os deste capítulo. Uma regra baseada no nível do preço precisa do preço efetivamente negociado na data; a série sem ajuste do livro se aproxima dele, mas já vem corrigida por desdobramentos (Capítulo 1).

7.5.1 Um experimento: o mesmo sinal, com e sem defasagem

O Código 7.3 mede o tamanho do estrago em duas regras. A lenta é o cruzamento 20/100. A rápida fica comprada quando o fechamento está acima da média dos últimos cinco fechamentos, e troca de posição 65 vezes por ano. Em cada uma, a única diferença entre as duas execuções é o argumento do `shift`.

Código 7.3 A regra rápida, com defasagem configurável (`cap07/l04_lookahead.py`, trecho)

```
def regra_rapida(defasagem):
    """Comprado se o fechamento supera a média de 5 dias."""
    sinal = (preco > preco.rolling(5).mean()).astype(float)
    posicao = sinal.shift(defasagem).fillna(0.0)
    giro = posicao.diff().abs().fillna(0.0)
    return posicao * r + (1 - posicao) * rf - 0.001 * giro
```

```
regra_rapida, shift(0): retorno a.a. 482.5%   Sharpe  6.21
regra_rapida, shift(1): retorno a.a.   4.1%   Sharpe -0.05
regra_lenta, shift(0): retorno a.a.  12.3%   Sharpe  0.22
regra_lenta, shift(1): retorno a.a.   8.5%   Sharpe  0.11
trocas de posição por ano: 65
dias de alta com o sinal do próprio dia ligado: 74.5%; desligado: 25.3%
```

Sem a defasagem, a regra rápida rende 483% ao ano com Sharpe acima de 6. A explicação está na última linha da saída: o fechamento de hoje fica acima da média de cinco dias principalmente quando hoje foi um dia de alta. Nos dias em que o sinal contemporâneo está ligado, a ação subiu em 74,5% das vezes; nos demais, em 25,3%. O "sinal" é em boa parte o próprio retorno que ele finge prever. Com o `shift(1)` a mesma regra rende 4,1% ao ano, bem abaixo do CDI.

O caso da regra lenta é mais perigoso justamente por ser discreto. O sinal muda poucas vezes, e o erro só afeta os dias de troca; ainda assim o retorno passa de 8,5% para 12,3% ao ano, e a estratégia que perdia para o CDI passa a ganhar dele. Nenhum alarme dispara: 12% ao ano com Sharpe 0,22 é um resultado perfeitamente plausível. Um Sharpe de 6 denuncia o erro sozinho, e

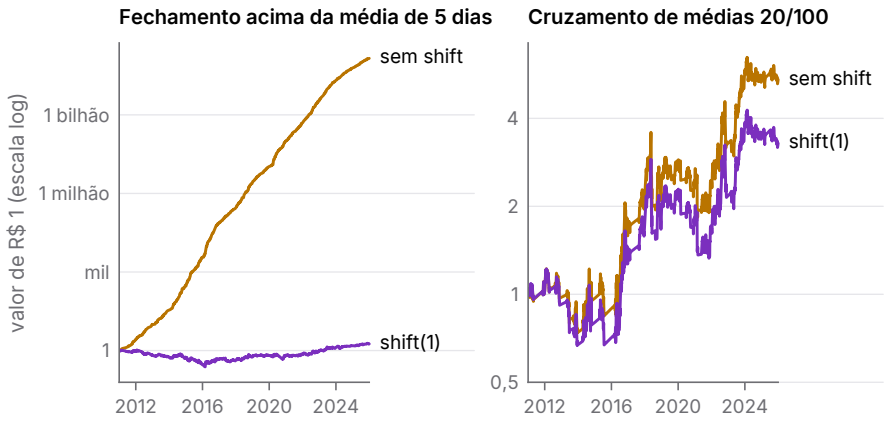


Figura 7.4 Curvas de capital com e sem a defasagem de um dia, de 2011 a 2025, líquidas de custos. Na regra rápida (esquerda) o erro transforma uma estratégia que perde para o CDI em uma que multiplica o capital por mais de 140 bilhões. Na regra lenta (direita) o efeito é menor, mas suficiente para inverter a conclusão.

um ganho de três ou quatro pontos por ano passa despercebido.

Regra prática: Sharpe acima de 2 ou 3 em dados diários, com uma regra simples, é sintoma de erro até prova em contrário.

7.6 Viés de sobrevivência

O **viés de sobrevivência** (*survivorship bias*) aparece quando o universo de ativos do backtest é definido com informação de hoje. Quem baixa "as ações do Ibovespa" e testa uma regra desde 2010 usa a composição atual do índice, da qual já saíram as empresas que quebraram, foram incorporadas ou encolheram até perder relevância. A OGX, que integrava o índice e pediu recuperação judicial em 2013, não está em nenhuma lista atual, mas uma estratégia rodando em 2012 poderia tê-la comprado. Brown et al. (1992) mostraram que amostras de fundos truncadas pela sobrevivência podem fabricar até a aparência de persistência de desempenho.

O conjunto de dados deste livro tem esse viés, e é bom vê-lo em números. As ações de `precos_ajustados.csv` foram escolhidas ao montar o livro, depois dos fatos, por serem grandes, líquidas e conhecidas. O script `cap07/l05_escolhidas.py` compara as seis que têm histórico desde 2010 com o Ibovespa, de 2011 a 2025:

```
      PETR4 VALE3  ITUB4  WEGE3 ABEV3 BBAS3 ^BVSP
a.a.  11.7%  8.6%  13.7%  23.2%  8.0%  9.8%  6.0%
carteira das seis: 14.4% a.a.
```

Todas as seis superaram o índice, e uma carteira com pesos iguais nelas rendeu 14,4% ao ano contra 6,0%. Isso não mede habilidade nenhuma: mede o fato de que hoje chamamos de "principais ações" as que foram bem. Qualquer regra testada só nesses papéis herda essa vantagem, e a diferença de mais de oito pontos percentuais ao ano não é uma estimativa limpa do viés (os pesos também diferem dos do índice), mas dá a ordem de grandeza do que está em jogo.

O Ibovespa é um índice de retorno total: reinveste os proventos. A comparação com preços ajustados é, nesse aspecto, justa.

7.6.1 Simulando o que os dados não mostram

Como as empresas que desapareceram não estão no arquivo, o efeito puro só pode ser isolado em simulação. O script `cap07/sobrevivencia.py` cria 500 ações fictícias com a mesma distribuição de retornos (movimento browniano

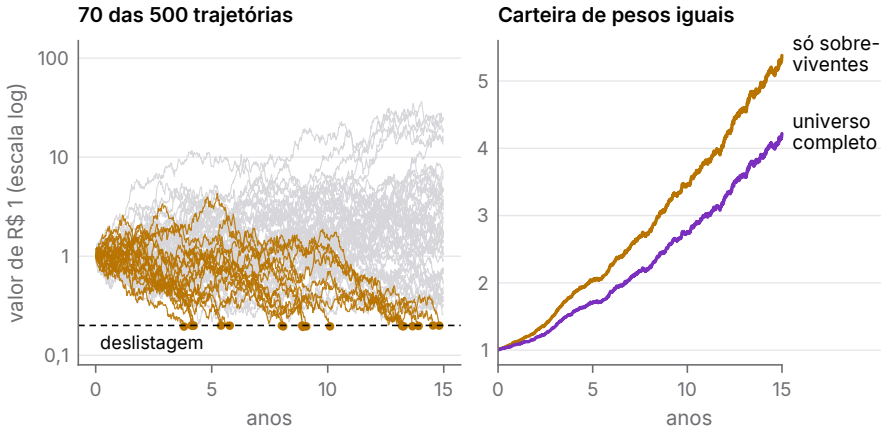


Figura 7.5 Universo simulado de 500 ações idênticas em distribuição. À esquerda, 70 trajetórias: as que cruzam a barreira de deslistagem (em ocre) somem das bases de dados montadas depois. À direita, a carteira de pesos iguais calculada com todas as ações e só com as sobreviventes.

geométrico com retorno esperado de 10% e volatilidade de 40% ao ano, independentes entre si) e as acompanha por quinze anos. Uma ação é deslistada quando perde 80% do valor inicial; quem a tinha fica com o que sobrou, parado. Nenhuma ação é melhor do que outra: tudo o que distingue as sobreviventes é sorte. (O script `aula7/02_survivorship_bias.py` faz uma simulação parecida, com falências sorteadas ao acaso; aqui a deslistagem decorre do desempenho ruim, que é o mecanismo responsável pelo viés.)

```
deslistadas: 112 de 500 (22.4%)
universo completo:  R$ 1 vira 4.22  (10.1% a.a.)
só sobreviventes:   R$ 1 vira 5.38  (11.9% a.a.)
viés: 1.8% ao ano
perdedoras, universo completo: 7.2% a.a.; 30 das 50 deslistadas depois
perdedoras, universo sobreviventes: 13.0% a.a.; 0 das 50 deslistadas depois
```

Comprar todas as ações no início e segurar rende 10,1% ao ano. O mesmo cálculo feito só com as 388 que chegaram ao fim dá 11,9%: quase dois pontos percentuais por ano criados por um filtro que ninguém aplicou de propósito (Figura 7.5).

As duas últimas linhas mostram que o tamanho do viés depende da estratégia. A regra testada é "ao fim do quinto ano, comprar as 50 ações que mais caíram e segurar por dez anos". No universo completo ela rende 7,2% ao ano, e 30 das 50 escolhidas acabam deslistadas. No universo de sobreviventes rende 13,0%, porque a base de dados já retirou, de antemão, justamente as perdedoras que continuaram caindo. Estratégias que compram ativos baratos, depreciados ou pequenos são as mais expostas, pois pescam na região em que estão os futuros desaparecidos.

A correção depende dos dados, pois nenhum código recupera empresas que não estão no arquivo: é preciso uma base que registre, para cada data, quais ativos existiam e faziam parte do universo naquele dia (*point-in-time*), incluindo os que deixaram de ser negociados. Bases gratuitas raramente oferecem isso. Quando não houver alternativa, restrinja as conclusões: uma regra de *timing* sobre um único ativo muito líquido, como a deste capítulo, sofre menos do que uma regra de seleção de ações, embora a própria escolha do ativo já carregue o problema.

7.7 Sobreajuste e a garimpagem de dados

O terceiro viés nasce do processo de pesquisa, com dados e código corretos. Você testa o cruzamento 20/100 e o resultado decepciona. Tenta 10/50, depois 5/20, acrescenta um filtro de volume, troca a ação. Na vigésima tentativa aparece um Sharpe de 1. O **sobreajuste** (*overfitting*) é o ajuste de uma regra ao ruído de uma amostra específica; a **garimpagem de dados** (*data snooping*) é o nome do procedimento que leva a ele: testar muitas variantes e relatar a melhor. A lógica é a dos testes de hipóteses do Capítulo 3: a 5% de significância, uma em cada vinte regras sem valor passa no teste, e quem testa vinte regras deve esperar uma aprovada.

7.7.1 Um experimento com estratégias sem habilidade

O Código 7.4 simula a situação em que não há nada para ser encontrado. Cada "estratégia" é ruído puro: retornos diários normais, de média zero, independentes. Sorteiam-se N delas, escolhe-se a de maior Sharpe em cinco anos e mede-se a mesma estratégia nos cinco anos seguintes. Repete-se tudo mil vezes para cada N.

Código 7.4 A melhor de N estratégias que não sabem nada (`cap07/sobreajuste.py`, trecho)

```
def experimento(n, repeticoes=1000, semente=2026):
    rng = np.random.default_rng(semente + n)
    dentro, fora = np.empty(repeticoes), np.empty(repeticoes)
    for k in range(repeticoes):
        r = rng.normal(0.0, 0.01, (n, 2 * T))
        s_dentro = sharpe(r[:, :T])
        melhor = s_dentro.argmax()            # escolhida antes
        dentro[k] = s_dentro[melhor]
        fora[k] = sharpe(r[[melhor], T:])[0]  # e levada adiante
    return dentro, fora
```

```
   N  dentro(média)  teoria   fora(média)
   1         -0.00    0.00        -0.02
  10          0.69    0.70         0.03
 100          1.12    1.13         0.01
1000          1.45    1.46        -0.01
Sharpe 1.0 esperado em 5 anos com N = 46
```

Com uma única tentativa o Sharpe médio é zero, como deve ser. Com cem tentativas, a melhor tem em média Sharpe de 1,12 dentro da amostra, e em 70% das repetições passa de 1. Com mil, a média é 1,45 e nenhuma repetição fica abaixo de 1. Fora da amostra, em todos os casos, a média volta a zero (Figura 7.6). As estratégias são as mesmas nos quatro casos; a única coisa que varia é o número de tentativas.

A coluna "teoria" vem de um resultado sobre o máximo de variáveis normais usado por Bailey et al. (2014) e por Bailey e López de Prado (2014).

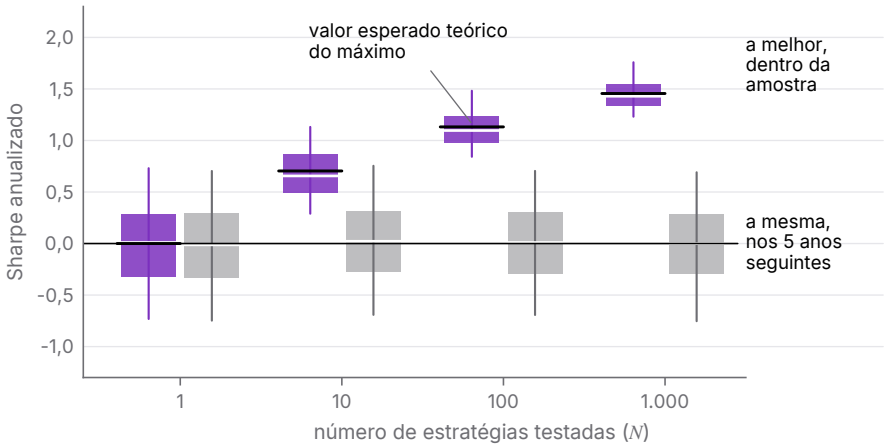


Figura 7.6 Distribuição do Sharpe da melhor entre N estratégias sem habilidade, em mil repetições: caixas do 1º ao 3º quartil, hastes do percentil 5 ao 95. Em roxo, o Sharpe nos cinco anos usados para escolher; em cinza, o da mesma estratégia nos cinco anos seguintes. O traço preto é o valor esperado da Equação (7.8).

Resultado 7.5 Sharpe esperado do máximo de N tentativas

Se N estratégias independentes têm Sharpe verdadeiro igual a zero e seus Sharpes estimados são aproximadamente normais com desvio-padrão σ_{SR}, então, para N grande,

$$\mathbb{E}\Big[\max_{n\le N}\widehat{\mathrm{SR}}_n\Big] \approx \sigma_{\mathrm{SR}}\left[(1-\gamma)\,\Phi^{-1}\Big(1-\frac{1}{N}\Big)+\gamma\,\Phi^{-1}\Big(1-\frac{1}{Ne}\Big)\right], \qquad (7.8)$$

em que Φ^{-1} é a inversa da distribuição normal padrão, e é o número de Euler e $\gamma \approx 0{,}5772$ é a constante de Euler-Mascheroni.

Pela Equação (7.7), $\sigma_{\mathrm{SR}} \approx 1/\sqrt{n}$ para n anos de dados, e a simulação confirma a aproximação com erro de um ou dois centésimos. O valor esperado do máximo cresce devagar com N (aproximadamente como $\sqrt{2\ln N}$), mas nunca para de crescer, e diminui com o tamanho do histórico. A última linha da saída inverte a pergunta: com cinco anos de dados, 46 tentativas independentes bastam para que o melhor Sharpe esperado chegue a 1 sem haver habilidade alguma. Bailey et al. (2014) chamam essa relação de comprimento mínimo do backtest e chegam ao mesmo limite, de cerca de 45 configurações independentes para cinco anos.

7.7.2 Descontando o número de tentativas

Se o melhor resultado de uma busca precisa ser comparado com o que o acaso produziria, a correção natural é trocar a pergunta "o Sharpe é maior que zero?" por "o Sharpe é maior do que o máximo esperado de N tentativas sem habilidade?". Essa é a ideia do *deflated Sharpe ratio* de Bailey e López de Prado (2014): a probabilidade de o Sharpe verdadeiro ser positivo, calculada depois de (i) elevar o valor de referência do teste de zero para o da Equação (7.8), (ii) corrigir o erro-padrão pela assimetria e pela curtose dos retornos e (iii) levar em conta o tamanho da amostra. Para aplicá-la é preciso saber quantas tentativas foram feitas e quão dispersos foram seus resultados, o que obriga a registrar todos os testes, inclusive os descartados.

A mesma preocupação aparece em outros trabalhos. White (2000) propôs um teste por *bootstrap*, o *reality check*, para decidir se o melhor modelo de um

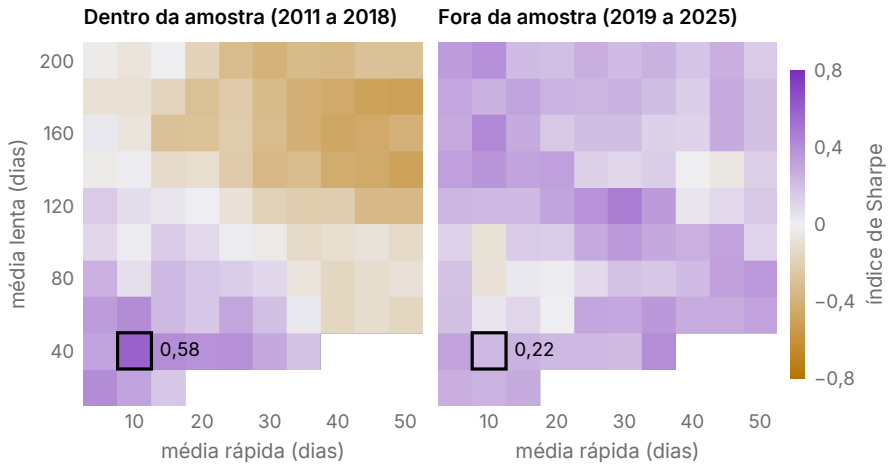


Figura 7.7 Sharpe do cruzamento de médias em PETR4 para cada par de janelas. O quadrado marca a combinação campeã dentro da amostra (10/40). Fora da amostra ela tem Sharpe de 0,22, exatamente a mediana das 90 combinações, e o desenho do mapa não se repete.

conjunto supera um benchmark depois de considerada a busca, e Sullivan et al. (1999) o aplicaram a milhares de regras de análise técnica. Harvey et al. (2016), diante das centenas de fatores já publicados na literatura de apreçamento de ativos, defendem que um fator novo só seja aceito com estatística *t* acima de 3, e não de 2.

7.7.3 Otimização de parâmetros, dentro e fora da amostra

Na prática a garimpagem tem cara respeitável: chama-se otimização de parâmetros. O script `cap07/otimizacao.py` calcula o Sharpe do cruzamento de médias em PETR4 para 90 pares de janelas (rápida de 5 a 50 dias, lenta de 20 a 200) e divide o histórico em dois. O período de 2011 a 2018 é a amostra de desenvolvimento, e tudo o que se mede nele está **dentro da amostra** (*in-sample*): é ali que se escolhem os parâmetros. O período de 2019 a 2025 fica **fora da amostra** (*out-of-sample*): não participa de nenhuma escolha e só é consultado depois, uma vez, para conferir.

```
combinações testadas: 90
 rapida  lenta  dentro  fora
     10     40    0.58  0.22
      5     20    0.41  0.27
     10     60    0.41  0.04
mediana dentro: -0.1   mediana fora: 0.22
a campeã de dentro é a 45ª fora da amostra
correlação de postos dentro x fora: 0.12
máximo esperado sem habilidade, N = 10: 0.56
máximo esperado sem habilidade, N = 90: 0.88
Sharpe de comprar e manter, dentro: 0.03
Sharpe de comprar e manter, fora: 0.56
primeiro retorno de fora: 6.1%   Sharpe sem ele: 0.54
```

A campeã dentro da amostra é o par 10/40, com Sharpe de 0,58. Fora da amostra ela cai para 0,22 e ocupa a 45ª posição entre 90: saber quem ganhou no primeiro período não informa nada sobre o segundo. A correlação de postos entre os dois períodos, de 0,12, diz o mesmo para a grade inteira, e a Figura 7.7 mostra que a região boa de um mapa não é a região boa do outro. O Sharpe de 0,58 tampouco era notável. Com oito anos de dados o erro-padrão é de

$1/\sqrt{8} = 0{,}35$, e a Equação (7.8) diz que dez tentativas independentes sem habilidade já produziriam, em média, um máximo de 0,56, e noventa, de 0,88. As 90 combinações da grade são muito correlacionadas entre si e equivalem a bem menos que 90 tentativas independentes, mas uma dezena já basta para explicar o resultado.

Repare ainda na mediana, que sobe de $-0{,}10$ para $0{,}22$ do primeiro período para o segundo. Isso reflete a ação, não a regra. Comprar e manter PETR4 teve Sharpe de 0,03 de 2011 a 2018 e de 0,56 de 2019 a 2025, acima de todas as 90 variantes. Esse 0,56 conta a partir do último fechamento de 2018 e inclui a alta de 6,1% de 2 de janeiro de 2019; contando a partir do fechamento desse dia, como faz o Capítulo 9, o número é 0,54.

Um mapa de calor serve mais para avaliar estabilidade do que para escolher o melhor ponto. Um pico isolado cercado de valores ruins é ruído; uma região larga de resultados parecidos é um indício melhor, embora não uma garantia, como o exemplo mostra. E uma única divisão em dois períodos desperdiça dados e ainda depende de onde se corta. O Capítulo 8 generaliza a ideia com a análise *walk-forward* de Pardo (2008), em que a regra é reotimizada e testada em janelas sucessivas, e com esquemas de validação cruzada próprios para séries financeiras.

Armadilha **O período de teste usado duas vezes**

O período fora da amostra só vale enquanto não influencia decisões. Se você olha o resultado de 2019 a 2025, não gosta, muda a regra e testa de novo, esse período passou a fazer parte da amostra de desenvolvimento, e o número que ele produz é tão otimista quanto o de dentro. Decida as regras do jogo antes, registre quantas variantes testou e consulte o período final uma única vez.

No repositório

`aula7/03_overfitting.py` faz a busca em grade com BBAS3, dividindo o histórico em 70% e 30% e usando o retorno acumulado como critério. O script `aula7/05_backtrader_otimizacao.py` repete o exercício com ITUB4 dentro do backtrader.

7.8 Backtests orientados a eventos: o backtrader

O motor do Código 7.1 é **vetorizado**: calcula todas as posições de uma vez, com operações sobre colunas inteiras. É rápido, cabe em poucas linhas e é ideal para explorar ideias. Tem dois limites. O acesso à série completa torna o *look-ahead* fácil de cometer, como se viu. E regras que dependem do estado da carteira (um *stop* calculado a partir do preço de entrada, um limite de capital, ordens parcialmente executadas) ficam difíceis de escrever em forma de colunas.

Um motor **orientado a eventos** percorre o histórico barra por barra, na ordem do tempo. A cada barra entrega à estratégia apenas o que já aconteceu, recebe ordens e as repassa a uma corretora simulada, que decide a que preço e quando são executadas. É mais lento e mais verboso, mas a estrutura do código é a de um sistema real, e olhar o futuro passa a exigir esforço deliberado. O *backtrader* é uma biblioteca aberta de Python desse tipo. Suas peças são:

- `Cerebro`: o orquestrador. Recebe dados, estratégia e configurações, e executa o laço sobre as barras.

- ***Data feed***: a série de preços. `PandasData` converte um `DataFrame` com colunas de abertura, máxima, mínima, fechamento e volume.
- **`Strategy`**: a classe que você escreve. Em `__init__` declaram-se os indicadores; `next` é chamado uma vez por barra e contém a decisão. O índice `[0]` de uma linha é o valor da barra atual e `[-1]`, o da anterior.
- ***Broker***: a corretora simulada, com caixa inicial, comissão e regras de execução.
- ***Analyzers***: coletores de estatísticas, como retornos por período, drawdown e a lista de operações.

O Código 7.5 escreve a mesma regra 20/100 nesse formato, lendo o arquivo `petr4_ohlcv.csv`.

Código 7.5 Cruzamento de médias no backtrader: a estratégia (`cap07/l06_backtrader.py`, trecho)

```
class CruzamentoMedias(bt.Strategy):
    params = (("rapida", 20), ("lenta", 100))

    def __init__(self):
        fecho = self.data.close
        media_r = bt.ind.SMA(fecho, period=self.p.rapida)
        media_l = bt.ind.SMA(fecho, period=self.p.lenta)
        self.acima = media_r > media_l       # linha de 0 e 1

    def next(self):                          # uma vez por barra
        if self.acima[0] and not self.position:
            self.order_target_percent(target=0.95)
        elif not self.acima[0] and self.position:
            self.close()
```

A classe só descreve a regra. Dados, corretora e coletores de estatísticas são entregues ao `Cerebro`, que executa tudo:

```
dados = pd.read_csv("../../dados/petr4_ohlcv.csv",
                    index_col=0, parse_dates=True)
cerebro = bt.Cerebro()
cerebro.adddata(bt.feeds.PandasData(dataname=dados))
cerebro.addstrategy(CruzamentoMedias)
cerebro.broker.setcash(100_000)
cerebro.broker.setcommission(commission=0.001)   # 0,1% por ordem
cerebro.addanalyzer(bt.analyzers.TimeReturn, _name="diario")
cerebro.addanalyzer(bt.analyzers.TradeAnalyzer, _name="ops")
estrategia = cerebro.run()[0]
```

```
patrimônio final: R$ 198,381.96
retorno a.a. 4.8%   vol a.a. 29.3%   drawdown máx. -54.9%
operações encerradas: 27   taxa de acerto: 44.4%
ganho médio: R$ 21,667   perda média: R$ -10,406   payoff: 2.08
fator de lucro: 1.67
```

Três comportamentos do backtrader explicam o que o código não mostra. O método `next` só começa a ser chamado quando todos os indicadores têm valor, isto é, depois de 100 barras. Uma ordem a mercado criada em `next` é executada na *abertura da barra seguinte*: a decisão usa o fechamento de hoje e o negócio acontece amanhã cedo. É a hipótese intermediária da Seção 7.2,

mais conservadora que a execução no fechamento do sinal e menos que o pregão inteiro de atraso. E `order_target_percent(target=0.95)` dimensiona a compra para 95% do patrimônio, deixando folga para a comissão e para o caso de a abertura seguinte vir acima do fechamento usado na conta.

A opção *cheat-on-close* do backtrader executa no fechamento da própria barra. É a execução no fechamento do sinal do Código 7.1; o nome lembra que ela é otimista.

O retorno de 4,8% ao ano é menor do que os 8,5% do motor vetorizado porque aqui o caixa não rende nada. Feita essa mesma hipótese no Código 7.1 (script `cap07/l07_confere_motores.py`), o motor vetorizado dá 4,5% ao ano tanto com a execução no fechamento do sinal quanto com um pregão de atraso. Os dois programas, escritos de formas completamente diferentes e com três hipóteses de execução, ficam a 0,3 ponto percentual ao ano um do outro, distância compatível com a troca do preço de execução em 55 ordens e com a exposição de 95%. Essa conferência cruzada é o melhor teste de um motor de backtest: se duas implementações independentes divergem muito, uma delas tem um erro.

Armadilha **O Sharpe pronto do backtrader**

O *analyzer* `SharpeRatio` do backtrader, se usado sem argumentos, trabalha com retornos *anuais* e com uma taxa livre de risco fixa de 1% ao ano. Em um backtest de quatro anos isso é um Sharpe calculado com quatro observações contra uma taxa que não tem relação com o CDI. Prefira o que fez o Código 7.5: extraia os retornos diários com `TimeReturn` e aplique as suas próprias funções, cuja definição você conhece.

7.8.1 Estatísticas por operação

Um motor orientado a eventos registra cada operação (*trade*), da compra à venda, o que permite outro conjunto de medidas. A **taxa de acerto** p é a fração de operações encerradas com lucro. O **payoff** é o ganho médio das operações vencedoras dividido pela perda média das perdedoras. O **fator de lucro** é a soma de todos os ganhos dividida pela soma de todas as perdas, e as três se ligam por uma identidade:

$$\text{fator de lucro} = \text{payoff} \times \frac{p}{1-p}. \tag{7.9}$$

No exemplo, $p = 12/27$ e $2{,}08 \times 12/15 = 1{,}66$, que só difere do 1,67 impresso pelo arredondamento do payoff. A estratégia ganha dinheiro, em média, quando o fator de lucro é maior que 1, isto é, quando o payoff supera $(1-p)/p$.

Os números do exemplo têm o perfil típico de regras que seguem tendência: erram mais do que acertam (44% de acerto), com ganhos médios que são o dobro das perdas médias. Regras de reversão à média costumam ter o perfil oposto, com taxa de acerto alta e payoff abaixo de 1. Por isso a taxa de acerto, sozinha, não diz nada: uma regra com 90% de acerto e payoff de 0,1 perde dinheiro. E 27 operações são uma amostra minúscula. Antes de se impressionar com um fator de lucro de 1,67, lembre que o Sharpe correspondente não se distingue de zero.

No repositório

`aula7/04_backtrader_intro.py` traz uma versão comentada desta estratégia em VALE3, com `notify_order` e `notify_trade` para imprimir cada execução, e o gráfico padrão do backtrader (`cerebro.plot()`).

7.9 Lista de conferência

Além do *look-ahead*, da sobrevivência e do sobreajuste, há vieses menores que somados fazem diferença. A Tabela 7.2 resume os mais frequentes.

Tabela 7.2 Outros vieses comuns em backtests

Viés	Como aparece e como evitar
Custos ignorados	Corretagem, emolumentos, diferença entre compra e venda, aluguel de ações vendidas a descoberto e impostos. Pesam mais quanto maior o giro: refaça o teste com o dobro do custo.
Liquidez e capacidade	O backtest executa qualquer quantidade ao preço da barra. Compare o tamanho das ordens com o volume negociado no dia.
Período escolhido a dedo	Começar o teste logo depois de uma crise ou terminar antes dela. Use todo o histórico disponível e mostre o resultado por subperíodo.
Dados revisados ou com erro	Séries corrigidas depois da divulgação, preços repetidos, proventos mal ajustados. Use dados já conferidos e limpos, como os do Capítulo 2.
Narrativa a posteriori	Encontrar o padrão primeiro e inventar a explicação econômica depois. Escreva a hipótese antes de rodar o teste.

Antes de levar um backtest a sério, percorra a lista abaixo. Ela não garante que a estratégia funcione; garante que o resultado não é fruto dos erros conhecidos.

1. **Defasagem.** O retorno de cada dia é multiplicado pela posição decidida na véspera? Procure `shift(-1)`, `center=True` e estatísticas calculadas na amostra inteira.
2. **Execução.** A hipótese está declarada (fechamento do sinal, abertura seguinte)? O resultado sobrevive a um pregão de atraso?
3. **Custos.** Estão incluídos? O resultado sobrevive ao dobro deles? Qual é o giro anual?
4. **Universo.** Os ativos foram escolhidos com informação da época ou de hoje?
5. **Benchmark.** O resultado é comparado com o CDI e com comprar e manter no mesmo período?
6. **Incerteza.** Qual é o erro-padrão do Sharpe? O intervalo de confiança exclui o zero?
7. **Tentativas.** Quantas variantes foram testadas antes desta? O Sharpe supera o máximo esperado da Equação (7.8)?
8. **Fora da amostra.** Existe um período que não participou de nenhuma escolha, consultado uma única vez?
9. **Vizinhança.** Parâmetros próximos dão resultados parecidos?
10. **Bom demais.** Sharpe acima de 2 ou curva de capital sem quedas são sintomas de erro. Procure-o antes de comemorar.
11. **Reprodução.** Outra implementação, ou outra pessoa, chega ao mesmo número?

O que levar deste capítulo

- Um backtest é uma simulação sobre uma única trajetória histórica. Serve para descartar ideias e conhecer o perfil de risco de uma regra, não para provar que ela dará lucro.
- O retorno do dia t é multiplicado pela posição decidida até o fechamento de $t-1$. Esquecer essa defasagem é *look-ahead*: no exemplo, transformou 4% ao ano em 483% e, na versão discreta do erro, 8,5% em 12,3%. A execução no fechamento do sinal, convenção do livro, não é *look-ahead*, mas é otimista: declara-se e testa-se com um pregão de atraso.
- Sharpe, Sortino, Calmar e Treynor dividem um ganho por medidas diferentes de risco: oscilação total, oscilação para baixo, pior queda acumulada e risco de mercado. Compare sempre com o CDI.
- O erro-padrão de um Sharpe anualizado é cerca de um sobre a raiz do número de anos. Um Sharpe estimado de 0,5 só é significativo com mais de quinze anos de dados, e detectar com segurança um Sharpe verdadeiro desse tamanho exige cerca de trinta.
- Universos definidos hoje excluem os ativos que desapareceram e inflam o resultado, sobretudo o de regras que compram ativos depreciados.
- O melhor de muitos testes é enviesado para cima mesmo quando nenhuma regra tem valor. Registre o número de tentativas, reserve um período fora da amostra e desconfie do que parecer bom demais.

Exercícios

7.1 [básico] Uma série de cinco retornos diários em excesso do alvo é +2%, −1%, +1%, −4%, +1%. Calcule à mão o desvio abaixo do alvo pela Equação (7.3) e compare com o desvio-padrão amostral dos dois retornos negativos. Por que os números diferem?

7.2 [básico] Uma estratégia tem taxa de acerto de 35% e payoff de 2,2. Calcule o fator de lucro. Qual é o payoff mínimo para que uma regra com 60% de acerto não perca dinheiro?

7.3 [básico] Um gestor apresenta três anos de histórico diário com Sharpe de 0,9. Calcule o erro-padrão aproximado e o intervalo de 95%. O resultado é estatisticamente diferente de zero?

7.4 [intermediário] Rode `cap07/l01_vetorizado.py` com custo de 0%, 0,1%, 0,3% e 0,5% por ponta. Repita para a regra rápida do Código 7.3. Qual das duas regras é mais sensível ao custo, e por quê?

7.5 [intermediário] Modifique o Código 7.1 para que a posição fora da ação renda zero em vez do CDI, e recalcule a tabela de métricas. Quanto do retorno da estratégia vinha da renda fixa?

7.6 [intermediário] No Código 7.5, ative a execução no fechamento da própria barra com `cerebro.broker.set_coc(True)` e compare o patrimônio final. A que hipótese de execução da Seção 7.2 esse resultado corresponde?

7.7 [desafio] Altere `cap07/sobreajuste.py` para que as N estratégias tenham correlação 0,5 entre si (some a cada uma um fator comum). O Sharpe médio da melhor fica acima ou abaixo do previsto pela Equação (7.8)? O que isso implica para a contagem de tentativas em uma busca em grade?

7.8 [desafio] Refaça a simulação de `cap07/sobrevivencia.py` com dez sementes diferentes e com barreiras de deslistagem de 10%, 20% e 40% do valor inicial. Como o viés anual da carteira de sobreviventes varia com a barreira e com a volatilidade das ações?

Para ir além

- López de Prado (2018), capítulos 11 e 14, para os perigos do backtesting e as estatísticas de avaliação de uma estratégia.
- Bailey et al. (2014), que apresenta o comprimento mínimo do backtest em linguagem acessível e com exemplos numéricos.
- Lo (2002), para a distribuição do índice de Sharpe com e sem autocorrelação nos retornos.
- Chan (2021), capítulo 3, com uma discussão prática de vieses e custos por quem opera estratégias próprias.
- Pardo (2008), a referência clássica sobre otimização de parâmetros e análise *walk-forward*, que o Capítulo 8 põe em prática.

Capítulo 8

Estratégias e validação fora da amostra

O Capítulo 7 ensinou a medir uma regra de negociação e a desconfiar da medida. Falta escrever regras que tenham uma razão para existir e submetê-las ao teste que mais importa: escolher os parâmetros com uma parte dos dados e ver o que acontece na parte que a escolha não viu. Quase toda estratégia piora nesse teste, e o tamanho da piora é a informação mais útil que um backtest produz.

Ao terminar este capítulo você consegue

- formular a tese de uma estratégia e dizer quem está do outro lado da operação;
- implementar regras de momentum, de reversão à média e de rompimento sobre um mesmo motor de backtest, com defasagem e custos explícitos;
- reconhecer o perfil de retornos de cada família e dimensionar a posição por um alvo de volatilidade;
- validar uma regra por *walk-forward*, costurando apenas trechos fora da amostra;
- explicar por que a validação cruzada comum vaza informação em séries financeiras e o que *purging* e embargo corrigem;
- redigir o relatório de backtest padronizado do Núcleo.

8.1 De onde vem uma estratégia

Uma regra de negociação é uma aposta de que algum padrão dos preços vai se repetir. Para cada real ganho acima do mercado, alguém ganhou um real a menos, e por isso a primeira pergunta diante de qualquer regra é por que essa outra pessoa continuaria aceitando perder. A resposta é a **tese** da estratégia. Sem ela, a única evidência a favor da regra é o próprio backtest, e o Capítulo 7 mostrou quanto vale um backtest sozinho: testando variantes suficientes, sempre aparece uma que teria funcionado (Bailey et al., 2014).

As teses que sobrevivem a esse exame costumam cair em três grupos.

- **Prêmio de risco.** A estratégia ganha em média porque carrega um risco que os outros preferem não carregar e perde justamente quando a perda dói mais. Quem está do outro lado paga para se proteger, e continuará pagando.
- **Comportamento.** Investidores reagem devagar a notícias, vendem cedo o que subiu e seguram o que caiu, extrapolam o passado recente. Esses desvios produzem padrões que duram enquanto as pessoas continuarem agindo assim.
- **Estrutura do mercado.** Fundos obrigados a vender um papel que saiu de um índice, investidores que precisam de liquidez imediata e aceitam um preço pior, limites regulatórios. Quem está do outro lado opera por obrigação, não por opinião.

Uma tese bem escrita diz também quando a estratégia deve perder. Essa parte é a que permite distinguir, mais tarde, uma fase ruim prevista de uma regra que deixou de funcionar. As três famílias deste capítulo têm teses conhecidas e públicas, resumidas na Tabela 8.1. Padrões públicos atraem capital, e capital reduz o ganho; o que se testa em dados de 2010 a 2025 é o que restou depois disso.

Um aviso vale para todos os resultados que seguem. As sete ações do arquivo congelado foram escolhidas hoje, e o viés de sobrevivência que isso cria (Capítulo 7) favorece qualquer estratégia comprada: uma carteira montada com pesos iguais nessas sete em junho de 2012 e nunca mais mexida rendeu 16,6% ao ano até dezembro de 2025, contra 8,6% do BOVA11 (Código 8.3). Os números deste capítulo servem para aprender o método, e nenhum deles é evidência de que uma regra dá lucro.

8.2 Um motor de backtest em catorze linhas

Todas as regras do capítulo passam pela mesma função. Ela recebe a **posição-alvo**, isto é, a fração do capital que a regra quer ter em cada ativo, decidida com os dados até o fechamento de cada dia, e devolve o retorno diário líquido de custos. A notação é a do Capítulo 7: $w_{i,t}$ é o peso do ativo i assumido no fechamento do dia t, que rende de t a $t+1$; $R_{i,t}$ é o retorno simples do ativo; R^f_t é o CDI acumulado entre os fechamentos de $t-1$ e t; c é o custo por ponta. O retorno da estratégia é

$$R_t = R^f_t + \sum_i w_{i,t-1}\,(R_{i,t} - R^f_t) - c \sum_i |w_{i,t-1} - w^-_{i,t-1}|, \tag{8.1}$$

com $w^-_{i,t-1} = w_{i,t-2}(1 + R_{i,t-1})/(1 + R^{\text{bruto}}_{t-1})$. O primeiro termo diz que o capital parado rende CDI. O segundo é o ganho das posições acima do CDI, e nele o retorno de hoje é multiplicado pela posição decidida ontem, como na Equação (7.1). O terceiro cobra o custo sobre o giro (Capítulo 7), a fração do capital negociada. O peso $w^-_{i,t-1}$ é aquele a que a posição anterior chegou sozinha, depois que os preços andaram (R^{bruto} é o retorno da carteira antes dos custos): um papel que subiu mais que a carteira passa a pesar mais, e trazê-lo de volta ao alvo é uma negociação que paga custo. Com um ativo só e posição 0 ou 1 não há deriva, e a fórmula se reduz à do Capítulo 7.

Código 8.1 O motor de backtest do capítulo (`cap08/motor.py`)

```
def backtest(alvo, ret, cdi, custo=0.001, defasagem=1):
    """Retorno diário líquido de custos de uma regra.
```

```
    alvo: fração do capital desejada em cada ativo, decidida
    com a informação disponível no fechamento de cada dia.
    """
    if isinstance(alvo, pd.Series):
        alvo, ret = alvo.to_frame("a"), ret.to_frame("a")
    pos = alvo.shift(defasagem).fillna(0.0)  # w de t-1 rende em t
    bruto = cdi + (pos * ret.sub(cdi, axis=0)).sum(axis=1)
    # peso de ontem depois de derivar com os preços de ontem
    antes = (pos * (1 + ret)).div(1 + bruto, axis=0).shift(1)
    giro = (pos - antes.fillna(0.0)).abs().sum(axis=1)
    return bruto - custo * giro, giro
```

Cada linha esconde uma hipótese, e um relatório honesto as declara. A execução é no fechamento do sinal: a ordem sai no mesmo fechamento em que o sinal é calculado, o que supõe que ele fique pronto instantes antes do leilão. É a convenção do Capítulo 7, otimista. O parâmetro `defasagem` conta os pregões entre o cálculo do sinal e o retorno que a posição passa a ganhar: 1 é a execução no fechamento do sinal, 2 é um pregão de atraso, usado como teste na Seção 8.9, e 0 seria *look-ahead*. O custo é proporcional ao valor negociado, com $c = 0{,}10\%$ por ponta, hipótese única que resume taxas, metade do *spread* e *slippage* de ordens pequenas; não inclui o impacto de mercado de ordens grandes, tratado no Capítulo 11. Posições vendidas e alavancadas se financiam a CDI, sem aluguel de ações. Os pesos voltam ao alvo todos os dias, e esse rebalanceamento diário é cobrado; uma carteira que só negocia uma vez por mês giraria menos. E as médias e os canais precisam de um período de aquecimento, durante o qual a regra fica em caixa: as métricas do capítulo incluem esse trecho, que rende CDI.

O Código 8.2 faz três verificações antes de qualquer uso. A função `carregar`, do mesmo arquivo, devolve preços ajustados, retornos e o CDI nas mesmas datas.

A taxa do CDI vem da série 12 do SGS, do Banco Central, baixada uma vez por `cap08/baixar_cdi.py`. A taxa datada de um dia rende até o dia útil seguinte.

Código 8.2 Três testes do motor (`cap08/l01_motor.py`)

```
# teste 1: comprado o tempo todo, sem custo = o próprio ativo
sempre = pd.Series(1.0, index=p.index)
r, giro = backtest(sempre, ret, cdi, custo=0)
print(f"motor: {(1 + r.iloc[1:]).prod():.6f}   "
      f"preço final / inicial: {p.iloc[-1] / p.iloc[0]:.6f}")

# teste 2: uma regra que "decide" olhando o retorno do dia
vidente = (ret > 0).astype(float)
for d in (0, 1):
    r, giro = backtest(vidente, ret, cdi, custo=0, defasagem=d)
    aa = (1 + r).prod() ** (252 / len(r)) - 1
    print(f"defasagem de {d} dia(s): {aa:8.1%} ao ano")

# teste 3: o custo cobrado é custo x giro
r0, giro = backtest(vidente, ret, cdi, custo=0)
r1, _ = backtest(vidente, ret, cdi, custo=0.001)
print(f"giro anual: {giro.sum() / len(giro) * 252:.0f} vezes o "
      f"capital   custo anual: {(r0 - r1).mean() * 252:.1%}")
```

```
motor: 2.260372   preço final / inicial: 2.260372
defasagem de 0 dia(s):   316.4% ao ano
defasagem de 1 dia(s):     3.2% ao ano
```

```
giro anual: 129 vezes o capital   custo anual: 12.9%
```

Comprado o tempo todo e sem custo, o motor reproduz a variação do preço até a sexta casa decimal. O segundo teste usa uma regra absurda, que fica comprada nos dias em que o retorno do próprio dia é positivo. Com defasagem zero ela rende 316% ao ano, anualizados pela convenção de 252 dias (Seção 1.9); com defasagem de um dia, 3,2%, bem abaixo do CDI. Todo backtest com resultado extraordinário merece esse exame. O terceiro teste confere a conta do custo: 129 vezes o capital negociado por ano, a 0,10% por ponta, custam 12,9% ao ano.

8.3 Momentum

Chama-se **momentum** a aposta de que o que subiu continua subindo por algum tempo. Há duas versões, que respondem a perguntas diferentes.

8.3.1 Momentum de série temporal

Na versão de **série temporal**, também chamada de seguir tendência, cada ativo é comparado com o próprio passado: fica-se comprado se o retorno dos últimos meses foi positivo, ou se o preço está acima de uma média móvel longa. Moskowitz et al. (2012) documentaram o efeito em 58 contratos futuros e a termo de índices de ações, moedas, commodities e títulos: o retorno dos doze meses anteriores ajuda a prever o do mês seguinte. As teses usuais são comportamentais (a informação se espalha devagar e os preços se ajustam aos poucos) e de prêmio (quem segue tendência compra o que os outros estão sendo forçados a vender em crises longas).

A regra mais simples cabe em uma linha:

```python
def tendencia(p, n=200):
    """Comprado enquanto o preço supera a média de n dias."""
    return (p > p.rolling(n).mean()).astype(float)
```

Aplicada ao BOVA11 com média de 200 dias, ela produz a Figura 8.1 e os números abaixo (`cap08/l02_tendencia.py`).

```
                   BOVA11 regra s/ custo regra líquida
retorno a.a.         5.4%           4.4%          3.4%
vol a.a.            23.8%          15.4%         15.4%
Sharpe              -0.06          -0.26         -0.33
Sortino             -0.08          -0.36         -0.45
drawdown máx.      -49.7%         -27.1%        -30.1%
submerso (pregões)   1644           1121          1417
Calmar               0.11           0.16          0.11
tempo comprado: 58%   giro anual: 10.1   CDI a.a.: 10.0%
assimetria mensal: regra +0.15   BOVA11 -0.52
```

As métricas são as do Capítulo 7, calculadas pela função `metricas` do motor: Sharpe e Sortino sobre o excesso em relação ao CDI, drawdown máximo (Capítulo 5) e Calmar como retorno anual dividido por ele. A linha `submerso` é o tempo submerso, a maior sequência de pregões abaixo do pico anterior; nos termos do Capítulo 5, é a duração da queda somada ao tempo de recuperação do episódio mais longo. As taxas anuais seguem a convenção de 252 dias, que as infla ligeiramente porque a B3 tem cerca de 248 pregões por ano; é por ela que o CDI de 2010 a 2025 aparece como 10,0% ao ano. A regra fez o que um seguidor de tendência promete: reduziu a volatilidade de 23,8% para 15,4% e

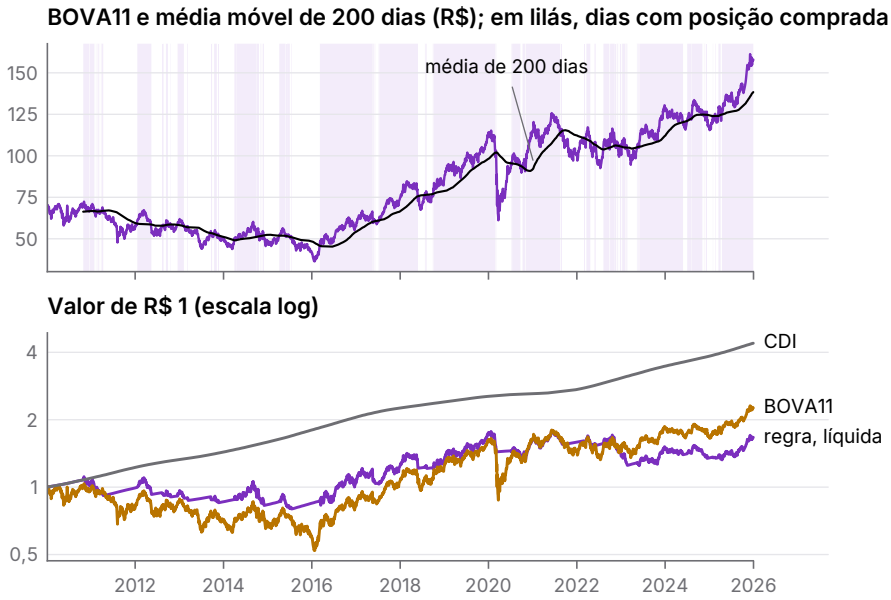


Figura 8.1 Regra de tendência no BOVA11, de 2010 a 2025. A regra evita boa parte das quedas de 2011 a 2015 e de 2020, mas compra atrasada nas retomadas e perde para o próprio índice a partir de 2016. As duas curvas terminam abaixo do CDI.

o drawdown máximo de 49,7% para 30,1%, e trocou a assimetria negativa dos retornos mensais do índice (−0,52) por uma levemente positiva (+0,15). O que ela não fez foi ganhar dinheiro. Rendeu 3,4% ao ano depois dos custos, menos que o índice e muito menos que o CDI, e por isso o Sharpe é negativo. A diferença entre a coluna sem custo e a líquida é o giro de 10,1 vezes o capital por ano multiplicado por 0,10%.

8.3.2 Momentum transversal

Na versão **transversal** (*cross-sectional*), os ativos são comparados entre si: compram-se os que mais subiram e vendem-se os que mais caíram, qualquer que tenha sido a direção do mercado. O trabalho de referência é o de Jegadeesh e Titman (1993). Com ações da NYSE e da AMEX entre 1965 e 1989, os autores ordenaram os papéis pelo retorno dos J meses anteriores ($J = 3$, 6, 9 ou 12), compraram o décimo de maior retorno, venderam o décimo de menor retorno, com pesos iguais, e mantiveram as posições por K meses ($K = 3$, 6, 9 ou 12). A combinação de seis meses de formação com seis de manutenção rendeu cerca de 1% ao mês, e o artigo mostra que parte desse ganho se desfaz nos dois anos seguintes. Uma variante pula uma semana entre a formação e a compra para afastar efeitos de microestrutura; a prática que se firmou depois foi pular o mês mais recente, o chamado momentum 12–1.

Com sete ações não há décimos. O Código 8.3 adapta a ideia: no último pregão de cada mês ordena os papéis pelo retorno de doze meses atrás até um mês atrás, compra os três primeiros e, na versão comprada e vendida, vende os três últimos. As duas últimas colunas da saída são carteiras de referência com as mesmas sete ações.

Código 8.3 Momentum transversal 12–1 com sete ações (`cap08/l03_momentum.py`)

```
# preços no último pregão de cada mês
dias = p.index.to_series()
pm = p.loc[dias.groupby(p.index.to_period("M")).max()]
```

```
# retorno de 12 meses atrás até 1 mês atrás (pula o último)
sinal = pm.shift(1) / pm.shift(12) - 1
posto = sinal.rank(axis=1, ascending=False)

vencedoras = (posto <= 3) / 3            # as 3 melhores, 1/3 cada
perdedoras = (posto >= 5) / 3            # as 3 piores
casos = {"3 venc.": vencedoras,
         "venc.-perd.": vencedoras - perdedoras,
         "1/7 diário": sinal.notna() / 7}
inicio = sinal.dropna().index[0]
res, giros = {}, {}
for nome, w in casos.items():
    alvo = w.astype(float).reindex(p.index).ffill().fillna(0)
    r, giro = backtest(alvo, ret, cdi, custo=0.001)
    res[nome] = r.loc[inicio:].iloc[1:]
    giros[nome] = f"{giro.loc[inicio:].mean() * 252:.1f}"
# referência sem negociar: 1/7 em cada ação no início, e só
dentro = p.loc[inicio:]
res["mantidas"] = (dentro / dentro.iloc[0]).mean(axis=1) \
    .pct_change().iloc[1:]
```

```
                  3 venc. venc.-perd. 1/7 diário mantidas
retorno a.a.        34.8%       23.9%      20.0%    16.6%
vol a.a.            28.1%       31.3%      26.3%    31.2%
Sharpe               0.87        0.54       0.47     0.35
Sortino              1.26        0.76       0.67     0.50
drawdown máx.      -41.3%      -54.9%     -51.0%   -74.7%
submerso (pregões)    870         490        597     1277
Calmar               0.84        0.43       0.39     0.22
giro anual: 6.0 | 20.2 | 3.6
01/06/2012 a 30/12/2025; BOVA11: 8.6% a.a.
assimetria mensal, 3 venc.: +0.69
assimetria mensal, venc.-perd.: -0.10
```

Quase 35% ao ano é o tipo de número que pede desconfiança, e ela se confirma em três etapas. A primeira é a referência. Há duas carteiras passivas na tabela: a que volta a 1/7 em cada ação todos os dias e a que compra 1/7 de cada uma no início e não negocia mais. A primeira rende 20,0% ao ano mesmo pagando por um giro de 3,6 vezes o capital; a segunda, 16,6%. As duas diferem na composição ao longo do tempo: a que não rebalanceia deixa o peso de cada papel acompanhar o preço, e a outra volta a pesos iguais todos os dias. As duas ficam muito acima do BOVA11 por causa do viés de sobrevivência. O ganho atribuível à ordenação é o que sobra acima dessas colunas.

A segunda etapa é o giro. Manter três pesos de 1/3 constantes exige negociar 6,0 vezes o capital por ano, e a versão comprada e vendida, 20,2 vezes; só a troca mensal de papéis responderia por 3,2 e 8,1 vezes (`cap08/x03_sem_mglu.py`). Quem implementasse a regra negociando apenas no fim do mês giraria menos, e esse é o tipo de decisão de execução que o Capítulo 11 mede.

A terceira é a dependência de um único papel. MGLU3 teve o preço ajustado multiplicado por mais de trezentos entre o fim de 2015 e o fim de 2020 e esteve entre as três vencedoras em quase todos os meses de meados de 2016 a meados de 2021. Sem ela no universo, as três vencedoras de seis ações rendem 18,1% ao ano, menos que os 18,8% dos pesos iguais (`cap08/x03_sem_mglu.py`): o efeito some com a retirada de um ativo. A coluna comprada e vendida inclui o CDI sobre o capital, ignora o aluguel das ações vendidas e tem drawdown de 55%;

Daniel e Moskowitz (2016) mostram que carteiras desse tipo sofrem perdas abruptas quando o mercado se recupera depois de uma queda forte, porque as perdedoras vendidas são as que mais sobem na virada.

8.4 Reversão à média

A aposta contrária é que um preço que se afastou demais de um valor de referência volta a ele. A tese mais sólida é de estrutura de mercado: quem compra o que acabou de cair está fornecendo liquidez a vendedores apressados e cobra por isso. A dificuldade está em definir o valor de referência. A escolha comum é uma média móvel, com o afastamento medido em desvios-padrão.

Definição 8.1 z-score em relação à média móvel

Sendo $x_t = \ln P_t$, m_t a média de x nos últimos n dias e s_t o desvio-padrão na mesma janela, o **z-score** é

$$z_t = \frac{x_t - m_t}{s_t}. \tag{8.2}$$

As **bandas de Bollinger** são as curvas $m_t \pm k\, s_t$, usualmente com $n = 20$ e $k = 2$; estar abaixo da banda inferior é o mesmo que ter $z_t < -k$. Aqui elas são calculadas sobre o log-preço, com o desvio-padrão amostral; a versão original de Bollinger, que o Capítulo 9 segue, usa o preço e o desvio populacional, e a diferença é pequena em janelas de 20 dias.

A regra do capítulo compra o BOVA11 quando $z_t < -2$ e zera a posição quando o preço volta à média ($z_t \geq 0$). Uma regra de entrada e saída tem memória, porque a posição de hoje depende de já se estar comprado, e a função `estado` resolve isso sem laço.

Código 8.4 Estado da posição, z-score, regra de reversão e meia-vida (`cap08/regras.py`)

```
def estado(entra, sai):
    """Posição 1 do sinal de entrada até o sinal de saída."""
    pos = pd.Series(np.nan, index=entra.index)
    pos[entra] = 1.0
    pos[sai] = 0.0
    return pos.ffill().fillna(0.0)


def zscore(p, n=20):
    """Desvio do log-preço à média móvel, em desvios."""
    x = np.log(p)
    return (x - x.rolling(n).mean()) / x.rolling(n).std()


def reversao(p, n=20, entrada=2.0):
    """Compra quando z < -entrada; sai quando z volta a zero."""
    z = zscore(p, n)
    return estado(z < -entrada, z >= 0)


def meia_vida(x):
    """Meia-vida (em dias) de um AR(1) ajustado à série x."""
```

```
    x = x.dropna()
    phi = np.polyfit(x[:-1], x[1:], 1)[0]
    return -np.log(2) / np.log(phi), phi
```

8.4.1 O que se supõe reverter precisa ser estacionário

Reversão à média só existe em séries estacionárias (Capítulo 4), e o primeiro passo é o teste ADF sobre aquilo que se supõe reverter. Para uma série estacionária, a velocidade da volta se mede ajustando um AR(1).

Resultado 8.2 Meia-vida da reversão

Se o desvio em relação à média μ segue $x_t - \mu = \phi\,(x_{t-1} - \mu) + \varepsilon_t$, com $0 < \phi < 1$, o desvio esperado daqui a h dias é $\phi^h(x_t - \mu)$. A **meia-vida** é o prazo em que ele cai à metade:

$$\phi^h = \tfrac{1}{2} \quad \Longrightarrow \quad h_{1/2} = -\frac{\ln 2}{\ln \phi}. \tag{8.3}$$

Chan (2013) chega à mesma medida regredindo Δx_t sobre x_{t-1}: o coeficiente é $\lambda = \phi - 1$ e a meia-vida é aproximada por $-\ln 2/\lambda$, o que coincide com a Equação (8.3) quando ϕ está perto de 1, pois $\ln\phi \approx \phi - 1$. A meia-vida orienta a janela da média e o prazo esperado de cada operação. O script `cap08/l04_reversao.py` faz as contas para o BOVA11:

```
ADF, log-preço: p = 0.859
ADF, desvio à média de 20 dias: p = 9.5e-18
AR(1) do desvio: phi = 0.917, meia-vida = 8.0 dias
passeio aleatório: meia-vida mediana = 8.7 dias (5% a 95%: 7.7 a 9.7)
```

O log-preço não é estacionário, como esperado. O desvio em relação à média de 20 dias passa no teste com folga e tem meia-vida de oito dias. A quarta linha desfaz o entusiasmo: 500 passeios aleatórios simulados, que por construção não têm reversão nenhuma, dão meia-vida mediana de 8,7 dias para o mesmo desvio, e o valor do BOVA11 cai dentro da faixa simulada.

Armadilha **Estacionário por construção**

Subtrair de uma série a sua própria média móvel produz uma série estacionária sempre que as variações diárias forem estacionárias, porque a média anda junto com o preço. O teste ADF e a meia-vida calculados sobre esse desvio medem a janela escolhida, e não uma propriedade do ativo. O teste só informa alguma coisa quando o valor de referência não é função do próprio preço recente, como na diferença entre dois ativos ligados por uma relação econômica.

O resultado da regra confirma o diagnóstico.

	sem custo	líquida
retorno a.a.	10.2%	9.2%
vol a.a.	14.6%	14.6%
Sharpe	0.09	0.02
Sortino	0.12	0.03
drawdown máx.	-40.7%	-40.7%
submerso (pregões)	943	998
Calmar	0.25	0.23

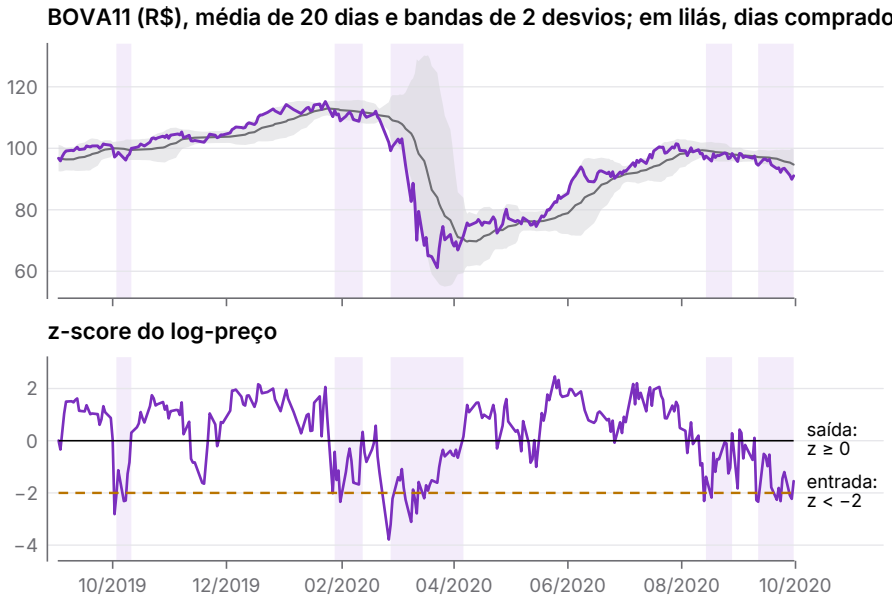


Figura 8.2 A regra de reversão no BOVA11 entre setembro de 2019 e setembro de 2020. As operações curtas à esquerda e à direita são o caso típico. No fechamento de 26 de fevereiro de 2020 a regra compra uma queda que continuou por semanas; a posição só é zerada em 6 de abril, com perda bruta de 29,8%.

```
72 operações, 67% com ganho; ganho médio +2.9%, perda média -5.6%
pior operação: -30.3%   tempo comprado: 23%   assimetria mensal: -3.51
```

Depois dos custos o Sharpe é 0,02: a regra empata com o CDI. O formato dos resultados interessa mais que a média. Dois terços das operações dão lucro, o ganho típico é pequeno e a perda média é quase o dobro do ganho médio. A assimetria dos retornos mensais é $-3{,}51$, e vem quase toda do episódio da Figura 8.2: sem o mês de março de 2020 ela seria $-0{,}62$. Sem limite de perda, uma regra de reversão continua comprada enquanto a queda durar.

8.4.2 Pares e cointegração

A saída para a armadilha do valor de referência é procurar uma combinação de ativos que seja estacionária por razões econômicas: duas classes de ação da mesma empresa, dois bancos expostos aos mesmos fatores, um fundo de índice e a cesta que ele replica. Se existe β tal que $\ln P_{A,t} - \beta \ln P_{B,t}$ é estacionário, as duas séries são **cointegradas**, e a estratégia de pares compra a combinação quando ela está abaixo da média e vende quando está acima. O teste é um ADF sobre o resíduo da regressão de um log-preço no outro, com valores críticos próprios (função `coint` do `statsmodels`). Chan (2013) desenvolve o assunto, incluindo o caso de mais de dois ativos. Uma relação estimada em uma janela pode se desfazer na seguinte, e o β precisa de validação fora da amostra como qualquer outro parâmetro.

8.5 Rompimento

Uma regra de **rompimento** (*breakout*) compra quando o preço supera a máxima de um período recente. O **canal de Donchian**, que leva o nome de Richard Donchian, um dos primeiros a negociar futuros por regras, é formado pela maior máxima e pela menor mínima dos últimos n pregões. A regra abaixo



Figura 8.3 Canal de Donchian em PETR4 entre outubro de 2015 e dezembro de 2016. A regra fica fora durante a queda de janeiro, compra no rompimento de março, é tirada por um recuo em maio e volta em junho para a alta do segundo semestre.

compra quando o fechamento supera o teto dos n dias anteriores e sai quando o fechamento perde o piso dos $n/2$ dias anteriores.

```python
def donchian(ohlc, n=20):
    """Compra no rompimento da máxima de n dias; sai quando o
    fechamento perde a mínima dos últimos n/2 dias."""
    teto = ohlc["High"].rolling(n).max().shift(1)
    piso = ohlc["Low"].rolling(n // 2).min().shift(1)
    c = ohlc["Close"]
    return estado(c > teto, c < piso)
```

O `shift(1)` garante que o canal de hoje use apenas pregões anteriores; sem ele o fechamento nunca superaria uma máxima que o inclui. A tese é a mesma do momentum de série temporal, e o rompimento é uma forma de seguir tendência: uma nova máxima de 20 dias só acontece se o preço vem subindo. A diferença está no gatilho, que exige um movimento recente e por isso reage mais depressa que uma média de 200 dias, ao preço de mais sinais falsos. A Figura 8.3 mostra a regra com $n = 20$ em PETR4, usando máximas e mínimas de `petr4_ohlcv.csv`, e a saída a seguir vem de `cap08/l05_donchian.py`.

O arquivo traz preços ajustados por proventos, o que evita rompimentos falsos do piso nas datas ex. Em 2016 a ação era negociada a valores bem maiores que os do gráfico.

```
                    PETR4 Donchian 20
retorno a.a.         9.0%       22.0%
vol a.a.            43.5%       24.9%
Sharpe               0.20        0.54
Sortino              0.28        0.80
drawdown máx.      -86.3%      -29.3%
submerso (pregões)   2380         880
Calmar               0.10        0.75
59 operações, 42% com ganho; ganho médio +16.2%, perda média -5.3%
melhor operação: +68.3%   tempo comprado: 40%   assimetria mensal: +2.47
as 3 maiores operações somam 71% do resultado
```

Dois detalhes antes de olhar a regra. PETR4 comprada rendeu 9,0% ao ano, menos que o CDI, e mesmo assim tem Sharpe positivo: o índice usa a média aritmética dos retornos diários, que supera o retorno composto em cerca de $\sigma^2/2$, e com volatilidade de 43,5% isso são mais de nove pontos percentuais ao ano. E o valor $n = 20$ não saiu de uma tese; é o número habitual, e a Seção 8.7

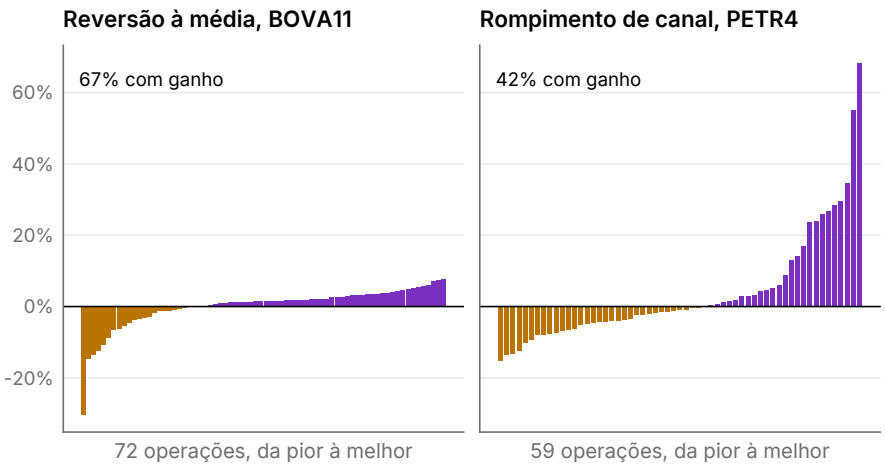


Figura 8.4 Resultado de cada operação, acima do CDI e líquido de custos, da pior à melhor. A reversão acumula muitos ganhos pequenos e uma perda de 30%. O rompimento acumula perdas pequenas e depende de poucos ganhos grandes.

Tabela 8.1 As três famílias de estratégias. A última coluna traz os números obtidos neste capítulo, com um ativo por regra; eles ilustram o perfil e não o generalizam.

Família	Tese	Ganha quando	Perde quando	Perfil nos dados do capítulo
Momentum (tendência e transversal)	informação incorporada aos poucos; investidores que extrapolam; vendas forçadas em crises	movimentos longos em uma direção	mercado lateral; viradas bruscas	BOVA11, média de 200 dias: assimetria mensal +0,15 (índice: −0,52)
Reversão à média	remuneração de quem fornece liquidez; exageros de curto prazo	oscilação em torno de um nível estável	o nível muda e o preço não volta	BOVA11: 67% de acerto, perda média maior que o ganho médio, assimetria −3,51
Rompimento	a mesma da tendência, com gatilho mais rápido	o rompimento inicia uma tendência	rompimentos falsos em sequência	PETR4: 42% de acerto, ganho médio triplo da perda média, assimetria +2,47

mostra o que acontece quando ele é escolhido sem olhar o futuro.

A regra ganha em 42% das operações e perde na maioria. O resultado vem de poucas operações longas: as três melhores, de um total de 59, respondem por 71% da soma dos resultados. É o perfil oposto ao da reversão, e a Figura 8.4 põe os dois lado a lado.

Intuição

Taxa de acerto alta e estratégia boa são coisas diferentes. A reversão acerta duas em cada três operações e empata com o CDI; o rompimento erra mais da metade e tem o melhor Sharpe entre as três regras de um ativo só. O que conta é o produto da frequência pelo tamanho, e a forma da distribuição diz que tipo de dia ruim esperar: muitas perdas pequenas seguidas, no rompimento, ou uma perda grande e rara, na reversão.

8.6 Tamanho da posição e custos

Até aqui toda posição valia 0 ou 1. Decidir quanto comprar é uma segunda regra, independente do sinal, e a mais usada é o **alvo de volatilidade**: escolher o tamanho de modo que o risco esperado da posição seja constante,

$$w_t = \min\left(\frac{\sigma^*}{\hat{\sigma}_t},\, w_{\max}\right), \tag{8.4}$$

em que σ^* é a volatilidade anual desejada, $\hat{\sigma}_t$ é a volatilidade estimada com dados até t e $w_{\max}$ limita a alavancagem. A ideia se apoia no agrupamento de volatilidade do Capítulo 4: a volatilidade recente prevê razoavelmente a dos próximos dias, muito melhor do que o retorno recente prevê o retorno.

```
ALVO = 0.20                                  # 20% ao ano
vol = ret.rolling(20).std() * np.sqrt(252)  # vol. realizada
tamanho = (ALVO / vol).clip(upper=1.0)      # sem alavancagem
```

```
                    PETR4 PETR4+alvo Donchian Donch.+alvo
retorno a.a.         9.0%      11.3%    22.0%       13.5%
vol a.a.            43.5%      21.9%    24.9%       13.8%
Sharpe               0.20       0.16     0.54        0.30
Sortino              0.28       0.23     0.80        0.42
drawdown máx.      -86.3%     -48.7%   -29.3%      -20.2%
submerso (pregões)   2380       1843      880         880
Calmar               0.10       0.23     0.75        0.67
tamanho médio: 0.61   em março de 2020: 0.18
```

Em PETR4 comprada de 2010 a 2025, o alvo de 20% entregou volatilidade de 21,9% e cortou o drawdown máximo de 86% para 49%. Em março de 2020 a posição média foi de 0,18 do capital, e a queda daquele ano passou de 63% para 26%. O Sharpe não melhorou, e na regra de Donchian caiu de 0,54 para 0,30. O alvo de volatilidade é uma ferramenta de controle de risco e de comparação entre ativos, e deve ser avaliado como tal. Ele também gira a carteira: 5,5 vezes o capital por ano só para manter o risco de uma posição comprada (`cap08/l06_vol_alvo.py`).

Esse giro é o elo entre a regra e o custo. Pela Equação (8.1), o custo anual é c vezes o giro anual, e a conta inversa dá o **custo de equilíbrio**: o custo por ponta que zera o ganho, $c^* =$ ganho bruto anual/giro anual. Para uma estratégia que mantém o capital em caixa, o ganho a zerar é o excesso bruto sobre o CDI; para uma carteira comprada e vendida de soma zero, como a do Capítulo 11, o retorno bruto já é um excesso e a fórmula é a mesma. Uma regra que gira 129 vezes o capital por ano, como a do segundo teste do motor, precisa de 12,9% ao ano de excesso bruto só para pagar um custo de 0,10% por ponta. Regras de reversão de poucos dias costumam ter esse perfil, e são as mais sensíveis à hipótese de custo.

8.7 Dentro e fora da amostra: walk-forward

O valor $n = 20$ do canal de Donchian foi usado sem justificativa. Suponha que ele tivesse sido escolhido da forma habitual, testando $n = 10, 15, \ldots, 100$ e ficando com o melhor Sharpe. O painel esquerdo da Figura 8.8, mais adiante, mostra que $n = 20$ estaria no topo: dos 19 valores, só $n = 15$ faz ligeiramente melhor na amostra inteira (0,55 contra 0,54). Um Sharpe assim é o máximo

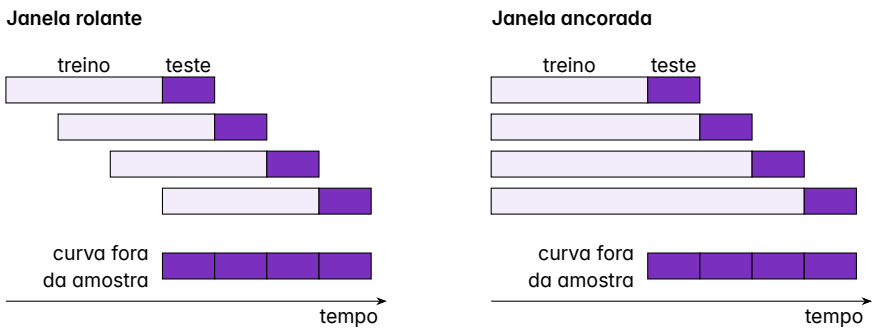


Figura 8.5 Duas formas de walk-forward. Em cada linha o parâmetro é escolhido no trecho claro e aplicado no trecho escuro seguinte. A curva de capital avaliada é a costura dos trechos de teste, na última linha; nenhum dia dela participou da escolha do parâmetro que o operou.

de 19 tentativas, e parte dele é sorte. Na linguagem do Capítulo 7, os dados usados para escolher parâmetros estão dentro da amostra (*in-sample*), e o desempenho que interessa é o dos dados que a escolha não viu, fora da amostra (*out-of-sample*).

A forma mais simples de separar as duas coisas é uma divisão única: otimizar de 2010 a 2018, por exemplo, e testar de 2019 a 2025. Ela é frágil por três razões. O resultado depende da data de corte, e mover o corte um ano pode mudar a conclusão. O período de teste é um só, com um único regime de mercado. E a tentação de olhar o teste, ajustar a regra e olhar de novo transforma o teste em treino sem que nada no código registre isso.

A **análise walk-forward**, sistematizada por Pardo (2008), repete a divisão muitas vezes, imitando o que um operador faria na prática: otimiza com os dados que existiam em uma data, opera o período seguinte com o parâmetro escolhido, e ao fim desse período refaz a otimização. A janela de treino pode ser **rolante**, de tamanho fixo, ou **ancorada**, sempre começando no primeiro dado (Figura 8.5). A rolante se adapta a mudanças de regime e descarta dados antigos; a ancorada usa tudo e muda de ideia mais devagar. O Capítulo 9 chama as mesmas janelas de deslizante e expansível.

O Código 8.5 aplica o procedimento à regra de Donchian em PETR4, com 756 pregões de treino (cerca de três anos) e 252 de teste.

Código 8.5 Walk-forward com reotimização de n a cada 252 pregões (`cap08/l07_walkforward.py`)

```
GRADE = range(10, 105, 5)       # candidatos a n: 10, 15, ..., 100
TREINO, TESTE = 756, 252        # três anos e um ano, em pregões
# posição e retorno líquido de cada candidato, amostra inteira
alvos = pd.DataFrame({n: donchian(ohlc, n) for n in GRADE})
rets = alvos.apply(lambda a: backtest(a, ret, cdi)[0])


def walk_forward(ancorada=False):
    costura = pd.Series(0.0, index=ohlc.index)
    linhas = []
    for ini in range(TREINO, len(ohlc), TESTE):
        comeco = 0 if ancorada else ini - TREINO
        treino = rets.iloc[comeco:ini]
        n = treino.apply(sharpe).idxmax()   # só dados passados
        fim = min(ini + TESTE, len(ohlc))
        costura.iloc[ini:fim] = alvos[n].iloc[ini:fim]
        teste = rets[n].iloc[ini:fim]
```

```
            linhas.append([ohlc.index[ini].year, n, sharpe(treino[n]),
                           sharpe(teste), excesso(treino[n]),
                           excesso(teste)])
    r, _ = backtest(costura, ret, cdi)      # custos na costura
    cols = ["ano", "n", "SR treino", "SR teste", "ex_tr", "ex_te"]
    tab = pd.DataFrame(linhas, columns=cols)
    return r.iloc[TREINO:], tab, costura
```

As funções `sharpe` e `excesso`, omitidas da listagem, calculam o índice de Sharpe e o retorno anual acima do CDI de uma série. O laço percorre o início de cada janela de teste: mede o Sharpe dos 19 candidatos apenas no treino, guarda o melhor e copia para a série `costura` as posições desse candidato no trecho de teste. No fim, o motor roda uma única vez sobre a série costurada, de modo que a troca de parâmetro entre janelas também paga custo quando muda a posição. Dois detalhes: como o ano da B3 tem cerca de 248 pregões, as janelas de 252 começam cada vez um pouco mais tarde no calendário; e o parâmetro novo passa a valer na posição do segundo dia de cada janela, um pregão de atraso que não usa o futuro.

```
 ano  n  SR treino  SR teste
2013 40       0.09      0.17
2014 40       0.13     -0.21
2015 20       0.36      1.83
2016 20       1.12      1.41
2017 20       1.37      0.89
2018 20       1.41      0.38
2019 15       1.28      0.03
2020 15       0.99      0.81
2021 15       0.84      0.86
2022 30       0.79     -0.21
2023 10       0.85      1.45
2024 10       1.12     -0.54
2025 10       0.69     -1.70
médias: SR treino 0.85, SR teste 0.40
excesso anual médio: treino 23.6%, teste 14.7%; razão 62%
melhor n fixo no período: 15 (Sharpe 0.77); mediana da grade: 0.13
                  n fixo ótimo WF rolante WF ancorada
retorno a.a.              29.9%      24.3%       19.6%
vol a.a.                  26.2%      27.5%       26.9%
Sharpe                     0.77       0.59        0.45
Sortino                    1.18       0.86        0.65
drawdown máx.            -27.3%     -33.6%      -27.8%
submerso (pregões)          384        367         412
Calmar                     1.09       0.72        0.71
n na ancorada: [40, 40, 40, 30, 20, 20, 15, 15, 20, 20, 20, 20, 20]
```

A coluna `ano` é o ano em que cada janela de teste começa; a primeira vai de 27 de março de 2013 a abril de 2014. A leitura tem três camadas.

O parâmetro escolhido muda: 40, depois 20, depois 15, e 10 nas três últimas janelas. Em cada janela o Sharpe do treino é o do vencedor entre 19, e a média de 0,85 no treino cai para 0,40 no teste. Essa queda dá a ordem de grandeza do que a otimização prometia a mais; parte dela é mudança de regime entre treino e teste. Pardo (2008) resume a comparação em um número, a **eficiência walk-forward**: a razão entre o lucro anualizado fora da amostra e o lucro anualizado obtido na otimização. Calculada aqui com o excesso sobre o CDI, ela é de 62%. Pardo toma valores a partir de 50% como sinal aceitável; abaixo disso, a regra provavelmente foi ajustada ao ruído do treino.

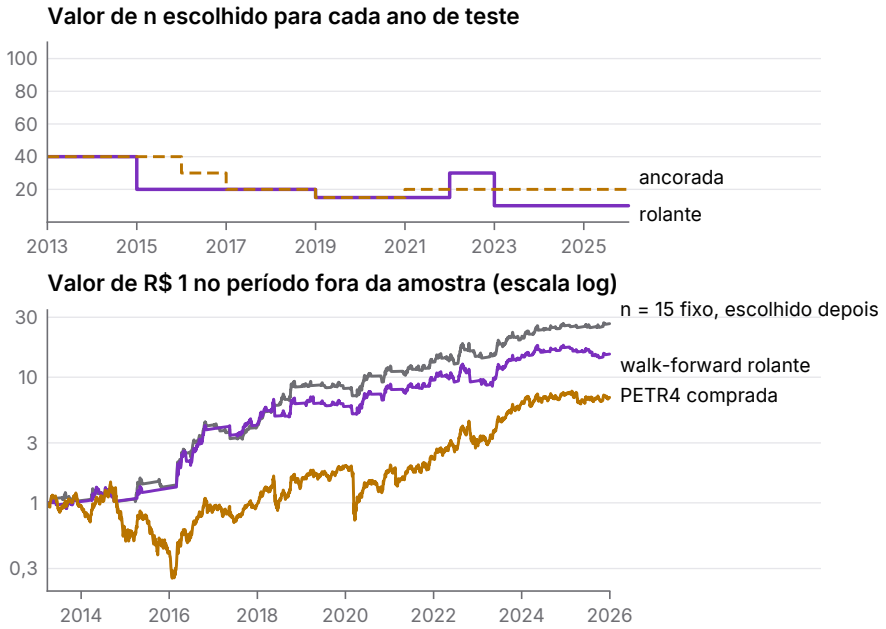


Figura 8.6 Walk-forward da regra de Donchian em PETR4. Em cima, o valor de n escolhido em cada reotimização, nas janelas rolante e ancorada. Embaixo, R$ 1 aplicado em março de 2013 no n fixo escolhido depois, no walk-forward rolante e em PETR4 comprada. Das duas curvas da regra, só a roxa poderia ter sido obtida.

A segunda camada compara três formas de chegar ao mesmo período, de março de 2013 a dezembro de 2025. Quem escolhesse o melhor n fixo olhando o período inteiro relataria Sharpe de 0,77. O walk-forward rolante, que nunca usou o futuro, obteve 0,59, e o ancorado, 0,45. A diferença entre 0,77 e 0,59 é o que a escolha a posteriori acrescenta ao relatório sem acrescentar ao bolso. Ela é uma estimativa ruidosa: o sinal é o esperado, o tamanho não é preciso. A mediana da grade, 0,13, é o resultado típico de um n escolhido ao acaso (Figura 8.6).

O Sharpe do período inteiro (0,59) e a média dos Sharpes das janelas (0,40) são estatísticas diferentes. Na primeira pesam mais os anos de ganhos grandes; a segunda dá o mesmo peso a todas as janelas.

A terceira camada é a que impede a comemoração. O Sharpe fora da amostra é negativo em quatro das treze janelas e quase nulo em outras duas, e a Seção 8.9 mostra de onde vem o resultado positivo.

Armadilha **Walk-forward também se sobreajusta**

O procedimento tem os seus próprios parâmetros: tamanho do treino e do teste, grade de candidatos, critério de escolha. Quem o roda dez vezes mudando essas escolhas e relata a melhor voltou ao ponto de partida. López de Prado (2018) aponta outra limitação: ele testa um único caminho, a história tal como aconteceu, e uma regra pode ir bem nele por causa da ordem particular dos eventos. Fixe a configuração antes de olhar o resultado e conte no relatório quantas foram tentadas.

8.8 Validação cruzada em séries financeiras

O walk-forward usa cada observação como teste uma única vez e nunca treina com dados posteriores ao teste. A validação cruzada em k partes (*k-fold*), padrão em aprendizado de máquina e retomada no Capítulo 9, aproveita melhor os dados: divide a amostra em k blocos, e cada bloco serve de teste uma vez, com

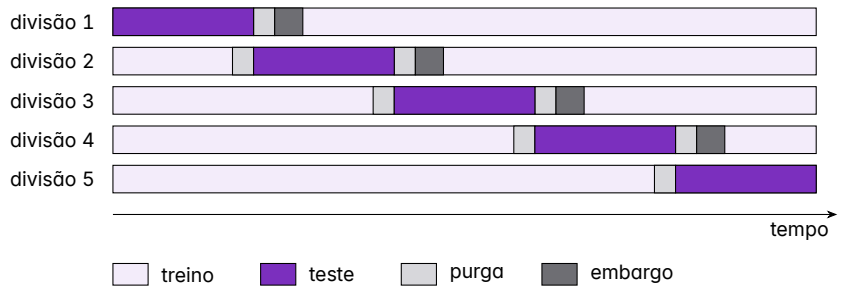


Figura 8.7 Validação cruzada em cinco blocos com purga e embargo. Cada linha é uma divisão: o bloco escuro é o teste, e o treino usa o restante, antes e depois, menos as faixas vizinhas. Para um rótulo de horizonte h a purga remove $h - 1$ observações de cada lado (Capítulo 9); o embargo só existe depois do teste.

o modelo treinado em todos os outros, anteriores e posteriores. Em dados financeiros a versão comum vaza informação do teste para o treino por dois caminhos.

O primeiro é o embaralhamento. Sorteando observações ao acaso, o dia 15 de março cai no teste e os dias 14 e 16 no treino. Como indicadores calculados em janelas e a volatilidade mudam pouco de um dia para o outro, o modelo é testado em algo que praticamente já viu. O remédio é usar blocos contíguos no tempo.

O segundo persiste mesmo com blocos contíguos. Em um modelo que prevê o retorno dos 20 dias seguintes, a observação do dia t tem um rótulo que depende dos preços de t a $t + 20$. Uma observação de treino cinco dias antes do início do bloco de teste tem um rótulo calculado com quinze dias de preços que pertencem ao teste. López de Prado (2018), no capítulo 7, propõe duas correções (Figura 8.7).

- **Purga** (*purging*). Eliminar do treino toda observação cujo rótulo se sobrepõe no tempo a algum rótulo do teste, dos dois lados do bloco.
- **Embargo.** Eliminar também um trecho do treino logo depois do teste, porque a dependência serial das variáveis faz com que as primeiras observações seguintes ainda carreguem informação dele.

Uma regra sem rótulo de previsão, como o canal de Donchian, tem um problema análogo: uma operação aberta no treino pode atravessar a fronteira e terminar dentro do teste. A solução é a mesma, descartar do treino as operações que cruzam a fronteira.

A validação cruzada purgada ainda produz uma única curva fora da amostra. A **validação cruzada combinatória purgada** (CPCV), do capítulo 12 do mesmo livro, divide a amostra em N grupos e usa como teste todas as combinações de k grupos. Com $N = 6$ e $k = 2$ são 15 divisões, e cada grupo aparece no teste cinco vezes, o que permite montar cinco caminhos completos fora da amostra em vez de um. O resultado passa a ser uma distribuição de Sharpes, e não um número. O preço é treinar com dados posteriores ao teste, o que nenhum operador consegue fazer: o walk-forward continua sendo a simulação mais fiel da operação, e a CPCV, o exame mais exigente da regra.

8.9 Testes de robustez

Uma estratégia é robusta quando o resultado sobrevive a mudanças pequenas naquilo que não deveria importar. Há quatro perguntas de rotina, e o script `cap08/x08_robustez.py` as faz ao walk-forward de Donchian. Um aviso sobre

a escala: o erro-padrão de um Sharpe anual estimado em T anos é da ordem de $1/\sqrt{T}$ quando os retornos são independentes (Lo, 2002), o que dá cerca de 0,28 para os doze anos e meio do teste. O erro da diferença entre duas variantes na mesma amostra é menor, porque os retornos delas são correlacionados, mas escolhas arbitrárias, como a data em que as janelas começam, movem o Sharpe em mais de um décimo, como se verá. Não se escolhe entre variantes por um décimo de Sharpe.

- **Parâmetros: platô ou pico?** Se a tese é verdadeira, valores vizinhos do parâmetro devem dar resultados parecidos, e o gráfico do desempenho contra o parâmetro deve ter um platô. Um pico isolado indica que o valor ótimo encaixou em acidentes da amostra. No painel esquerdo da Figura 8.8, apenas quatro dos 19 valores (15, 20, 25 e 30) têm Sharpe acima de 0,30, e de 40 em diante o resultado fica perto de zero. É mais um pico do que um platô. O tamanho das janelas do walk-forward também pesa: com treino de dois a cinco anos e teste de seis meses ou um ano, as oito combinações dão Sharpe entre 0,44 e 0,76 no trecho comum a todas, de abril de 2015 em diante. E basta mudar o dia em que as janelas começam: deslocando o início de 21 em 21 pregões, até um ano, o Sharpe do trecho comum vai de 0,34 a 0,68. Em dez dos treze deslocamentos ele fica acima de 0,60; nos outros três, perto de 0,34, abaixo do 0,41 de PETR4 comprada no mesmo trecho. O 0,59 do relatório é um sorteio dessa distribuição.
- **Custos e execução.** A Tabela 8.2 refaz o walk-forward com custos de zero a 0,50% por ponta, na execução no fechamento do sinal e com um pregão de atraso. A regra gira pouco e resiste a custos altos quando a execução é no fechamento do sinal. Com um pregão de atraso o Sharpe cai de 0,59 para 0,50 no custo-padrão e para 0,31 no custo mais alto: parte do resultado depende de comprar no próprio dia do rompimento.
- **Subperíodos.** Aqui aparece a fragilidade. O Sharpe de 0,59 se decompõe em 0,95 nos quatro primeiros anos, 0,60 nos quatro seguintes e 0,21 nos cinco últimos, quando a ação simplesmente comprada teve 0,66. A estratégia superou o CDI em sete dos treze anos civis, e sem 2016, ano em que rendeu 197%, o Sharpe cai para 0,33. O resultado repousa em boa parte na recuperação de PETR4 em 2015 e 2016.
- **Ativos fora do conjunto.** O teste mais barato contra o sobreajuste é aplicar a regra, sem alteração, a ativos que não participaram do seu desenvolvimento. O painel direito da Figura 8.8 roda o mesmo walk-forward nos outros sete ativos do arquivo, e em PETR4 para comparação, com canal de fechamentos porque só PETR4 tem máximas e mínimas. Nos sete, o Sharpe médio é 0,22, contra 0,25 dos ativos comprados, e a regra supera o ativo em três, com folga apenas em MGLU3 e no BOVA11. A própria PETR4, nas mesmas datas, cai de 0,59 para 0,25 quando o canal passa a usar fechamentos, abaixo do 0,36 da ação comprada; uma mudança de implementação que não deveria importar leva embora mais da metade do resultado. Não há, nesses dados, evidência de que a regra acrescente algo a simplesmente carregar o ativo.

8.10 O relatório de backtest e as entregas

Um backtest só pode ser avaliado por outra pessoa se vier acompanhado de tudo o que foi suposto e de tudo o que foi tentado. O Núcleo adota um relatório de backtest de uma página com nove campos fixos, sempre na mesma ordem, para que duas estratégias possam ser comparadas lado a lado.

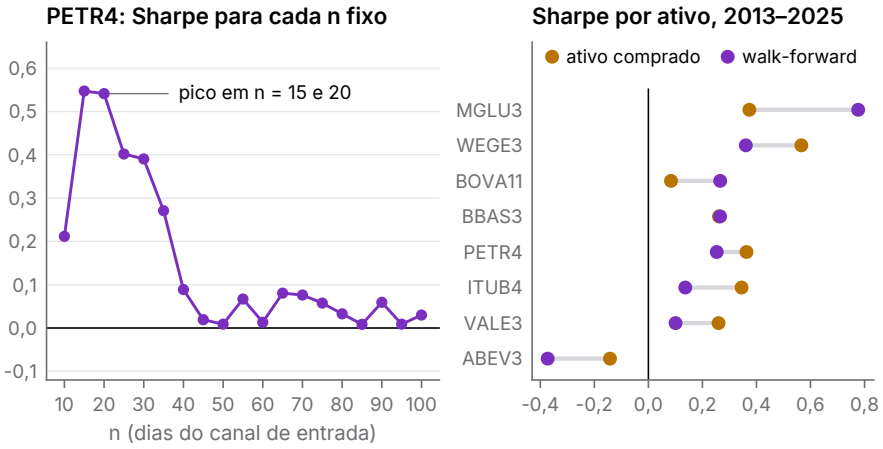


Figura 8.8 À esquerda, Sharpe da regra de Donchian em PETR4 para cada n fixo, na amostra inteira: o desempenho se concentra em uma faixa estreita. À direita, o mesmo walk-forward aplicado a cada ativo do conjunto, com canal de fechamentos (MGLU3 a partir de junho de 2014); ele supera o ativo comprado em três de oito, e PETR4 não está entre os três.

Tabela 8.2 Sensibilidade do walk-forward de Donchian em PETR4, de 2013 a 2025. À esquerda, Sharpe por custo por ponta e por hipótese de execução: no fechamento do sinal (Fech., `defasagem=1`) ou com um pregão de atraso (`defasagem=2`). À direita, subperíodos da configuração-base; a última coluna é o Sharpe da ação comprada.

Custo	Fech.	+1 pregão	Período	Ret. a.a.	Sharpe	PETR4
0	0,62	0,60	2013–2016	44,4%	0,95	0,02
0,05%	0,60	0,58	2017–2020	21,0%	0,60	0,50
0,10%	0,59	0,50	2021–2025	13,4%	0,21	0,66
0,20%	0,56	0,46	sem 2016		0,33	
0,50%	0,48	0,31				

1. **Tese.** Por que a regra deveria funcionar, quem está do outro lado e em que condições ela deve perder.
2. **Dados e universo.** Fonte, período, frequência, ajuste por proventos, critério de escolha dos ativos e o viés de sobrevivência que ele implica.
3. **Regras.** Entrada, saída e tamanho da posição, descritos de modo que outra pessoa consiga programá-los.
4. **Execução e custos.** Momento da execução em relação ao sinal, custo por ponta, tratamento do caixa, da venda a descoberto e da alavancagem.
5. **Resultados.** A tabela fixa de dez métricas, dentro e fora da amostra, ao lado de uma referência (*benchmark*) que o leitor poderia ter comprado; curva de capital e drawdown.
6. **Robustez.** Parâmetros, custos, subperíodos e outros ativos.
7. **Limitações.** O que o backtest não captura e o que faria você abandonar a estratégia.
8. **Variantes testadas.** Quantas regras, parâmetros e configurações foram experimentados até chegar à versão relatada, inclusive os descartados.
9. **Reprodução.** Arquivos, versão dos dados e sementes aleatórias.

O oitavo campo é o menos habitual e o que mais importa. Ele é o número de tentativas que o Sharpe deflacionado do Capítulo 7 pede, e sem ele ninguém consegue julgar o quinto. A Figura 8.9 traz o modelo preenchido com a estratégia das seções anteriores, com os números de `cap08/x09_relatorio.py`. A

conclusão do relatório é negativa, e ele está completo assim: as razões estão expostas e qualquer membro do Núcleo consegue auditar cada número.

8.10.1 As entregas do Núcleo e a rubrica

A proposta do Núcleo prevê quatro entregas ao longo do ciclo: a análise exploratória, na quarta semana (Seção 4.10); o relatório de risco, na sexta (Seção 6.10); o backtest completo, na oitava; e o projeto final, na décima segunda (Capítulo 12). Todas são avaliadas pela mesma rubrica de quatro dimensões com peso igual, reproduzida na Tabela 8.3. As perguntas são as da proposta; a terceira coluna diz o que o avaliador procura em qualquer uma das entregas.

Tabela 8.3 Rubrica das entregas do Núcleo. A segunda coluna reproduz as perguntas da proposta; a terceira vale para as quatro entregas.

Dimensão	Pergunta da proposta	O que se observa
Rigor técnico	Os métodos estão corretos? Os pressupostos foram verificados? O código é limpo e funcional?	Cada método aplicado onde suas hipóteses valem, com a verificação à vista (um teste, um gráfico, uma conta conferida); defeitos dos dados tratados e declarados; nenhuma informação do futuro.
Clareza	O notebook é autoexplicativo? Há narrativa conectando as etapas? Os gráficos estão rotulados?	Texto antes de cada bloco de código dizendo o que ele faz e por quê; gráficos com título, eixos e unidades; uma frase de leitura para cada tabela e figura.
Reprodutibilidade	Outra pessoa consegue rodar o notebook do zero e obter os mesmos resultados?	Dados lidos de arquivo versionado; sementes fixas; `requirements.txt`; *Restart & Run All* sem erro.
Interpretação	Os resultados foram discutidos de forma crítica? Limitações foram apontadas? Há conclusão objetiva?	O que os números permitem afirmar e o que não permitem; limitações dos dados e do método; uma conclusão que os números sustentem, mesmo que negativa.

A entrega 3 tem o seguinte enunciado na proposta: *implementação de uma estratégia quantitativa com backtest rigoroso, métricas de performance e análise de robustez.* O produto é um notebook que roda de cima a baixo e termina com o relatório de uma página. Você pode partir de uma das três famílias deste capítulo ou propor outra, desde que escreva a tese antes de rodar o primeiro teste. Nela, rigor técnico quer dizer defasagem entre sinal e posição testada, custos incluídos e parâmetros escolhidos sem usar o período de teste; clareza, os nove campos do relatório na ordem; interpretação, a comparação entre dentro e fora da amostra, os testes de robustez e a contagem de variantes. A proposta acrescenta que entregar no prazo com estrutura razoável vale mais do que entregar atrasado com código impecável: um walk-forward simples, com resultado ruim e limitações declaradas, atende às quatro dimensões.

No repositório

Os scripts deste capítulo estão em `livro/scripts/cap08/`. O motor (`motor.py`) e as regras (`regras.py`) foram escritos para ser copiados para o notebook da entrega; `l07_walkforward.py` serve de ponto de partida para a validação, e `x09_relatorio.py` calcula a tabela de métricas do relatório.

Relatório de backtest

Núcleo Quant · LMF-UFU · modelo de uma página

Rompimento de canal de Donchian em PETR4, com n reotimizado por walk-forward

1. TESE Seguir tendência: informação incorporada aos poucos faz com que uma nova máxima recente seja seguida, em média, por mais alta. Do outro lado, quem realiza lucro cedo. Deve perder em mercado lateral, com rompimentos falsos em sequência.

2. DADOS PETR4, diário, 04/01/2010 a 30/12/2025, preços ajustados (`petr4_ohlcv.csv`); CDI do SGS/BCB. Um único ativo, escolhido depois de conhecida a sua história; arquivo limpo conforme o Capítulo 2, com uma barra incoerente mantida (02/04/2014, fechamento abaixo da mínima).

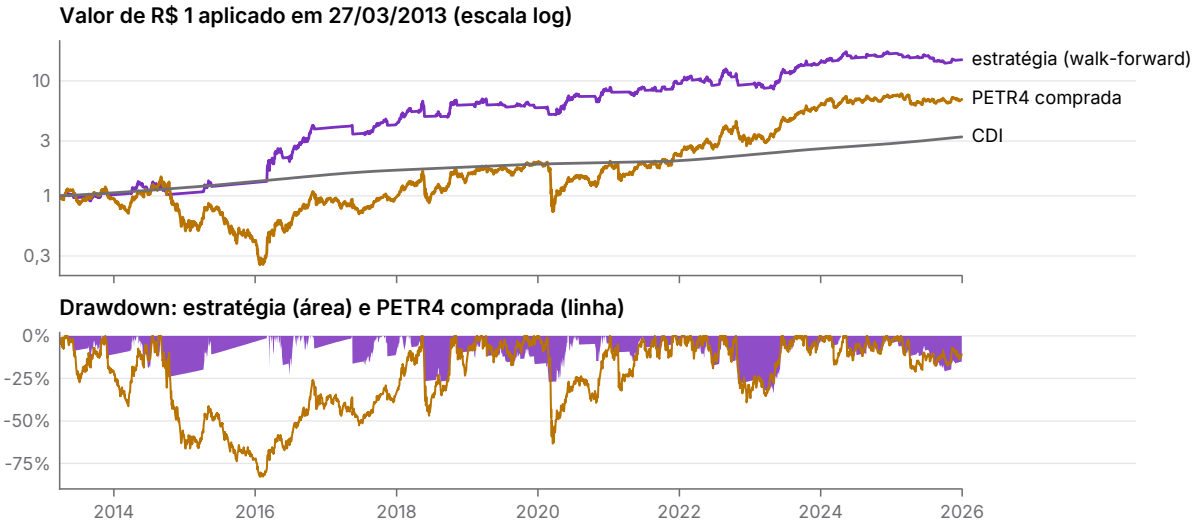
3. REGRAS Compra 100% do capital quando o fechamento supera a máxima dos n pregões anteriores; zera quando o fechamento perde a mínima dos $n/2$ anteriores. Sem venda a descoberto. A cada 252 pregões, $n \in \{10, 15, \ldots, 100\}$ é reescolhido pelo maior Sharpe dos 756 pregões anteriores.

4. EXECUÇÃO Execução no fechamento do sinal (hipótese otimista); custo de 0,10% por ponta; caixa a 100% do CDI; sem impacto de mercado.

5. RESULTADOS 27/03/2013 a 30/12/2025 (3 163 pregões)

	Na amostra ($n = 15$)	Fora da amostra	PETR4 comprada
Retorno a.a.	29,9%	24,3%	16,7%
Volatilidade a.a.	26,2%	27,5%	46,0%
Sharpe (sobre CDI)	0,77	0,59	0,36
Sortino	1,18	0,86	0,51
Drawdown máximo	−27,3%	−33,6%	−82,9%
Submerso (pregões)	384	367	908
Calmar	1,09	0,72	0,20
Nº de operações	65	59	1
Taxa de acerto	48%	47%	
Giro anual	10,4	9,4	0,1

CDI no período: 9,9% a.a.; taxas anuais pela convenção de 252 dias. "Na amostra": melhor n fixo, escolhido olhando o período inteiro.



6. ROBUSTEZ Parâmetro: só n de 15 a 30 tem Sharpe acima de 0,30 na amostra inteira (pico). Custos: Sharpe de 0,62 sem custo a 0,48 com 0,50% por ponta; com um pregão de atraso: 0,50. Subperíodos: 0,95 (2013–2016), 0,60 (2017–2020), 0,21 (2021–2025); sem 2016, 0,33. Início das janelas deslocado em até um ano: 0,34 a 0,68. Outros sete ativos: supera o ativo comprado em 3 (Sharpe médio 0,22 contra 0,25).

7. LIMITAÇÕES Um ativo, sobrevivente; resultado concentrado em 2015–2016; 7 de 13 anos acima do CDI; em drawdown desde 11/12/2024, ainda aberto no fim da amostra; canal de fechamentos reduz o Sharpe a 0,25, abaixo da ação comprada. Sem liquidez nem impacto. **Conclusão: não recomendada para capital real.**

8. VARIANTES TESTADAS 19 valores de n por janela; 2 tipos de janela (rolante, ancorada: Sharpe 0,45); 8 tamanhos de treino e teste (0,44 a 0,76 de abril de 2015 em diante); 13 datas de início; 2 definições de canal. Saída em $n/2$ fixada antes dos testes.

9. REPRODUÇÃO `livro/scripts/cap08/`: `motor.py`, `regras.py`, `l07_walkforward.py`, `x08_robustez.py`, `x09_relatorio.py`; dados congelados em `livro/dados/`; sem números aleatórios.

Figura 8.9 O relatório de backtest padronizado do Núcleo, preenchido com a estratégia deste capítulo. Na coluna "fora da amostra", cada dia foi operado com um parâmetro escolhido apenas com dados anteriores.

O que levar deste capítulo

- Toda regra precisa de uma tese que diga por que o padrão existe, quem está do outro lado e quando a regra deve perder. Sem tese, o backtest é a única evidência, e ele é fraco.
- O retorno de hoje é multiplicado pela posição decidida ontem. Um motor de backtest se resume a essa defasagem, ao custo sobre o giro e a hipóteses de execução que precisam estar escritas.
- Nos dados do capítulo, tendência e rompimento erram a maioria das operações e dependem de poucos ganhos grandes; a reversão acerta a maioria e concentra o risco em perdas raras e grandes.
- O walk-forward escolhe parâmetros apenas com dados passados e avalia a costura dos trechos de teste. A diferença para a otimização na amostra inteira (Sharpe de 0,77 contra 0,59 no exemplo) mede o que a escolha a posteriori infla.
- Um resultado fora da amostra ainda precisa passar por parâmetros vizinhos, datas de início das janelas, custos, subperíodos e outros ativos. O relatório diz quantas variantes foram tentadas.

Exercícios

8.1 [básico] Uma série de desvios segue um AR(1) com $\phi = 0{,}95$. Calcule a meia-vida pela Equação (8.3) e pela aproximação $-\ln 2/(\phi - 1)$. Repita para $\phi = 0{,}5$ e explique por que a aproximação piora.

8.2 [básico] Uma regra gira 25 vezes o capital por ano e rende, antes dos custos, 4 pontos percentuais ao ano acima do CDI. Qual é o custo de equilíbrio? Compare com o custo por ponta de 0,10%.

8.3 [intermediário] Teste se `ITUB4.SA` e `BBAS3.SA` são cointegrados de 2010 a 2025 com a função `coint` do `statsmodels`. Calcule a meia-vida da diferença dos log-preços e compare com a meia-vida do desvio de cada papel em relação à própria média de 20 dias. O que os dois números dizem?

8.4 [intermediário] No Código 8.5, troque o critério de escolha de n do Sharpe para o índice de Calmar no treino. Os valores escolhidos mudam? A mudança no Sharpe fora da amostra é maior do que as diferenças que as escolhas arbitrárias da Seção 8.9 produzem?

8.5 [desafio] Implemente a validação cruzada em cinco blocos contíguos para a escolha de n na regra de Donchian, descartando do treino as operações que cruzam a fronteira de um bloco de teste. Compare o Sharpe médio de teste com o do walk-forward e discuta por que os dois podem diferir.

8.6 [desafio] Escolha uma das três famílias, escreva a tese em um parágrafo antes de rodar qualquer código e preencha o relatório de uma página para um ativo que não seja PETR4. Registre no campo 8 todas as variantes que experimentar.

Para ir além

- Pardo (2008), para a análise walk-forward tratada por quem a sistematizou.
- López de Prado (2018), capítulo 7, para purga e embargo, e capítulos 11 e 12, para os perigos do backtest, as limitações do walk-forward e a validação combinatória.
- Chan (2013), capítulos 2 a 4, para reversão e pares, e capítulo 6, para momentum.

Parte III

Machine learning, execução e projeto

Modelos que aprendem com dados, o caminho de um sinal até uma ordem no mercado e a montagem de uma estratégia completa, da tese ao resultado.



Capítulo 9

Machine learning I: regressão, features e validação temporal

9

Um modelo de aprendizado supervisionado aprende, com exemplos do passado, uma regra que liga o que se sabe hoje ao que acontece depois. Em mercados essa regra é fraca, muda com o tempo e se confunde com ruído. Por isso este capítulo usa dois modelos lineares simples e concentra o esforço no que decide se um resultado merece confiança: as variáveis, o alvo e a validação.

Ao terminar este capítulo você consegue

- formular uma previsão financeira como problema supervisionado, datando cada variável;
- estimar e interpretar uma regressão linear e uma regressão logística;
- construir features de preço e volume usando só informação disponível na data;
- rotular observações por horizonte fixo e pelo método da tripla barreira;
- validar um modelo respeitando a ordem do tempo e reconhecer vazamentos;
- levar uma previsão até um backtest com custos e compará-la com referências simples.

9.1 Por que mercados são um problema diferente

Reconhecer um gato em uma foto é um problema em que o aprendizado de máquina funciona muito bem: o rótulo é inequívoco, gatos não mudam de aparência de um ano para o outro e cada foto é um exemplo independente das demais. Prever o retorno de uma ação viola as três condições, e ainda acrescenta uma quarta dificuldade.

- **Razão sinal-ruído muito baixa.** A parte previsível de um retorno é uma fração mínima da sua variação. Gu et al. (2020) compararam mais de uma dezena de métodos na previsão do retorno mensal de ações americanas e os melhores, redes neurais, alcançaram um R^2 fora da amostra da ordem de 0,4% ao mês. Mais de 99% da variação ficou sem explicação, e ainda assim os autores mostram que esse pouco tem valor econômico.

- **Não estacionariedade.** A relação entre as variáveis muda com o regime de juros, com a liquidez e com a composição dos participantes. Um padrão estimado em uma década pode não existir na seguinte.
- **Poucos dados independentes.** Dezesseis anos de pregões são cerca de 4 mil observações diárias de um ativo, pouco para os padrões de aprendizado de máquina, e elas dependem umas das outras no tempo.
- **Adversários.** Um padrão lucrativo atrai capital, e as ordens de quem o explora movem os preços até que ele desapareça. Parte da baixa razão sinal-ruído vem dessa força de arbitragem.

Com tão pouco sinal, um bom ajuste ao passado se obtém quase sempre e informa pouco. O trabalho está em verificar se a regra sobrevive em dados que o modelo de fato não viu.

9.1.1 O problema supervisionado e sua linha do tempo

Em **aprendizado supervisionado** cada exemplo é um par formado por um vetor de variáveis explicativas, as **features**, e por uma resposta, o **rótulo** ou alvo. O modelo é uma função f ajustada para que $f(\mathbf{x})$ fique perto de y nos exemplos conhecidos. Se y é um número, como um retorno, a tarefa é de **regressão**; se é uma categoria, como "sobe" ou "cai", é de **classificação**.

Com dados de mercado, o cuidado está nas datas. As features $\mathbf{x}_t$ só podem conter o que se sabia no fechamento do dia t. O alvo é algo que acontece depois, por exemplo o log-retorno dos h pregões seguintes,

$$y_t = r_{t \to t+h} = \ln P_{t+h} - \ln P_t, \tag{9.1}$$

e só fica conhecido em $t+h$. A Figura 9.1 desenha duas observações consecutivas com $h = 3$. Guarde dois fatos que voltam no capítulo inteiro: nenhuma seta de informação cruza a data t da direita para a esquerda, e os rótulos de dias vizinhos se sobrepõem quando $h > 1$.

9.2 Regressão linear

A regressão linear supõe que a resposta é uma soma ponderada das features mais um erro:

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_k x_{ik} + \varepsilon_i. \tag{9.2}$$

Empilhando as n observações, $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$, em que $\mathbf{X}$ é a matriz $n \times (k+1)$ com uma coluna de uns para o intercepto. O método dos **mínimos quadrados ordi-**

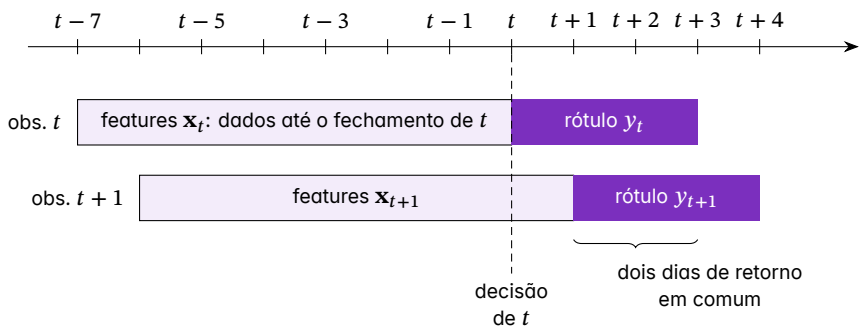


Figura 9.1 Linha do tempo de duas observações com horizonte $h = 3$. As features da observação t usam apenas o passado; o rótulo é o retorno dos três pregões seguintes e só existe em $t+3$. Os rótulos de t e de $t+1$ compartilham dois dos três retornos diários.

nários (MQO) escolhe os coeficientes que minimizam a soma dos quadrados dos resíduos.

Resultado 9.1 Estimador de mínimos quadrados

O vetor que minimiza $S(\boldsymbol{\beta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^{\mathsf{T}}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$ é

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^{\mathsf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathsf{T}}\mathbf{y}, \tag{9.3}$$

desde que as colunas de $\mathbf{X}$ sejam linearmente independentes. Com uma única feature, a inclinação é $\hat{\beta}_1 = \widehat{\mathrm{Cov}}(x, y)/\widehat{\mathrm{Var}}(x)$.

A Equação (9.3) sai de derivar S em relação a $\boldsymbol{\beta}$ e igualar a zero, o que dá $\mathbf{X}^{\mathsf{T}}(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = \mathbf{0}$: os resíduos são ortogonais a todas as features. Cada coeficiente $\hat{\beta}_j$ é a variação esperada de y quando x_j aumenta uma unidade e as demais features ficam fixas. A qualidade do ajuste é resumida pelo **coeficiente de determinação**,

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \tag{9.4}$$

a fração da variância de y que o modelo reproduz na amostra em que foi estimado.

9.2.1 Um exemplo que funciona: o beta de uma ação

O **modelo de mercado** explica o retorno de uma ação pelo retorno do índice no mesmo dia, $R_{i,t} = \alpha_i + \beta_i R_{m,t} + \varepsilon_{i,t}$ (Campbell et al., 1997). O coeficiente β_i, o beta, mede a sensibilidade do papel ao mercado e já apareceu nos cenários de estresse do Capítulo 6. O Código 9.1 o estima para PETR4 contra o Ibovespa com a biblioteca statsmodels, que imprime a tabela clássica de regressão.

Código 9.1 Beta de PETR4 pelo modelo de mercado (`cap09/l01_beta.py`)

```
import pandas as pd
import statsmodels.api as sm

precos = pd.read_csv("../../dados/precos_ajustados.csv",
                     index_col=0, parse_dates=True)
p = precos[["PETR4.SA", "^BVSP"]].dropna()
R = p.pct_change().dropna().loc["2019":"2024"]

X = sm.add_constant(R["^BVSP"])     # coluna de uns: o intercepto
mqo = sm.OLS(R["PETR4.SA"], X).fit()
for tabela in mqo.summary(slim=True).tables:
    print(tabela)

# mesmos coeficientes, erros-padrão de Newey-West (HAC)
hac = sm.OLS(R["PETR4.SA"], X).fit(
    cov_type="HAC", cov_kwds={"maxlags": 5})
hc0 = sm.OLS(R["PETR4.SA"], X).fit(cov_type="HC0")
print(f"erro-padrão do beta: MQO {mqo.bse.iloc[1]:.4f}"
      f" | HC0 {hc0.bse.iloc[1]:.4f}"
      f" | HAC {hac.bse.iloc[1]:.4f}")
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                PETR4.SA   R-squared:                       0.544
Model:                            OLS   Adj. R-squared:                  0.544
No. Observations:                1492   F-statistic:                     1779.
Covariance Type:            nonrobust   Prob (F-statistic):          1.58e-256
==============================================================================
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0010      0.000      2.239      0.025       0.000       0.002
^BVSP          1.2671      0.030     42.182      0.000       1.208       1.326
==============================================================================
erro-padrão do beta: MQO 0.0300 | HC0 0.0648 | HAC 0.0749
```

A coluna `coef` traz as estimativas. O beta de 1,27 diz que, entre 2019 e 2024, um dia em que o Ibovespa subiu 1% foi em média um dia em que PETR4 subiu 1,27%. A coluna `std err` é o erro-padrão de cada estimativa, `t` é a razão entre as duas e `P>|t|` é o valor-p do teste de que o coeficiente verdadeiro é zero. As duas últimas colunas formam o intervalo de 95% de confiança. O R^2 de 0,544 indica que o índice reproduz pouco mais da metade da variância diária da ação; na regressão com uma feature ele é o quadrado da correlação entre as duas séries. A Figura 9.2 mostra a nuvem de pontos e a reta ajustada.

9.2.2 As hipóteses e o que falha com dados financeiros

O estimador da Equação (9.3) pode ser calculado para quaisquer dados. As boas propriedades que os livros lhe atribuem, e os erros-padrão que o programa imprime, dependem de hipóteses sobre o erro ε: média zero dadas as features, variância constante (homocedasticidade), ausência de correlação entre erros de datas diferentes e features que não sejam combinação linear umas das outras. James et al. (2021), no capítulo 3, discutem cada uma. Com retornos, três delas falham de forma previsível.

- **Heterocedasticidade.** A volatilidade se agrupa no tempo (Capítulo 4), de modo que a variância do erro em março de 2020 não é a de um mês calmo. Os coeficientes continuam bem estimados em média, mas os erros-padrão usuais ficam errados, quase sempre pequenos demais.

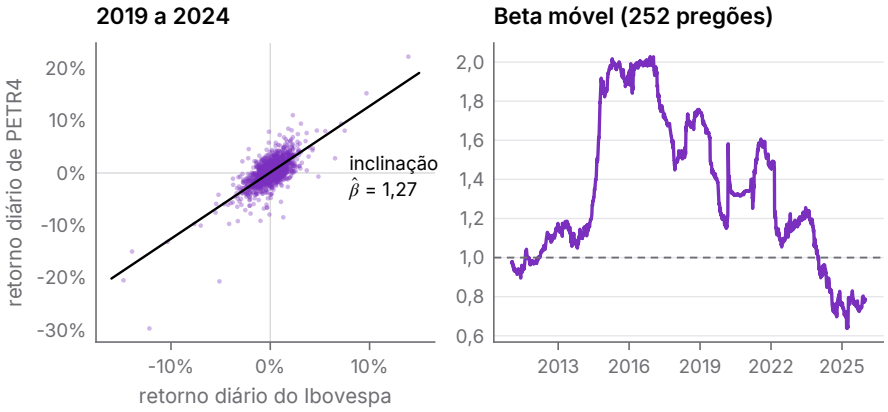


Figura 9.2 Modelo de mercado para PETR4. À esquerda, cada ponto é um pregão de 2019 a 2024 e a reta é o ajuste de MQO. À direita, o mesmo beta estimado em janelas móveis de 252 pregões: ele passou de 2,0 em 2016 e ficou abaixo de 0,7 em 2025. O coeficiente da regressão é uma média de um parâmetro que não ficou parado.

- **Autocorrelação dos erros.** Aparece sempre que o alvo cobre vários dias e as observações são diárias, pela sobreposição da Figura 9.1. O efeito é o mesmo: uma precisão aparente que não existe.
- **Multicolinearidade.** Indicadores técnicos são transformações do mesmo preço e andam juntos. Quando duas features são quase iguais, a matriz $\mathbf{X}^{\top}\mathbf{X}$ fica próxima de singular e os coeficientes individuais se tornam instáveis, com sinais opostos que se compensam, embora a previsão conjunta pouco mude.

Para os dois primeiros problemas existe um remédio padrão: manter os coeficientes de MQO e trocar a fórmula dos erros-padrão por uma que admita variância variável e autocorrelação, a de Newey e West (1987), conhecida pela sigla HAC. A última linha da saída do Código 9.1 mostra a diferença: o erro-padrão do beta passa de 0,030 para 0,075, duas vezes e meia maior. Quase todo o aumento vem da heterocedasticidade, pois a correção só para ela (HC0) já leva a 0,065; a parcela da autocorrelação, aqui com cinco defasagens, é pequena, como se espera de retornos de um dia. O intervalo de 95% se alarga de $[1{,}21;\ 1{,}33]$ para cerca de $[1{,}12;\ 1{,}41]$. A conclusão de que o beta é maior que 1 sobrevive, mas com bem menos folga do que a tabela sugeria. Para a multicolinearidade, as saídas são descartar features redundantes ou penalizar o tamanho dos coeficientes, a regularização do Capítulo 10.

Há um quarto problema, que nenhuma correção de erro-padrão resolve: o parâmetro muda. O painel direito da Figura 9.2 mostra o beta de PETR4 variando entre 0,64 e 2,03 conforme a janela de um ano escolhida.

Armadilha **Regressão contemporânea não é previsão**

O R^2 de 54% do modelo de mercado não significa que se consegue prever PETR4. O retorno do Ibovespa no dia t só é conhecido no fechamento de t, junto com o da ação. A regressão descreve como os dois se movem juntos, o que serve para medir risco e montar proteções. Para prever, o lado direito precisa estar datado antes do lado esquerdo.

9.2.3 A regressão preditiva e o R^2 fora da amostra

Mude agora uma única coisa: as features passam a ser dez indicadores calculados com dados até o dia t (a Seção 9.4 mostra como) e o alvo passa a ser o log-retorno de t a $t+1$. O modelo é estimado com os pregões de 2010 a 2018 e avaliado nos de 2019 a 2025, que ele nunca viu. Dentro da amostra de estimação o R^2 da Equação (9.4) nunca é negativo e nunca diminui quando se acrescentam features, mesmo que sejam ruído. A medida honesta usa os dados de teste e compara o modelo com uma previsão ingênua.

Definição 9.2 R^2 fora da amostra

Sejam $\hat{y}_t$ as previsões do modelo para as observações de teste e $\bar{y}_{\text{ref},t}$ uma previsão de referência que não usa as features. O R^2 **fora da amostra** é

$$R^2_{\text{fora}} = 1 - \frac{\sum_{t \in \text{teste}} (y_t - \hat{y}_t)^2}{\sum_{t \in \text{teste}} (y_t - \bar{y}_{\text{ref},t})^2}. \tag{9.5}$$

Ele é positivo se o modelo erra menos que a referência e negativo se erra mais.

Duas referências são usuais. Campbell e Thompson (2008) usam a média histórica dos retornos disponível até a data da previsão. Gu et al. (2020) usam zero, com o argumento de que a média histórica de uma ação é estimada com tanto ruído que prever zero é uma referência mais difícil de bater. O Código 9.2 mostra o núcleo da conta, com a média histórica atualizada a cada dia; a função `base`, apresentada na Seção 9.4, devolve as features e o alvo já alinhados, e `treino` e `teste` marcam as linhas de cada período.

Código 9.2 Regressão preditiva do retorno do dia seguinte (`cap09/l02_preditiva.py`)

```
modelo = LinearRegression().fit(X[treino], y[treino])
prev = modelo.predict(X[teste])


def r2_fora(y, prev, referencia):
    """1 - SQE do modelo / SQE da previsão de referência."""
    sqe = ((y - prev) ** 2).sum()
    return 1 - sqe / ((y - referencia) ** 2).sum()


# média histórica até t-1, atualizada a cada dia
media = y.expanding().mean().shift(1)[teste]
```

```
treino: 2129 dias | teste: 1741 dias
R2 dentro da amostra:       0.5482%
R2 fora, ref. média hist.:  0.1122%
R2 fora, ref. zero:         0.1133%
R2 fora por ano (ref. zero):
  2019: -0.0429%
  2020:  0.7883%
  2021: -0.1457%
  2022: -0.1222%
  2023:  0.1287%
  2024: -2.9243%
  2025: -0.7310%
```

Dentro da amostra, dez indicadores explicam 0,55% da variância do retorno do dia seguinte. Parece pouco e é menos ainda: quando k features sem qualquer relação com o alvo entram em uma regressão com n observações e erros normais, o R^2 esperado é $k/(n-1)$, aqui $10/2128 = 0{,}47\%$. O ajuste obtido é o que o puro acaso produziria. Fora da amostra o R^2 fica em torno de 0,1%, positivo no agregado, mas negativo em cinco dos sete anos. O número agregado é dominado por 2020, o ano de maior variância do período, que por isso pesa mais nas duas somas da Equação (9.5). A leitura correta é que este modelo linear, com estas features, não prevê o retorno diário de PETR4 de maneira distinguível de zero.

Resultados dessa ordem são a regra em previsão de retornos. Campbell e Thompson (2008) mostram que valores de R^2 fora da amostra muito pequenos já podem ter importância econômica para um investidor, e os números de Gu et al. (2020) citados na abertura são da mesma ordem de grandeza. Quem encontra 5% de R^2 fora da amostra em retornos diários deve procurar o vazamento antes de comemorar.

O método `score` do scikit-learn, aplicado aos dados de teste, usa como referência a média do próprio teste, que não era conhecida na hora da previsão. Prefira escrever a Equação (9.5) explicitamente.

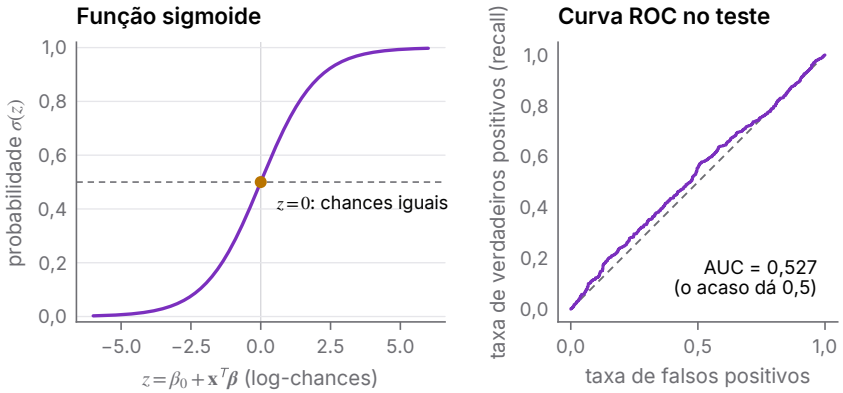


Figura 9.3 À esquerda, a função sigmoide, que converte o escore linear z em probabilidade. À direita, a curva ROC da regressão logística do Código 9.3 nos dados de teste: ela quase coincide com a diagonal, que é o desempenho de um sorteio.

9.3 Classificação e regressão logística

Para decidir entre comprar e ficar de fora, basta acertar o lado do retorno, sem acertar o tamanho. Trocar o alvo contínuo pelo seu sinal transforma a regressão em classificação, com duas vantagens: o alvo deixa de ser dominado por poucos dias extremos, e a saída do modelo passa a ser uma probabilidade, que se converte naturalmente em decisão. A desvantagem é jogar fora a magnitude.

Defina $y_t = 1$ se o retorno do período seguinte for positivo e $y_t = 0$ caso contrário. Ajustar uma reta a um alvo que só vale 0 ou 1 produziria "probabilidades" abaixo de zero e acima de um. A **regressão logística** resolve isso passando a combinação linear das features por uma função que comprime qualquer número real para o intervalo entre 0 e 1.

Definição 9.3 Regressão logística

O modelo supõe que a probabilidade da classe 1 é

$$p(\mathbf{x}) = \Pr(y = 1 \mid \mathbf{x}) = \sigma(\beta_0 + \mathbf{x}^{\mathsf{T}}\boldsymbol{\beta}), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}, \tag{9.6}$$

em que σ é a **função sigmoide**. De forma equivalente, o logaritmo da razão de chances é linear nas features:

$$\ln \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} = \beta_0 + \mathbf{x}^{\mathsf{T}}\boldsymbol{\beta}. \tag{9.7}$$

A razão $p/(1-p)$ são as **chances** (*odds*): uma probabilidade de 60% corresponde a chances de $0{,}6/0{,}4 = 1{,}5$ para 1. A Equação (9.7) dá a interpretação dos coeficientes. Aumentar x_j em uma unidade soma β_j ao logaritmo das chances, isto é, multiplica as chances por e^{β_j}, a **razão de chances**. Um coeficiente de 0,25 multiplica as chances por $e^{0{,}25} \approx 1{,}28$. O efeito sobre a probabilidade não é constante: como mostra o painel esquerdo da Figura 9.3, ele é máximo perto de $p = 0{,}5$ e some nas extremidades.

Os coeficientes são estimados por máxima verossimilhança. Para dados $(\mathbf{x}_i, y_i)$ independentes, maximizar a verossimilhança equivale a minimizar a **perda logarítmica** (*log-loss*),

$$L(\boldsymbol{\beta}) = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \ln p_i + (1 - y_i)\ln(1 - p_i)\Big], \qquad p_i = p(\mathbf{x}_i), \tag{9.8}$$

que castiga com força a previsão confiante e errada: atribuir $p_i = 0{,}99$ a um dia que acabou em queda custa $-\ln 0{,}01 = 4{,}6$, contra $-\ln 0{,}5 = 0{,}69$ de uma previsão neutra. Não há fórmula fechada como a de MQO; o mínimo é encontrado por um algoritmo iterativo (Hastie et al., 2009, seção 4.4).

A probabilidade vira decisão por meio de um **limiar**: prevê-se a classe 1 quando $p(\mathbf{x})$ passa de um valor escolhido, em geral 0,5. O limiar é uma escolha de quem usa o modelo. Subi-lo para 0,55 produz menos sinais de compra, mais seletivos.

9.3.1 Medir um classificador

O Código 9.3 ajusta a logística às mesmas dez features, com a mesma divisão de treino e teste, e prevê se o dia seguinte será de alta.

Código 9.3 Regressão logística para o sinal do retorno de amanhã (`cap09/l03_logistica.py`)

```
X, ret = base(h=1)
y = (ret > 0).astype(int)         # 1 = alta amanhã, 0 = queda
treino = X.index < "2019"
teste = ~treino

# o Pipeline ajusta média e desvio do padronizador só no treino
modelo = make_pipeline(StandardScaler(), LogisticRegression())
modelo.fit(X[treino], y[treino])
prob = modelo.predict_proba(X[teste])[:, 1]   # P(alta)
prev = (prob > 0.5).astype(int)               # limiar de decisão
```

```
matriz de confusão (linhas: real 0, 1):
[[482 348]
 [496 415]]
acurácia  0.5152
precisão  0.5439
recall    0.4555
F1        0.4958
AUC       0.5271
sempre alta (classe majoritária): 0.5233
repetir o sinal de hoje: 0.4957
variação diária média, em módulo: 0.0164
                  ret_1  ret_5  ret_21  vol_21  p_mm50
beta             -0.134  0.080   0.040   0.026   0.135
razão de chances  0.875  1.083   1.041   1.026   1.144
                  rsi_14  macd_h  pct_b  vol_rel  sexta
beta              -0.333  -0.028  0.246    0.069  0.008
razão de chances   0.717   0.972  1.279    1.072  1.008
```

A **matriz de confusão** cruza o que aconteceu (linhas) com o que foi previsto (colunas). Dos 1741 dias de teste, o modelo previu alta em $348 + 415 = 763$. Acertou 415, os verdadeiros positivos (VP), e errou 348, os falsos positivos (FP). Dos dias em que previu queda, 482 foram verdadeiros negativos (VN) e 496 foram altas que ele perdeu, os falsos negativos (FN). Todas as métricas usuais saem desses quatro números.

- **Acurácia**, $(\text{VP} + \text{VN})/n = 897/1741 = 51{,}5\%$: a fração de acertos.
- **Precisão**, $\text{VP}/(\text{VP} + \text{FP}) = 415/763 = 54{,}4\%$: quando o modelo diz "alta", quantas vezes acerta. É a métrica que importa para quem só opera quando há sinal.

- **Recall** (ou sensibilidade), VP/(VP + FN) = 415/911 = 45,6%: de todas as altas que ocorreram, quantas o modelo apontou.
- **F1**, a média harmônica de precisão e recall, $2 \times 0{,}544 \times 0{,}456/(0{,}544 + 0{,}456) = 0{,}496$, que só é alta quando as duas são.
- **AUC**, a área sob a curva ROC. A curva percorre todos os limiares possíveis e marca, para cada um, a taxa de verdadeiros positivos contra a de falsos positivos. A área tem uma leitura direta: é a probabilidade de o modelo dar nota maior a um dia de alta sorteado ao acaso do que a um dia de queda sorteado ao acaso. Vale 0,5 para um sorteio e 1 para um classificador perfeito, e não depende do limiar escolhido.

> **Armadilha** **Acurácia sem referência não diz nada**
>
> Um modelo que prevê "alta" todos os dias, sem olhar feature alguma, acerta 52,3% do teste, porque essa é a proporção de dias de alta no período. A logística, com 51,5%, perde para ele. Em problemas com classes desbalanceadas a distorção é pior: se o evento de interesse ocorre em 5% dos dias, o classificador que nunca o prevê tem 95% de acurácia e nenhuma utilidade. Sempre compare com a classe majoritária e olhe precisão, recall e AUC.

Os coeficientes impressos referem-se a features padronizadas, de modo que cada razão de chances corresponde ao aumento de um desvio-padrão na feature. Os dois maiores em módulo têm sinais opostos: −0,33 para o RSI e +0,25 para o %B de Bollinger, duas features cuja correlação na amostra é de 0,89. É o sintoma de multicolinearidade descrito na Seção 9.2, e seria um erro ler ali uma descoberta sobre o RSI.

A classe `LogisticRegression` aplica por padrão uma penalização do tipo ridge, com `C=1`. Com 2 mil observações e dez features o efeito é desprezível. O Capítulo 10 trata de regularização.

9.3.2 Quanto vale acertar 53% das vezes

Suponha um modelo hipotético que acerte o lado em 53% dos dias. Essa acurácia parece irrelevante e pode ser a base de um negócio, ou não valer nada. Três contas separam os dois casos.

A primeira é o ganho esperado por aposta. Se acertos e erros tiverem a mesma magnitude média m, quem acerta uma fração p das vezes ganha em média $(2p - 1)\,m$ por operação. Com $p = 0{,}53$ e $m = 1{,}64\%$, a variação diária média de PETR4 no período de teste, isso dá cerca de 0,1% por dia, antes de custos. A segunda conta são os custos: a 0,10% por ponta, o custo padrão deste livro, o ganho inteiro é consumido se a posição mudar todos os dias. A terceira é a incerteza da própria acurácia. Com n previsões independentes e p perto de 0,5, o erro-padrão da acurácia medida é $\sqrt{0{,}25/n}$; para os 1741 dias do teste, 1,2 ponto percentual. Mesmo que o modelo hipotético medisse 52,4% nesse teste, estaria a apenas dois erros-padrão de 50%, no limite do que o acaso explica sozinho.

A hipótese de magnitudes iguais é a que mais falha: um modelo pode acertar a maioria dos dias pequenos, errar os grandes e perder dinheiro com acurácia acima de 50%. E um acerto pequeno só vira resultado quando é repetido em muitas apostas pouco correlacionadas, ideia que Grinold e Kahn (2000) formalizam ao relacionar o desempenho de um gestor à qualidade de cada previsão multiplicada pela raiz do número de apostas independentes.

9.4 Engenharia de features com dados de mercado

Chama-se **engenharia de features** o trabalho de transformar dados brutos em variáveis que carreguem informação em um formato que o modelo consiga usar. Com preço e volume diários, as famílias mais comuns estão na Tabela 9.1, e o Código 9.4 as implementa para PETR4.

Código 9.4 Construção das features (`cap09/features.py`, trecho)

```
def wilder(x, n):
    """Média de Wilder: começa na média simples dos n primeiros
    valores e depois segue m_t = m_{t-1} + (x_t - m_{t-1}) / n."""
    m = x.copy()
    m.iloc[:n - 1] = np.nan
    m.iloc[n - 1] = x.iloc[:n].mean()
    return m.ewm(alpha=1 / n, adjust=False).mean()


def rsi(p, n=14):
    d = p.diff().dropna()
    ganho = wilder(d.clip(lower=0), n)       # altas médias
    perda = wilder(-d.clip(upper=0), n)      # quedas médias
    return 100 - 100 / (1 + ganho / perda)


def mme(x, n):
    """Média móvel exponencial com alfa = 2 / (n + 1)."""
    return x.ewm(span=n, adjust=False).mean()


def montar(d):
    """Cada linha t usa apenas dados até o fechamento de t."""
    p, v = d["Close"], d["Volume"]
    r = np.log(p).diff()
    X = pd.DataFrame(index=d.index)
    X["ret_1"] = r                           # log-retorno do dia
    X["ret_5"] = r.rolling(5).sum()          # momentum semanal
    X["ret_21"] = r.rolling(21).sum()        # momentum mensal
    X["vol_21"] = r.rolling(21).std()        # vol. realizada
    X["p_mm50"] = p / p.rolling(50).mean() - 1
    X["rsi_14"] = rsi(p, 14)
    macd = (mme(p, 12) - mme(p, 26)) / p     # em fração do preço
    X["macd_h"] = macd - mme(macd, 9)        # histograma
    mm, dp = p.rolling(20).mean(), p.rolling(20).std(ddof=0)
    X["pct_b"] = (p - (mm - 2 * dp)) / (4 * dp)
    X["vol_rel"] = np.log(v / v.rolling(20).mean())
    # t é sexta: o alvo de 1 dia é, em geral, o retorno de segunda
    X["sexta"] = (d.index.dayofweek == 4).astype(float)
    return X
```

```
pregões no arquivo: 3920 | intervalos de mais de 5 dias corridos: 5
linhas completas: 3870, de 2010-03-17 a 2025-12-29
linha de 2020-03-09 igual sem os dados futuros: True
rsi_14  pct_b     0.89
ret_21  p_mm50    0.88
p_mm50  rsi_14    0.85
```

Tabela 9.1 Features do capítulo, todas calculadas com dados até o fechamento de t

Coluna	Definição e ideia
`ret_1, ret_5, ret_21`	Log-retorno acumulado de 1, 5 e 21 pregões. Medem *momentum* (ou reversão) em vários horizontes.
`vol_21`	Desvio-padrão dos log-retornos diários em 21 pregões: a volatilidade realizada recente.
`p_mm50`	Preço dividido pela média móvel de 50 dias, menos 1. Distância em relação à tendência.
`rsi_14`	Índice de força relativa de Wilder: peso das altas no movimento recente, de 0 a 100.
`macd_h`	Histograma do MACD: diferença entre médias exponenciais de 12 e 26 dias, menos a sua própria média de 9 dias, em fração do preço.
`pct_b`	%B de Bollinger: posição do preço entre as bandas de dois desvios em torno da média de 20 dias (0 na inferior, 1 na superior).
`vol_rel`	Logaritmo do volume do dia sobre o volume médio de 20 dias.
`sexta`	Indicadora de que t é sexta-feira. Efeito de calendário: o alvo de um dia é então, em geral, o retorno da segunda.

A primeira linha da saída lembra uma consequência da limpeza do Capítulo 2: em cinco pontos de 2011 e 2012 o arquivo salta mais de cinco dias corridos, porque os pregões sem cotação confiável foram removidos. O retorno seguinte a cada lacuna cobre vários pregões e é tratado aqui como se fosse diário. Todos esses pontos ficam no período de treino.

9.4.1 RSI, MACD e bandas de Bollinger

O **índice de força relativa** (RSI, de *relative strength index*) foi proposto por Wilder (1978) e circula em mais de uma versão. Seja $\Delta P_t = P_t - P_{t-1}$. Separe as variações em altas, $U_t = \max(\Delta P_t, 0)$, e quedas, $D_t = \max(-\Delta P_t, 0)$. Wilder suaviza cada série com uma média própria: o primeiro valor é a média simples das n primeiras observações e, a partir daí,

$$\bar{U}_t = \frac{(n-1)\,\bar{U}_{t-1} + U_t}{n}, \qquad \bar{D}_t = \frac{(n-1)\,\bar{D}_{t-1} + D_t}{n}, \tag{9.9}$$

com $n = 14$ na versão original. O indicador é

$$\mathrm{RSI}_t = 100 - \frac{100}{1 + \bar{U}_t/\bar{D}_t} = 100\,\frac{\bar{U}_t}{\bar{U}_t + \bar{D}_t}. \tag{9.10}$$

A segunda forma deixa claro o que ele mede: a fração do movimento recente que foi de alta. Valores acima de 70 são lidos, por convenção, como mercado "sobrecomprado" e abaixo de 30 como "sobrevendido". A suavização da Equação (9.9) é uma média exponencial com fator $\alpha = 1/n$, e não $2/(n+1)$, que é a convenção das médias exponenciais comuns. Implementações que usam média móvel simples ou o fator errado produzem outro indicador com o mesmo nome. A função `wilder` do Código 9.4 segue a definição original, inclusive na semente.

O **MACD** (*moving average convergence divergence*) é a diferença entre duas médias móveis exponenciais do preço, uma rápida de 12 dias e uma lenta de 26. Sua própria média exponencial de 9 dias é chamada de linha de sinal, e

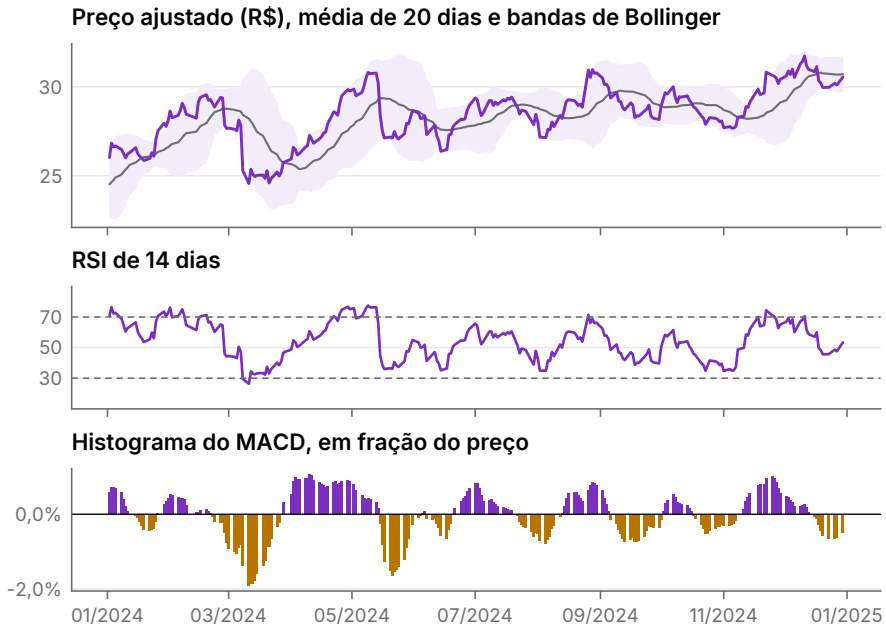


Figura 9.4 Três indicadores calculados sobre PETR4 em 2024. No alto, o preço, a média de 20 dias e a faixa das bandas de Bollinger; no meio, o RSI com as linhas de 30 e 70; embaixo, o histograma do MACD. Os três são funções do mesmo preço e contam, em boa parte, a mesma história.

a diferença entre as duas é o histograma: positivo quando a tendência curta acelera em relação à longa. As bandas de Bollinger, que o Capítulo 8 usou como regra de reversão, ficam aqui dois desvios-padrão móveis acima e abaixo da média de 20 dias do preço (lá, do logaritmo do preço), e o %B localiza o preço dentro delas. A Figura 9.4 mostra os três em 2024.

Os indicadores técnicos são formas diferentes de resumir a trajetória recente do preço, e a saída do Código 9.4 confirma a redundância: RSI e %B têm correlação de 0,89. O que eles oferecem a um modelo são transformações não lineares e já normalizadas dos retornos passados. Se há informação nelas é uma questão empírica.

9.4.2 A regra de ouro: só o passado

Toda feature da linha t deve poder ser calculada no fechamento de t, com os dados que existiam naquele momento. No Código 9.4 isso é garantido porque `rolling` e `ewm` só olham para trás, e o único deslocamento para a frente, `shift(-h)`, aparece na construção do alvo, na função `base` do mesmo arquivo. A terceira linha da saída é um teste que vale a pena repetir em todo projeto: recalcular as features com a base truncada em uma data e conferir que a linha daquela data não muda.

O preço ajustado muda retroativamente a cada provento (Capítulo 1). Como um ajuste futuro multiplica todo o passado pelo mesmo fator, features que são razões de preços não se alteram. Uma feature em nível de preço se alteraria.

Armadilha **Features que enxergam o futuro**

Os vazamentos mais comuns entram por funções de aparência inocente: `rolling(..., center=True)`, que centraliza a janela e usa dias seguintes; `shift` com sinal trocado; preenchimento de lacunas com `bfill`, que copia o valor futuro para trás; máximos e mínimos calculados sobre a série inteira; dados de balanço datados pelo trimestre a que se referem, e não pelo dia em que foram divulgados. Todos são casos de *look-ahead* (Capítulo 7), e um único deles basta para inflar o resultado de um backtest.

9.4.3 Estacionariedade e escala

Um modelo só generaliza se as features do futuro se parecerem com as do treino. O preço em nível não cumpre isso: PETR4 ajustada foi de pouco mais de R$ 1 a mais de R$ 30 na amostra, e um modelo treinado com preços de um dígito nunca viu os valores que encontrará depois. Por isso todas as colunas da Tabela 9.1 são retornos, razões ou osciladores limitados, que flutuam em torno de um nível estável.

A transformação tem um custo. O retorno diário é estacionário, mas quase não guarda memória do caminho que o preço percorreu. López de Prado (2018), no capítulo 5, propõe um meio-termo, a **diferenciação fracionária**: em vez de diferenciar a série uma vez inteira, aplica-se uma diferença de ordem d entre 0 e 1, a menor que torne a série estacionária, preservando o máximo de memória. Fica aqui o registro da ideia; não vamos implementá-la.

Features em escalas muito diferentes, como um RSI que vai de 0 a 100 e uma volatilidade diária de 0,02, atrapalham os métodos de otimização e tornam os coeficientes incomparáveis. O remédio é **padronizar**: subtrair a média e dividir pelo desvio-padrão. A média e o desvio precisam ser calculados apenas nos dados de treino e depois aplicados ao teste. Calculá-los na amostra inteira informa ao modelo, ainda que de leve, qual será a volatilidade do futuro. No scikit-learn (Pedregosa et al., 2011), o objeto `Pipeline` encadeia o padronizador e o modelo e cuida disso sozinho: cada vez que `fit` é chamado, inclusive dentro de cada dobra de uma validação cruzada, o padronizador é reajustado só com os dados daquele treino. Os classificadores deste capítulo são sempre ajustados dentro de `make_pipeline` por essa razão; a regressão por MQO do Código 9.2 dispensa o cuidado, porque suas previsões não mudam com a escala das features.

9.5 Rotulagem: o que exatamente se quer prever

Em finanças o rótulo é uma decisão de projeto, e muda o problema. O mais simples é o de **horizonte fixo**: o sinal do retorno dos próximos h dias, como na Equação (9.1). López de Prado (2018), no capítulo 3, critica esse rótulo. Dois defeitos interessam aqui. O primeiro é ignorar a volatilidade: um movimento de 2% em dez dias é grande em um período calmo e irrelevante em uma crise, mas recebe o mesmo rótulo. O segundo é ignorar o caminho. Uma posição que caiu 15% no terceiro dia e terminou o décimo em leve alta ganha rótulo positivo, embora nenhum operador com limite de perda a tivesse carregado até lá.

O **método da tripla barreira**, do mesmo autor, rotula cada observação pelo que teria acontecido com uma operação de verdade. A partir da entrada em t traçam-se três barreiras: uma superior, de realização de lucro; uma inferior,

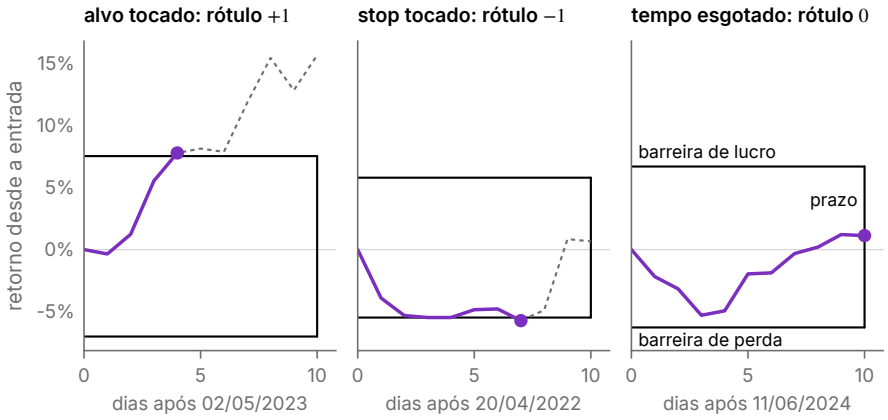


Figura 9.5 Tripla barreira em três entradas reais em PETR4, com prazo de 10 pregões e barreiras a um desvio-padrão do retorno de 10 dias. A linha roxa é o caminho até o primeiro toque; a pontilhada, o que veio depois. No painel do meio a perda limite foi atingida no sétimo dia e o preço se recuperou em seguida: pelo horizonte fixo esse rótulo seria positivo.

de limite de perda (*stop*); e uma vertical, o prazo máximo de h dias. As duas horizontais ficam a uma distância proporcional à volatilidade estimada em t. O rótulo é dado pela primeira barreira tocada: $+1$ para a superior, -1 para a inferior. Se o prazo vencer antes, o autor discute duas opções, rotular com zero ou com o sinal do retorno acumulado até ali; usamos zero. A Figura 9.5 mostra os três desfechos.

Código 9.5 Tripla barreira com fechamentos diários (`cap09/l04_barreiras.py`, trecho)

```
def tripla_barreira(p, sigma, h=10, k=1.0):
    """Para cada dia t: rótulo (+1 alvo, -1 stop, 0 tempo) e
    número de dias até a primeira barreira tocada."""
    lp = np.log(p.to_numpy())
    larg = k * np.sqrt(h) * sigma.to_numpy()  # largura em t
    rot = np.full(len(lp), np.nan)
    dias = np.full(len(lp), np.nan)
    for t in range(len(lp) - h):         # exige h dias de futuro
        if np.isnan(larg[t]):
            continue
        caminho = lp[t + 1:t + h + 1] - lp[t]
        toque = np.flatnonzero(np.abs(caminho) >= larg[t])
        if len(toque) > 0:               # barreira horizontal
            rot[t] = np.sign(caminho[toque[0]])
            dias[t] = toque[0] + 1
        else:                            # barreira vertical
            rot[t], dias[t] = 0, h
    return pd.DataFrame({"rotulo": rot, "dias": dias},
                        index=p.index)
```

```
        fração  dias até o toque
rotulo
-1.0     0.228             5.573
 0.0     0.525            10.000
 1.0     0.247             5.861
barreira tocada e retorno de 10 dias com sinal oposto: 102 de 1848
sem rótulo no fim da amostra: 2025-12-15 a 2025-12-30
```

No Código 9.5, `sigma` é a volatilidade diária dos 21 pregões até t, e a largura das barreiras é $k\sqrt{h}\,\sigma_t$ em log-retorno, a regra da raiz do tempo do Capítulo 1. Com $k = 1$ e $h = 10$, cerca de metade das entradas termina por prazo, 24,7% tocam primeiro o alvo e 22,8% o *stop*, em média depois de pouco menos de seis pregões. Em 102 das 1848 entradas que tocaram uma barreira horizontal, o retorno de dez dias terminou com o sinal oposto ao do rótulo: são os casos em que o horizonte fixo e a tripla barreira contam histórias contrárias.

Esta implementação simplifica o método em dois pontos: as barreiras são simétricas, quando nada impede distâncias diferentes para lucro e perda, e o toque é verificado só no fechamento, quando máximas e mínimas intradiárias atingiriam as barreiras mais cedo. Note também que as últimas h datas ficam sem rótulo, porque o futuro delas ainda não existe.

9.6 Validação temporal

Para estimar o desempenho de um modelo em dados novos, a receita dos livros de aprendizado de máquina é a **validação cruzada** em k dobras (*k-fold*): embaralham-se as observações, divide-se a amostra em k partes, e cada parte serve uma vez de teste para um modelo treinado nas outras $k - 1$ (James et al., 2021, capítulo 5). O procedimento supõe que as observações são independentes e vêm da mesma distribuição. Com séries financeiras as duas hipóteses falham, e o embaralhamento produz dois vazamentos.

O primeiro é treinar com o futuro. Uma dobra de teste sorteada ao acaso contém dias de 2012 avaliados por um modelo que viu 2020. O segundo é mais

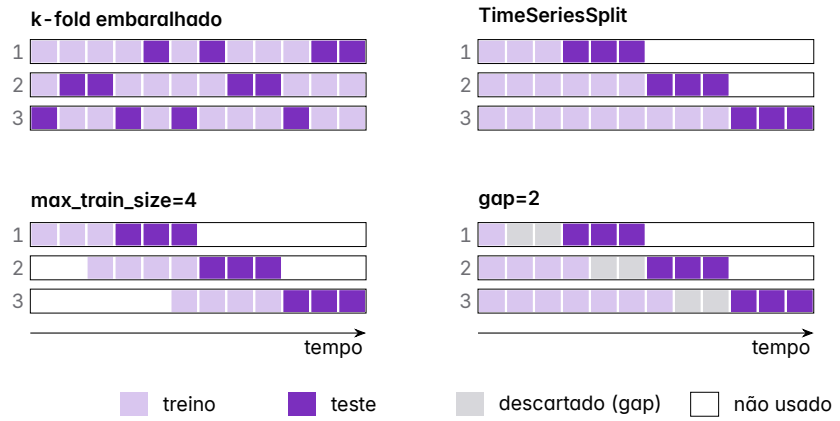


Figura 9.6 Três dobras sobre 12 observações em quatro esquemas (índices gerados por `cap09/l05_dobras.py`). No `k-fold` embaralhado as observações de teste ficam cercadas por vizinhas de treino, anteriores e posteriores. No `TimeSeriesSplit` o treino sempre termina antes do teste; `max_train_size` limita o treino ao passado recente e `gap` abre um intervalo entre os dois.

sutil e mais grave, e vem da sobreposição de rótulos. Com alvo de 20 dias, o rótulo do dia t e o do dia $t+1$ compartilham 19 dos 20 retornos diários, e as features dos dois dias também são quase idênticas. Se o embaralhamento manda t para o teste e $t+1$ para o treino, o modelo é avaliado em uma pergunta cuja resposta ele praticamente já recebeu.

Intuição

Embaralhar uma série com rótulos sobrepostos é aplicar uma prova em que cada questão é uma cópia, com uma palavra trocada, de um exercício resolvido na apostila.

9.6.1 Dobras que respeitam o tempo

A alternativa é nunca treinar com dados posteriores ao teste. A classe `TimeSeriesSplit` do scikit-learn faz isso com uma **janela expansível**, a janela ancorada do Capítulo 8: a amostra é cortada em $k+1$ blocos consecutivos, e a dobra i treina com os i primeiros blocos e testa no seguinte. A Figura 9.6 compara os esquemas para 12 observações. Dois parâmetros modificam o desenho básico. Com `max_train_size`, o treino fica limitado às observações mais recentes, uma janela deslizante (a rolante do Capítulo 8) que esquece o passado distante e faz sentido quando se acredita que o regime muda. Com `gap`, um número fixo de observações entre o treino e o teste é descartado.

O `gap` é a ferramenta contra a sobreposição de rótulos. Se o rótulo de t usa retornos até $t+h$, as últimas observações do treino têm rótulos que invadem o período de teste e, o que é o mesmo defeito visto de outro ângulo, que ainda não seriam conhecidos na data em que o teste começa. Para que o último rótulo do treino termine antes de começar o primeiro do teste é preciso descartar $h-1$ observações. É a purga (*purging*) apresentada na Seção 8.8. Em esquemas nos quais o treino também pode vir depois do teste, como o k-fold em blocos, purga-se dos dois lados e ainda se acrescenta o embargo, uma margem extra depois do teste, porque a dependência serial das features faz a contaminação durar um pouco mais que h (López de Prado, 2018, capítulo 7).

9.6.2 O experimento: o mesmo modelo, quatro notas

O Código 9.6 põe os esquemas à prova em um problema com sobreposição forte: prever, a cada dia, se o retorno de PETR4 nos 20 pregões seguintes será positivo (no código, `H = 20`). Dois modelos recebem as mesmas features. A regressão logística tem onze parâmetros e pouca capacidade de memorizar. O segundo modelo, de *k* vizinhos mais próximos, é o oposto: para classificar um dia, procura os 15 dias de treino com features mais parecidas e responde com a proporção de altas entre eles. Ele entra aqui só como exemplo de modelo flexível, da mesma família dos que o Capítulo 10 estuda.

Código 9.6 Quatro esquemas de validação para o mesmo problema (`cap09/l06_validacao.py`, trecho)

```
modelos = {
    "logística": LogisticRegression(),
    "15 vizinhos": KNeighborsClassifier(n_neighbors=15),
}
esquemas = {
    "k-fold embaralhado": KFold(5, shuffle=True, random_state=0),
    "k-fold em blocos": KFold(5),
    "TimeSeriesSplit": TimeSeriesSplit(5),
    "TimeSeriesSplit + gap": TimeSeriesSplit(5, gap=H - 1),
}
print(f"{len(y)} observações, {y.mean():.1%} de altas")
for nome_m, m in modelos.items():
    modelo = make_pipeline(StandardScaler(), m)
    for nome_e, cv in esquemas.items():
        auc = cross_val_score(modelo, X, y, cv=cv,
                              scoring="roc_auc")
```

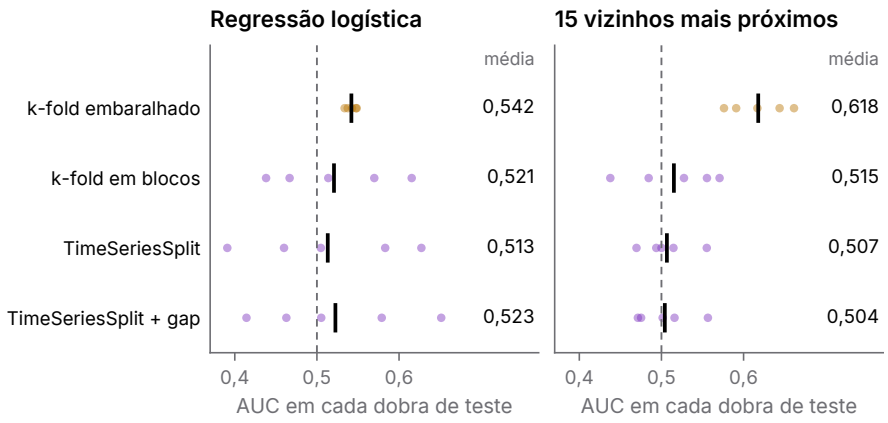


Figura 9.7 AUC em cada uma das cinco dobras de teste (pontos) e sua média (traço), para os quatro esquemas do Código 9.6. Com o `k-fold embaralhado`, em ocre, as dobras concordam entre si e apontam um modelo com poder de previsão. Nos demais esquemas a média volta para perto de 0,5 e a dispersão entre dobras revela quão incerta é a estimativa.

```
3851 observações, 54.5% de altas
logística    k-fold embaralhado     AUC 0.542  (dobras: 0.534 a 0.548)
logística    k-fold em blocos       AUC 0.521  (dobras: 0.438 a 0.615)
logística    TimeSeriesSplit        AUC 0.513  (dobras: 0.391 a 0.627)
logística    TimeSeriesSplit + gap  AUC 0.523  (dobras: 0.414 a 0.651)
15 vizinhos  k-fold embaralhado     AUC 0.618  (dobras: 0.576 a 0.661)
15 vizinhos  k-fold em blocos       AUC 0.515  (dobras: 0.438 a 0.571)
15 vizinhos  TimeSeriesSplit        AUC 0.507  (dobras: 0.470 a 0.555)
15 vizinhos  TimeSeriesSplit + gap  AUC 0.504  (dobras: 0.471 a 0.557)
vizinhos a menos de 20 pregões: 1.35 de 15 (ao acaso: 0.15)
vizinhos a 1 pregão: 0.37 de 15
distância mediana: 867 pregões
```

O modelo de vizinhos recebe AUC de 0,62 no k-fold embaralhado e de 0,50 a 0,51 quando o treino termina antes do teste, com ou sem purga. Nada mudou no modelo nem nos dados; mudou apenas quem estava no treino quando cada dia foi avaliado. As três últimas linhas da saída vêm de um diagnóstico no fim do script, que refaz a busca de vizinhos de cada dobra embaralhada, e mostram o mecanismo. A maior parte dos 15 dias de treino "mais parecidos" com um dia t de teste está longe dele no calendário, a uma distância mediana de 867 pregões. Mas em média 1,35 deles cai a menos de 20 pregões de t (0,37 é vizinho imediato), nove vezes mais do que o 0,15 que o acaso colocaria ali, e esses têm rótulos que compartilham retornos com o de t. Pouco mais de um vizinho com a resposta quase pronta, em quinze, leva a AUC de 0,50 a 0,62.

A logística sofre menos. Sua média cai de 0,542 para algo entre 0,51 e 0,52, uma diferença pequena diante da dispersão entre dobras, porque um modelo linear com onze parâmetros não tem como decorar vizinhanças. O vazamento por embaralhamento cresce com a flexibilidade do modelo, e por isso é mais perigoso justamente com os modelos do próximo capítulo.

A Figura 9.7 mostra um segundo engano do embaralhamento. As cinco dobras embaralhadas dão resultados quase iguais, de 0,534 a 0,548 para a logística, o que passa uma impressão de estabilidade. As dobras cronológicas vão de 0,39 a 0,65. Essa dispersão é a incerteza verdadeira: cada bloco de teste tem cerca de 640 dias, que com rótulos de 20 dias equivalem a pouco mais de 30 janelas sem sobreposição. A amostra inteira, de 3851 linhas, contém menos de 200 retornos de 20 dias independentes.

O k-fold em blocos, sem embaralhar, já elimina quase toda a ilusão, porque

Tabela 9.2 Vazamentos típicos em aprendizado de máquina financeiro

Vazamento	Como acontece	Como evitar
Feature com futuro	Janela centralizada, `shift` invertido, `bfill`, dado datado antes da divulgação	Teste de truncamento; datar cada dado pelo dia em que ficou disponível
Normalização global	Média e desvio calculados na amostra inteira antes de separar o teste	`Pipeline`: o ajuste ocorre só no treino de cada dobra
Seleção de features na amostra inteira	Escolher as variáveis mais correlacionadas com o alvo usando todos os dados e só depois validar	Fazer a seleção dentro de cada dobra (Hastie et al., 2009, seção 7.10)
Hiperparâmetros ajustados no teste	Experimentar configurações até a métrica de teste melhorar	Três conjuntos: treino, validação e um teste final usado uma vez (Capítulo 10)
Rótulos sobrepostos	Alvo de vários dias com treino e teste vizinhos	Purga de $h-1$ observações e embargo (Seção 8.8)

nele a sobreposição só ocorre nas bordas de cada bloco. Ele continua treinando com o futuro, mas a comparação isola a causa: o problema estava nos vizinhos.

9.6.3 Vazamentos típicos e referências obrigatórias

O embaralhamento é um caso de **vazamento de informação** (*data leakage*): qualquer caminho pelo qual dados do período de teste influenciam o modelo que será avaliado nele. A Tabela 9.2 reúne os mais frequentes. Todos têm o mesmo efeito, uma métrica otimista que desaparece em produção, e a mesma prevenção: tudo o que é ajustado aos dados, do padronizador à lista de features, fica dentro do laço de validação.

O quarto item é o que menos parece trapaça. Cada vez que você olha o resultado no teste e volta para mudar o modelo, o teste passa a fazer parte do treino, como no sobreajuste de backtest dos Capítulos 7 e 8.

Por fim, todo modelo deve ser comparado, no mesmo teste, com pelo menos duas referências: a classe majoritária (ou a previsão zero, em regressão) e uma regra simples, de uma linha. No Código 9.3, "sempre alta" acertou 52,3% e "repetir o sinal de hoje" acertou 49,6%. Um modelo que não supera as duas não aprendeu nada que valha a sua complexidade.

9.7 Da previsão à estratégia

Entre uma probabilidade e uma ordem há uma regra de posição, que diz quanto comprar para cada valor do sinal, e um backtest com custos, nos moldes dos Capítulos 7 e 8. O Código 9.7 percorre esse caminho da forma mais simples possível. No início de cada ano, de 2019 a 2025, a logística é reajustada com todos os dados anteriores e gera probabilidades para o ano que começa, um esquema *walk-forward* de janela expansível. A posição w_t vale 1 (comprado em PETR4) se a probabilidade de alta passa de 0,5 e 0 (aplicado no CDI) caso contrário. O retorno da estratégia entre os fechamentos de t e $t+1$ é

$$R_{t+1} = w_t\, R^{\text{PETR4}}_{t+1} + (1 - w_t)\, R^{f}_{t+1} - c\,|w_t - w_{t-1}|, \tag{9.11}$$

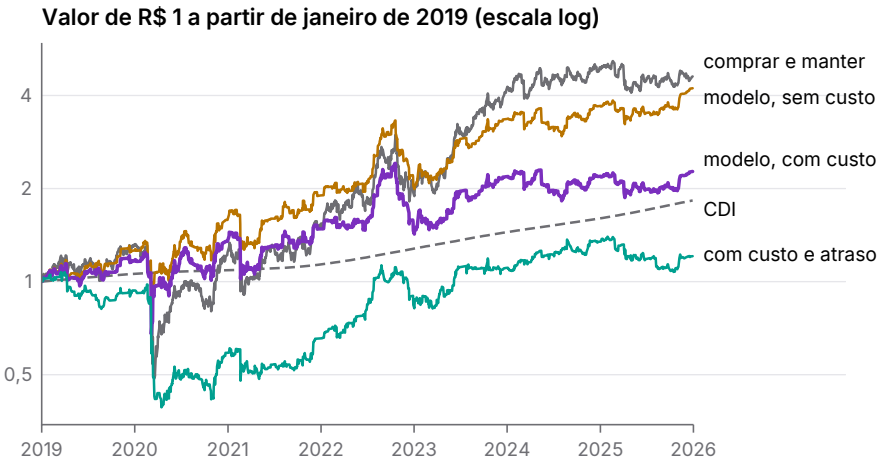


Figura 9.8 Resultado fora da amostra da estratégia guiada pela regressão logística, de 2019 a 2025, contra comprar e manter PETR4 e contra o CDI. Sem custos o modelo fica perto da ação; com 0,10% por ponta fica bem abaixo dela; com um pregão de atraso na execução perde para o CDI.

com R^f a taxa diária do CDI (arquivo `dados/c09_cdi.csv`, gravado por `cap09/baixar_cdi.py`) e $c = 0{,}10\%$ o custo por ponta, a hipótese única do livro, que resume taxas, metade do *spread* e *slippage* de ordens pequenas; não inclui o impacto de mercado de ordens grandes, tratado no Capítulo 11. A convenção é a de *execução no fechamento do sinal*, a mesma do motor do Capítulo 7: a posição é assumida no próprio fechamento de t que gerou o sinal. Não há *look-ahead*, porque o retorno do dia t não é capturado, mas a hipótese é otimista e o código a testa repetindo a conta com um pregão de atraso. A referência é **comprar e manter** (*buy and hold*): adquirir a ação no primeiro dia e não negociar mais.

Código 9.7 Do sinal à posição e ao resultado líquido (`cap09/l07_backtest.py`, trecho)

```
prob = pd.Series(np.nan, index=X.index)
for ano in range(2019, 2026):     # reajusta no início de cada ano
    treino, teste = X.index.year < ano, X.index.year == ano
    m = make_pipeline(StandardScaler(), LogisticRegression())
    m.fit(X[treino], y[treino])
    prob[teste] = m.predict_proba(X[teste])[:, 1]

prob = prob.dropna()
w = (prob > 0.5).astype(float)    # 1 = comprado, 0 = no CDI
R, rf = R[prob.index], rf[prob.index]


def estrategia(w, custo):
    giro = w.diff().abs().fillna(w.iloc[0])
    return w * R + (1 - w) * rf - custo * giro


bruto = estrategia(w, 0.0)
liquido = estrategia(w, CUSTO)
atraso = estrategia(w.shift(1).fillna(0), CUSTO)
```

```
modelo, sem custo     23.2% a.a.  Sharpe  0.55  DD máx. -45.2%
modelo, com custo     12.6% a.a.  Sharpe  0.26  DD máx. -45.6%
com custo e atraso     2.8% a.a.  Sharpe -0.05  DD máx. -63.5%
comprar e manter      24.8% a.a.  Sharpe  0.54  DD máx. -63.4%
CDI no período: 9.1% a.a.
dias comprado: 48.9% | pontas: 620 | acerto do sinal: 50.7%
```

As taxas são anualizadas pela convenção de 252 dias (Seção 1.9), o índice de Sharpe usa o retorno em excesso ao CDI, e a última coluna é o drawdown máximo do Capítulo 5. Antes de custos, a estratégia rendeu 23,2% ao ano, um pouco menos que os 24,8% de comprar e manter, com Sharpe praticamente igual (0,55 e 0,54). As duas verificações seguintes tiram o que restava. As 620 pontas negociadas custam, somadas, 62 pontos percentuais de retorno em sete anos, e o retorno líquido cai para 12,6% ao ano, com Sharpe de 0,26. Com um pregão de atraso na execução, o retorno cai para 2,8% ao ano, abaixo do CDI de 9,1%. O sinal acerta 50,7% dos dias e troca de lado a cada três pregões: não há previsão explorável aqui.

O drawdown máximo menor, de 45% contra 63%, tem uma explicação que dispensa o modelo: a carteira passou metade dos dias fora da ação. Na versão com atraso ele é igual ao da própria ação.

Duas ressalvas valem para qualquer leitura desses números. O ativo foi escolhido hoje, sabendo que a Petrobras atravessou o período e pagou dividendos elevados; é o viés de sobrevivência da Seção 7.6, e 24,8% ao ano não é o que se espera de uma ação tomada ao acaso em 2019. E o teste cobre um único papel em um único intervalo.

O resultado é decepcionante e é o esperado: indicadores técnicos diários sobre uma das ações mais negociadas do país são informação que todos os participantes têm. O que fica do exercício é o encanamento, que agora existe e não vaza: features datadas, padronização dentro do *pipeline*, validação cronológica com purga, referências ingênuas, custos e teste de atraso. É nele que o Capítulo 10 vai ligar modelos mais flexíveis.

O Capítulo 7 obtém Sharpe de 0,56 para comprar e manter PETR4 em 2019–2025. A diferença vem da amostra: lá entra o retorno do pregão de 2 de janeiro de 2019; aqui a primeira posição é assumida no fechamento desse dia.

O que levar deste capítulo

- Em mercados a razão sinal-ruído é mínima, as relações mudam e as observações dependem umas das outras. Um R^2 fora da amostra de décimos de ponto percentual é o resultado normal, e métricas muito melhores pedem primeiro uma busca por vazamento.
- A regressão linear por MQO funciona bem para descrever relações contemporâneas, como o beta. Com retornos, os erros-padrão usuais são otimistas e devem ser trocados por versões HAC; o próprio coeficiente muda com o tempo.
- A regressão logística transforma features em probabilidade por meio da sigmoide. Avalie com matriz de confusão, precisão, recall e AUC, sempre ao lado da classe majoritária e de uma regra simples.
- Toda feature da data t usa apenas dados até t, em forma estacionária, e qualquer transformação ajustada aos dados, como a padronização, é estimada só no treino.
- O rótulo é uma escolha: o horizonte fixo ignora volatilidade e caminho, e a tripla barreira os incorpora. Rótulos de vários dias se sobrepõem e reduzem a informação efetiva da amostra.

- Embaralhar séries temporais vaza o futuro para o treino. Use divisões cronológicas, purgue $h-1$ observações entre treino e teste e só acredite em uma previsão depois de vê-la sobreviver a um backtest com custos.

Exercícios

9.1 [básico] Um classificador produz, em 200 dias de teste, 60 verdadeiros positivos, 50 falsos positivos, 50 falsos negativos e 40 verdadeiros negativos. Calcule acurácia, precisão, recall e F1, e a acurácia de quem prevê sempre a classe majoritária.

9.2 [básico] Uma regressão logística tem $\beta_0 = 0{,}1$ e um único coeficiente $\beta_1 = -0{,}4$ para uma feature padronizada. Calcule a probabilidade prevista para $x = 0$ e para $x = 2$, e a razão de chances associada a um desvio-padrão de aumento em x.

9.3 [intermediário] Escreva uma versão do RSI que use média móvel simples no lugar da suavização de Wilder e outra que use `ewm(span=14)`. Compare as três séries para PETR4 em 2024: qual é a maior diferença, em pontos, e em quantos dias elas discordam sobre o indicador estar acima de 70?

9.4 [intermediário] Introduza de propósito um vazamento no Código 9.4, trocando a média móvel de 50 dias por uma versão com `center=True`. Refaça o Código 9.3 e relate o que acontece com a acurácia e a AUC. O teste de truncamento detecta o erro?

9.5 [intermediário] No Código 9.6, varie o horizonte do rótulo para 1, 5, 10 e 40 dias e registre a diferença de AUC entre o k-fold embaralhado e o `TimeSeriesSplit` com `gap`, para os dois modelos. Explique o padrão que aparece.

9.6 [desafio] Use os rótulos da tripla barreira do Código 9.5, descartando os de prazo, como alvo de uma regressão logística. Valide com `TimeSeriesSplit`, escolhendo o `gap` adequado, e compare com a classe majoritária em cada dobra.

9.7 [desafio] Refaça o backtest do Código 9.7 para os sete ativos brasileiros de `precos_ajustados.csv`, usando apenas as features que não dependem de volume. Em quantos deles a estratégia líquida supera comprar e manter em índice de Sharpe? O que esse número diria se fosse quatro de sete?

Para ir além

- James et al. (2021), capítulos 3, 4 e 5, para regressão linear, classificação e validação cruzada em nível introdutório.
- Hastie et al. (2009), capítulos 3, 4 e 7, com o tratamento formal dos mesmos temas; a seção 7.10 discute o jeito errado e o jeito certo de fazer validação cruzada.
- López de Prado (2018), capítulos 3, 5 e 7, para tripla barreira, diferenciação fracionária e validação com purga e embargo; López de Prado (2020), seção 5, retoma a rotulagem em texto mais curto.
- Jansen (2020), capítulos 4, 6 e 7, com muitos exemplos em Python de construção de features e de modelos lineares aplicados a retornos.

Capítulo 10

Machine learning II: ensembles, regularização e sobreajuste

10

Uma random forest com parâmetros de fábrica acerta 100% dos dias em que as ações deste capítulo subiram ou caíram nos dados em que foi treinada. No período seguinte ela acerta 50%, o mesmo que uma moeda. Este capítulo apresenta os modelos mais usados em dados tabulares, explica por que essa distância entre treino e teste é a regra em finanças e mostra o que fazer para medi-la antes que ela custe dinheiro.

Ao terminar este capítulo você consegue

- enunciar a decomposição viés-variância e reconhecê-la em curvas de erro de treino e de teste;
- regularizar modelos lineares com ridge, lasso e elastic net, escolhendo a penalidade por validação temporal;
- explicar como árvores, random forests e gradient boosting são construídos e quais hiperparâmetros controlam sua complexidade;
- comparar modelos sem se enganar, reservando um período final que só é usado uma vez;
- interpretar um modelo com importância por permutação e dependência parcial, sabendo os limites de cada medida;
- transformar uma probabilidade prevista em posição e avaliar o resultado com custos.

10.1 O dilema entre viés e variância

O Capítulo 9 tratou de dois modelos lineares, a regressão e a logística. Eles são rígidos: por mais dados que recebam, só conseguem desenhar retas e planos. Os modelos deste capítulo são flexíveis e conseguem reproduzir quase qualquer relação entre features e alvo. Flexibilidade tem um custo, e a melhor forma de vê-lo é um experimento em que a resposta certa é conhecida.

Suponha que o alvo seja gerado por $y = f(x) + \varepsilon$, em que f é uma função fixa e ε é um ruído de média zero e variância σ^2. Um modelo $\hat{f}$ é ajustado a

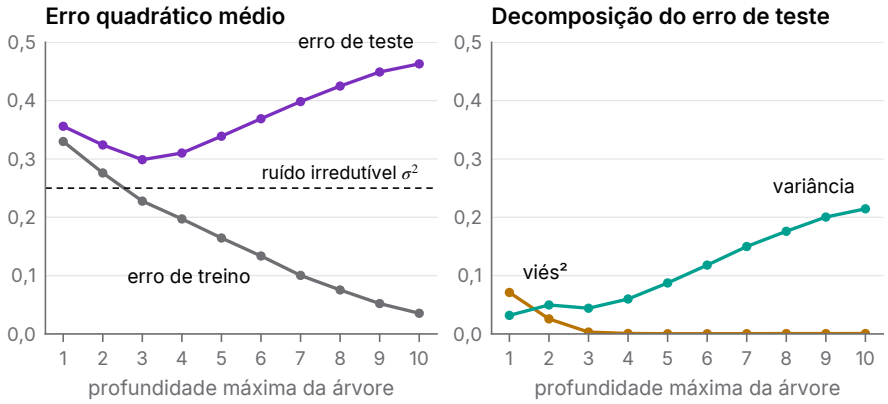


Figura 10.1 Experimento simulado com árvores de regressão. À esquerda, o erro de treino cai sem parar com a complexidade, enquanto o erro em dados novos passa por um mínimo na profundidade 3 e volta a subir. À direita, a causa: o viés some depressa, e a variância cresce quase sem parar e domina o erro a partir da profundidade 3.

uma amostra de treino. Se o experimento fosse repetido com outra amostra, $\hat{f}$ sairia diferente. O erro esperado em um ponto novo depende de duas coisas: de quanto a média dessas versões de $\hat{f}$ erra o alvo e de quanto elas variam entre si.

Resultado 10.1 Decomposição viés-variância

Para um ponto x_0 fixo e uma nova observação $y_0 = f(x_0) + \varepsilon_0$, o erro quadrático esperado da previsão, tomando a esperança sobre as amostras de treino e sobre o ruído ε_0, é

$$\mathbb{E}\Big[\big(y_0 - \hat{f}(x_0)\big)^2\Big] = \underbrace{\sigma^2}_{\text{ruído}} + \underbrace{\big(\mathbb{E}[\hat{f}(x_0)] - f(x_0)\big)^2}_{\text{viés}^2} + \underbrace{\operatorname{Var}\big(\hat{f}(x_0)\big)}_{\text{variância}}. \tag{10.1}$$

O primeiro termo não depende do modelo e nenhum algoritmo o reduz. O **viés** mede o erro sistemático de um modelo simples demais para a forma de f. A **variância** mede a sensibilidade do modelo à amostra que por acaso foi usada no treino. A demonstração está em Hastie et al. (2009), seção 7.3; a decomposição vale para o erro quadrático, e para classificação a ideia se mantém com outra álgebra.

A Figura 10.1 mostra a Equação (10.1) acontecendo. O processo simulado é $f(x) = \operatorname{sen}(2\pi x)$ com $\sigma = 0{,}5$, e o modelo é uma árvore de regressão (Seção 10.4) cuja profundidade máxima vai de 1 a 10. Para cada profundidade foram sorteadas 300 amostras de treino de 200 pontos.

Com profundidade 1, o viés ao quadrado é 0,071 e a variância 0,032: o modelo erra por rigidez. Com profundidade 10 o viés é praticamente nulo e a variância chega a 0,215. O erro de teste é mínimo na profundidade 3, com 0,299, pouco acima do piso de $\sigma^2 = 0{,}25$. O erro de treino continua caindo e chega a 0,035 na profundidade 10, quando o erro de teste já subiu para 0,463. Esse é o retrato do **sobreajuste** (*overfitting*): o modelo passou a ajustar o ruído da amostra, que não se repete.

Intuição

O erro de treino sempre melhora quando o modelo ganha liberdade, e por isso não serve para escolher o modelo. A distância entre o erro de treino e o erro em dados novos é a medida do quanto o modelo decorou. Todo o capítulo gira em torno de estimar o segundo número sem se enganar.

O ponto ótimo depende da quantidade de ruído. Repetindo o experimento com $\sigma = 0{,}25$, a melhor profundidade sobe para 4; com $\sigma = 1$ ela cai para 2, e com $\sigma = 2$, para 1. Quanto mais ruído em relação ao sinal, mais simples é o melhor modelo. Retornos de ações estão no extremo ruidoso dessa escala, e a Seção 10.7.3 volta a esse ponto com dados reais.

10.2 Um painel para o capítulo inteiro

Todos os exemplos com dados reais usam a mesma tabela, montada pelo arquivo `cap10/painel.py`. Ela empilha sete ações do conjunto de dados do livro (PETR4, VALE3, ITUB4, WEGE3, ABEV3, BBAS3 e MGLU3), uma linha por ação e por pregão em que a ação tem cotação, com dez features calculadas apenas com preços até o fechamento do dia. Seis delas são versões das features do Capítulo 9, aqui com retornos simples em vez de log-retornos: os retornos de 1, 5 e 21 pregões, a volatilidade de 21 pregões, a distância do preço à média de 50 pregões e o RSI de 14. Quatro são novas: `ret_63`, o retorno de 63 pregões; `vol_razao`, a volatilidade de 21 pregões dividida pela de 126; `dd_252`, a distância do preço ao máximo dos últimos 252 pregões; e `ibov_21`, o retorno de 21 pregões do BOVA11, usado como medida do mercado.

Código 10.1 As dez features do painel (trecho de `cap10/painel.py`)

```
def features(p, merc_21):
    """Tudo calculado só com preços até o fechamento de t."""
    r = p.pct_change()
    f = pd.DataFrame(index=p.index)
    f["ret_1"] = r
    f["ret_5"] = p.pct_change(5)
    f["ret_21"] = p.pct_change(21)
    f["ret_63"] = p.pct_change(63)
    f["vol_21"] = r.rolling(21).std()
    f["vol_razao"] = f["vol_21"] / r.rolling(126).std()
    f["pm_50"] = p / p.rolling(50).mean() - 1
    f["rsi_14"] = rsi(p)
    f["dd_252"] = p / p.rolling(252).max() - 1
    f["ibov_21"] = merc_21
    return f
```

A função `painel`, no mesmo arquivo, aplica `features` a cada ação e acrescenta o rótulo:

```
    merc_21 = precos["BOVA11.SA"].dropna().pct_change(21)
    blocos = []
    for a in ACOES:
        p = precos[a].dropna()          # só pregões com cotação
        f = features(p, merc_21.reindex(p.index, method="ffill"))
        f["fwd"] = p.shift(-H) / p - 1     # retorno de t a t+H
        f["ret_seg"] = p.shift(-1) / p - 1  # retorno de t a t+1
        f["acao"] = a
        blocos.append(f)
    d = pd.concat(blocos).dropna().sort_index(kind="stable")
    d["y"] = (d["fwd"] > 0).astype(int)   # 1 se subiu em H dias
```

O rótulo y vale 1 quando o retorno dos cinco pregões seguintes, de t a $t + 5$, é positivo. É um rótulo simples de propósito; a tripla barreira do Capítulo 9 seria uma escolha melhor para uma estratégia de verdade. O arquivo de preços tem valores ausentes isolados, pregões em que a fonte não registrou negócio com o papel. O `dropna` aplicado a cada ação faz com que retornos, janelas e rótulo sejam contados em pregões com cotação daquela ação; nenhum preço é inventado, e depois de uma lacuna o retorno "de um pregão" cobre mais de um dia de calendário. O painel vai de 7 de janeiro de 2011 a 19 de dezembro de 2025 e tem 25.492 linhas. MGLU3 abriu capital em maio de 2011 e, descontado o ano de aquecimento das janelas, só entra no painel em maio de 2012. Por isso, e pelos valores ausentes, 346 dos pregões do período de desenvolvimento têm menos de sete ações.

O RSI do painel usa a média exponencial de Wilder sem a semente de média simples do Capítulo 9. Depois do aquecimento de 252 pregões as duas versões coincidem na prática.

Duas decisões de validação ficam fixas desde já. O período de 2022 em diante é separado como **teste final** e só aparece na Seção 10.7.2. O restante, 18.506 linhas até 22 de dezembro de 2021, é o conjunto de desenvolvimento, em que se ajustam modelos e hiperparâmetros com validação temporal. A função `cortes`, no mesmo arquivo, aplica o `TimeSeriesSplit` do Capítulo 9 às datas, e não às linhas, para que todas as ações de um mesmo dia caiam do mesmo lado, e deixa uma folga de cinco pregões entre treino e validação, porque o rótulo de um dia usa preços dos cinco dias seguintes. Quatro pregões, $h - 1$, bastariam para separar os rótulos; o quinto cobre um eventual dia sem cotação.

As sete ações foram escolhidas em 2025 entre empresas grandes e líquidas que continuam listadas; as que quebraram ou perderam relevância não estão na amostra. Esse viés de sobrevivência (Capítulo 7) infla o retorno de qualquer estratégia comprada testada aqui, inclusive o da referência de comprar tudo. Os números deste capítulo servem para comparar métodos, e não como estimativa de retorno futuro.

10.3 Regularização em modelos lineares

Antes de trocar de modelo, vale controlar a complexidade do modelo linear. A regressão por mínimos quadrados ordinários (MQO) escolhe os coeficientes que minimizam a soma dos quadrados dos resíduos, e nada mais. Com features correlacionadas e pouco sinal, ela produz coeficientes grandes de sinais opostos que se cancelam na amostra de treino e erram fora dela. A **regularização** acrescenta ao critério uma penalidade sobre o tamanho dos coeficientes.

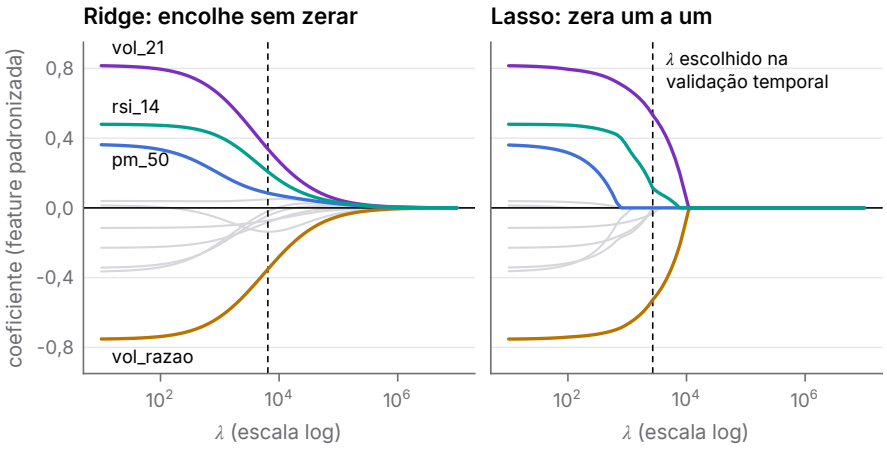


Figura 10.2 Coeficientes das dez features em função de λ, com quatro delas destacadas. O ridge encolhe o vetor de coeficientes de forma contínua, sem zerar nenhum antes do limite (um coeficiente isolado pode até crescer, como o de dd_252, uma das linhas cinzas). O lasso zera um coeficiente de cada vez, e acima de $\lambda \approx 11.200$ não sobra nenhum. As linhas tracejadas marcam o λ de melhor desempenho na validação temporal.

Definição 10.2 Ridge, lasso e elastic net

Com as features padronizadas e o alvo centrado, os três estimadores minimizam a soma dos quadrados dos resíduos mais uma penalidade de intensidade $\lambda \geq 0$:

$$\text{ridge:} \quad \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \sum_j \beta_j^2, \tag{10.2}$$

$$\text{lasso:} \quad \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \sum_j |\beta_j|, \tag{10.3}$$

$$\text{elastic net:} \quad \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \sum_j \left[\alpha|\beta_j| + (1-\alpha)\beta_j^2\right], \quad 0 \leq \alpha \leq 1.$$

O **ridge** (Hoerl e Kennard, 1970) usa a penalidade L2, a soma dos quadrados. O problema continua sendo quadrático e tem solução fechada:

$$\hat{\boldsymbol{\beta}}_{\text{ridge}} = \left(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}\right)^{-1}\mathbf{X}^\top\mathbf{y}. \tag{10.4}$$

Com $\lambda = 0$ recupera-se o MQO. Somar λ à diagonal de $\mathbf{X}^\top\mathbf{X}$ torna a matriz bem condicionada mesmo quando há features quase colineares, que é quando o MQO fica instável. Em troca de um pouco de viés, a variância dos coeficientes cai.

O **lasso** (Tibshirani, 1996) usa a penalidade L1, a soma dos valores absolutos. Não há fórmula fechada em geral, mas há uma diferença de comportamento que importa mais do que a álgebra: o lasso leva coeficientes a exatamente zero. Ele faz ao mesmo tempo o ajuste e a **seleção de features**. O **elastic net** (Zou e Hastie, 2005) mistura as duas penalidades e é útil quando há grupos de features muito correlacionadas, caso em que o lasso tende a ficar com uma do grupo e descartar as outras.

A padronização é obrigatória: a penalidade trata todos os coeficientes na mesma moeda, e um coeficiente de uma feature medida em porcentagem não é comparável ao de uma medida em pontos de RSI. O intercepto não é penalizado, o que se consegue centrando o alvo. A média e o desvio-padrão usados para padronizar são calculados só no treino, pela razão discutida no Capítulo 9.

O Código 10.2 compara MQO, ridge e lasso no conjunto de desenvolvimento. A matriz `X` tem as $p = 10$ features padronizadas e `y` é o retorno dos cinco dias seguintes, em porcentagem e centrado.

Código 10.2 Ridge pela fórmula fechada e lasso sobre as features do painel (trecho de `cap10/l01_ridge_lasso.py`)

```
# ridge pela fórmula fechada, conferida com o scikit-learn
lam = 2500.0
b_ridge = np.linalg.solve(X.T @ X + lam * np.eye(p), X.T @ y)
sk = Ridge(alpha=lam, fit_intercept=False).fit(X, y)
print("fórmula = sklearn:", np.allclose(b_ridge, sk.coef_))

b_mqo = np.linalg.solve(X.T @ X, X.T @ y)
# o Lasso do sklearn minimiza SQR/(2n) + alpha*|b|_1
lasso = Lasso(alpha=lam / (2 * n), fit_intercept=False)
b_lasso = lasso.fit(X, y).coef_
tab = pd.DataFrame({"MQO": b_mqo, "ridge": b_ridge,
                    "lasso": b_lasso}, index=FEATS)
print(tab.round(3).to_string())
print("norma L2:", np.linalg.norm(tab, axis=0).round(3))
```

```
fórmula = sklearn: True
             MQO  ridge  lasso
ret_1     -0.115 -0.092 -0.037
ret_5     -0.229 -0.136 -0.020
ret_21    -0.367 -0.102 -0.000
ret_63    -0.344 -0.124 -0.020
vol_21     0.818  0.507  0.550
vol_razao -0.754 -0.508 -0.543
pm_50      0.365  0.128  0.000
rsi_14     0.480  0.322  0.138
dd_252     0.015 -0.110 -0.000
ibov_21    0.040  0.042  0.000
norma L2: [1.386 0.838 0.787]
```

A primeira linha confirma que a Equação (10.4) dá o mesmo resultado que a biblioteca. Na tabela, repare em `ret_21` e `pm_50`. As duas features têm correlação de 0,89 no painel, e o MQO lhes dá coeficientes de $-0{,}367$ e $+0{,}365$, uma aposta grande na pequena diferença entre duas medidas quase iguais. O ridge encolhe os dois; o lasso zera os dois, além de `dd_252` e `ibov_21`. Repare também que o ridge não encolhe cada coeficiente: o de `dd_252` passa de 0,015 para $-0{,}110$, maior em módulo. O que encolhe é a norma do vetor, na última linha da saída; com features correlacionadas, coeficientes isolados podem crescer e trocar de sinal. Um detalhe de escala: o scikit-learn escreve o objetivo do lasso como a soma dos quadrados dividida por $2n$ mais `alpha` vezes a soma dos valores absolutos, de modo que a solução da Equação (10.3) se obtém com `alpha` igual a $\lambda/(2n)$, em que n é o número de linhas do treino.

10.3.1 Escolha de λ por validação temporal

A Figura 10.2 mostra a família inteira de soluções, o chamado caminho de regularização. Falta escolher um ponto dele, e λ é um hiperparâmetro como qualquer outro: escolhe-se o valor que dá o menor erro em dados posteriores ao treino. O script `cap10/fig_caminho.py` percorre 80 valores de λ nas cinco divisões temporais do conjunto de desenvolvimento e mede o R^2 fora da

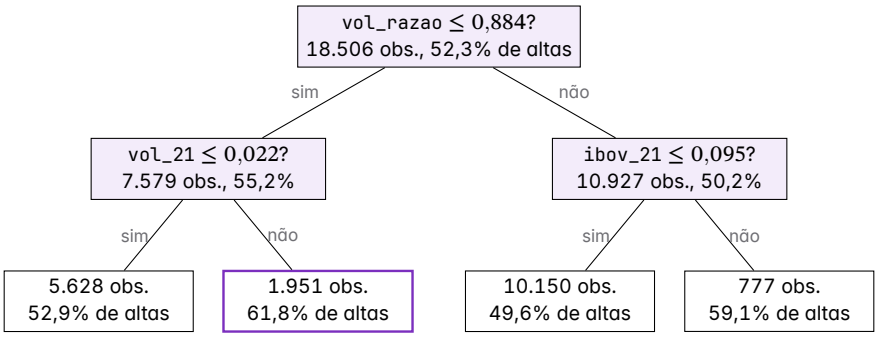


Figura 10.3 Árvore de classificação de profundidade 2 ajustada ao painel (saída de `cap10/l02_arvore.py`). Cada nó interno faz uma pergunta sobre uma feature; cada folha prevê a proporção de altas das observações que chegaram a ela. A folha destacada parece uma descoberta e é discutida no texto.

amostra, que compara o erro do modelo com o de prever sempre a média do treino. Há um cuidado de escala. Nas Equações (10.2) e (10.3), λ multiplica a penalidade e disputa com uma soma sobre n linhas, de modo que o mesmo λ regulariza muito mais um treino de 2.690 linhas, o da primeira divisão, do que um de 15.314, o da última. O que se mantém constante entre as divisões é a penalidade por observação, λ/n: em um treino de n_{tr} linhas usa-se $\lambda\, n_{\text{tr}}/n$. Os valores de λ citados a seguir e marcados na Figura 10.2 já estão na escala das 18.506 linhas do ajuste completo.

O resultado dá a dimensão do problema. O MQO tem R^2 de 1,6% dentro da amostra e de $-0{,}53\%$ fora dela: é pior do que prever a média. O melhor ridge ($\lambda \approx 6.500$) chega a $+0{,}26\%$, e o melhor lasso ($\lambda \approx 2.700$, com seis coeficientes não nulos) a $+0{,}20\%$. A regularização transformou um modelo que atrapalha em um que ajuda um pouco. Esses dois números ainda são otimistas, porque o λ foi escolhido para maximizá-los; a Seção 10.7.2 trata dessa contaminação.

10.4 Árvores de decisão

Uma **árvore de decisão** divide os dados com perguntas do tipo "a feature j é menor ou igual a s?" e repete o processo dentro de cada grupo, até chegar a grupos finais, as **folhas**. A previsão de uma folha é a proporção de cada classe entre as observações de treino que caíram nela (ou a média do alvo, em regressão). A Figura 10.3 mostra a árvore de profundidade 2 ajustada ao conjunto de desenvolvimento.

10.4.1 Como a árvore escolhe as divisões

O algoritmo é guloso. Em cada nó ele testa todas as features e todos os pontos de corte possíveis e fica com a divisão que deixa os dois filhos mais "puros", isto é, com uma classe mais predominante. Depois repete o procedimento em cada filho, sem nunca rever uma divisão já feita. Esse esquema de partição recursiva é o do algoritmo CART (Breiman et al., 1984).

Definição 10.3 Impureza de Gini e entropia

Em um nó com proporção p_k de observações da classe k, as duas medidas usuais de impureza são

$$G = 1 - \sum_k p_k^2 \qquad \text{e} \qquad H = -\sum_k p_k \log_2 p_k. \tag{10.5}$$

Ambas valem zero em um nó puro e são máximas quando as classes têm a mesma proporção. Com duas classes, $G = 2p(1-p)$, com máximo de 0,5. A divisão escolhida é a que mais reduz a impureza média dos filhos, ponderada pelo número de observações de cada um.

Em regressão o critério é o mesmo, com a variância do alvo no lugar da impureza. Gini e entropia costumam levar a árvores muito parecidas, e o scikit-learn usa Gini por padrão.

Exemplo 10.1 O ganho da primeira divisão

Na raiz da Figura 10.3 há 9.672 altas em 18.506 observações, $p = 0{,}5226$, e a impureza de Gini é $2 \times 0{,}5226 \times 0{,}4774 = 0{,}4990$. A divisão em `vol_razao` cria um filho com 7.579 observações e $G = 0{,}4946$ e outro com 10.927 e $G = 0{,}5000$. A impureza média ponderada é $(7.579 \times 0{,}4946 + 10.927 \times 0{,}5000)/18.506 = 0{,}4978$. Com mais casas, a redução é de 0,49897 para 0,49780: um ganho de 0,0012, em uma escala cujo máximo é 0,5. A melhor pergunta possível sobre dez features quase não separa altas de quedas.

Deixada livre, a árvore cresce até que cada folha seja pura, o que no limite significa uma folha por observação. Os **critérios de parada** limitam esse crescimento: profundidade máxima (`max_depth`), número mínimo de observações por folha (`min_samples_leaf`) e ganho mínimo para dividir (`min_impurity_decrease`). São eles os hiperparâmetros que posicionam a árvore no eixo horizontal da Figura 10.1.

A folha destacada na Figura 10.3 mostra o perigo. Ela diz que uma ação com volatilidade diária acima de 2,2% nos últimos 21 pregões, mas abaixo de 88% da sua volatilidade de seis meses, sobe em 62% dos casos, o que parece uma regra de operação. Das 1.951 observações, 35% são de MGLU3 e 70% caem em quatro anos (2016, 2017, 2018 e 2020), em que ações voláteis se recuperavam de quedas fortes; em 2012 e em 2015 a mesma folha teve menos de 50% de altas. A regra descreve alguns episódios de poucas ações, com rótulos sobrepostos de dias vizinhos, e uma árvore mais funda produz dezenas de folhas como essa.

10.4.2 Por que uma árvore sozinha é instável

Uma árvore tem viés baixo, porque com profundidade suficiente aproxima qualquer função, e variância alta. A razão está na estrutura hierárquica: uma pequena mudança nos dados pode trocar a divisão da raiz, e tudo o que vem abaixo muda junto. O script `cap10/l02_arvore.py` mostra isso reamostrando com reposição os dias do conjunto de desenvolvimento e reajustando a mesma árvore rasa cinco vezes. A raiz dividiu em `ibov_21` em quatro das amostras (com cortes em 0,082, 0,079 e, duas vezes, 0,003) e em `vol_razao`, a feature da raiz original, em apenas uma. Quando a escolha depende tanto do sorteio, o que se está vendo é variância. A Figura 10.4 mostra o mesmo fenômeno no espaço de duas features.

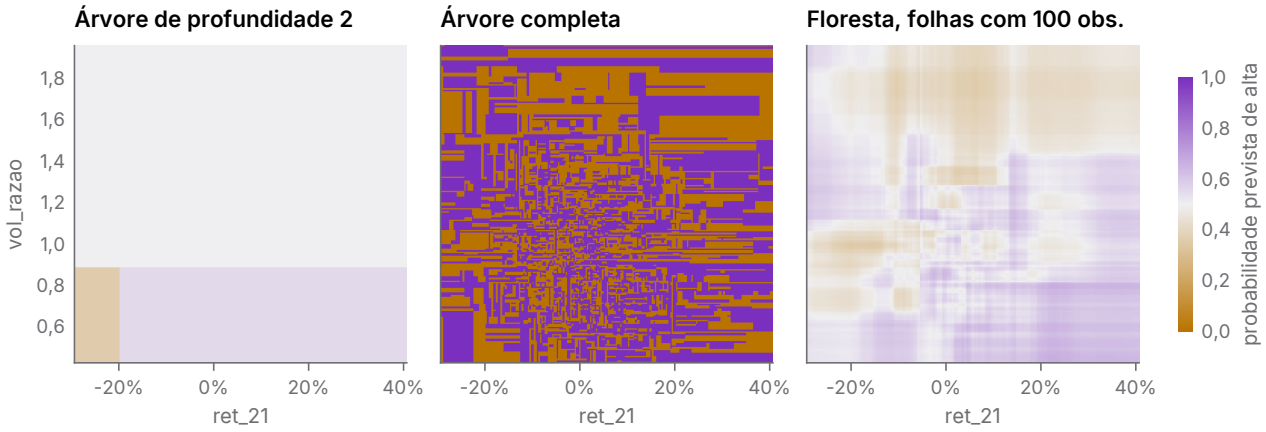


Figura 10.4 Probabilidade de alta prevista a partir de duas features. A árvore rasa mal se afasta de 50%. A árvore completa tem 6.482 folhas, acerta 100% do treino e afirma 0 ou 1 em cada retalho do plano, um desenho que só pode ser ruído. A floresta, média de 300 árvores com folhas de pelo menos 100 observações, devolve uma superfície suave e probabilidades moderadas.

10.5 Bagging e random forest

Se o problema da árvore é a variância, a solução estatística clássica é tirar a média de muitas. A dificuldade é que só existe uma amostra de treino. O **bootstrap** contorna isso: sorteiam-se n observações da amostra original com reposição, o que gera uma amostra nova do mesmo tamanho em que algumas observações aparecem repetidas e outras ficam de fora. A probabilidade de uma observação ficar de fora de uma amostra bootstrap é $(1-1/n)^n \approx e^{-1} \approx 36{,}8\%$, de modo que cada amostra bootstrap contém cerca de 63,2% das observações distintas.

O **bagging** (de *bootstrap aggregating*), proposto por Breiman (1996), ajusta uma árvore funda a cada uma de B amostras bootstrap e tira a média das previsões. É o primeiro exemplo de **ensemble**, um modelo formado pela combinação de vários. O quanto a média ajuda depende de quão parecidas são as árvores.

Resultado 10.4 Variância da média de previsões correlacionadas

Se B previsões têm a mesma variância σ^2 e correlação ρ entre cada par, a variância da média é

$$\operatorname{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x)\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2. \tag{10.6}$$

A conta sai da variância de uma soma: são B termos de variância σ^2 e $B(B-1)$ covariâncias $\rho\sigma^2$, tudo dividido por B^2. A Equação (10.6), que é a equação 15.1 de Hastie et al. (2009), tem duas leituras. O segundo termo some quando B cresce, e por isso acrescentar árvores nunca piora o ensemble, apenas custa tempo. O primeiro termo não depende de B: se as árvores são muito parecidas, a média pouco ajuda. Árvores de bagging são parecidas, porque todas enxergam as mesmas features e tendem a começar pelas mesmas divisões.

A **random forest** de Breiman (2001) ataca o primeiro termo. Em cada divisão de cada árvore, o algoritmo sorteia m das p features e só pode escolher entre elas. Uma feature dominante deixa de estar disponível em boa parte dos nós, as árvores passam a usar caminhos diferentes e a correlação entre elas

cai. Cada árvore fica um pouco pior; a média fica melhor. O padrão usual em classificação é $m = \sqrt{p}$.

Código 10.3 Bagging e random forest na última divisão temporal do painel (trecho de `cap10/l03_floresta.py`)

```
casos = (("bagging (m = p)", None), ("floresta (m = 3)", 3))
for nome, m in casos:
    rf = RandomForestClassifier(
        n_estimators=300, max_features=m, min_samples_leaf=50,
        oob_score=True, n_jobs=4, random_state=0)
    rf.fit(X[tr], y[tr])
    print(f"{nome}: correlação {corr_media(rf, X[va]):.3f}"
          f" | OOB {rf.oob_score_:.3f}"
          f" | validação {rf.score(X[va], y[va]):.3f}")
```

```
bagging (m = p): correlação 0.148 | OOB 0.623 | validação 0.519
floresta (m = 3): correlação 0.116 | OOB 0.607 | validação 0.516
árvores completas: treino 1.000 | OOB 0.719 | validação 0.508
```

A função `corr_media`, definida no mesmo arquivo, calcula a correlação média entre as probabilidades previstas por pares de árvores. Ela cai de 0,148 para 0,116 quando m passa de 10 para 3. Essa medida, feita ao longo das observações de validação, não é exatamente o ρ da Equação (10.6), definido sobre sorteios da amostra de treino, mas dá a ordem de grandeza: com ρ em torno de 0,1 e $B = 300$, a variância da média fica pouco acima de um décimo da variância de uma árvore.

10.5.1 O erro out-of-bag e por que desconfiar dele

Cada árvore deixa de fora cerca de 37% das observações. Prevendo cada observação apenas com as árvores que não a viram, obtém-se o **erro out-of-bag** (OOB), uma estimativa de erro fora da amostra que sai de graça, sem separar validação. Com observações independentes ela funciona bem.

No painel ela falha, e a saída do Código 10.3 mostra o tamanho da falha: acurácia OOB de 61% a 62%, contra 52% na validação temporal. Com árvores completas, 72% contra 51%.

> **Armadilha** **OOB com observações que se sobrepõem no tempo**
>
> O rótulo do dia t usa o retorno de t a $t + 5$; o do dia $t + 1$ compartilha quatro desses cinco dias, e as features dos dois dias são quase iguais. Quando o dia t fica fora de uma árvore, é muito provável que $t - 1$ ou $t + 1$ estejam dentro, e a árvore "prevê" t lembrando do vizinho. As outras ações do mesmo dia, que se movem juntas, agravam o efeito. López de Prado (2018) dedica parte do capítulo 6 aos problemas que a redundância entre observações cria para o bagging. Em séries financeiras, avalie com validação temporal com folga (Capítulo 9) ou com a validação cruzada purgada do Capítulo 8.

Os hiperparâmetros que importam em uma random forest são poucos. O número de árvores (`n_estimators`) deve ser grande o bastante para a previsão estabilizar; algumas centenas bastam. O número de features por divisão (`max_features`) controla a correlação entre árvores. O tamanho mínimo da

folha (`min_samples_leaf`) ou a profundidade máxima controlam a variância de cada árvore e, em dados ruidosos, são os que mais mudam o resultado. Por fim, `max_samples` reduz o tamanho de cada amostra bootstrap, o que López de Prado (2018) recomenda quando as observações se sobrepõem.

10.6 Gradient boosting

O bagging constrói árvores fundas, ajustadas separadamente, e tira a média para reduzir variância. O **boosting** segue o caminho oposto: constrói árvores rasas em sequência, cada uma corrigindo os erros das anteriores, e reduz o viés aos poucos. O modelo final é uma soma,

$$F_M(x) = F_0 + \nu \sum_{m=1}^{M} h_m(x), \tag{10.7}$$

em que F_0 é uma constante, cada h_m é uma árvore pequena e $0 < \nu \leq 1$ é a **taxa de aprendizado**.

A contribuição de Friedman (2001) foi mostrar como escolher cada h_m para uma função de perda L qualquer. Na etapa m, calcula-se para cada observação o gradiente negativo da perda em relação à previsão atual,

$$r_{im} = -\left.\frac{\partial L\big(y_i, F(x_i)\big)}{\partial F(x_i)}\right|_{F=F_{m-1}}, \tag{10.8}$$

e ajusta-se uma árvore de regressão a esses valores. É uma descida de gradiente em que cada passo é uma árvore. Para a perda quadrática $L = \frac{1}{2}(y - F)^2$, o gradiente negativo é $y_i - F_{m-1}(x_i)$, o resíduo: cada árvore aprende o que as anteriores erraram. Para a perda logarítmica da classificação binária, com F na escala de log-chance, ele é $y_i - p_i$, a diferença entre o rótulo e a probabilidade prevista. Fora do caso quadrático há uma segunda etapa: definida a estrutura da árvore, o valor de cada folha é recalculado para minimizar a perda nas observações que caíram nela. O Código 10.4 implementa o caso quadrático em poucas linhas, sobre 200 pontos do processo simulado da Seção 10.1 (`X`, `y`), e o script confere que o resultado coincide com o do `GradientBoostingRegressor` do scikit-learn.

Código 10.4 Gradient boosting escrito à mão: cada árvore é ajustada ao resíduo das anteriores (trecho de `cap10/l04_boosting.py`)

```
eta, M = 0.1, 100
F = np.full(len(y), y.mean())          # F_0: a melhor constante
for m in range(1, M + 1):
    residuo = y - F                    # gradiente negativo da perda
    arvore = DecisionTreeRegressor(max_depth=2)
    F += eta * arvore.fit(X, residuo).predict(X)
    if m in (1, 10, 100):
        print(f"{m:3d} árvores: EQM de treino "
              f"{np.mean((y - F) ** 2):.4f}")
```

```
  1 árvores: EQM de treino 0.6258
 10 árvores: EQM de treino 0.3121
100 árvores: EQM de treino 0.1672
igual ao scikit-learn: True
```

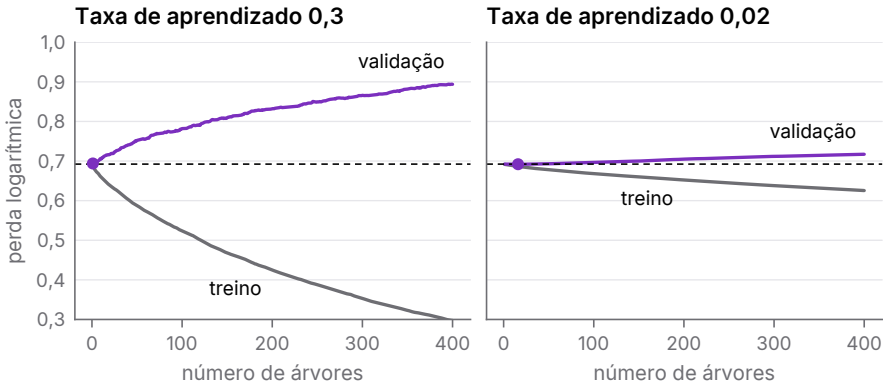


Figura 10.5 Perda logarítmica do LightGBM no painel, por número de árvores (árvores de 15 folhas; a linha tracejada é a perda de prever sempre a frequência de altas do treino). A perda de treino cai indefinidamente. A de validação tem seu mínimo, marcado com um ponto, logo no começo, e depois só piora. Com taxa de aprendizado menor o estrago é mais lento, mas a direção é a mesma.

O ruído desses dados tem variância 0,25. Com 100 árvores o erro de treino já está em 0,167, abaixo de σ^2, o mínimo possível para o erro em dados novos: o modelo já está ajustando ruído. O boosting não para sozinho. Quatro hiperparâmetros controlam isso.

- **Taxa de aprendizado e número de árvores.** Andam juntos. Um ν pequeno pede mais árvores e costuma generalizar melhor, porque nenhuma árvore isolada pesa muito.
- **Tamanho das árvores.** Profundidade ou número de folhas. Árvores de profundidade d capturam interações entre até d features. Valores pequenos são a regra.
- **Subamostragem.** Ajustar cada árvore em uma fração das linhas (Hastie et al., 2009, seção 10.12) ou, nas bibliotecas atuais, também das colunas acrescenta aleatoriedade e reduz a variância, como na random forest.
- **Parada antecipada.** Em vez de fixar M, acompanha-se a perda em um conjunto de validação e interrompe-se o treino quando ela para de cair (*early stopping*).

A Figura 10.5 mostra o que a parada antecipada encontraria no painel, e o resultado é desconcertante. Com $\nu = 0{,}3$, a perda de validação é mínima com uma única árvore, e mesmo esse mínimo (0,6939) fica acima da referência sem modelo (0,6923); com 400 árvores ela chega a 0,894, enquanto a de treino desce a 0,30. Com $\nu = 0{,}02$ o mínimo ocorre com 16 árvores e vale 0,6920. O algoritmo capaz de ajustar qualquer coisa conclui, quando avaliado com honestidade, que há pouquíssimo a ajustar. Lembre também que o conjunto usado para decidir quando parar deixa de ser um teste: ele participou de uma escolha.

10.6.1 O que XGBoost e LightGBM acrescentam

As duas bibliotecas mais usadas implementam a ideia de Friedman com melhorias de engenharia e de regularização. O XGBoost (Chen e Guestrin, 2016) coloca a penalidade dentro do objetivo: a cada árvore soma-se um termo $\gamma T + \frac{1}{2}\lambda \sum_j w_j^2$, em que T é o número de folhas e w_j o valor de cada folha. Usando uma aproximação de segunda ordem da perda, o valor ótimo de uma folha é $w_j^* = -G_j/(H_j + \lambda)$, com G_j e H_j as somas das primeiras e segundas derivadas da perda nas observações da folha. O λ encolhe as folhas como no

ridge, e uma divisão só é feita se o ganho superar γ. O artigo também descreve o tratamento nativo de valores ausentes e a busca aproximada de pontos de corte.

O LightGBM (Ke et al., 2017) mira a velocidade. Ele agrupa os valores de cada feature em faixas e procura cortes em histogramas, e propõe duas técnicas: amostrar preferencialmente as observações de gradiente grande (GOSS) e empacotar features esparsas que raramente são não nulas ao mesmo tempo (EFB). Segundo a documentação da biblioteca, as árvores crescem folha a folha (*leaf-wise*): a cada passo divide-se a folha de maior ganho, onde quer que esteja, em vez de completar um nível inteiro. Para o mesmo número de folhas isso reduz mais a perda de treino e produz árvores assimétricas e fundas, razão pela qual o hiperparâmetro central do LightGBM é `num_leaves`, e não a profundidade. Os demais controles têm nomes quase iguais nas duas bibliotecas: `learning_rate`, `n_estimators`, `subsample`, `colsample_bytree` e `reg_lambda`; o mínimo de observações por folha é `min_child_samples` no LightGBM e `min_child_weight` (uma soma de segundas derivadas) no XGBoost.

10.7 Uma comparação honesta

Com os três tipos de modelo em mãos, a pergunta prática é qual deles prevê melhor o rótulo do painel. O script `cap10/l05_comparacao.py` ajusta uma regressão logística (com as features padronizadas em um `Pipeline`), uma random forest, um XGBoost e um LightGBM, todos com parâmetros de fábrica e 300 árvores, nas cinco divisões temporais do conjunto de desenvolvimento, e mede a AUC (Capítulo 9) no treino e na validação de cada divisão. A saída traz a média das cinco divisões e, para a validação, também a pior e a melhor.

```
               AUC treino  AUC valid.   pior  melhor
logística           0.549       0.506  0.492   0.541
random forest       1.000       0.507  0.484   0.525
xgboost             1.000       0.510  0.485   0.534
lightgbm            0.994       0.504  0.484   0.523
```

A coluna de treino cresce com a flexibilidade do modelo: 0,55 para a logística e 1,00 para os ensembles, que separam perfeitamente altas de quedas nos dados que viram. A coluna de validação não se mexe. Os quatro modelos ficam entre 0,504 e 0,510, e as colunas de pior e melhor divisão mostram que a diferença entre modelos é muito menor do que a variação de um mesmo modelo entre períodos. Uma AUC de 0,50 é a de um sorteio.

10.7.1 Ajuste de hiperparâmetros

Parâmetros de fábrica são feitos para problemas com muito mais sinal. O passo seguinte é procurar uma configuração mais contida, com a mesma validação temporal. O Código 10.5 sorteia 40 combinações de hiperparâmetros do LightGBM (uma **busca aleatória**) e avalia cada uma nas cinco divisões.

Código 10.5 Busca aleatória com validação temporal (trecho de `cap10/l06_busca.py`)

```
grade = {
    "n_estimators": [25, 50, 100, 200, 400],
    "learning_rate": [0.01, 0.02, 0.05, 0.1],
    "num_leaves": [3, 5, 9, 15, 31],
    "min_child_samples": [20, 100, 300, 1000],
    "subsample": [0.5, 0.7, 1.0],
    "colsample_bytree": [0.5, 0.7, 1.0],
    "reg_lambda": [0, 1, 10, 100],
}
base = LGBMClassifier(subsample_freq=1, n_jobs=1, random_state=0,
                      verbose=-1)
busca = RandomizedSearchCV(
    base, grade, n_iter=40, scoring="roc_auc",
    cv=cortes(dev.index), n_jobs=4, random_state=0).fit(X, y)
```

```
melhor configuração:
{'subsample': 0.5, 'reg_lambda': 100, 'num_leaves': 15}
{'n_estimators': 400, 'min_child_samples': 20, 'learning_rate': 0.02}
{'colsample_bytree': 1.0}
AUC de validação das 40 tentativas:
média 0.5057 | pior 0.4964 | melhor 0.5108
cinco melhores: [0.5108, 0.5106, 0.5105, 0.5102, 0.5095]
```

A configuração vencedora mistura sinais: usa a maior penalidade L2 da grade, taxa de aprendizado baixa e metade das linhas por árvore, que apontam para um modelo contido, mas também o maior número de árvores e o menor mínimo por folha. Não convém ler muito nela. A AUC de 0,5108 é o máximo de 40 tentativas cuja média foi 0,5057, as cinco melhores diferem entre si em menos de 0,002, e o máximo de muitos números ruidosos tende a superar o valor verdadeiro do escolhido. Qualquer pequena mudança nos dados troca a vencedora.

O argumento passado em `cv` é a lista de divisões por data, e a escolha importa. O script `cap10/t_cv.py` avalia o mesmo LightGBM de fábrica de três maneiras. Um `KFold` com `shuffle=True` põe observações vizinhas dos dois lados e repete a ilusão do OOB: AUC média de 0,70. O padrão do scikit-learn, `cv=5`, não embaralha; usa blocos contíguos, mas treina com dados posteriores à validação e não deixa folga entre os blocos, e dá 0,46, com dobras entre 0,41 e 0,51. As divisões temporais com folga dão 0,50. Três procedimentos, três respostas para o mesmo modelo e os mesmos dados; só o último imita a situação de quem vai operar.

10.7.2 O teste final, usado uma única vez

Tudo o que foi decidido até aqui (features, rótulo, modelo, hiperparâmetros) usou o conjunto de desenvolvimento. A validação temporal protegeu cada avaliação individual, mas as escolhas se acumularam sobre os mesmos dados. A única estimativa limpa vem de um período que não participou de nenhuma decisão. O script `cap10/t_teste_final.py` treina cada modelo uma vez em todo o desenvolvimento e o aplica a 2022–2025.

A Tabela 10.1 deve ser lida sem torcida. A linha que conta é a última, a do modelo escolhido pela validação; as outras três estão ali para ilustrar, e em um projeto real não seriam calculadas. O LightGBM ajustado teve AUC de 0,499. Uma reamostragem por blocos de dez pregões dá a essa AUC um desvio-padrão

Tabela 10.1 Desempenho no período intocado, de 3 de janeiro de 2022 a 19 de dezembro de 2025 (6.951 observações, 51,5% de altas)

Modelo	AUC treino	AUC teste	Acurácia teste
Regressão logística	0,539	0,493	0,506
Random forest (padrão)	1,000	0,504	0,511
LightGBM (padrão)	0,969	0,503	0,504
LightGBM (ajustado na busca)	0,680	0,499	0,501

de 0,011: ela não se distingue de 0,5, e as outras três também não. Nenhum dos quatro modelos alcança a acurácia de 51,5% de quem prevê alta todos os dias.

Armadilha **Trocar de modelo depois de ver o teste**

A tentação agora é adotar a random forest de fábrica, porque "foi a melhor no teste". No instante em que o teste é usado para escolher, ele vira validação, e não resta nenhum dado limpo para estimar o desempenho da escolha. O período final serve para uma única pergunta, feita uma única vez: o modelo que eu já tinha decidido usar funciona?

10.7.3 Por que modelos complexos falham em finanças

O resultado não é um acidente deste conjunto de dados. Cinco características do problema empurram na mesma direção.

- **Pouco sinal para muito ruído.** Pela Seção 10.1, quanto maior o ruído, mais simples é o melhor modelo. Gu et al. (2020), com cerca de 30 mil ações americanas e décadas de dados, encontram R^2 fora da amostra abaixo de 1% ao mês para os melhores modelos, e ainda assim mostram que isso tem valor econômico.
- **Poucas observações independentes.** O desenvolvimento tem 18.506 linhas, mas elas vêm de 2.707 pregões, isto é, 541 períodos de cinco dias sem sobreposição, e a correlação média entre os retornos de cinco dias das sete ações é de 0,36. O número efetivo de observações é uma pequena fração do número de linhas.
- **Não estacionariedade.** A relação entre features e retornos muda com o regime de juros, de liquidez e de competição. Um padrão que foi lucrativo tende a atrair capital até deixar de sê-lo. O modelo aprende uma média de regimes passados, e a variação entre divisões na comparação acima é o sintoma.
- **Múltiplos testes.** Cada feature, rótulo, modelo e hiperparâmetro experimentado é um teste, e o melhor de muitos testes é enviesado para cima, como se viu na busca. É o mesmo problema que o Sharpe deflacionado do Capítulo 7 corrige, tratado por Bailey et al. (2014) e Bailey e López de Prado (2014). Anote quantas variantes você tentou.
- **Vazamento.** Com modelos flexíveis, qualquer informação do futuro que escape para as features ou para o treino é explorada integralmente. O OOB inflado da Seção 10.5 é um exemplo; o Capítulo 9 lista os outros.

Nada disso quer dizer que ensembles sejam inúteis em finanças. O mesmo estudo de Gu et al. (2020) conclui que árvores e redes neurais superam modelos lineares, graças a interações não lineares entre preditores. A diferença está na

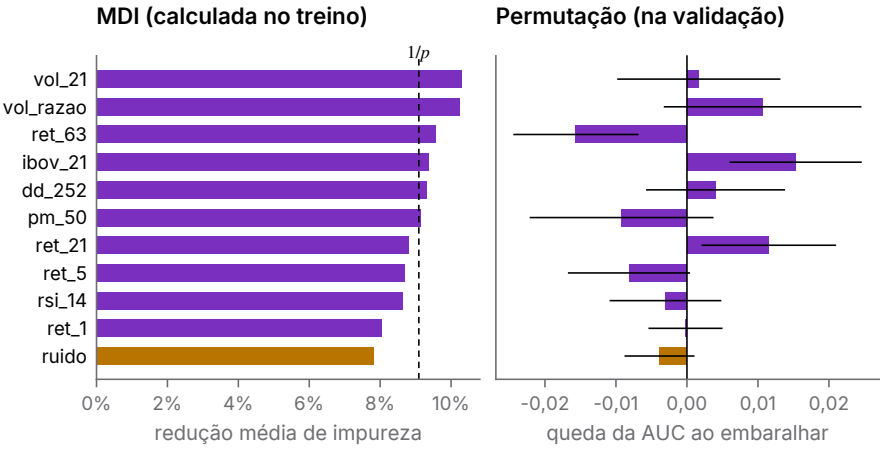


Figura 10.6 Duas medidas de importância para a mesma floresta, com uma feature de ruído puro (em ocre). A MDI dá ao ruído 7,8% da importância total, pouco abaixo da parcela igualitária de $1/p = 9{,}1\%$. Na permutação fora da amostra, com barras de erro de dois desvios-padrão entre 30 embaralhamentos, metade das features reais tem importância negativa: embaralhá-las melhorou a AUC.

escala: milhares de ativos, quase uma centena de características por ação e décadas de história, contra sete ações e dez indicadores de preço. Com poucos dados, o modelo simples e bem regularizado é o ponto de partida, e o complexo precisa provar fora da amostra que merece o lugar.

10.8 Interpretar o modelo

Saber quais features o modelo usa serve para depurar (uma feature com importância absurda costuma ser vazamento), para simplificar e para confrontar o modelo com a intuição econômica. López de Prado (2018) dedica o capítulo 8 ao tema e defende que a análise de importância é uma ferramenta de pesquisa melhor do que rodar backtests repetidos. Há três famílias de medida.

A **importância por impureza**, ou MDI (*mean decrease in impurity*), soma, para cada feature, a redução de impureza de todas as divisões em que ela foi usada, ponderada pelo número de observações, e normaliza o total para 1. É o atributo `feature_importances_` do scikit-learn. Ela é calculada inteiramente no treino e, portanto, mede o quanto a feature foi usada para ajustar, o que inclui ajustar ruído. Também favorece features contínuas ou com muitos valores distintos, que oferecem mais pontos de corte (Strobl et al., 2007).

A **importância por permutação**, ou MDA (*mean decrease in accuracy*), embaralha os valores de uma feature em dados fora da amostra e mede o quanto a métrica piora. Se o modelo não depende da feature, ou se a dependência é ruído, nada muda. Ela vale para qualquer modelo e qualquer métrica.

A Figura 10.6 compara as duas em uma random forest padrão, depois de acrescentar ao painel uma décima primeira coluna de números aleatórios, sem relação nenhuma com o alvo.

A MDI não consegue dizer que uma feature é inútil. O ruído recebe 7,8%, a feature mais importante (`vol_21`) recebe 10,3%, e o gráfico sugere onze features de relevância parecida. A permutação conta outra história. A floresta tem AUC de 0,506 nessa validação, e uma importância negativa quer dizer que embaralhar a feature melhorou a AUC. Cinco das dez features reais têm importância negativa, e `ret_63` fica mais de três desvios abaixo de zero: o que a floresta aprendeu com ela no treino atrapalha na validação. Duas ficam mais

de dois desvios acima de zero, `ibov_21` e `ret_21`; em onze testes, um ou outro resultado assim ao acaso não surpreende. A coluna de ruído fica em −0,004, no meio das outras, e a maior parte das features reais não se distingue dela. As barras refletem só o sorteio da permutação, não a incerteza da amostra, que é maior.

Armadilha **Features correlacionadas dividem a importância**

Quando duas features carregam a mesma informação, a MDI reparte a importância entre elas e as duas parecem fracas. Na permutação, embaralhar uma delas quase não afeta o modelo, que se apoia na outra, e as duas parecem dispensáveis. López de Prado (2018), no capítulo 8, chama isso de efeito de substituição. No painel, `pm_50`, `ret_21` e `rsi_14` têm correlações entre 0,80 e 0,89. O remédio é agrupar features parecidas e medir a importância do grupo.

A terceira família são os valores SHAP (Lundberg e Lee, 2017), que decompõem cada previsão individual em uma soma de contribuições, uma por feature, mais um valor base. As contribuições são os valores de Shapley da teoria dos jogos: a contribuição média de uma feature, considerando todas as ordens em que as features poderiam ser reveladas ao modelo. Para ensembles de árvores existe um algoritmo exato e rápido, e o LightGBM o oferece no argumento `pred_contrib`. Para a última observação do conjunto de desenvolvimento, o script `cap10/l07_contribuicoes.py` obtém um valor base de 0,0926 em log-chance (probabilidade de 52,3%, praticamente a frequência de altas do treino) e contribuições de +0,1256 para `ret_63`, −0,0766 para `pm_50` e −0,0546 para `rsi_14`, com as demais menores. A soma de todas com o valor base, 0,1644, é exatamente o escore do modelo, que equivale a uma probabilidade de 54,1%. Essa aditividade é o que torna o SHAP útil para explicar uma previsão específica. A média dos valores absolutos por feature dá uma medida global de importância, que herda da MDI a limitação de descrever o que o modelo faz, e não se o que ele faz generaliza.

10.8.1 Dependência parcial

Importância diz quanto uma feature pesa, mas não em que direção. A **dependência parcial**, também introduzida por Friedman (2001), responde a isso: fixa-se a feature em um valor para todas as observações, mantendo as demais como estão, e tira-se a média das previsões; repetindo para uma grade de valores, obtém-se uma curva.

Na Figura 10.7, a probabilidade prevista de alta cai de 0,574 para 0,475 entre os extremos de `vol_razao`, e de 0,563 para 0,501 entre os de `ret_5`, com idas e vindas no caminho. A leitura econômica seria que ações com volatilidade recente baixa em relação à própria história e ações que caíram na última semana tendem a subir um pouco mais. As curvas cinzas dão a medida da confiança. Nas cinco janelas a curva termina abaixo de onde começa, para as duas features, mas a amplitude vai de 6 a 13 pontos percentuais conforme a janela, e as ondulações de uma janela não se repetem na outra. As janelas são aninhadas, o que por si só produz alguma concordância. E a Tabela 10.1 já mostrou que esse modelo não previu o período seguinte. Um gráfico de dependência parcial descreve o modelo; para descrever o mercado, o modelo precisa primeiro funcionar fora da amostra. A curva também pode enganar quando as features são correlacionadas, porque a grade inclui combinações de valores que nunca ocorrem juntas.

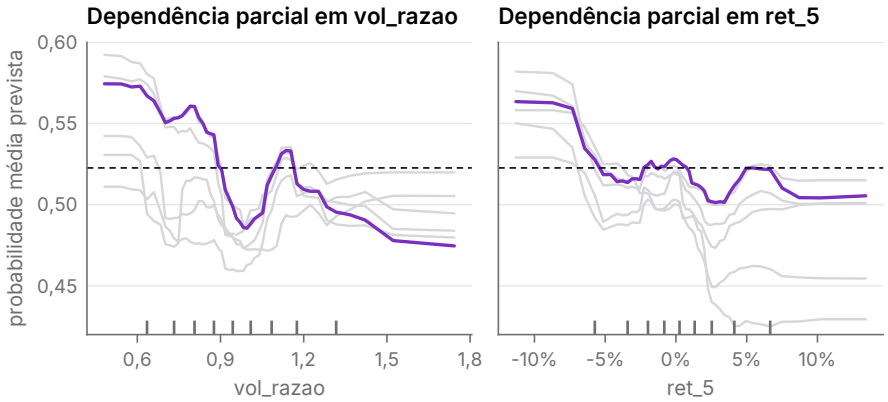


Figura 10.7 Dependência parcial do LightGBM ajustado em duas features. Em roxo, o modelo treinado em todo o desenvolvimento; em cinza, os modelos das cinco janelas de treino da validação temporal. A linha tracejada é a frequência de altas e os traços na base marcam os decis da feature. O sentido geral das curvas se repete entre janelas; a amplitude e as ondulações, não.

10.8.2 Seleção de features

As três medidas servem para descartar features, e o lasso da Seção 10.3 faz isso sozinho. Um critério prático de corte é a própria coluna de ruído: uma feature cuja importância por permutação fora da amostra não supera a do ruído é candidata a sair. Menos features significam menos variância e menos oportunidades de ajustar coincidências.

O cuidado é o mesmo da padronização: a seleção faz parte do treino. Se você escolhe as melhores features olhando o conjunto inteiro e só depois valida, a validação já foi vista, e o desempenho medido é otimista mesmo que o modelo final seja simples. A seleção deve ser refeita dentro de cada divisão, apenas com o treino daquela divisão, o que no scikit-learn se consegue colocando o seletor em um `Pipeline` junto com o modelo. E cada rodada de seleção guiada pela validação é mais um teste a entrar na conta dos múltiplos testes.

10.9 Da previsão à posição

Um classificador devolve uma probabilidade, e uma carteira precisa de quantidades. A regra mais simples é um limiar: comprar se $p > 0{,}5$. Ela descarta informação, pois trata $p = 0{,}51$ e $p = 0{,}80$ da mesma forma. A alternativa é fazer o **tamanho da aposta** crescer com a confiança. A versão linear é $m = 2p - 1$, que vai de -1 (vendido) a $+1$ (comprado). López de Prado (2018) dedica o capítulo 10 a esse problema. Uma regra dessa família converte a probabilidade em uma estatística $z = (p - \frac{1}{2})/\sqrt{p(1-p)}$ e usa $m = 2\Phi(z) - 1$, com Φ a distribuição acumulada da normal padrão. Para $p = 0{,}55$ isso dá $z = 0{,}10$ e $m = 0{,}08$: uma previsão pouco confiante vira uma posição de 8% do limite. Qualquer que seja a fórmula, ela pressupõe probabilidades calibradas, e as de uma floresta com árvores completas, que chegam a 0 e 1 no treino, não são.

10.9.1 Meta-labeling

López de Prado (2018), no capítulo 3, propõe separar duas decisões que costumam ser pedidas ao mesmo modelo: o lado da aposta e o tamanho dela. No **meta-labeling**, um modelo primário decide o lado. Ele pode ser uma regra simples e interpretável, como um cruzamento de médias (Capítulo 7) ou um si-

nal de reversão (Capítulo 8). Um segundo modelo, de aprendizado de máquina, é treinado apenas nos momentos em que o primário deu sinal, com um rótulo binário: 1 se a operação indicada teria dado lucro, 0 se não. A probabilidade prevista pelo segundo modelo decide se a operação é feita e com que tamanho.

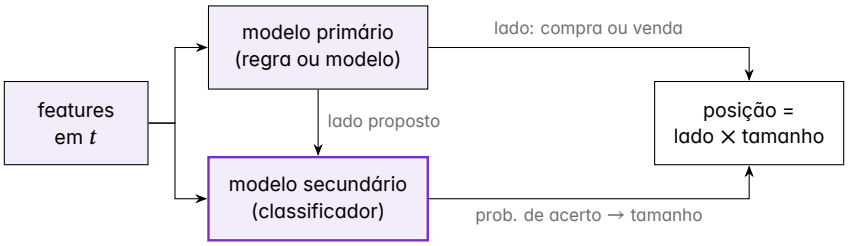


Figura 10.8 Meta-labeling. O modelo primário escolhe o lado. O secundário recebe as features e o lado proposto, estima a probabilidade de a operação dar certo e, com ela, define se e quanto apostar.

A divisão de trabalho tem uma lógica de métricas. O modelo primário pode ser calibrado para não perder oportunidades (recall alto), mesmo que gere muitos sinais falsos, e o secundário filtra os falsos positivos e aumenta a precisão. O aprendizado de máquina fica com a tarefa mais limitada de dizer quando confiar em uma regra que tem justificativa econômica. O meta-labeling não cria sinal onde o modelo primário não tem nenhum, e o modelo secundário está sujeito a todos os problemas de validação deste capítulo.

10.9.2 Um backtest sem maquiagem

Falta ver o que o modelo escolhido na validação, o LightGBM ajustado, teria feito com dinheiro no período final. É a segunda e última consulta a esse período, sobre o mesmo modelo. A regra é a mais simples possível: a cada fechamento, um sétimo do capital em cada ação com $p > 0{,}5$, e o restante em caixa rendendo o CDI. A convenção é a do motor dos Capítulos 7 e 8, a *execução no fechamento do sinal*: o peso w_t é decidido e assumido no fechamento de t e rende o retorno de t a $t+1$. É uma hipótese otimista, e por isso a tabela traz também a versão com um pregão de atraso. O custo por ponta é $c = 0{,}10\%$ do valor negociado. A referência é manter as sete ações em pesos iguais. No Código 10.6, `ret` é a tabela de retornos de t a $t+1$ (uma coluna por ação), `cdi` é a taxa acumulada entre pregões e `CUSTO` vale `0.001`; a função é chamada com os pesos `(prob > 0.5) / n` para o modelo, os mesmos pesos defasados em um dia para a versão com atraso e `1 / n` para a referência. O Sharpe é calculado sobre o excesso em relação ao CDI, como no Capítulo 7, e as taxas são anualizadas pela convenção de 252 dias (Seção 1.9).

Código 10.6 Backtest do modelo escolhido no período final, com custos e caixa no CDI (trecho de `cap10/l08_backtest.py`)

```
def resultado(pesos):
    caixa = 1 - pesos.sum(axis=1)           # rende o CDI
    giro = pesos.diff().abs().sum(axis=1)
    giro.iloc[0] = pesos.iloc[0].abs().sum()
    bruto = (pesos * ret).sum(axis=1) + caixa * cdi
    liq = bruto - CUSTO * giro
    curva = (1 + liq).cumprod()
    exc = liq - cdi                         # excesso sobre o CDI
    anos = len(liq) / 252
    return {"retorno a.a.": curva.iloc[-1] ** (1 / anos) - 1,
            "sem custos": (1 + bruto).prod() ** (1 / anos) - 1,
            "vol a.a.": liq.std() * np.sqrt(252),
            "Sharpe": exc.mean() / exc.std() * np.sqrt(252),
            "drawdown máx.": (curva / curva.cummax() - 1).min(),
            "exposição": pesos.sum(axis=1).mean(),
            "giro a.a.": giro.mean() * 252}
```

```
03/01/2022 a 19/12/2025, 994 pregões | CDI: 12.7% a.a.
                modelo  com atraso  comprar tudo
retorno a.a.     0.030       0.037         0.118
sem custos       0.085       0.092         0.118
vol a.a.         0.136       0.138         0.200
Sharpe          -0.594      -0.537         0.058
drawdown máx.   -0.161      -0.193        -0.252
exposição        0.607       0.607         1.000
giro a.a.       52.080      51.972         0.254
```

O modelo rendeu 3,0% ao ano depois dos custos, contra 11,8% de comprar tudo e 12,7% do CDI. Antes dos custos rendia 8,5%: girar a carteira 52 vezes por ano custou mais de 5 pontos percentuais. A volatilidade e o drawdown máximo foram menores, mas só porque o modelo ficou em média 61% investido; o Sharpe de $-0{,}59$ mostra que ele perdeu do CDI de forma consistente. É o resultado esperado para um classificador com AUC de 0,499: entra e sai ao acaso, perde parte da alta e paga custos por isso. A versão com um pregão de atraso rende um pouco mais (3,7%), sinal de que não há informação no momento exato do sinal.

Comprar tudo tem Sharpe ligeiramente positivo embora sua taxa composta fique abaixo da do CDI. O índice usa a média aritmética dos retornos diários, que supera a taxa composta em cerca de $\sigma^2/2$ ao ano (Capítulo 1), aqui dois pontos percentuais. Duas ressalvas completam o quadro: o backtest não cobra o rebalanceamento diário para pesos iguais, que existe nas três colunas, e o viés de sobrevivência da Seção 10.2 favorece todas elas.

Os scripts `cap10/c_contas.py` e `cap10/baixar_cdi.py` completam a pasta do capítulo: o primeiro refaz as contas feitas à mão no texto (impureza de Gini, tamanho da aposta, correlações, contagens do painel) e o segundo baixa a série do CDI usada aqui.

No repositório

O projeto em `complex/` combina um `GradientBoostingRegressor` com uma regressão `Ridge`, com pesos que mudam conforme o regime de mercado (`complex/thesis_and_model.py` e `complex/config.py`). A demonstração roda sobre dados sintéticos. Um bom exercício é percorrer esse código com a lista abaixo em mãos.

10.9.3 Lista de conferência

1. Comece por um modelo linear regularizado. Ele é a referência que o modelo complexo precisa bater fora da amostra.
2. Separe um período final antes de olhar qualquer resultado e use-o uma vez.
3. Valide respeitando o tempo, com folga do tamanho do horizonte do rótulo. Não confie em OOB nem em validação cruzada embaralhada.
4. Ponha padronização, seleção de features e ajuste de hiperparâmetros dentro da validação.
5. Compare sempre treino e validação. Uma AUC de treino perto de 1 mede a capacidade de decorar.
6. Registre quantas variantes foram testadas e desconte o melhor resultado de acordo.
7. Inclua uma feature de ruído. O que tiver importância parecida com a dela é ruído também.
8. Avalie em dinheiro, com custos e giro, contra uma referência simples.
9. Declare os vieses do conjunto de dados, a começar pelo de sobrevivência.

O que levar deste capítulo

- O erro em dados novos é a soma de ruído, viés ao quadrado e variância. Mais flexibilidade troca viés por variância, e quanto mais ruidoso o problema, mais simples é o melhor modelo.
- Ridge encolhe coeficientes e estabiliza features correlacionadas; lasso zera coeficientes e seleciona features. A intensidade da penalidade se escolhe por validação temporal.
- Random forest reduz variância tirando a média de árvores fundas e descorrelacionadas. Gradient boosting reduz viés somando árvores rasas, cada uma ajustada ao gradiente da perda, e precisa de taxa de aprendizado baixa, árvores pequenas e parada antecipada.
- No painel de sete ações, os ensembles chegaram a AUC de 1,00 no treino e ficaram entre 0,50 e 0,52 na validação e no teste final, sem se distinguir da regressão logística nem de um sorteio.
- Erro out-of-bag e importância por impureza são calculados com dados de treino e enganam em séries financeiras. Prefira validação temporal e importância por permutação fora da amostra.
- A previsão só vira estratégia depois de uma regra de posição, de custos e de uma comparação com uma referência simples em um período que não participou de nenhuma escolha.

Exercícios

10.1 [básico] Um nó tem 60 observações da classe 1 e 40 da classe 0. Uma divisão o separa em um filho com 45 e 15 e outro com 15 e 25. Calcule a impureza de Gini do nó e dos filhos e o ganho da divisão.

10.2 [básico] Use a Equação (10.6) para calcular a variância da média de $B = 10, 100$ e 1.000 árvores com $\sigma^2 = 1$, para $\rho = 0{,}5$ e $\rho = 0{,}05$. A partir de que ponto acrescentar árvores deixa de ajudar em cada caso?

10.3 [intermediário] No Código 10.2, acrescente ao painel uma cópia de `pm_50` somada a um ruído pequeno. Compare o que MQO, ridge e lasso fazem com os coeficientes da feature original e da cópia.

10.4 [intermediário] Repita o Código 10.3 com o horizonte do rótulo em um dia (`H = 1` em `painel.py`). A distância entre a acurácia OOB e a de validação diminui? Explique.

10.5 [intermediário] Troque o rótulo do painel por "o retorno de cinco dias da ação ficou acima da mediana das sete ações naquele dia" e refaça a comparação de `cap10/l05_comparacao.py`. Que parte do problema esse rótulo remove?

10.6 [desafio] Implemente um meta-labeling mínimo: use como modelo primário a regra "comprado quando o preço está acima da média de 50 dias" e treine um classificador para prever, apenas nos dias com sinal, se o retorno dos cinco dias seguintes foi positivo. Avalie precisão e recall com e sem o filtro, na validação temporal.

10.7 [desafio] Rode a busca do Código 10.5 com 10, 40 e 160 tentativas e registre a melhor AUC de validação e a AUC da configuração vencedora em um período que não entrou na busca (reserve 2020–2021 para isso, sem tocar no teste final). Como a diferença entre as duas evolui com o número de tentativas?

Para ir além

- James et al. (2021), capítulos 6 e 8, para regularização e métodos baseados em árvores no nível deste capítulo, com mais exemplos.
- Hastie et al. (2009), seções 3.4 e 7.3 e capítulos 10 e 15, para a teoria de ridge e lasso, viés-variância, boosting e random forests.
- López de Prado (2018), capítulos 3, 6, 7 e 8, sobre meta-labeling, ensembles com observações sobrepostas, validação cruzada em finanças e importância de features; López de Prado (2020), seção 6, aprofunda a importância com features correlacionadas.
- Gu et al. (2020), a comparação empírica de referência entre modelos lineares, árvores e redes neurais na previsão de retornos.
- Jansen (2020), capítulos 11 e 12, com random forests e gradient boosting aplicados a estratégias, em Python.

Capítulo 11

Algoritmos de trading: ordens, custos e execução

11

Um backtest termina em uma curva de capital. Entre essa curva e dinheiro na conta existe uma camada que os capítulos anteriores resumiram em um único número, o custo por ponta: a ordem precisa chegar à bolsa, encontrar contraparte, ser executada a algum preço e pagar por isso. Este capítulo trata dessa camada, do livro de ofertas ao programa que conversa com a corretora, e mostra quanto dela uma estratégia aguenta.

Ao terminar este capítulo você consegue

- descrever os componentes de um bot de trading e o laço de eventos que os liga;
- ler um livro de ofertas e escolher entre ordem a mercado, limitada e stop sabendo o que cada uma garante;
- estimar o custo de uma operação somando taxas, meio spread e impacto de mercado;
- calcular o custo de equilíbrio de uma estratégia e relacioná-lo ao giro;
- construir e testar uma corretora simulada sobre barras diárias, conhecendo seus limites;
- dimensionar posições e cercar um sistema automático com controles de risco operacional.

11.1 Do backtest ao mercado

Um backtest como os do Capítulo 7 usa a convenção de execução no fechamento do sinal e cobra um custo fixo de 0,10% por ponta. Por trás disso há três suposições: a ordem é executada no instante em que o sinal fica pronto, pelo preço de fechamento que o gerou, na quantidade inteira. No mercado nenhuma das três vale por construção, e o custo de 0,10% é uma estimativa que este capítulo ensina a abrir.

- **Latência.** Entre o dado chegar, o programa decidir e a ordem alcançar a bolsa passa um tempo, de milissegundos a minutos conforme a infraestrutura. Nesse intervalo o preço anda.

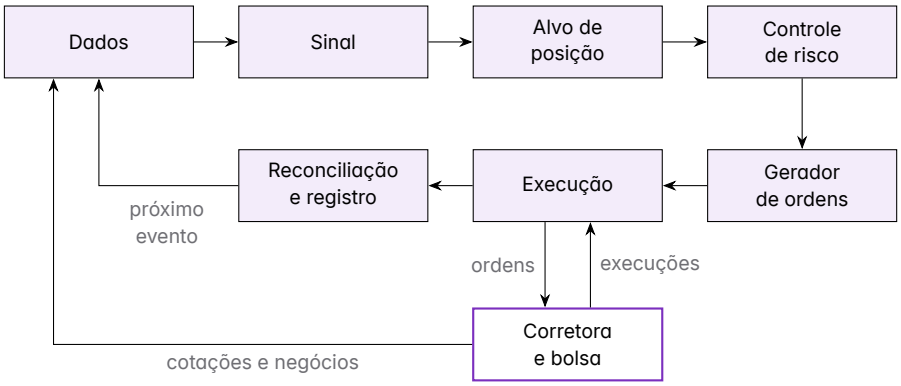


Figura 11.1 O laço de eventos de um bot. A linha de cima decide o que se quer ter; a de baixo transforma a decisão em ordens e confere o que de fato aconteceu. A estratégia em si ocupa só a caixa *Sinal*.

- **Preço de execução.** Você compra pelo preço de quem aceita vender, que é maior que o de quem aceita comprar, e uma ordem grande consome várias ofertas a preços cada vez piores. A diferença entre o preço que a regra supôs e o preço obtido chama-se **slippage**, ou deslizamento.
- **Preenchimento parcial ou nenhum.** Uma ordem com limite de preço pode ser executada em parte, ou ficar sem execução enquanto o mercado se afasta.
- **Dados sujos.** O histórico que você baixa foi corrigido depois; o fluxo em tempo real chega com negócios cancelados, cotações repetidas e buracos. Nem o histórico chega limpo: a cópia bruta das barras de PETR4, em `dados/bruto/`, tem 54 barras com volume zero e os quatro preços idênticos; o Capítulo 2 as identifica, e todas foram removidas do arquivo limpo.
- **Falhas.** A conexão cai, a corretora rejeita a ordem, o programa trava com uma posição aberta. Nada disso acontece em um backtest.

Chan (2021) trata a distância entre resultado simulado e resultado real como parte normal do trabalho e recomenda um período de operação simulada antes de arriscar capital.

11.2 A anatomia de um bot

Um *bot* de trading é um programa que repete um ciclo: recebe um evento (uma barra nova, um negócio, a confirmação de uma ordem), atualiza o que sabe e decide se envia, altera ou cancela ordens. A Figura 11.1 separa esse ciclo em sete responsabilidades. Em um sistema pequeno elas podem ser sete funções no mesmo arquivo, desde que cada uma faça uma coisa só.

A camada de *dados* recebe cotações e negócios, descarta o que é inválido e monta as barras e os indicadores. O *sinal* é a regra da estratégia, a mesma função testada no backtest, e devolve uma opinião: "comprado", ou uma previsão de retorno. O *alvo de posição* converte a opinião em quantidade de ações (Seção 11.9). O *controle de risco* recusa ou reduz alvos que violem limites (Seção 11.10). O *gerador de ordens* compara o alvo com a posição atual e com as ordens já abertas e produz a diferença. A *execução* envia as ordens, acompanha seus estados e trata rejeições e execuções parciais. A *reconciliação* confere a posição que o programa julga ter com a que a corretora informa, e o *registro* grava tudo o que aconteceu.

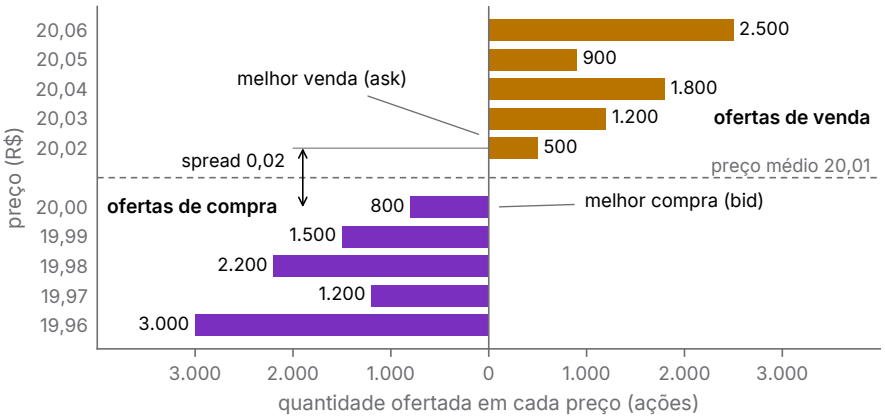


Figura 11.2 Livro de ofertas esquemático. Cada barra é a quantidade somada das ordens que aguardam em um preço. Não há "o preço" da ação: há o melhor preço para vender agora (20,00) e o melhor para comprar agora (20,02).

Trabalhar com *alvo de posição*, e não com "sinal de compra", evita uma família inteira de erros. Se o programa reinicia no meio do dia, ele relê a posição na corretora, recalcula o alvo e envia só a diferença. Um bot que reage a sinais de compra, ao reiniciar, compra de novo.

11.3 Microestrutura mínima: o livro de ofertas

Nas bolsas eletrônicas, a B3 entre elas, o preço se forma em um **livro de ofertas** (*order book*): a lista das ordens de compra e de venda que aguardam contraparte, organizada por preço. A Figura 11.2 mostra um livro esquemático, com números inventados para o exemplo.

A **melhor oferta de compra** (*bid*) é o maior preço que alguém aceita pagar, e a **melhor oferta de venda** (*ask*), o menor preço pelo qual alguém aceita vender. A distância entre as duas é o **spread** (*bid-ask spread*), e o ponto médio entre elas é a referência usual de "preço justo" do instante. A quantidade disponível em cada nível é a **profundidade**. As ordens são atendidas por prioridade de preço e, no mesmo preço, por ordem de chegada; por isso existe uma fila em cada nível (Harris, 2003).

Quem tem pressa paga o spread. Uma compra imediata é executada contra o lado de venda, começando pelo melhor preço e avançando pelos seguintes até completar a quantidade. O Código 11.1 faz essa conta para o livro da figura.

Código 11.1 Uma ordem a mercado percorre o livro (`cap11/l01_livro.py`)

```
def varrer(lado_oposto, qtd):
    """Preço médio de uma ordem a mercado contra o livro."""
    falta, gasto = qtd, 0.0
    for preco, disponivel in lado_oposto:
        q = min(falta, disponivel)
        gasto += q * preco
        falta -= q
        if falta == 0:
            return gasto / qtd
    raise ValueError("livro sem profundidade para a ordem")
```

```
spread: 0.02 = 10.0 pb do preço médio 20.01
compra de  100: preço médio 20.0200,  5.0 pb acima do meio
compra de  500: preço médio 20.0200,  5.0 pb acima do meio
compra de 2000: preço médio 20.0290,  9.5 pb acima do meio
compra de 4000: preço médio 20.0358, 12.9 pb acima do meio
```

A unidade dessas contas é o **ponto-base** (pb), um centésimo de ponto percentual: 10 pb são 0,10%. Enquanto a ordem cabe no primeiro nível, o custo em relação ao preço médio é metade do spread, 5 pb. A compra de 2.000 ações esgota as 500 ofertadas a 20,02, as 1.200 a 20,03 e ainda leva 300 a 20,04; o preço médio sobe para 20,029 e o custo quase dobra. O spread mede o custo de uma ordem pequena e a profundidade determina o de uma ordem grande.

11.3.1 Leilões de abertura e de fechamento

O pregão começa e termina com um **leilão**: durante alguns minutos as ordens se acumulam sem gerar negócios, e ao final as que puderem ser casadas são executadas a um preço único, aquele que maximiza a quantidade negociada. Os preços de abertura e de fechamento das ações líquidas saem desses leilões. Para quem simula com barras diárias, a consequência é que "comprar na abertura" ou "vender no fechamento" só é realizável participando do leilão, com a ordem enviada antes de o preço ser conhecido.

11.4 Ordens: tipos, validade e ciclo de vida

Uma ordem diz o que se quer (comprar ou vender, qual ativo, que quantidade) e em que condições. As condições de preço definem três tipos básicos, descritos em detalhe por Harris (2003).

Uma **ordem a mercado** (*market order*) manda executar já, pelo melhor preço disponível. Ela garante a execução e deixa o preço em aberto: é o caso do Código 11.1.

Uma **ordem limitada** (*limit order*) fixa o pior preço aceitável: uma compra limitada a 19,99 só é executada a 19,99 ou menos. Se não houver contraparte, ela entra no livro e espera. Garante o preço e deixa a execução em aberto. Quem usa ordem limitada deixa de pagar o spread e pode até recebê-lo, mas corre dois riscos. O primeiro é a não execução: o preço sobe sem voltar e a compra nunca acontece. O segundo é a **seleção adversa**: a ordem parada no livro é uma oferta firme que qualquer um pode aceitar, e quem a aceita com mais frequência é quem sabe de algo. Suas compras limitadas tendem a ser executadas justamente quando o preço está caindo e vai continuar a cair.

Uma **ordem stop** (*stop order*) fica dormente até o mercado negociar a um preço de disparo. Um stop de venda a 9,50 é acionado quando sai um negócio a 9,50 ou menos, e nesse momento se transforma em outra ordem. No *stop a mercado* ela vira ordem a mercado; no **stop-limite** vira ordem limitada, e é preciso informar dois preços, o de disparo e o limite. O uso mais comum é limitar a perda de uma posição. Na B3, a oferta stop é do segundo tipo: ao ser disparada, entra no livro como oferta limitada (B3, 2026). A bolsa oferece ainda a oferta stop com proteção, cujo limite é o preço de disparo mais ou menos um valor de proteção. O stop a mercado que aparece em algumas plataformas é montado sobre esses mecanismos, ou pela própria plataforma, com um limite distante do disparo. A Tabela 11.1 compara os tipos.

Tabela 11.1 O que cada tipo de ordem garante

Tipo	Garante	Deixa em aberto	Uso típico
A mercado	execução imediata	o preço: paga o spread e, se for grande, percorre o livro	entrar ou sair com urgência
Limitada	o preço máximo de compra (mínimo de venda)	se e quando executa; seleção adversa	entrar com paciência; realizar lucro em um alvo
Stop a mercado	que a ordem é enviada quando o disparo ocorre	o preço, sobretudo em gap	limitar perda; entrar em rompimento
Stop-limite	disparo e preço-limite	a execução: em queda rápida pode não sair	limitar perda aceitando o risco de ficar na posição

Armadilha **Stop não garante preço**

O preço de disparo de um stop não é o preço de execução. Se a ação fecha a 10,00 e abre no dia seguinte a 8,00 depois de uma notícia, um stop de venda a 9,50 dispara na abertura e é executado perto de 8,00: nenhum negócio aconteceu entre 9,50 e 8,00, e o livro não tinha compradores ali. O stop-limite troca esse risco por outro, o de a ordem ficar parada acima do mercado enquanto o preço despenca. A Seção 11.7 mostra um caso real em que a diferença foi de 14%.

11.4.1 Validade

Toda ordem tem também um prazo. As duas validades que você encontra em praticamente qualquer plataforma são a *do dia*, que expira no encerramento do pregão, e a *até cancelar*. Bolsas e corretoras costumam oferecer ainda a validade até uma data e duas condições de execução imediata: *executa ou cancela* (em inglês *immediate or cancel*, IOC), que executa o que for possível na hora e cancela o saldo, e *tudo ou nada* (*fill or kill*, FOK), que é executada na hora pela quantidade inteira ou cancelada. Em documentação em inglês, não confunda com *all or none*, que exige a quantidade inteira mas não a execução imediata. O conjunto disponível muda conforme a corretora, a plataforma e o ativo; antes de programar, leia a documentação da sua.

11.4.2 Ciclo de vida

Depois de enviada, a ordem passa por estados, e o bot precisa acompanhar cada transição (Figura 11.3). A corretora e a bolsa primeiro validam a ordem: saldo ou margem, limites, preço dentro da faixa permitida. Se passar, ela é aceita e fica aberta. Daí pode ser executada de uma vez, executada aos pedaços, cancelada por você ou expirar.

Dois detalhes desse diagrama causam a maior parte dos erros de quem começa. Enviar não é o mesmo que ter a ordem aceita: até a confirmação chegar, o programa não sabe se ela existe. E pedir o cancelamento não cancela na hora: o pedido viaja até a bolsa, e a ordem pode ser executada enquanto isso. O bot só deve alterar sua contabilidade quando recebe a confirmação de cada evento.

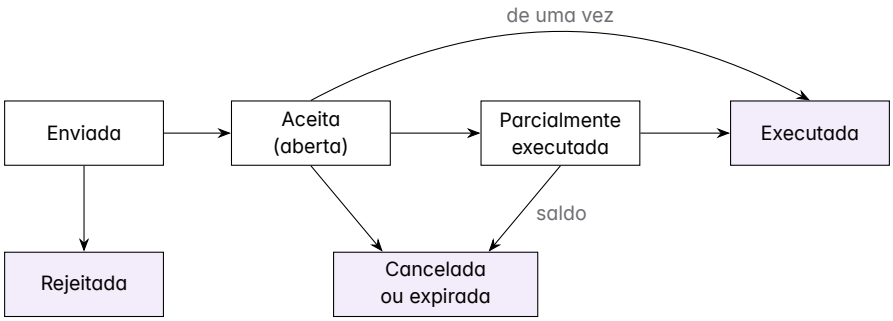


Figura 11.3 Estados de uma ordem. Os estados finais estão sombreados. Uma ordem parcialmente executada continua aberta pelo saldo, que ainda pode ser executado, cancelado ou expirar.

11.5 Custos de transação

Cada operação tem custos de dois tipos. Os **custos explícitos** aparecem na nota de corretagem:

- **Corretagem**, cobrada pela corretora. Pode ser um valor fixo por ordem, um percentual do volume ou zero, conforme a corretora e o tipo de operação.
- **Emolumentos e taxas da bolsa**, cobrados pela B3 pela negociação e pela liquidação, como percentual do valor negociado. Para ações são da ordem de poucos pontos-base, com tabelas distintas para operações comuns e para as iniciadas e encerradas no mesmo dia (*day trade*).
- **Impostos** sobre o ganho líquido, com regras próprias de apuração, de compensação de prejuízos e de retenção na fonte.

Este livro não reproduz percentuais de propósito. As tarifas da B3 e as regras tributárias mudam, e um número impresso envelhece. Consulte a tabela de tarifas vigente no site da B3, a tabela de custos da sua corretora e a regra da Receita Federal em vigor, e coloque os valores atuais no seu modelo.

Os **custos implícitos** não aparecem em nota nenhuma. Estão embutidos no preço de execução e, para quem gira a carteira com frequência, costumam ser a parte maior:

- **Meio spread.** Comprar no *ask* e vender no *bid* custa metade do spread em cada ponta, medido contra o preço médio.
- **Slippage de latência.** O preço anda entre a decisão e a execução.
- **Impacto de mercado.** A própria ordem move o preço contra quem a envia: consome a profundidade e sinaliza ao mercado que há um comprador com pressa.

Um modelo de custo por operação soma as três partes que se conseguem estimar. Para uma ordem de valor Q, o custo por ponta (cada compra e cada venda é uma ponta), como fração do valor negociado, é

$$c(Q) = \underbrace{c_{\text{taxas}}}_{\text{explícito}} + \underbrace{\frac{s}{2}}_{\text{meio spread}} + \underbrace{I(Q)}_{\text{impacto}}, \tag{11.1}$$

em que s é o spread relativo e $I(Q)$ o custo de impacto, a única parcela que depende do tamanho. O custo padrão de 0,10% por ponta dos outros capítulos, 10 pb, é uma hipótese única que resume taxas, metade do spread e slippage de ordens pequenas. Ele não inclui o impacto de mercado de ordens grandes, que entra à parte, pelo modelo a seguir.

11.5.1 A lei da raiz quadrada

Para ordens pequenas em relação ao volume do dia, estudos com dados de execução de diferentes mercados encontram a mesma forma para o deslocamento do preço entre o primeiro e o último negócio de uma ordem executada aos poucos:

$$\Delta(Q) \approx Y\sigma\sqrt{\frac{Q}{V}}, \tag{11.2}$$

em que σ é a volatilidade diária do ativo, V o volume diário negociado e Y uma constante de ordem um. A relação ficou conhecida como **lei da raiz quadrada**. Tóth et al. (2011) a enunciam nessa forma e propõem uma explicação para ela; Grinold e Kahn (2000) chegam à mesma forma por um argumento de risco de estoque de quem provê liquidez. Ela é uma regularidade empírica, e o expoente varia entre os estudos, de 0,4 a 0,7 segundo Tóth et al. (2011); Almgren et al. (2005), com dados da mesa de ações de um grande banco americano, estimam 0,6 para a parte temporária do impacto.

O deslocamento Δ é o impacto ao fim da ordem, e o custo é menor que ele. Quem compra aos poucos paga, em média, menos que o preço final: se o preço sobe com a raiz quadrada da fração já executada, o preço médio de compra fica a dois terços de Δ acima do inicial. O custo de impacto da Equação (11.1) é, então,

$$I(Q) \approx \tfrac{2}{3}\, Y\sigma\sqrt{\frac{Q}{V}}. \tag{11.3}$$

Os estudos diferem em medir o impacto pelo preço final ou pelo custo médio de execução, e por isso Y só é conhecido até um fator dessa ordem. Use essas expressões para obter ordens de grandeza, e não previsões com casas decimais.

> **Intuição**
>
> A forma côncava diz que os primeiros reais de uma ordem são os que mais movem o preço por real negociado. Quadruplicar a ordem dobra o impacto por ação. E a escala natural do tamanho é a fração do volume diário: R$ 1 milhão é uma ordem irrelevante em PETR4 e um evento em uma ação pouco negociada.

O script `cap11/l02_custo_ordem.py` aplica as Equações (11.1) e (11.3) a PETR4 com os dados de 2025. A volatilidade e o volume saem dos arquivos; as taxas de 3 pb e a constante $Y = 1$ são hipóteses de trabalho, e o spread é suposto igual a um centavo, a menor variação de preço das ações na B3. A coluna `fixo` soma taxas e meio spread; as três colunas de custo estão em pontos-base.

```
preço 30.82  volume mediano R$ 983 mi  vol. diária 1.43%
   ordem (R$)  % do volume  fixo  impacto  total  custo (R$)
       10,000      0.001%   4.6      0.3    4.9           5
      100,000      0.010%   4.6      1.0    5.6          56
    1,000,000      0.102%   4.6      3.0    7.7         766
   10,000,000      1.017%   4.6      9.6   14.2      14,224
  100,000,000     10.170%   4.6     30.4   35.0     349,861
```

Para o investidor que negocia R$ 10 mil, o custo é quase todo fixo, cerca de 5 pb, e o impacto não existe na prática. Em R$ 10 milhões, 1% do volume do dia, o impacto já é o dobro da parte fixa, e cada real negociado custa quase o triplo do que custa à pessoa física. PETR4 é uma das ações mais negociadas da bolsa. Em ações menos líquidas a mesma conta, com V dez ou cem vezes menor e spread de vários centavos, dá custos bem maiores para todos. Para

ordens pequenas em ações líquidas, o custo padrão de 10 pb do livro fica, por essa conta, do lado conservador; para ordens grandes, o impacto precisa ser somado a ele.

11.6 Quanto custo uma estratégia aguenta

O custo sai do retorno na proporção em que a estratégia negocia. O giro (*turnover*) dos Capítulos 7 e 8 é o valor negociado em um período, somando compras e vendas, dividido pelo capital. Se cada real negociado custa c, o retorno líquido é

$$R_{\text{líq}} = R_{\text{bruto}} - c \times \text{giro}. \tag{11.4}$$

O custo de equilíbrio (*break-even*) foi definido no Capítulo 8 como o custo por ponta que zera o ganho bruto acima da referência. Com R^f a taxa da referência no período (o CDI),

$$c^* = \frac{R_{\text{bruto}} - R^f}{\text{giro}}. \tag{11.5}$$

É essa a definição usada no livro. Em uma carteira comprada e vendida de soma zero, o retorno bruto já é um ganho acima da referência, e a expressão se reduz a $c^* = R_{\text{bruto}}/\text{giro}$. O número resume a folga da estratégia: se o custo que você estima pagar está acima de c^*, ela não se paga.

11.6.1 Um experimento: o mesmo sinal, dois giros

Para isolar o efeito do giro, o Código 11.2 usa um único sinal em duas frequências de rebalanceamento. O sinal é o momentum 12–1 do Capítulo 8, o retorno de 12 meses sem o mês mais recente, variante que se firmou depois de Jegadeesh e Titman (1993), aplicado a seis ações (PETR4, VALE3, ITUB4, WEGE3, ABEV3 e BBAS3). A carteira compra as que subiram mais que a média e vende a descoberto as que subiram menos, com pesos proporcionais à distância da média, soma de pesos igual a zero e soma dos valores absolutos igual a um. Em uma versão os pesos são levados ao alvo todos os dias; na outra, só no último pregão de cada mês.

Código 11.2 Momentum com rebalanceamento diário e mensal: retorno bruto e giro (`cap11/l03_custos.py`)

```
# momentum 12-1: retorno de 12 meses, pulando o mês mais recente
sinal = (p.shift(21) / p.shift(252) - 1).dropna()
d = sinal.sub(sinal.mean(axis=1), axis=0)
alvo = d.div(d.abs().sum(axis=1), axis=0)   # soma 0, soma |w| = 1


def simular(alvo, R, rebalancear):
    """Retorno bruto e giro por dia, com capital constante 1."""
    v = np.zeros(alvo.shape[1])             # posições em R$
    bruto, giro = [], []
    for dia, w in alvo.iterrows():
        r = R.loc[dia].values
        bruto.append(v @ r)                 # resultado do dia
        v = v * (1 + r)                     # posições derivam
        negocia = rebalancear(dia)
        giro.append(np.abs(w.values - v).sum() if negocia else 0)
        v = w.values if negocia else v      # no fechamento
```

```
    return (pd.Series(bruto, alvo.index),
            pd.Series(giro, alvo.index))
```

```
2011-01-11 a 2025-12-30
rebalanceamento         diário            mensal
giro anual (x capital)    24.9               5.3
custo por ponta    retorno  Sharpe   retorno  Sharpe
        0 pb         3.26%    0.19     2.95%    0.17
        5 pb         2.01%    0.12     2.69%    0.16
       10 pb         0.77%    0.05     2.42%    0.14
       20 pb        -1.72%   -0.10     1.89%    0.11
custo de equilíbrio    13.1 pb           55.6 pb
```

A função `simular` recebe os pesos-alvo, os retornos e uma função que diz em que dias se negocia. Ela mantém o capital constante em 1 e acompanha o valor de cada posição: entre dois rebalanceamentos as posições derivam com os retornos, e o giro de um rebalanceamento é a soma dos valores absolutos das diferenças entre o alvo e o que a carteira tinha. A execução segue a convenção padrão do livro, no fechamento do sinal: os pesos calculados com o fechamento de um dia são assumidos nesse fechamento e rendem a partir do dia seguinte. Como a carteira tem soma zero, o que ela aplica nas compras vem das vendas, e seu retorno já é um retorno em excesso. Por isso o Sharpe aqui é a média sobre o desvio-padrão do próprio retorno, sem descontar o CDI, anualizado pela convenção de 252 dias (Seção 1.9); a conta ignora o aluguel das ações vendidas a descoberto.

Sem custos, as duas versões rendem quase o mesmo, perto de 3% ao ano. A diária negocia 24,9 vezes o capital por ano, quase cinco vezes o giro da mensal. Com o custo padrão de 10 pb por ponta, a diária perde cerca de três quartos do retorno e a mensal perde menos de um quinto. O custo de equilíbrio é de 13,1 pb em uma e 55,6 pb na outra, que é o que a Equação (11.5) prevê: $3{,}26\%/24{,}9 = 13{,}1$ pb. A Figura 11.4 mostra as duas curvas.

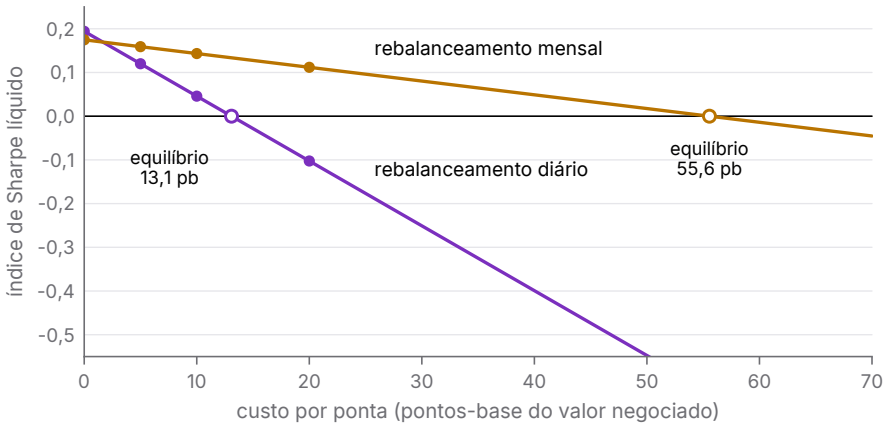


Figura 11.4 Sharpe líquido do momentum de seis ações em função do custo por ponta. As duas versões partem de valores próximos; a inclinação de cada curva é aproximadamente proporcional ao giro. Os círculos vazados marcam o custo de equilíbrio.

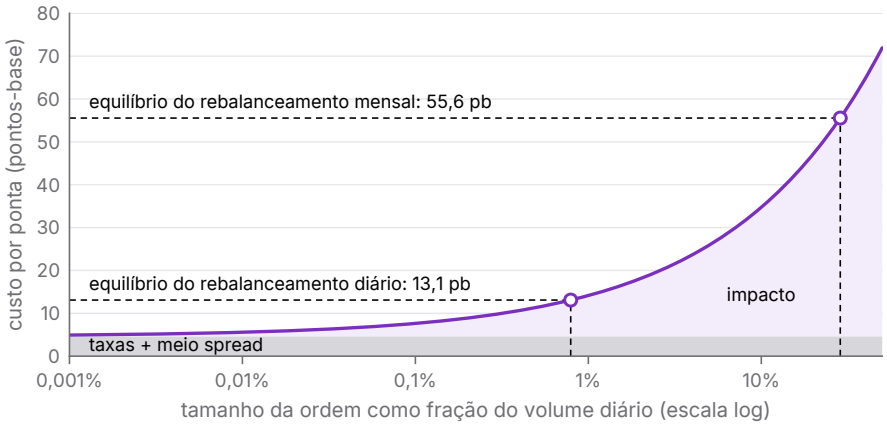


Figura 11.5 Custo por ponta em PETR4 em função do tamanho da ordem, sob as hipóteses da Seção 11.5. A versão diária do momentum deixa de se pagar com ordens acima de 0,8% do volume do dia; a mensal suporta ordens dezenas de vezes maiores.

Armadilha **O Sharpe bruto deste exemplo não prova nada**

Um Sharpe de 0,19 em quase quinze anos tem erro-padrão próximo de 0,26 (Lo, 2002): não se distingue de zero. Seis ações são muito pouco para uma estratégia de momentum. O experimento serve para mostrar a aritmética do custo, que vale para qualquer estratégia, e não para sugerir que esta dê dinheiro. A lição prática é outra: uma vantagem pequena só sobrevive com giro baixo, e antes de refinar o sinal convém perguntar quantas vezes por ano ele manda negociar.

11.6.2 Capacidade

A Equação (11.1) torna o custo crescente com o tamanho, e daí sai um teto. A **capacidade** de uma estratégia é o volume de capital que ela comporta antes de o impacto das próprias ordens consumir a vantagem. A Figura 11.5 junta as duas contas: a curva de custo de PETR4 da seção anterior e os custos de equilíbrio do experimento.

As últimas linhas da saída do mesmo script fazem a conta inversa: dado o custo de equilíbrio de cada versão, qual é a maior ordem que ainda se paga em um papel com a liquidez de PETR4.

```
versão  equilíbrio  ordem máxima   % do volume  capital
diário     13.1 pb  R$    7.8 mi        0.79%  R$ 472 mi
mensal     55.6 pb  R$  281.4 mi       28.62%  R$ 3,946 mi
```

A versão diária deixa de se pagar com ordens acima de R$ 8 milhões em cada papel, 0,8% do volume do dia. Para a mensal a conta dá R$ 280 milhões, 29% do volume, um tamanho em que a lei da raiz quadrada, medida em ordens pequenas, já não merece confiança; leia o número como “dezenas de vezes maior”. A última coluna converte ordem em capital, supondo seis papéis igualmente líquidos: a versão diária negocia perto de 10% do capital a cada dia, e a mensal, 43% a cada mês, de modo que a razão de 36 vezes entre os tamanhos de ordem vira cerca de oito vezes em capital. São tetos otimistas, porque quatro das seis ações negociam menos que PETR4 e no teto o lucro já é zero: o ponto de lucro máximo em reais chega antes. A ordem de grandeza não muda com ajustes razoáveis das hipóteses: estratégias de giro alto são negócios pequenos.

11.6.3 Executar uma ordem grande

Quem precisa negociar uma fração relevante do volume do dia não envia uma ordem a mercado única, que percorreria o livro como no Código 11.1. Divide a ordem em fatias e as distribui no tempo. Os dois roteiros mais comuns têm nome. O **TWAP** (*time-weighted average price*) usa fatias iguais em intervalos iguais. O **VWAP** (*volume-weighted average price*) dimensiona as fatias na proporção do volume esperado em cada intervalo do dia, para que a ordem seja sempre a mesma fração do mercado; como o volume costuma se concentrar perto da abertura e do fechamento, um VWAP negocia mais nessas horas. A lei da raiz quadrada já descreve ordens executadas assim, aos poucos; fatiar não escapa dela, apenas evita o custo ainda maior de esvaziar o livro de uma vez.

Resta decidir em quanto tempo executar. Almgren e Chriss (2001) formularam o dilema: executar depressa concentra as ordens e aumenta o impacto; executar devagar reduz o impacto, mas deixa a parte ainda não executada exposta à variação do preço durante mais tempo. Eles minimizam o custo esperado mais um múltiplo da variância do custo, com um parâmetro de aversão a risco que regula a troca. Sem aversão a risco, a solução é o ritmo constante do TWAP, que minimiza o impacto. Quanto maior a aversão a risco ou a volatilidade do ativo, mais a execução ótima se concentra no início do prazo, pagando impacto para reduzir a incerteza.

11.7 Uma corretora simulada

Antes de ligar um bot a uma corretora real, ele deve rodar contra uma corretora de mentira que responda como a real: aceite ordens, devolva estados, cobre custos e mantenha caixa e posição. A classe `CorretoraPapel` do Código 11.3 faz isso sobre barras diárias de abertura, máxima, mínima e fechamento (OHLC). Como uma barra diária esconde o que aconteceu dentro do dia, as regras de preenchimento precisam ser explícitas, e na dúvida escolhemos a hipótese pior para quem opera:

- o bot manda processar a barra do dia antes de decidir, de modo que uma ordem só é executada em barras posteriores àquela em que foi criada;
- ordem **a mercado**: executa na abertura da barra seguinte, com um slippage contra você;
- ordem **limitada**: executa ao preço-limite apenas se a barra *atravessar* o limite. Tocar não basta, nem na abertura, porque no preço exato haveria uma fila na sua frente. Se a barra abrir além do limite a seu favor, a ordem é tratada como ordem a mercado na abertura, com slippage, e o limite continua sendo o pior preço possível;
- ordem **stop**: é um stop a mercado. Dispara se a barra alcançar o gatilho e é executada ao gatilho com slippage; se a barra abrir além do gatilho, na abertura com slippage;
- havendo várias ordens abertas, executam primeiro as que a abertura já decide, porque a abertura é o primeiro negócio do dia. Para o que acontece dentro do dia, cuja sequência a barra não registra, os stops vêm antes das limitadas;
- barras sem volume ou sem preço não executam nada;
- toda execução paga uma comissão proporcional ao valor, e a corretora rejeita, no envio, ordens malformadas e, no momento da execução, compras sem caixa e vendas sem posição, anotando o motivo na ordem. Uma corretora real faz a conferência de saldo já no envio;

Código 11.3 Corretora simulada sobre barras diárias (`cap11/papel.py`)

```
class CorretoraPapel:
    def __init__(self, caixa, comissao=0.0003, slippage=0.0005):
        self.caixa, self.posicao = caixa, 0
        self.comissao, self.slippage = comissao, slippage
        self.ordens, self.extrato = [], []

    def enviar(self, lado, qtd, tipo="mercado", preco=None):
        o = Ordem(len(self.ordens) + 1, lado, qtd, tipo, preco)
        inteiro = (isinstance(qtd, Integral)
                   and not isinstance(qtd, bool))
        if not (inteiro and qtd > 0):
            o.motivo = "quantidade inválida"
        elif (lado not in ("compra", "venda")
              or tipo not in ("mercado", "limitada", "stop")):
            o.motivo = "lado ou tipo desconhecido"
        elif tipo != "mercado" and not (
                preco and isfinite(preco) and preco > 0):
            o.motivo = "preço inválido"
        else:
            o.qtd = int(qtd)           # aceita inteiros do numpy
        if o.motivo:
            o.estado = "rejeitada"
        self.ordens.append(o)
        return o

    def cancelar(self, o):
        if o.estado == "aceita":
            o.estado = "cancelada"

    def abertas(self):
        return [o for o in self.ordens if o.estado == "aceita"]

    def patrimonio(self, preco):
        return self.caixa + self.posicao * preco

    def _preco(self, o, b):
        """Preço de execução na barra b, ou None."""
        s = 1 if o.lado == "compra" else -1
        pior = 1 + s * self.slippage       # compra paga mais
        if o.tipo == "mercado":
            return b.Open * pior
        if o.tipo == "limitada":
            if s * (b.Open - o.preco) < 0:     # abriu melhor:
                p = b.Open * pior              # paga como mercado
                return s * min(s * p, s * o.preco)  # teto: limite
            extremo = b.Low if s == 1 else b.High
            if s * (extremo - o.preco) < 0:    # atravessou
                return o.preco
            return None
        # stop: a compra dispara na alta; a venda, na queda
        if s * (b.Open - o.preco) >= 0:        # abriu além (gap)
            return b.Open * pior
        extremo = b.High if s == 1 else b.Low
        if s * (extremo - o.preco) >= 0:       # tocou o gatilho
            return o.preco * pior
```

```
        return None

    def _na_abertura(self, o, b):
        """A abertura da barra b já decide a execução de o?"""
        if o.tipo == "mercado":
            return True
        d = (1 if o.lado == "compra" else -1) * (b.Open - o.preco)
        return d < 0 if o.tipo == "limitada" else d >= 0

    def processar(self, data, b):
        """Tenta executar na barra b as ordens já aceitas."""
        if not (b.Volume > 0 and b.Open > 0):  # barra inválida
            return

        def vez(o):    # 1° a abertura; no dia, stops antes
            return (not self._na_abertura(o, b),
                    o.tipo != "stop", o.id)

        for o in sorted(self.abertas(), key=vez):
            p = self._preco(o, b)
            if p is None:
                continue
            s = 1 if o.lado == "compra" else -1
            taxa = self.comissao * p * o.qtd
            sem_caixa = s == 1 and p * o.qtd + taxa > self.caixa
            sem_papel = s == -1 and o.qtd > self.posicao
            if sem_caixa or sem_papel:
                o.estado = "rejeitada"
                o.motivo = "sem papel"
                if sem_caixa:
                    o.motivo = "sem caixa"
                continue
            self.caixa -= s * p * o.qtd + taxa
            self.posicao += s * o.qtd
            o.estado, o.preco_exec = "executada", p
            self.extrato.append(
                (data, o.id, o.lado, o.tipo, o.qtd, p, taxa,
                 self.posicao, self.caixa))
```

A classe `Ordem`, definida no começo do arquivo, é um registro com identificador, lado, quantidade, tipo, preço e estado. O método `_preco` concentra as regras: a variável `s` vale $+1$ para compra e -1 para venda, o que permite escrever cada regra uma vez para os dois lados. Executar na abertura supõe participar do leilão de abertura, e o slippage faz o papel do custo implícito da Seção 11.5; com ele, o preço de execução pode cair fora do intervalo da barra, por uma distância que é no máximo o slippage configurado. As simplificações mais fortes são duas. Não há execução parcial: cada ordem é executada inteira ou não é. E as ordens não se conhecem: se o stop de uma posição é executado, o alvo continua aberto até que o bot o cancele.

No repositório

O arquivo `livro/scripts/cap11/teste_papel.py` tem 19 testes, que cobrem as regras acima: limitada que apenas toca o preço, limitada que não sai mais barata que a ordem a mercado, stop com abertura em gap, stop e alvo na mesma barra com e sem gap, rejeições com motivo, barra sem volume. O arquivo `conf_numeros.py` imprime os números citados no texto que não aparecem nas saídas. Rode-os antes de alterar a classe: uma corretora simulada com erro de contabilidade produz backtests errados sem dar aviso.

11.7.1 Um bot completo

O Código 11.4 liga um bot à corretora simulada, usando PETR4 de 2019 a 2024. A regra é simples de propósito e seus parâmetros foram fixados sem busca. Quando o fechamento está acima da média de 100 dias, o bot deixa uma compra limitada 2% abaixo do fechamento, renovada todo dia. Executada a compra, envia duas ordens de saída: um stop 5% abaixo do preço de entrada e uma venda limitada 8% acima. Se nenhuma for executada em dez pregões, cancela as duas e sai a mercado. A quantidade é escolhida para que a perda planejada no stop seja 1% do patrimônio, arredondada para lotes de 100 ações.

Código 11.4 Bot de exemplo sobre a corretora simulada (`cap11/l04_bot.py`)

```
import pandas as pd
from papel import CorretoraPapel

b = pd.read_csv("../../dados/petr4_ohlcv.csv", index_col=0,
                parse_dates=True).loc["2018":"2024"]
media = b.Close.rolling(100).mean()

RISCO, STOP, ALVO, RECUO, PRAZO = 0.01, 0.05, 0.08, 0.02, 10
cor = CorretoraPapel(caixa=100_000)
curva, dias = {}, 0

for dia, barra in b.loc["2019":].iterrows():
    cor.processar(dia, barra)          # 1. execuções da barra
    abertas, c = cor.abertas(), barra.Close
    if cor.posicao == 0:               # 2. decisão no fechamento
        for o in abertas:              # entrada ou perna órfã
            cor.cancelar(o)
        if c > media[dia]:             # só em tendência de alta
            limite = round(c * (1 - RECUO), 2)
            risco = RISCO * cor.patrimonio(c)
            qtd = int(risco / (STOP * limite) / 100) * 100
            cor.enviar("compra", qtd, "limitada", limite)
        dias = 0
    elif not abertas:                  # comprou: protege
        pe = cor.extrato[-1][5]        # preço de entrada
        q = cor.posicao
        cor.enviar("venda", q, "stop", round(pe * (1 - STOP), 2))
        alvo = round(pe * (1 + ALVO), 2)
        cor.enviar("venda", q, "limitada", alvo)
    else:
        dias += 1
        if dias == PRAZO:              # saída por tempo
            for o in abertas:
                cor.cancelar(o)
```

```
            cor.enviar("venda", cor.posicao)
    curva[dia] = cor.patrimonio(c)     # 3. registro

curva = pd.Series(curva)
```

```
data       ordem lado   tipo      qtd   preço  taxa  pos.     caixa
2019-01-22 14    compra limitada 3000  6.5400  5.89  3000  80374.11
2019-02-07 17    venda  mercado  3000  6.6868  6.02     0 100428.46
2019-02-11 19    compra limitada 3100  6.4400  5.99  3100  80458.47
2019-02-14 21    venda  limitada 3100  6.9600  6.47     0 102028.00
2019-03-08 35    compra limitada 2900  6.8600  5.97  2900  82128.03
2019-03-15 37    venda  limitada 2900  7.4100  6.45     0 103610.58
2019-03-21 41    compra limitada 2700  7.4200  6.01  2700  83570.57
2019-03-25 42    venda  stop     2700  7.0081  5.68     0 102486.68
cancelada    529
executada    191
aceita         1
compra limitada: 96 executadas de 495 enviadas
venda stop: 35 executadas de 96 enviadas
venda limitada: 27 executadas de 96 enviadas
venda mercado: 33 executadas de 34 enviadas
pior stop: 2021-02-22, gatilho 7.19, executado a 6.1534
patrimônio final: 111,600.88   taxas pagas: 1,184.50
drawdown máximo: -5.6%   comprar e manter: +386%
```

O laço tem a forma da Figura 11.1: primeiro a corretora processa a barra do dia e executa o que estava pendente; depois o bot olha o fechamento e decide; por fim registra o patrimônio. As ordens criadas no passo 2 só podem ser executadas a partir da barra seguinte, e é essa ordem das chamadas que impede o bot de usar informação do futuro. Em relação à convenção padrão do livro, a execução no fechamento do sinal, o bot é mais conservador: decide com o fechamento do dia t e só negocia a partir da abertura de $t + 1$, como faz o backtrader no Capítulo 7. Fica entre a convenção padrão e a de um pregão inteiro de atraso usada no Capítulo 12.

Os preços do arquivo são ajustados por proventos (Capítulo 1); por isso PETR4 aparece a R$ 6,54 em 2019. Preços e quantidades do extrato não são os de tela da época.

No extrato, a primeira compra foi encerrada por tempo com uma ordem a mercado, a segunda e a terceira atingiram o alvo e a quarta foi estopada. Das 495 compras limitadas enviadas, 96 foram executadas, 11 delas na abertura, por um preço melhor que o limite; as outras foram canceladas no fechamento seguinte e, se a tendência continuava, trocadas por uma nova ordem ao novo preço, o que em uma corretora real consome limite de mensagens.

A linha do pior stop merece atenção. Em 19 de fevereiro de 2021 o bot comprou a 7,57 e posicionou o stop a 7,19. No pregão seguinte a ação abriu 16,6% abaixo do fechamento anterior, e o stop foi executado a 6,15, 14,4% abaixo do gatilho. A operação perdeu 18,7% em vez dos 5% planejados, e o capital caiu 3,1% em um dia, contra o risco de 1% que o dimensionamento supunha. A Figura 11.6 marca esse dia.

O resultado em si é modesto: 11,6% em seis anos, com o bot posicionado em 44% dos dias e cerca de um quinto do patrimônio em cada operação, enquanto comprar e manter rendeu 386% no mesmo período de alta forte. Como estratégia o bot não se recomenda. Ele serve de encanamento completo, que funciona igual para qualquer regra colocada no passo 2.

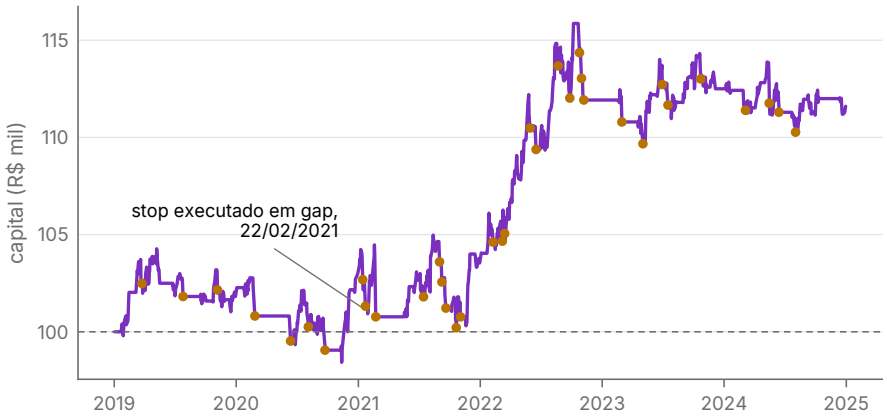


Figura 11.6 Patrimônio do bot de exemplo, de 2019 a 2024. Os pontos marcam as saídas por stop, dimensionadas para custar 1% do patrimônio; a de 22 de fevereiro de 2021 custou três vezes isso.

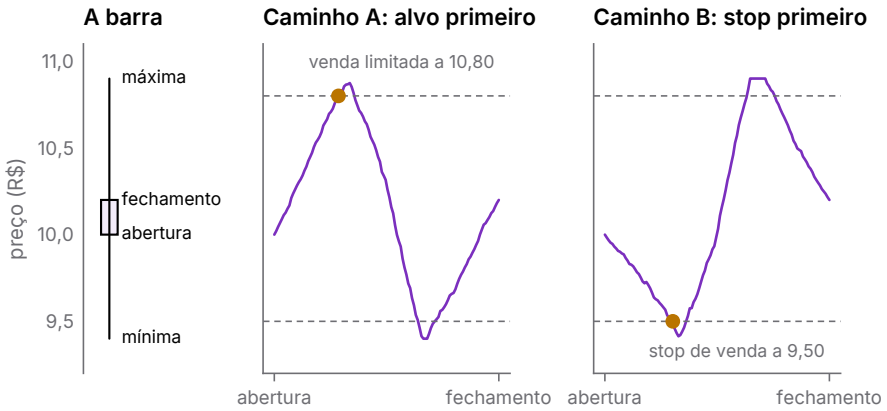


Figura 11.7 Uma barra, dois dias possíveis. A máxima passa do alvo e a mínima passa do stop. No caminho A a venda limitada é executada primeiro, com lucro; no caminho B o stop dispara antes, com prejuízo. O arquivo de barras é o mesmo nos dois casos.

11.7.2 O que uma barra não conta

A barra diária registra quatro preços e nada sobre a ordem em que ocorreram. Para uma posição com stop e alvo, isso às vezes decide o resultado (Figura 11.7).

Quando a abertura já decide, não há ambiguidade: se a barra abre abaixo do stop, o stop sai na abertura; se abre acima do alvo, sai o alvo. A `CorretoraPapel` executa primeiro esses casos. Para o que acontece dentro do dia ela adota a hipótese pior: quando stop e alvo poderiam ser executados na mesma barra, vale o stop, e a venda limitada é rejeitada por falta de posição. Na amostra do exemplo esse caso não ocorreu, mas outra limitação da mesma natureza sim: no dia da compra a posição ainda não tem stop, nem alvo, e em 2 das 96 entradas a mínima daquele mesmo dia ficou abaixo do nível em que o stop seria colocado (a máxima não passou do alvo em nenhuma). Quem precisa de respostas melhores usa barras de minutos ou os negócios um a um, o que muda o volume de dados, mas não a estrutura do simulador.

11.7.3 Paper trading

O passo seguinte é o **paper trading**: o bot roda em tempo real, ligado aos dados ao vivo e a uma conta de simulação da corretora, sem dinheiro. Ele testa o que o simulador de barras não alcança: a conexão, o formato dos dados

em tempo real, o tratamento de rejeições, o comportamento do programa na abertura e no fechamento, a rotina de ligar e desligar. Algumas semanas de paper trading sem intervenção manual costumam revelar a maior parte dos erros de programação.

> **Armadilha** **Paper trading não mede custo**
>
> A conta de simulação executa suas ordens contra o livro real sem alterá-lo. Suas ordens não consomem profundidade, não movem o preço e não ocupam lugar na fila; uma ordem limitada costuma ser dada como executada assim que o mercado toca o preço. A documentação da Alpaca, por exemplo, avisa que seu ambiente de simulação não reproduz impacto de mercado, slippage de latência nem posição na fila. O paper trading valida o programa. O custo só aparece com dinheiro real, e por isso se começa com posições pequenas.

11.8 Conectar-se a uma corretora

As interfaces de programação (API) das corretoras variam nos detalhes e se parecem na estrutura:

- **Autenticação por chave.** A corretora emite uma chave e um segredo, que identificam sua conta em cada requisição. Costuma haver chaves separadas para o ambiente de simulação e para o real.
- **REST para ordens e consultas.** Enviar, cancelar e consultar ordens, posição e saldo são requisições HTTP com resposta em JSON.
- **Websocket para tempo real.** Cotações, negócios e mudanças de estado das suas ordens chegam por uma conexão que fica aberta, sem que você precise perguntar.
- **Limites de requisição.** Há um teto de chamadas por minuto; quem o ultrapassa é bloqueado por um tempo. O bot precisa contar suas chamadas.
- **Identificador de ordem do cliente.** Você atribui a cada ordem um identificador único, e a corretora recusa uma segunda ordem com o mesmo identificador. É o que torna o envio **idempotente**: se a resposta se perder e você reenviar, não compra duas vezes.

O bot não deve conhecer nenhum desses detalhes. Ele conversa com uma interface abstrata, e cada corretora ganha um adaptador que traduz a interface para a API correspondente. O Código 11.5 é um esboço que não foi executado: o endereço e os campos são fictícios, e uma versão real seguiria a documentação da corretora escolhida.

Código 11.5 Esboço não executado de uma interface de corretora e de um adaptador REST

```
import os
from abc import ABC, abstractmethod

import requests


class Corretora(ABC):
    """Tudo o que o bot precisa saber pedir."""

    @abstractmethod
```

```
    def enviar(self, id_cliente, lado, qtd, tipo, preco=None): ...

    @abstractmethod
    def cancelar(self, id_cliente): ...

    @abstractmethod
    def estado(self, id_cliente): ...

    @abstractmethod
    def posicao(self, ativo): ...


class CorretoraREST(Corretora):
    URL = "https://api.corretora.exemplo/v1"     # fictício

    def __init__(self, ativo):
        chave = os.environ["CORRETORA_CHAVE"]    # nunca no código
        self.ativo = ativo
        self.http = requests.Session()
        self.http.headers["Authorization"] = f"Bearer {chave}"

    def enviar(self, id_cliente, lado, qtd, tipo, preco=None):
        corpo = {"id_cliente": id_cliente, "ativo": self.ativo,
                 "lado": lado, "qtd": qtd, "tipo": tipo,
                 "preco": preco}
        r = self.http.post(f"{self.URL}/ordens", json=corpo,
                           timeout=5)
        r.raise_for_status()                     # erro vira exceção
        return r.json()["estado"]
    # cancelar, estado e posicao seguem o mesmo padrão
```

O identificador `id_cliente` é criado pelo bot e gravado em disco *antes* da primeira tentativa de envio. Se a requisição estourar o tempo, o bot reenvia com o mesmo identificador; gerar um novo a cada tentativa anularia a proteção. A `CorretoraPapel` pode ser embrulhada na mesma interface, e então o mesmo bot roda no simulador, na conta de simulação e na conta real trocando uma linha.

Armadilha **Chave de API no repositório**

Uma chave de API com permissão de negociação equivale à senha da conta. Ela não entra no código nem no Git: fica em uma variável de ambiente ou em um arquivo listado no `.gitignore`, como no Código 11.5. Se uma chave for publicada por engano, apagar o arquivo não resolve, porque o histórico a guarda; revogue-a na corretora. Quando a corretora permitir, crie a chave sem permissão de saque e restrita ao seu endereço IP.

11.8.1 Caminhos usados por pessoas físicas

Três caminhos aparecem com frequência, e os três têm documentação pública. Para o mercado brasileiro, algumas corretoras oferecem a plataforma MetaTrader 5, e a MetaQuotes, sua desenvolvedora, publica o pacote Python `MetaTrader5`, que se comunica com o terminal instalado no computador (o pacote é distribuído para Windows) e expõe funções para obter cotações, con-

sultar posições e enviar ordens. Para ações americanas, a Alpaca oferece uma API REST com ambiente de simulação gratuito. Para criptoativos, a biblioteca de código aberto `ccxt` dá uma interface única para as APIs de mais de cem corretoras. Nomes de funções e parâmetros mudam entre versões: trabalhe com a documentação atual.

11.9 Dimensionamento de posições

Falta a caixa *Alvo de posição*: dado um sinal, quanto comprar. Três regras cobrem a maior parte dos usos.

A **fração fixa** aplica em cada operação uma parcela constante do patrimônio, ou, na versão usada pelo bot do Código 11.4, arrisca uma parcela constante: a quantidade é o risco aceito dividido pela perda por ação até o stop. É simples e faz a posição encolher depois de perdas. Como se viu, o risco efetivo pode superar o planejado quando o stop é atravessado.

O alvo de volatilidade da Equação (8.4), $w_t = \min(\sigma^*/\hat{\sigma}_t,\ w_{\max})$, ajusta a exposição para que o risco da posição fique constante. Com alvo de 10% e um ativo que oscila 40% ao ano, aplica-se um quarto do capital; se a volatilidade cai para 20%, metade. A estimativa $\hat{\sigma}_t$ vem dos modelos dos Capítulos 4 e 5.

O **critério de Kelly** responde a outra pergunta: qual fração maximiza o crescimento do capital no longo prazo? Kelly (1956) resolveu o caso de uma aposta repetida. Se a probabilidade de ganhar é p, a de perder é $q = 1 - p$ e cada real apostado rende b reais em caso de ganho, a fração ótima é

$$f^* = p - \frac{q}{b}. \tag{11.6}$$

Ela sai de maximizar o valor esperado do logaritmo da riqueza após uma aposta, $g(f) = p\ln(1 + bf) + q\ln(1 - f)$: derivando e igualando a zero chega-se à Equação (11.6). Com $p = 0{,}55$ e $b = 1$, $f^* = 0{,}55 - 0{,}45 = 0{,}10$.

Para um ativo com retorno em excesso esperado μ e volatilidade σ, a taxa de crescimento com alavancagem f é aproximadamente $g(f) = f\mu - f^2\sigma^2/2$ acima da taxa livre de risco, e o máximo está em

$$f^* = \frac{\mu}{\sigma^2}. \tag{11.7}$$

Com $\mu = 6\%$ e $\sigma = 20\%$, $f^* = 0{,}06/0{,}04 = 1{,}5$: o critério manda alavancar uma vez e meia.

O script `cap11/l05_kelly.py` calcula $g(f)$ para a aposta binária do exemplo e simula 10 mil sequências de 500 apostas com três frações. Para cada uma imprime a taxa g, a riqueza final mediana, partindo de 1, e a mediana da maior queda sofrida no caminho:

```
f* = 0.10
f = 0.05 (0.5 x Kelly): g = +0.00375  mediana final  6.53  queda 53%
f = 0.10 (1.0 x Kelly): g = +0.00501  mediana final 12.23  queda 83%
f = 0.20 (2.0 x Kelly): g = -0.00014  mediana final  0.93  queda 99%
```

A Figura 11.8 mostra a curva $g(f)$.

Na prática quase ninguém aposta o Kelly cheio, por duas razões que a saída deixa ver. A primeira é o caminho: no Kelly cheio, metade das trajetórias passa por uma queda de 83% ou mais; com metade da fração esse número é 53% e o crescimento esperado fica em três quartos do máximo (0,00375 contra 0,00501). A segunda é o erro de estimação. As Equações (11.6) e (11.7) supõem p e μ conhecidos, e eles são estimados com muito ruído. Quem superestima a

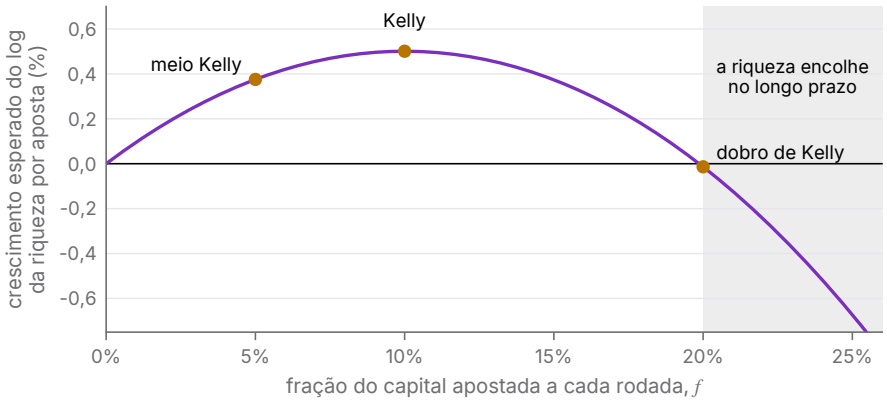


Figura 11.8 Crescimento esperado do logaritmo da riqueza em uma aposta com $p = 0{,}55$ e $b = 1$. A curva é chata perto do máximo e assimétrica nas consequências: apostar metade de Kelly custa um quarto do crescimento; apostar o dobro o elimina.

vantagem em duas vezes aposta o dobro de Kelly sem saber, e a última linha da saída mostra o destino: crescimento esperado nulo. Por isso se usa uma **fração de Kelly**, tipicamente a metade ou menos; Chan (2021) discute o critério e a prática de operar com metade do valor de Kelly.

11.10 Risco operacional

Um programa que envia ordens sozinho erra na velocidade em que opera. No fim de julho de 2012, a corretora americana Knight Capital instalou de forma incorreta uma versão nova do código de seu roteador de ordens, e uma função antiga e defeituosa, que nunca tinha sido removida, voltou a ser acionada. Segundo a SEC (2013), nos primeiros 45 minutos do pregão de 1º de agosto o sistema enviou mais de 4 milhões de ordens ao tentar executar 212 ordens de clientes, negociou mais de 397 milhões de ações e deixou a empresa com um prejuízo superior a 460 milhões de dólares. O órgão regulador concluiu que faltavam controles capazes de barrar ordens erradas antes de chegarem ao mercado.

Um bot pessoal precisa das mesmas travas, em escala menor. Elas ficam na caixa *Controle de risco* da Figura 11.1 e são independentes do sinal.

- **Verificação pré-ordem.** Antes de qualquer envio: a quantidade está abaixo do máximo por ordem? O preço está a poucos por cento do último negócio? A posição resultante respeita o limite? O número de ordens no último minuto é plausível? Uma resposta negativa bloqueia a ordem e gera um alerta.
- **Limites de posição e de perda diária.** Um teto de exposição por ativo e total, e uma perda no dia a partir da qual o bot para de abrir posições.
- **Botão de emergência** (*kill switch*). Um comando único que cancela todas as ordens abertas e impede novos envios, acionável por você e pelo próprio programa. Zerar as posições é uma decisão separada, que pode ou não ser automática.
- **Reconciliação.** A intervalos regulares, e sempre ao iniciar, o bot compara posição, caixa e ordens abertas com o que a corretora informa. Havendo diferença, vale a corretora, e o bot para até alguém entender a causa.
- **Registro** (*logging*). Cada dado recebido, decisão, ordem e resposta vai para um arquivo com horário. Sem registro não há como reconstituir o que

aconteceu.

A queda de conexão merece um plano próprio. As ordens que já estão na bolsa continuam vivas enquanto o programa está fora do ar: um stop registrado na corretora protege a posição mesmo com o computador desligado, e um stop que existe só na memória do bot não protege nada. Ao reconectar, o programa não deve supor que nada mudou. Ele consulta ordens e posição, reconcilia e só então volta a decidir.

11.10.1 Antes de ligar

Operar com dinheiro real coloca você sob as regras da corretora e da Comissão de Valores Mobiliários (CVM). Leia o contrato e a política da corretora para ordens automatizadas, verifique se o uso da API é permitido para o seu tipo de conta e informe-se sobre a tributação das suas operações. Este livro é material de ensino: nada aqui é recomendação de investimento, e as estratégias mostradas servem para ilustrar métodos.

A lista abaixo reúne, na ordem em que devem ser cumpridas, as condições para um bot sair da simulação.

1. O backtest passou pelos testes dos Capítulos 7 e 8, incluindo validação fora da amostra.
2. O custo estimado por operação, com dados atuais de taxas e spread, está bem abaixo do custo de equilíbrio da estratégia.
3. O bot rodou na corretora simulada e, depois, em paper trading por algumas semanas sem intervenção, com ordens iguais às que o backtest geraria nos mesmos dias.
4. Limites de ordem, de posição e de perda diária estão configurados, e o botão de emergência foi testado.
5. A reconciliação com a corretora roda ao iniciar e durante o dia; quedas de conexão e reinícios foram ensaiados.
6. As chaves estão fora do código, sem permissão de saque.
7. O capital inicial é pequeno, e há um critério escrito para aumentá-lo e outro para desligar o bot.

No projeto final do Capítulo 12, os resultados devem ser apresentados líquidos de custos, com as ferramentas deste capítulo.

O que levar deste capítulo

- Um bot é um laço de eventos com responsabilidades separadas. Trabalhar com alvo de posição e reconciliar com a corretora permite reiniciar sem duplicar ordens.
- A ordem a mercado garante execução e não preço; a limitada garante preço e não execução, e sofre seleção adversa; o stop dispara uma ordem e não garante preço, sobretudo em gap.
- O custo por operação soma taxas, meio spread e impacto, e o impacto cresce aproximadamente com a raiz quadrada da fração do volume diário negociada.
- O retorno líquido é o bruto menos custo vezes giro. O custo de equilíbrio, ganho bruto acima da referência dividido pelo giro, diz se a estratégia tem folga; giro alto limita a capacidade.

- Simular com barras exige regras de preenchimento explícitas e conservadoras. O paper trading valida o programa, não os custos.
- Limites, verificação pré-ordem, botão de emergência, reconciliação e registro protegem contra o erro do próprio programa. Dimensione com alvo de volatilidade ou com uma fração do valor de Kelly.

Exercícios

11.1 [básico] No livro da Figura 11.2, calcule à mão o preço médio de uma venda a mercado de 3.000 ações e o custo em pontos-base em relação ao preço médio de 20,01. Confira com a função `varrer`.

11.2 [básico] Uma estratégia rende, antes de custos, 8% ao ano acima do CDI e gira o capital 40 vezes por ano, somando compras e vendas. Qual é seu custo de equilíbrio por ponta? Com custo de 12 pb, quanto sobra do ganho acima do CDI?

11.3 [básico] Uma aposta rende R$ 2 de ganho por real apostado com probabilidade 0,4 e perde o valor apostado com probabilidade 0,6. Calcule a fração de Kelly e a taxa de crescimento g em f^*, em $f^*/2$ e em $2f^*$.

11.4 [intermediário] Acrescente à `CorretoraPapel` a validade "do dia": uma ordem não executada na primeira barra em que poderia ser é marcada como expirada. Escreva o teste antes de escrever o código.

11.5 [intermediário] No Código 11.2, acrescente uma versão com rebalanceamento semanal e uma versão diária que só negocia quando a diferença entre o peso atual e o alvo passa de um limiar (uma *banda de não negociação*). Como ficam o giro e o custo de equilíbrio?

11.6 [intermediário] Rode o bot do Código 11.4 com slippage de 0, 5, 20 e 50 pb e com a regra da ordem limitada trocada para "executa se tocar". Quanto do resultado depende de cada hipótese de preenchimento?

11.7 [desafio] Embrulhe a `CorretoraPapel` na interface `Corretora` do Código 11.5 e acrescente ao bot um módulo de risco com limite de posição, limite de perda diária e botão de emergência. Escreva um teste em que um sinal defeituoso manda comprar todos os dias e mostre que o módulo o contém.

Para ir além

- Harris (2003), capítulo 4, para os tipos de ordem e suas propriedades; o restante do livro cobre a microestrutura de mercado sem matemática pesada.
- Chan (2021), capítulos 5 e 6, sobre sistemas de execução, paper trading e gestão de capital com o critério de Kelly.
- Grinold e Kahn (2000), capítulo 16, para a relação entre custos de transação, giro e valor adicionado de uma estratégia.
- Almgren e Chriss (2001) e Tóth et al. (2011), os artigos de referência sobre execução ótima e sobre a lei da raiz quadrada do impacto.

Capítulo 12

Projeto final: da tese ao resultado

12

Os onze capítulos anteriores entregaram ferramentas. O projeto final pede que você as use em sequência para responder a uma pergunta sobre o mercado, e que deixe o caminho documentado a ponto de outra pessoa refazê-lo e chegar aos mesmos números. Este capítulo descreve como montar esse projeto do zero e traz um exemplo completo, executado com os dados do livro, que serve de gabarito de formato. O exemplo termina com um resultado inconclusivo, e isso é parte do que ele ensina.

Ao terminar este capítulo você consegue

- escrever uma tese falsificável, com mecanismo, universo, horizonte e critério de refutação;
- registrar, antes de olhar o período de teste, tudo o que poderia ser ajustado depois;
- conduzir um backtest com divisão temporal, defasagem de execução, custos e benchmark adequado, e reportar as métricas exigidas pelo Núcleo;
- testar a sensibilidade do resultado a parâmetros, custos, subperíodos e composição do universo;
- redigir uma conclusão que confirme, refute ou declare inconclusiva a tese, com limitações e próximos passos;
- organizar o repositório de entrega e preparar a defesa de dez minutos.

12.1 O que o Núcleo pede

O enunciado do projeto final fixa um fluxo obrigatório, mostrado na Figura 12.1: tese, modelo, backtest e resultado. O trabalho é feito em grupos de duas a quatro pessoas, defendido presencialmente em dez minutos com slides e entregue em um repositório público no GitHub. A Tabela 12.1 reúne as exigências e indica em que capítulo cada ferramenta foi apresentada.

Nenhum item da tabela pede que a estratégia dê lucro. O enunciado pede que você teste “rigorosamente se ela funciona” e apresente os resultados “com

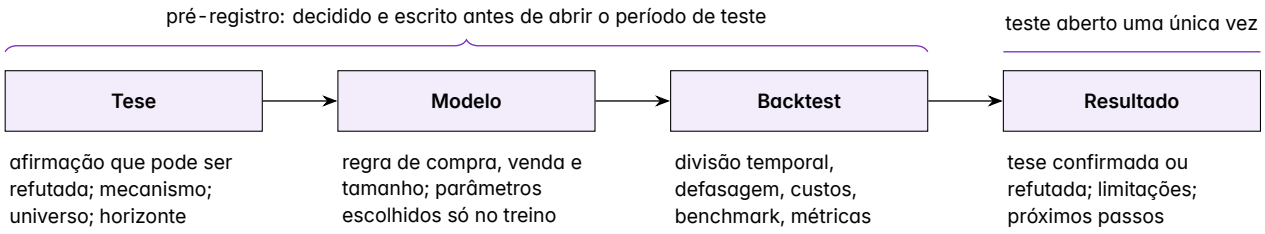


Figura 12.1 O fluxo obrigatório do projeto. As três primeiras etapas são fixadas por escrito antes de qualquer contato com o período de teste; a quarta consome esse período uma única vez.

Tabela 12.1 Exigências do enunciado do projeto final (ciclo 2026.1) e onde o livro trata de cada uma

Item	Exigência	Onde está
Tese	ideia central, mecanismo econômico ou comportamental, ativo ou universo, horizonte de operação; deve aparecer nos slides da apresentação	Seção 12.2
Modelo	implementado em Python, comentado, gerando os sinais de entrada e de saída	Capítulos 4, 8, 9 e 10
Dados	no mínimo cinco anos	Capítulo 2
Divisão temporal	período de treino (*in-sample*) para calibrar e período de teste (*out-of-sample*) reservado à avaliação final	Capítulos 7 e 8
Métricas obrigatórias	retorno acumulado (o "retorno total" do enunciado) e anualizado; volatilidade anualizada; índice de Sharpe; drawdown máximo e duração da recuperação; número de operações e taxa de acerto	Capítulos 1, 5 e 7
Benchmark	o índice mais adequado à estratégia (BOVA11 para ações brasileiras), comparado em retorno e em Sharpe	Capítulo 7
Custos	corretagem e *spread* em toda movimentação	Capítulo 11
Conclusão	tese confirmada ou refutada; significância econômica; limitações; o que faria diferente	Seção 12.7
Entrega	repositório público com `README.md` e `requirements.txt`; defesa de dez minutos com slides	Seções 12.8 e 12.9

honestidade", e a rubrica da Seção 12.9 avalia rigor técnico, clareza, reprodutibilidade e interpretação. Um grupo que refuta a própria tese com um backtest limpo cumpre todas as exigências.

12.2 A tese

A tese, termo que o Capítulo 8 já usou, é uma afirmação sobre o mercado que, se verdadeira, gera uma oportunidade de ganho. Ela vem antes do código porque é ela que diz o que medir. Quem começa pelo backtest, variando indicadores até a curva de capital ficar bonita, termina com uma regra que descreve o passado e com uma explicação inventada depois para justificá-la.

Uma tese utilizável tem cinco partes. A **afirmação** diz o que deve acontecer, em termos que os dados possam contrariar. O **mecanismo** explica por que o efeito existiria e por que ainda não foi eliminado por quem opera: um viés de comportamento, uma restrição institucional, um prêmio por carregar um risco que outros não querem. O **universo** delimita os ativos, e o **horizonte** diz de quanto em quanto tempo a posição muda. A quinta parte costuma faltar: o **critério de refutação**, ou seja, que resultado faria você concluir que a tese está errada.

Tabela 12.2 Teses como chegam e como ficam depois de reescritas

Primeira versão	O que falta	Versão testável
"Vou usar XGBoost para prever PETR4."	É um método, e não uma afirmação. Nada diz por que haveria algo a prever.	"O sinal do retorno de PETR4 em cinco dias é previsível a partir da variação do petróleo e do câmbio na semana anterior, com acurácia fora da amostra acima da de prever sempre alta."
"Ações baratas sobem."	Barata em que medida, em que prazo, contra o quê.	"Entre as ações do Ibovespa, o quinto de menor preço sobre lucro rende mais que o índice nos doze meses seguintes."
"Cruzamento de médias de 9 e 21 dias funciona no mini-índice."	Não há mecanismo, e os números 9 e 21 já parecem escolhidos olhando o gráfico.	"Tendências de um a três meses no Ibovespa persistem; uma regra de médias móveis com parâmetros escolhidos no treino supera comprar e manter em Sharpe, líquida de custos."

Intuição

Uma tese é uma aposta que pode ser perdida. Se nenhum resultado imaginável do backtest faria você abandoná-la, ela não afirma nada sobre o mercado, e o backtest deixa de ser um experimento para virar uma ilustração.

A Tabela 12.2 mostra três teses do jeito que costumam aparecer na primeira reunião de um grupo e o que falta para que possam ser testadas.

A tese do projeto-exemplo deste capítulo é o *momentum* transversal. Jegadeesh e Titman (1993) mostraram, com ações americanas de 1965 a 1989, que comprar as ações que mais subiram e vender as que mais caíram nos 3 a 12 meses anteriores gerou retornos positivos significativos nos 3 a 12 meses seguintes. Os autores descartam como explicação o risco sistemático e a reação atrasada a fatores comuns, o que deixa como candidata uma reação lenta dos preços a informações específicas de cada empresa. A versão do exemplo é pequena o bastante para caber em um capítulo: entre sete ações líquidas da B3, as duas a quatro que mais subiram nos últimos meses rendem mais, no mês seguinte, do que a média das sete.

12.2.1 Pré-registro

O Capítulo 7 mostrou que o sobreajuste nasce do número de tentativas. Bailey et al. (2014) demonstram que, testando variantes suficientes sobre a mesma amostra, chega-se a um Sharpe alto dentro da amostra mesmo quando o Sharpe verdadeiro de todas elas é zero. As tentativas que contam não são só as combinações de parâmetros. Trocar o benchmark depois de ver o resultado, encurtar o período de teste para excluir um ano ruim e escolher entre Sharpe e retorno acumulado a métrica em que a estratégia se sai melhor também são tentativas, e nenhuma aparece no código final.

O remédio é o **pré-registro**: um documento curto, escrito e datado antes de rodar qualquer coisa no período de teste, que fixa as decisões capazes de mudar a conclusão. Um *commit* no repositório com o arquivo `PRE_REGISTRO.md` serve de carimbo de data. A Tabela 12.3 é a ficha do projeto-exemplo.

Três escolhas da ficha merecem justificativa. O benchmark principal é a carteira de pesos iguais do próprio universo porque a tese é sobre *qual* das sete ações carregar, e a comparação com um índice misturaria essa pergunta com outra, a de quais sete ações estão na lista. DeMiguel et al. (2009) mostraram,

Tabela 12.3 Ficha de pré-registro do projeto-exemplo

Decisão	O que foi fixado
Afirmação	Entre sete ações da B3, uma carteira das K de maior retorno nos últimos J meses supera, no mês seguinte, a carteira de pesos iguais das sete, líquida de custos.
Mecanismo	Reação lenta dos preços a informação específica das empresas, hipótese compatível com os resultados de Jegadeesh e Titman (1993).
Universo e horizonte	PETR4, VALE3, ITUB4, WEGE3, ABEV3, BBAS3 e MGLU3; somente posições compradas; rebalanceamento mensal.
Períodos	Aquecimento de 2010 a 2012; treino de 2013 a 2019; teste de 2020 a 2025.
Variantes	Doze: $J \in \{3, 6, 9, 12\}$ e $K \in \{2, 3, 4\}$. Escolhe-se a de maior Sharpe líquido no treino.
Execução e custos	Sinal com o fechamento do último pregão do mês; ordem no fechamento do pregão seguinte; custo por ponta de 0,10% do valor negociado.
Benchmarks	Principal: pesos iguais nas sete ações, com o mesmo rebalanceamento e os mesmos custos. Secundário: BOVA11 comprado e mantido.
Métricas	As obrigatórias do enunciado, mais a razão de informação e a estatística t do retorno ativo mensal contra o benchmark principal.
Regra de decisão	Confirmada se, no teste, o Sharpe supera o do benchmark principal e $t \geq 2$. Refutada se o retorno ativo médio é nulo ou negativo. Inconclusiva em qualquer outro caso.

além disso, que a carteira $1/N$ é um adversário difícil: nenhum dos catorze modelos de otimização que testaram a superou de forma consistente fora da amostra. A regra de decisão tem três saídas, e não duas, porque seis anos de teste raramente bastam para um veredito, como a conta da Seção 12.5 vai mostrar. E o limiar $t \geq 2$ é o convencional (5% bilateral); Harvey et al. (2016) argumentam que, depois de décadas de garimpo de fatores, um efeito novo deveria passar de $t = 3$.

A ficha do exemplo precisa de uma ressalva, e ela vale como lição. Trata-se de uma reconstituição didática, que o exemplo não cumpre à risca. O Capítulo 8 já aplicou momentum transversal a estas mesmas sete ações com dados de 2012 a 2025 e já mostrou que o ganho dependia de MGLU3. Quem escreveu a ficha conhecia, portanto, o período de teste, e a expressão "teste aberto uma única vez" não descreve o que aconteceu neste livro. Tampouco existe um *commit* datado anterior à rodada; o arquivo `PRE_REGISTRO.md` da pasta `livro/scripts/cap12/` repete a ficha com essa mesma ressalva. O que o exemplo preserva é o procedimento: J e K saem só do treino, e os números do teste são reportados como saíram. A contaminação entra na lista de limitações da conclusão. No seu projeto, a ficha vem de fato antes.

Armadilha **Abrir o período de teste mais de uma vez**

Se você roda a estratégia no teste, não gosta do que vê, muda um parâmetro e roda de novo, o período de teste virou treino e não existe mais avaliação fora da amostra. O resultado da primeira rodada é o resultado do projeto. Ideias que surgirem depois dele entram na seção de próximos passos, e as análises de sensibilidade feitas no teste (Seção 12.6) são apresentadas como diagnóstico, nunca como nova rodada de escolha.

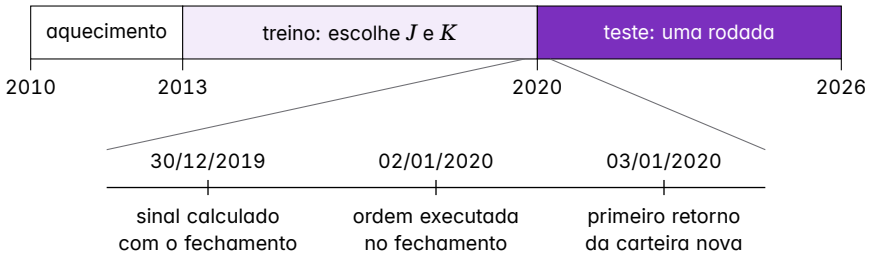


Figura 12.2 Divisão temporal do projeto-exemplo e, no detalhe, a defasagem de execução na virada de 2019 para 2020. O sinal calculado com o fechamento de um pregão só vira posição no fechamento do pregão seguinte.

12.3 Dados e universo

O projeto-exemplo usa o conjunto congelado do livro: preços de fechamento ajustados por proventos (Capítulo 1), diários, de 2010 a 2025. A taxa livre de risco é o CDI diário, série 12 do Sistema Gerenciador de Séries Temporais do Banco Central. O script `cap12/baixar_cdi.py` baixa a série e a grava em `dados/c12_cdi.csv`, a cópia usada neste capítulo. Baixar uma vez e ler do arquivo é o que garante que os números não mudem entre a véspera e o dia da defesa. Dois tratamentos ficam declarados. Os preços têm valores ausentes isolados, em pregões sem negócio registrado para um papel; nesses dias o código mantém a última cotação, de modo que o retorno do dia é zero e a variação aparece no pregão seguinte. E o CDI é acumulado de um pregão ao seguinte, porque há dias úteis bancários sem pregão em que ele continua rendendo.

A Figura 12.2 mostra a divisão temporal. Os três primeiros anos servem de aquecimento: MGLU3 começou a ser negociada em maio de 2011, e o sinal de doze meses só existe para as sete ações a partir de maio de 2012. O treino tem sete anos e o teste tem seis, ambos acima do mínimo de cinco anos de dados que o enunciado exige para o projeto inteiro.

Declarar as limitações dos dados faz parte da seção de dados, e a mais séria aqui é o viés de sobrevivência (Seção 7.6). As sete ações foram escolhidas hoje, entre empresas que chegaram a 2025 grandes e líquidas. Ninguém que montasse uma lista em janeiro de 2013 saberia que MGLU3, então uma varejista pequena, multiplicaria seu preço ajustado por 177 entre o fim de 2015 e o fim de 2019. Qualquer regra aplicada a esse universo herda a sorte de quem o escolheu, e a Tabela 12.4 vai mostrar quanto isso pesa. As demais limitações são menores e também ficam registradas: sete ativos são um universo muito pequeno para uma estratégia transversal; o preço de fechamento ajustado não é um preço em que se consiga negociar qualquer volume; e o custo de 0,10% por ponta é uma hipótese única que resume taxas, metade do *spread* e *slippage* de ordens pequenas, e não inclui o impacto de mercado de ordens grandes, tratado no Capítulo 11.

12.4 Modelo: da tese à regra

O modelo traduz a tese em pesos. No último pregão de cada mês calcula-se, para cada ação, o retorno acumulado nos J meses anteriores; as K ações de maior retorno recebem peso $1/K$ e as demais recebem zero. O Código 12.1 mostra as constantes do projeto e as duas funções que fazem essa conta. Todo o projeto está em um único módulo, `cap12/projeto.py`, que os scripts numerados importam.

Código 12.1 Constantes e sinal de momentum (`cap12/projeto.py`)

```
DADOS = Path(__file__).resolve().parents[2] / "dados"
ACOES = ["PETR4.SA", "VALE3.SA", "ITUB4.SA", "WEGE3.SA",
         "ABEV3.SA", "BBAS3.SA", "MGLU3.SA"]
TREINO = ("2013-01-01", "2019-12-31")
TESTE = ("2020-01-01", "2025-12-31")
CUSTO = 0.001          # 0,10% do valor negociado, por ponta
GRADE_J, GRADE_K = [3, 6, 9, 12], [2, 3, 4]   # 12 variantes
def fins_de_mes(px):
    """Linhas do último pregão de cada mês."""
    return px.groupby(px.index.to_period("M")).tail(1)


def alvo_momentum(px, j, k, acoes=ACOES):
    """Pesos-alvo: 1/k nos k papéis de maior alta em j meses."""
    fim = fins_de_mes(px[acoes])
    mom = fim / fim.shift(j) - 1        # só preços até o sinal
    posto = mom.rank(axis=1, ascending=False)
    alvo = (posto <= k).astype(float) / k
    return alvo[mom.notna().all(axis=1)]
```

A função `alvo_momentum` devolve uma tabela com uma linha por fim de mês e uma coluna por ação. A linha datada de 30 de dezembro de 2019 usa preços até esse dia e nenhum posterior; é isso que o comentário "só preços até o sinal" registra, e é o primeiro lugar em que a banca vai procurar um *look-ahead*.

Os parâmetros J e K são escolhidos pelo critério pré-registrado, o maior Sharpe líquido de custos, usando somente o período de treino. O script `cap12/l02_selecao.py` percorre as doze variantes em dois laços, chama o simulador da próxima seção apenas com o intervalo `TREINO` e imprime a grade.

```
Sharpe líquido no treino (2013 a 2019)
K      2     3     4
J
3   0.93  0.66  0.62
6   1.02  0.60  0.65
9   0.94  0.78  0.75
12  1.06  0.98  0.76
escolha: J = 12, K = 2
mediana da grade: 0.77
```

A variante escolhida é $J = 12$ e $K = 2$, com Sharpe de 1,06 no treino. Antes de seguir, leia a grade inteira. A escolhida não é um pico isolado: toda a coluna $K = 2$ fica entre 0,93 e 1,06, o que indica que o resultado depende pouco de J. Pardo (2008) recomenda exatamente essa leitura, preferindo regiões estáveis do espaço de parâmetros a máximos cercados de vizinhos ruins. Por outro lado, 1,06 é o maior de doze números, e o máximo de uma amostra superestima o que se deve esperar; a mediana da grade, 0,77, é uma previsão mais sóbria para o que vem depois. A partir deste ponto, J e K ficam escritos como constantes no script de teste e não mudam mais.

12.5 Backtest

12.5.1 O simulador e um teste de sanidade

O Código 12.2 é o motor do backtest, com três cuidados do Capítulo 7 visíveis: defasagem, custo sobre o valor negociado e período isolado. A defasagem está na chamada `shift(atraso)`: a linha de pesos calculada no pregão t só aparece como ordem em $t+1$, e só rende a partir de $t+2$. É um pregão a mais do que a convenção padrão do livro, a execução no fechamento do sinal dos Capítulos 7 e 8, em que a posição é assumida no próprio fechamento de t e rende de t para $t+1$. Aquela convenção não ganha o retorno do dia que gerou o sinal, mas supõe o sinal pronto instantes antes do leilão de fechamento, quando o preço de fechamento ainda não é conhecido. O enunciado do projeto final é mais rígido do que os capítulos anteriores neste ponto: ele dá exatamente esse caso, decidir e comprar no fechamento de t com um indicador que usa o fechamento de t, como exemplo de look-ahead. No projeto, portanto, use ao menos um pregão de atraso entre o sinal e a execução. O exemplo faz isso e deixa a outra como teste de sensibilidade (`atraso=0`) na Seção 12.6. Quem levar código de um capítulo a outro deve notar que o parâmetro `atraso` conta pregões de espera, e o parâmetro `defasagem` do motor dos Capítulos 7 e 8 vale um a mais: `atraso` é `defasagem` menos um. O custo é cobrado sobre o valor efetivamente negociado, que é a diferença entre os pesos-alvo e os pesos a que a carteira chegou depois de um mês de oscilação dos preços. E a função só enxerga o intervalo `periodo`, de modo que cada rodada começa em dinheiro e não carrega nada do período anterior.

Código 12.2 Simulação diária com defasagem e custos (`cap12/projeto.py`)

```
def simular(alvo, ret, periodo, custo=CUSTO, atraso=1):
    """Carteira dia a dia: retorno líquido, posições e giro."""
    ini, fim = periodo
    # o sinal de um pregão vira ordem no fechamento do seguinte
    ordens = alvo.reindex(ret.index).shift(atraso).loc[ini:fim]
    r = ret.loc[ini:fim, alvo.columns].fillna(0.0)
    w = np.zeros(r.shape[1])
    liquido, posicoes, giro = [], [], []
    for r_dia, ordem in zip(r.to_numpy(), ordens.to_numpy()):
        bruto = w @ r_dia
        w = w * (1 + r_dia) / (1 + bruto)     # pesos derivam
        negociado = 0.0
        if not np.isnan(ordem).any():         # dia de ordem
            negociado = np.abs(ordem - w).sum()
            w = ordem
        liquido.append((1 + bruto) * (1 - custo * negociado) - 1)
        posicoes.append(w)
        giro.append(negociado)
    return (pd.Series(liquido, r.index),
            pd.DataFrame(posicoes, r.index, alvo.columns),
            pd.Series(giro, r.index))
```

Um simulador escrito pelo grupo precisa ser testado antes de ser usado, com um caso cuja resposta se conhece por outro caminho. O mais simples é comprar e manter um único ativo: o resultado tem de coincidir com a razão entre o preço final e o preço de compra, descontado um custo de entrada.

Código 12.3 Teste de sanidade do simulador (cap12/l01_sanidade.py)

```
from projeto import *

px, ret, cdi = carregar()
bova = alvo_fixo(px, ["BOVA11.SA"])
r, pos, giro = simular(bova, ret, TESTE)

# conta direta: compra no fechamento do primeiro pregão do
# período, paga o custo uma vez e segura até o fim
p = px.loc[TESTE[0]:TESTE[1], "BOVA11.SA"]
direto = p.iloc[-1] / p.iloc[0] * (1 - CUSTO) - 1

print(f"simulador:    {(1 + r).prod() - 1:.6f}")
print(f"conta direta: {direto:.6f}")
print(f"giro total:   {giro.sum():.4f}")
print(f"primeira ordem em {giro[giro > 0].index[0].date()}, "
      f"sinal de {bova.loc[:TESTE[0]].index[-1].date()}")
```

```
simulador:    0.381670
conta direta: 0.381670
giro total:   1.0000
primeira ordem em 2020-01-02, sinal de 2019-12-30
```

Os dois caminhos coincidem até a sexta casa, o giro total é exatamente uma compra, e as datas confirmam a defasagem da Figura 12.2.

12.5.2 Métricas e o retorno ativo

O Código 12.4 calcula as métricas obrigatórias. A função `drawdown`, definida logo acima no módulo, devolve a queda da cota em relação ao maior valor já atingido. O Sharpe usa o excesso diário sobre o CDI, e as taxas são anualizadas pela convenção de 252 dias (Seção 1.9). Do episódio de drawdown máximo saem os três números do Capítulo 5: a profundidade, a duração da queda do pico ao vale e o tempo de recuperação do vale até a cota voltar ao pico anterior, os dois últimos em pregões. A "duração da recuperação" que o enunciado pede é esse tempo de recuperação. Uma **operação** é definida como a permanência contínua de um papel na carteira, da entrada à saída (função `operacoes` do módulo), e seu retorno desconta o custo das duas pontas; a taxa de acerto é a fração de operações com retorno positivo. Os ajustes mensais de peso de um papel que permanece na carteira entram no giro e no custo, mas não contam como operação nova. Operações ainda abertas no último pregão do período são contadas como se fossem encerradas nesse dia, com o custo de saída.

Código 12.4 Métricas obrigatórias (cap12/projeto.py)

```
def metricas(ret, cdi):
    """Métricas obrigatórias, a partir do retorno diário."""
    cota = (1 + ret).cumprod()
    anos = len(ret) / 252               # convenção de 252 dias
    excesso = ret - cdi.reindex(ret.index)
    dd = drawdown(ret)
    vale = dd.idxmin()
    pico = dd.loc[:vale][dd.loc[:vale] == 0].index
    queda = dd.loc[pico[-1]:vale].size - 1 if len(pico) else (
        dd.loc[:vale].size)             # pico no capital inicial
```

```
    apos = dd.loc[vale:]
    volta = apos[apos == 0].index       # cota de volta ao pico
    rec = apos.loc[:volta[0]].size - 1 if len(volta) else np.nan
    return pd.Series({
        "retorno acumulado": cota.iloc[-1] - 1,
        "retorno a.a.": cota.iloc[-1] ** (1 / anos) - 1,
        "vol a.a.": ret.std() * np.sqrt(252),
        "Sharpe": excesso.mean() / excesso.std() * np.sqrt(252),
        "drawdown máx.": dd.min(),
        "queda (pregões)": queda,
        "recuperação (pregões)": rec,
    })
```

Para a tese do exemplo, a métrica que decide é a do retorno ativo.

Definição 12.1 Retorno ativo e razão de informação

O **retorno ativo** no mês m é a diferença $a_m = R_{p,m} - R_{b,m}$ entre o retorno da estratégia e o do benchmark. Com média $\bar{a}$ e desvio-padrão s_a em n meses, a **razão de informação** anualizada e a estatística t da hipótese de retorno ativo médio nulo são

$$\mathrm{RI} = \frac{\bar{a}}{s_a}\sqrt{12}, \qquad t = \frac{\bar{a}}{s_a/\sqrt{n}} = \mathrm{RI}\sqrt{n/12}. \tag{12.1}$$

A frequência é mensal porque a tese é mensal; o Capítulo 7 define a mesma razão, com o nome em inglês *information ratio*, a partir de retornos ativos diários e $\sqrt{252}$, e os dois valores diferem um pouco. A segunda igualdade da Equação (12.1) diz que a estatística t é a razão de informação multiplicada pela raiz do número de anos. Ela permite uma conta de planejamento antes de qualquer backtest: uma estratégia com razão de informação verdadeira de 0,5, que já é respeitável, precisa de $(2/0{,}5)^2 = 16$ anos de dados para alcançar $t = 2$. Em seis anos de teste, só uma razão de informação acima de $2/\sqrt{6} \approx 0{,}82$ passaria do limiar. É por isso que a ficha previa a saída “inconclusiva”.

12.5.3 A rodada de teste

O script `cap12/l03_teste.py` aplica a variante escolhida ao período de teste, junto com os dois benchmarks, que passam pelo mesmo simulador e pagam os mesmos custos. A Tabela 12.4 reúne a saída e a Figura 12.3 mostra as curvas de capital e de drawdown.

Pela regra da ficha, o resultado é *inconclusivo*. No teste, a estratégia teve Sharpe de 0,45 contra 0,14 do benchmark principal e retorno ativo médio de 0,81% ao mês, mas a estatística t ficou em 1,08, abaixo de 2. A leitura da tabela não termina no veredito; há pelo menos quatro coisas nela que um grupo deveria comentar.

O Sharpe caiu de 1,06 no treino para 0,45 no teste. Parte da queda é do período, e não do modelo: o benchmark de pesos iguais caiu de 0,70 para 0,14. A vantagem sobre o benchmark também encolheu, embora menos: a distância entre os dois Sharpes passou de 0,36 para 0,31, e a razão de informação, de 0,55 para 0,44. O nível do treino dependia de um universo excepcionalmente favorável, e nem a parte atribuível à regra se repetiu por inteiro.

A linha do BOVA11 mostra o viés de sobrevivência em números. No treino, a carteira de pesos iguais das sete ações rendeu 27,5% ao ano contra 9,1% do BOVA11, sem modelo nenhum. Essa diferença de dezoito pontos percentuais

Tabela 12.4 Métricas do projeto-exemplo, líquidas de custo por ponta de 0,10% (saída de `cap12/l03_teste.py`). Pela convenção de 252 dias, o CDI rendeu 9,94% ao ano no treino e 9,65% ao ano no teste. O giro inclui a compra inicial, de 1,0 vez o patrimônio em cada período. Número de operações e taxa de acerto só são informativos para a estratégia: nas carteiras passivas, as operações são as compras iniciais.

	Treino, 2013 a 2019			Teste, 2020 a 2025		
Métrica	**momentum**	**pesos iguais**	**BOVA11**	**momentum**	**pesos iguais**	**BOVA11**
Retorno acumulado	1.192,6%	431,1%	81,4%	198,2%	74,6%	38,2%
Retorno anualizado	45,2%	27,5%	9,1%	20,2%	9,9%	5,6%
Volatilidade anualizada	30,8%	26,2%	22,8%	31,7%	26,5%	24,5%
Índice de Sharpe	1,06	0,70	0,08	0,45	0,14	−0,03
Drawdown máximo	−26,3%	−48,1%	−41,1%	−47,8%	−42,1%	−46,9%
Queda do pico ao vale (pregões)	44	343	756	18	21	40
Tempo de recuperação (pregões)	30	132	187	55	81	195
Giro anual (vezes o patrimônio)	5,5	1,0	0,1	4,7	1,0	0,2
Número de operações	36			27		
Taxa de acerto	52,8%			48,1%		
Retorno ativo médio ao mês	1,18%			0,81%		
Razão de informação	0,55			0,44		
Estatística t do retorno ativo	1,44			1,08		

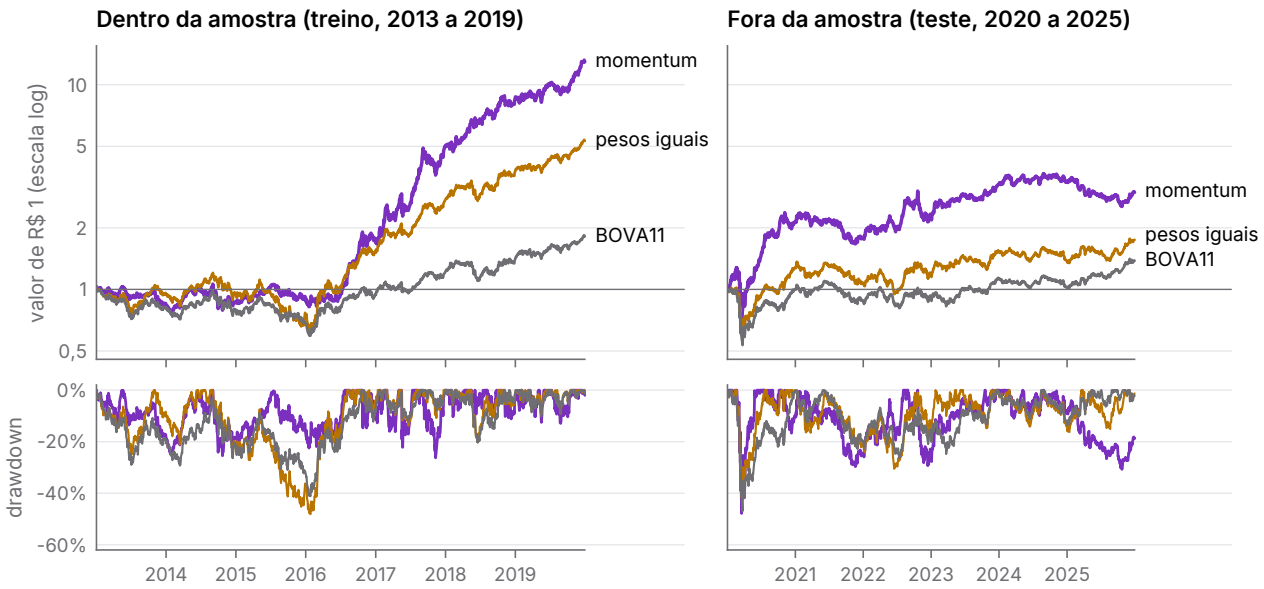


Figura 12.3 Valor de R$ 1 e drawdown da estratégia de momentum ($J = 12$, $K = 2$), da carteira de pesos iguais e do BOVA11, cada período começando em R$ 1. Fora da amostra a estratégia ainda termina à frente, mas quase todo o ganho relativo vem de 2020, e ela fecha 2025 em queda enquanto a carteira de pesos iguais sobe.

por ano é o prêmio de ter escolhido as ações conhecendo o futuro delas. Uma frase como "a estratégia rendeu 20,2% ao ano contra 5,6% do BOVA11" atribuiria ao momentum um mérito que é quase todo da lista.

A mesma linha tem um detalhe que parece contradição. O BOVA11 rendeu menos que o CDI no treino (9,1% contra 9,94% ao ano) e ainda assim aparece com Sharpe positivo. O retorno anualizado é uma taxa composta, e o Sharpe usa a média simples dos excessos diários, que fica acima da taxa composta em cerca de metade da variância (Capítulo 1): com volatilidade de 22,8%, são perto

de 2,6 pontos percentuais ao ano.

> **Armadilha** **Bater o índice com ações escolhidas hoje**
>
> Se o universo do projeto é uma lista de ações montada com o conhecimento de hoje, compare a estratégia com uma carteira passiva *do mesmo universo*, com os mesmos custos. Só a diferença entre as duas mede o que a regra acrescenta. O índice continua na tabela, como exige o enunciado, com a ressalva escrita ao lado.

O risco aumentou. A volatilidade da estratégia no teste foi de 31,7% contra 26,5% do benchmark, e o drawdown máximo, de 47,8%, atingido em março de 2020 depois de apenas 18 pregões de queda, foi pior que o dos dois benchmarks. Concentrar a carteira em duas ações tem esse preço.

E a incerteza é grande. Pela fórmula de Lo (2002) para retornos independentes, o erro-padrão de um Sharpe anualizado estimado com 1494 pregões é de 0,41. O Sharpe de 0,45 da estratégia está a pouco mais de um erro-padrão de zero.

12.6 Robustez

A análise de robustez pergunta de que o resultado depende. Ela é feita depois da rodada de teste e não altera o veredito; serve para dizer ao leitor quanta confiança o número merece. O script `cap12/l04_robustez.py` refaz o teste variando, um de cada vez, o custo, a defasagem de execução e a composição do universo, e separa o resultado por ano. As três colunas da saída são o Sharpe da estratégia, o do benchmark de pesos iguais e a estatística *t* do retorno ativo.

```
teste, 2020 a 2025     mom.  iguais      t
custo de 0.00%         0.47    0.14   1.12
custo de 0.10%         0.45    0.14   1.08
custo de 0.20%         0.44    0.14   1.03
custo de 0.50%         0.39    0.13   0.91
atraso de 0 dia(s)     0.42    0.14   0.97
atraso de 1 dia(s)     0.45    0.14   1.08
atraso de 2 dia(s)     0.46    0.15   1.08
atraso de 5 dia(s)     0.42    0.14   1.01
sem PETR4              0.48    0.04   1.63
sem VALE3              0.36    0.08   0.86
sem ITUB4              0.38    0.12   0.96
sem WEGE3              0.54    0.05   2.06
sem ABEV3              0.50    0.19   1.13
sem BBAS3              0.47    0.16   1.14
sem MGLU3              0.36    0.35   0.29
             2020    2021    2022    2023    2024    2025
momentum  +112.9%  -16.5%  +29.5%  +41.5%   +2.5%  -10.6%
iguais     +27.0%  -15.1%  +11.7%  +28.2%  -10.8%  +26.8%
ativo      +85.8%   -1.4%  +17.8%  +13.3%  +13.3%  -37.4%
```

- **Custos e defasagem.** A estratégia gira 4,7 vezes o patrimônio por ano, o que a 0,10% por ponta custa cerca de 0,47% ao ano. Dobrar o custo tira um centésimo do Sharpe; multiplicá-lo por cinco leva o Sharpe a 0,39. Esperar dois ou cinco pregões para executar quase não muda nada. A linha de atraso zero é a execução no fechamento do sinal, a convenção dos Capítulos 7 e 8:

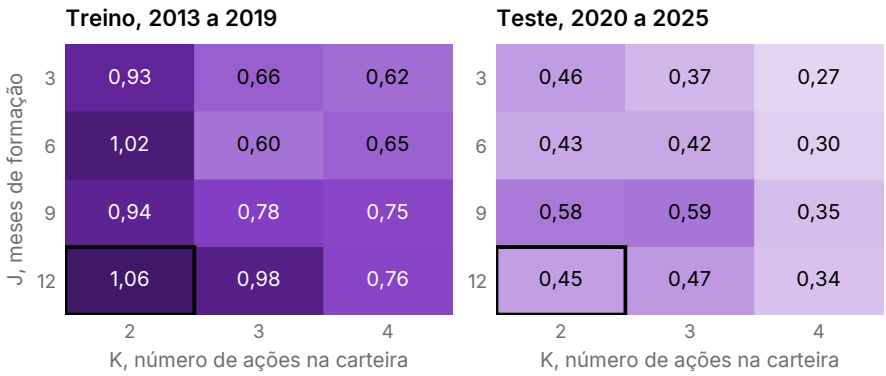


Figura 12.4 Mapa de sensibilidade: Sharpe líquido das doze variantes no treino e no teste, na mesma escala de cor. A célula contornada é a variante escolhida no treino. O nível caiu em toda a grade, para algo entre 0,42 e 0,76 do valor do treino, e a vantagem de $K = 2$ sobre $K = 3$ desapareceu.

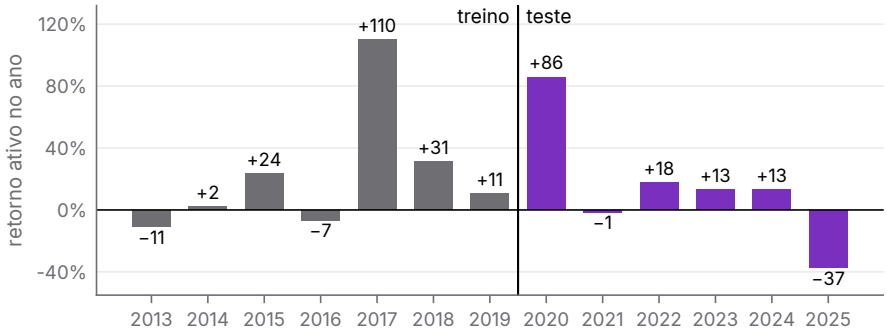


Figura 12.5 Retorno ativo da estratégia sobre a carteira de pesos iguais, por ano civil. Dois anos, 2017 no treino e 2020 no teste, concentram a maior parte do ganho acumulado.

aqui ela daria 0,42, um pouco menos que a versão com atraso, sinal de que a diferença entre as duas hipóteses é ruído nesta amostra. O resultado não é frágil nessas direções, o que se espera de uma regra mensal com poucas trocas.

- **Parâmetros.** A Figura 12.4 coloca lado a lado o Sharpe das doze variantes nos dois períodos. No teste, todas ficaram entre 0,27 e 0,59, acima do 0,14 do benchmark, e a ordenação das variantes se preservou em parte (correlação de postos de 0,65 entre as duas grades). A escolhida no treino não foi a melhor no teste, e não havia razão para que fosse. A mediana da grade caiu de 0,77 para 0,42.
- **Subperíodos.** A Figura 12.5 mostra o retorno ativo por ano civil. No teste, a estratégia ganhou do benchmark em quatro dos seis anos, e os tamanhos são muito desiguais: 86 pontos percentuais em 2020 e −37 em 2025. O treino tem o mesmo desenho, com 2017 respondendo por 110 pontos.
- **Universo.** O teste mais revelador é retirar uma ação de cada vez e refazer tudo com as seis restantes, incluindo o benchmark (Figura 12.6). Sem MGLU3 a estatística t cai para 0,29, e o Sharpe da estratégia (0,36) praticamente empata com o do benchmark (0,35); sem WEGE3 ela sobe para 2,06 e cruzaria o limiar de confirmação. Uma conclusão que muda de "nada" para "confirmada" conforme a presença de um único papel diz respeito a essas sete ações nesses seis anos, e não ao momentum. É a mesma dependência de MGLU3 que o Capítulo 8 encontrou com outra variante da regra.

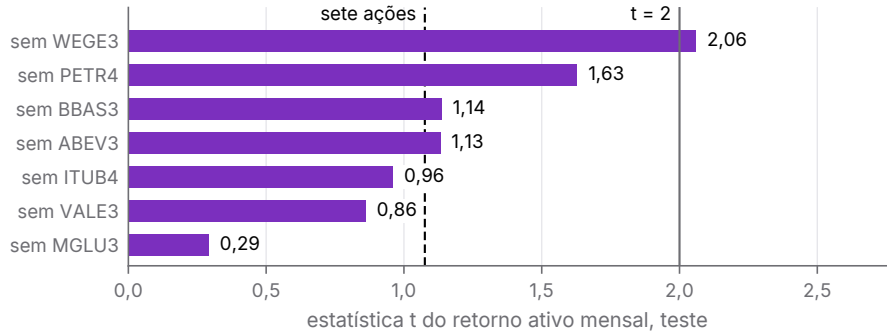


Figura 12.6 Estatística *t* do retorno ativo no teste quando uma das sete ações é retirada do universo. A linha tracejada marca o valor com as sete ações, 1,08. O resultado varia de 0,29 a 2,06 com a troca de um único papel.

A lista de operações, produzida por `cap12/l05_operacoes.py`, explica a origem dessa dependência.

```
operações: 27, com ganho: 13
ganho médio: 35.2%   perda média: -9.7%
duração mediana: 91 dias
ativo    entrada      saida retorno  dias
WEGE3 2020-01-02 2021-03-01 +123.9%   424
MGLU3 2020-01-02 2021-02-01 +102.4%   396
PETR4 2021-10-01 2023-03-01  +62.4%   516
último topo em 2024-08-26; fim da amostra a -18.7% dele
```

Das 27 operações, 14 deram prejuízo. A taxa de acerto de 48% convive com um resultado positivo porque o ganho médio das operações vencedoras, 35,2%, é mais de três vezes a perda média das perdedoras, um perfil comum em estratégias de tendência. As duas maiores operações do teste foram abertas no primeiro pregão, com um sinal calculado inteiramente com dados do treino, e mantidas por mais de um ano. Vinte e sete operações, duas das quais carregam o resultado, são uma amostra pequena para qualquer afirmação. A última linha completa o quadro: a estratégia terminou 2025 a 18,7% do topo de agosto de 2024.

12.7 Resultado e conclusão

O enunciado pede que a conclusão responda a quatro perguntas: a tese foi confirmada ou refutada; o resultado é economicamente significativo; quais são as limitações; o que você faria diferente com mais tempo. Uma conclusão que apenas repete a tabela de métricas não responde a nenhuma. Para a segunda pergunta o enunciado dá uma referência: um Sharpe de 0,3 fora da amostra, sem custos, provavelmente não é explorável na prática. E a seção de backtest do enunciado acrescenta uma quinta: a estratégia bateu o índice em retorno, e em retorno ajustado ao risco? O Exemplo a seguir é a conclusão do projeto deste capítulo, do tamanho que caberia em um `README.md` ou em um slide final.

Exemplo 12.1 Conclusão do projeto-exemplo

Veredito. Pela regra da ficha, o resultado é inconclusivo. De 2020 a 2025, a carteira das duas ações de maior retorno em doze meses teve Sharpe de 0,45

contra 0,14 da carteira de pesos iguais das mesmas sete ações, líquido de custo por ponta de 0,10%, mas a estatística t do retorno ativo foi de 1,08. Contra o BOVA11, a estratégia ganhou em retorno (20,2% contra 5,6% ao ano) e em Sharpe (0,45 contra $-0{,}03$), mas quase toda essa diferença vem da lista de ações, e não da regra.

Significância econômica. Um retorno ativo de 0,81% ao mês seria grande se fosse confiável. Não é possível afirmar que seja: o erro-padrão do Sharpe é de 0,41, um único ano (2020) responde por 86 pontos percentuais de retorno ativo, e a estatística t varia de 0,29 a 2,06 quando se retira uma ação do universo. A estratégia também teve mais volatilidade e drawdown maior que o benchmark.

Limitações. As sete ações foram escolhidas em 2025, o que favorece qualquer regra que compre as que mais subiram. O universo é pequeno demais para uma estratégia transversal. O custo por ponta é uma hipótese única, sem impacto de mercado. Foram testadas doze variantes no treino, e a escolhida era o máximo da grade. E o período de teste não era inédito: o Capítulo 8 já tinha examinado momentum nestas ações com dados até 2025, de modo que a ficha foi escrita por quem conhecia o desfecho.

Próximos passos. Repetir o teste em um universo amplo com a composição histórica do índice em cada data; trocar a divisão única por uma análise *walk-forward* (Capítulo 8); comparar com carteiras de duas ações sorteadas ao acaso, para saber onde o resultado fica na distribuição do que a sorte produz.

Duas regras de redação saem desse exemplo. Dê o veredito na primeira frase, com o número que o sustenta. E, em cada afirmação favorável, ponha ao lado a evidência que a enfraquece, se ela existir; a banca vai encontrá-la de qualquer modo.

12.7.1 Quando o resultado é negativo

A maior parte das teses testadas com cuidado não se confirma. Se a sua foi refutada, o projeto continua completo, desde que a refutação esteja bem documentada. Isso significa mostrar que o teste era capaz de detectar o efeito se ele existisse (período longo o bastante, custos realistas, simulador verificado), reportar o tamanho do efeito encontrado com sua incerteza em vez de escrever "não funcionou", e propor uma explicação que possa ser verificada: o efeito existe antes dos custos e desaparece depois deles, só aparece em um subperíodo, ou não aparece em parte alguma. O que não se faz é trocar de tese na última semana e apresentar a segunda como se fosse a primeira.

12.8 O repositório de entrega

O enunciado pede um repositório público no GitHub, enviado pelo WhatsApp ao gerente do Quant até 23h59 da véspera da apresentação, e deixa a organização livre: um único script ou vários arquivos. Só dois arquivos são obrigatórios, `README.md` e `requirements.txt`. A estrutura abaixo é a da pasta `livro/scripts/cap12/`, onde o projeto-exemplo está inteiro, com os três arquivos de documentação que este capítulo pede. Ela separa o que foi decidido, o que foi calculado e o que é resultado.

```
cap12/
|-- README.md           tese, como rodar, conclusão
|-- requirements.txt    bibliotecas com versões exatas
```

```
|-- PRE_REGISTRO.md      a ficha, com a ressalva do exemplo
|-- baixar_cdi.py        roda uma vez e grava o CSV do CDI
|-- projeto.py           sinal, simulador e métricas
|-- l01_sanidade.py      testa o simulador
|-- l02_selecao.py       usa só o período de treino
|-- l03_teste.py         período de teste
|-- l04_robustez.py      sensibilidade e subperíodos
|-- l05_operacoes.py     lista de operações
`-- fig_*.py             geram as figuras
```

Os números na frente dos scripts dizem a ordem de execução, e essa ordem conta a mesma história da apresentação. Em um repositório de grupo, os dados congelados ficam em uma pasta `dados/` dentro dele, e os slides entram como `slides.pdf`. O `README.md` deve conter o que o enunciado lista: nomes e cursos dos integrantes, a tese em duas ou três frases, as instruções para rodar e a principal conclusão. O do exemplo é este:

```
# Momentum entre sete ações da B3

Projeto-exemplo do Capítulo 12 do livro do Núcleo Quant (LMF,
UFU). No projeto do seu grupo, esta linha traz os nomes e os
cursos dos integrantes.

## Tese
Entre sete ações líquidas da B3, as duas de maior retorno nos
últimos doze meses rendem mais, no mês seguinte, do que a
carteira de pesos iguais das sete, líquido de custos.

## Como rodar (Python 3.12)
    pip install -r requirements.txt
    python l01_sanidade.py
    python l02_selecao.py
    python l03_teste.py
    python l04_robustez.py
    python l05_operacoes.py

A pasta não é autossuficiente: os dados ficam congelados em
../../dados (preços do livro e c12_cdi.csv, gravado uma vez
por baixar_cdi.py), e os fig_*.py, que refazem as figuras,
importam ../estilo.py.

## Conclusão
Inconclusivo. No teste (2020 a 2025): Sharpe 0,45 contra 0,14
do benchmark de pesos iguais, t do retorno ativo = 1,08.
Contra o BOVA11 a estratégia ganha em retorno (20,2% contra
5,6% ao ano) e em Sharpe (0,45 contra -0,03), mas quase toda
essa diferença vem da lista de ações. O resultado depende de
2020 e da presença de MGLU3, e o universo já tinha sido
examinado no Capítulo 8 com dados até 2025.
```

O `requirements.txt` sai de `pip freeze > requirements.txt`, executado dentro do ambiente virtual do projeto (Capítulo 1), e traz versões exatas, uma por linha, no formato `pandas==3.0.3`. O do exemplo foi reduzido à mão às três bibliotecas que o projeto importa, para caber na página; no seu projeto, entregue a saída inteira do `pip freeze`. Os números deste capítulo foram gerados com Python 3.12, numpy 2.4.4, pandas 3.0.3 e matplotlib 3.10.8. A versão exata

Tabela 12.5 Roteiro sugerido para a defesa de dez minutos

Slide	Tempo	O que mostrar
1. Tese	1,5 min	A afirmação em uma frase, o mecanismo, o universo e o horizonte, que o enunciado exige nos slides. O critério de refutação.
2. Dados e pré-registro	1,5 min	Linha do tempo com treino e teste. Benchmark, custos, número de variantes e regra de decisão, com a data em que foram fixados.
3. Modelo	1,5 min	Como foi implementado: a regra em palavras ou em uma fórmula, sem código. As decisões técnicas e o motivo de cada uma. Como os parâmetros foram escolhidos, com a grade do treino.
4. Backtest	2,5 min	Curva de capital e drawdown contra o benchmark, dentro e fora da amostra, e a tabela de métricas obrigatórias.
5. Robustez	1,5 min	Um ou dois gráficos: o que acontece com outros parâmetros, custos maiores, subperíodos.
6. Resultado	1,5 min	Veredito, limitações e próximos passos. Funcionou? Por quê?

importa porque bibliotecas mudam comportamentos padrão entre versões, e um script pode rodar sem erro e produzir outro número. Dois outros cuidados completam a reprodutibilidade. Se o projeto usa qualquer sorteio (divisão aleatória, *bootstrap*, random forest, pesos iniciais), fixe a semente no início do script e passe-a explicitamente, por exemplo com `np.random.default_rng(42)` ou o argumento `random_state` do scikit-learn. E os dados ficam congelados em arquivo: um script que baixa os preços a cada execução devolve séries ajustadas diferentes a cada novo provento.

No repositório

A pasta `complex/` do repositório do Núcleo é um exemplo de projeto grande organizado pelo mesmo fluxo: o código está separado em `thesis_and_model.py` e `backtest_engine.py`, e a pasta `complex/reports/tex/` traz quatro relatórios, `relatorio_1_tese.tex` a `relatorio_4_resultados.tex`, um para cada etapa. Use-a como referência de organização. Os resultados que o `README.md` de lá informa não foram auditados neste livro.

12.9 A defesa e a avaliação

A defesa presencial é, segundo o enunciado, o principal componente do projeto: dez minutos com slides, seguidos de perguntas, cobrindo os quatro elementos do fluxo. Dez minutos comportam seis ou sete slides. A Tabela 12.5 sugere uma distribuição.

Os erros de apresentação mais comuns são poucos e se repetem. Gastar quatro minutos explicando o que é momentum ou o que é uma random forest e chegar ao resultado sem tempo. Mostrar código no slide. Exibir a curva de capital sem o benchmark no mesmo gráfico, ou em escala linear em um período longo. Apresentar só o período completo, sem separar treino e teste. Ler uma tabela de vinte números em voz alta em vez de apontar os três que importam. Deixar as limitações para a fase de perguntas.

Tabela 12.6 Rubrica de avaliação: as quatro dimensões da proposta do Núcleo, com descritores

Dimensão	O que a proposta pergunta	Atende plenamente	Sinais de problema
Rigor técnico	Os métodos estão corretos? Os pressupostos foram verificados? O código é limpo e funcional?	Divisão temporal respeitada; sinal sem dado futuro e ao menos um pregão entre sinal e execução; custos incluídos; simulador testado; métricas calculadas como definidas.	Parâmetros escolhidos olhando o teste; sinal que usa dado posterior à ordem; sinal e execução no mesmo fechamento; Sharpe sem taxa livre de risco declarada.
Clareza	O notebook é autoexplicativo? Há narrativa conectando as etapas? Os gráficos estão rotulados?	Cada etapa do fluxo identificada; gráficos com eixos, unidades e benchmark; uma frase de leitura por figura.	Células soltas sem texto; gráficos sem rótulo; números sem unidade ou sem período.
Reprodutibilidade	Outra pessoa consegue rodar o notebook do zero e obter os mesmos resultados?	Versões exatas, dados congelados, sementes fixas, instruções que funcionam em um ambiente limpo.	Caminhos do computador do autor; dados baixados a cada execução; notebook que depende da ordem das células.
Interpretação	Os resultados foram discutidos de forma crítica? As limitações foram apontadas? Há conclusão objetiva?	Veredito explícito sobre a tese; incerteza quantificada; limitações específicas; próximos passos executáveis.	Conclusão que só repete as métricas; limitações genéricas; resultado negativo escondido ou reembalado.

As perguntas da banca são previsíveis, porque quase todas verificam um item deste livro. Prepare uma resposta de trinta segundos, com um número, para cada uma destas:

1. Onde poderia haver look-ahead no seu código, e como você verificou que não há?
2. Quantas variantes você testou até chegar a esta, contando as que não estão no repositório? Alguma decisão foi tomada depois de ver o período de teste?
3. O que acontece com o resultado se o custo dobrar?
4. Por que este benchmark? A estratégia ganha dele em retorno e também em Sharpe?
5. Como o universo foi escolhido? Há viés de sobrevivência?
6. O resultado depende de um ano, de um ativo ou de poucas operações?
7. Qual é a incerteza do seu Sharpe? Ele é distinguível de zero, ou do Sharpe do benchmark?
8. Qual foi o pior momento da estratégia, e você teria continuado operando durante ele?
9. Se eu clonar o repositório agora e rodar, chego aos mesmos números dos slides?
10. Que resultado teria feito você concluir que a tese estava errada?

A proposta do Núcleo avalia cada entrega em quatro dimensões de peso igual: rigor técnico, clareza, reprodutibilidade e interpretação. As perguntas da segunda coluna da Tabela 12.6 são as da proposta, que fala em notebook porque as entregas intermediárias são notebooks; os descritores das outras duas são a leitura deste livro sobre o que elas significam no projeto final.

A proposta acrescenta que a meta é progresso consistente: entregar no prazo com estrutura razoável vale mais do que entregar atrasado com código

Tabela 12.7 Erros frequentes: como reconhecer e como evitar

Sintoma	Causa provável	Como evitar
Sharpe muito alto e curva de capital quase sem quedas	Look-ahead: a posição ganha o retorno do próprio dia cujo fechamento gerou o sinal, ou o sinal usa uma estatística calculada com a amostra inteira	Defasar a posição em relação ao sinal e executar ao menos um pregão depois dele; calcular médias, desvios e normalizações só com dados anteriores; testar o simulador com um caso conhecido
Resultado excelente no treino e medíocre no teste	Sobreajuste: muitas variantes ou muitos parâmetros para poucos dados	Limitar e declarar o número de variantes; preferir regiões estáveis da grade; walk-forward (Capítulo 8)
Qualquer regra bate o Ibovespa	Viés de sobrevivência no universo	Benchmark passivo do mesmo universo; composição histórica do índice quando disponível
Lucro some ao incluir custos	Giro alto com ganho pequeno por operação	Medir o giro; incluir custos desde a primeira rodada; reduzir a frequência
Números mudam a cada execução	Dados baixados de novo; semente não fixada	Congelar os dados em arquivo; fixar sementes
O repositório não roda em outra máquina	Versões não registradas; caminhos absolutos; estado oculto do notebook	`pip freeze`; caminhos relativos; *Restart & Run All* em ambiente limpo

impecável. Repare também que a rubrica não tem dimensão para o retorno da estratégia. Um resultado negativo bem documentado pode alcançar a nota máxima em rigor e em interpretação.

A Tabela 12.7 lista os problemas que mais aparecem em projetos de backtest, pelo sintoma que os denuncia.

Na véspera da entrega, percorra esta lista com o repositório aberto.

1. A tese está escrita em duas ou três frases no `README.md` e aparece nos slides, com mecanismo, universo e horizonte.
2. Há pelo menos cinco anos de dados, com treino e teste declarados por datas.
3. Todo parâmetro foi escolhido com dados do treino, e o número de variantes testadas está informado.
4. O sinal de cada data usa apenas dados disponíveis até ela, e a ordem é executada ao menos um pregão depois do sinal.
5. Os custos de transação estão no backtest, e há uma análise com custo maior.
6. As métricas obrigatórias aparecem para a estratégia e para o benchmark, no treino e no teste.
7. A conclusão responde às quatro perguntas do enunciado e dá um veredito sobre a tese.
8. Em uma pasta nova, `git clone`, `pip install -r requirements.txt` e os scripts, na ordem do `README.md`, reproduzem os números dos slides.
9. O repositório é público e o endereço foi enviado ao gerente do Quant, pelo WhatsApp, dentro do prazo.

12.10 Depois do ciclo

A proposta do Núcleo foi desenhada tendo o Itaú Quant Challenge como competição de referência, e cita ainda o Top Gestor, hackathons de dados

financeiros e processos seletivos de *buy-side* e de mesas quantitativas como destinos para o que se aprende aqui. O formato de cada competição muda de uma edição para outra, e a primeira providência é ler o regulamento vigente. O que se transfere de um contexto para outro é o processo deste capítulo: uma pergunta bem formulada, um protocolo fixado antes do teste, um backtest que resiste a perguntas e uma conclusão do tamanho da evidência. Para continuar estudando, a seção "Para ir além", no fim do capítulo, organiza os livros da lista de referências em uma trilha comentada.

O projeto-exemplo deste capítulo terminou com um efeito de sinal positivo e incerteza demais para afirmá-lo: um único ano e um único papel respondem pela maior parte do ganho, e o período de teste já era conhecido. Esse é um desfecho comum de pesquisa feita com poucos dados e relatada sem cortes. Resultados individuais dependem em boa parte do período em que caíram. O que se repete de um projeto para o outro, e está sob o controle de quem pesquisa, é o processo: a tese escrita antes, o teste reservado, a conta conferida por outro caminho, a limitação declarada. As quatro dimensões da rubrica do Núcleo avaliam esse processo.

O que levar deste capítulo

- O projeto segue o fluxo tese, modelo, backtest e resultado. A tese é uma afirmação que os dados podem contrariar, com mecanismo, universo, horizonte e critério de refutação.
- O pré-registro fixa por escrito, antes do teste, períodos, variantes, benchmark, custos, métricas e regra de decisão. O período de teste é usado uma única vez, e quando isso não foi possível, como no exemplo deste capítulo, a contaminação é declarada.
- O benchmark precisa responder à mesma pergunta que a tese. Com um universo escolhido hoje, a referência justa é a carteira passiva do mesmo universo; o índice entra com a ressalva do viés de sobrevivência.
- A estatística t do retorno ativo é a razão de informação vezes a raiz do número de anos. Com poucos anos de teste, "inconclusivo" é um veredito legítimo e frequente.
- A análise de robustez diz de que o resultado depende (parâmetros, custos, subperíodos, ativos individuais) e qualifica a conclusão sem substituí-la.
- A avaliação considera rigor, clareza, reprodutibilidade e interpretação. Um resultado negativo ou inconclusivo, bem documentado, cumpre todas as exigências.

Exercícios

12.1 [básico] Reescreva em forma testável a tese "alta volatilidade hoje prediz alta volatilidade amanhã", citada no enunciado do projeto. Preencha para ela uma ficha como a da Tabela 12.3. Observe que ela afirma algo sobre volatilidade, e não sobre retorno: que estratégia a transformaria em ganho?

12.2 [básico] Rode os scripts de `livro/scripts/cap12/` na ordem e confira os números da Tabela 12.4. Depois altere `CUSTO` para 0,2% em `projeto.py` e verifique que o Sharpe do teste coincide com a linha correspondente da saída da Seção 12.6.

12.3 [intermediário] Projeto: momentum de série temporal em BOVA11. Fique comprado quando o retorno dos últimos J meses for positivo e no CDI quando for

negativo. Use treino até 2019, teste de 2020 a 2025, e comprar e manter como benchmark. A regra reduz o drawdown? A que custo em retorno? Como extensão, acrescente um alvo de volatilidade (Capítulos 8 e 11), com a volatilidade estimada com dados até a véspera.

12.4 [intermediário] Refaça a escolha de parâmetros do projeto-exemplo em formato walk-forward: a cada ano, escolha J e K com todos os dados anteriores e aplique-os no ano seguinte. Compare o resultado de 2020 a 2025 com o da Tabela 12.4. Os parâmetros escolhidos mudam ao longo do tempo?

12.5 [intermediário] Projeto: reversão de curto prazo. Entre as sete ações, compre a cada semana as duas de pior retorno na semana anterior. Estime o giro anual antes de rodar o backtest e calcule o custo por ponta que zera o retorno ativo. Ele não é o custo de equilíbrio do Capítulo 11, que divide o ganho bruto acima do CDI pelo giro: aqui a referência é a carteira de pesos iguais, que também paga custo, e o divisor é a diferença de giro entre as duas.

12.6 [desafio] Sorteie, com semente fixa, mil estratégias que escolhem duas das sete ações ao acaso a cada mês, e passe-as pelo mesmo simulador no período de teste. Em que percentil da distribuição de Sharpe fica a estratégia de momentum? Compare essa resposta com a estatística t da Tabela 12.4.

12.7 [desafio] Projeto: use um classificador dos Capítulos 9 ou 10 para prever o sinal do retorno de BOVA11 no mês seguinte, com validação temporal. Compare a estratégia resultante com ficar sempre comprado, e conte, na ficha de pré-registro, cada combinação de variáveis e hiperparâmetros como uma variante.

Para ir além

- Chan (2021) e Hilpisch (2018), para consolidar a prática. O primeiro percorre o caminho da ideia à execução do ponto de vista de quem opera sozinho, com atenção aos vieses de backtest do Capítulo 7; o segundo é a referência do ferramental em Python.
- López de Prado (2018), que a proposta do Núcleo chama de referência central em aprendizado de máquina aplicado a finanças. Reorganiza o processo de pesquisa em torno do sobreajuste: como rotular dados financeiros, como fazer validação cruzada quando as observações se sobrepõem no tempo, como avaliar um backtest. A parte do livro sobre os perigos do backtest continua este capítulo. Rende mais depois de um primeiro projeto, porque responde a problemas que só aparecem na prática. Jansen (2020) acompanha bem essa leitura, com o fluxo completo de aprendizado de máquina para estratégias implementado em código.
- Ang (2014), para a base teórica: trata fatores de risco como a unidade de análise de uma carteira e ajuda a escrever o "mecanismo" de uma tese. Tsay (2010) e Hastie et al. (2009) aprofundam a econometria do Capítulo 4 e o aprendizado estatístico dos Capítulos 9 e 10.
- Bailey et al. (2014) e Bailey e López de Prado (2014), sobre o Sharpe que se obtém por acaso ao testar muitas variantes e como corrigi-lo; Harvey et al. (2016), sobre limiares de significância depois de muitas hipóteses testadas.
- Pardo (2008), para a análise walk-forward e a estabilidade de parâmetros, e Jegadeesh e Titman (1993), como modelo de artigo que enuncia uma tese, testa-a e discute explicações alternativas.

Apêndice A

Notação e formulário

As fórmulas mais usadas do livro, na notação em que aparecem nos capítulos, com a indicação de onde cada uma é explicada. Serve para consulta rápida; a justificativa de cada expressão está no capítulo indicado.

A.1 Notação

Símbolo	Significado
P_t	preço de um ativo no instante t (ajustado por proventos, salvo aviso)
R_t	retorno simples entre $t-1$ e t
r_t	log-retorno entre $t-1$ e t
R^f, R^f_t	taxa livre de risco (no Brasil, o CDI): a taxa anual e a do dia t
μ, σ, σ^2	média, desvio-padrão (volatilidade) e variância de um retorno
$\bar{x}$, s	média e desvio-padrão amostrais
ρ_{ij}, σ_{ij}	correlação e covariância entre os ativos i e j
$\mathbf{w}$	vetor de pesos de uma carteira, com $\sum_i w_i = 1$
$\boldsymbol{\mu}$, $\boldsymbol{\Sigma}$	vetor de retornos esperados e matriz de covariância
$\mathbf{1}$	vetor de uns
α	nível de confiança de uma medida de risco (95%, 99%); no GARCH, o coeficiente do choque
z_α	quantil de ordem α da normal padrão ($z_{0,95} = 1{,}645$; $z_{0,99} = 2{,}326$)
φ, Φ	densidade e função de distribuição da normal padrão
L	perda de uma posição, com sinal positivo para prejuízo
$\mathbb{E}$, Var, Cov, Corr	esperança, variância, covariância e correlação
T, n	número de observações (n no Capítulo 3); 252 dias úteis por ano
w_t, c	em um backtest, posição assumida no fechamento de t e custo por ponta

Algumas letras mudam de sentido conforme o capítulo. A letra α é nível de confiança no Capítulo 5, nível de significância no Capítulo 3, coeficiente do choque no GARCH (Capítulo 4), intensidade de encolhimento no Capítulo 6, alfa de Jensen no Capítulo 7 e parâmetro de mistura do *elastic net* no Capítulo 10. A letra K é a curtose nos Capítulos 3 e 5 e um número de meses ou de ações nos Capítulos 8 e 12. A letra τ é o coeficiente de Kendall no Capítulo 3 e a incerteza do equilíbrio no modelo de Black–Litterman (Capítulo 6). A letra c é a constante do AR(1) no Capítulo 4 e o custo por ponta do Capítulo 7 em diante. Em cada caso o texto do capítulo define o símbolo antes de usá-lo.

A.2 Retornos (Capítulo 1)

$$R_t = \frac{P_t}{P_{t-1}} - 1, \qquad r_t = \ln\frac{P_t}{P_{t-1}} = \ln(1 + R_t),$$

$$1 + R_{0\to T} = \prod_{t=1}^{T}(1 + R_t), \qquad r_{0\to T} = \sum_{t=1}^{T} r_t,$$

$$R_{\text{a.a.}} = \left(\frac{P_T}{P_0}\right)^{1/A} - 1, \qquad \sigma_{\text{a.a.}} = \sigma_d\sqrt{252}.$$

Em A fica o número de anos do período, medido no calendário ou aproximado por $T/252$. O retorno simples de uma carteira é $R_p = \sum_i w_i R_i$. A anualização

da volatilidade pela raiz do tempo supõe retornos não correlacionados no tempo e de variância constante.

A.3 Estatística (Capítulo 3)

Para uma amostra $x_1, \dots, x_n$:

$$\bar{x} = \frac{1}{n}\sum_{t=1}^{n} x_t, \qquad s^2 = \frac{1}{n-1}\sum_{t=1}^{n}(x_t - \bar{x})^2,$$

$$\text{assimetria} = \frac{\mathbb{E}\big[(X-\mu)^3\big]}{\sigma^3}, \qquad \text{curtose} = \frac{\mathbb{E}\big[(X-\mu)^4\big]}{\sigma^4}.$$

A distribuição normal tem assimetria 0 e curtose 3; o excesso de curtose é a curtose menos 3. O erro-padrão da média amostral de observações não correlacionadas é $s/\sqrt{n}$. Covariância e correlação:

$$\operatorname{Cov}(X,Y) = \mathbb{E}[(X-\mu_X)(Y-\mu_Y)], \qquad \rho_{XY} = \frac{\operatorname{Cov}(X,Y)}{\sigma_X\,\sigma_Y} \in [-1,1].$$

A estatística de Jarque–Bera para uma amostra de tamanho n, com assimetria amostral $\hat{S}$ e curtose amostral $\hat{K}$, é $JB = \frac{n}{6}\left[\hat{S}^2 + \frac{(\hat{K}-3)^2}{4}\right]$, com distribuição assintótica qui-quadrado de 2 graus de liberdade sob normalidade.

A.4 Séries temporais (Capítulo 4)

A autocorrelação de ordem k de uma série estacionária é

$$\rho_k = \frac{\operatorname{Cov}(x_t, x_{t-k})}{\operatorname{Var}(x_t)}.$$

Se as observações são independentes e identicamente distribuídas, as autocorrelações amostrais ficam aproximadamente dentro de $\pm 1{,}96/\sqrt{T}$ em 95% dos casos; com volatilidade agrupada essa faixa é estreita demais.

$$\begin{aligned}
\text{AR(1):}\quad & x_t = c + \phi\, x_{t-1} + \varepsilon_t, \quad |\phi| < 1,\\
& \mathbb{E}[x_t] = \frac{c}{1-\phi}, \quad \operatorname{Var}(x_t) = \frac{\sigma^2}{1-\phi^2},\\
\text{MA(1):}\quad & x_t = \mu + \varepsilon_t + \theta\,\varepsilon_{t-1},\\
\text{GARCH(1,1):}\quad & r_t = \mu + \varepsilon_t, \quad \varepsilon_t = \sigma_t z_t, \quad \sigma_t^2 = \omega + \alpha\,\varepsilon_{t-1}^2 + \beta\,\sigma_{t-1}^2.
\end{aligned}$$

No AR(1) e no MA(1), ε_t é um ruído branco de variância σ^2. No GARCH(1,1), com $\omega > 0$, $\alpha, \beta \geq 0$ e $\alpha + \beta < 1$, a variância incondicional é $\omega/(1-\alpha-\beta)$.

A.5 Risco (Capítulo 5)

Com a perda L medida como número positivo e nível de confiança α:

$$\text{VaR}_\alpha = \inf\{\ell : P(L \leq \ell) \geq \alpha\}, \qquad \text{CVaR}_\alpha = \mathbb{E}[L \mid L \geq \text{VaR}_\alpha].$$

A segunda expressão vale para distribuições contínuas; no caso geral, o CVaR (ou *expected shortfall*, ES) é a média dos VaR de todos os níveis acima de α.

Se o retorno é normal com média μ e desvio-padrão σ, por unidade de capital investido,

$$\mathrm{VaR}_\alpha = -\mu + z_\alpha\,\sigma, \qquad \mathrm{CVaR}_\alpha = -\mu + \sigma\,\frac{\varphi(z_\alpha)}{1-\alpha}.$$

Drawdown no instante t de uma curva de capital V_t: $D_t = V_t / \max_{s \le t} V_s - 1$. O drawdown máximo é $\min_t D_t$.

A.6 Carteiras (Capítulo 6)

$$\begin{aligned}
\text{retorno e variância:} &\quad \mu_p = \mathbf{w}^\top \boldsymbol{\mu}, \qquad \sigma_p^2 = \mathbf{w}^\top \boldsymbol{\Sigma}\, \mathbf{w},\\
\text{dois ativos:} &\quad \sigma_p^2 = w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + 2w_1 w_2\,\rho_{12}\,\sigma_1\sigma_2,\\
\text{mínima variância:} &\quad \mathbf{w} = \frac{\boldsymbol{\Sigma}^{-1}\mathbf{1}}{\mathbf{1}^\top\boldsymbol{\Sigma}^{-1}\mathbf{1}},\\
\text{carteira tangente:} &\quad \mathbf{w} = \frac{\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - R^f\mathbf{1})}{\mathbf{1}^\top\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - R^f\mathbf{1})}.
\end{aligned}$$

As duas fórmulas fechadas valem sem restrição de sinal dos pesos (vendas a descoberto permitidas); a da carteira tangente supõe denominador positivo.

A.7 Desempenho (Capítulo 7)

Com R_t o retorno diário da estratégia, R_t^f o da taxa livre de risco, R_t^m o do mercado, e os excessos $d_t = R_t - R_t^f$ e $d_t^m = R_t^m - R_t^f$:

$$\begin{aligned}
\text{Sharpe} &= \frac{\bar{d}}{\hat{\sigma}_d}\sqrt{252}, & \text{Calmar} &= \frac{R_{\text{a.a.}}}{|\text{drawdown máximo}|},\\
\text{Sortino} &= \frac{\overline{R - A}}{\mathrm{DD}}\sqrt{252}, & \mathrm{DD} &= \sqrt{\frac{1}{T}\sum_{t=1}^{T}\min(R_t - A_t, 0)^2},\\
\text{Treynor} &= \frac{252\,\bar{d}}{\hat{\beta}}, & \hat{\beta} &= \frac{\mathrm{Cov}(d, d^m)}{\mathrm{Var}(d^m)}.
\end{aligned}$$

Em A_t fica o retorno mínimo aceitável do índice de Sortino (no livro, o CDI do dia). A anualização por $\sqrt{252}$ vale sob as mesmas hipóteses da regra da raiz do tempo.

Apêndice B

Glossário inglês–português

B

O vocabulário de finanças quantitativas é quase todo em inglês, e boa parte dele se usa sem tradução nas mesas e na literatura. Esta lista reúne os termos que aparecem no livro, com a tradução adotada e o capítulo em que são tratados.

Termo	Significado	Cap.
adjusted close	preço de fechamento ajustado por proventos e eventos societários	1
backtest	simulação de uma estratégia sobre dados históricos	7
benchmark	carteira ou índice de referência contra o qual se compara o desempenho	7
bid, *ask*	melhor preço de compra e melhor preço de venda no livro de ofertas	11
bid-ask spread	diferença entre o melhor preço de venda e o melhor preço de compra	11
breakout	rompimento: preço que ultrapassa a máxima ou a mínima de uma janela	8
buy and hold	comprar e manter, sem negociar	7
cross-validation	validação cruzada	9
data snooping	escolher um resultado depois de testar muitas variantes nos mesmos dados	7
drawdown	queda percentual em relação ao pico anterior da curva de capital	2, 5
ensemble	modelo formado pela combinação de vários modelos	10
expected shortfall	perda média nos cenários além do VaR; o mesmo que CVaR	5
fat tails	caudas pesadas: eventos extremos mais frequentes que na normal	3
feature	variável explicativa de um modelo de aprendizado de máquina	9

Termo	Significado	Cap.
high-water mark	maior valor já atingido pela curva de capital	5
in-sample, *out-of-sample*	dentro da amostra (dados usados para ajustar) e fora da amostra (dados reservados para avaliar)	7
label	rótulo: a variável que o modelo supervisionado tenta prever	9
limit order	ordem limitada: executa apenas ao preço indicado ou melhor	11
look-ahead bias	viés de antecipação: uso de informação que não existia no momento da decisão	2, 7
market order	ordem a mercado: executa de imediato ao melhor preço disponível	11
mean reversion	reversão à média	8
momentum	tendência de ativos que subiram (ou caíram) continuarem na mesma direção	8
overfitting	sobreajuste: modelo que aprende o ruído da amostra e não generaliza	7, 10
paper trading	operação simulada com dados ao vivo e dinheiro fictício	11
random walk	passeio aleatório	4
rolling window	janela móvel	2
slippage	diferença entre o preço esperado e o preço efetivamente obtido na execução	11
stop order	ordem que vira ordem a mercado quando o preço atinge um gatilho	11
stress test	teste de estresse: perda da carteira em cenários extremos	6
survivorship bias	viés de sobrevivência: amostra que só contém os ativos que continuaram existindo	7
ticker	código de negociação de um ativo	2
turnover	giro: fração da carteira negociada em um período	7
volatility clustering	agrupamento de volatilidade: dias agitados tendem a vir juntos	4
walk-forward	validação que reotimiza a estratégia em janelas sucessivas e avalia sempre no trecho seguinte	8

Referências

Acerbi, C.; Tasche, D. On the coherence of expected shortfall. *Journal of Banking & Finance*, v. 26, n. 7, p. 1487–1503, 2002.

Almgren, R.; Chriss, N. Optimal execution of portfolio transactions. *Journal of Risk*, v. 3, n. 2, p. 5–39, 2001.

Almgren, R.; Thum, C.; Hauptmann, E.; Li, H. Direct estimation of equity market impact. *Risk*, v. 18, n. 7, p. 58–62, 2005.

Ang, A. *Asset Management: A Systematic Approach to Factor Investing*. New York: Oxford University Press, 2014.

Artzner, P.; Delbaen, F.; Eber, J.-M.; Heath, D. Coherent measures of risk. *Mathematical Finance*, v. 9, n. 3, p. 203–228, 1999.

B3. *Tipos de ofertas*. PUMA Trading System: regras e parâmetros de negociação. São Paulo: B3. Disponível em: https://www.b3.com.br/pt_br/solucoes/plataformas/puma-trading-system/para-participantes-e-traders/regras-e-parametros-de-negociacao/tipos-de-ofertas. Acesso em: 6 out. 2026.

Bailey, D. H.; Borwein, J. M.; López de Prado, M.; Zhu, Q. J. Pseudo-mathematics and financial charlatanism: the effects of backtest overfitting on out-of-sample performance. *Notices of the American Mathematical Society*, v. 61, n. 5, p. 458–471, 2014.

Bailey, D. H.; López de Prado, M. The Sharpe ratio efficient frontier. *Journal of Risk*, v. 15, n. 2, p. 3–44, 2012.

Bailey, D. H.; López de Prado, M. The deflated Sharpe ratio: correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, v. 40, n. 5, p. 94–107, 2014.

BCBS (Basel Committee on Banking Supervision). *Minimum capital requirements for market risk*. Basileia: Bank for International Settlements, jan. 2019 (rev. fev. 2019). Disponível em: https://www.bis.org/bcbs/publ/d457.htm. Acesso em: 6 out. 2026.

Black, F.; Litterman, R. Global portfolio optimization. *Financial Analysts Journal*, v. 48, n. 5, p. 28–43, 1992.

Bollerslev, T. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, v. 31, n. 3, p. 307–327, 1986.

Bollerslev, T. A conditionally heteroskedastic time series model for speculative prices and rates of return. *The Review of Economics and Statistics*, v. 69, n. 3, p. 542–547, 1987.

Box, G. E. P.; Jenkins, G. M.; Reinsel, G. C.; Ljung, G. M. *Time Series Analysis: Forecasting and Control*. 5. ed. Hoboken: Wiley, 2015.

Breiman, L.; Friedman, J. H.; Olshen, R. A.; Stone, C. J. *Classification and Regression Trees*. Belmont: Wadsworth, 1984.

Breiman, L. Bagging predictors. *Machine Learning*, v. 24, n. 2, p. 123–140, 1996.

Breiman, L. Random forests. *Machine Learning*, v. 45, n. 1, p. 5–32, 2001.

Brown, S. J.; Goetzmann, W.; Ibbotson, R. G.; Ross, S. A. Survivorship bias in performance studies. *The Review of Financial Studies*, v. 5, n. 4, p. 553–580, 1992.

Campbell, J. Y.; Lo, A. W.; MacKinlay, A. C. *The Econometrics of Financial Markets*. Princeton: Princeton University Press, 1997.

Campbell, J. Y.; Thompson, S. B. Predicting excess stock returns out of sample: can

anything beat the historical average? *The Review of Financial Studies*, v. 21, n. 4, p. 1509–1531, 2008.

Chan, E. P. *Algorithmic Trading: Winning Strategies and Their Rationale*. Hoboken: Wiley, 2013.

Chan, E. P. *Quantitative Trading: How to Build Your Own Algorithmic Trading Business*. 2. ed. Hoboken: Wiley, 2021.

Chen, T.; Guestrin, C. XGBoost: a scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York: ACM, 2016. p. 785–794.

Chopra, V. K.; Ziemba, W. T. The effect of errors in means, variances, and covariances on optimal portfolio choice. *The Journal of Portfolio Management*, v. 19, n. 2, p. 6–11, 1993.

Christoffersen, P. F. Evaluating interval forecasts. *International Economic Review*, v. 39, n. 4, p. 841–862, 1998.

Cont, R. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, v. 1, n. 2, p. 223–236, 2001.

Daniel, K.; Moskowitz, T. J. Momentum crashes. *Journal of Financial Economics*, v. 122, n. 2, p. 221–247, 2016.

DeMiguel, V.; Garlappi, L.; Uppal, R. Optimal versus naive diversification: how inefficient is the 1/N portfolio strategy? *The Review of Financial Studies*, v. 22, n. 5, p. 1915–1953, 2009.

Dickey, D. A.; Fuller, W. A. Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, v. 74, n. 366, p. 427–431, 1979.

Engle, R. F. Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, v. 50, n. 4, p. 987–1007, 1982.

Forbes, K. J.; Rigobon, R. No contagion, only interdependence: measuring stock market comovements. *The Journal of Finance*, v. 57, n. 5, p. 2223–2261, 2002.

Friedman, J. H. Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, v. 29, n. 5, p. 1189–1232, 2001.

Glosten, L. R.; Jagannathan, R.; Runkle, D. E. On the relation between the expected value and the volatility of the nominal excess return on stocks. *The Journal of Finance*, v. 48, n. 5, p. 1779–1801, 1993.

Granger, C. W. J.; Newbold, P. Spurious regressions in econometrics. *Journal of Econometrics*, v. 2, n. 2, p. 111–120, 1974.

Grinold, R. C.; Kahn, R. N. *Active Portfolio Management: A Quantitative Approach for Producing Superior Returns and Controlling Risk*. 2. ed. New York: McGraw-Hill, 2000.

Gu, S.; Kelly, B.; Xiu, D. Empirical asset pricing via machine learning. *The Review of Financial Studies*, v. 33, n. 5, p. 2223–2273, 2020.

Hamilton, J. D. *Time Series Analysis*. Princeton: Princeton University Press, 1994.

Harris, L. *Trading and Exchanges: Market Microstructure for Practitioners*. New York: Oxford University Press, 2003.

Harvey, C. R.; Liu, Y.; Zhu, H. … and the cross-section of expected returns. *The Review of Financial Studies*, v. 29, n. 1, p. 5–68, 2016.

Hastie, T.; Tibshirani, R.; Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2. ed. New York: Springer, 2009.

Hilpisch, Y. *Python for Finance: Mastering Data-Driven Finance*. 2. ed. Sebastopol: O'Reilly, 2018.

Hoerl, A. E.; Kennard, R. W. Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, v. 12, n. 1, p. 55–67, 1970.

Hull, J. C. *Risk Management and Financial Institutions*. 5. ed. Hoboken: Wiley, 2018.

Hunter, J. D. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, v. 9, n. 3, p. 90–95, 2007.

Jagannathan, R.; Ma, T. Risk reduction in large portfolios: why imposing the wrong constraints helps. *The Journal of Finance*, v. 58, n. 4, p. 1651–1683, 2003.

James, G.; Witten, D.; Hastie, T.; Tibshirani, R. *An Introduction to Statistical Learning: with Applications in R*. 2. ed. New York: Springer, 2021.

Jansen, S. *Machine Learning for Algorithmic Trading*. 2. ed. Birmingham: Packt, 2020.

Jarque, C. M.; Bera, A. K. A test for normality of observations and regression residuals. *International Statistical Review*, v. 55, n. 2, p. 163–172, 1987.

Jegadeesh, N.; Titman, S. Returns to buying winners and selling losers: implications for stock market efficiency. *The Journal of Finance*, v. 48, n. 1, p. 65–91, 1993.

Jensen, M. C. The performance of mutual funds in the period 1945–1964. *The Journal of Finance*, v. 23, n. 2, p. 389–416, 1968.

Jorion, P. *Value at Risk: The New Benchmark for Managing Financial Risk*. 3. ed. New York: McGraw-Hill, 2007.

Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.-Y. LightGBM: a highly efficient gradient boosting decision tree. In: *Advances in Neural Information Processing Systems 30*. 2017. p. 3146–3154.

Kelly, J. L., Jr. A new interpretation of information rate. *The Bell System Technical Journal*, v. 35, n. 4, p. 917–926, 1956.

Kupiec, P. H. Techniques for verifying the accuracy of risk measurement models. *The Journal of Derivatives*, v. 3, n. 2, p. 73–84, 1995.

Kwiatkowski, D.; Phillips, P. C. B.; Schmidt, P.; Shin, Y. Testing the null hypothesis of stationarity against the alternative of a unit root: how sure are we that economic time series have a unit root? *Journal of Econometrics*, v. 54, n. 1–3, p. 159–178, 1992.

Ledoit, O.; Wolf, M. Honey, I shrunk the sample covariance matrix. *The Journal of Portfolio Management*, v. 30, n. 4, p. 110–119, 2004.

Ledoit, O.; Wolf, M. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, v. 88, n. 2, p. 365–411, 2004.

Ljung, G. M.; Box, G. E. P. On a measure of lack of fit in time series models. *Biometrika*, v. 65, n. 2, p. 297–303, 1978.

Lo, A. W. The statistics of Sharpe ratios. *Financial Analysts Journal*, v. 58, n. 4, p. 36–52, 2002.

Longin, F.; Solnik, B. Extreme correlation of international equity markets. *The Journal of Finance*, v. 56, n. 2, p. 649–676, 2001.

López de Prado, M. *Advances in Financial Machine Learning*. Hoboken: Wiley, 2018.

López de Prado, M. *Machine Learning for Asset Managers*. Cambridge: Cambridge University Press, 2020.

Lundberg, S. M.; Lee, S.-I. A unified approach to interpreting model predictions. In: *Advances in Neural Information Processing Systems 30*. 2017. p. 4765–4774.

Maillard, S.; Roncalli, T.; Teïletche, J. The properties of equally weighted risk contribution portfolios. *The Journal of Portfolio Management*, v. 36, n. 4, p. 60–70, 2010.

Mandelbrot, B. The variation of certain speculative prices. *The Journal of Business*, v. 36, n. 4, p. 394–419, 1963.

Markowitz, H. Portfolio selection. *The Journal of Finance*, v. 7, n. 1, p. 77–91, 1952.

McKinney, W. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. 3. ed. Sebastopol: O'Reilly, 2022.

McNeil, A. J.; Frey, R.; Embrechts, P. *Quantitative Risk Management: Concepts, Techniques and Tools*. Ed. rev. Princeton: Princeton University Press, 2015.

Merton, R. C. On estimating the expected return on the market: an exploratory investigation. *Journal of Financial Economics*, v. 8, n. 4, p. 323–361, 1980.

Michaud, R. O. The Markowitz optimization enigma: is 'optimized' optimal? *Financial Analysts Journal*, v. 45, n. 1, p. 31–42, 1989.

Moskowitz, T. J.; Ooi, Y. H.; Pedersen, L. H. Time series momentum. *Journal of Financial Economics*, v. 104, n. 2, p. 228–250, 2012.

Nelson, D. B. Conditional heteroskedasticity in asset returns: a new approach. *Econometrica*, v. 59, n. 2, p. 347–370, 1991.

Newey, W. K.; West, K. D. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, v. 55, n. 3, p. 703–708, 1987.

Pardo, R. *The Evaluation and Optimization of Trading Strategies*. 2. ed. Hoboken: Wiley, 2008.

Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; Vanderplas, J.; Passos, A.; Cournapeau, D.; Brucher, M.; Perrot, M.; Duchesnay, É. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.

Rockafellar, R. T.; Uryasev, S. Optimization of conditional value-at-risk. *Journal of Risk*, v. 2, n. 3, p. 21–41, 2000.

Said, S. E.; Dickey, D. A. Testing for unit roots in autoregressive-moving average models of unknown order. *Biometrika*, v. 71, n. 3, p. 599–607, 1984.

SEC (U.S. Securities and Exchange Commission). *SEC charges Knight Capital with violations of market access rule*. Press release 2013-222. Washington, 16 out. 2013. Disponível em: https://www.sec.gov/news/press-release/2013-222. Acesso em: 6 out. 2026.

Sharpe, W. F. Mutual fund performance. *The Journal of Business*, v. 39, n. 1, parte 2, p. 119–138, 1966.

Sharpe, W. F. The Sharpe ratio. *The Journal of Portfolio Management*, v. 21, n. 1, p. 49–58, 1994.

Sortino, F. A.; Price, L. N. Performance measurement in a downside risk framework. *The Journal of Investing*, v. 3, n. 3, p. 59–64, 1994.

Strobl, C.; Boulesteix, A.-L.; Zeileis, A.; Hothorn, T. Bias in random forest variable importance measures: illustrations, sources and a solution. *BMC Bioinformatics*, v. 8, art. 25, 2007.

Sullivan, R.; Timmermann, A.; White, H. Data-snooping, technical trading rule performance, and the bootstrap. *The Journal of Finance*, v. 54, n. 5, p. 1647–1691, 1999.

Tibshirani, R. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, v. 58, n. 1, p. 267–288, 1996.

Tóth, B.; Lempérière, Y.; Deremble, C.; de Lataillade, J.; Kockelkoren, J.; Bouchaud, J.-P. Anomalous price impact and the critical nature of liquidity in financial markets. *Physical Review X*, v. 1, n. 2, 021006, 2011.

Treynor, J. L. How to rate management of investment funds. *Harvard Business Review*, v. 43, n. 1, p. 63–75, 1965.

Tsay, R. S. *Analysis of Financial Time Series*. 3. ed. Hoboken: Wiley, 2010.

Tufte, E. R. *The Visual Display of Quantitative Information*. 2. ed. Cheshire: Graphics Press, 2001.

Tukey, J. W. *Exploratory Data Analysis*. Reading: Addison-Wesley, 1977.

White, H. A reality check for data snooping. *Econometrica*, v. 68, n. 5, p. 1097–1126, 2000.

Wilder, J. W. *New Concepts in Technical Trading Systems*. Greensboro: Trend Research, 1978.

Young, T. W. Calmar ratio: a smoother tool. *Futures*, v. 20, out. 1991.

Zou, H.; Hastie, T. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, v. 67, n. 2, p. 301–320, 2005.

Índice remissivo

ADF, 62
aditividade no tempo, 7
agrupamento de volatilidade, 71
alfa de Jensen, 130
alvo de volatilidade, 230
alvo de volatilidade, 157
ambiente virtual, 3
análise de sensibilidade, 119
análise exploratória de dados, 32
análise walk-forward, 158
ancorada, 158
aprendizado supervisionado, 169
ARCH, 71
ARIMA, 66
árvore de decisão, 196
assimetria, 40
AUC, 176, 202
autocorrelação, 42, 64
autorregressivo, 65
Axes (Matplotlib), 27

backtest, 124
bagging, 198
bandas de Bollinger, 152
barra OHLCV, 14
base 100, 33
benchmark, 236
benchmark, 127
beta, 118, 129, 170
Black–Litterman, 116
Bonferroni, 51
boosting, 200
bootstrap, 100
bootstrap, 198
botão de emergência, 231
Box-Jenkins, 66
break-even, 219
busca aleatória, 202

canal de Donchian, 154
capacidade, 221
caudas pesadas, 40
CDI, 16, 108, 125, 147, 187, 208
chances, 174
classificação, 169
coeficiente de determinação, 170
cointegração, 61
cointegradas, 154
comprar e manter, 187
contribuição de risco, 115
correlação, 34, 52
correlação de Spearman, 52
correlação móvel, 55
critério de Kelly, 230
critérios de parada, 197
curtose, 40
curva ROC, 176
custo de equilíbrio, 157, 219
custo por ponta, 176
custos explícitos, 217
custos implícitos, 217
CVaR, 94

decomposição de Cholesky, 89
dentro da amostra, 139
dependência parcial, 206
deriva, 59
desvio abaixo do alvo, 129
desvio-padrão, 39
Dickey-Fuller, 61
diferenciação fracionária, 180
dispersão, 39
distribuição normal, 41
drawdown, 96
drawdown máximo, 96

efeito fantasma, 88
eficiência walk-forward, 159
eficiente, 106
elastic net, 194
embargo, 161
encolhimento, 115
engenharia de features, 177
ensemble, 198
erro out-of-bag, 199
erro tipo I, 46
erro tipo II, 46
erro-padrão da média, 47
estacionária, 51
estacionariedade estrita, 59

estatística de teste, 46
EWMA, 75, 91
excesso de curtose, 40
excesso de retorno, 128
execução no fechamento do sinal, 125, 187
expected shortfall, 94

fator de lucro, 142
fatos estilizados, 42
features, 169
Figure (Matplotlib), 27
folhas, 196
fora da amostra, 139
fracamente estacionário, 59
fração de Kelly, 231
fração fixa, 230
fronteira eficiente, 106
FRTB, 96
função de autocorrelação, 64
função de autocorrelação parcial, 64
função sigmoide, 174

GARCH, 71
garimpagem de dados, 137
giro, 219
giro, 126
graus de liberdade, 43

heterocedasticidade, 171
high-water mark, 96
hipótese nula, 46
horizonte, 83
horizonte fixo, 180

idempotente, 228
importância por impureza, 205
importância por permutação, 205
índice de Calmar, 129
índice de Calmar, 97
índice de força relativa, 178
índice de Sharpe, 128
índice de Sharpe, 107
índice de Sortino, 129
índice de Treynor, 130
information ratio, 130
interface orientada a objetos, 27
intervalo de confiança, 48
intervalo interquartil, 39

janela expansível, 183
janela móvel, 33

kill switch, 231

KPSS, 62

lasso, 194
lei da raiz quadrada, 218
leilão, 215
leptocurtose, 40
LightGBM, 202
limiar, 175
linha de alocação de capital, 107
linha de mercado de capitais, 107
livro de ofertas, 214
Ljung-Box, 64
locação e escala, 44
log-retorno, 6
look-ahead, 20, 133, 148, 180, 239

MACD, 178
Markowitz, 106
Matplotlib, 27
matriz de confusão, 175
matriz de correlação, 54
matriz de covariância, 54
média, 39
mediana, 39
médias móveis, 65
medida coerente de risco, 93
medida de posição, 39
meia-vida, 71, 153
melhor oferta de compra, 214
melhor oferta de venda, 214
meta-labeling, 207
método da tripla barreira, 180
mínimos quadrados ordinários, 170, 193
modelo de mercado, 170
momentum, 149
Monte Carlo, 89
multicolinearidade, 172
múltiplos testes, 51

nível de confiança, 83
nível de significância, 46

operação, 241
ordem a mercado, 215
ordem limitada, 215
ordem stop, 215
order book, 214
orientado a eventos, 140

p-valor, 46
padronizar, 180
paper trading, 227
paridade de risco, 116
passeio aleatório, 59

payoff, 142
perda, 82
perda logarítmica, 174
persistência, 71
Plotly, 29
poder, 46
ponto-base, 215
posição-alvo, 147
pré-registro, 236
preço ajustado, 9
primeira diferença, 60
processo estocástico, 58
profundidade, 214
PTAX, 16
purga, 161

QQ-plot, 45
quantil, 39

R^2 fora da amostra, 172
raiz unitária, 60
random forest, 198
razão de cauda, 99
razão de chances, 174
razão de informação, 242
regressão, 169
regressão espúria, 61
regressão logística, 174
regularização, 193
relatório de backtest, 162
resíduos padronizados, 74
retorno ativo, 242
retorno simples, 6
retorno total, 9
ridge, 194
risco diversificável, 105
risco sistemático, 105
rolante, 158
rompimento, 154
rótulo, 169
ruído branco, 59

seleção adversa, 215
seleção de features, 194
série temporal, 58, 149
SHAP, 206
slippage, 213
sobreajuste, 137, 191
spread, 214
stop a mercado, 215
stop-limite, 215

t padronizada, 43
tamanho da aposta, 207
tau de Kendall, 52
taxa de acerto, 142
taxa de aprendizado, 200
tempo de recuperação, 96
tempo submerso, 97
teorema central do limite, 42
tese, 146
teste de estresse, 117
teste de Jarque-Bera, 49
teste de Kolmogorov-Smirnov, 50
teste de Kupiec, 91
teste de Ljung-Box, 50
teste de Shapiro-Wilk, 49
teste final, 193
teste *t*, 47
ticker, 16
TimeSeriesSplit, 183
transversal, 150
TWAP, 222

underwater, 97

validação cruzada, 160, 182
validação cruzada combinatória purgada, 161
Value at Risk, 83
VaR, *veja* Value at Risk
variância, 191
vazamento de informação, 186
venda a descoberto, 106
vetorização, 4
vetorizado, 140
viés, 191
viés de antecipação, 20
viés de sobrevivência, 135, 147, 193, 238
violação, 90
volatilidade, 39
  em janela móvel, 33
volatilidade condicional, 71
VWAP, 222

walk-forward, 186

XGBoost, 201

z-score, 152